TYPO3 CMS Core ChangeLog

Release 8

TYPO3 CMS Core Team

2017-12-18 12:19

CONTENTS

1 Sitemap **3**

2 ChangeLog v8 **5**
2.1 8.7.x Changes . . . 5
2.2 8.7 Changes . . . 8
2.3 8.6 Changes . . . 57
2.4 8.5 Changes . . . 152
2.5 8.4 Changes . . . 187
2.6 8.3 Changes . . . 208
2.7 8.2 Changes . . . 239
2.8 8.1 Changes . . . 253
2.9 8.0 Changes . . . 274

3 ChangeLog v7 **367**
3.1 7.6.x Changes . . . 367
3.2 7.6 Changes . . . 370
3.3 7.5 Changes . . . 411
3.4 7.4 Changes . . . 473
3.5 7.3 Changes . . . 533
3.6 7.2 Changes . . . 569
3.7 7.1 Changes . . . 600
3.8 7.0 Changes . . . 642

4 Documenting Changes **697**
4.1 Motivation and goals . . . 697
4.2 Different changelog types . . . 697
4.3 Location . . . 698
4.4 Filename convention . . . 698
4.5 File content . . . 698
4.6 Tagging . . . 699

Index **701**

Rendered 2017-12-18 12:18

Every change to the TYPO3 Core which might affect your site is documented here.

CHAPTER

ONE

SITEMAP

CHAPTER

TWO

CHANGELOG V8

Every change to the TYPO3 Core which might affect your site is documented here.

2.1 8.7.x Changes

Changes after 8.7.0 LTS release.

Table of contents

- *Features*
- *Important*

2.1.1 Features

Feature: #78161 - Introduce .typoscript file extension

See Issue #78161

Description

The new file extension .typoscript will be the default for TypoScript configuration files and is the only recommended one from now on. This effort is made to introduce a dedicated file extension for TypoScript configuration files, and to avoid conflicts with already existing and more spread file extensions like ".ts" for TypeScript or Video Transport Stream Files.

New prioritised files for static templates: * constants.typoscript * setup.typoscript

New prioritised files for extension statics: * ext_typoscript_constants.typoscript * ext_typoscript_setup.typoscript

For more details please head over to the decision platform: * https://decisions.typo3.org/t/file-endings-for-typoscript-files-and-tsconfig-files/43 * https://decisions.typo3.org/t/file-endings-for-typoscript-and-tsconfig-files-results/71

Impact

The ".typoscript" file extension is prioritised over the legacy .txt and .ts file extensions, and the only recommended file extension for typoscript configuration files.

2.1.2 Important

Important: #23178 - New TYPO3_CONF_VARS option FE|pageNotFound_handling_accessdeniedheader

See Issue #23178

Description

In order to send a correct HTTP header to the browser when access to a page is denied, a new option TYPO3_CONF_VARS is introduced.

The option `FE|pageNotFound_handling_accessdeniedheader` allows to configure the header which defaults to `HTTP/1.0 403 Access denied`.

Important: #77830 - CSC-HeaderLinkRespectsGlobalPageTarget

See Issue #77830

Description

Setting the global configuration `lib.parseTarget` was not respected by the header_link field. Now the configuration is properly applied and might change the output in the frontend.

Impact

If the global setting `lib.parseTarget` is set, the field header_link will now respect it. If in addition the target of header_link is set in a content element, it will take precedence over `lib.parseTarget`.

Important: #81751 - DBAL compatible quoting in TCA

See Issue #81751

Description

Names of tables and columns used in SQL fragments of `TCA` definitions need proper quoting to be compatible with different database drivers. The database framework of the core now applies proper quoting to table and column names if they are wrapped as `{#tableName}.{#columnName}`

It is advised to adapt extensions accordingly to run successfully on databases like PostgreSQL.

Example for a `TCA` definition snippet:

```
'columns' => [
    'aField' => [
        'config' => [
            'foreign_table' => 'tt_content',
            'foreign_table_where' => 'AND {#tt_content}.{#CType} IN (\'text\',\
↪'textpic\',\'textmedia\') ORDER BY {#tt_content}.{#CType} ASC',
            ...
        ],
    ],
    ...
],

'columns' => [
```

```
    'aField' => [
        'config' => [
            'type' => 'text',
            'search' => [
                'andWhere' => '{#CType}=\'text\' OR {#CType}=\'textpic\' OR {#CType}
↪=\'textmedia\''
            ],
            ...
        ],
    ],
    ...
],
```

Important: #82763 - Fluid config for ExpressionNodeType and TemplatePreProcessor made global

See Issue #82763

Description

Before, Fluid's arrays of class names for `ExpressionNodeType` and `TemplatePreProcessor` were hard-coded into the `RenderingContext` constructor and were not possible to modify except when having access to the `RenderingContext`.

Now, these two arrays of class names are possible to configure in `TYPO3_CONF_VARS` which allows extensions or site administrators to add and remove such Fluid components on a global level.

Example:

```
// Add one new ExpressionNodeType and one new TemplatePreProcessor to be used in␣
↪every RenderingContext
// For example from an ext_localconf.php file in an extension.
$GLOBALS['TYPO3_CONF_VARS']['SYS']['fluid']['expressionNodeTypes'][] =␣
↪\MyVendor\MyExtension\MyFluidExpressionNodeType::class;
$GLOBALS['TYPO3_CONF_VARS']['SYS']['fluid']['preProcessors'][] =␣
↪\MyVendor\MyExtension\MyFluidTemplatePreProcessor::class;
```

These classes will then be *added to the list of existing implementations* and will be automatically used as defaults when new `RenderingContext` instances are created.

See for reference how to create such implementations:

- `\TYPO3Fluid\Fluid\Core\Parser\TemplateProcessorInterface`
- `\TYPO3Fluid\Fluid\Core\Parser\SyntaxTree\Expression\ExpressionNodeInterface`

Both interfaces contain the documentation for how they must be implemented, when/why functions get called and what the expected return types are. The interfaces must of course be implemented by classes you add to `TYPO3_CONF_VARS` in these configuration sections. Due to their global nature you should be very careful to implement the classes and in particular observe the return types.

Important: #82794 - Added config.sys_language_mode = fallback;3,2,stop

See Issue #82794

Description

If a translation (language UID 5) has a TypoScript configuration to `config.sys_language_mode = content_fallback;3,2` the definition is that if this page is not available in this translation (language=5) then check if a translation for 3 and after that "2" is set.

However, if none of the page translations is available, the fallback to "0" always applies.

On a set up like: - language=0 is german - language=2 is english-worldwide - language=3 is english-US - language=5 is russian

You would not want to fall back to german AT ALL.

It is now possible to define a special keyword called "pageNotFound" to not fall back to sys_language_uid=0 if any other fallbacks do not work - so a 404 error page is thrown. `config.sys_language_mode = content_fallback;3,2,pageNotFound`

2.2 8.7 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.2.1 Breaking Changes

Breaking: #79615 - QueryBuilder getQueriedTables result format change

See Issue #79615

Description

`QueryBuilder::getQueriedTables` return value has been changed from array with key being table name and value being table alias (or null) to array with a key being table alias and value being table name. Both keys and the value should be filled. This allows to return multiple entries for the same table (in case inner join is made).

Example for the `tt_content` table inner joined with self and joined with `sys_language`:

```
[
  'tt_content_alias' => 'tt_content',
  'tt_content' => 'tt_content',
  'sys_language' => 'sys_language'
]
```

Previously the array (for the same case) looked like:

```
[
  'tt_content' => 'tt_content_alias',
  'sys_language' => null
]
```

Impact

All code which rely on the result format of the `getQueriedTables` method needs to be adapted. The first parameter of the `QueryRestrictionInterface::buildExpression` (`$queriedTables`) expects a new array structure.

Affected Installations

All installations with custom implementation of Query Restriction (classes implementing `QueryRestrictionInterface`). All installations where table array passed to `buildExpression` method is created manually (without using `QueryBuilder::getQueriedTables`).

Migration

The code of the `buildExpression` method in custom `QueryRestrictionInterface` implementations needs to adapted to be able to handle the new incoming array structure. Format of the first parameter passed to `buildExpression` needs to be adapted in case a query restriction is used directly (without using `QueryBuilder::getQueriedTables`).

Breaking: #80050 - Remove option cHashIncludePageId from cHash calculation

See Issue #80050

Description

The global configuration option `$GLOBALS['TYPO3_CONF_VARS']['FE']['cHashIncludePageId']` has been removed, as the functionality is now always activated.

The option was introduced in 2016 as part of a security bugfix for existing releases to allow the inclusion the cHash calculation (= caching identifier for pages with different GET variables) and was active for new installations.

Impact

Setting the option has no effect anymore.

If the option was disabled before, all existing cached contents and existing cHash calculations for URL rewrites (e.g. RealURL) of existing pages are invalidated and will throw a "page not found" exception if called directly.

Affected Installations

Any existing TYPO3 installation that did not have the option activated before.

Breaking: #80149 - Remove $GLOBALS['TYPO3_CONF_VARS']['FE']['pageOverlayFields']

See Issue #80149

Description

The configuration `$GLOBALS['TYPO3_CONF_VARS']['FE']['pageOverlayFields']` is removed from the default configuration as well as from the overlay handling in PageRepository and RootlineUtility.

This setting has been used to determine overlay fields in the table `pages_language_overlay` at a time in the runtime processing when the complete TCA was not fully available. Since the `allowLanguageSynchronization` possibility has been integrated into TYPO3 CMS 8, `l10n_mode` was available already and the TCA is loaded as well, the `pageOverlayFields` settings are superfluous.

Impact

Since `$GLOBALS['TYPO3_CONF_VARS']['FE']['pageOverlayFields']` was used as a filter for field names to be taken from `pages_language_overlay` and merged onto those fields in `pages`, all fields are overlaid per default.

Affected Installations

All installations having custom fields in table `pages_language_overlay` and custom settings in `$GLOBALS['TYPO3_CONF_VARS']['FE']['pageOverlayFields']`.

Migration

Check the TCA of `pages_language_overlay` and remove l10n_mode for those fields that previously were not defined in `$GLOBALS['TYPO3_CONF_VARS']['FE']['pageOverlayFields']` and thus should not be overlaid.

Breaking: #80171 - Remove lib.parseFunc_RTE inline styles from parsed blockquote tag

See Issue #80171

Description

With https://forge.typo3.org/issues/44879 the inline styles were removed from CSC. However, the change was not applied to FSC.

Impact

`lib.parseFunc_RTE` doesn't overwrite the attributes of `<blockquote/>` and doesn't add the style attribute anymore.

Affected Installations

All installations using the `lib.parseFunc_RTE` provided by EXT:fluid_styled_content where

```
lib.parseFunc_RTE {
   externalBlocks {
      blockquote {
         callRecursive {
            tagStdWrap {
               HTMLparser = 1
               HTMLparser.tags.blockquote.overrideAttribs = style="margin-bottom:0;
↪margin-top:0;"
            }
         }
      }
   }
}
```

was not overwritten manually.

Migration

If your frontend relies on this inline CSS styles, make sure to add following CSS on your own:

```
blockquote {
   margin-top: 0;
   margin-bottom: 0;
}
```

Additionally you have to check if some other attributes are now added/preserved to `<blockquote/>` due the fact, that the former typoscript has removed all attributes before adding the style attribute.

Breaking: #80374 - Default content element configuration for frontend login adapts fluid styled content

See Issue #80374

Description

Default configuration for the frontend login content element was adapted to match fluid styled content instead of css styled content by default. Css styled content was adapted and works there as before.

Rendering for css styled content

```
tt_content.login = COA
tt_content.login {
   10 =< lib.stdheader
   20 =< plugin.tx_felogin_pi1
}
```

Rendering for fluid styled content

```
tt_content.login =< lib.contentElement
tt_content.login {
   templateName = Generic
   variables {
      content =< plugin.tx_felogin_pi1
   }
}
```

Impact

Adjustments made manually to the TypoScript rendering definition of `tt_content.login` might not work in fluid styled content as expected.

Affected Installations

Installations that are using fluid styled content and directly modify configuration of `tt_content.login`.

Migration

Manual adaption is necessary.

Breaking: #80412 - New shared content element TypoScript libary object for Fluid Styled Content

See Issue #80412

Description

To solve an inconsistency issue for API based content element registration between CSS Styled Content (CSC) and Fluid Styled Content (FSC) through `Extbase` or `addPItoST43` we are now introducing a new shared content object for content elements and drop the active usage of `lib.fluidContent`.

The generated code relied on the existence of the removed `lib.stdheader` and also ignored layouts, frames, spacebefore, spaceafter in context of Fluid Styled Content.

For content element registration the TypoScript `lib.contentElement` is now used for `CSC` and `FSC` and replaces the usage of `lib.contentElement`. The generated code was slightly adjusted to match the requirements of all content rendering definitions and can be adapted to the specific needs of a content element rendering definition anytime since a reference is used now instead of a hard definition.

Generated code before change

```
tt_content.myce = COA
tt_content.myce {
   10 =< lib.stdheader
   20 =< plugin.myContent
}
```

Generated code after change

```
tt_content.myce =< lib.contentElement
tt_content.myce {
   templateName = Generic
   20 =< plugin.myContent
}
```

CSS Styled Content

CSS Styled Content adds the missing `lib.stdheader` and everything works as before, no migration or adjustments to your code nessesary. Because `COA` does not understand the option `templateName` it will simply be ignored.

```
lib.contentElement = COA
lib.contentElement {
   10 =< lib.stdheader
}
```

Fluid Styled Content

Fluid Styled Content adds the logic it needs through `lib.contentElement`. All content elements registered through the TYPO3 APIs will now share a multifunctional `Generic` template. That will provide the necessary layouts and overriding options known from FSC.

```
lib.contentElement = FLUIDTEMPLATE
lib.contentElement {
   templateRootPaths {
      0 = EXT:fluid_styled_content/Resources/Private/Templates/
      10 = {$styles.templates.templateRootPath}
   }
   partialRootPaths {
      0 = EXT:fluid_styled_content/Resources/Private/Partials/
      10 = {$styles.templates.partialRootPath}
   }
   layoutRootPaths {
      0 = EXT:fluid_styled_content/Resources/Private/Layouts/
      10 = {$styles.templates.layoutRootPath}
   }
   ...
}
```

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers" data-namespace-
↪typo3-fluid="true">
<f:layout name="Default" />
<f:section name="Main">

   <f:comment>This templates is used to provide necessary functionality for␣
↪external processed content and could be used across multiple sources, for␣
↪example the frontend login content element.</f:comment>
   <f:if condition="{content}">
      <f:then>{content -> f:format.raw()}</f:then>
      <f:else><f:cObject typoscriptObjectPath="tt_content.{data.CType}.20" data="
↪{data}" table="tt_content" /></f:else>
   </f:if>

</f:section>
</html>
```

The TypoScript Object `lib.fluidContent` will be kept as copy of `lib.contentElement` for compatibility for the duration of TYPO3 v8 LTS and will be removed in TYPO3 v9.

Impact

Assignments and overrides made directly to `lib.fluidContent` are not recognized anymore for core content elements provided by Fluid Styled Content. They need to be migrated to `lib.contentElement`. Only not modified versions of `lib.fluidContent` will keep working as expected.

Affected Installations

Installations that directly modify `lib.fluidContent`.

Migration

Rename assignments and modifications to `lib.contentElement`.

Breaking: #80628 - Extension rtehmlarea moved to TER

See Issue #80628

Description

The legacy extension `EXT:rtehtmlarea` has been removed from the TYPO3 CMS core and is only available as TER extension.

Impact

The new extension `EXT:rte_ckeditor` is loaded by default, if you need features of the old rtehmlarea extension, you have to install `EXT:rtehtmlarea` from TER. An upgrade wizard can do this for you in the upgrade process of the install tool. If you have allowed images in RTE, you should install the rtehtmlarea extension, the ckeditor extension does not support images in RTE.

Affected Installations

Most installations are not affected. Instances are only affected if a loaded extension has a dependency to `EXT:rtehtmlarea` extension, or if the instance has used special plugins.

Migration

Use the upgrade wizard provided by the install tool to fetch and load the extensions from TER if you really need it.

2.2.2 Features

Feature: #79343 - Allow overriding PATH_site via environment variable

See Issue #79343

Description

It is now possible to define the `PATH_site` constant, which acts as a basis for any entry point running a TYPO3 system, via the environment variable `TYPO3_PATH_ROOT`.

This variable is automatically calculated and set for any TYPO3 installation set up via composer, making it possible to run the TYPO3 command line interface from any location of the system.

Impact

When using the command line entry-point `typo3/sysext/core/bin/typo3` with composer, it can also be called from a projects' default `bin/` directory.

Feature: #79812 - Allow overriding cropVariants for Image Manipulation

See Issue #79812

Description

With the introduction of Issue #75880 you now can define multiple cropVariants in TCA. With this feature it is now possible to change or override these cropVariants via TSconfig.

Setting a FormEngine option through `TCEFORM.sys_file_reference.crop.config.cropVariants.*` does now work.

```
TCEFORM.sys_file_reference.crop.config.cropVariants {
    default {
        title = Default desktop
        selectedRatio = NaN
        allowedAspectRatios {
            NaN {
                title = free
                value = 0.0
            }
        }
    }
    specialMobile {
        title = Our special mobile variant
        selectedRatio = NaN
        allowedAspectRatios {
            4:3 {
                title = ratio 4/3
                value = 1.3333333
            }
        }
    }
}
```

Impact

It is not possible to change or override cropVariants via Page and User TSconfig.

Feature: #79883 - Add cropVariant support to TypoScript rendering of images

See Issue #79883

Description

The introduction of the new crop variants Issue #75880 broke the handling of cropped images when using TypoScript to render file(reference)'s. This feature fixes this and introduces a new TypoScript option to use a different cropVariant.

To use a different `cropVariant` as default you can provide the `cropVariant` name now in your TypoScript configuration. If `cropVariant` isn't provided the default variant will be used.

```
# Use specific cropVariant for the images
tt_content.image.20.1.file.cropVariant = mobile
```

Impact

If multiple cropVariants are available (see Issue #75880) you can now configure which variant to use with the `cropVariant` option of `imgResource`.

Feature: #80126 maximum field length not set as attribute "maxlength"

See Issue #80126 See Issue #80128

Description

If a form element is set to be use the 'String length' server side validation through the form editor, the client side validation properties `minlength` and `maxlength` will be rendered.

Result:

```
renderables:
  -
    type: <formElementType>
    ...
    properties:
      fluidAdditionalAttributes:
        minlength: 2
        maxlength: 3
        ...
    validators:
      -
        identifier: StringLength
        options:
          minimum: 2
          maximum: 3
```

Feature: #80154 - Retrieve session data in TS

See Issue #80154

Description

As the session API has been modified, it is no longer possible to access session data from TypoScript by accessing the formerly public property of the fe_user with:

```
page.10 = TEXT
page.10.data = TSFE:fe_user|sesData|myext|mydata
```

This is being replaced by a more direct way, which allows for the same functionality:

```
page.10 = TEXT
page.10.data = session:myext|mydata
```

Feature: #80196 - EXT:form - support multiple form elements per row

See Issue #80196

Description

Two new form element types have been added to the form framework:

- GridContainer
- GridRow

Using these 'container' form elements will enable you to define multiple form elements per row.

Example:

```yaml
type: Form
identifier: example-form-gridcontainer
label: 'Form Grid Container'
prototypeName: standard
renderables:
  -
    type: Page
    identifier: page-1
    label: Page
    renderables:
      -
        type: GridContainer
        identifier: gridcontainer-2
        label: 'Grid: Container'
        renderables:
          -
            type: GridRow
            identifier: gridrow-2
            label: 'Grid: Row'
            renderables:
              -
                type: SingleSelect
                identifier: singleselect-1
                label: 'Single select'
                properties:
                  gridColumnClassAutoConfiguration:
                    viewPorts:
                      xs:
                        numbersOfColumnsToUse: 12
                      lg:
                        numbersOfColumnsToUse: 2
              -
                type: Text
                identifier: text-1
                label: Text
                properties:
                  gridColumnClassAutoConfiguration:
                    viewPorts:
                      xs:
                        numbersOfColumnsToUse: 6
                      lg:
                        numbersOfColumnsToUse: 5
              -
                type: MultiSelect
                identifier: multiselect-1
                label: 'Multi select'
                properties:
                  gridColumnClassAutoConfiguration:
                    viewPorts:
                      xs:
                        numbersOfColumnsToUse: 6
                      sm:
                        numbersOfColumnsToUse: 5
      -
        type: GridContainer
        identifier: gridcontainer-1
        label: 'Grid: Container'
        renderables:
          -
            type: GridRow
            identifier: gridrow-1
            label: 'Grid: Row'
            renderables:
```

```
            -
              type: Password
              identifier: password-1
              label: Password
```

Per default, the resulting markup is compatible to Twitter Bootstrap.

The following options are available now:

```
GridContainer:
  ...
  properties:
    columnClassAutoConfiguration:
      gridSize: 12
      viewPorts:
        xs:
          classPattern: 'col-xs-{@numbersOfColumnsToUse}'
        sm:
          classPattern: 'col-sm-{@numbersOfColumnsToUse}'
        md:
          classPattern: 'col-md-{@numbersOfColumnsToUse}'
        lg:
          classPattern: 'col-lg-{@numbersOfColumnsToUse}'
```

and

```
<formElementIdentifier>:
  ...
  properties:
    gridColumnClassAutoConfiguration:
      viewPorts:
        xs:
          numbersOfColumnsToUse: 12
        ...
        lg:
          numbersOfColumnsToUse: 2
```

GridContainer.properties.columnClassAutoConfiguration

The example form definition shown above generates the following HTML markup

```
<div class="container">
    <div class="row">
        <div class="col-xs-12 col-sm-3 col-md-4 col-lg-2">
            ...
        </div>
        <div class="col-xs-6 col-sm-3 col-md-4 col-lg-5">
            ...
        </div>
        <div class="col-xs-6 col-sm-5 col-md-4 col-lg-5">
            ...
        </div>
    </div>
</div>
```

GridContainer.properties.columnClassAutoConfiguration.gridSize

Total amount of grid columns (default: 12).

GridContainer.properties.columnClassAutoConfiguration.viewPorts.<viewPortName>.classPattern

This pattern will be used to generate the HTML class attribute values for each viewport. The wildcard '{@numbersOfColumnsToUse}' will be replaced with the calculated grid column numbers. At the end, all 'classPattern'

items for each viewport will be merged together and written into the class attribute of each form element (all form elements within a 'GridRow').

The calculation depends on the option 'gridSize', the amount of the form elements within the 'GridRow' form element and the optional option 'gridColumnClassAutoConfiguration' from the form element configurations.

<formElementIdentifier>.properties.gridColumnClassAutoConfiguration (otional)

Each form elements within a 'GridRow' element can define the number of grid columns to use on a 'per viewport' base.

<formElementIdentifier>.properties.gridColumnClassAutoConfiguration.viewPorts.<viewPortName>

The array keys '<viewPortName>' must match with the array keys '<viewPortName>' from the configuration 'GridContainer.properties.columnClassAutoConfiguration.viewPorts.<viewPortName>'

<formElementIdentifier>.properties.gridColumnClassAutoConfiguration.viewPorts.<viewPortName>.numbersOfColumnsT

The number of grid columns to be used by this element for the viewport '<viewPortName>'.

This number goes hard to the '{@numbersOfColumnsToUse}' wildcard from the configuration 'GridContainer.properties.columnClassAutoConfiguration.viewPorts.<viewPortName>.classPattern'

If nothing is set, the {@numbersOfColumnsToUse} will be calculated automatically.

Impact

You are now able to add multiple form elements per row via the API and the form editor.

Feature: #80374 - Add generic fluid template for already rendered content

See Issue #80374

Description

To provide better support for content elements where the content itself is not processed by fluid we introduce a new generic template, to make it easy to benefit from the universal layouts of fluid styled content.

The generic template only wrapps already generated html that have been assigned to the variable `content`. This eliminates the need for extensions to provide custom templates to wrap their external rendered content to achieve the same behaviour as other fluid styled content elements.

Template

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers" data-namespace-
↪typo3-fluid="true">
<f:layout name="Default" />
<f:section name="Main">
   <f:comment>This templates is used to provide nessesary functionality for␣
↪external processed content and could be used across multiple sources, for␣
↪example the frontend login content element.</f:comment>
   {content -> f:format.raw()}
</f:section>
</html>
```

Example Usage

```
tt_content.mycontent =< lib.contentElement
tt_content.mycontent {
   templateName = Generic
   variables {
      content = USER_INT
      content {
         userFunc = ACME\ContentExtension\Controller\SuperController->main
      }
   }
}
```

Feature: #80374 - Frontend Login configuration now available through TypoScript constants

See Issue #80374

Description

The most common configuration options for the "Frontend Login" configuration are now available as TypoScript constants, and moved to a new section "Frontend Login" in the constant editor.

Storage

styles.content.loginform.pid Storage Folder: Define the Storage Folder with the Website User Records, using a comma separated list or single value

styles.content.loginform.recursive Recursive: If set, also any subfolders of the storagePid will be used

Template

styles.content.loginform.templateFile Login template: Enter the path for the HTML template to be used

styles.content.loginform.feloginBaseURL BaseURL for generated links: Base url if something other than the system base URL is needed

styles.content.loginform.dateFormat Date format: Format for the link is valid until message (forget password email)

Features

styles.content.loginform.showForgotPasswordLink Display Password Recovery Link: If set, the section in the template to display the link to the forget password dialogue is visible.

styles.content.loginform.showPermaLogin Display Remember Login Option: If set, the section in the template to display the option to remember the login (with a cookie) is visible.

styles.content.loginform.showLogoutFormAfterLogin Disable redirect after successful login, but display logout-form: If set, the logout form will be displayed immediately after successful login.

E-Mail

styles.content.loginform.emailFrom E-Mail Sender Address: E-Mail address used as sender of the change password emails

styles.content.loginform.emailFromName E-Mail Sender Name: Name used as sender of the change password emails

styles.content.loginform.replyToEmail Reply To E-Mail Address: Reply-to address used in the change password emails

Redirects

styles.content.loginform.redirectMode Redirect Mode: Comma separated list of redirect modes. Possible values: groupLogin, userLogin, login, getpost, referer, refererDomains, loginError, logout

styles.content.loginform.redirectFirstMethod Use First Supported Mode from Selection: If set the first method from redirectMode which is possible will be used

styles.content.loginform.redirectPageLogin After Successful Login Redirect to Page: Page id to redirect to after Login

styles.content.loginform.redirectPageLoginError After Failed Login Redirect to Page: Page id to redirect to after Login Error

styles.content.loginform.redirectPageLogout After Logout Redirect to Page: Page id to redirect to after Logout

styles.content.loginform.redirectDisable Disable Redirect: If set redirecting is disabled

Security

styles.content.loginform.forgotLinkHashValidTime Time in hours how long the link for forget password is valid: How many hours the link for forget password is valid

styles.content.loginform.newPasswordMinLength Minimum amount of characters, when setting a new password: Minimum length of the new password a user sets

styles.content.loginform.domains Allowed Referrer-Redirect-Domains: Comma separated list of domains which are allowed for the referrer redirect mode

styles.content.loginform.exposeNonexistentUserInForgotPasswordDialog Expose existing users: Expose the information on whether or not the account for which a new password was requested exists. By default, that information is not disclosed for privacy reasons.

Impact

Frontend Login configuration is now always added first and not depending anymore and not depending anymore on the configuration of the TypoScript template. This allows reliable configuration since the configuration is not a moving target.

Feature: #80452 - Extbase CLI commands available via new CLI API

See Issue #80452

Description

Any Extbase Command Controller can now be accessed via the new Symfony Console CLI entrypoint by simply calling `typo3/sysext/core/bin/typo3 controller:command`.

Using the existing CLI entrypoint via `typo3/cli_dispatch.phpsh extbase controller:command` still works as expected.

Feature: #80579 - Improved JavaScript Modal API

See Issue #80579

Description

To improve the usability and flexibility for a unified handling of overlays in the backend we're opening the existing API for modals to be more flexible and adjustable to your needs for advanced usage. With the introduction of the advanced API it is now possible to pass configuration by json object. This enables more easy configuration and better fallback if the modal is not correctly configured.

For a unified experience all modals are now centered by default, will kept in place automatically and are available in different sized depending on your needs. In addition to this there is also a new type for loading content into an iframe. Buttons have now full support for the TYPO3 Icon API and data attributes that also can set by configuration.

Advanced API

Unlike the existing api functions like `Modal.confirm`, `Modal.loadUrl` or `Modal.show`, `Modal.advanced` uses a JavaScript object instead of fixed parameters.

```
require([
   'jquery',
   'TYPO3/CMS/Backend/Modal'
   ], function ($, Modal) {

      var configuration = {
         type: Modal.types.iframe,
         title: title,
         content: url,
         size: Modal.sizes.large,
         callback: function(currentModal) {
            currentModal.find('.t3js-modal-body')
               .addClass('custom-css-class');
         }
      };
      Modal.advanced(configuration);

   });
}
```

Configuration Options

Type

The `type` will define the behaviour of content loading, and only accepts `Modal.types.ajax`, `Modal.types.iframe` and the default `Modal.types.default`.

```
var configuration = {
   type: Modal.types.iframe,
   content: url || content
};
```

Modal.types.default Default will display static content set in the option `content`.

Modal.types.ajax Content will be grabbed from a url set in the option `content`

Modal.types.iframe Url provided in the option `content` will be loaded in a iframe in the modal. Also it will automaticly set the title from the contained document.

Title

The `title` will be display above the modal content. For the type `Modal.types.iframe` this option will have no effect. As soon as the content from the iframe is loaded, the title will be replaced with title of the contained document. The default will set the title to "Information".

```
var configuration = {
   title: 'My Title'
};
Modal.advanced(configuration);
```

Content

The `content` accepts only strings that can be either a HTML or a url for types `Modal.types.ajax` and `Modal.types.iframe`. The default will show a warning that there is a possible misconfiguration of the modal.

```
var configurationStatic = {
   type: Modal.types.default,
   content: 'My Title'
};
Modal.advanced(configurationStatic);
```

```
var configurationAjax = {
   type: Modal.types.ajax,
   content: 'http://www.google.de/'
};
Modal.advanced(configurationAjax);
```

```
var configurationIframe = {
   type: Modal.types.iframe,
   content: 'http://www.google.de/'
};
Modal.advanced(configurationIframe);
```

Severity

Severity is used to change the appearance of the modal window to represent a contextual state like success, information, warning or danger. The default is `Severity.notice`. Only options provided by the `Severity` object will be accepted.

```
var configuration = {
   severity: Severity.info,
};
Modal.advanced(configuration);
```

Buttons

Defined buttons will be display displayed at the bottom of the modal window. The configuration accepts an array of single button definitions.

```
var configuration = {
   buttons: [
      {
         text: 'Save changes',
         name: 'save',
         icon: 'actions-document-save'
```

```
            active: true,
            btnClass: 'btn-primary',
            dataAttributes: {
               action: 'save'
            },
            trigger: function() {
               Modal.currentModal.trigger('modal-dismiss');
            }
         }
      ]
};
Modal.advanced(configuration);
```

text Text that will be displayed in the button

name Value of the name attribute of the button

icon Name of the icon that will be displayed in front of the text

active Activated button after opening the modal window

btnClass Additional css class that will be added to the button

dataAttributes Object of data attributes that will be added to the button

trigger Callback function that will be triggered then the button is clicked

Style

The `style` option will change the appearance of the modal like the `severity` both without contextual meaning. The default option is `Modal.styles.light`. The second available option is `Modal.styles.dark` will override all contextual styling.

```
var configuration = {
   style: Modal.styles.default
}
Modal.advanced(configuration);
```

Size

While the modal itself adapts to the window, there are several options available to limit the maximal size of the modal. The sizes `Modal.sizes.small` and the default `Modal.sizes.default` will automaticly adapt to the content and are only limited to the width of the modal. `Modal.sizes.large` and `Modal.sizes.full` are designed to contain a undefined lenth of content in a fixed sized modal. These are suited best for `Modal.types.ajax` or `Modal.types.iframe` content.

```
var configuration = {
   size: Modal.sizes.large
}
Modal.advanced(configuration);
```

Modal.sizes.small Limited to 400px width

Modal.sizes.default Limited to 600px width

Modal.sizes.large Limited to 800px width and 600px height

Modal.sizes.full Limited to 1800px width and 1200px height

Additional CSS Classes

The option `additionalCssClasses` accepts an array of css classes that will be added to the modal frame.

```
var configuration = {
   additionalCssClasses: [
      'class1',
      'class2'
   ]
}
Modal.advanced(configuration);
```

Callback

Callback function that will be called after the modal is processed.

```
var configuration = {
   callback: function(currentModal) {
      currentModal.find('.t3js-modal-body')
         .addClass('custom-css-class');
   }
}
Modal.advanced(configuration);
```

Callback after ajax processing

Callback function that will be called after the ajax call has been done and the response added to the desired location. This option is only available for type `Modal.types.ajax`.

```
var configuration = {
   type: Modal.types.ajax,
   ajaxCallback: function() {
      do();
   }
}
Modal.advanced(configuration);
```

Target for ajax response

The ajax response will be added to the body of the particular modal window by default but can be set to a different selector if necessary. This option is only available for type `Modal.types.ajax`.

```
var configuration = {
   type: Modal.types.ajax,
   ajaxTarget: '.t3js-modal-footer'
}
Modal.advanced(configuration);
```

Feature: #80619 - Extend Link Generation within TypoLink

See Issue #80619

Description

Generating a link to a page, email, url, email in the TYPO3 Frontend is usually handled via the so-called `typolink` functionality. Generating links is now flexible, extensions can register their own link-building functionality via `$GLOBALS[TYPO3_CONF_VARS][FE][typolinkBuilder][$linkType]` in the extensions `ext_localconf.php`.

All existing functionality for TypoLink via TypoScript etc. still works as before.

Impact

The TYPO3 Core itself handles all native link types (email, url, page, record, file, folder) via this functionality already, and it can be overridden.

The functionality goes hand-in-hand with the LinkService registration functionality for setting links of a specific type.

2.2.3 Deprecation

Deprecation: #78650 - TemplateService->splitConfArray

See Issue #78650

Description

The method `TemplateService->splitConfArray` which has been used for building the "optionSplit" functionality has been marked as deprecated.

The method is now moved to a new class called `TypoScriptService`, effectively removing the dependency on `$TSFE->tmpl` within a ContentObject.

Impact

Calling `TemplateService->splitConfArray` will throw a deprecation warning.

Affected Installations

Any installation using an extension that calls this method.

Migration

Use the new method `TypoScriptService->explodeConfigurationForOptionSplit` instead.

Deprecation: #79122 - Deprecate method getRecordsByField

See Issue #79122

Description

The method `TYPO3\CMS\Backend\Utility\BackendUtility::getRecordsByField()` has been deprecated and should not be used any longer.

Impact

Calling the deprecated `getRecordsByField()` method will trigger a deprecation log entry.

Affected Installations

Any installation using the mentioned method `getRecordsByField()`.

Migration

Use the `ConnectionPool` and the `QueryBuilder` classes directly to query the database from your code.

Deprecation: #79580 - Deprecate methods in DataHandler related to page delete access

See Issue #79580

Description

The following methods have been marked as deprecated:

- `TYPO3\CMS\Core\DataHandling\DataHandler->rmComma()`
- `TYPO3\CMS\Core\DataHandling\DataHandler->noRecordsFromUnallowedTables()`

Impact

Calling these methods will trigger a deprecation log entry.

Affected Installations

Any TYPO3 extension calling one of these methods.

Migration

Use native `rtrim($input,',')` instead of `TYPO3\CMS\Core\DataHandling\DataHandler->rmComma()`.
No migration available for `TYPO3\CMS\Core\DataHandling\DataHandler->noRecordsFromUnallowedTables(`

Deprecation: #79591 - Extbase command controllers admin role methods

See Issue #79591

Description

The methods `CommandController->ensureAdminRoleIfRequested()` and `CommandController->restoreUserRole()` have been marked as deprecated.

All CLI scripts are now executed with administrator access rights, so this functionality is obsolete.

Impact

Calling any of the methods above will trigger a deprecation log warning.

Affected Installations

Any installation with custom CLI Extbase Command Controllers using the methods above.

Migration

Remove the affected lines where the methods are called, as they are not necessary anymore.

Deprecation: #79770 - Deprecate inline localizationMode

See Issue #79770

Description

The `localizationMode` for inline relational record editing types is deprecated.

Impact

Using `localizationMode` set to `keep` and having `allowLanguageSynchronization` enabled at the same time is counter-productive, since it will deny the synchronization process for the affected field. That's why `localizationMode` is unset only if `allowLanguageSynchronization` is enabled.

Affected Installations

All having `$TCA[<table-name>]['columns'][<field-name>]['config']['behaviour']['localizationM` defined for database tables that support translations.

Migration

Remove `$TCA[<table-name>]['columns'][<field-name>]['config']['behaviour']['localizationMo` definitions and make use of either one of the following

- `$TCA[<table-name>]['columns'][<field-name>]['config']['behaviour']['allowLanguageSyn = true` if editors can decide whether to provide custom child references or synchronize all references from the language parent record - this comes close to `localizationMode=select` without having the possibility to selectively translate child references
- `$TCA[<table-name>]['columns'][<field-name>]['l10n_mode'] = 'exclude'` if editors don't have a choice to translate child references - this corresponds to `localizationMode=keep`

Deprecation: #79858 - TSFE-related properties and methods

See Issue #79858

Description

The following properties within TypoScriptFrontendController have been marked as deprecated:

- `$compensateFieldWidth`
- `$excludeCHashVars`
- `$scriptParseTime`

The following methods have been marked as deprecated:

- `TypoScriptFrontendController->generatePage_whichScript()` (used via `config.pageGenScript`)
- `TypoScriptFrontendController->encryptEmail()`
- `TypoScriptFrontendController->encryptCharcode()`
- `PageGenerator::pagegenInit()`

The following TypoScript properties have been marked as deprecated:

- `config.pageGenScript`
- `config.compensateFieldWidth`

Impact

Calling any of the PHP methods will trigger a deprecation log entry.

All properties and options are still callable with the according output, however there are alternatives to achieve the same.

Affected Installations

Any TYPO3 installation working with custom extensions that use any of these functionalities, methods or properties.

Migration

All of the functionality is obsolete or outdated and should be handled differently from now on:

1. The `compensateFieldWidth` option was used for forms built with TYPO3 4.x (before TYPO3 4.6), instead, any other form framework should be used for forms and for field width calculations, where styling of form fields are also handled via CSS.

2. An alternative `config.pageGenScript` can be used and set via hooks in PHP classes nowadays and executed, instead of configuring this functionality on a high-end TypoScript level to execute include spaghetti PHP code within a file.

3. `PageGenerator::pagegenInit()` is solely working on public properties of the TSFE PHP class, which belongs to the TSFE object itself (thus, the logic is copied to `$TSFE->preparePageContentGeneration()`)

4. Calculating the debug parse time for the web page is not part of the controller logic but more certainly belongs to the request handling itself, where it is handled in a cleaner way for PHP, waiting for further refactorings in TYPO3 v9.

5. The methods `TypoScriptFrontendController->encryptEmail()` and `encryptCharcode()` have been moved to ContentObjectRenderer.

Deprecation: #79972 - Deprecated Fluid Overrides

See Issue #79972

Description

following methods have been marked as deprecated:

- `\TYPO3\CMS\Fluid\Core\Rendering\RenderingContext::injectObjectManager()`
- `\TYPO3\CMS\Fluid\Core\Rendering\RenderingContext::getObjectManager()`
- `\TYPO3\CMS\Fluid\Core\Rendering\RenderingContext::setLegacyMode()`

following methods have been removed:

- `\TYPO3\CMS\Fluid\Core\Rendering\RenderingContext::getExpressionNodeTypes()`
- `\TYPO3\CMS\Fluid\Core\Rendering\RenderingContext::getViewHelperVariableContainer()`

following classes have been removed:

- `\TYPO3\CMS\Fluid\Core\Parser\PreProcessor\XmlnsNamespaceTemplatePreProcessor`
- `\TYPO3\CMS\Fluid\Core\Parser\SyntaxTree\Expression\LegacyNamespaceExpressionNode`

Impact

Calling any of the methods marked as deprecated above will trigger a deprecation log entry. Accessing any of the removed classes or methods will cause a PHP Fatal Error.

Affected Installations

Any TYPO3 instances which uses the above described methods or classes.

Migration

- Remove usage of classes / methods.

Deprecation: #80000 - InlineOverrideChildTca

See Issue #80000

Description

These `TCA` `type=inline` properties have been deprecated and superseded with the more general property `overrideChildTca`:

- foreign_types
- foreign_selector_fieldTcaOverride
- foreign_record_defaults

Impact

It is now possible to override display (FormEngine) related columns properties and the types section of child `TCA` from within the parent `TCA`. This is also allowed in a parents `['types']['columnsOverrides']` section.

Affected Installations

Instances using one of the above inline properties should adapt to the new `overrideChildTca` property.

Migration

A `TCA` auto-migration is in place. It will transfer the old settings to the new property as shown below and logs deprecation entries if there is no `overrideChildTca` defined. This allows extension authors to keep both the old and the new settings to support CMS v7 and v8 at the same time without having deprecations logged.

foreign_types before and after transition to overrideChildTca:

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'foreign_types' => [
                'aForeignType' => [
                    'showitem' => 'aChildField',
                ],
            ],
            ...
        ],
    ],
    ...
],
```

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'overrideChildTca => [
                'types' => [
                    'aForeignType' => [
                        'showitem' => 'aChildField',
                    ],
                ],
            ],
            ...
        ],
    ],
    ...
],
```

foreign_selector_fieldTcaOverride before and after transition to overrideChildTca:

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'foreign_selector' => 'uid_local',
            'foreign_selector_fieldTcaOverride' => [
                'config' => [
                    'appearance' => [
                        'elementBrowserType' => 'file',
                    ],
                ],
            ],
            ...
        ],
    ],
```

```
    ...
],
```

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'foreign_selector' => 'uid_local',
            'overrideChildTca => [
                'columns' => [
                    'uid_local' => [
                        'config' => [
                            'appearance' => [
                                'elementBrowserType' => 'file',
                            ],
                        ],
                    ],
                ],
            ],
            ...
        ],
    ],
    ...
],
```

foreign_record_defaults before and after transition to overrideChildTca:

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'foreign_record_defaults' => [
                'aChildField' => 42,
            ],
            ...
        ],
    ],
    ...
],
```

```
'columns' => [
    'aField' => [
        'config' => [
            'type' => 'inline',
            'overrideChildTca' => [
                'columns' => [
                    'aChildField' => [
                        'config' => [
                            'default' => 42,
                        ],
                    ],
                ],
            ],
            ...
        ],
    ],
    ...
],
```

Deprecation: #80027 - Remove TCA config 'max' on inputDateTime fields

See Issue #80027

Description

The TCA migration removes the 'max' config option for renderType="inputDateTime" since this should not be set for this renderType.

Impact

Has an impact on performance during saving of records.

Affected Installations

All installations using the 'max' config option for renderType="inputDateTime".

Migration

Remove the 'max' config option for renderType="inputDateTime".

Deprecation: #80047 - Deprecate jQuery and extJS for BE viewhelpers

See Issue #80047

Description

The BE related ViewHelpers `<f:be.container>` and `<f:be.pageRenderer>` have properties to activate ExtJS and jQuery with various options. The usage of ExtJS has been deprecated and jQuery is always loaded. Therefore the following attributes have been marked as deprecated.

`<f:be.container>`

- `enableClickMenu`
- `loadExtJs`
- `loadExtJsTheme`
- `enableExtJsDebug`
- `loadJQuery`
- `jQueryNamespace`

`<f:be.pageRenderer>`

- `loadExtJs`
- `loadExtJsTheme`
- `enableExtJsDebug`
- `loadJQuery`
- `jQueryNamespace`

Impact

Using these attributes will trigger a deprecation log entry. Code using them will work until these methods are removed in TYPO3 v9.

Affected Installations

Any installation using the mentioned attributes.

Migration

Use `includeRequireJsModules` property of the `<f:be.pageRenderer>` or `<f:be.container>` ViewHelpers to add needed RequireJS modules.

Example:

```
<f:be.pageRenderer
   includeRequireJsModules="{
      0:'TYPO3/CMS/Backend/ContextMenu'
   }"
/>
```

See also documentation about RequireJS documentation

Deprecation: #80048 - Mark ExtJS related API calls as deprecated

See Issue #80048

Description

The usage of ExtJS has been marked as deprecated. Therefore the following methods of `ExtensionManagementUtility` have been marked as deprecated:

- `addExtJSModule`
- `registerExtDirectComponent`

Impact

Calling any of the PHP methods will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation working with custom extensions that use any of these methods.

Migration

All of the functionality is obsolete or outdated and should be handled differently from now on:

1. Use `ExtensionManagementUtility::addModule` instead of `addExtJSModule`.
2. Some ajax routes instead of ExtDirect.

Deprecation: #80053 - Extbase CLI Console Output different method signature for infinite attempts

See Issue #80053

Description

When using Extbase's CLI functionality to ask for a question via `ConsoleOutput->select()` or `ConsoleOutput->askAndValidate()` the option to define infinite attempts has changed from "false" to "null".

Impact

Calling any of the methods with `$attempts = false` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 extension shipping custom CLI commands for Extbase using the methods above with the option to have infinite attempts.

Migration

Set the method argument from "false" to "null" in the Extbase Command of your extension.

Deprecation: #80076 - TypoScript option page.insertClassesFromRTE

See Issue #80076

Description

The TypoScript setting `page.insertClassesFromRTE` has been marked as deprecated.

The option enabled loading of CSS classes defined by backend PageTs config `RTE.classes` as inline CSS into a frontend page. However it did not take merged RTE options and userTS/pageTS overrides into account.

Impact

Setting `page.insertClassesFromRTE` in TypoScript will trigger a deprecation log entry.

Affected Installations

Any installation having the option activated in TypoScript.

Migration

In order to separate the functionality, all CSS classes which have been used in the RTE should be defined separately for the frontend rendering in a custom CSS/LESS/SASS file.

Deprecation: #80079 - Deprecated method Bootstrap::loadExtensionTables

See Issue #80079

Description

The internal method `TYPO3\CMS\Core\Core\Bootstrap::loadExtensionTables()` has been deprecated and should not be used any longer.

Impact

Calling the deprecated `Bootstrap::loadExtensionTables()` method will trigger a deprecation log entry.

Affected Installations

Any installation using the mentioned method `Bootstrap::loadExtensionTables()`. Please note that this method is marked as internal and should not be called at all from outside the TYPO3 core.

Migration

If you need to call the internal Bootstrap method, you can use `Bootstrap::loadBaseTca()` and `Bootstrap::loadExtTables()` now. Please note that both methods are marked as internal and don't belong to public TYPO3 core API. This means that the methods can be adjusted anytime by the core itself.

Deprecation: #80317 - Deprecate BackendUtility::getRecordRaw()

See Issue #80317

Description

Method `BackendUtility::getRecordRaw()` has been deprecated and should not be used any longer.

Impact

Extensions using above methods will throw a deprecation warning.

Affected Installations

All installations and extensions using the method `BackendUtility::getRecordRaw()`.

Migration

Use the QueryBuilder instead and remove all restrictions. For further information follow this link: querybuilder

Deprecation: #80440 - EXT:lowlevel ArrayBrowser->wrapValue

See Issue #80440

Description

The method `ArrayBrowser->wrapValue` in EXT:lowlevel has been marked as deprecated, since the sole logic was to wrap the incoming string into `htmlspecialchars()`.

Impact

Calling the method will trigger a deprecation warning.

Affected Installations

Any TYPO3 installation using the EXT:lowlevel ArrayBrowser class in a custom extension.

Migration

Remove the call to the method and directly use `htmlspecialchars()` instead.

Deprecation: #80444 - TypoScriptFrontendController-> beLoginLinkIPList

See Issue #80444

Description

The method `TypoScriptFrontendController->beLoginLinkIPList` has been marked as deprecated.

Impact

Calling the PHP method directly will trigger a deprecation warning.

Affected Installations

Any installation instantiating a custom frontend-related RequestHandler or using the method above when rendering the frontend. Also, any custom extension using this method.

Migration

The functionality is moved to EXT:compatibility7.

Deprecation: #80445 - Deprecate printContent methods

See Issue #80445

Description

The methods `TaskModuleController::printContent()` and `PageLayoutController::printContent()` have been marked as deprecated.

Impact

Calling the deprecated methods will trigger a deprecation log entry.

Deprecation: #80449 - GeneralUtility::freetypeDpiComp

See Issue #80449

Description

The method `GeneralUtility::freetypeDpiComp` has been marked as deprecated.

Impact

Calling this method will trigger a deprecation log entry.

Affected Installations

Any installation using custom GraphicalFunctions where GDlib/Freetype does custom calculations.

Migration

No substitution available.

Deprecation: #80451 - Deprecate GeneralUtility::csvValues

See Issue #80451

Description

The method `GeneralUtility::csvValues()` has been marked as deprecated.

Impact

Calling the deprecated methods will trigger a deprecation log entry.

Migration

Use the new method `CsvUtility::csvValues()`

Deprecation: #80468 - Command Line Interface: cliKeys and cli_dispatch.phpsh

See Issue #80468

Description

The functionality to register any command line script via `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['GLOBAL']` has been marked as deprecated.

The entry-point ‘`typo3/cli_dispatch.phpsh`` as well as the corresponding `Application` class and the `CliRequestHandler` class have been marked as deprecated as well.

The functionality has been superseded by Symfony Console and the new entry-point within `typo3/sysext/core/bin/typo3` which is able to handle all functionality the same way including all Extbase-related Command Controllers.

Impact

Calling the CLI entrypoint `typo3/cli_dispatch.phpsh` to call a CLI script will trigger a deprecation warning.

Affected Installations

Any installation using `typo3/cli_dispatch.phpsh` in any deployment or cronjob / scheduler functionality.

Migration

All functionality related to Extbase, EXT:lowlevel, or scheduler tasks can be called via the new entrypoint `typo3/sysext/core/bin/typo3` with a similar call.

Update all cronjobs and automated and manual running scripts called via the command line to use the new entrypoint.

If there any custom cliKeys registered, migrate them to a Symfony Command or an Extbase Command Controller.

Deprecation: #80485 - Method parameter of TSFE->whichWorkspace to return the workspace title

See Issue #80485

Description

The method `TypoScriptFrontendController->whichWorkspace()` has an optional first parameter to return the workspace title of the current workspace instead of the current workspace UID.

This parameter has been marked as deprecated.

Impact

When calling the method above with the method parameter set to "true", a deprecation message is triggered.

Affected Installations

Any installation using this PHP method with the parameter set to "true" via a custom extension which deals with workspaces for frontend output (e.g. for editors to know in which workspace a user is currently previewing a page).

Migration

If the workspace title is necessary, a separate SQL call should be done right after `whichWorkspace()` is called in the extensions' PHP code.

Deprecation: #80486 - Setting charset via LocalizationParserInterface->getParsedData()

See Issue #80486

Description

The `LocalizationParserInterface->getParsedData()` contains a third parameter to hand over a value for the charset used.

This third parameter has been marked as deprecated, as it is not in use anymore.

Affected Installations

Any installation with an extension that extends the LocalizationParser functionality with a custom PHP class implementing the `LocalizationParserInterface`.

Migration

If implementing the `LocalizationParserInterface`, be aware that this third parameter will be dropped in TYPO3 v9.

Deprecation: #80491 - BackendController inclusion hooks

See Issue #80491

Description

The hook within BackendController `$TYPO3_CONF_VARS["typo3/backend.php"]["additionalBackendItems"]` has been marked as deprecated.

Loading ExtJS module JS/CSS files via `ExtensionManagementUtility::addExtJSModule()` inside the module configuration has been deprecated.

Calling `BackendController->addJavascriptFile()`, `BackendController->addJavascript()` and `BackendController->addCssFile()` will trigger a deprecation log entry.

Impact

Registering a hook via `$TYPO3_CONF_VARS["typo3/backend.php"]["additionalBackendItems"]` and then calling the Backend main page will trigger a deprecation log warning.

Registering any backend module which should load a global CSS/JS file within a module configuration will trigger a deprecation log warning.

Calling any of the methods above will trigger a deprecation log warning.

Affected Installations

Any installation using the hook or PHP methods directly in a custom extension, or using any of the public methods above in a custom PHP script.

Migration

Use the "constructPostProcess" hook within BackendController to load additional resources to achieve the same functionality.

Deprecation: #80510 - ContentObjectRenderer->URLqMark

See Issue #80510

Description

The PHP method `ContentObjectRenderer->URLqMark()` has been marked as deprecated. It was used to add a `?` between two strings if the first one does contain a `?` already.

Its main purpose is to add query string parameters to a given URL.

Impact

Calling the method above will trigger a deprecation warning.

Affected Installations

Any installation using custom extensions calling this method.

Migration

Implement this functionality with PHP's native `(strpos($haystack,'?') !== false ? '?' : '')` one-liner directly.

Deprecation: #80511 - AbstractFunctionModule->incLocalLang and $thisPath

See Issue #80511

Description

The method `AbstractFunctionModule->incLocalLang()` and the public property `AbstractFunctionModule->thisPath` have been marked as deprecated.

Impact

Calling method above will trigger a deprecation log entry.

Affected Installations

Any extension extending the AbstractFunctionModule and calling the mentioned method.

Migration

The functionality of loading a locallang file is now taken care of by `LanguageService::includeLLFile()` and takes care of everything automatically.

If any specific calls to the method is made, it should be replaced by the LanguageService equivalent.

The property `$thisPath` contains the path to the class, which can be accessed via Reflection, as it is done currently as well, should be implemented in the custom extension itself that needs this information.

Deprecation: #80512 - DocumentTemplate->extJScode property

See Issue #80512

Description

The property `DocumentTemplate->extJScode` to load ExtJS-specific code "onExtJSReady" has been marked as deprecated.

Impact

If the property is filled and added to the response output, a deprecation warning will be triggered.

Affected Installations

Any installation with custom extensions using (or mis-using) this property to inject ExtJS-specific code.

Migration

Use the PageRenderer object directly to inject `addExtOnReadyCode` in a backend response.

Deprecation: #80513 - DataHandler: Various methods and method arguments

See Issue #80513

Description

The method `DataHandler->destPathFromUploadFolder()` has been marked as deprecated.

The fourth parameter `$func` of the method `DataHandler->extFileFunctions()` has been deprecated.

Impact

Calling the method `DataHandler->destPathFromUploadFolder()` will trigger a deprecation log entry.

Calling the method `DataHandler->extFileFunctions()` with the fourth parameter (usually set to 'deleteAll') will trigger a deprecation log entry.

Affected Installations

Any installation with custom extension logic using the DataHandler and specifically these methods.

Migration

Replace the function call `DataHandler->destPathFromUploadFolder()` by prepend the constant `PATH_site` before the string to be handed over to the deprecated method.

Remove the fourth parameter of the callee of `DataHandler->extFileFunctions()`.

Deprecation: #80514 - GraphicalFunctions->tempPath and createTempSubDir()

See Issue #80514

Description

The method `GraphicalFunctions->createTempSubDir()` and the property `GraphicalFunctions->tempPath` have been marked as deprecated.

Impact

Calling the method above will trigger a deprecation log entry.

Affected Installations

Any instance with custom extensions extending the PHP class GraphicalFunctions.

Migration

Use `GeneralUtility::mkdir_deep()` with the full path (including the PHP constant `PATH_site`) directly.

Deprecation: #80516 - TypoScript config.setJS_mouseOver and config.setJS_openPic

See Issue #80516

Description

The TypoScript properties `config.setJS_mouseOver` and `config.setJS_openPic` have been marked as deprecated.

Impact

Setting any of the TypoScript properties will trigger a deprecation log entry.

Affected Installations

Any installation using these TypoScript options.

Migration

Include the small JavaScript files directly in your custom JavaScript file or inline via `page.inlineJS`.

Deprecation: #80524 - PageRepository::getHash and PageRepository::storeHash

See Issue #80524

Description

The two static methods `PageRepository::getHash()` and `PageRepository::storeHash()`, that act as simple wrappers for the Caching Frameworks's "cache_hash" frontend, have been deprecated.

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any installation with a custom installation using any of the methods.

Migration

Use the Caching Framework directly. Simply spoken, the code that still exists in the functions, can simply be copied into the third-party extensions' code.

Deprecation: #80527 - Marker-related methods in ContentObjectRenderer

See Issue #80527

Description

The following methods within `ContentObjectRenderer` PHP class have been marked as deprecated:

- `getSubpart()`
- `substituteSubpart()`
- `substituteSubpartArray()`
- `substituteMarker()`
- `substituteMarkerArrayCached()`
- `substituteMarkerArray()`
- `substituteMarkerInObject()`
- `substituteMarkerAndSubpartArrayRecursive()`
- `fillInMarkerArray()`

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any installation using custom extensions calling these API methods.

Migration

Instantiate the class `MarkerBasedTemplateService` available in TYPO3 v7, which contains equivalents to all methods that have been marked as deprecated with the same functionality and namings.

Deprecation: #80532 - GifBuilder-related methods in ContentObjectRenderer

See Issue #80532

Description

The following methods related to `GifBuilder` within `ContentObjectRenderer` have been marked as deprecated.

- `clearTSProperties()`
- `gifBuilderTextBox()`
- `linebreaks()`

Impact

Calling any of the methods above will trigger a deprecation message.

Affected Installations

Any installation using these methods in custom extensions.

Deprecation: #80579 - Modal.center has been marked as deprecated

See Issue #80579

Description

The method `Modal.center` has been marked as deprecated. Alignment is now handled via CSS and this method is now obsolete.

Impact

Calling `Modal.center` will trigger a console warning in the browser.

Affected Installations

All 3rd party extensions using `Modal.center` are affected.

Migration

Remove obsolete calls to `Modal.center()`.

Deprecation: #80583 - TYPO3_CONF_VARS_extensionAdded

See Issue #80583

Description

The global array `$GLOBALS['TYPO3_CONF_VARS_extensionAdded']` has been deprecated along with the method `ExtensionManagementUtility::appendToTypoConfVars()`

Impact

Using method `appendToTypoConfVars()` throws a deprecation warning and accessing `$GLOBALS['TYPO3_CONF_VARS_extensionAdded']` will stop working with core version 9.

Affected Installations

Extensions using `$GLOBALS['TYPO3_CONF_VARS_extensionAdded']` or method `appendToTypoConfVars()`

Migration

Access `$GLOBALS['TYPO3_CONF_VARS']` directly.

Deprecation: #80601 - Change duplicate icon identifiers to actions-close

See Issue #80601

Description

The icon `actions-document-close` has been marked as deprecated.

Impact

Using the icon identifier `actions-document-close` will trigger a deprecation log entry.

Affected Installations

Any installation using the icon identifier.

Migration

Use the new icon identifier `actions-close` instead.

Deprecation: #80603 - Change duplicate icon identifiers to actions-add

See Issue #80603

Description

The icon `actions-edit-add` has been marked as deprecated.

Impact

Using the icon identifier `actions-edit-add` will trigger a deprecation log entry.

Affected Installations

Any installation using the icon identifier.

Migration

Use the new icon identifier `actions-add` instead.

Deprecation: #80614 - TCA itemListStyle and selectedListStyle

See Issue #80614

Description

The TCA property `itemListStyle` available in renderType `selectSingleBox` and `selectMultipleSideBySide`, as well as the property `selectedListStyle` available in renderType `selectMultipleSideBySide` have been deprecated.

Impact

Using these properties throws a deprecation warning and will not be considered with TYPO3 v9.

Affected Installations

Instances using one of the above properties in TCA

Migration

The properties can be dropped. Changing styles of above elements should be done in own renderTypes or by overloading CSS in the backend.

2.2.4 Important

Important: #71095 - Add language debug mode to All Configuration

See Issue #71095

Description

Previously it was possible to set `$TYPO3_CONF_VARS['BE']['lang']['debug']` in order to enable debug in LanguageService.

However this could not be configured in the install tool. In order to enable this possibility it has been renamed to `$TYPO3_CONF_VARS['BE']['languageDebug']`

Important: #78650 - TypoScriptService class moved from Extbase to Core

See Issue #78650

Description

The PHP class `TypoScriptService` has been moved to the core extension, as it has no direct link to Extbase, and a lot of other system extensions are using the class.

The old class name is still registered as a class alias, so extensions can call the class via the Extbase PHP namespace in TYPO3 v8 without any downsides.

Important: #79847 - Fluid bugs fixed and features added (Fluid 2.3.1)

See Issue #79847

Description

The Fluid engine dependency is raised to version 2.3.1 which fixes a few important bugs and adds a couple of features:

- Namespace declarations (`{namespace foo=Bar\Baz\ViewHelpers}` style) are now removed from output https://github.com/TYPO3/Fluid/pull/262
- The TemplatePaths object now accepts arrays for `sanitizePath` like the TYPO3 CMS adapter does. https://github.com/TYPO3/Fluid/pull/263
- Compiler is reset after each rendering - this fixes an issue where rendering the new `HeaderAssets` and `FooterAssets` sections would fail to attach the assets until the Fluid template had been compiled (first page hit after cache flush). https://github.com/TYPO3/Fluid/pull/269

And in the new features department, two new features are added:

- XML namespace extraction is brought into sync with TYPO3 CMS adapter https://github.com/TYPO3/Fluid/pull/264
- An escaping modifier pre-processor has been added https://github.com/TYPO3/Fluid/pull/266

This means two things:

1. For template developers this means you can use `{escaping off}` in a template to completely disable the escaping which is normally done - which can be particularly helpful in non-HTML templates.
2. For the TYPO3 core this means it becomes possible to drop two classes (Fluid overrides) completely from the source; namely the `XmlnsNamespaceTemplatePreProcessor` and `LegacyNamespaceExpressionNode`. Thus increasing the parsing efficiency of Fluid as it is integrated with TYPO3 CMS.

Important: #79942 - Version selector view moved to compatibility7

See Issue #79942

Description

The custom Backend module for showing and selecting versions of a specific page, which was part of EXT:version has been moved to EXT:compatibility7. The backend module is solely available via the ClickMenu as it is formally superseded by the Workspaces Backend module.

All related PHP classes and template files have been moved as well.

Important: #80236 - EXT:form Configuration for form VH attributes

See Issue #80236

Description

Move some fixed configurations from within the fluid templates into the form settings

- f:form -> additionalParams
- f:form -> addQueryString
- f:form -> argumentsToBeExcludedFromQueryString
- f:form -> action
- f:form -> enctype
- f:form -> method

within the EXT:Form configuration.

```
TYPO3:
    CMS:
        Form:
            prototypes:
                <prototypeName>:
                    formElementsDefinition:
                        Form:
                            renderingOptions:
                                addQueryString: false
                                argumentsToBeExcludedFromQueryString: []
                                additionalParams: []
                                controllerAction: perform
                                httpMethod: post
                                httpEnctype: 'multipart/form-data'
```

Impact

An integrator can configure the FLUID form properties

f:form -> additionalParams f:form -> addQueryString f:form -> argumentsToBeExcludedFromQueryString f:form -> action f:form -> enctype f:form -> method

Important: #80241 - EXT:form simplify translation handling

See Issue #80241

Description

If an integrator wants to add translations for new form elements he can only define a new translation file which must contain all translation keys. This patch makes it possible to define multiple translation files.

Before this patch:

```
TYPO3:
  CMS:
    Form:
      prototypes:
        standard:
          formElementsDefinition:
            Form:
              renderingOptions:
                translation:
                  translationFile: 'EXT:form/Resources/Private/Language/locallang.
↪xlf'
```

After this patch:

```
TYPO3:
  CMS:
    Form:
      prototypes:
        standard:
          formElementsDefinition:
            Form:
              renderingOptions:
                translation:
                  translationFile:
                    10: 'EXT:form/Resources/Private/Language/locallang.xlf'
                    20: 'EXT:my_ext/Resources/Private/Language/locallang.xlf'
```

The translation keys will be searched within the referenced files. The search order is from the key with the highest number to the lowest. If a translation key is found within one of these files the search will stop.

This makes it possible to only define new keys within the custom translations and use the default form translations as well. The default settings keep the translationFile property as string because of backward compatibility.

Before this patch the "BaseFormElementMixin" inherits the "translationSettingsMixin". Thus, the "renderingOptions.translation..." are copied to each form element. This is inconvenient if an integrator defines his own prototype which inherits from the standard prototype because he must redefine the "renderingOptions.translation..." options for each form element.

Since there already is a fallback strategy to the "renderingOptions.translation..." options from the root form element - if this option is not set within the child form elements - we can simply apply the "translationSettingsMixin" to the "Form" element and remove it from the "BaseFormElementMixin". Now, the rendering options are only set for the "Form" element and rules as a prototype wide frontend translation setting.

This patch adds a fallback for the form engine translation if there is no "translationFile" setting within the "FormEngine" option.

```
TYPO3:
  CMS:
    Form:
      prototypes:
        standard:
          formEngine:
            translationFile:
              10: 'EXT:form/Resources/Private/Language/Database.xlf'
              20: 'EXT:ext_form_example1484232130/Resources/Private/Language/
↪Database.xlf'
```

Now, there is one prototype wide form engine (plugin settings) translation setting.

Summary

With this patch, an integrator has prototype wide translation settings for the 4 aspects of the form framework. Furthermore, the integrator is able to define multiple translation files to avoid copying the whole default translation files or using locallangXMLOverride.

```
TYPO3:
  CMS:
    Form:
      formManager:
        selectablePrototypesConfiguration:
          1484232130:
            translationFile:
              # translations for the form managers "new form" modal
              10: 'EXT:form/Resources/Private/Language/Database.xlf'
              20: 'EXT:my_ext/Resources/Private/Language/Database.xlf'

      prototypes:
        <prototypeName>:
          formEditor:
            translationFile:
              # translations for the form editor
              10: 'EXT:form/Resources/Private/Language/Database.xlf'
              20: 'EXT:my_ext/Resources/Private/Language/Database.xlf'

          formEngine:
            translationFile:
              # translations for the form plugin (finisher overrides)
              10: 'EXT:form/Resources/Private/Language/Database.xlf'
              20: 'EXT:my_ext/Resources/Private/Language/Database.xlf'

          formElementsDefinition:
            Form:
              renderingOptions:
                translation:
                  translationFile:
                    # translations for the frontend
                    10: 'EXT:form/Resources/Private/Language/locallang.xlf'
                    20: 'EXT:my_ext/Resources/Private/Language/locallang.xlf'
```

Impact

Easier to use, less maintenance.

Important: #80266 - Moved config.sys_language_softExclude to compatibility7

See Issue #80266

Description

The TypoScript option `config.sys_language_softExclude` to set certain TCA table fields to `l10n_mode=exclude` during frontend request runtime has been moved to compatibility7.

If any installation depends on this option in the TYPO3 frontend, the extension should be installed.

However, as the TCA option `l10n_mode=exclude` has been superseded by the TCA option `allowLanguageSynchronization` the actual use-case for this TypoScript setting should be re-evaluated.

Important: #80301 - EXT:form - Cleanup / callback migration

See Issue #80301

Description

The callback 'onBuildingFinished' is deprecated and will be removed in TYPO3 v9.

Use the new hook 'afterBuildingFinished' instead.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['afterBuildingFinished'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface $renderable
 * @return void
 */
public function␣
↪afterBuildingFinished(\TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface␣
↪$renderable)
{
}
```

This hook will be called for each renderable.

The callback 'beforeRendering' is deprecated and will be removed in TYPO3 v9.

Use the new hook 'beforeRendering' instead.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['beforeRendering'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the signal:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Runtime\FormRuntime $formRuntime
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\RootRenderableInterface
↪$renderable
 * @return void
 */
public function beforeRendering(\TYPO3\CMS\Form\Domain\Runtime\FormRuntime
↪$formRuntime, \TYPO3\CMS\Form\Domain\Model\Renderable\RootRenderableInterface
↪$renderable)
{
}
```

This hook will be called for each renderable.

The callback 'onSubmit' is deprecated and will be removed in TYPO3 v9.

Use the new hook 'afterSubmit' instead.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['afterSubmit'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Runtime\FormRuntime $formRuntime
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface $renderable
 * @param mixed $elementValue submitted value of the element *before post␣
↪processing*
 * @param array $requestArguments submitted raw request values
 * @return void
 */
public function onSubmit(\TYPO3\CMS\Form\Domain\Runtime\FormRuntime $formRuntime,␣
↪\TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface $renderable,
↪$elementValue, array $requestArguments = [])
{
    return $elementValue;
}
```

This hook will be called for each renderable.

The callback 'initializeFormElement' call the 'initializeFormElement' hook.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['initializeFormElement'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface $renderable
 * @return void
 */
public function␣
↪initializeFormElement(\TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface␣
↪$renderable)
{
}
```

This enables you to override the 'initializeFormElement' method within your custom implementation class. If you do not call the parents 'initializeFormElement' then no hook will be thrown. Furthermore, you can connect to the hook and initialize the generic form elements without defining a custom implementation to access the 'initializeFormElement' method. You only need a class which connects to this hook. Then detect the form element you wish to initialize. This saves you a lot of configuration!

The hook 'beforeRemoveFromParentRenderable' will be called for each renderable.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form'][
↪'beforeRemoveFromParentRenderable'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface $renderable
 * @return void
 */
public function␣
↪beforeRemoveFromParentRenderable(\TYPO3\CMS\Form\Domain\Model\Renderable\RenderableInterface␣
↪$renderable)
{
}
```

The hook 'afterInitializeCurrentPage' will be called after a page is initialized.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['afterInitializeCurrentPage
↪']['']
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param \TYPO3\CMS\Form\Domain\Runtime\FormRuntime $formRuntime
 * @param \TYPO3\CMS\Form\Domain\Model\Renderable\CompositeRenderableInterface
↪$currentPage
 * @param␣
↪null|\TYPO3\CMS\Form\Domain\Model\Renderable\CompositeRenderableInterface
↪$lastPage
 * @param mixed $elementValue submitted value of the element *before post␣
↪processing*
 * @return \TYPO3\CMS\Form\Domain\Model\Renderable\CompositeRenderableInterface
 */
public function␣
↪afterInitializeCurrentPage(\TYPO3\CMS\Form\Domain\Runtime\FormRuntime
↪$formRuntime,␣
↪\TYPO3\CMS\Form\Domain\Model\Renderable\CompositeRenderableInterface $currentPage,
↪ \TYPO3\CMS\Form\Domain\Model\Renderable\CompositeRenderableInterface $lastPage␣
↪= null, array $requestArguments = []): CompositeRenderableInterface
{
    return $currentPage;
}
```

The form manager call the 'beforeFormCreate' hook.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['beforeFormCreate']['']
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param string $formPersistenceIdentifier
 * @param array $formDefinition
 * @return array
 */
public function beforeFormCreate(string $formPersistenceIdentifier, array
↪$formDefinition): array
{
```

```
    return $formDefinition;
}
```

The form manager call the 'beforeFormDuplicate' hook.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['beforeFormDuplicate'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param string $formPersistenceIdentifier
 * @param array $formDefinition
 * @return array
 */
public function beforeFormDuplicate(string $formPersistenceIdentifier, array
 ↪$formDefinition): array
{
    return $formDefinition;
}
```

The form manager call the 'beforeFormDelete' hook.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['beforeFormDelete'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the signal:

```
/**
 * @param string $formPersistenceIdentifier
 * @return void
 */
public function beforeFormDelete(string $formPersistenceIdentifier)
{
}
```

The form editor call the 'beforeFormSave' hook.

Connect to the hook:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/form']['beforeFormSave'][]
    = \VENDOR\YourNamespace\YourClass::class;
```

Use the hook:

```
/**
 * @param string $formPersistenceIdentifier
 * @param array $formDefinition
 * @return array
 */
public function beforeFormSave(string $formPersistenceIdentifier, array
 ↪$formDefinition): array
{
```

```
    return $formDefinition;
}
```

New form element property: properties.fluidAdditionalAttributes

In order to deal with fluid ViewHelpers 'additionalAttributes' it is necessary to introduce a new configuration scope "properties.fluidAdditionalAttributes" for each form element. This configuration property will be used to fill the fluid ViewHelper property "additionalAttributes".

Important: #80391 - Css Styled Content will not reset TypoScript Constants

See Issue #80391

Description

Previously the TypoScript definition from CSS Styled Content reset all constants that were set before the static template was included to preserve the namespace `styles.content`.

Since there is no need to reset the constants, this behaviour is removed.

Removed Code

```
# Clear out any constants in this reserved room!
styles.content >
```

Important: #80444 - config.beLoginLinkIPList moved to compatibility7

See Issue #80444

Description

The TypoScript option `config.beLoginLinkIPList` which renders a link to login/logout to the TYPO3 Backend when visiting a frontend page from a certain IP address, has been moved to EXT:compatibility7.

Important: #80450 - MonitorUtilityMovedToCompatibility

See Issue #80450

Description

The "peak memory measurement" in the frontend has been moved to extension compatiblity7. The functionality is semi useful and should live a happy life in an extension for people who may need it, but there is no need to have that within the core on each frontend call.

Important: #80506 - Dbal compatible field quoting in TypoScript

See Issue #80506

Description

Properties in `TypoScript` dealing with SQL fragments need proper quoting of field names to be compatible with different database drivers. The database framework of the core now applies proper quoting to field names if they are wrapped as `{#fieldName}`

It is advised to adapt extensions accordingly to run successfully on databases like PostgreSQL.

Example for a `select.where` TypoScript snippet:

```
select.where = {#colPos}=0
```

Important: #80553 - Simplify important actions in Install Tool

See Issue #80553

Description

To simplify the Install Tool the following settings have been removed from the section "Important actions":

- Change site name
- Change encryption key

The configuration can be still changed in the section "All configuration"

Important: #80606 - Testing Framework Removal / Use composer package instead

See Issue #80606

Description

The TYPO3 core base testing classes were moved to a separate composer package and removed from the TYPO3 core. If you want to continue using the testing framework classes please use composer and require the package typo3/testing-framework.

To make sure you don't run into path issues, you can set the following environment variable: TYPO3_PATH_ROOT = The path to your TYPO3 root directory

Find the new package on github

And on packagist

2.3 8.6 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.3.1 Breaking Changes

Breaking: #70316 - AbstractUserAuthentication properties and methods dropped and changed

See Issue #70316

Description

The property `AbstractUserAuthentication::session_table` has been dropped. The property `FrontendUserAuthentication::sessionDataTimestamp` has been dropped. The property `FrontendUserAuthentication::sesData` has been moved to `AbstractUserAuthentication::sessionData` and is protected now.

The method `FrontendUserAuthentication::fetchSessionData()` has been removed and its logic has been integrated into `AbstractUserAuthentication::fetchUserSession()`.

Impact

Accessing one of these properties will raise a PHP warning. Calling the method `fetchSessionData()` will cause a PHP fatal error.

Moreover it is not possible anymore to use the getData function from within TypoScript to access session data. This functionality is replaced by a new API. (see Issue #80154)

Affected Installations

All extensions accessing these properties will most likely not work properly anymore. Extensions accessing the removed method will not work at all.

Migration

Use configuration from `DatabaseSessionBackend` located in `$GLOBALS['TYPO3_CONF_VARS']['SYS']['session' Session Identifier */]['table']` or use `AbstractUserAuthentication::loginType` to distinguish between FE or BE login types.

Session data can be manipulated with the following methods in `AbstractUserAuthentication`

- `getSessionData()`
- `setSessionData()`

Calls to `FrontendUserAuthentication::fetchSessionData()` can safely be removed.

Breaking: #77934 - Remove field `select_key` from content element preview

See Issue #77934

Description

The value of the field `select_key` has been shown in the preview of a content element in the page module. This field has been removed and therefore also the preview has been removed.

Impact

The preview of this field is not available anymore.

Affected Installations

Every installation or third party extension which uses the field.

Migration

No migration available.

Breaking: #78192 - Refactor click menu (context menu)

See Issue #78192

Description

Due to the refactoring and unification of the click/context-menu handling in the TYPO3 Backend, a few breaking changes have been introduced.

Classes removed

- `\TYPO3\CMS\Backend\ClickMenu\ClickMenu`
- `\TYPO3\CMS\Backend\ContextMenu\ContextMenuAction`
- `\TYPO3\CMS\Backend\ContextMenu\ContextMenuActionCollection`
- `\TYPO3\CMS\Backend\ContextMenu\Pagetree\ContextMenuDataProvider`
- `\TYPO3\CMS\Backend\ContextMenu\Pagetree\Extdirect\ContextMenuConfiguration`
- `\TYPO3\CMS\Backend\Controller\ClickMenuController`
- `\TYPO3\CMS\Impexp\Clickmenu` (replaced by new hook implementation: `TYPO3\CMS\Impexp\Hook\ContextMenuModifyItemsHook`)
- `\TYPO3\CMS\Impexp\Hook\ContextMenuDisableItemsHook`
- `\TYPO3\CMS\Version\ClickMenu\VersionClickMenu`

ExtJS component removed

- The `TYPO3.Components.PageTree.ContextMenu` component defined in contextmenu.js has been removed.
- The `contextMenuProvider` property as well as `enableContextMenu` and `openContextMenu` methods of the `TYPO3.Components.PageTree.Tree` component have been removed.

Migration

Migrate your code to a requireJS module for custom click-menu actions.

ClickMenu requireJS component removed

The `TYPO3/CMS/Backend/ClickMenu` requireJS component (ClickMenu.js) has been removed.

Migration

Use the new requireJS component: `TYPO3/CMS/Backend/ContextMenu`.

Page TSConfig change

The pagetree context-menu configuration in Page TSConfig has been removed (except for the `disableItems` part). The list of available menu items is now provided by `ItemProviders` e.g. `\TYPO3\CMS\Backend\ContextMenu\ItemProviders\PageProvider`.

The TSConfig options for disabling click-menu items has been streamlined. Also some items names have changed (e.g. `new_wizard` is now called `newWizard`, `db_list` is now `openListModule`). Refer to the provider class for correct naming.

Migration

Migrate TSConfig from:

`options.contextMenu.folderList.disableItems` to `options.contextMenu.sys_file.disableItems`
`options.contextMenu.folderTree.disableItems` to `options.contextMenu.sys_file.tree.disableIt`
`options.contextMenu.pageList.disableItems` to `options.contextMenu.pages.disableItems`
`options.contextMenu.pageTree.disableItems` to `options.contextMenu.pages.tree.disableItems`

Hooks removed

The following two hooks have been removed:

- `$GLOBALS['TBE_MODULES_EXT']['xMOD_alt_clickmenu']['extendCMclasses']`
- `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['backend']['contextMenu']['disableItems']`

Migration

Use the new ItemsProvider API to add or modify click-menu items. See existing usage of the API in the core `TYPO3\CMS\Filelist\ContextMenu\ItemProviders\FileProvider` or `\TYPO3\CMS\Beuser\ContextMenu\ItemProvider`.

Legacy Tree

Support for drag & drop menu for LegacyTree.js of pages has been dropped.

Changed markup (data attributes) for click menu

- `data-listFrame` has been replaced with the optional attribute `data-context` attribute. Context is set to "tree" for click-menus triggered from trees.
- **for files, `data-table` now contains the real table name "sys_file" while before it contained the combined identifier e.**
  the `data-uid` attribute now contains the combined identifier of the file (before it was empty). Thus the `data-uid` attribute value is not always an int.
- the class which triggers the context-menu has changed from `t3-js-clickmenutrigger` to `t3js-contextmenutrigger`

Migration

To trigger click-menus for files, use the correct class-attribute as well as the table and uid data attributes. Replace `data-listFrame="0"` with `data-context="tree"`, `data-listFrame="1"` can be removed (it's a default context now).

Impact

Instantiating the PHP class will result in a fatal PHP error. Accessing removed JavaScript properties will result in a JavaScript error.

Removed hooks will not influence the menu rendering process.

Affected Installations

Any installation using the removed PHP classes, JS components or hooks.

Migration

Adapt your code to the new click menu API.

Breaking: #78477 - FlashMessageViewHelper no longer inherits from TagBasedViewHelper

See Issue #78477

Description

The `FlashMessageViewHelper` has been refactored and no longer inherits from the `TagBasedViewHelper`.

Impact

The `FlashMessageViewHelper` outputs default context specific markup. Adding own classes or tag attributes is no longer possible.

Affected Installations

All installations using the `FlashMessageViewHelper` with tag specific attributes.

Migration

Remove the tag specific attributes and style the default output. If you need custom output use the possibility to render FlashMessages yourself, for example:

```
<f:flashMessages as="flashMessages">
    <dl class="messages">
        <f:for each="{flashMessages}" as="flashMessage">
           <dt>CODE!! {flashMessage.code}</dt>
           <dd>MESSAGE:: {flashMessage.message}</dd>
        </f:for>
    </dl>
</f:flashMessages>
```

Breaking: #78477 - Remove method FlashMessage->getMessageAsMarkup()

See Issue #78477

Description

The method `FlashMessage->getMessageAsMarkup()` has been removed.

Impact

Using this method will stop working immediately.

Affected Installations

All installations using the mentioned method above.

Migration

Use the new `FlashMessageRendererResolver::class`, for example:

```
GeneralUtility::makeInstance(FlashMessageRendererResolver::class)->resolve()->
↪render()
```

Breaking: #78899 - Remove `extJSCODE` from FormEngine result array

See Issue #78899

Description

The key `extJSCODE` in the array returned by FormEngine's `Container` and `Element` (initialized in `AbstractNode->initializeResultArray()`) has been removed.

Impact

JavaScript code added to `extJSCODE` by custom elements will not be evaluated anymore.

Affected Installations

Search extensions for the string `extJSCODE`. This array is used rather seldom, but if there are matches in combination with Backend Form classes, they should be adapted.

Migration

For a simple solution, add according JavaScript to the return key `additionalJavaScriptPost` for now. Both keys were used nearly identically anyway. Be aware that both keys `additionalJavaScriptPost` and `additionalJavaScriptSubmit` are target of a later removal as soon as a better JavaScript side event handling for those scenarios is in place. See if the current code injected at this point could be done with casual `RequireJsModules` instead already.

Breaking: #78899 - Remove methods, hook and property in FormEngine

See Issue #78899

Description

The following methods have been removed:

- `TYPO3\CMS\Backend\Form\Element\AbstractFormElement->dbFileIcons()`
- `TYPO3\CMS\Backend\Form\Element\AbstractFormElement->getClipboardElements()`
- `TYPO3\CMS\Backend\Form\Container\SingleFieldContainer->getMergeBehaviourIcon()`
- `TYPO3\CMS\Backend\Form\Container\SingleFieldContainer->renderDefaultLanguageDiff()`
- `TYPO3\CMS\Backend\Form\Container\SingleFieldContainer->renderDefaultLanguageContent`
- `TYPO3\CMS\Backend\Form\Container\AbstractContainer->previewFieldValue()`

The following property has been removed:

- `TYPO3\CMS\Backend\Form\Element\AbstractFormElement->clipboard`

The following hook interface has been removed and registered hooks in `dbFileIcons` are no longer called:

- `TYPO3\CMS\Backend\Form\DatabaseFileIconsHookInterface`

TCA wizards registered as `userFunc` no longer receive the element HTML by reference, so they can no longer change given HTML string of a given element.

Impact

Using above methods, properties and hooks will result in fatal PHP errors or fail silently.

Affected Installations

Check extensions for usages of above methods and especially implementations of the hook interface.

Migration

The methods have been partially moved to the `TcaGroup` data provider and merged to the two FormEngine elements `GroupElement` and `SelectMultipleSideBySideElement`. Those can be changed and extended via FormEngine's internal `NodeFactory` and data provider resolvers.

Breaking: #78988 - Remove optional Fluid TypoScript template

See Issue #78988

Description

The static include file "Fluid: (Optional) default ajax configuration (fluid)" was meant as an example/showcase on how to use Fluid Widgets in FE. But the currently used includes are outdated or broken. Furthermore the way of including files with `page.headerData` instead of `page.includeJSLibs` or `page.includeCSSLibs` is not the preferred way anymore. Also in many situations this way of including JavaScript and CSS conflicts with other included JavaScript libs and CSS files.

Including the files manually has many benefits:

- more control of what versions of the JavaScript libs are included
- prevent multiple jquery.js includes
- more control of adjusting styling without resetting/overriding styles delivered by jquery-ui-theme.css

Impact

The jQuery JavaScript and CSS files are not included anymore so the AJAX handling in the frontend will not work anymore when the site relies on these files.

Affected Installations

All installations that depend on the jQuery includes added by the static TypoScript template "Fluid: (Optional) default ajax configuration (fluid)".

Migration

Include the needed file manually in your TypoScript template.

```
page.includeJSLibs {
    jquery = https://code.jquery.com/jquery-3.1.1.slim.min.js
    jquery.external = 1
    jquery.integrity = sha256-/SIrNqv8h6QGKDuNoLGA4iret+kyesCkHGzVUUV0shc=
    jqueryUi = https://code.jquery.com/ui/1.12.1/jquery-ui.min.js
    jqueryUi.external = 1
    jqueryUi.integrity = sha256-VazP97ZCwtekAsvgPBSUwPFKdrwD3unUfSGVYrahUqU=
}

page.includeCSSLibs {
    jqueryUI = https://code.jquery.com/ui/1.12.1/themes/smoothness/jquery-ui.css
    jqueryUi.external = 1
}
```

Breaking: #79025 - Extract testing framework for TYPO3

See Issue #79025

Description

Since the .gitattributes export change, a lot of base test classes for writing own tests are missing in distribution builds. To get a sustainable future-proof solution, the TYPO3 core testing framework will be extracted to an own component.

Impact

All test classes that are considered as part of the TYPO3 core testing framework are moved to components/testing_framework and will in the long run be released as an own package that can be required for dev environments.

Moving the classes results in changed namespaces. To ensure compatibility with earlier TYPO3 versions all classes that were previously available in distribution (non-source) installations are additionally provided by their old namespace names and will be provided for 8 LTS.

Affected Installations

All installations using core testing components as base.

Migration

Change the namespace from TYPO3CMSCoreTests to TYPO3TestingFrameworkCore or in case of the xml fixtures the corresponding file path.

If you need to ensure compatibility with multiple TYPO3 versions, use the base test classes with their old names.

Breaking: #79100 - ext:felogin: Remove default CSS

See Issue #79100

Description

The applied default CSS styles delivered by EXT:felogin may break the frontend output, for example if CSS frameworks are used. The default styles need to get overridden in such case.

Impact

EXT:felogin doesn't add default CSS styles anymore.

Affected Installations

All installations using EXT:felogin with default/non-overwritten `plugin.tx_felogin_pi1._CSS_DEFAULT_STYLE` TypoScript setup are affected.

Migration

If your frontend relies on the default EXT:felogin CSS styles, make sure to add following CSS on your own:

```
.tx-felogin-pi1 label {
   display: block;
}
```

Breaking: #79109 - Lowlevel VersionsCommand parameters changed

See Issue #79109

Description

The existing CLI command within EXT:lowlevel for showing and cleaning up versions (from EXT:version / EXT:workspaces) has been migrated to a Symfony Console command.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner versions` is now available via `./typo3/sysext/core/bin/typo3 cleanup:versions` and allows the following CLI options to be set:

The following options can be set:

- `--action={nameofaction}` to clean up versioned records, one of the following actions are possible:
  - "versions_in_live": Delete versioned records in the live workspace
  - "published_versions": Delete versions of published records
  - "invalid_workspace": Move records inside a non-existing workspace ID into the live workspace
  - "unused_placeholders": Remove placeholders which are not used anymore from the database
- `-v` and `-vv` to show more detailed information on the records affected
- `--pid=23` or `-p=23` to only find versions with page ID 23 (otherwise "0" is taken)
- `--depth=4` or `-d=4` to only clean recursively until a certain page tree level.
- `--dry-run` to only show the records to be changed / deleted

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\VersionsCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner versions` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding and cleaning up versioned records.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

Breaking: #79120 - Remove legacy CLI-related constants and variables

See Issue #79120

Description

The deprecated PHP constants `TYPO3_cliKey` and `TYPO3_cliInclude`, and the global variables `$GLOBALS['temp_cliScriptPath']` and `$GLOBALS['temp_cliKey']` which had been filled when running a CLI command have been removed.

Impact

Calling one of the PHP constants above will result in a PHP error. Accessing the global variables will result in a PHP warning.

Affected Installations

Any installation with third-party CLI commands which use these constants or global variables.

Breaking: #79196 - Toolbar item event handling changed

See Issue #79196

Description

With the introduction of the topbar reloading mechanism, the event handling of toolbar items has changed. Reason is that the event information gets lost, as the whole topbar is rendered from scratch after a reload.

Impact

After reloading the topbar, non-migrated events will not get triggered anymore.

Affected Installations

All installations with old-fashioned toolbar item registrations.

Migration

In most cases it's sufficient to replace the register function with `Viewport.Topbar.Toolbar.registerEvent()`.

Example:

```
define(['jquery', 'TYPO3/CMS/Backend/Viewport'], function($, Viewport) {
        // old registration
        $(MyAwesomeItem.doStuff)

        // new registration
        Viewport.Topbar.Toolbar.registerEvent(MyAwesomeItem.doStuff);
});
```

Breaking: #79201 - EXT:form: Split TypoScript Includes

See Issue #79201

Description

The frontend specific TypoScript setup for EXT:form isn't loaded automatically anymore and must be added manually through static includes. With this change a TYPO3 integrator can easier decide where the extension Typoscript is included.

Impact

Using the extension without adding static includes of EXT:form will result in an erroneous frontend output.

Affected Installations

Any installation with activated EXT:form extension.

Migration

Make sure to include the static TypoScript "Form" in your (root) template record. Same procedure as with static includes of fluid_styled_content or css_styled_content.

Breaking: #79227 - Removed ExtDirect State Provider

See Issue #79227

Description

The ExtDirect based State Provider for ExtJS applications (endpoint `TYPO3.ExtDirectStateProvider.ExtDirect`) has been removed.

The ExtDirect endpoint `TYPO3.ExtDirectStateProvider.ExtDirect` is no longer available.

The following PHP classes have been removed: * `\TYPO3\CMS\Backend\InterfaceState\ExtDirect\DataProvider` * `\TYPO3\CMS\Backend\Tree\AbstractTreeStateProvider` * `\TYPO3\CMS\Backend\Tree\AbstractExtJ`

The relevant JavaScript file `ExtDirect.StateProvider.js` has been removed.

The PHP method php:`DocumentTemplate->setExtDirectStateProvider()` to load the JavaScript file has been removed.

Instead the jQuery-based AMD module `TYPO3\CMS\Backend\Storage` is incorporated to load the data the same way via an anonymous State Provider which is handed to ExtJS as long as ExtJS is still available in the TYPO3 Core.

Impact

Accessing the ExtDirect endpoint will result in a JavaScript error. Loading the JavaScript file will result in a HTTP 404 error.

Instantiating the PHP class will result in a fatal PHP error.

Affected Installations

Any installation using custom implementations with ExtDirect and the State Provider shipped with the TYPO3 Core.

Migration

Include the `TYPO3\CMS\Backend\Storage`, and use the UserSettingsController class directly on the PHP side to access the user settings.

See the implementation of the JavaScript Storage object for a more detailed usage.

Breaking: #79228 - Remove ExtJS Pagetree indicator functionality

See Issue #79228

Description

The functionality to enhance the page tree with custom indicators has been removed. The functionality was never documented and was hidden in the code.

Impact

Custom indicators are not shown in the TYPO3 Backend Pagetree anymore.

Affected Installations

Any installation using indicators of the page tree.

Breaking: #79242 - Remove l10n_mode noCopy

See Issue #79242

Description

The setting `noCopy` has been removed without replacement from the list of possible values of the TCA column property `l10n_mode`.

Impact

Previously `noCopy` prevented that values of the parent language record were copied to a particular localization when that was created. Now, this value is duplicated during the creation of the localized record and has to be cleared manually if required.

Affected Installations

All having `$GLOBALS['TCA'][<table-name>]['columns'][<column-name>]['l10n_mode']` set to `noCopy`.

Migration

Remove setting `$GLOBALS['TCA'][<table-name>]['columns'][<column-name>]['l10n_mode']` if it is set to `noCopy`.

Breaking: #79243 - Remove l10n_mode mergeIfNotBlank

See Issue #79243

Description

The setting `mergeIfNotBlank` has been removed without replacement from the list of possible values of the TCA column property `l10n_mode`.

Impact

Previously values of a localization having a dependent parent record were taken from the parent record if `l10n_mode` for the particular field was set to `mergeIfNotBlank` and the value in the localization was empty. Now, this value is duplicated during the creation of the localized record and has to be modified manually if required.

Affected Installations

All instances with extensions setting TCA options and having `$GLOBALS['TCA'][<table-name>]['columns'][<column`
set to `mergeIfNotBlank`.

Migration

First execute the upgrade wizard **Migrate values in database records having "l10n_mode" set** in the install tool. After that, remove `$GLOBALS['TCA'][<table-name>]['columns'][<column-name>]['l10n_mode']` if it is set to `mergeIfNotBlank`. If `l10n_mode` is removed before the upgrade wizard has been executed, nothing will be migrated - thus, it's important to keep that order of migration.

The upgrade wizard executes the following field usages:

- inline children, pointing to `sys_file_reference`: file references are localized for the the localization, if missing there
- group fields, basically not using MM intermediate tables: value is cloned to the accordant field in the localization, if empty there
- any other field type: value is cloned to the accordant field in the localization, is blank there

The term `blank` refers to an empty string (`''`), `empty` refers to an empty string, null values and zero values (numeric and string).

Breaking: #79243 - Remove sys_language_softMergeIfNotBlank

See Issue #79243

Description

The TypoScript setting `config.sys_language_softMergeIfNotBlank` has been removed without any replacement. This is a result of removing the TCA setting `mergeIfNotBlank` from the list of possible values for `l10n_mode`.

Migration

Remove TypoScript setting `config.sys_language_softMergeIfNotBlank`.

Breaking: #79259 - EXT:t3skin removed

See Issue #79259

Description

The system extension `t3skin` has been removed, as all functionality has been migrated into other system extensions.

All ExtJS-related images and css files have been moved to EXT:core.

All other images have been unused for a while now, and have been deleted from the TYPO3 core.

Impact

Any direct references to EXT:t3skin now lead to missing styling or image(s).

Affected Installations

All installations that use CSS files or images from EXT:t3skin.

Migration

Do not use ExtJS styling or images anymore, as ExtJS will be removed from the core.

Other direct references to image(s) in EXT:t3skin should be migrated to have the image(s) in custom extension.

Breaking: #79263 - Scheduler CLI Controller class removed

See Issue #79263

Description

The PHP class `TYPO3\CMS\Scheduler\Controller\SchedulerCliController` has been removed from the system extension "scheduler" due to the migration to a native Symfony Command.

Impact

Instantiating the mentioned PHP class will result in a fatal PHP error.

Affected Installations

Any installation with a custom extension using this PHP class directly.

Please note that this does not affect any calls via CLI to trigger the scheduler via `typo3/cli_dispatch.phpsh scheduler` directly. This still works as before.

Migration

Remove any direct calls to the PHP class and use the provided APIs via CLI instead.

Breaking: #79270 - Removed RTE processing option disableUnifyLineBreaks

See Issue #79270

Description

The RTE option that could be set via PageTSconfig `RTE.default.proc.disableUnifyLineBreaks` has been removed.

The option was never set by default.

If activated, it allowed that both line feeds (LFs) and carriage returns (CRs) were left as is. If the option was not set, all line breaks were converted to CRLFs after processing (Windows-syntax) to have a unified style of line breaks in the database.

The option was only there for historic reasons in TYPO3 v3 and TYPO3 v4 to allow to simulate old behaviour when no RTE was available.

Impact

When editing or saving a rich-text content element, all line breaks are converted to CRLFs at any time. If this option is set, it is not evaluated anymore.

Affected Installations

Any installation having the mentioned option explicitly activated in PageTSConfig and counting on the non-unified behaviour.

Migration

Remove the option from TSconfig as it is not necessary anymore.

Breaking: #79273 - Removed RteHtmlParser proc options

See Issue #79273

Description

The following TSconfig options for processing content of RTE fields have been removed:

- RTE.default.proc.dontConvBRtoParagraph
- RTE.default.proc.dontProtectUnknownTags_rte
- RTE.default.proc.dontConvAmpInNBSP_rte

Impact

Setting any of these options has no effect anymore.

Content coming from the database towards the RTE will now always keep unknown tags (but HSC'ed), and never have any double-encoded   characters - this was a default since a decade already.

Content stored in the database will now always treat <br> tags as intentional and not treat them like paragraphs, a behaviour which is common in modern Rich Text Editors.

Affected Installations

Installations explicitly setting `RTE.default.proc.dontConvBRtoParagraph=0`, `RTE.default.proc.dontProtectUnknownTags_rte=1` or `RTE.default.proc.dontConvAmpInNBSP_rte=1` might experience different results when editing and saving content via an RTE.

Migration

Remove the TSconfig options, as they have no effect anymore. Any custom implementation which is necessary should be handled via separate entryHtmlParser and exitHtmlParsers in both directions.

Breaking: #79300 - Removed RTE proc.transformBoldAndItalicTags option

See Issue #79300

Description

The RTE processing TSconfig option `RTE.default.proc.transformBoldAndItalicTags` has been removed from the processing functionality.

It was a shortcut to change all <b> and <i> tags coming from the database to and when loading the RTE. In return when storing the content again from the RTE, the and tags were moved to <b> and <i> again.

If an integrator wanted to explicitly disable this functionality (basically having and in the database), he/she needed to explicitly disable the option (setting it to "0", not just unsetting the option via PageTSconfig).

Impact

Setting this option does not transform the tags anymore when loading the RTE or storing in DB. Instead, and are stored in the database when editing a record.

Affected Installations

Any installation having custom RTE configuration and explicitly setting this option without having a proper HTMLparser replacement mapping in place.

Migration

Any default configuration of RTEHtmlArea that was in place before 8.6.0 has a simple replacement to ensure the same functionality now:

This code does the same as having `proc.transformBoldAndItalicTags=1`:

```
RTE.default.proc {
        # make <strong> and <em> tags when sending to the RTE
        HTMLparser_rte {
                tags {
                        b.remap = strong
                        i.remap = em
                }
        }
        # make <b> and <i> tags when sending to the DB
        HTMLparser_db {
```

```
                tags {
                        strong.remap = B
                        em.remap = I
                }
        }
}
```

If having the option explicitly turned off (allowing strong, b, em, and i tags) is what is wanted the configuration should look like this:

```
RTE.default.proc {
        # no remapping should happen, tags should stay as they are
        HTMLparser_rte {
                tags {
                        b.remap >
                        i.remap >
                }
        }
        # no remapping should happen, tags should stay as they are
        HTMLparser_db {
                tags {
                        strong.remap >
                        em.remap >
                }
        }
}
```

Please note that this migration is only necessary if custom RTE options are in place, as the default RTE HTMLArea configuration does that automatically.

Breaking: #79302 - Moved pages.url_scheme to compatibility7 extension

See Issue #79302

Description

The database field "pages.url_scheme" functionality has been moved to the compatibility7 extension.

The field allows to force the HTTP or HTTPS protocol for a specific page to be set by an editor in the page properties on a per-page basis. However, it is common today to ensure (if a SSL certificate is available) to use HTTPS for a whole website or even only for a specific area (inc. subpages) to force the protocol.

Impact

If the functionality was used before, it will not work anymore, thus links will not be forced to be generated with a forced HTTP/HTTPS url scheme and redirects on pages that had the option set will not happen anymore, unless the compatibility7 extension is installed.

Generating preview links with pages that have an enforced scheme out of the TYPO3 backend will not work anymore.

Affected Installations

Any TYPO3 instance that depends on the `url_scheme` database field, having any value filled in.

Migration

Install the compatibility7 extension to have the same functionality as before, or use HTTPS enforcing via server configuration (.htaccess) or any SSL related extension in the TYPO3 Extension Repository (TER) that provides superior functionality.

To ensure a certain protocol when previewing a page the TSconfig option `TCEMAIN.previewDomain` can be used to set a preview prefix including the URL scheme.

Breaking: #79327 - The veriCode - vC parameter is not evaluated any more

See Issue #79327

Description

The `veriCode` (`&vC=...`) parameter generated by `AbstractUserAuthentication::veriCode` is not evaluated anymore in:

- `ImportExportController::checkUpload()`
- `FileController::main()`
- `EditDocumentController::processData()`
- `SimpleDataHandlerController::main()`

Also the following properties have been removed:

- `EditDocumentController::vC`
- `SimpleDataHandlerController::vC`
- `ImportExportController::vC`

Impact

Any code reading from the removed `vC` properties will now throw an "Undefined property" notice.

Affected Installations

Any installation having code relying on 'vC' property being present in aforementioned classes, or relying on `vC` parameter being checked.

Migration

Remove calls to `veriCode` or any `vC` HTTP parameter evaluation from your code. Ensure your code uses `moduleToken` to protect backend urls.

Breaking: #79364 - Move page module function `QuickEdit` to compatibility7

See Issue #79364

Description

The function `QuickEdit` in the page module has been moved to EXT:compatibility7 and will not be developed further. EXT:compatibility7 will be moved to TER before the release of 8 LTS.

Impact

Installation of EXT:compatibility7 is required to continue using the `QuickEdit` function of the page module.

Affected Installations

All installations depending on the `QuickEdit` function in the page module.

Breaking: #79464 - EXT:form - Refactor fluid rendering

See Issue #79464

Description

EXT:form uses "fluid" as the default rendering strategy. Therefore, EXT:form has to work closely with the concepts of fluid to avoid current and future problems. Until now, EXT:form tried to reuse a fluid view instance by reconfiguring the instance on each nesting level, but fluid is not intended for such a purpose. This change reduces the complexity of the rendering process and works closer with the concepts of fluid.

Impact

The configuration options `renderingOptions.templateRootPaths`, `renderingOptions.partialRootPaths` and `renderingOptions.layoutRootPaths` for form elements are from now on only rules for the root form element ('Form') and will be applied for all child form elements. If you configure `renderingOptions.templateRootPaths` etc. for other form elements it will have no effect.

The configuration option `renderingOptions.templatePathAndFilename` for form elements was removed from the configuration and will have no effect. To define a template file name which should be used instead of a filename which is named like the form element type, there is a new option `renderingOptions.templateName`.

The internal setting `renderingOptions.renderableNameInTemplate` for form elements has been removed from the configuration and will have no effect.

The setting `rendererClassName` for form elements are from now on only rules for the root form element ('Form'). If you define this option for other form elements, an `invalid configuration` exception will be thrown.

The configuration for the backend editor inline templates which are used by editor javascript has changed. The configuration path `prototypes.<prototypeName>.formEditor.formEditorTemplates` has been renamed and has no effect anymore. The fluid configuration part moved from `prototypes.<prototypeName>.formEditor.formEditorTemplates` to a new section `prototypes.<prototypeName>.formEditor.formEditorFluidConfiguration`. The backend editor inline template mapping moved to a new section `prototypes.<prototypeName>.formEditor.formEditorPartials`. The inline template mapping for stage templates has been condensed. If you define custom form editor stage templates which use a default stage template it could result in a javascript error within the form editor.

The template files moved from `Resources/Private/Frontend/Templates/FormElements/` to `Resources/Private/Frontend/Partials`. The template structure has changed. Without adaptation of your overridden templates, no form elements are visible within the frontend.

Affected Installations

All installations since TYPO3 8.5 which use the new EXT:form extension and create or extend custom form elements through configuration and / or override EXT:form template files.

Migration

If you override/ extend

```
TYPO3.CMS.Form.mixins.formElementMixins.BaseFormElementMixin.renderingOptions.templa
TYPO3.CMS.Form.mixins.formElementMixins.BaseFormElementMixin.renderingOptions.partia
TYPO3.CMS.Form.mixins.formElementMixins.BaseFormElementMixin.renderingOptions.layout
```

move it to

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
```

If you override/ extend

```
TYPO3.CMS.Form.mixins.formElementMixins.BaseFormElementMixin.renderingOptions.skipUn
```

move it to

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
```

If you defined

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.<formElementType>.r
```

for a <formElementType> which is *NOT* 'Form', you have to remove this setting.

If you defined

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
```

you have to use

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptio
```

`templateName` is the partial path, relative to `TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDef`

If you defined custom form editor templates within

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formEditor.formEditorTemplates
```

you have to move this to

```
TYPO3.CMS.Form.prototypes.<prototypeName>.formEditor.formEditorPartials
```

If you defined a custom form editor stage template which depends on a default form editor stage template you have to redefine it:

> Stage/Text => Stage/SimpleTemplate Stage/Password => Stage/SimpleTemplate Stage/Advanced-Password => Stage/SimpleTemplate Stage/Textarea => Stage/SimpleTemplate Stage/Checkbox => Stage/SimpleTemplate Stage/MultiCheckbox => Stage/SelectTemplate Stage/MultiSelect => Stage/SelectTemplate Stage/RadioButton => Stage/SelectTemplate Stage/SingleSelect => Stage/SelectTemplate Stage/DatePicker => Stage/SimpleTemplate Stage/Hidden => Stage/SimpleTemplate Stage/FileUpload => Stage/FileUploadTemplate Stage/ImageUpload => Stage/FileUploadTemplate

All form element templates except the template for the `Form` element moved from templates to partials. You have to move this too, if you extended the fluid search paths. The 'Form' element is a template and will be found through `TYPO3.CMS.Form.prototypes.<prototypeName>.formElementsDefinition.Form.renderingOptions.t`
All other form elements are partials and will be found through `TYPO3.CMS.Form.prototypes.<prototypeName>.formEl`

The template/partial structure has changed. You have to adapt this to your custom templates. Please look at the files within `EXT:form/Resources/Private/Frontend/Partials` to see what has happened. The main change is that you have to wrap the markup with

```
<formvh:renderRenderable renderable="{element}">
    some form element
</formvh:renderRenderable>
```

Breaking: #79513 - Removed session locking based on useragent

See Issue #79513

Description

When using session data or user-login functionality with TYPO3, the default configuration was to harden the session binding to the User Agent information sent by the HTTP request. If the user agent information does not match, the session gets renewed and the user gets logged out.

The options `$TYPO3_CONF_VARS['FE']['lockHashKeyWords']` and `$TYPO3_CONF_VARS['BE']['lockHashKeyWords']` were set to "useragent" by default to use this additional session locking check.

This case is especially problematic when having a larger website (e.g. a community platform) with 100K frontend users and the session lifetime set to 6 months. After every security update of the browser or possibly a plugin, or if a version update is happening on Evergreen Browsers, then all users would get logged out, which is inconvenient.

Based on the additional security level on top versus the user experience on the site, the "useragent" functionality has been dropped. Since the "lockHashKeyWords" options did only work on "useragent" and no other functionality was integrated, the option (and related, the database fields "ses_hashlock" as well) has been removed without substitution.

Impact

The options `$TYPO3_CONF_VARS['FE']['lockHashKeyWords']` and `$TYPO3_CONF_VARS['BE']['lockHashKeyWords']` are removed automatically when hitting the install tool.

The database fields 'fe_sessions.ses_hashlock' and 'be_sessions.ses_hashlock' have been removed.

The public property `$lockHashKeyWords` of the PHP class `AbstractUserAuthentication` has been removed and will throw a PHP Notice when trying to access it.

All other functionality related to sessions still works the same.

Affected Installations

Any installation using the configuration options for custom checks based on the session handling with third-party extensions, which is very unlikely.

Migration

The TYPO3 Install Tool removes the configuration option for existing installations. Using the "Database Comparison" view, it is possible to remove the fields from the database.

Breaking: #79622 – CSS Styled Content and TypoScript

See Issue #79622

Description

In order to streamline CSS Styled Content and Fluid Styled Content several options of CSS Styled Content have been dropped without replacement.

Options Dropped: - TCA image_compression - TCA image_effects - TCA image_noRows - TypoScript IMAGE noRows dropped - TypoScript IMAGE noCols dropped - TypoScript IMAGE noRowsStdWrap dropped - TypoScript IMGTEXT captionAlign dropped

Impact

The options mentioned above will have no effect.

Affected Installations

All Installations that use the options mentioned above.

Migration

Image Compression

Use global image compression configuration of TYPO3 instead of deciding compression on content element level.

Image Effects

Use CSS to apply effects on Images.

No Rows

Use CSS to style the output.

Caption Alignment

Use CSS to align the caption text to your preference.

Breaking: #79622 – CSS Styled Content Bullet Content Element Adjustments

See Issue #79622

Description

In order to streamline the options and enhance compatibility across CSS Styled Content and Fluid Styled Content the bullet content element has been partly refactored.

The bullet content elements of CSS Styled Content now also uses the field `bullets_type` instead of `layout` to match the behavior of Fluid Styled Content.

Affected Installations

Installations that use the CSS Styled Content element bullets.

Migration

Run the upgrade wizard in the install tool to migrate the layout field to the dedicated database field `bullets_type`.

Breaking: #79622 - CSS Styled Content table content element adjustments

See Issue #79622

Description

In order to streamline the options and enhance compatibility across CSS Styled Content and Fluid Styled Content the table content element has been partly refactored. All previous flexform configuration has been migrated to database fields, shared across both content rendering definitions.

Element options removed: - Table Summary - No CSS styles for this table

Element options changed: - Additional CSS Class

Rendering changes: - Additional CSS classes for tr, th, td have been dropped

TypoScript options removed: - color - tableParams_0 - tableParams_1 - tableParams_2 - tableParams_3 - border - cellpadding - cellspacing

Table Summary

The <table> summary attribute is not supported in HTML5 and has been dropped. No migration path available.

No CSS styles for table

The default CSS styling for CSS Styled Content is now optional. If no styling is required, simply do not include the optional static template or override the styling with CSS.

Additional CSS Class

The process of adding additional CSS classes for tables has been changed. To ease the work of the editor the CSS class field is no longer a simple input field. Adding CSS classes are now handled by predefined CSS classes that can be adjusted by the integrator. Classes will be prefixed with "contenttable-".

```
TCEFORM.tt_content.table_class {
   removeItems = striped,bordered
   addItems {
      hover = LLL:my_extension/Resources/Private/language.xlf:hover
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['table_class']['config']['items'][] = [
   0 = LLL:my_extension/Resources/Private/language.xlf:hover
   1 = hover
];
```

Rendering changes and removed TypoScript options

Style specific options have been removed and are no longer available. This includes the following options for table rendering: `color`, `tableParams_0`, `tableParams_1`, `tableParams_2`, `tableParams_3`, `border`, `cellpadding` and `cellspacing`. Also additional CSS classes for `tr`, `th` and `td` are no longer available.

Affected Installations

Installations that use the CSS Styled Content element table.

Migration

Run the upgrade wizard in the install tool to migrate all fields previously stored in flexforms to dedicated fields in the database.

Table summary

The <table> summary attribute is not supported in HTML5 and has been dropped. No migration path available.

No CSS styles for this table

Remove the optional "CSS Styled Content Styling" static template.

Additional CSS classes

Additional CSS Classes must be registered as items for the field `table_class`.

Rendering changes and removed TypoScript Options

Use CSS styling to restore the look of your tables.

Breaking: #79622 - Dedicated content elements for menus

See Issue #79622

Description

For better maintainability the currently existing content element menu has been split into dedicated content elements.

Database Key	Name	Description
menu_abstract	Abstracts	Menu of subpages of selected pages including abstracts
menu_categorized_content	Categorized content	Content elements for selected categories
menu_categorized_pages	Categorized pages	Pages for selected categories
menu_pages	Pages	Menu of selected pages
menu_subpages	Subpages	Menu of subpages of selected pages
menu_recently_updated	Recently updated pages	Menu of recently updated pages
menu_related_pages	Related pages	Menu of related pages based on keywords
menu_section	Section index	Page content marked for section menus
menu_section_pages	Section index of subpages	Menu of subpages of selected pages including sections
menu_sitemap	Sitemap	Expanded menu of all pages and subpages for selected pages
menu_sitemap_pages	Sitemaps of selected pages	Expanded menu of all subpages for selected pages

Affected Installations

All installations that use the content element "menu".

Migration

Run the migration wizard in the install tool. All shipped menu types from the TYPO3 core will be migrated to the new dedicated elements.

The migration is optional, you can also enable the extension `compatibility7` that will make the old menu content element available again.

Breaking: #79622 - Default content element changed for Fluid Styled Content

See Issue #79622

Description

The default content element has been streamlined with CSS Styled Content and has been changed to "Text".

Impact

The default content element is now "Text".

Affected Installations

All instances that have Fluid Styled Content installed.

Migration

To restore the configuration you need to set the default content element manually to your preferred choice. You can do this by simply overriding the configuration again in your `Configuration/TCA/Overrides/tt_content.php` file.

```
$GLOBALS['TCA']['tt_content']['columns']['CType']['config']['default'] = 'textmedia
↪';
```

```
$GLOBALS['TCA']['tt_content']['columns']['CType']['config']['default'] = 'header';
```

Breaking: #79622 - Default layouts for Fluid Styled Content changed

See Issue #79622

Description

The content element layouts for Fluid Styled Content have been changed to provide a better maintainability and to be more flexible.

Previously available content element layouts `ContentFooter`, `HeaderFooter` and `HeaderContentFooter` have been dropped and replaced with a single `Default` layout that is more flexible.

Impact

The content element layouts `ContentFooter`, `HeaderFooter` and `HeaderContentFooter` are no longer available. Referencing these layouts will result in an exception.

Affected Installations

All instances that override or implement custom content elements based on Fluid Styled Content that use the layouts `ContentFooter`, `HeaderFooter` or `HeaderContentFooter`.

Migration

All content elements and overrides need to be migrated to the new default layout. Have a look at the feature description on how to use the new layout.

Feature-79622-NewDefaultLayoutForFluidStyledContent.rst

Breaking: #79622 - Dropping thumbnail configuration for tt_content

See Issue #79622

Description

It is currently not possible to set the thumbnail field depending on the type of the record. Since tt_content uses different fields to store media we are removing this default configuration that was set to `images` by CSS Styled Content and `assets` by Fluid Styled Content.

Impact

Thumbnails in list view are no longer displayed for tt_content records.

Affected Installations

All instances.

Migration

To restore the configuration you need to set the thumbnail field manually to your preferred choice. You can do this by simply adding the configuration again in your `Configuration/TCA/Overrides/tt_content.php` file.

```
$GLOBALS['TCA']['tt_content']['ctrl']['thumbnail'] = 'image';
```

```
$GLOBALS['TCA']['tt_content']['ctrl']['thumbnail'] = 'assets';
```

Breaking: #79622 - Removal of Fluid Styled Content Menu ViewHelpers

See Issue #79622

Description

Fetching data directly in the view is not recommended and the temporary solution of menu ViewHelpers has been replaced by its successor, the menu processor that is based on HMENU.

Menu ViewHelpers have been moved to the `compatibility7` extension, and are replaced in the core menu content elements.

List of removed ViewHelpers: - menu.categories - menu.directory - menu.keywords - menu.list - menu.section - menu.updated

Affected Installations

All installations that use the `fluid_styled_content` menu ViewHelpers.

Migration

Use `TYPO3\CMS\Frontend\DataProcessing\MenuProcessor` instead of ViewHelpers.

For CMS 8 the ViewHelpers will be available as soon as `compatibility7` is installed, but it's highly recommended to migrate your configuration.

Example (Directory)

Before:

```
tt_content.menu_subpages.dataProcessing {
   10 = TYPO3\CMS\Frontend\DataProcessing\SplitProcessor
   10 {
      if.isTrue.field = pages
      fieldName = pages
      delimiter = ,
      removeEmptyEntries = 1
      filterIntegers = 1
      filterUnique = 1
      as = pageUids
   }
}
```

```
<ce:menu.directory pageUids="{pageUids}" as="pages" levelAs="level">
   <f:for each="{pages}" as="page">
      ...
   </f:for>
</ce.menu.directory>
```

After:

```
tt_content.menu_subpages.dataProcessing {
   10 = TYPO3\CMS\Frontend\DataProcessing\MenuProcessor
   10.special = directory
   10.special.value.field = pages
}
```

```
<f:for each="{menu}" as="page">
   ...
</f:for>
```

Breaking: #79622 - Section Frame for CSS Styled Content replaced with Frame Class

See Issue #79622

Description

The functionality provided by `Section Frame` has been streamlined with Fluid Styled Content and is now available as `Frame Class`. Previously, integers have been stored in the Database that required a mapping of non speaking values.

The new introduced speaking values provide a more valuable use, for example in Fluid Styled Content the values are used directly without mapping.

For CSS Styled Content the original behaviour has been mapped to the new keys in the database and the option `invisible` has been dropped.

Compatibility Table

Name	Previous Key	New Key	CSS Class	Additional Effects
Default	0	default	csc-frame csc-frame-default	•
Invisible	1	(dropped)	•	•
Ruler Before	5	ruler-before	csc-frame csc-frame-ruler-before	•
Ruler After	6	ruler-after	csc-frame csc-frame-ruler-after	•
Indent	10	indent	csc-frame csc-frame-indent	•
Indent, 33/66%	11	indent-left	csc-frame csc-frame-indent-left	•
Indent, 66/33%	12	indent-right	csc-frame csc-frame-indent-right	•
No Frame	66	none	(none)	No Frame is rendered

TypoScript Before

```
tt_content.stdWrap.innerWrap.cObject.key.field = section_frame
tt_content.stdWrap.innerWrap.cObject.5 =< tt_content.stdWrap.innerWrap.cObject.
↪default
tt_content.stdWrap.innerWrap.cObject.5.20.10.value = csc-frame csc-frame-ruler-
↪before
```

TypoScript After

```
tt_content.stdWrap.innerWrap.cObject.key.field = frame_class
tt_content.stdWrap.innerWrap.cObject.ruler-before =< tt_content.stdWrap.innerWrap.
↪cObject.default
tt_content.stdWrap.innerWrap.cObject.ruler-before.20.10.value = csc-frame csc-frame-
↪ruler-before
```

Affected Installations

Installations that use the CSS Styled Content.

Migration

Default fames can be automatically upgraded to the new field and values. Custom values will be prefixed with `custom-<key>` and will also be transferred to the new field.

Note that custom values must be added again to the field configuration, and the mapping in the TypoScript rendering definition for CSS Styled Content needs also to adapt.

Add custom frame

```
TCEFORM.tt_content.frame_class {
   addItems {
      custom-1 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪customFrame
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['frame_class']['config']['items'][] = [
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:customFrame
   1 = custom-1
];
```

Adapt rendering definition

```
tt_content.stdWrap.innerWrap.cObject.custom-1 =< tt_content.stdWrap.innerWrap.
↪cObject.default
tt_content.stdWrap.innerWrap.cObject.custom-1.20.10.value = csc-frame csc-frame-
↪custom-1
```

Breaking: #79622 - SpaceBefore and SpaceAfter adjustments for CSS Styled Content

See Issue #79622

Description

CSS Styled Content provided the possibility to the editor to fine-tune distances between content elements. The concept of CSC relied on the editor understanding what `margins` are, how they are calculated and had to maintain an overview of pixels that were used on the site he/she is maintaining.

This led to different problems not only for the editor but also for the integrator because he had no control about what the editor fills into these fields. Also it was hardly controllable when these distances should be variable and change on certain viewports for mobile usage.

To regain control over this behaviour we are now introducing a new concept that purely relies on CSS classes, that can be defined by the integrator.

The original fields `spaceAfter` and `spaceBefore` have been dropped, and also the method `\TYPO3\CMS\CssStyledContent\Controller\CssStyledContentController::renderSpace()` is not called anymore.

Old TypoScript Rendering

```
tt_content.stdWrap.innerWrap.cObject.default.20.20 = USER
tt_content.stdWrap.innerWrap.cObject.default.20.20 {
   userFunc = TYPO3\CMS\CssStyledContent\Controller\CssStyledContentController->
 ↪renderSpace
   space = before
   constant = {$content.spaceBefore}
   classStdWrap {
      required = 1
      noTrimWrap = |csc-space-before-| |
   }
}
```

New TypoScript Rendering

```
tt_content.stdWrap.innerWrap.cObject.default.20.20 = TEXT
tt_content.stdWrap.innerWrap.cObject.default.20.20 {
   field = space_before_class
   required = 1
   noTrimWrap = |csc-space-before-| |
}
```

Impact

User-defined distances between content elements are missing.

Affected Installations

All instances that use CSS Styled Content and have spaceBefore or spaceAfter values set to generate more space between their content elements.

Check if your site is affected

```
SELECT
   uid,
   pid,
```

```
   spaceBefore,
   spaceAfter
FROM
   tt_content
WHERE
   (spaceBefore > 0 OR spaceAfter > 0)
   AND deleted = 0
```

Migration

There is no automatic migration available. If a migration is necessary you need to check the new presets available and migrate the pixels defined before to the a preset of your choice.

Example

```
UPDATE
   tt_content
SET
   spaceAfter = 0,
   space_after_class = 'medium'
WHERE
   spaceAfter >= 42
   AND spaceAfter < 56
```

Replacement Documentation

For detailed information about the replacement of spaceBefore and spaceAfter, please head over to the feature documentation of SpaceBefore- and SpaceAfterClass for CSS Styled Content.

Feature-79622-SpaceBeforeAndSpaceAfterClassForCssStyledContent.rst

Breaking: #79622 - Streamlining structure of CSS Styled Content and Fluid Styled Content

See Issue #79622

Description

The file structures of CSS Styled Content and Fluid Styled Content have been streamlined.

File structure of CSS Styled Content

```
- Configuration/TypoScript
| - ContentElement
| | - Bullets.txt
| | - Div.txt
| | - Header.txt
| | - Html.txt
| | - Image.txt
| | - List.txt
| | - MenuAbstract.txt
| | - MenuCategorizedContent.txt
| | - MenuCategorizedPages.txt
| | - MenuPages.txt
```

```
| | - MenuRecentlyUpdated.txt
| | - MenuRelatedPages.txt
| | - MenuSection.txt
| | - MenuSectionPages.txt
| | - MenuSitemap.txt
| | - MenuSitemapPages.txt
| | - MenuSubpages.txt
| | - Shortcut.txt
| | - Table.txt
| | - Text.txt
| | - Textmedia.txt
| | - Textpic.txt
| | - Uploads.txt
| - ContentElementPartials
| | - Menu.txt
| - Helper
| | - ParseFunc.txt
| | - StandardHeader.txt
| | - StylesContent.txt
| - Styling
| | - setup.txt
| - constants.txt
| - setup.txt
```

File structure of Fluid Styled Content

```
- Configuration/TypoScript
| - ContentElement
| | - Bullets.txt
| | - Div.txt
| | - Header.txt
| | - Html.txt
| | - Image.txt
| | - List.txt
| | - MenuAbstract.txt
| | - MenuCategorizedContent.txt
| | - MenuCategorizedPages.txt
| | - MenuPages.txt
| | - MenuRecentlyUpdated.txt
| | - MenuRelatedPages.txt
| | - MenuSection.txt
| | - MenuSectionPages.txt
| | - MenuSitemap.txt
| | - MenuSitemapPages.txt
| | - MenuSubpages.txt
| | - Shortcut.txt
| | - Table.txt
| | - Text.txt
| | - Textmedia.txt
| | - Textpic.txt
| | - Uploads.txt
| - Helper
| | - FluidContent.txt
| | - ParseFunc.txt
| - Styling
| | - setup.txt
| - constants.txt
| - setup.txt
```

Impact

TYPO3 will fail to load the rendering definitions correctly if the paths are not included matching the new file locations.

Affected Installations

All installations that are referring to the previous location of the rendering definitions. Please check if your using any of these paths for including the rendering definitions.

CSS Styled Content

- EXT:css_styled_content/static/v4.5
- EXT:css_styled_content/static/v4.6
- EXT:css_styled_content/static/v4.7
- EXT:css_styled_content/static/v6.0
- EXT:css_styled_content/static/v6.1
- EXT:css_styled_content/static/v6.2
- EXT:css_styled_content/static
- EXT:css_styled_content/Configuration/TypoScript/v7

Fluid Styled Content

- EXT:fluid_styled_content/TypoScript/Static

Migration

Database entries can be automatically upgraded to the new locations. If you have references in your TypoScript files you need to do the migration manually.

Use the new locations for accessing the TypoScript configuration.

- `CSS Styled Content` = EXT:css_styled_content/Configuration/TypoScript/
- `Fluid Styled Content` = EXT:fluid_styled_content/Configuration/TypoScript/

Breaking: #79622 - TypoScript Standard Header has been removed from Fluid Styled Content

See Issue #79622

Description

The TypoScript standard header rendering definition `lib.stdHeader` has been introduced in CSS Styled Content to reference it across multiple content elements to simplify maintenance.

For Fluid Styled Content a workaround for compatibility with CMS 7 has been introduced to simplify migration. However, it only renders the header and misses all frames, and additional are options necessary to generate a streamlined rendering output if the content element layout was not implemented correctly.

Example

TypoScript Configuration

```
tt_content.simple_content = COA
tt_content.simple_content {
   10 < lib.stdHeader
   20 = TEXT
   20.field = bodytext
}
```

Generated Output

```
<header>
   <h1>Nunc vel libero dignissim</h1>
</header>
<p>
   Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed cursus
   vel lectus vel placerat. Suspendisse non metus sed lorem sagittis
   consequat non vel nulla.
</p>
```

Expected Output

```
<div id="c53" class="frame frame-default frame-type-header frame-layout-0">
   <a id="c62"></a>
   <header>
      <h1>Nunc vel libero dignissim</h1>
   </header>
   <p>
      Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed cursus
      vel lectus vel placerat. Suspendisse non metus sed lorem sagittis
      consequat non vel nulla.
   </p>
   <p>
      <a href="#top">Top</a>
   </p>
</div>
```

Impact

Content elements using lib.stdHeader will miss the header in the frontend output.

Affected Installations

All installations that have content elements relying on lib.stdHeader and use Fluid Styled Content as content rendering definition.

Migration

To fully embrace support for Fluid Styled Content the setup needs to be changed. This is a very simple example, it is highly recommended to migrate content elements and plugins to only use fluid. The default fluid layout has all

information necessary to render the header, frame and everything else correctly. That means that you do not need to care about streamlined rendering.

Example for Content Element with TypoScript Rendering

If you need TypoScript to process the rendering of your content element the recommended way is to use a variable to pass the rendering to fluid.

TypoScript Setup

```
tt_content.simple_content =< lib.fluidContent
tt_content.simple_content {
   templateName = SimpleContent
   templateRootPaths.12022017 = EXT:my_simple_content/Resources/Private/Templates/
   variables.content = COA
   variables.content {
      10 = TEXT
      10.field = bodytext
   }
}
```

Fluid Template

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers" data-namespace-
↪typo3-fluid="true">
<f:layout name="Default" />
<f:section name="Main">
   <f:format.raw>{content}</f:format.raw>
</f:section>
</html>
```

Recommended Content Element Rendering

It's highly recommended to migrate content elements to embrace fluid and streamline the rendering definitions.

TypoScript Setup

```
tt_content.simple_content =< lib.fluidContent
tt_content.simple_content {
   templateName = SimpleContent
   templateRootPaths.12022017 = EXT:my_simple_content/Resources/Private/Templates/
}
```

Fluid Template

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers" data-namespace-
↪typo3-fluid="true">
<f:layout name="Default" />
<f:section name="Main">
   <f:format.html>{data.bodytext}</f:format.html>
</f:section>
</html>
```

Customize header rendering

Section in the default layout are optional and have fallbacks. To change for example the header rendering, place a `Header` section in a template and place your adjustments. For unsetting the header simply place an empty `Header` section.

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers" data-namespace-
↪typo3-fluid="true">
<f:layout name="Default" />
<f:section name="Header">
   <h1>{data.header}</h1>
</f:section>
<f:section name="Main">
   <f:format.html>{data.bodytext}</f:format.html>
</f:section>
</html>
```

2.3.2 Features

Feature: #12211 - Usability: Scheduler provide page browser to choose start page

See Issue #12211

Description

To improve the usability of the linkvalidator scheduler task, the page browser is provided to choose the start page.

Impact

Scheduler tasks that need a page `uid` can now add a button for the page browser popup.

In the `ValidatorTaskAdditionalFieldProvider` two additional fields have to be added.

```
'browser' => 'page',
```

If the additional field `browser` is set to `page` then the `SchedulerModuleController` adds a button for calling the page browser popup to the field.

```
'pageTitle' => $pageTitle,
```

The `pageTitle` contains the title of the page that is shown next to the browse button.

Feature: #28171 - Improved link field in FormEngine

See Issue #28171

Description

The handling of link fields when managing records has been improved and now shows a human readable string, an icon and an additional help text instead of the cryptic `t3://` syntax. This is enabled by default for all `renderType="inputLink"` elements.

Impact

Better UX of link fields in the Backend

Feature: #45537 - Run manually executed tasks on next cron-run

See Issue #45537

Description

There is a new action icon to mark a task to be run by cron. Also a new button "Execute selected tasks on next cron job" has been added to mark all selected actions to be run by next cron job.

Feature: #47006 - Extend the widget identifier with custom string

See Issue #47006

Description

The parameter `customWidgetId` has been introduced for fluid widgets. This string is used in the widget identifier in addition to the `nextWidgetNumber`.

The widget identifier is used to create the GET parameter names.

A good value for the `customWidgetId` is the {contentObjectData.uid} to ensure no collisions happen.

Example: .. code-block:: html

```
<f:widget.paginate customWidgetId="{contentObjectData.uid}" ...></f:widget.paginate>
```

Impact

Allows to use the same fluid widget more than once on one page in different content elements.

Feature: #47135 - Paste icons available at pasting position and use modal now

See Issue #47135

Description

As soon as the normal clipboard contains an item, a single paste icon becomes available in the page module. The icon is located at each possible pasting position directly besides the [content +] buttons. When the user clicks on the icon, a modal pops up to have the user confirm the action. Depending on the clipboard mode this will either be "Copy" or "Move" together with the title of the item in the clipboard and a "Cancel" button.

Feature: #51291 - Synchronized field values in localized records

See Issue #51291

Description

The localized record overlay behaviour has been changed to make localization-rows standalone.

Previously, if fields in `TCA` columns were set to `l10n_mode` `exclude` or `mergeIfNotBlank`, the localized record overlay did not contain values, and those values were "pulled up" from the underlying default language records.

This has been changed, the `DataHandler` now copies those values over to the localized record and synchronizes them if the default language record is changed.

As a substitution of the `mergeIfNotBlank` feature, the new configuration `allowLanguageSynchronization` has been added. Setting this adds a wizard to single fields and an editor can select if a field of a localized record should be kept in sync with the default language record, or the localized record it was derived from.

A typical configuration looks like that:

```
'columns' => [
    ...
    'header' => [
        'label' => 'My header',
        'config' => [
            'type' => 'input',
            'behaviour' => [
                'allowLanguageSynchronization' => true,
            ],
        ],
    ],
],
```

`TCA` tables that configure the language localization get the field `l10n_state` added by the schema analyzer which stores a json string with field names and the values `custom`, `parent` or `source` to specify if and from which record a single field gets its value.

Feature: #67243 - Implement folding of scheduler task groups

See Issue #67243

Description

When task groups are used, the tasks are displayed grouped in the list of tasks. Clicking on the row with the group title hides or shows the tasks of the group now.

Feature: #69572 - Page module Notice "Content is also shown on:"

See Issue #69572

Description

When page content is inherited from a different page via "Show content from page" there is a notice displayed on the page that is pulling in content from a different page.

As of now, the page whose content is used on other pages gets an info box that indicates which other pages use these contents.

Impact

On pages that are inherited elsewhere you see a notice which links to the pages where the content is inherited.

Feature: #70316 - Introduce Session Storage Framework

See Issue #70316

Description

A new session storage framework has been introduced. The goal of this framework is to create interoperability between different session storages (called "backends") like database, file storage, Redis, etc.

Impact

An integrator may configure session backends based on `TYPO3_MODE`, which is either `BE` or `FE`.

The following session backends are available by default:

- `\TYPO3\CMS\Core\Session\Backend\DatabaseSessionBackend`
- `\TYPO3\CMS\Core\Session\Backend\RedisSessionBackend`

The default session backend for `BE` and `FE` is `DatabaseSessionBackend` with `table` set to `fe_sessions` and `be_sessions` respectively.

The configuration of the backend for each `TYPO3_MODE` is stored within `SYS/session`:

```
'SYS' => [
    'session' => [
        'BE' => [
            'backend' => \TYPO3\CMS\Core\Session\Backend\RedisSessionBackend::class,
↪
            'options' => [
                'hostname' => 'localhost',
                'database' => 2
            ]
        ],
    ],
],
```

The `DatabaseSessionBackend` requires a `table` as option. If the backend is used to holds non-authenticated sessions (default for `FE`), the `has_anonymous` option must be set to true.

The `RedisSessionBackend` requires a running PHP redis module (PHP extension "redis") and a running redis service. By default, a connection will be made to `hostname` 127.0.0.1 and `port` 6379. You may also specify a `database` number which to store the sessions in (default database is 0) and a `password` for the connection.

A developer may implement a custom session backend. To achieve this, the interface `\TYPO3\CMS\Core\Session\Backend\SessionBackendInterface` has to be implemented.

Feature: #72749 - CLI support for T3D import

See Issue #72749

Description

EXT:impexp now allows to import data files (T3D or XML) via the command line interface through a Symfony Command.

Impact

The command line allows the following options:

```
Imports a T3D file into a page.

USAGE:
 ./typo3/sysext/core/bin/typo3 impexp:import [<options>] <file> <pageId>

ARGUMENTS:
  --file      The path / filename to import (.t3d or .xml), the EXT: syntax can be␣
↪used as well
  --pageId    The page id where the page should be started from, defaults to "0"␣
↪if not set

OPTIONS:
  --updateRecords   Force updating existing records
  --ignorePid       Don't correct page ids of updated records
  --enableLog       log all database action
```

Feature: #75880 - Implement multiple cropping variants in image manipulation tool

See Issue #75880

Description

The `imageManipulation` TCA type is now capable to handle multiple crop variants if configured.

The default configuration is to have only one variant with the same possible aspect ratios like in older TYPO3 versions.

For that the TCA configuration has been extended. The following example configures two crop variants, one with the id "mobile", one with the id "desktop". The array key defines the crop variant id, which will be used when rendering an image with the image view helper.

The allowed crop areas are now also configured differently. The array key is used as identifier for the ratio and the label is specified with the "title" and the actual (floating point) ratio with the "value" key. The value **should** be of PHP type float, not only a string.

```
'config' => [
     'type' => 'imageManipulation',
     'cropVariants' => [
         'mobile' => [
             'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.mobile',
             'allowedAspectRatios' => [
                 '4:3' => [
                     'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.4_3',
                     'value' => 4 / 3
                 ],
                 'NaN' => [
                     'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.free',
```

```
                        'value' => 0.0
                    ],
                ],
            ],
            'desktop' => [
                'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.desktop',
                'allowedAspectRatios' => [
                    '4:3' => [
                        'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.4_3',
                        'value' => 4 / 3
                    ],
                    'NaN' => [
                        'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.free',
                        'value' => 0.0
                    ],
                ],
            ],
        ]
]
```

It is now also possible to define an initial crop area. If no initial crop area is defined, the default selected crop area will cover the complete image. Crop areas are defined relatively with floating point numbers. The x and y coordinates and width and height must be specified for that. The below example has an initial crop area in the size the previous image cropper provided by default.

```
'config' => [
    'type' => 'imageManipulation',
    'cropVariants' => [
        'mobile' => [
            'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.mobile',
            'cropArea' => [
                'x' => 0.1,
                'y' => 0.1,
                'width' => 0.8,
                'height' => 0.8,
            ],
        ],
    ],
]
```

Users can also select a focus area, when configured. The focus area is always **inside** the crop area and mark the area in the image which must be visible for the image to transport its meaning. The selected area is persisted to the database but will have no effect on image processing. The data points are however made available as data attribute when using the `<f:image />` view helper.

The below example adds a focus area, which is initially one third of the size of the image and centered.

```
'config' => [
    'type' => 'imageManipulation',
    'cropVariants' => [
        'mobile' => [
            'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.mobile',
            'focusArea' => [
                'x' => 1 / 3,
                'y' => 1 / 3,
                'width' => 1 / 3,
                'height' => 1 / 3,
```

```
            ],
        ],
    ],
]
```

Very often images are used in a context, where there are overlaid with other DOM elements like a headline. To give editors a hint which area of the image is affected, when selecting a crop area, it is possible to define multiple so called cover areas. These areas are shown inside the crop area. The focus area cannot intersect with any of the cover areas.

```
'config' => [
    'type' => 'imageManipulation',
    'cropVariants' => [
        'mobile' => [
            'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.mobile',
            'coverAreas' => [
                [
                    'x' => 0.05,
                    'y' => 0.85,
                    'width' => 0.9,
                    'height' => 0.1,
                ]
            ],
        ],
    ],
]
```

The above configuration examples are basically meant to add one single cropping configuration to sys_file_reference, which will then apply in every record, which reference images.

It is however also possible to provide a configuration per content element. If you for example want a different cropping configuration for tt_content images, then you can add the following to your `image` field configuration of tt_content records:

```
'config' => [
    'overrideChildTca' => [
        'columns' => [
            'crop' => [
                'config' => [
                    'cropVariants' => [
                        'mobile' => [
                            'title' => 'LLL:EXT:ext_key/Resources/Private/Language/
↪locallang.xlf:imageManipulation.mobile',
                            'cropArea' => [
                                'x' => 0.1,
                                'y' => 0.1,
                                'width' => 0.8,
                                'height' => 0.8,
                            ],
                        ],
                    ],
                ],
            ],
        ],
    ],
]
```

Please note, that you need to specify the target column name as array key. Most of the time this will be `crop` as this is the default field name for image manipulation in `sys_file_reference`

It is also possible to set the cropping configuration only for a specific tt_content element type by using the `columnOverrides` feature:

```
$GLOBALS['TCA']['tt_content']['types']['textmedia']['columnsOverrides']['assets']['
↪'config']['overrideChildTca']['columns']['crop']['config'] = [
    'cropVariants' => [
        'default' => [
            'disabled' => true,
        ],
        'mobile' => [
            'title' => 'LLL:EXT:ext_key/Resources/Private/Language/locallang.xlf:
↪imageManipulation.mobile',
            'cropArea' => [
                'x' => 0.1,
                'y' => 0.1,
                'width' => 0.8,
                'height' => 0.8,
            ],
            'allowedAspectRatios' => [
                '4:3' => [
                    'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.4_3',
                    'value' => 4 / 3
                ],
                'NaN' => [
                    'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_
↪wizards.xlf:imwizard.ratio.free',
                    'value' => 0.0
                ],
            ],
        ],
    ],
];
```

Please note, that the array for `overrideChildTca` is merged with the child TCA, so are the crop variants that are defined in the child TCA (most likely sys_file_reference). Because you cannot remove crop variants easily, it is possible to disable them for certain field types by setting the array key for a crop variant `disabled` to the value `true`

To render crop variants, the variants can be specified as argument to the image view helper:

```
<f:image image="{data.image}" cropVariant="mobile" width="800" />
```

Impact

TCA configuration for field type "imageManipulation" has changed. Old configuration options still work but are deprecated and issue a warning when used.

The TCA configuration option `enableZoom` has been removed for now. It wasn't really usable anyway and will need some proper UX design before re-implementation. Setting the option will have no effect.

Feature: #78169 - Introduce "Translation Source" field for tt_content

See Issue #78169

Description

The new database field `l10n_source` for tt_content table has been introduced together with a new TCA ctrl configuration `translationSource`. The `translationSource` field contains a uid of the record used as a translation source, no matter whether the record was translated in the free or connected mode. The new TCA configuration `translationSource` contains column name, similar to the `transOrigPointerField`. e.g.

```
$GLOBALS['TCA']['tt_content']['ctrl']['translationSource'] = 'l10n_source';
```

The new field solves few issues:

1. There was no way to detect whether a record was translated using connected mode or free mode. If a record has value > 0 in the `transOrigPointerField` (e.g. `l10n_parent`) field, it means it was translated using "connected mode". If the `transOrigPointerField` is 0 but the `translationSource` field is > 0 it means it was translated using "free mode". If both are 0, it means the record was not translated but created manually.
2. TYPO3 allows to use a record in non-default language as a translation source. In this case the information about the translation source was lost. Now, the `translationSource` field always contains a uid of the record used as a translation source.
3. In some places `origUid` (e.g. `t3_origuid`) fields were misused as a translation source. Now these places can be refactored to use the `translationSource` field.

Difference between `translationSource` and other existing fields

1. `transOrigPointerField` (e.g. `l10n_parent`) - "translation parent" - this field contains uid of the record in the *default language* representing the same content. The `translationSource` field can contain a uid of the record in non-default language.
2. `origUid` (e.g. `t3_origuid`) - "copy source" - this field contains uid of the record, current record was *copied from*. It might be equal to `translationSource` as localization is a copy internally, but often it is different.

See following test scenarios to see how data is handled in details.

- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::localizeCo`
- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::localizeCo`
- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::copyContent`
- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::copyPage`
- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::copyPageFre`
- `\TYPO3\CMS\Core\Tests\Functional\DataHandling\Regular\Modify\ActionTest::localizePa`

Impact

Introduction of the new field is a base step for further refactorings, e.g. - it opens a way to implement features like "reconnecting" free-mode translations back to the "connected mode" - replace usage of the `t3_origuid` with the `l10n_source` where `t3_origuid` is misused for language handling purposes (e.g. in LocalizationRepository)

Feature: #78192 - Refactor click menu (context menu)

See Issue #78192

Description

Click-menu (context-menu) handling has been refactored and unified. The ExtJS/ExtDirect click-menu used on the page tree has been replaced with a jQuery based implementation. The same context-menu implementation is used in all places in the Backend (page tree, page module, list module, file list, folder tree...).

Context-menu rendering flow

The context-menu is shown after click on the HTML element which has `class="t3js-contextmenutrigger"` together with `data-table`, `data-uid` and optional `data-context` attributes.

The JavaScript click event handler is implemented in the `TYPO3/CMS/Backend/ContextMenu` requireJS module. It takes the data attributes mentioned above and executes an ajax call to the `\TYPO3\CMS\Backend\Controller\ContextMenuController->getContextMenuAction()`.

`ContextMenuController` asks `\TYPO3\CMS\Backend\ContextMenu\ContextMenu` to generate an array of items. ContextMenu builds a list of available item providers by asking each whether it can provide items (`->canHandle()`), and what priority it has (`->getPriority()`).

Custom item providers can be registered in `$GLOBALS['TYPO3_CONF_VARS']['BE']['ContextMenu']['ItemProvid` They must implement `\TYPO3\CMS\Backend\ContextMenu\ItemProviders\ProviderInterface` and can extend `\TYPO3\CMS\Backend\ContextMenu\ItemProviders\AbstractProvider`.

A list of providers is sorted by priority, and then each provider is asked to add items. The generated array of items is passed from an item provider with higher priority to a provider with lower priority.

After that, a compiled list of items is returned to the `ContextMenuController` which passes it back to the ContextMenu.js as JSON.

Example of the JSON response:

```
{
   "view":{
      "type":"item",
      "label":"Show",
      "icon":"<span class=\"t3js-icon icon icon-size-small icon-state-default icon-
↪actions-document-view\" data-identifier=\"actions-document-view\">\n\t<span␣
↪class=\"icon-markup\">\n<img src=\"\/typo3\/sysext\/core\/Resources\/Public\/
↪Icons\/T3Icons\/actions\/actions-document-view.svg\" width=\"16\" height=\"16\"␣
↪\/>\n\t<\/span>\n\t\n<\/span>",
      "additionalAttributes":{
         "data-preview-url":"http:\/\/typo37.local\/index.php?id=47"
      },
      "callbackAction":"viewRecord"
   },
   "edit":{
      "type":"item",
      "label":"Edit",
      "icon":"",
      "additionalAttributes":[
      ],
      "callbackAction":"editRecord"
   },
   "divider1":{
      "type":"divider",
      "label":"",
      "icon":"",
      "additionalAttributes":[

      ],
      "callbackAction":""
   },
   "more":{
      "type":"submenu",
      "label":"More options...",
      "icon":"",
      "additionalAttributes":[

      ],
```

```
        "callbackAction":"openSubmenu",
        "childItems":{
            "newWizard":{
                "type":"item",
                "label":"'Create New' wizard",
                "icon":"",
                "additionalAttributes":{
                },
                "callbackAction":"newContentWizard"
            }
        }
    }
}
```

Based on the JSON data ContextMenu.js is rendering a context-menu. If one of the items is clicked, the according JS `callbackAction` is executed on the `TYPO3/CMS/Backend/ContextMenuActions` JS module or other modules defined in the `additionalAttributes['data-callback-module']`.

For example usage of this API see:

- Beuser item provider `\TYPO3\CMS\Beuser\ContextMenu\ItemProvider` and requireJS module `TYPO3/CMS/Beuser/ContextMenuActions`
- Impexp item provider `\TYPO3\CMS\Impexp\ContextMenu\ItemProvider` and requireJS module `TYPO3/CMS/Impexp/ContextMenuActions`
- Version item provider `\TYPO3\CMS\Version\ContextMenu\ItemProvider` and requireJS module `TYPO3/CMS/Version/ContextMenuActions`
- Version item provider `\TYPO3\CMS\Version\ContextMenu\ItemProvider` and requireJS module `TYPO3/CMS/Version/ContextMenuActions`
- Filelist item providers `\TYPO3\CMS\Filelist\ContextMenu\ItemProviders\FileDragProvider`, `\TYPO3\CMS\Filelist\ContextMenu\ItemProviders\FileProvider`, `\TYPO3\CMS\Filelist\ContextMenu\ItemProviders\FileStorageProvider`, `\TYPO3\CMS\Filelist\ContextMenu\ItemProviders\FilemountsProvider` and requireJS module `TYPO3/CMS/Filelist/ContextMenuActions`

Feature: #78477 - Refactoring of FlashMessage rendering

See Issue #78477

Description

The implementation of rendering FlashMessages in the core has been optimized. With Issue #73698 a cleanup has been started to centralize the markup within the FlashMessage class.

A new class called `FlashMessageRendererResolver` has been introduced. This class detects the context and renders the given FlashMessages in the correct output format. It can handle any kind of output format. The following FlashMessageRendererResolver classes have been introduced:

- `TYPO3\CMS\Core\Messaging\Renderer\BootstrapRenderer` (is used in backend context by default)
- `TYPO3\CMS\Core\Messaging\Renderer\ListRenderer` (is used in frontend context by default)
- `TYPO3\CMS\Core\Messaging\Renderer\PlaintextRenderer` (is used in CLI context by default)

All new rendering classes have to implement the `TYPO3\CMS\Core\Messaging\Renderer\FlashMessageRendererIn` interface.

Impact

The core has been modified to use the new `FlashMessageRendererResolver`. Any third party extension should use the provided `FlashMessageViewHelper` or the new `FlashMessageRendererResolver` class:

```
$out = GeneralUtility::makeInstance(FlashMessageRendererResolver::class)
    ->resolve()
    ->render($flashMessages);
```

Feature: #78899 - TCA maxitems optional

See Issue #78899

Description

The `TCA` config setting `maxitems` for `type=select` and `type=group` fields is now an optional setting that defaults to a high value (99999) instead of 1 as before.

Impact

Fields that typically relate to multiple relations like the group element and some select elements no longer need `maxitems` set to some value to enable multiple values.

Example before:

```
aField => [
    'config' => [
        'type' => `select',
        'renderType' => 'multipleSideBySide',
        'maxitems' => 99999,
    ],
],
```

Example after:

```
aField => [
    'config' => [
        'type' => `select',
        'renderType' => 'multipleSideBySide',
    ],
],
```

This simplifies `TCA` of those fields and removes a cross dependency between `renderType` and `maxitems`.

Feature: #79121 - Implement hook in typolink for modification of page params

See Issue #79121

Description

A new hook has been implemented in ContentObjectRenderer::typoLink for links to pages. With this hook you can modify the link configuration, for example enriching it with additional parameters or meta data from the page row.

Impact

You can now register a hook via:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['typolinkProcessing']['
↪'typolinkModifyParameterForPageLinks'][] = \Your\Namespace\Hooks\MyBeautifulHook::
↪class;
```

Your hook has to implement `TypolinkModifyLinkConfigForPageLinksHookInterface` with its method `modifyPageLinkConfiguration(array $linkConfiguration,array $linkDetails,array $pageRow)`. In `$linkConfiguration` you get the configuration array for the link - this is what your hook can modify and **has to** return. `$linkDetails` contains additional information for your link and `$pageRow` is the full database row of the page.

For more information as to which configuration options may be changed, see TSRef.

Example implementation:

```
public function modifyPageLinkConfiguration(array $linkConfiguration, array
↪$linkDetails, array $pageRow) : array
{
    $linkConfiguration['additionalParams'] .= $pageRow['myAdditionalParamsField'];
    return $linkConfiguration;
}
```

Feature: #79124 - Allow overwriting of template paths in BackendTemplateView

See Issue #79124

Description

BackendTemplateView now allows overwriting of template paths to add your own locations for templates, partials and layouts in a BackendTemplateView based backend module.

Impact

You can now do for example

```
$frameworkConfiguration = $this->configurationManager->
↪getConfiguration(ConfigurationManagerInterface::CONFIGURATION_TYPE_FRAMEWORK);
$viewConfiguration = [
   'view' => [
      'templateRootPaths' => ['EXT:myext/Resources/Private/Backend/Templates'],
      'partialRootPaths' => ['EXT:myext/Resources/Private/Backend/Partials'],
      'layoutRootPaths' => ['EXT:myext/Resources/Private/Backend/Layouts'],
   ],
];
$this->configurationManager->setConfiguration(array_merge($frameworkConfiguration,
↪$viewConfiguration));
```

Feature: #79140 - Add hook to add custom TypoScript templates

See Issue #79140

Description

A new hook in TemplateService allows to add or modify existing TypoScript templates.

Register the hook via `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['Core/TypoScript/TemplateService` in the extensions' ext_localconf.php file.

Example

An example implementation could look like this:

EXT:my_site/ext_localconf.php

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['Core/TypoScript/TemplateService']['runThroughTemplatesPostProcessing'][1313131313] =
    \MyVendor\MySite\Hooks\TypoScriptHook::class . '->addCustomTypoScriptTemplate';
```

EXT:my_site/Classes/Hooks/TypoScriptHook.php

```
namespace MyVendor\MySite\Hooks;

class TypoScriptHook
{

    /**
     * Hooks into TemplateService after
     * @param array $parameters
     * @param \TYPO3\CMS\Core\TypoScript\TemplateService $parentObject
     * @return void
     */
    public function addCustomTypoScriptTemplate($parameters, $parentObject)
    {
        // Disable the inclusion of default TypoScript set via TYPO3_CONF_VARS
        $parameters['isDefaultTypoScriptAdded'] = true;
        // Disable the inclusion of ext_typoscript_setup.txt of all extensions
        $parameters['processExtensionStatics'] = false;

        // No template was found in rootline so far, so a custom "fake" sys_template record is added
        if ($parentObject->outermostRootlineIndexWithTemplate === 0) {
            $row = [
                'uid' => 'my_site_template',
                'config' => '<INCLUDE_TYPOSCRIPT: source="FILE:EXT:my_site/Configuration/TypoScript/site_setup.t3s">',
                'root' => 1,
                'pid' => 0
            ];
            $parentObject->processTemplate($row, 'sys_' . $row['uid'], 0, 'sys_' . $row['uid']);
        }
    }
}
```

Feature: #79196 - Allow reload of topbar

See Issue #79196

Description

A new JavaScript API to reload the backend's topbar has been introduced to the TYPO3 Core.

Impact

The toolbar reloading may be triggered on JavaScript and PHP code-level. To enforce the reloading on PHP side, call `\TYPO3\CMS\Backend\Utility\BackendUtility::setUpdateSignal('updateTopbar')`.

Reloading the topbar via JavaScript requires the following code:

```
// Either: RequireJS style
define(['TYPO3/CMS/Backend/Viewport'], function(Viewport) {
   Viewport.Topbar.refresh();
});

// Or: old-fashioned JavaScript
if (top && top.TYPO3.Backend && top.TYPO3.Backend.Topbar) {
   top.TYPO3.Backend.Topbar.refresh();
}';
```

In case a toolbar item registers to the `load` event of the page, the registration must be changed. Reason is that the event information gets lost, as the whole toolbar is rendered from scratch after a reload.

Example:

```
define(['jquery', 'TYPO3/CMS/Backend/Viewport'], function($, Viewport) {
   // old registration
   $(MyAwesomeItem.doStuff)

   // new registration
   Viewport.Topbar.Toolbar.registerEvent(MyAwesomeItem.doStuff);
});
```

Feature: #79216 - Add YAML configuration for CKEditor RTE

See Issue #79216

Description

The CKEditor-flavored RTE can now be configured via YAML files, defined as *presets*.

A preset contains both the RTE configuration and the HTML processing when storing the content in the database.

A YAML file for RTE configurations can be registered by any extension in `ext_localconf.php`:

```
$GLOBALS['TYPO3_CONF_VARS']['RTE']['Presets']['default'] =
'EXT:rte_ckeditor/Configuration/RTE/Default.yaml';
```

The TYPO3 Core ships with three flavors for the RTE configuration which can also be overridden via PageTSconfig on a per-field/type level:

```
RTE.default.preset = minimal
RTE.config.tt_content.bodytext.types.textmedia.preset = full
```

The PageTSconfig allows to use the minimal configuration everywhere, but to use the full configuration on the tt_content.bodytext field (but only for textmedia content types).

With the YAML configuration files, an "imports" functionality allows to import other configuration and just override the necessary values for a custom configuration for a specific site. This way, the processing part of EXT:rte_ckeditor can be used directly (which acts as best practice) but the editor part can be completely customized.

The YAML format thus states three important parts considered by the RTE configuration preset:

1. "imports" Allows to import other files via the "resource" sub-property
2. "processing" uses the former "proc" options to hand over to RteHtmlParser to sanitize the content - the option are the same as for RTEHtmlArea
3. "editor" A configuration for CKEditor, where all CKEditor-related options can be set which are available from the ckeditor configuration specifications (see http://docs.ckeditor.com/#!/api/CKEDITOR.config for all options).

Feature: #79225 - Plugin preview with Fluid

See Issue #79225

Description

The page TSconfig to render a preview of a single content element in the Backend has been improved by allowing the rendering of plugins as well.

The following option allows to override the default output of a plugin via page TSconfig:

```
mod.web_layout.tt_content.preview.list.example = EXT:site_mysite/Resources/Private/
↪Templates/Preview/ExamplePlugin.html
```

All properties of the tt_content record are available in the template directly. Any data of the flexform field `pi_flexform` is available with the property `pi_flexform_transformed` as an array.

Note: If a PHP hook already is set to render the element, it will take precedence over the Fluid-based preview.

Feature: #79235 - Add button to delete similar errors from sys_log

See Issue #79235

Description

The log module of TYPO3 now shows a button to delete multiple errors at once based on the `details` field of the `sys_log` table. This comes in handy when you fixed an error that spammed the log before.

Feature: #79240 - Single cli user for cli commands

See Issue #79240

Description

Accessing TYPO3 functionality from the command line has been simplified. Single commands no longer require single users in the database, instead all cli command use the username `_cli_`. This user is created on demand by the framework if it does not exist at the first command line call. The `_cli_` user has admin rights and no longer needs specific access rights assigned to perform specific tasks like manipulating database content using the `DataHandler`.

Impact

Creating and managing command line tasks has been simplified, all existing `_cli_*` users can be deleted.

Feature: #79250 - EXT:form extend the extension location functionality

See Issue #79250

Description

EXT:form has a feature to load custom form definitions from within extension locations. These locations can be configured through the `allowedExtensionPaths` setting. To define whether forms can be changed from within extension locations through the form editor, a setting named `allowSaveToExtensionPaths` exists. But this setting affects only already existing form definitions within extension locations. This feature makes it possible to store new forms within extension locations through the form manager as well. You can also define whether forms can be deleted within extension locations through the form manager with a new setting called `allowDeleteFromExtensionPaths`. By default both settings `allowSaveToExtensionPaths` and `allowDeleteFromExtensionPaths` are disabled.

Summary

With this patch is it possible to:

- save existing forms within extension locations ("allowedExtensionPaths") if "allowSaveToExtensionPaths" is set to true (like before)
- save new created forms within extension locations ("allowedExtensionPaths") if "allowSaveToExtensionPaths" is set to true
- delete forms within extension locations ("allowedExtensionPaths") if "allowDeleteFromExtensionPaths" is set to true

Impact

Example to allow edit form definitions within 'EXT:my_ext/Resources/Private/Forms/':

```
TYPO3:
     CMS:
       Form:
         persistenceManager:
           allowSaveToExtensionPaths: true
           allowedExtensionPaths:
             100: EXT:my_ext/Resources/Private/Forms/
```

Example to allow remove form definitions within 'EXT:my_ext/Resources/Private/Forms/':

```
TYPO3:
  CMS:
    Form:
      persistenceManager:
        allowDeleteFromExtensionPaths: true
        allowedExtensionPaths:
          100: EXT:my_ext/Resources/Private/Forms/
```

Feature: #79262 - Add possibility to create TRIM expression with Doctrine DBAL

See Issue #79262

Description

The possibility to create TRIM expressions using Doctrine DBAL has been integrated. However, when using this in comparisons, ExpressionBuilder::comparison() has to be invoked explicitly - otherwise the created TRIM expression would be quoted if e.g. used with ExpressionBuilder::eq().

```
$queryBuilder->expr()->comparison(
    $queryBuilder->expr()->trim($fieldName),
    ExpressionBuilder::EQ,
    $queryBuilder->createNamedParameter('', \PDO::PARAM_STR)
);
```

The call to `$queryBuilder->expr()-trim()` can be one of the following:

- `trim('fieldName')` results in `TRIM("tableName"."fieldName")`
- `trim('fieldName',AbstractPlatform::TRIM_LEADING,'x')` results in `TRIM(LEADING "x" FROM "tableName"."fieldName")`
- `trim('fieldName',AbstractPlatform::TRIM_TRAILING,'x')` results in `TRIM(TRAILING "x" FROM "tableName"."fieldName")`
- `trim('fieldName',AbstractPlatform::TRIM_BOTH,'x')` results in `TRIM(BOTH "x" FROM "tableName"."fieldName")`

Feature: #79263 - Scheduler CLI available as Symfony Command

See Issue #79263

Description

Calling the scheduler to process a task is now callable via CLI through `typo3/cli_dispatch.phpsh scheduler` and `typo3/sysext/core/bin/typo3 scheduler:run`.

The following aliases for the scheduler options are now available: * `--task=13` or `--task 13` as synonym to `-i 13` to run a specific task * `--force` as synonym to `-f` to force to run a specific task in combination with `--task` or `-i` * `--stop` as synonym to `-s` to stop a specific task in combination with `--task` or `-i`

Feature: #79337 - Add useCacheHash parameter to f:link.typolink and f:uri.typolink

See Issue #79337

Description

The older implementation of the two typolink ViewHelpers was lacking support of the useCacheHash parameter.

The boolean argument `useCacheHash` has been added to the typoscript Viewhelpers.

```
<f:link.typolink parameter="{link}" useCacheHash="true">
```

Feature: #79341 - TCA richtext configuration in config section

See Issue #79341

Description

A new config setting `enableRichtext` has been introduced. It enables richtext editing on the text field and replaces the old setting `defaultExtras`.

Impact

Setting `enableRichtext` will result in the text field being rendered with a richtext editor. Config example:

```
'columns' => [
    'content' => [
        'config' => [
            'type' => 'text',
            'enableRichtext' => true,
        ],
    ],
];
```

Feature: #79387 - Add signal to exclude tables from ReferenceIndex

See Issue #79387

Description

A new signal `shouldExcludeTableFromReferenceIndex` is emitted in `TYPO3\CMS\Core\Database\ReferenceIndex` which allows extensions to define tables which should be excluded from ReferenceIndex.

Register the class which excludes tables in `ext_localconf.php`:

```
$dispatcher = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Extbase\SignalSlot\Dispatcher::class);
$dispatcher->connect(
    \TYPO3\CMS\Core\Database\ReferenceIndex::class,
    'shouldExcludeTableFromReferenceIndex',
    \MyVendor\MyExtension\Slots\ReferenceIndexSlot::class,
    'shouldExcludeTableFromReferenceIndex'
);
```

Your class could look like this:

```
namespace MyVendor\MyExtension\Slot;

class ReferenceIndexSlot {

   /**
    * Exclude tables from ReferenceIndex which cannot contain a reference
    *
    * @param string $tableName Name of the table
    * @param bool &$excludeTable Reference to a boolean whether to exclude the␣
↪table from ReferenceIndex or not
    */
   public function shouldExcludeTableFromReferenceIndex($tableName, &
↪$excludeTable) {
      if ($tableName === 'tx_myextension_mytable') {
         $excludeTable = true;
      }
   }
```

```
}
```

Impact

This signal allows extensions to speed up the process of maintaining the ReferenceIndex. If an extension has tables in which by definition none of its columns can contain any relations to other records these can be excluded from the ReferenceIndex.

Only exclude tables from ReferenceIndex which do not contain any relations and never did since existing references won't be deleted if it is excluded! There is no need to add tables without a definition in `$GLOBALS['TCA]` since ReferenceIndex only handles those.

Feature: #79402 - New Fluid ViewHelper f:variable added

See Issue #79402

Description

A new ViewHelper `f:variable` has been added in Fluid 2.2.0 which is now minimum required dependency for TYPO3. The ViewHelper allows variables to be assigned in the template:

```
<f:variable name="myvariable">My variable's content</f:variable>
<f:variable name="myvariable" value="My variable's content"/>
{f:variable(name: 'myvariable', value: 'My variable\'s content')}
{myoriginalvariable -> f:variable.set(name: 'mynewvariable')}
```

Impact

The new ViewHelper is now available in any and all Fluid templates being rendered in TYPO3.

Feature: #73409 - Auto-render Assets sections in Fluid template with controller

See Issue #73409

Description

ActionController has received a new method, `renderAssetsForRequest` which receives the `RequestInterface` of the Request currently being processed. The ActionController has a default implementation of this method which attempts to render two sections in the Fluid template that is associated with the controller action being called:

- `<f:section name="HeaderAssets">` for assets intended for the `<head>` tag
- `<f:section name="FooterAssets">` for assets intended for the end of the `<body>` tag

Both sections are optional.

When rendering, `{request}` is available as template variable in both sections, as is `{arguments}`, allowing you to make decisions based on various request/controller arguments. As usual, `{settings}` is also available.

All content you write into these sections will be output in the respective location as is, meaning you must write the entire `<script>` or whichever tag you are writing, including all attributes. You can of course use various Fluid ViewHelpers to resolve extension asset paths.

The feature only applies to ActionController (thus excluding CommandController) and will only attempt to render the section if the view is an instance of `TYPO3Fluid\\Fluid\\View\\TemplateView` (thus including any View in TYPO3 which extends either TemplateView or AbstractTemplateView from TYPO3's Fluid adapter).

Impact

- Fluid templates rendered through any ActionController using a TemplateView may now contain two new sections for either `HeaderAssets` or `FooterAssets` depending on desired output. Content of these sections will be rendered and assigned via PageRenderer to either header or footer.
- ActionControllers can override the `renderAssetsForRequest` method to perform asset insertion using other means. The method sits at a very opportune point right after the action method itself gets called, when the entire controller is fully initialized with arguments etc. but no forwarding/redirection has happened in the controller action.

Feature: #79413 - Auto-render Assets sections in FLUIDTEMPLATE content object

See Issue #79413

Description

FLUIDTEMPLATE content object will now automatically render two sections and insert the rendered content as assets via PageRenderer. The two sections can be defined in the template file rendered by the FLUIDTEMPLATE object.

- `<f:section name="HeaderAssets">` for assets intended for the `<head>` tag
- `<f:section name="FooterAssets">` for assets intended for the end of the `<body>` tag

Both sections are optional.

When rendering, `{contentObject}` is available as template variable in both sections, allowing you to make decisions based on various aspects of the configured content object instance. In addition, all variables you declared for the content object are available when rendering either section.

All content you write into these sections will be output in the respective location as is, meaning you must write the entire `<script>` or whichever tag you are writing, including all attributes. You can of course use various Fluid ViewHelpers to resolve extension asset paths.

Impact

- Fluid templates rendered through the FLUIDTEMPLATE content object may now contain two new sections for either `HeaderAssets` or `FooterAssets` depending on desired output. Content of these sections will be rendered and assigned via PageRenderer to either header or footer.

Feature: #79420 - Hide files from list of documentation

See Issue #79420

Description

The list of displayed documentation files gets pretty long over time. The user might want to hide those already tackled or of no interest from listing. This is possible via checkboxes.

A new section has been introduced that lists those hidden files to bring them back if necessary.

Impact

The list of documentation files can be shortened now.

Feature: #79438 - Add configuration option to disable validation of stored records

See Issue #79438

Description

Two configuration options have been added to the Install Tool, which are used when saving records using the DataHandler.

The first option allows to disable the check if the contents of a record match the data that should have been written after saving it to the database for the whole TYPO3 installation. This allows a speed-up especially when inserting/copying large amounts of data at the cost of possible differences, if values could not be written as requested. This may happen, if e.g. data types do not match or columns are too small to store the data.

Disable checking the stored records:

```
$GLOBALS['TYPO3_CONF_VARS']['BE']['checkStoredRecords'] = false;
```

If checking the stored records is enabled, the second option allows to make the comparison of the record contents strict. If a loose comparison is made (which is the default), comparing an empty string '' with the number 0 is not considered as an error ('' == 0), while using strict comparison it is ('' !== 0).

Make the comparison of record contents strict:

```
$GLOBALS['TYPO3_CONF_VARS']['BE']['checkStoredRecordsLoose'] = false;
```

Impact

In TYPO3 installations where the administrator has configured the database to run in strict-mode disabling the validation of stored records can speed-up inserts and updates by a factor of 2. With strict-mode enabled the record will not be saved at all, if there are errors.

If the validation of stored records is disabled, the entry in the protocol (sys_log) does not contain the record title (it is displayed as "[No title]" instead) - however table and uid are still provided and can be used for finding the record.

Affected Installations

None as default.

Feature: #79440 - FormEngine Element Expansion

See Issue #79440

Description

A new API in FormEngine has been introduced that allows fine grained additions to single elements and containers without substituting the whole element.

For elements within the `TCA` config section, three new options have been introduced:

- `fieldInformation` An array of single field information. This could be additionally describing text that is rendered between the element label and the element itself. Field information are restricted, only a couple of HTML tags are allowed within the result HTML.
- `fieldControl` An array of single field controls. These are icons with JavaScript or links to further functionality of the framework. They are usually displayed next to the element. Each control must return an icon identifier, a title, and an array of a-tag attributes.
- `fieldWizard` Additional functionality enriching the element. These are typically shown below the element. Wizards may return any HTML.

For FormEngine containers, the same API has been introduced, but it is currently only implemented within the `OuterWrapContainer` which renders the record title and delegates the main record rendering to a different container. Adding `fieldInformation` or `fieldWizard` here allows embedding additional functionality between the record title an the main record body.

Single elements and containers may register default information, control and wizards. The configuration is merged with any possibly given configuration from `TCA`.

Example from `GroupElement`:

```
class GroupElement extends AbstractFormElement
{
    /**
     * Default field controls for this element.
     *
     * @var array
     */
    protected $defaultFieldControl = [
        'elementBrowser' => [
            'renderType' => 'elementBrowser',
        ],
        'insertClipboard' => [
            'renderType' => 'insertClipboard',
            'after' => [ 'elementBrowser' ],
        ],
        'editPopup' => [
            'renderType' => 'editPopup',
            'disabled' => true,
            'after' => [ 'insertClipboard' ],
        ],
        'addRecord' => [
            'renderType' => 'addRecord',
            'disabled' => true,
            'after' => [ 'editPopup' ],
        ],
        'listModule' => [
            'renderType' => 'listModule',
            'disabled' => true,
            'after' => [ 'addRecord' ],
        ],
    ];

    public function render()
    {
        ...
        $fieldControlResult = $this->renderFieldControl();
        $fieldControlHtml = $legacyFieldControlHtml . $fieldControlResult['html'];
        $resultArray = $this->mergeChildReturnIntoExistingResult($resultArray,
↪$fieldControlResult, false);
    }
}
```

This element registers five default field controls (icons next to the element), renders them and later adds the HTML

at an appropriate place within the HTML of the main element. The `defaultFieldControl` can be overwritten on `TCA` level of single fields:

```
'columns' => [
    'aField' => [
        'label' => 'aField',
        'config' => [
            'fieldControl' => [
                'elementBrowser' => [
                    'disabled' => true,
                ],
                'editPopup' => [
                    'disabled' => false,
                ],
                'aNewControl' => [
                    'renderType' => 'myOwnTypeGroupControl',
                    'before' => [ 'elementBrowser' ],
                ],
            ],
        ],
    ],
],
```

The above configuration disables the element browser which is enabled by default, it enabled the edit popup control which exists in default configuration but is disabled by default, and it adds a further control called `aNewControl` with `renderType=myOwnTypeGroupControl`. The renderType instructs the FormEngine `NodeFactory` to instantiate the class that in configured for that renderType, identical to other usages of the NodeFactory, this lookup can be manipulated on configuration and code level. In the example above, a new renderType should be registered in `ext_localconf.php`:

```
$GLOBALS['TYPO3_CONF_VARS']['SYS']['formEngine']['nodeRegistry'][1485351217] = [
    'nodeName' => 'myOwnTypeGroupControl',
    'priority' => 30,
    'class' => \My\ExtensionName\Form\FieldControl\MyFancyControl::class,
];
```

Note the above configuration also uses the `DependencyResolver`: It is possible to resort single elements by adding `before` and `after` to the configuration. In above example, the new control `aNewControl` would be shown as first control, before `elementBrowser`.

The first level below `fieldControl` contains speaking names of single controls, each control must have a `renderType` defined (either on TCA level or via defaultFieldControl), and they may have a `before` and `after` and a `disabled` setting. Each control also may have a `options` sub array with further settings given to the specific control.

In `TCA`, the configuration name for field controls is `fieldControl`, for wizards it is `fieldWizard`, and for information it is `fieldInformation`. All three follow the same structure. It is up to a single element if all three of these are actually called and rendered. For instance, it sometimes does not make sense to have field controls in all elements, so some elements skip that.

For containers, the configuration of `fieldInformation`, `fieldControl` and `fieldWizard` is within the `ctrl` section of `TCA`. This is currently only implemented within the `OuterWrapContainer` for `fieldInformation` and `fieldWizard`.

Example:

```
'ctrl' => [
    ...
    'container' => [
        'outerWrapContainer' => [
            'fieldInformation' => [
                'myHelloWorld' => [
                    'renderType' => 'helloWorld',
```

```
                    ],
                ],
            ],
        ],
    ],
```

The above example would instruct the system to call the class registered for renderType `helloWorld` within the OuterWrapContainer.

Impact

The new API brings lots of new options to add functionality to single elements without substituting the full element.

Feature: #79442 - EXT:form - add element selector for text editors

See Issue #79442

Description

A new button has been added to the text editors of the form editor. Clicking on this button opens an overlay with available form elements of the current form. The user has the possibility to choose one of these form elements. The process adds the dynamic identifier (e.g. "{text-1}" to the current text editor field.

Feature: #79467 - EXT:form - add form settings button to module header

See Issue #79467

Description

A new button has been added to the module header of the form editor. Clicking on this button shows the form settings within the inspector.

Feature: #79521 - Show list of failed input elements in FormEngine

See Issue #79521

Description

When validating input fields of the FormEngine fails, a button is now rendered into the button bar in the module document header. Clicking the button renders a list of all input elements whose validation failed.

Clicking onto a field in that list automatically focuses the field in the form.

Feature: #79530 - EXT:form - Extend SaveToDatabase finisher

See Issue #79530

Description

The SaveToDatabase finisher has been extended by the following functions:

Perform multiple database operations

You can set options as an array to perform multiple database operations.

```
finishers:
  -
    identifier: SaveToDatabase
    options:
      -
        table: 'my_table'
        mode: insert
        databaseColumnMappings:
          some_column:
            value: 'cool'
      -
        table: 'my_other_table'
        mode: update
        whereClause:
          pid: 1
        databaseColumnMappings:
          some_other_column:
            uid_foreign: '{SaveToDatabase.insertedUids.0}'
```

Access the inserted uids from previous database inserts

You can access the inserted uids via `{SaveToDatabase.insertedUids.<theArrayKeyNumberWithinOptions>}`. If you perform an insert operation, the value of the inserted database row will be stored within the FinisherVariableProvider. `<theArrayKeyNumberWithinOptions>` references to the numeric key from the `options` array within which the insert operation is executed.

Add a special option value '{__currentTimestamp}'

You can write `{__currentTimestamp}` as an option value which returns the current timestamp.

```
finishers:
  -
    identifier: SaveToDatabase
    options:
      -
        table: 'my_table'
        mode: insert
        databaseColumnMappings:
          tstamp:
            value: '{__currentTimestamp}'
```

Add a variable container object which is passed through all finishers

There is a simple data storage object available within the `\TYPO3\CMS\Form\Domain\Finishers\FinisherContext`. You can access this from within a finisher with

```
$this->finisherContext->getFinisherVariableProvider()
```

Each finisher can write and/or read data from this object. All data has to be prefixed with the finisher identifier, so you can determine what data from which finisher you want.

Prototype to add some data:

```
$this->finisherContext->getFinisherVariableProvider()->add(
   $this->shortFinisherIdentifier,
   'some.data',
   $yourData
);
```

Prototype to get some data:

```
$otherFinisherData = $this->finisherContext->getFinisherVariableProvider()->get(
   'SomeFinisherIdentifier',
   'some.data'
);
```

You can access this data within the form definition:

For example, a finisher with identifier 'SomeFinisherIdentifier' writes data with the key 'some.data'

```
finishers:
  -
    identifier: SaveToDatabase
    options:
      -
        table: 'my_table'
        mode: insert
        databaseColumnMappings:
          tstamp:
            value: '{SomeFinisherIdentifier.some.key}'
```

You can read more about the configuration options within the SaveToDatabase inline documentation. Please see `\TYPO3\CMS\Form\Domain\Finishers\SaveToDatabaseFinisher`.

Feature: #79531 - EXT:form - Add multiselect inspector editor

See Issue #79531

Description

A new inspector editor, i.e. a new field type of the form editor, has been added. If applied, multi-select fields can be added to the inspector. A multi-select field allows the selection of multiple meta properties for a field and stores them in the defined property path.

For example: If you have a file upload element in your form, until now you could only select a single mime type restriction. With the new multi-select option, the mime-type field was converted and you can now select multiple mime-types (for example docx, doc and odt together).

Feature: #79622 - Header Position support for Fluid Styled Content

See Issue #79622

Description

Header position as known from CSS Styled Content is now also supported by Fluid Styled Content. This will allow the editor to have more control about the alignment of the header in the frontend.

By default all CSS classes for header alignment are prefixed with `ce-headline-` to make the css class unique and to allow even more adjustments without breaking your styling somewhere else.

Predefined values for header alignment and resulting CSS classes

Name	Value	CSS Class
Default	(empty)	(No CSS Class added)
Center	center	ce-headline-center
Right	right	ce-headline-right
Left	left	ce-headline-left

Implementation Example

The following examples are taken from the partials of fluid_styled_content that can be found here `EXT:fluid_styled_content/Resources/Private/Partials/Header/All.html` and here `EXT:fluid_styled_content/Resources/Private/Partials/Header/Header.html`.

```
<f:render partial="Header/Header" arguments="{
   header: data.header,
   layout: data.header_layout,
   positionClass: '{f:if(condition: data.header_position, then: \'ce-headline-{data.
↪header_position}\')}',
   link: data.header_link,
   default: settings.defaultHeaderType}" />
```

```
<h1 class="{positionClass}">
  <f:link.typolink parameter="{link}">{header}</f:link.typolink>
</h1>
```

Edit Predefined Options

```
TCEFORM.tt_content.header_position {
   removeItems = center,left,right
   addItems {
      fancyheader = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪fancyHeader
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['header_position']['config']['items'][] =␣
↪[
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:fancyHeader
   1 = fancyheader
];
```

Impact

Header positions are now available to all editors by default.

Feature: #79622 - Introducing Frame Class for Fluid Styled Content

See Issue #79622

Description

In CSS Styled Content it is possible to provide additional CSS classes for the wrapping container element. This feature is now available for Fluid Styled Content, too.

The default layout of Fluid Styled Content is now passing the value of `Frame Class` directly to the template and prefixes the value by default with `frame-<key>`.

Implementation in Fluid Styled Content

```
<div id="c{data.uid}" class="frame frame-{data.frame_class} ...">
   ...
</div>
```

Explanation of Keys and Effects of Frame Classes

Name	Key	CSS Class	Additional Effects
Default	default	frame-default	•
Ruler Before	ruler-before	frame-ruler-before	A ruler is added after the output.
Ruler After	ruler-after	frame-ruler-after	A ruler is added after the output.
Indent	indent	frame-indent	Margin of 15% is added to the left and right side.
Indent, 33/66%	intent-left	frame-indent-left	Margin of 33% is added to the left side.
Indent, 66/33%	indent-right	frame-indent-right	Margin of 33% is added to the right side.
No Frame	none	(none)	No Frame is rendered.

Please note that you need to include the optional static template "Fluid Styled Content Styling" to have a visual effect on the new added CSS classes.

Edit Predefined Options

```
TCEFORM.tt_content.frame_class {
   removeItems = default,ruler-before,ruler-after,indent,indent-left,indent-right,
↪none
   addItems {
      superframe = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪superframe
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['frame_class']['config']['items'][] = [
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:superframe
   1 = superframe
];
```

Impact

`Frame Class` is now available to all Fluid Styled Content elements.

Feature: #79622 - Introducing Table Class for Fluid Styled Content

See Issue #79622

Description

In CSS Styled Content it is possible to provide additional CSS classes for the rendering of the table content element. This feature is now available for Fluid Styled Content, too.

The default layout of the Fluid Styled Content element table is now passing the value of `Table Class` directly to the template and prefixes the value by default with `ce-table-<key>`.

Implementation in Fluid Styled Content

```
<table class="ce-table{f:if(condition: data.table_class, then: ' ce-table-{data.
↪table_class}')}">
   ...
</table>
```

Explanation of Keys and Effects of Table Classes

Name	Key	CSS Class	Additional Effects
Default	(empty)	(none)	(none)
Stiped	striped	ce-table-striped	Odd rows will be highlighted
Bordered	bordered	ce-table-bordered	A border will be displayed around the table

Please note that you need to include the optional static template "Fluid Styled Content Styling" to have a visual effect on the new added CSS classes.

Default styling for optional "Fluid Styled Content Styling"

```
/* Table */
.ce-table {
   width: 100%;
   max-width: 100%;
}
.ce-table th,
.ce-table td {
   padding: 0.5em 0.75em;
   vertical-align: top;
}
.ce-table thead th {
   border-bottom: 2px solid #dadada;
}
.ce-table th,
.ce-table td {
   border-top: 1px solid #dadada;
}
.ce-table-striped tbody tr:nth-of-type(odd) {
   background-color: rgba(0,0,0,.05);
}
.ce-table-bordered th,
.ce-table-bordered td {
   border: 1px solid #dadada;
}
```

Edit predefined options

```
TCEFORM.tt_content.table_class {
   removeItems = striped,bordered
   addItems {
      supertable = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪supertable
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['table_class']['config']['items'][] = [
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:supertable
   1 = supertable
];
```

Impact

`Table Class` is now available for the Fluid Styled Content table content element.

Feature: #79622 - New Content Elements for Fluid Styled Content

See Issue #79622

Description

Content Elements from CSS Styled Content that were previously not supported by Fluid Styled Content are now making their comeback in order to share the same feature set across both content element renderings.

Name	cType	Description
Text	text	A regular text element with header and bodytext fields.
Text and Images	textpic	Any number of images wrapped right around a regular text element.
Images	image	Any number of images aligned in columns and rows with a caption.

Impact

The content elements Text, Text and Images, Images are now also available when Fluid Styled Content is used as content rendering definition.

Feature: #79622 - New default layout for Fluid Styled Content

See Issue #79622

Description

Previously there have been three layouts you could choose from when you were defining your own custom content elements or overriding an existing template.

To provide a better maintainability and ease of use of overrides we are reducing these to a single layout that is named `Default` with all sections optional and fallbacks if the section is not set. Also we are introducing the "DropIn" concept.

Structure

The `Default` layout consists of five predefined sections that can be utilized to shape the output for your content rendering. In most cases you will not have to care about other section than `Main`. The sections will be rendered in that exact ordering.

- Before
- Header
- Main
- Footer
- After

```
<f:spaceless>
   <f:if condition="{data.frame_class} != none">
      <f:then>
         <div id="c{data.uid}" class="frame frame-{data.frame_class} frame-type-
↪{data.CType} frame-layout-{data.layout}{f:if(condition: data.space_before_class,␣
↪then: ' frame-space-before-{data.space_before_class}')}{f:if(condition: data.
↪space_after_class, then: ' frame-space-after-{data.space_after_class}')}">
            <f:if condition="{data._LOCALIZED_UID}">
               <a id="c{data._LOCALIZED_UID}"></a>
            </f:if>
            <f:render section="Before" optional="true">
               <f:render partial="DropIn/Before/All" arguments="{_all}" />
            </f:render>
            <f:render section="Header" optional="true">
               <f:render partial="Header/All" arguments="{_all}" />
            </f:render>
            <f:render section="Main" optional="true" />
            <f:render section="Footer" optional="true">
               <f:render partial="Footer/All" arguments="{_all}" />
            </f:render>
            <f:render section="After" optional="true">
               <f:render partial="DropIn/After/All" arguments="{_all}" />
            </f:render>
         </div>
      </f:then>
      <f:else>
         <a id="c{data.uid}"></a>
         <f:if condition="{data._LOCALIZED_UID}">
            <a id="c{data._LOCALIZED_UID}"></a>
         </f:if>
         <f:if condition="{data.space_before_class}">
            <div class="frame-space-before-{data.space_before_class}"></div>
         </f:if>
         <f:render section="Before" optional="true">
            <f:render partial="DropIn/Before/All" arguments="{_all}" />
         </f:render>
         <f:render section="Header" optional="true">
            <f:render partial="Header/All" arguments="{_all}" />
         </f:render>
         <f:render section="Main" optional="true" />
         <f:render section="Footer" optional="true">
            <f:render partial="Footer/All" arguments="{_all}" />
         </f:render>
         <f:render section="After" optional="true">
            <f:render partial="DropIn/After/All" arguments="{_all}" />
         </f:render>
         <f:if condition="{data.space_after_class}">
            <div class="frame-space-after-{data.space_after_class}"></div>
```

```
        </f:if>
      </f:else>
   </f:if>
</f:spaceless>
```

DropIn

The sections `Before` and `After` are so called "DropIn" sections. DropIns have been introduced to be able to place additional functionality to all content elements without overriding layouts or templates. DropIns are basically placeholders/empty partials that are meant to be overridden if necessary.

DropIn Locations:

- Resources/Private/Partials/DropIn/Before/All.html
- Resources/Private/Partials/DropIn/After/All.html

Handling Optional Sections

Since all sections are optional you do not need to reference them in your templates. All sections except the `Main` section have a fallback to a default behaviour if they are not set in the template. This is for example used to render the content element header.

```
<f:render section="Header" optional="true">
   <f:render partial="Header/All" arguments="{_all}" />
</f:render>
```

In some cases it can be useful to disable or override the default fallback of a section. For example if the HTML does not want to render the header at all.

```
<f:layout name="Default" />
<f:section name="Header" />
<f:section name="Main">
   <f:format.raw>{data.bodytext}</f:format.raw>
</f:section>
```

```
<f:layout name="Default" />
<f:section name="Header">
   <f:if condition="{gallery.position.noWrap} != 1">
      <f:render partial="Header/All" arguments="{_all}" />
   </f:if>
</f:section>
<f:section name="Main">
   ...
</f:section>
```

Basic Usage

```
<f:layout name="Default" />
<f:section name="Main">
   ...
</f:section>
```

Feature: #79622 - SpaceBefore- and SpaceAfterClass for CSS Styled Content

See Issue #79622

Description

CSS Styled Content provided the possibility to the editor to fine-tune distances between content elements. The concept of CSC relied on the editor understanding what `margins` are, how they are calculated and had to maintain an overview of pixels that were used on the site he/she is maintaining.

This led to different problems not only for the editor but also for the integrator because he had no control about what the editor fills into these fields. Also it was hardly controllable when these distances should be variable and change on certain viewports for mobile usage.

To regain control over this behaviour we are now introducing a new concept that purely relies on CSS classes, that can be defined by the integrator.

All CSS classes are prefixed with `frame-space-before-` or `frame-spacer-after-` by default and added to the surrounding frame when available. If the frame of the content element is set to none, placeholder elements are placed before and after to generate that distance.

The default CSS definitions are placed in the optional static template `TypoScript Content Elements CSS (optional)`. If this is not included only the CSS classes will be added but without having CSS rules matching these classes.

Example for before classes

Name	Value	CSS Class	Margin
None	(empty)	(No CSS Class added)	(No Margin)
Extra Small	extra-small	csc-space-before-extra-small	1em
Small	small	csc-space-before-small	2em
Medium	medium	csc-space-before-medium	3em
Large	large	csc-space-before-large	4em
Extra Large	extra-large	csc-space-before-extra-large	5em

Example Output

```
<div id="c43" class="... csc-space-before-medium">
   ...
</div>
```

```
<a id="c43"></a>
<div class="csc-space-before-medium"></div>
...
<div class="csc-space-after-medium"></div>
```

Edit Predefined Options

```
TCEFORM.tt_content.space_before_class {
   removeItems = extra-small,small,medium,large,extra-large
   addItems {
      superspace = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪superSpace
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['space_before_class']['config']['items
↪']['] = [
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:superSpace
```

```
    1 = superspace
];
```

Impact

SpaceBefore and SpaceAfter is now available to all CSS Styled Content elements.

Feature: #79622 - SpaceBefore- and SpaceAfterClass for Fluid Styled Content

See Issue #79622

Description

CSS Styled Content provided the possibility to the editor to fine-tune distances between content elements. The concept of CSC relied on the editor understanding what `margins` are, how they are calculated and had to maintain an overview of pixels that were used on the site he/she is maintaining.

This led to different problems not only for the editor but also for the integrator because he had no control about what the editor fills into these fields. Also it was hardly controllable when these distances should be variable and change on certain viewports for mobile usage.

To regain control over this behaviour we are now introducing a new concept that purely relies on CSS classes, that can be defined by the integrator.

All CSS classes are prefixed with `frame-space-before-` or `frame-spacer-after-` by default and added to the surrounding frame when available. If the frame of the content element is set to none, placeholder elements are placed before and after to generate that distance.

The default CSS definitions are placed in the optional static template `Fluid Content Elements CSS (optional)`. If this is not included only the CSS classes will be added but without having CSS rules matching these classes.

Example for before classes

Name	Value	CSS Class	Margin
None	(empty)	(No CSS Class added)	(No Margin)
Extra Small	extra-small	frame-space-before-extra-small	1em
Small	small	frame-space-before-small	2em
Medium	medium	frame-space-before-medium	3em
Large	large	frame-space-before-large	4em
Extra Large	extra-large	frame-space-before-extra-large	5em

Example Output

```
<div id="c43" class="frame ... frame-space-before-medium">
   ...
</div>
```

```
<a id="c43"></a>
<div class="frame-space-before-medium"></div>
...
<div class="frame-space-after-medium"></div>
```

Edit predefined options

```
TCEFORM.tt_content.space_before_class {
   removeItems = extra-small,small,medium,large,extra-large
   addItems {
      superspace = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:
↪superSpace
   }
}
```

```
$GLOBALS['TCA']['tt_content']['columns']['space_before_class']['config']['items
↪']['] = [
   0 = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:superSpace
   1 = superspace
];
```

Impact

SpaceBefore and SpaceAfter is now available to all Fluid Styled Content elements.

Feature: #79622 - Textmedia support for CSS Styled Content

See Issue #79622

Description

CSS Styled Content now comes with support for the content element "Text and Media" that was previously exclusive to Fluid Styled Content, to make the transition from CSS Styled Content to Fluid Styled content easier.

The "Text and Media" implementation uses the fluid rendering for the Gallery from the Fluid Styled Content implementation and also the ClickEnlarge ViewHelper. This is only a temporary solution until we remove CSS Styled Content from the TYPO3 Core with CMS 9.

Impact

"Text and Media" content element is now also available for CSS Styled Content.

Feature: #79626 - Integrate record link handler

See Issue #79626

Description

The functionality of EXT:linkhandler has been integrated into the core. It enables editors to link to single records.

The configuration consists of the following parts:

PageTsConfig is used to create a new tab in the LinkBrowser to be able to select records:

```
TCEMAIN.linkHandler.anIdentifier {
    handler = TYPO3\CMS\Recordlist\LinkHandler\RecordLinkHandler
    label = LLL:EXT:extension/Resources/Private/Language/locallang.xlf:link.
↪customTab
    configuration {
```

```
        table = tx_example_domain_model_item
    }
    scanBefore = page
}
```

The following optional configuration is available:

- `configuration.hidePageTree = 1`: Hide the page tree in the link browser
- `configuration.storagePid = 1`: Let the link browser start with the given page
- `configuration.pageTreeMountPoints = 123,456`: Mount the given pages instead of the regular page tree

You can position your own handlers in order as defined in https://docs.typo3.org/typo3cms/extensions/core/7.6/Changelog/7.6/Feature-66369-AddedLinkBrowserAPIs.html

TypoScript is used to generate the actual link in the frontend

```
config.recordLinks.anIdentifier {
    // Do not force link generation when the record is hidden
    forceLink = 0

    typolink {
        parameter = 123
        additionalParams.data = field:uid
        additionalParams.wrap = &tx_example_pi1[item]=|&tx_example_
↪pi1[controller]=Item&tx_example_pi1[action]=show
        useCacheHash = 1
    }
}
```

2.3.3 Deprecation

Deprecation: #51291 - PageRepository shouldFieldBeOverlaid()

See Issue #51291

Description

The following method has been deprecated:

- `TYPO3\CMS\Frontend\Page\PageRepository->shouldFieldBeOverlaid()`

Impact

Localized record fields are always "overlaid", the method returns true in all cases.

Affected Installations

Instances with extensions calling this method

Migration

The deprecated method returns TRUE in all cases, the call can be omitted.

Deprecation: #70316 - Frontend basket with recs

See Issue #70316

Description

The TypoScriptFrontendController has a basic mechanism to automatically register session data if the GET/-POST variable `recs` is given. This has been deprecated. This additionally obsoletes the configuration variable `$GLOBALS['TYPO3_CONF_VARS']['FE']['maxSessionDataSize']` which has been deprecated, too.

Impact

Handling baskets or other session data in `recs` throws a deprecation warning.

Affected Installations

Extensions with a legacy that rely on this automatic basket (`tt_products` for example) should be adapted. Searching extensions for string `recs` should reveal affected parts.

Migration

Use the session functions `setKey()` and `getKey()` of `$GLOBALS['TSFE']->fe_user` directly to store session data like basket information from within the extension.

Deprecation: #77934 - Deprecate tt_content field select_key

See Issue #77934

Description

The field `select_key` of the table `tt_content` is not used in the core and has been removed.

Impact

The field `select_key` is not available by default anymore.

Affected Installations

All installations and extensions using the field `select_key` of the table `tt_content`.

Migration

Install the extension `compatibility7` to enable the field again.

Deprecation: #78225 - Legacy PreparedStatements within Extbase

See Issue #78225

Description

Extbase has a way to set raw statements with PreparedStatements based on the legacy DatabaseConnection a.k.a. `TYPO3_DB`. This functionality has been marked as deprecated.

Impact

Calling a query within Extbase with `$query->setStatement($preparedStatement)` using a `\TYPO3\CMS\Core\Database\PreparedStatement` object will trigger a deprecation log entry.

Affected Installations

Any TYPO3 extension with Extbase functionality using custom prepared statements with the legacy database API.

Migration

Use the same method `setStatement()` and provide a QueryBuilder object or a Statement object based on Doctrine DBAL.

Deprecation: #78477 - Refactoring of FlashMessage rendering

See Issue #78477

Description

The following methods and properties within `FlashMessage::class` have been marked as deprecated:

- `FlashMessage->classes`
- `FlashMessage->icons`
- `FlashMessage->getClass()`
- `FlashMessage->getIconName()`

Impact

Using these properties and methods will stop working in TYPO3 v9.

Affected Installations

All installations using the mentioned methods and properties above.

Migration

Use the new `FlashMessageRendererResolver::class`, for example:

```
GeneralUtility::makeInstance(FlashMessageRendererResolver::class)->resolve()->
↪render()
```

Deprecation: #78899 - FormEngine Methods

See Issue #78899

Description

The following methods have been marked as deprecated:

- `TYPO3\CMS\Core\Database\RelationHandler->readyForInterface()`
- `TYPO3\CMS\Backend\Form\FormDataProvider->sanitizeMaxItems()`
- `TYPO3\CMS\Backend\Utility::getSpecConfParts()`
- `TYPO3\CMS\Backend\Controller\Wizard\ColorpickerController` and backend route `wizard_colorpicker`
- `TYPO3\CMS\Backend\Form\Wizard\SuggestWizard` and template `typo3/sysext/backend/Resources/Private/Templates/Wizards/SuggestWizard.html`
- `TYPO3\CMS\Backend\Form\AbstractNode->getValidationDataAsDataAttribute()`
- `TYPO3\CMS\Backend\Form\Element->renderWizards()`

Impact

Using above methods will throw a deprecation warning.

Affected Installations

Extensions using above methods.

Migration

- `sanitizeMaxItems()` has been merged into calling methods using a default value and sanitizing with `MathUtility::forceIntegerInRange()`.
- `readyForInterface()` has been substituted with the easier to parse method `getResolvedItemArray()`.
- `getSpecConfParts()` is obsolete with the removal of `defaultExtras` TCA
- `ColorpickerController` is obsolete with the JavaScript based colorpicker in the backend
- `SuggestWizard` has been merged into `GroupElement` directly, the standalone class is obsolete
- `getValidationDataAsDataAttribute()` - use `getValidationDataAsJsonString()` and `htmlspecialchars()` the result or use `GeneralUtility::implodeAttributes()` with second argument set to true.
- `renderWizards()` has been substituted with the new API `NodeExpansion`. Old `popup,userFunc,script` wizards are still called and rendered, but the method usage should be avoided and extensions should switch to the new API.

Extensions using above methods should consider to switch away from those methods.

Deprecation: #78899 - TCA ctrl field requestUpdate dropped

See Issue #78899

Description

The `TCA` `ctrl` configuration option `['ctrl']['requestUpdate']` has been dropped. This option was often used together with `displayCond` fields to re-evaluate display conditions if referenced fields changed their value. Typically, a "Refresh required" popup is raised to the editor in those cases, if the editor did not disable that. The field has been moved and is now located within the `['columns']` section of the single field as `'onChange' => 'reload'`.

Impact

The field is just moved from `ctrl` section to the single field `columns` section. An automatic TCA migration does that and logs deprecation messages.

Affected Installations

All `TCA` tables that use `requestUpdate` in `ctrl` section.

Migration

Monitor the deprecation log for according messages, remove the `ctrl` field and add `'onChange' => 'reload'` to fields listed in `requestUpdate` parallel to `label` and `config` section of the field in question. The option can be added to multiple fields.

Deprecation: #79258 - Methods getRecordLocalization() and getPreviousLocalizedRecordUid() in LocalizationRepository

See Issue #79258

Description

The methods `LocalizationRepository::getRecordLocalization()` and `LocalizationRepository::getPreviousLocalizedRecordUid()` have been marked as deprecated as they are not used in the core anymore, since https://review.typo3.org/#/c/47645/ was merged.

Impact

Calling these methods will trigger a deprecation log entry. Code using them will work until these methods are removed in TYPO3 v9.

Affected Installations

Any installation using the mentioned methods `LocalizationRepository::getRecordLocalization()` and `LocalizationRepository::getPreviousLocalizedRecordUid()`.

Migration

No migration available.

Deprecation: #79265 - CommandLineController and Cleaner Command

See Issue #79265

Description

The following PHP classes for using CLI commands without Extbase Command Controllers or native Symfony Commands which were introduced in TYPO3 v4 have been marked as deprecated:

- `TYPO3\CMS\Core\Controller\CommandLineController`
- `TYPO3\CMS\Lowlevel\CleanerCommand`

Impact

Instantiating any of the PHP classes above will trigger a deprecation log entry.

Affected Installations

TYPO3 instances with extensions that use the old command line controllers or cleaner commands.

Migration

Use native Symfony Commands or Extbase Command Controller logic instead for creating CLI-based functionality.

Deprecation: #79316 - Deprecate ArrayUtility::inArray()

See Issue #79316

Description

Deprecate ArrayUtility::inArray()

Impact

Calling `ArrayUtility::inArray()` method will trigger a deprecation log entry. Code using this method will work until it is removed in TYPO3 v9.

Affected Installations

Any installation using the mentioned method `ArrayUtility::inArray()`.

Migration

Use the native `in_array()` function of PHP. It is strongly recommended to ensure the same type is used everywhere and the 3rd parameter of `in_array()` is set to `true` to activate the type check.

Deprecation: #79327 - Deprecate AbstractUserAuthentication::veriCode method

See Issue #79327

Description

The `AbstractUserAuthentication::veriCode` method has been marked as deprecated.

Right now all Backend urls require module token, so veriCode is not needed any more. The Veri token was used as an alternative verification when the JavaScript interface executes cmds to tce_db.php from eg. MSIE 5.0 because the proper referrer is not passed with this browser...

Impact

Calling `AbstractUserAuthentication::veriCode` will trigger a deprecation log entry.

Affected Installations

Any installation having extensions calling `AbstractUserAuthentication::veriCode`

Migration

Remove calls to `veriCode` or any `vC` HTTP parameter evaluation from your code. Ensure your code uses `moduleToken` to protect backend urls.

Deprecation: #79341 - Methods related to richtext configuration

See Issue #79341

Description

The following methods and method arguments have been deprecated:

- Method `\TYPO3\CMS\Backend\Utility\BackendUtility::getSpecConfParametersFromArray()`
- Method `\TYPO3\CMS\Backend\Utility\BackendUtility::RTEsetup()`
- Second argument `$specConf` of `\TYPO3\CMS\Core\Html\RteHtmlParser->RTE_transform()`

Impact

Using above methods or arguments trigger deprecation log entries, the according methods will vanish with TYPO3 v9.

Affected Installations

Loaded extensions using one of the above methods.

Migration

If not otherwise possible, class `\TYPO3\CMS\Core\Configuration\Richtext` can be used to fetch richtext configuration. Be aware this class is marked @internal and is likely to change or vanish in TYPO3 v9 again.

Deprecation: #79341 - TCA richtext configuration in defaultExtras dropped

See Issue #79341

Description

Enabling richtext rendering for fields in the Backend record editor has been simplified.

In the past, a typical `TCA` configuration of a richtext field looked like:

```
'columns' => [
    'content' => [
        'config' => [
            'type' => 'text',
        ],
        'defaultExtras' => 'richtext:rte_transform',
    ],
];
```

The `defaultExtras` is obsolete and substituted with `enableRichtext` within the `config` section:

```
'columns' => [
    'content' => [
        'config' => [
            'type' => 'text',
            'enableRichtext' => true,
        ],
    ],
];
```

If the RTE was enabled for a specific type only, it looked like this:

```
'columns' => [
    'content' => [
        'config' => [
            'type' => 'text',
        ],
    ],
],
'types' => [
    'myType' => [
        'columnsOverrides' => [
            'aField' => [
                'defaultExtras' => 'richtext:rte_transform',
            ],
        ],
    ],
],
```

This is now:

```
'columns' => [
    'content' => [
        'config' => [
            'type' => 'text',
        ],
    ],
],
'types' => [
    'myType' => [
        'columnsOverrides' => [
            'aField' => [
```

```
                'config' => [
                    'enableRichtext' => true,
                ],
            ],
        ],
    ],
],
```

Impact

Using defaultExtras to enable richtext editor will stop working in TYPO3 v9. An automatic `TCA` migration transfers to the new syntax in TYPO3 v8 and logs deprecations.

Affected Installations

All installations using `defaultExtras` for richtext configuration.

Migration

Remove the defaultExtras line and set `'enableRichtext' => true,` within the config section of the field. This is allowed in `columnsOverrides` for specific record types, too.

Deprecation: #79364 - Deprecate members in PageLayoutController

See Issue #79364

Description

Deprecate the members `\TYPO3\CMS\Backend\Controller\PageLayoutController::edit_record` and `\TYPO3\CMS\Backend\Controller\PageLayoutController::new_unique_uid`.

Impact

Installation of EXT:compatibility7 is required to continue using this members until they are removed in TYPO3 CMS 9.

Affected Installations

Any installation using the mentioned members `\TYPO3\CMS\Backend\Controller\PageLayoutController::edit_` and `\TYPO3\CMS\Backend\Controller\PageLayoutController::new_unique_uid`.

Deprecation: #79440 - TCA Changes

See Issue #79440

Description

The `TCA` on field level has been changed. Nearly all column types are affected. In general, the sub-section `wizards` is gone and replaced by a combination of new `renderType's` and a new set of configuration options. Wizards are now divided into three different kinds:

- `fieldInformation` - Informational HTML, typically displayed between the element label and the element itself.
- `fieldControl` - Icons, typically displayed right next to the element, used to trigger certain actions, for instance to jump to the link view.
- `fieldWizard` - HTML typically shown below the element to enrich the element with further functionality. Example is the rendering of thumbnails below a type=group element.

Other wizards like the "suggest" functionality have been merged into the affected elements itself.

Additionally, the config option `defaultExtras`, which was often set within `columnsOverrides` has been removed. The options were transferred to config options of the elements itself and can be set within `columnsOverrides` directly.

A `TCA` migration transforms old configuration options to new ones and throws descriptive log entries.

The first list covers all former existing wizards and shows where and how the functionality is now placed.

The second list below gives examples from before and after based on `type`, together with detail information on certain configuration options.

Wizard list

Add wizard, edit wizard and list wizard

The three wizards `wizard_add`, `wizard_edit` and `wizard_list` usually used in `type=group` and `type=select` with `renderType=selectMultipleSideBySide` are now default controls of these two elements and just need to be enabled. `options` are optional, the render engine selects a fallback title, a default pid and table name if needed.

Example before:

```
'wizards' => [
    '_VERTICAL' => 1,
    'edit' => [
        'type' => 'popup',
        'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_tca.xlf:be_
↪users.usergroup_edit_title',
        'module' => [
            'name' => 'wizard_edit',
        'popup_onlyOpenIfSelected' => true,
        'icon' => 'actions-open',
        'JSopenParams' => 'width=800,height=600,status=0,menubar=0,scrollbars=1'
    ],
    'add' => [
        'type' => 'script',
        'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_tca.xlf:be_
↪users.usergroup_add_title',
        'icon' => 'actions-add',
        'params' => [
            'table' => 'be_groups',
            'pid' => 0,
            'setValue' => 'prepend'
        ],
        'module' => [
            'name' => 'wizard_add'
```

```
        ]
    ],
    'list' => [
        'type' => 'script',
        'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_tca.xlf:be_
↪users.usergroup_list_title',
        'icon' => 'actions-system-list-open',
        'params' => [
            'table' => 'be_groups',
            'pid' => 0
        ],
        'module' => [
            'name' => 'wizard_list'
        ]
    ]
]
```

Example after:

```
'fieldControl' => [
    'editPopup' => [
        'disabled' => false,
    ],
    'addRecord' => [
        'disabled' => false,
        'options' => [
            'setValue' => 'prepend',
        ],
    'listModule' => [
        'disabled' => false,
    ],
],
```

Color picker

The color picker wizard has been changed to `type=input` element with `renderType=colorpicker`

Example before:

```
'input_34' => [
    'label' => 'input_34',
    'config' => [
        'type' => 'input',
        'wizards' => [
            'colorChoice' => [
               'type' => 'colorbox',
               'title' => 'LLL:EXT:examples/Resources/Private/Language/locallang_db.
↪xlf:tx_examples_haiku.colorPick',
               'module' => [
                  'name' => 'wizard_colorpicker',
               ],
               'JSopenParams' => 'height=600,width=380,status=0,menubar=0,
↪scrollbars=1',
               'exampleImg' => 'EXT:examples/res/images/japanese_garden.jpg',
            ]
        ],
    ],
],
```

Example after:

```
'input_34' => [
    'label' => 'input_34',
    'config' => [
        'type' => 'input',
        'renderType' => 'colorpicker',
    ],
],
```

Table wizard

The table wizard has been embedded in `type=text` element with `renderType=textTable`

Example before:

```
'text_17' => [
    'label' => 'text_17',
    'config' => [
        'type' => 'text',
        'cols' => '40',
        'rows' => '5',
        'wizards' => [
            'table' => [
                'notNewRecords' => 1,
                'type' => 'script',
                'title' => 'LLL:EXT:cms/locallang_ttc.xlf:bodytext.W.table',
                'icon' => 'content-table',
                'module' => [
                    'name' => 'wizard_table'
                ],
                'params' => [
                    'xmlOutput' => 0
                ]
            ],
        ],
    ],
],
```

Example after:

```
'text_17' => [
    'label' => 'text_17',
    'config' => [
        'type' => 'text',
        'renderType' => 'textTable',
        'cols' => '40',
        'rows' => '5',
    ],
],
```

RTE wizard

The RTE wizard that jumps to a full screen view of a text field has been embedded into `EXT:rtehtmlarea` directly and just needs to be turned on. This additionally obsoletes the `defaultExtras=rte_only` setting.

Example before:

```
'rte_1' => [
    'label' => 'rte_1',
    'config' => [
```

```
        'type' => 'text',
        'enableRichtext' => true,
        'RTE' => [
            'notNewRecords' => 1,
            'RTEonly' => 1,
            'type' => 'script',
            'title' => 'LLL:EXT:frontend/Resources/Private/Language/locallang_ttc.
↪xlf:bodytext.W.RTE',
            'icon' => 'actions-wizard-rte',
            'module' => [
                'name' => 'wizard_rte'
            ]
        ],
    ],
],
```

Example after:

```
'rte_1' => [
    'label' => 'rte_1',
    'config' => [
        'type' => 'text',
        'enableRichtext' => true,
        'fieldControl' => [
            'fullScreenRichtext' => [
                'disabled' => false,
            ],
        ],
    ],
],
```

Link browser

The link browser icon wizard has been embedded in new :code:renderType='inputLink' directly. The parameters `blindLinkOptions`, `blindLinkFields` and `allowedExtensions` are now all optional and options of section `fieldControl`.

Example before:

```
'input_29' => [
    'label' => 'input_29 link',
    'config' => [
        'type' => 'input',
        'wizards' => [
            'link' => [
            'type' => 'popup',
            'title' => 'LLL:EXT:frontend/Resources/Private/Language/locallang_ttc.
↪xlf:header_link_formlabel',
            'icon' => 'actions-wizard-link',
            'module' => [
               'name' => 'wizard_link',
            ],
            'JSopenParams' => 'height=800,width=600,status=0,menubar=0,scrollbars=1
↪',
            'params' => [
                'blindLinkOptions' => 'folder',
                'blindLinkFields' => 'class, target',
                'allowedExtensions' => 'jpg',
            ],
        ],
```

```
    ],
],
```

Example after:

```
'input_29' => [
    'label' => 'input_29',
    'config' => [
        'type' => 'input',
        'renderType' => 'inputLink',
        'fieldControl' => [
            'linkPopup' => [
                'options' => [
                    'blindLinkOptions' => 'folder',
                    'blindLinkFields' => 'class, target',
                    'allowedExtensions' => 'jpg',
                ],
            ],
        ],
    ],
],
```

Select wizard

The select wizard has been directly embedded in `type=input` and `type=text` elements, only works with a static list of items and is called `valuePicker`.

Example before:

```
'input_33' => [
    'label' => 'input_33',
    'config' => [
        'type' => 'input',
        'wizards' => [
            'select' => [
                'items' => [
                    [ 'spring', 'Spring', ],
                    [ 'summer', 'Summer', ],
                    [ 'autumn', 'Autumn', ],
                    [ 'winter', 'Winter', ],
                ],
            ],
        ],
    ],
],
```

Example after:

```
'input_33' => [
    'label' => 'input_33',
    'config' => [
        'type' => 'input',
        'valuePicker' => [
            'items' => [
                [ 'spring', 'Spring', ],
                [ 'summer', 'Summer', ],
                [ 'autumn', 'Autumn', ],
                [ 'winter', 'Winter', ],
            ],
        ],
```

```
    ],
],
```

Suggest wizard

The suggest wizard has been directly embedded in `type=group` element and is enabled by default. It can be disabled by setting `hideSuggest=true` in config section and suggest options can be added in `suggestOptions`.

Example before:

```
'group_db_8' => [
    'label' => 'group_db_8',
    'config' => [
        'type' => 'group',
        'internal_type' => 'db',
        'allowed' => 'tx_styleguide_staticdata',
        'wizards' => [
            '_POSITION' => 'top',
                'suggest' => [
                    'type' => 'suggest',
                    'default' => [
                        'pidList' => 42,
                    ],
                ],
            ],
        ],
    ],
],
```

Example after:

```
'group_db_8' => [
    'label' => 'group_db_8',
    'config' => [
        'type' => 'group',
        'internal_type' => 'db',
        'allowed' => 'tx_styleguide_staticdata',
        'suggestOptions' => [
            'default' => [
                'pidList' => 42,
            ]
        ],
    ],
],
```

Slider wizard

The slider wizard has been embedded in `type=text` as option `slider` within the config section.

Example before:

```
'input_30' => [
    'label' => 'input_30',
    'config' => [
        'type' => 'input',
        'size' => 5,
        'eval' => 'trim,int',
        'range' => [
            'lower' => -90,
```

```
            'upper' => 90,
        ],
        'default' => 0,
        'wizards' => [
            'angle' => [
                'type' => 'slider',
                'step' => 10,
                'width' => 200,
            ],
        ],
    ],
],
```

Example after:

```
'input_30' => [
    'label' => 'input_30',
    'config' => [
        'type' => 'input',
        'size' => 5,
        'eval' => 'trim,int',
        'range' => [
            'lower' => -90,
            'upper' => 90,
        ],
        'default' => 0,
        'slider' => [
            'step' => 10,
            'width' => 200,
        ],
    ],
],
```

Type list

type=input

- The wizard `slider` has been directly embedded in this element. The new config option `slider` can be used to configure the slider. The slider appears if the option `slider` exists and is an arary.
- The wizard `select` has been directly embedded in this element. The new config option `valuePicker` has been introduced to configure items of the drop down.
- The four date related `eval` options `date`, `datetime`, `time` and `timesec` have been moved to `renderType=inputDateTime`.
- The wizard `wizard_link` has been removed a field now displays the link wizard by setting `renderType=inputLink` for a `type=input` element.

type=text

- The wizard `select` has been directly embedded in this element. The new config option `valuePicker` has been introduced to configure items of the drop down.
- The wizard `wizard_table` has been given the own `renderType=textTable`.
- The wizard `RTE` has been changed to a `fieldControl` of the richtext element implemented by extension `rtehtmlarea`.
- `defaultExtras=enable-tab` has been moved to config option `enableTabulator`

- `defaultExtrasfixed-font` has been moved to config option `fixedFont`
- `defaultExtras=nowrap` has been moved to config option `wrap=off`

type=select with renderType=selectSingle

- Config option `showIconTable` has been dropped. Showing assigned images below the select field has been migrated to a `fieldWizard` and is disabled by default.
- Config option `selicon_cols` has been dropped without substitution, the render engine now shows as many images in a row as fit into the view.

Example to enable the icon display:

```
'select_single_5' => [
    'label' => 'select_single_5',
    'config' => [
        'type' => 'select',
        'renderType' => 'selectSingle',
        'items' => [
            ['foo 1', 'foo1', 'EXT:styleguide/Resources/Public/Icons/tx_styleguide.
↪svg'],
            ['foo 2', 'foo2', 'EXT:styleguide/Resources/Public/Icons/tx_styleguide.
↪svg'],
        ],
        'fieldWizard' => [
            'selectIcons' => [
                'disabled' => false,
            ],
        ],
    ],
],
```

type=select with renderType=multipleSideBySide

Configuration option changes of the `renderType=multipleSideBySide` element are similar to the `type=group` changes. In detail, the following changes have been applied:

- Config option `selectedListStyle` has been dropped without substitution.
- Wizards `wizard_add`, `wizard_edit` and `wizard_list` have been changed to `fieldControl` and must be enabled per element. All options are still valid, but the system tries to determine sane fallback values if no options are given. For example, the `table` option of `wizard_add` and `wizard_list` fall back to the value of `foreign_table` if not explicitly given.

Example configuration of a multipleSideBySide field before change:

```
'select_multiplesidebyside_6' => [
    'exclude' => 1,
    'label' => 'select_multiplesidebyside_6',
    'config' => [
        'type' => 'select',
        'renderType' => 'selectMultipleSideBySide',
        'foreign_table' => 'tx_styleguide_staticdata',
        'rootLevel' => 1,
        'size' => 5,
        'autoSizeMax' => 20,
        'wizards' => [
            '_VERTICAL' => 1,
            'edit' => [
                'type' => 'popup',
                'title' => 'edit',
```

```
                'module' => [
                    'name' => 'wizard_edit',
                ],
                'icon' => 'actions-open',
                'popup_onlyOpenIfSelected' => 1,
                'JSopenParams' => 'height=350,width=580,status=0,menubar=0,
↪scrollbars=1',
            ],
            'add' => [
                'type' => 'script',
                'title' => 'add',
                'icon' => 'actions-add',
                'module' => [
                    'name' => 'wizard_add',
                ],
                'params' => [
                    'table' => 'tx_styleguide_staticdata',
                    'pid' => '0',
                    'setValue' => 'prepend',
                ],
            ],
            'list' => [
                'type' => 'script',
                'title' => 'list',
                'icon' => 'actions-system-list-open',
                'module' => [
                    'name' => 'wizard_list',
                ],
                'params' => [
                    'table' => 'tx_styleguide_staticdata',
                    'pid' => '0',
                ],
            ],
        ],
    ],
],
```

Example after:

```
'select_multiplesidebyside_6' => [
    'exclude' => 1,
    'label' => 'select_multiplesidebyside_6 wizards',
    'config' => [
        'type' => 'select',
        'renderType' => 'selectMultipleSideBySide',
        'foreign_table' => 'tx_styleguide_staticdata',
        'rootLevel' => 1,
        'size' => 5,
        'autoSizeMax' => 20,
        'fieldControls' => [
            'editPopup' => [
                'disabled' => false,
            ],
            'addRecord' => [
                'disabled' => false,
            ],
            'listModule' => [
                'disabled' => false,
            ],
        ],
    ],
],
```

type=group

This element got most changes in this series. A number of options have been fine-tuned and got better default values:

- Similar to `type=select` with `type=multipleSideBySide`, the three wizards `wizard_add`, `wizard_edit` and `wizard_list` have been changed to `fieldControl` and are disabled by default.
- Config option `selectedListStyle` has been dropped without substitution.
- The suggest wizard has been directly embedded in `type=group` and has been enabled by default for `internal_type=db` elements. It can be disabled with `hideSuggest=true` and options of the `suggest` can be hand over in config option `suggestOptions`.
- The config option `show_thumbs` showed two different things in the past: With `internal_type=db`, it displayed the list of selected records as a table below the element, with `internal_type=file` it rendered thumbnails of selected files and displayed the below the element. `show_thumbs` has been dropped, the functionality has been transferred to `fieldWizard` as `recordsOverview` and `fileThumbnails` respectively and are enabled by default. They can be disabled by setting `disabled=true`.
- The config option `disable_controls` has been obsoleted, single parts of the group element can be disabled by adding `disabled=true` to the according `fieldControl` or `fieldWizard`.

Example of a typical group field before:

```
'group_db_1' => [
    'label' => 'group_db_1',
    'config' => [
        'type' => 'group',
        'internal_type' => 'db',
        'allowed' => 'be_users,be_groups',
        'wizards' => [
            'edit' => [
                'type' => 'popup',
                'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_core.
↪xlf:labels.edit',
                'module' => [
                    'name' => 'wizard_edit',
                ],
                'popup_onlyOpenIfSelected' => 1,
                'icon' => 'actions-open',
                'JSopenParams' => 'height=350,width=580,status=0,menubar=0,
↪scrollbars=1'
            ],
            'add' => [
                'type' => 'script',
                'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_core.
↪xlf:labels.createNewPage',
                'icon' => 'actions-add',
                'params' => [
                    'table' => 'be_users',
                    'pid' => 0,
                    'setValue' => 'append'
                ],
                'module' => [
                'name' => 'wizard_add'
                ],
            ],
            'list' => [
                'type' => 'script',
                'title' => 'LLL:EXT:lang/Resources/Private/Language/locallang_core.
↪xlf:labels.list',
                'icon' => 'actions-system-list-open',
                'params' => [
```

```
                    'table' => 'be_groups',
                    'pid' => '0'
                ],
                'module' => [
                    'name' => 'wizard_list'
                ]
            ]
        ],
    ],
],
```

Example after:

```
'group_db_1' => [
    'label' => 'group_db_1',
    'config' => [
        'type' => 'group',
        'internal_type' => 'db',
        'allowed' => 'be_users,be_groups',
        'fieldControls' => [
            'editPopup' => [
                'disabled' => false,
            ],
            'addRecord' => [
                'disabled' => false,
            ],
            'listModule' => [
                'renderType' => 'listModule',
                'options' => [
                'disabled' => false,
            ],
        ],
    ],
],
```

Disable other parts of type=group:

```
'group_db_1' => [
    'label' => 'group_db_1',
    'config' => [
        'type' => 'group',
        'internal_type' => 'db',
        'allowed' => 'be_users,be_groups',
        'fieldControl' => [
            // Disable element browser icon
            'elementBrowser' => [
                'disabled' => true,
            ],
            // Disable insert from clipboard icon
            'insertClipboard' => [
                'disabled' => true,
            ],
        ],
        'fieldWizard => [
            // Disable button list of allowed tables
            'tableList' => [
                'disabled' => true,
            ],
            // Disable list of allowed file types
            'fileTypeList' => [
                'disabled' => true,
            ],
            // Disable thumbnail view of selected files
```

```
            'fileThumbnails' => [
                'disabled' => true,
            ],
            // Disable table view of selected records
            'recordsOverview' => [
                'disabled' => true,
            ],
            // Disable direct file upload button
            'fileUpload' => [
                'disabled' => true,
            ],
        ],
    ],
],
```

Impact

Using old TCA settings as outlined above will throw a deprecation warnings.

Affected Installations

Most installations are affected by this change.

Migration

An automatic TCA migration transfers from old TCA settings to new ones and throws deprecation log entries with hints which changes should be incorporated. For flex form data structure definitions, the TCA migration is called when opening an according record and logs, too.

Deprecation: #79441 - Deprecate visibility internal caching arrays

See Issue #79441

Description

The following variables have been marked as deprecated in DataHandler since their visibility will change from public to protected or even be replaced by a run-time cache. The documentation states that these are "internal-cache" variables and hence the visibility public is misleading.

```
public $recUpdateAccessCache = [];
public $recInsertAccessCache = [];
public $isRecordInWebMount_Cache = [];
public $isInWebMount_Cache = [];
public $cachedTSconfig = [];
public $pageCache = [];
```

The following variable has been marked as deprecated in the DataHandler since it is not referenced in the class.

```
public $checkWorkspaceCache = [];
```

Impact

These variables should not be accessed in DataHandler from outside the class since their visibility or even implementation will change with TYPO3 v9.

Affected Installations

Extensions using one of the above variables.

Migration

None - since public internal

Deprecation: #79560 - Deprecate ClientUtility::getDeviceType

See Issue #79560

Description

The method `\TYPO3\CMS\Core\Utility\ClientUtility::getDeviceType` is not used and completely outdated and has been marked as deprecated.

Impact

Calling `\TYPO3\CMS\Core\Utility\ClientUtility::getDeviceType` method will trigger a deprecation log entry. Code using this method will work until it is removed in TYPO3 v9.

Affected Installations

Any installation using the mentioned method `\TYPO3\CMS\Core\Utility\ClientUtility::getDeviceType`.

Deprecation: #79622 - Deprecation of CSS Styled Content

See Issue #79622

Description

CSS Styled Content has been the preferred way of rendering content in the frontend for a long time. Fluid Styled Content has been introduced as successor of CSC, but the feature set diverged from the beginning. The lack of flexibility and incomplete feature set in comparison to CSC made it hard to migrate existing instances.

Since TYPO3 CMS 7.6 Fluid-Templates are the defined standard and official recommendation for content rendering. The feature set of FSC is now matching CSC. Both content renderings are now streamlined to be fully compatible with each other. For the period of CMS 8 CSC will share the same capabilities to make a transition as easy as possible. CSC is now deprecated and goes into maintenance mode and will be removed with CMS 9.

Affected Installations

All installations that still use or rely on the content rendering of `css_styled_content`.

Migration

Create a custom content rendering definition or switch to a maintained one like `fluid_styled_content`.

2.3.4 Important

Important: #75400 - New DataHandler command 'copyToLanguage'

See Issue #75400

Description

A new DataHandler command `copyToLanguage` has been introduced. It behaves like `localize` command (both record and child records are copied to a given language), but does not set `transOrigPointerField` fields (e.g. l10n_parent).

The `copyToLanguage` command should be used when localizing records in "Free Mode". This command is used when localizing content elements using the translation wizard's "Copy" strategy.

The `localize` DataHandler command should be used when translating records in "Connected Mode" (strict translation of records from the default language). This command is used when selecting "Translate" strategy in content elements translation wizard.

Important: #78899 - displayCond strict parsing

See Issue #78899

Description

The parser handling `displayCond` in `TCA` fields is now strict and throws exceptions if the documented condition syntax is not valid or if referenced fields are not found. This should help debugging faulty conditions a lot.

Important: #79005 - Included missing support for persistent connection in Redis cache backend

See Issue #79005

Description

phpredis has support for persistent connections, but until now the Redis cache backend had the regular connect call hard-coded. For unknown reasons this is the only connection setting missing from the Redis cache backend but has now been implemented.

The configuration setting is named `persistentConnection`. It is an optional boolean option.

For other configuration options see https://docs.typo3.org/typo3cms/CoreApiReference/CachingFramework/FrontendsBackends/Index.html#redis-backend

Impact

None. Non-persistent connections remain the default.

Important: #79119 - Removed PageRepository->versioningPreview_where_hid_del property

See Issue #79119

Description

The public property `$versioningPreview_where_hid_del` inside the PHP class `PageRepository` was not used anymore due to the refactoring of the database queries based on Doctrine DBAL and has been removed.

Important: #79221 - Use $ instead of TYPO3.jQuery

See Issue #79221

Description

The TYPO3 Core uses jQuery in the TYPO3 Backend with a default namespace of `TYPO3.jQuery` and `jQuery` in the global object namespace, and is now also available as global `$` when no other namespace is given. This was not possible before due to conflicts with prototype.js.

If using the shipped jQuery code in the Frontend explicitly via ts:`page.javascriptLibs.jQuery.noConflict.namespac = default` then the global `$` is also available in frontend scripts.

2.4 8.5 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.4.1 Breaking Changes

Breaking: #73016 - Renaming of Clipboard->printContentFromTab to getContentFromTab

See Issue #73016

Description

During the fluidification of the clipboard, it became obvious that the method `printContentFromTab()` doesn't describe the method correctly anymore. So it has been renamed to `getContentFromTab()`.

Impact

This is a public method, so it could be that some unknown extension calls the old function. But as no TER extension or the core itself calls the method, no deprecation is needed.

Affected Installations

Every extension that calls `Clipboard->printContentFromTab()`.

Migration

Change the call from `Clipboard->printContentFromTab()` to `Clipboard->getContentFromTab()`.

Breaking: #78002 - Enforce cHash argument for Extbase actions

See Issue #78002

Description

URIs to Extbase actions now need a valid cHash per default. This is required for both cached and uncached actions. The behavior can be disabled for all actions using the feature switch `requireCHashArgumentForActionArguments`.

Impact

All generated links to Extbase actions without having a valid cHash will fail.

Affected Installations

All generated links to Extbase actions that explicitly disabled the cHash are affected - like `<f:link.action action="..." noCacheHash="1"/>`

Migration

Either one of the following:

- ensure to use a valid cHash, e.g. by removing the `noCacheHash="1"` argument from link view-helpers
- disable the `feature.requireCHashArgumentForActionArguments` setting for the particular extension

Breaking: #78191 - Remove support for transForeignTable in TCA

See Issue #78191

Description

TCA allowed the definition of separate tables to hold localized and translated records. The property names used for that were `transForeignTable` (basically pointed to table `pages_language_overlay`) and `transOrigPointerTable` (basically pointed back to table `pages`). The mentioned two pages tables are the only tables that make use of this feature in the TYPO3 core.

To overcome special handling and to combine `pages_language_overlay` with `pages` at a later step, the configured table names have been replaced with hardcoded table names.

Impact

Modifications concerning the following two TCA control properties won't have any effect anymore:

- `$TCA[<tableName>]['ctrl']['transForeignTable']`
- `$TCA[<tableName>]['ctrl']['transOrigPointerTable']`

Affected Installations

All sites using localizations and translations for page hierarchies.

Migration

No special actions are required if just the core defaults are used. Special adjustments concerning the mentioned TCA properties should be verified and hard-coded for the time being.

- `$TCA['pages']['ctrl']['transForeignTable']`, use value `pages_language_overlay` directly
- `$TCA['pages_language_overlay']['ctrl']['transOrigPointerTable']`, use value `pages` directly

Breaking: #78383 - pages, tt_content, sys_file_metadata have been removed from defaultCategorizedTables

See Issue #78383

Description

The tables `pages`, `tt_content` and `sys_file_metadata` have been removed from `defaultCategorizedTables`. For these tables the core API `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::makeCategorizable` would be executed to define a common position of the categories field.

Impact

It is no longer possible to remove the category field for these tables by reset the configuration.

Affected Installations

Any TYPO3 instance that reset the configuration value.

Migration

None.

Use PageTSConfig to disable the field:

```
TCEFORM.pages.categories.disabled = 1
TCEFORM.tt_content.categories.disabled = 1
TCEFORM.sys_file_metadata.categories.disabled = 1
```

Breaking: #78384 - Frontend ignores TCA in ext_tables.php

See Issue #78384

Description

Frontend requests no longer load `ext_tables.php` in requests. The only exception is if a backend user is logged in to the backend at the same time to initialize the admin panel or frontend editing.

Impact

Since especially a not yet cached frontend call relies on initialized `$GLOBALS['TCA']`, changes to `TCA` done within `ext_tables.php` are now ignored and may fail.

Affected Installations

Extensions that still set, add or remove settings in `$GLOBALS['TCA']` need to be adapted. The install tool provides test "TCA ext_tables check" to find such extensions.

Migration

In `ext_tables.php` neither writing directly to `$GLOBALS['TCA']` and `$TCA` is allowed, nor writing indirectly via `ExtensionManagementUtility` methods. An example list of calls and their new positions:

- `$GLOBALS['TCA']['someTable'] = `: A full table `TCA is added. This must be moved to `Configuration/TCA/someTable.php`, see `ext:sys_note` as example.
- `ExtensionManagementUtility::addStaticFile()`: A static file is registered in `sys_template`. Add this to `Configuration/TCA/Overrides/sys_template.php`, see `ext:rtehtmlarea` as example.
- `ExtensionManagementUtility::addTCAcolumns()`: Columns are added to a table. Add this to `Configuration/TCA/Overrides/<table>.php`, see `ext:felogin` as example.
- `ExtensionManagementUtility::addToAllTCAtypes()`: Fields are added to types. Add this to `Configuration/TCA/Overrides/<table>.php`, see `ext:felogin` as example.
- `ExtensionManagementUtility::addPiFlexFormValue()`: A new flex from in `tt_content` is registered. Add this to `Configuration/TCA/Overrides/tt_content.php`, see `ext:felogin` as example.
- `ExtensionUtility::registerPlugin()` and `ExtensionManagementUtility::addPlugin`: A new type item is added to the `tt_content` table. Add this to `Configuration/TCA/Overrides/tt_content.php`.

Breaking: #78417 - Lowlevel DeletedRecordsCommand parameters changed

See Issue #78417

Description

The DeletedRecordsCommand is now using Symfony Console. The new command behaves like the old one, but allows using certain parameters and is located under the following path now:

```
./typo3/sysext/core/bin/typo3 cleanup:deletedrecords
```

The following options can be set `--dry-run` to only show the deleted records `-v` and `-vv` to show additional information `--pid=23` or `-p=23` to only find and delete records below page ID 23 (otherwise "0" is taken) `--depth=4` or `-d=4` to only delete recursively until a certain page tree level.

The PHP class `TYPO3\CMS\Lowlevel\DeletedRecordsCommand` has been removed.

Impact

Calling `typo3/cli_dispatch lowlevel cleaner deleted` will not work anymore.

Calling the PHP class results in a fatal PHP error.

Affected Installations

Any TYPO3 installation using the old CLI command or the related PHP class.

Migration

Use the new CLI command as shown above.

Breaking: #78439 - Lowlevel FlexForm Cleaning parameters changed

See Issue #78439

Description

The CleanFlexFormsRecordsCommand is now using Symfony Console. The new command behaves like the old functionality, but uses certain different parameters. It can now be called with the following CLI command:

```
./typo3/sysext/core/bin/typo3 cleanup:flexforms
```

The following options can be set `--dry-run` to only show the deleted records `-v` and `-vv` to show additional information `--pid=23` or `-p=23` to only find and clean up records with FlexForm XMLs below page ID 23 (otherwise "0" is taken) `--depth=4` or `-d=4` to only clean recursively until a certain page tree level.

The PHP class `TYPO3\CMS\Lowlevel\CleanFlexformCommand` has been removed.

Impact

Calling `typo3/cli_dispatch.phpsh lowlevel cleaner cleanflexform` will not work anymore.

Calling the PHP class results in a fatal PHP error.

Affected Installations

Any TYPO3 installation using the previously command callable via `cli_dispatch.phpsh` or the related PHP class.

Migration

Use the new CLI command as shown above.

Breaking: #78468 - Remove ExtDirect from EXT:workspaces

See Issue #78468

Description

To remove ExtJS the ExtDirect component has been removed too. A new class `TYPO3\CMS\Workspaces\Controller\AjaxDispatcher` has been added to implement the ExtDirect router functionality. This class is callable by a new AJAX route with the name `workspace_dispatch`.

Impact

The following classes have been moved:

- EXT:workspaces/Classes/ExtDirect/AbstractHandler.php => EXT:workspaces/Classes/Controller/Remote/AbstractHandler.php
- EXT:workspaces/Classes/ExtDirect/ActionHandler.php => EXT:workspaces/Classes/Controller/Remote/ActionHandler.php
- EXT:workspaces/Classes/ExtDirect/MassActionHandler.php => EXT:workspaces/Classes/Controller/Remote/MassActionHand
- EXT:workspaces/Classes/ExtDirect/ExtDirectServer.php => EXT:workspaces/Classes/Controller/Remote/RemoteServer.php

Affected Installations

Any TYPO3 installation using the previously classes.

Migration

Use the new classes as mentioned above.

Breaking: #78520 - Lowlevel Orphan Records Cleaning parameters changed

See Issue #78520

Description

The OrphanRecordsCommand is now using Symfony Console. The new command behaves like the old functionality, but uses certain different parameters. It can now be called with the following CLI command:

```
./typo3/sysext/core/bin/typo3 cleanup:orphanrecords
```

The following options can be set `--dry-run` to only show the orphaned records `-v` and `-vv` to show additional information

The PHP class `TYPO3\CMS\Lowlevel\OrphanRecordsCommand` has been removed.

Impact

Calling `typo3/cli_dispatch.phpsh lowlevel cleaner orphan_records` will not work anymore.

Calling the PHP class results in a fatal PHP error.

Affected Installations

Any TYPO3 installation using the previously command callable via `cli_dispatch.phpsh` or the related PHP class.

Migration

Use the new CLI command as shown above.

Breaking: #78521 - Drop unused JavaScript from backend.js

See Issue #78521

Description

The following JavaScript methods related to ExtJS have been removed from the Backend main frame as defined in the main `backend.js` file.

`TYPO3._instances` `TYPO3.addInstance` `TYPO3.getInstance` `TYPO3.helpers.split`

Impact

Any call to one of the above mentioned methods will result in a JavaScript error.

Affected Installations

Any installation that uses one of the methods mentioned above.

Breaking: #78522 - Removed backend user option debugInWindow

See Issue #78522

Description

The backend user option `debugInWindow` was unused in the core and has been removed, as the option of opening the debug information in a window was migrated already.

Impact

The setting is not available anymore in JavaScript under `TYPO3.configuration`.

Affected Installations

Any installation that uses the removed backend user option `debugInWindow`.

Breaking: #78525 - Removed unused configuration options for JavaScript

See Issue #78525

Description

Removed all options that are not used anymore from TYPO3.configuration in JavaScript context.

`TYPO3.configuration.moduleMenuWidth` `TYPO3.configuration.topBarHeight`

Impact

Both settings are not available anymore in JavaScript under `TYPO3.configuration`.

Affected Installations

Any installation that uses one of the mentioned options.

Migration

No migration.

Breaking: #78549 - Override New Page Creation Wizard via page TSconfig

See Issue #78549

Description

In the past it was possible to override the "New Page Creation Wizard" via custom scripts when using page TSconfig via `mod.web_list.newPageWiz.overrideWithExtension = myextension` to define an extension, which then needed a file placed under `mod1/index.php`. The script was then called with certain parameters instead of the wizard.

The new way of handling entry-points and custom scripts is now built via modules and routes. The former option `mod.web_list.newPageWiz.overrideWithExtension` has been removed and a new option `mod.newPageWizard.override` has been introduced instead. Instead of setting the option to a certain extension key, a custom module or route has to be specified.

Example:

```
mod.newPageWizard.override = my_custom_module
```

Impact

Using the old TSconfig option `mod.web_list.newPageWiz.overrideWithExtension` has no effect anymore and will fallback to the regular new page creation wizard provided by the TYPO3 Core.

Affected Installations

Any installation using this option with extensions providing custom New Page Wizards, e.g. EXT:templavoila.

Migration

The extension providing the script must be changed to register a route or module and set the TSconfig option to the route identifier, instead of a raw PHP script. Any usages in TSconfig need to be adapted to use the new TSconfig option.

Breaking: #78552 - Lowlevel LostFilesCommand parameters changed

See Issue #78552

Description

The existing CLI command within EXT:lowlevel for detecting and removing files within uploads/ which are not referenced by TYPO3 has been migrated to a Symfony Console command.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner lost_files` is now available via `./typo3/sysext/core/bin/typo3 cleanup:lostfiles` and allows the following CLI options to be set:

`--update-refindex` - updates the reference index before scanning for lost files. If not set, the user is asked if the task should be run `--exclude=uploads/mypics/,uploads/psa` - a list of paths of files to exclude within uploads/ `--dry-run` - do not delete the files but only list the files that are not connected to the TYPO3 system anymore

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\LostFilesCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner lost_files` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding and deleting lost files.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

Breaking: #78577 - Lowlevel MissingFilesCommand parameters changed

See Issue #78577

Description

The existing CLI command within EXT:lowlevel for showing missing files that are referenced by TYPO3 records has been migrated to a Symfony Console command.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner missing_files` is now available via `./typo3/sysext/core/bin/typo3 cleanup:missingfiles` and allows the following CLI options to be set:

`--update-refindex` - updates the reference index before scanning for missing files. If not set, the user is asked if the task should be run `--dry-run` - do not delete the references, files but only list the files that are missing but connected to the TYPO3 system

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\MissingFilesCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner missing_files` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding missing files in relations.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

Breaking: #78581 - FlexFormTools public properties dropped

See Issue #78581

Description

Two public properties have been dropped from PHP class `FlexFormTools`:

- `FlexFormTools->traverseFlexFormXMLData_DS`
- `FlexFormTools->traverseFlexFormXMLData_Data`

Impact

Accessing those properties will throw a warning.

Affected Installations

Extensions that access these properties. These two properties were of little use from an extensions point of view, it is very unlikely this actually breaks an extension.

Migration

No migration possible.

Breaking: #78581 - FormEngine TcaFlexFetch data provider removed

See Issue #78581

Description

The FormEngine data provider `TcaFlexFetch` has been merged into data provider `TcaFlexPrepare`.

Impact

If own registered data providers are declared to "depends" or "before" `TcaFlexFetch`, the `DependencyResolver` will be unable to find it and throws an exception or sorts the own data provider to an ambiguous place.

Affected Installations

An installation is only affected in the relatively unlikely case that an own data provider declared a dependency to `TcaFlexFetch`.

Migration

Move the dependency over to `TcaFlexPrepare`: The two data providers have been merged into one, it should be save for any data provider to hook in before or after `TcaFlexPrepare` instead. There is a little additional flex form processing in `TcaFlexPrepare`, so the flex structure might be a bit different. Have a look at methods `removeTceFormsArrayKeyFromDataStructureElements()` and `migrateFlexformTcaDataStructureElements()` for details.

Breaking: #78581 - Hook getFlexFormDSClass no longer called

See Issue #78581

Description

With the deprecation of `BackendUtility::getFlexFormDS()` the hook `getFlexFormDSClass` is no longer called and there is no substitution available.

Impact

The hook is no longer called and flex form field manipulation by extensions does not happen anymore.

Affected Installations

Extensions that extension flex form data structure definitions and use the hook `getFlexFormDSClass` for that purpose.

Migration

Method `BackendUtility::getFlexFormDS()` has been split into the methods `FlexFormTools->getDataStructureIdentifier()` and `FlexFormTools->parseDataStructureByIdentifier()`.

Those two new methods now provide four hooks to allow manipulation of the flex form data structure location and parsing. The methods and hooks are documented well, read their description for a deeper insight on which combination is the correct one for a specific extension need.

The new hooks are very powerful and must be used with special care to be as future proof as possible.

Since the old hook is used by some widespread extensions, the core team prepared a transition for some of them beforehand:

- EXT:news: The extension used the old hook to only remove a couple of fields from the flex form definition. This has been moved over to a "FormEngine" data provider: news
- EXT:flux: Flux implements a completely own way of locating and pointing to the flex form data structure that is needed in a specific context. The default core resolving does not work here. Flux now implements the hooks `getDataStructureIdentifierPreProcess` and `parseDataStructureByIdentifierPreProcess` to specify an own "identifier" syntax and to resolve that syntax to a data structure later: flux
- EXT:gridelements: Similar to flux, gridelements has a own logic to choose which specific data structure should be used. However, the data structures are located in database row fields, so the "record" syntax of the core can be re-used to refer to those. gridelements uses the hook `getDataStructureIdentifierPreProcess` together with a small implementation in `parseDataStructureByIdentifierPreProcess` for a fallback scenario: gridelements
- EXT:powermail: Powermail allows extending and changing existing flex form data structure definition depending on page TS. To do that, it now implements hook `getDataStructureIdentifierPostProcess` to add the needed pid to the existing identifier, and then implements hook `parseDataStructureByIdentifierPostProcess` to manipulate the resolved data structure: powermail

Breaking: #78623 - Lowlevel MissingRelationsCommand parameters changed

See Issue #78623

Description

The existing CLI command within EXT:lowlevel for showing relations and soft-references to non-existing records, offline versions and records marked as deleted has been migrated to a Symfony Console command.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner missing_relations` is now available via `./typo3/sysext/core/bin/typo3 cleanup:missingrelations` and allows the following CLI options to be set:

`--update-refindex` - updates the reference index before scanning for missing files. If not set, the user is asked if the task should be run `--dry-run` - do not delete the references but only list the references that are missing but connected to the TYPO3 system

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\MissingRelationsCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner missing_relations` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding relations pointing to deleted, offline versions or non-existing records.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

Breaking: #78627 - Lowlevel MissingRelationsCommand parameters changed

See Issue #78627

Description

The existing CLI command within EXT:lowlevel for showing files within uploads/ that are used by records twice (non-FAL) has been migrated to a Symfony Console command.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner double_files` is now available via `./typo3/sysext/core/bin/typo3 cleanup:multiplereferencedfiles` and allows the following CLI options to be set:

`--update-refindex` - updates the reference index before scanning for multiple-referenced files. If not set, the user is asked if the task should be run `--dry-run` - do not copy the files to single-reference them, but only list the references and files.

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\DoubleFilesCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner double_files` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding files with two records pointing to them.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

Breaking: #78759 - Fluidification of EditFileController

See Issue #78759

Description

While moving all HTML from PHP code to an own Fluid template the HTML string given to the hook after compiling the output is different now.

Impact

The HTML string given to the hook after compiling the output now contains the closing form tag (</form>).

Affected Installations

All installations that append text to the HTML code in the hook after compiling the output.

Migration

The hook code has to be changed to insert additional code before the closing form tag.

Breaking: #78855 - Remove obsolete sys_action translations

See Issue #78855

Description

These translations have been removed from `EXT:sys_action/Resources/Private/Language/locallang.xlf`:

- action_BEu_hidden
- action_BEu_username
- action_BEu_password
- action_BEu_realName
- action_BEu_email
- action_BEu_usergroups

These translations have been removed from `EXT:sys_action/Resources/Private/Language/locallang_tca.xlf`:

- tx_sys_action

Impact

Integrations / third party Extensions using these translation-keys within the mentioned files will output empty strings.

Affected Installations

Installations that use the removed translations in third party code.

Migration

Create your own `locallang.xlf` file and add the required translations.

Breaking: #78895 - Lowlevel RteImagesCommand parameters changed

See Issue #78895

Description

The existing CLI command within EXT:lowlevel for detecting and removing RTE files within uploads/ which are not referenced by TYPO3 has been migrated to a Symfony Console command. The same command is also used to copy RTE images which are used multiple times on multiple references, to be only used once.

The command previously available via `./typo3/cli_dispatch.phpsh lowlevel_cleaner rte_images` is now available via `./typo3/sysext/core/bin/typo3 cleanup:rteimages` and allows the following CLI options to be set:

`--update-refindex` - updates the reference index before scanning for lost files. If not set, the user is asked if the task should be run `--dry-run` - do not copy / delete the files but only list the files that are not wrongly connected or not connected at all.

The PHP class of the old CLI command `TYPO3\CMS\Lowlevel\RteImagesCommand` has been removed.

Impact

Calling the old CLI command `./typo3/cli_dispatch.phpsh lowlevel_cleaner rte_images` will result in an error message.

Affected Installations

Any TYPO3 instances using the lowlevel cleaner for finding RTE image files.

Migration

Update the CLI call on your servers to the new command line and available options as shown above.

2.4.2 Features

Feature: #29399 - OptionViewHelper and OptgroupViewHelper for use with SelectViewHelper

See Issue #29399

Description

Allows manually definition of all options and optgroups for the `f:form.select` parent field as tag contents of the select field. The added ViewHelpers are TagBasedViewHelpers which means they support all standard HTML attributes.

Note that while tag content rendering is now supported, it is **STILL** not possible to create `<option>` tags manually - you **HAVE** to use the form fields!

Example:

```
<f:form.select name="myproperty">
        <f:form.select.option value="1">Option one</f:form.select.option>
        <f:form.select.option value="2">Option two</f:form.select.option>
        <f:form.select.optgroup>
                <f:form.select.option value="3">Grouped option one</f:form.select.
↪option>
                <f:form.select.option value="4">Grouped option twi</f:form.select.
↪option>
        </f:form.select.optgroup>
</f:form.select>
```

Impact

- Adds two new ViewHelpers
- Changes `SelectViewHelper` to allow tag content (but not manual options created without using `f:form.select.*`)

Feature: #52286 - Add option to "system status updates" report-job to send all tests

See Issue #52286

Description

Sometimes it is useful to also get every test in the "System Status Updates (reports)" via email.

A checkbox was added to the job configuration for the decision to get a mail only if the system has WARNING or ERROR events, or just get a mail for everything. If the checkbox is not set (default) it works like before, including WARNING and ERROR events only.

Impact

If the checkbox `Notification for all type of status,not only warning and error` is checked, then the `System Status Update (reports)` contains all type of notifications.

Feature: #58637 - Purge language packs in language module

See Issue #58637

Description

The language module in the backend offers the possibility to activate and deactivate language packs. If deactivating a language pack that previously had been loaded, the data stays in `typo3conf/l10n/<locale>/`. A remove button has been added to the actions. With the remove action the language is disabled and the data is removed from the `typo3conf/l10n/<locale>/` directory.

Impact

The language data can now be removed from the installation file system using the backend user interface.

Feature: #67909 - Add hook to DataHandler - localize - translateToMessage

See Issue #67909

Description

By introducing a new hook to the `localize()` function (the `translateToMessage` part in particular) you are now able to use external translation services and speed-up translation of the content and even add a custom transliteration function that would handle various content transformations.

Impact

A new hook is available at:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_tcemain.php']['
↪'processTranslateToClass']
```

Implement it for example as follows:

```
class YourHookClass {
        public function processTranslateTo_copyAction(&$content, $lang,
↪$dataHandler) {
                // Do something with content (translate, transliterate etc)
        }
}
```

Feature: #73626 - numberOfResults should be configurable and report overflow

See Issue #73626

Description

Adds possibility to overwrite the maximum number of Indexed Search results with TypoScript which previously was limited to 100.

The TypoScript setting `plugin.tx_indexedsearch.settings.blind.numberOfResults` now became a list of values. If the number of results sent in the request does not match any value of this list, the first value will be used to protect against DoS attacks.

Values from `plugin.tx_indexedsearch.settings.blind.numberOfResults` are used as the options in the selectbox in advanced search mode. To keep backward compatibility the default values are 10, 25, 50 and 100.

Impact

The TypoScript setting `plugin.tx_indexedsearch.settings.blind.numberOfResults` can be now a list of available number of results. Because of that it is possible to overwrite the list of values displayed in the advanced search mode. The first value from the list will be used as default.

Feature: #76085 - Add fluid debug information to admin panel

See Issue #76085

Description

A new setting in the admin panel (Preview > Show fluid debug output) enables showing fluid debug output. If the checkbox is enabled, the path to the template file of a partial and the name of a section will be shown in the frontend directly above the markup. With this feature an integrator can easily find the correct template and section.

Impact

Activating this option can break the output in frontend or may result in unexpected behavior.

Feature: #77757 - Enable rechecking whether an UpdateWizard should run

See Issue #77757

Description

It is now possible to reset the upgrade wizards marked as done. In Install Tool you will find a list of wizards that have been marked as done, additionally with a checkbox for each to reset their status. Then the wizard will test whether it needs to be executed again.

Feature: #77910 - EXT:form - introduce new form framework

See Issue #77910

Description

A flexible framework for building forms has been integrated. It replaces the legacy 'form wizard' based on ExtJS and the depending frontend rendering system.

The new backend 'form editor' relies on vanilla JS and jQuery. Different JS patterns have been applied to ensure a modern architecture, high flexibility and extensibility.

A new backend module lists all existing forms and allows the creation of new ones. The 'mailform' content element has been reworked. It lists available forms and enables the backend editor to override certain settings, e.g. 'finisher' settings (formerly known as 'postProcessors').

Until now it was not possible to customize and extend the 'form editor'. To allow the registration of new finishers, validators and pre-defined form elements a lot of architectural changes would have been necessary. After a long conceptional phase the team decided to remove the former code base, backport the 'form' package of the Flow project and improve the given ideas and concepts. The result is a new form extension. A lot of code received major improvements and tons of additional features have been integrated.

The list of features is long and impressive. The documentation will explain the ideas, concept and architecture as well as the functionality in detail. The following list names some of them:

- YAML as configuration and description language including inheritances and overrides.
- File-based configuration.
- All JavaScript components of the form wizard (and the wizard itself) can be replaced or extended.
- Own PHP renderers for form and/or form elements are possible.
- Create entire forms via API.
- Create conditions for form elements and validators programmatically.
- Create 'prototypes' and use them as boilerplate.
- Create new form elements and use them in the wizard.
- Uploads are handled as FAL objects.
- Ships with a bunch of built-in finishers, like email, redirect, save to database.
- Create own finishers. Override those in the content element.
- Create and apply own validators.
- Multi language support.
- Multi step support.
- Multiple forms per page.
- Built-in spam protection (honeypot).

Impact

Happy little wizard.

Feature: #78002 - Enforce cHash argument for Extbase actions

See Issue #78002

Description

`TypoScriptFrontendController::reqCHash()` is now called for Extbase frontend plugin actions just like they are usually called for AbstractPlugin. This provides a more reliable page caching behavior by default and with zero configuration for extension authors.

With the feature switch `requireCHashArgumentForActionArguments` this behavior can be disabled, which could be useful, if all actions in a plugin are uncached or one wants to manually control the cHash behavior.

Impact

The enforcing of a cHash results in a 404, if plugin arguments are present but cHash is not, which would also happen if the plugin arguments were added to `cHashRequiredParameters` configuration.

Feature: #78103 - Add missing information status for addSystemMessage

See Issue #78103

Description

Adds the possibility to pass and set status parameter `TYPO3\CMS\Backend\Toolbar\Enumeration\InformationStatu` through `addSystemInformation()` in `SystemInformationToolbarItem`.

Impact

All system information added by `addSystemInformation()` will now pass `InformationStatus::STATUS_NOTICE` as default value.

Feature: #78116 - Extbase support for Doctrine's native DBAL Statement and QueryBuilder

See Issue #78116

Description

With the change to Doctrine DBAL Extbase's direct query functionality also supports `QueryBuilder` objects and instances of `\Doctrine\DBAL\Statement` as prepared statements instead of only `\TYPO3\CMS\Core\Database\PreparedStatement`.

The following example could happen inside any Extbase Repository using native Doctrine DBAL statements:

```
$connection = $this->objectManager->get(ConnectionPool::class)->
↪getConnectionForTable('mytable');
$statement = $this->objectManager->get(
   \Doctrine\DBAL\Statement::class
   'SELECT * FROM mytable WHERE uid=? OR title=?',
   $connection
);

$query = $this->createQuery();
$query->statement($statement, [$uid, $title]);
```

The following example shows the usage with the QueryBuilder object:

```
$queryBuilder = $this->objectManager->get(ConnectionPool::class)->
↪getQueryBuilderForTable('mytable');

... do the SQL query with the query builder.

$query = $this->createQuery();
$query->statement($queryBuilder);
```

Feature: #78384 - Check ext tables TCA changes in install tool

See Issue #78384

Description

The install tool has a new feature to check extensions for `ext_tables.php` files that still change the global `TCA` array.

Impact

Changing the global `TCA` array in `ext_tables.php` is not allowed and can lead to failing or incomplete frontend requests. The feature helps to find affected, loaded extensions.

Feature: #78415 - Global Fluid ViewHelper namespaces moved to TYPO3 configuration

See Issue #78415

Description

By storing Fluid's namespaces in `$GLOBALS['TYPO3_CONF_VARS']['SYS']['fluid']['namespaces']` we can allow adding or extending the global namespaces from third party packages in for example `ext_localconf.php` or by simply specifying the namespace arrays in `LocalConfiguration.php`.

In terms of performance there is nearly zero impact but in terms of flexibility this should provide the ultimate way to manage global namespaces as configuration; something that currently is only possible by implementing custom ViewHelperResolvers.

Impact

- Site administrators and third party ViewHelper packages will be able to manipulate the global namespace `f:` in configuration

- Third party ViewHelper packages will be able to register new global namespaces
- Template developers can use such global namespaces without first importing them and can use them in all Fluid templates regardless of context.

Feature: #78523 - Suggest wizard provides option to define ordering of results

See Issue #78523

Description

It is now possible to set the ordering of results delivered by the suggest wizard.

The new option is called php:`orderBy => 'somefield ASC'` and can hold the usual SQL order-by definition.

Example TCA configuration for `EXT:news` suggest wizard returning the related articles sorted by datetime:

```
'config' => [
    'type' => 'group',
    'internal_type' => 'db',
    'allowed' => 'tx_news_domain_model_news',
    'foreign_table' => 'tx_news_domain_model_news',
    'MM_opposite_field' => 'related_from',
    'size' => 5,
    'minitems' => 0,
    'maxitems' => 100,
    'MM' => 'tx_news_domain_model_news_related_mm',
    'wizards' => [
        'suggest' => [
            'type' => 'suggest',
            'default' => [
                'searchWholePhrase' => true,
                'addWhere' => ' AND tx_news_domain_model_news.uid != ###THIS_UID###
↪',
                'orderBy => 'datetime DESC',
            ]
        ],
    ],
]
```

Feature: #78575 - Enumeration constants don't provide their names

See Issue #78575

Description

Requesting the name of Enumeration constants has been introduced.

There are two methods you can use: `MyEnumerationClass::getName($value);` will return the name exactly as it is. Usually this will be all uppercase and underscores. `MyEnumerationClass::getHumanReadableName($value);` replaces underscores with spaces and turns all words into lowercase with the first letter uppercase.

Impact

You can use the constant names as part of your application more easily.

Feature: #78672 - Introduce fluid data processor for menus

See Issue #78672

Description

This menu processor utilizes HMENU to generate a json encoded menu string that will be decoded again and assigned to FLUIDTEMPLATE as a variable. Additional DataProcessing is supported and will be applied to each record.

Options: `as` The variable to be used within the result `levels` Number of levels of the menu `expandAll` If false, submenus will only render if the parent page is active `includeSpacer` If true, the doctype "Spacer" will be included in the menu `titleField` Field that should be used for the title

See HMENU docs for more options. https://docs.typo3.org/typo3cms/TyposcriptReference/ContentObjects/Hmenu/Index.html

Example TypoScript configuration:

```
10 = TYPO3\CMS\Frontend\DataProcessing\MenuProcessor
10 {
   special = list
   special.value.field = pages
   levels = 7
   as = menu
   expandAll = 1
   includeSpacer = 1
   titleField = nav_title // title
   dataProcessing {
      10 = TYPO3\CMS\Frontend\DataProcessing\FilesProcessor
      10 {
         references.fieldName = media
      }
   }
}
```

Feature: #78842 - Let FLUIDTEMPLATE mimic an actual extbase web request

See Issue #78842

Description

Adds the possibility to let the FLUIDTEMPLATE content element mimic an actual extbase web request. This makes it possible to access submitted data like in extbase with `->controllerContext->getRequest()->getArguments()`

Impact

Data which was submitted through a `FLUIDTEMPLATE` content element are now available within

```
$view->getRenderingContext()
     ->getControllerContext()
     ->getRequest()
     ->getArguments()
```

Affected Installations

Any installation which use the `FLUIDTEMPLATE` content element which are initialized with the following settings:

```
extbase.pluginName
extbase.controllerExtensionName
extbase.controllerName
extbase.controllerActionName
```

2.4.3 Deprecation

Deprecation: #57385 - Deprecate parameter $caseSensitive of Extbase Query->like comparison

See Issue #57385

Description

The argument `$caseSensitive` of the method `Query->like` has been marked as deprecated.

Impact

Using the argument will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using custom calls to `Query->like` using the mentioned argument.

Migration

For MySQL change the collation of the queried field to be stored in a case sensitive fashion. This requires using a collation with a suffix of `_cs` for the field or table. Alternatively a binary column type can be used. Both solutions will ensure the field will be queried in a case sensitive fashion.

Deprecation: #77296 - Deprecate public member parentMenuArr in AbstractMenuContentObject

See Issue #77296

Description

The previously undefined member `parentMenuArr` has been added as public member and marked as deprecated.

Impact

The parentMenuArr will be publicly accessible until it is changed to protected in TYPO3 v9.

Affected Installations

Instances that have menus with sublevels and using this member in the itemArrayProcFunc.

Migration

Use the provided API function `getParentMenuArr()` to get the parentMenuArr instead. This method always returns an array.

If you need the direct parent menuitem of the current sublevel use `getParentMenuItem()` method.

Deprecation: #77524 - Deprecated method fileResource of ContentObjectRenderer

See Issue #77524

Description

The method `ContentObjectRenderer::fileResource()` has been marked as deprecated.

Impact

Using the mentioned method will trigger a deprecation log entry.

Affected Installations

Instances that use the method.

Migration

Migrate your code to use `file_get_contents`. Use a call to `$GLOBALS['TSFE']->tmpl->getFileName($fileName)` for substituting strings like `EXT`.

Deprecation: #77732 - Deprecate methods of Extbase's ArrayUtility

See Issue #77732

Description

The class `\TYPO3\CMS\Extbase\Utility\ArrayUtility` has been marked as deprecated.

Impact

Calling any of the methods within the static class will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation calling the methods of that PHP class.

Migration

A migration is available for the following methods:

- `integerExplode`: Use `GeneralUtility::intExplode`
- `trimExplode`: Use `GeneralUtility::trimExplode`
- `arrayMergeRecursiveOverrule`: Use `\TYPO3\CMS\Core\Utility\ArrayUtility::mergeRecursiveW` or `array_replace_recursive`
- `getValueByPath`: Use `\TYPO3\CMS\Core\Utility\ArrayUtility::getValueByPath`
- `setValueByPath`: Use `\TYPO3\CMS\Core\Utility\ArrayUtility::setValueByPath`
- `unsetValueByPath`: Use `\TYPO3\CMS\Core\Utility\ArrayUtility::removeByPath`
- `sortArrayWithIntegerKeys`: Use `\TYPO3\CMS\Core\Utility\ArrayUtility::sortArrayWithInteg`

Deprecation: #78134 - Deprecate TypoScript option config.noScaleUp

See Issue #78317

Description

The TypoScript setting `config.noScaleUp` has been marked as deprecated.

Impact

Using this setting `config.noScaleUp` will trigger a deprecation log entry. It will work until it get's removed in TYPO3 v9.

Affected Installations

Instances that use this TypoScript setting.

Migration

Use the provided global TYPO3 configuration `$GLOBALS['TYPO3_CONF_VARS']['GFX']['processor_allowUpscali` to allow upscaling of images on a "per installation" basis.

Deprecation: #78217 - frameset and frame

See Issue #78217

Description

Frameset and frame are not supported in HTML5 anymore. The browser support for framesets could be dropped in the future.

Creating a layout based on framesets has been marked deprecated: * DocumentationFrame * DocumentationFrameset

The following TypoScript has been marked as deprecated: * `config.frameReloadIfNotInFrameset` * `config.doctype = xhtml_frames` * `config.xhtmlDoctype= xhtml_frames` * `frameSet` and its options * `FRAME` and its options * `FRAMESET` and its options

Furthermore the class `FramesetRenderer` has been marked as deprecated.

Impact

Using framesets will trigger deprecation log entries.

Affected Installations

All installations using framesets.

Migration

None.

Deprecation: #78244 - Deprecate TYPO3_DB and Prepared Statement class

See Issue #78244

Description

The classes `TYPO3\CMS\Core\Database\DatabaseConnection` and `TYPO3\CMS\Core\Database\PreparedStatement` have been marked as deprecated. This classes have been succeeded by Doctrine DBAL in TYPO3 v8, and will be removed in TYPO3 v9.

Impact

Calling any methods of the classes above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instances with references to `$GLOBALS['TYPO3_DB']` or use instances of the mentioned classes above.

Migration

Use the `ConnectionPool` and the `QueryBuilder` classes to achieve future-proof and proper database abstraction for future TYPO3 versions.

Deprecation: #78279 - Deprecate top.TYPO3.Backend.ContentContainer.iframe

See Issue #78279

Description

The property `top.TYPO3.Backend.ContentContainer.iframe` has been marked as deprecated.

Impact

Using this property will stop working in TYPO3 v9.

Affected Installations

All installations using `top.TYPO3.Backend.ContentContainer.iframe`.

Migration

Use accessor method `top.TYPO3.Backend.ContentContainer.get()` instead.

Deprecation: #78314 - AbstractFunctionModule->getBackPath

See Issue #78314

Description

The protected method `AbstractFunctionModule->getBackPath()` has been marked as deprecated, as it is not needed anymore.

Impact

Calling the PHP method will trigger a deprecation log entry.

Affected Installations

Any TYPO3 extension with a PHP class extending the `AbstractFunctionModule` and calling the method above.

Migration

As the method always returns an empty string (since the `backPath` functionality is not needed anymore) the PHP call can be removed.

Deprecation: #78524 - TCA option versioning_followPages removed

See Issue #78524

Description

The option `$TCA[$table][ctrl][versioning_followPages]` which was used for branch versioning has been removed.

Additionally the option `$TCA[$table][ctrl][versioningWS]` is now cast to boolean.

The branch / page versioning functionality was removed in TYPO3 v7, but the leftover functionality code has been completely removed as well.

Impact

A deprecation message is thrown when scanning the TCA tree for these options not being properly set or removed.

Affected Installations

Any TYPO3 installation with a TCA definition as mentioned above.

Migration

Remove the setting `$TCA[$table][ctrl][versioning_followPages]` from any TCA definition.

If a TCA table has workspaces enabled, set the option `$TCA[$table][ctrl][versioningWS]` to a boolean (true/false) directly.

Deprecation: #78581 - Flex form related parsing

See Issue #78581

Description

Three flex form data structure related parsing methods have been deprecated:

- `BackendUtility::getFlexFormDS()`
- `GeneralUtility::resolveSheetDefInDS()`
- `GeneralUtility::resolveAllSheetsInDS()`

Impact

Calling those PHP methods will trigger a deprecation log entry.

Affected Installations

Extensions calling one of the above methods.

Migration

`BackendUtility::getFlexFormDS()` has been refactored to a combination of two methods `FlexFormTools->getDataStructureIdentifier()` and `FlexFormTools->parseDataStructureByIdentifier()`. The two methods are heavily documented and the combination works in many cases just as before. Read the method comments for a detailed description of their purpose.

Warning: The hook `getFlexFormDSClass` within `BackendUtility::getFlexFormDS()` is no longer called by the core. Please refer to the according "Breaking" document for details on this topic.

Deprecation: #78628 - TCA tree pageTsConfig addItems icon path

See Issue #78628

Description

When adding items to `TCA type="select"` fields with `pageTSConfig`, the syntax for icons has been changed:

Example to add an item with icon in pages to field category before:

```
# Add an item with text "staticFromPageTs" to field category in pages
TCEFORM.pages.category.addItems.12345 = staticFromPageTs
# Assign icon to the element
TCEFORM.pages.category.addItems.12345.icon = EXT:any_extension/Resources/Public/
↪Icons/Smiley.png
```

The path has been deprecated and now accepts icon identifiers from the icon registry only:

```
# Add an item with text "staticFromPageTs" to field category in pages
TCEFORM.pages.category.addItems.12345 = staticFromPageTs
# Assign icon to the element
TCEFORM.pages.category.addItems.12345.icon = my-registered-icon
```

Impact

Using a file path syntax will trigger a deprecation log entry, but will work until TYPO3 v9.

Affected Installations

Instances that use this PageTSConfig setting with a file path instead of an icon identifier.

Migration

Register the icon within the `IconRegistry` and use an icon identifier instead of the file path.

Deprecation: #78647 - Move language files from EXT:lang/locallang_* to Resources/Private/Language

See Issue #78647

Description

Moved language files from `EXT:lang/locallang_*` to `EXT:lang/Resources/Private/Language`

Impact

Language files from `EXT:lang` have been moved to different places into the core.

Affected Installations

All 3rd party extensions that are using language labels from `EXT:lang`

Migration

Move the following references to the new location of the language file:

- lang/locallang_alt_doc.xlf -> lang/Resources/Private/Language/locallang_alt_doc.xlf
- lang/locallang_alt_intro.xlf -> lang/Resources/Private/Language/locallang_alt_intro.xlf
- lang/locallang_browse_links.xlf -> lang/Resources/Private/Language/locallang_browse_links.xlf
- lang/locallang_common.xlf -> lang/Resources/Private/Language/locallang_common.xlf
- lang/locallang_core.xlf -> lang/Resources/Private/Language/locallang_core.xlf
- lang/locallang_csh_be_groups.xlf -> lang/Resources/Private/Language/locallang_csh_be_groups.xlf
- lang/locallang_csh_be_users.xlf -> lang/Resources/Private/Language/locallang_csh_be_users.xlf
- lang/locallang_csh_corebe.xlf -> lang/Resources/Private/Language/locallang_csh_corebe.xlf
- lang/locallang_csh_pages.xlf -> lang/Resources/Private/Language/locallang_csh_pages.xlf
- lang/locallang_csh_sysfilem.xlf -> lang/Resources/Private/Language/locallang_csh_sysfilem.xlf
- lang/locallang_csh_syslang.xlf -> lang/Resources/Private/Language/locallang_csh_syslang.xlf
- lang/locallang_csh_sysnews.xlf -> lang/Resources/Private/Language/locallang_csh_sysnews.xlf
- lang/locallang_csh_web_func.xlf -> func/Resources/Private/Language/locallang_csh_web_func.xlf
- lang/locallang_csh_web_info.xlf -> lang/Resources/Private/Language/locallang_csh_web_info.xlf
- lang/locallang_general.xlf -> lang/Resources/Private/Language/locallang_general.xlf
- lang/locallang_login.xlf -> lang/Resources/Private/Language/locallang_login.xlf
- lang/locallang_misc.xlf -> lang/Resources/Private/Language/locallang_misc.xlf
- lang/locallang_mod_admintools.xlf -> lang/Resources/Private/Language/locallang_mod_admintools.xlf
- lang/locallang_mod_file_list.xlf -> lang/Resources/Private/Language/locallang_mod_file_list.xlf
- lang/locallang_mod_file.xlf -> lang/Resources/Private/Language/locallang_mod_file.xlf
- lang/locallang_mod_help_about.xlf -> lang/Resources/Private/Language/locallang_mod_help_about.xlf
- lang/locallang_mod_help_cshmanual.xlf -> lang/Resources/Private/Language/locallang_mod_help_cshmanual.xlf
- lang/locallang_mod_help.xlf -> lang/Resources/Private/Language/locallang_mod_help.xlf
- lang/locallang_mod_system.xlf -> lang/Resources/Private/Language/locallang_mod_system.xlf
- lang/locallang_mod_usertools.xlf -> lang/Resources/Private/Language/locallang_mod_usertools.xlf
- lang/locallang_mod_user_ws.xlf -> lang/Resources/Private/Language/locallang_mod_user_ws.xlf
- lang/locallang_mod_web_func.xlf -> func/Resources/Private/Language/locallang_mod_web_func.xlf
- lang/locallang_mod_web_info.xlf -> lang/Resources/Private/Language/locallang_mod_web_info.xlf
- lang/locallang_mod_web_list.xlf -> lang/Resources/Private/Language/locallang_mod_web_list.xlf
- lang/locallang_mod_web.xlf -> lang/Resources/Private/Language/locallang_mod_web.xlf
- lang/locallang_show_rechis.xlf -> lang/Resources/Private/Language/locallang_show_rechis.xlf
- lang/locallang_t3lib_fullsearch.xlf -> lang/Resources/Private/Language/locallang_t3lib_fullsearch.xlf
- lang/locallang_tca.xlf -> lang/Resources/Private/Language/locallang_tca.xlf
- lang/locallang_tcemain.xlf -> lang/Resources/Private/Language/locallang_tcemain.xlf
- lang/locallang_tsfe.xlf -> lang/Resources/Private/Language/locallang_tsfe.xlf
- lang/locallang_tsparser.xlf -> lang/Resources/Private/Language/locallang_tsparser.xlf

- lang/locallang_view_help.xlf -> lang/Resources/Private/Language/locallang_view_help.xlf
- lang/locallang_wizards.xlf -> lang/Resources/Private/Language/locallang_wizards.xlf

Deprecation: #78668 - TypoScript option config.mainScript

See Issue #78668

Description

The TypoScript option `config.mainScript` allows to set the frontend entrypoint from "index.php" to something else, and is respected when links are built, but not when e.g. previewing a page from the backend. This option has been marked as deprecated.

Impact

Setting this TypoScript option will trigger a deprecation log entry in the admin panel.

Affected Installations

Any installation using this TypoScript option.

Deprecation: #78670 - Deprecated CharsetConverter methods

See Issue #78670

Description

The `symfony/polyfill-mbstring` package provides us with `mb_string` functionality in all installations. Therefore some methods of `CharsetConverter` have been marked as deprecated, since the equivalent `mb_string` functions can be used directly:

- `strlen()`: use `mb_strlen()` directly
- `substr()`: use `mb_substr()` directly
- `strtrunc()`: use `mb_strcut()` directly
- `convCapitalize()`: use `mb_convert_case()` directly
- `conv_case()`: use `mb_strtolower()` or `mb_strtoupper()` directly
- `utf8_substr()`: use `mb_substr()` directly
- `utf8_strlen()`: use `mb_strlen()` directly
- `utf8_strtrunc()`: use `mb_strcut()` directly
- `utf8_strpos()`: use `mb_strpos()` directly
- `utf8_strrpos()`: use `mb_strrpos()` directly
- `utf8_byte2char_pos()`: no replacement
- `euc_strtrunc()`: use `mb_strcut()` directly
- `euc_substr()`: use `mb_substr()` directly
- `euc_strlen()`: use `mb_strlen()` directly
- `euc_char2byte_pos()`: no replacement

- `$fourByteSets`: no replacement

Impact

Calling the deprecated `CharsetConverter` methods will trigger a deprecation log entry.

Affected Installations

Any installation using third party extensions leveraging the mentioned `CharsetConverter` functionality.

Migration

Use the equivalent mb_string methods directly as denoted above.

Deprecation: #78679 - Crawler inclusion via require_once in Indexed Search

See Issue #78679

Description

The system extension "Indexed Search" has support for `EXT:crawler`, by using the crawler library to index a page.

This functionality is done under the hood via the Indexer class, which does a manual PHP call on "require_once" - code which is not necessary anymore, since the TYPO3 Core class loader is in place. The public PHP method `TYPO3\CMS\IndexedSearch\Indexer->includeCrawlerClass()` is therefore marked as deprecated.

Impact

Calling the method `TYPO3\CMS\IndexedSearch\Indexer->includeCrawlerClass()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with a custom indexer written in PHP, and Indexed Search and Crawler installed, and the custom indexer using the method call above.

Migration

Remove the function call, as TYPO3 includes the PHP class automatically.

Deprecation: #78733 - CallUserFunction "&" token for singleton objects

See Issue #78733

Description

The method `GeneralUtility::callUserFunction()` allows to send the callee (the user-defined function) to be prepended with a "&" before the method name to add the instantiated object to a "singleton" pool during a single request. This functionality has been marked as deprecated as it can easily be solved by implementing a class as singleton.

This way, the object is always a singleton, even when it is called via `GeneralUtility::makeInstance()`.

Impact

Calling `callUserFunction()` with a "&" symbol will trigger a deprecation log entry.

Affected Installations

Any installation with a hook or user function which is registered with an ampersand "&" symbol.

Migration

The class of the user function / method can implement the `SingletonInterface` to achieve the same behaviour.

Deprecation: #78872 - Deprecate method LocalizationController::getRecordUidsToCopy

See Issue #78872

Description

The method `\TYPO3\CMS\Backend\Controller\Page\LocalizationController::getRecordUidsToCopy()` is not used at any place in the TYPO3 core.

Impact

Calling the deprecated `getRecordUidsToCopy()` method will trigger a deprecation log entry.

Affected Installations

Any installation using the mentioned method `getRecordUidsToCopy()`.

Migration

No migration available.

2.4.4 Important

Important: #17904 - showAccessRestrictedPages does not work with special menus

See Issue #17904

Description

HMENU setting `showAccessRestrictedPages=NONE` now acts as documented in TypoScript reference.

Before: using the option renders `<a>Page title</a>` when page is inaccessible.

After: using the option renders `<a href="index.php?id=123">Page title</a>` when page is not accessible.

Important: #72050 - encapsLines does not render duplicate paragraphs anymore

See Issue #72050

Description

In the past the `_encapsLines` TypoScript function rendered two paragraphs for one empty trailing line-break in the content.

See encapsLines

This behaviour is now fixed.

Your Frontend appearance might change if you had multiple empty trailing paragraphs in your RTE content. The last paragraph is no longer rendered twice in the Frontend.

Important: #75232 - Spread TypeConverter priorities

See Issue #75232

Description

The priorities of the "TypeConverter" classes were quite packed. To be able to register own type converters between, before and after the core converters the priorities were spread from 0, 1 and 2 to 10 and 20.

If you register your own TypeConverter(s) make sure they are using the right priority.

Important: #77702 - Custom render types for date and datetime fields must use ISO-8601

See Issue #77702

Description

Historically, TYPO3 used its own special, localized formats for passing date and datetime values between server and client. To get rid of any possible problems with that, we now use ISO-8601, a standard format for date/time representations.

Due to that, you need to adapt your **custom FormEngine render types** if you use them for any date/datetime fields, even those stored as integers in the database (eval=date/datetime).

Important: #78383 - TCA: Streamline field positions in tabs for recurring fields

See Issue #78383

Description

In TYPO3 there are some recurring field definitions shared by a lot of records. These fields are mostly defined in `$GLOBALS['TCA']['<mytable>']['ctrl']`. Furthermore the generic categories are taken into account.

These fields are used by core records and third party extensions.

These fields should have a generic position in the edit form (`EditDocumentController` / `FormEngine`) to allow the editor or integrator to have a valid guess where to look for a common option. Furthermore the fields should be placed in the given order in the certain tab.

There should be no records not using tabs to group fields.

See the documentation for the definition of the recurring fields:

- ctrl
- categories

Legend

For the fields name in "Generic fields" the actual value of `$GLOBALS['TCA']['<mytable>']['ctrl']['<generic field>']` should be set in `$GLOBALS['TCA']['<mytable>']['types'][<mytype>]['showitem']`.

General (first tab)

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:general`

Generic fields:

- `type` (if there is a field that is not set as type in ctrl, but has a similar meaning, set it to the first position)
- `label`
- `label_alt` (if the fields are directly related to the label; especially if `label_alt_force` is set `true`)

Additional fields: Fields that reflect the main focus of an editor or integrator working with the record.

Following tabs

The following tabs should be defined by the specific record. They should have speaking names. Avoid unspecific labelling (for example options, settings, extended, miscellaneous) as those labels do not guide the editor.

Language

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:language`

Generic fields:

- `languageField`
- `transOrigPointerField`

Additional fields: Other fields that affects the language or translation handling.

Access

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:access`

Generic fields:

- `enablecolumns`

- `disabled`
- `starttime` (Use a palette for starttime and endtime)
- `endtime`
- `fe_group`
- `fe_admin_lock`
- `editlock`

Additional fields: Other fields that affects the access handling in FE or BE.

Categories

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:categories`

Generic fields: Field that is defined by `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::makeCatego`
It is not recommended to use the configuration option `defaultCategorizedTables` to make a table categorizable as the tab position might not be consistent.

Additional fields: Other fields that are category related (e.g. select main category)

Notes

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:notes`

Generic fields: `descriptionColumn`

Additional fields: Other fields for internal remarks of editors or integrators. These fields should not affect the website frontend.

Extended

Label: `LLL:EXT:core/Resources/Private/Language/Form/locallang_tabs.xlf:extended`

Generic fields: No.

Additional fields: No. There should be no additional field in this tab as the labelling is too generic to provide a good UX. It should be only added to prevent that accidentally added fields from third party extensions are placed in last tab.

2.5 8.4 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*

2.5.1 Breaking Changes

Breaking: #38496 - Shortcut redirects append all URL parameters

See Issue #38496

Description

When a user accesses a shortcut page, all provided URL parameters are appended to the target URL.

Example:

Page with UID 2 is a shortcut to page with UID 1 and these `linksVars` are configured:

```
config.linkVars = L
```

Old behavior:

http://mydomain.tld?id=2&L=1&customparam=X will redirect to http://mydomain.tld?id=1&L=1

New behavior:

http://mydomain.tld?id=2&L=1&customparam=X will redirect to http://mydomain.tld?id=1&L=1&customparam=X

Impact

The target URL of a shortcut may change when additional parameters are provided in the URL.

Affected Installations

All installations using shortcut pages are affected.

Migration

There is no migration available.

Breaking: #52877 - Remove ExtJS Viewport

See Issue #52877

Description

The ExtJS component `TYPO3.Viewport` has been removed from the TYPO3 Core, `Ext.layout` and `Ext.Viewport` are no longer used in the backend viewport.

Impact

- Calling the removed ExtJS components `TYPO3.Viewport` or `TYPO3.backendContentIframePanel` will result in an error
- The ability to stack content with cards with `TYPO3.Viewport.ContentCards` is no longer supported

Affected Installations

Any TYPO3 installations using custom extensions based on ExtJS which rely on the above mentioned components.

Migration

There is no migration available.

Breaking: #66995 - ObjectAccess behaviors changed

See Issue #66995

Description

The following changes have been implemented in ObjectAccess:

- Uses more native PHP methods where reasonable
- Uses fewer method calls where reasonable
- Gets rid of a variables passed by reference
- More cases return `null` rather than throw Exceptions
- Fastest decisions and access methods come first
- Reflection-based access isolated to edge cases and access with the "force direct access" flag enabled.
- Sacrifices ability to read objects of types other than persisted objects contained in an ObjectStorage or subclass of ObjectStorage.
- Changes verdict from `false` to `true` when determining if a dynamically added property exists on an object (these are by definition publicly accessible).

Impact

- Performance improvement; optimising for most frequent case coming first, skipping expensive Reflection etc.
- Reflection based access only happens when "force" flag is set to true in getProperty / getPropertyPath
- Removes support for using objects of types other than persisted objects in any ObjectStorage implementation.
- Changes behavior when accessing dynamically added properties on objects from previously false to now true (by definition they are public). This improves compatibility with JSON sources decoded to stdClass for example.

Affected Installations

- Any code using `ObjectStorage` combined with objects that are not persistent objects (Extbase models)
- Any code passing `stdClass` with dynamic properties to `ObjectAccess::getProperty()` and expecting `null` (real value will now be returned)

Migration

- If you have any `ObjectStorage` implementation containing other types than persisted objects, switch to any alternative (`Iterator`, `ArrayAccess`, etc.)
- If you have Extbase code or Fluid templates using `ObjectAccess` to read `stdClass` instances with dynamically added properties and your code expects a false verdict for such properties (highly unlikely!) inverse any conditions depending on the resolved value being null or empty to expect the actual value of the value you address.

Breaking: #75031 - Fluidification of TypoScriptTemplateInformationModuleFunctionController

See Issue #75301

Description

`TYPO3\CMS\Tstemplate\Controller\TypoScriptTemplateInformationModuleFunctionController->t`
has been removed.

Impact

Calling this method will result in a fatal error.

Affected Installations

Any installations calling `TYPO3\CMS\Tstemplate\Controller\TypoScriptTemplateInformationModuleFunct`

Migration

There is no migration available.

Breaking: #75032 - Fluidification of TypoScriptTemplateConstantEditorModuleFunctionController

See Issue #75032

Description

`TYPO3\CMS\Tstemplate\Controller\TypoScriptTemplateConstantEditorModuleFunctionController`
has been removed.

Impact

Calling this method will result in a fatal error.

Affected Installations

All installations using `TYPO3\CMS\Tstemplate\Controller\TypoScriptTemplateConstantEditorModuleFun`

Migration

There is no migration available, pleas write your own function.

Breaking: #77379 - Doctrine: Typo3DbQueryParser

See Issue #77379

Description

While migrating the database endpoint for the persistence functionality of Extbase to Doctrine DBAL, the `Typo3DbQueryParser` class has been completely rewritten to work on a `QueryBuilder` object instead of plain arrays and strings. The PHP method `Typo3DbQueryParser->parseQuery()` has been removed, instead the new equivalent `Typo3DbQueryParser->convertQueryToDoctrineQueryBuilder()` has been introduced.

Additionally, the PHP method `Typo3DBBackend->injectQueryParser()` has been removed, as the `Typo3DbQueryParser` class is not a singleton instance anymore but always rebuilt when needed.

Impact

Calling one of the methods above will result in a fatal PHP error.

Affected Installations

TYPO3 instances with custom Extbase database backend and parsing functionality.

Migration

Switch to Doctrine DBAL and `Typo3DbQueryParser->convertQueryToDoctrineQueryBuilder()` which results in the same behaviour.

Breaking: #77547 - Behaviour of RecordCollectionRepository::findByUid changed

See Issue #77547

Description

The behaviour of `RecordCollectionRepository::findByUid()` has changed. When TYPO3 is in Frontend mode, the method will now respect the configured enable fields. Instead of returning an object that is supposed to be disabled due to being hidden or having a start date in the future, or an end date in the past, it will now return `null`.

Impact

Using the `RecordCollectionRepository` expecting to fetch disabled records while TYPO3 is in Frontend mode will not yield the expected result.

Affected Installations

Any installation that uses the `FILES` cObject, e.g. via the `uploads` CType, as well as any installation with a 3rd party extension that uses the named method.

Migration

If the previous behaviour is wanted, the TCA of the used collection table needs to be overridden to not use the configured enable columns.

Breaking: #77592 - Dropped TCA option showIfRTE in type=check

See Issue #77592

Description

The TCA setting `showIfRTE` for type=check is not evaluated anymore and has been removed from the TCA on all fields.

Impact

All TCA columns having this option set will be shown at any time inside FormEngine. The option has been removed from the final TCA used inside TYPO3.

The TCA migration will throw a deprecation message when building the final TCA.

Affected Installations

TYPO3 instances using old extensions which provide custom TCA configurations having this option set.

Migration

Remove the setting from the TCA, and if still needed, use a custom display condition to achieve the same functionality.

Breaking: #77630 - Remove wizard icons

See Issue #77630

Description

The following icons have been removed from `typo3/sysext/backend/Resources/Public/Images/FormFieldWiza`

- wizard_add.gif
- wizard_edit.gif
- wizard_list.gif
- wizard_table.gif
- wizard_link.gif
- wizard_rte.gif

Impact

The mentioned icons can not be used anymore.

Affected Installations

Any installation using those icons.

Migration

The TCA migration migrates the icon calls to the new output if used as wizard icon.

- `wizard_add.gif` => `actions-add`
- `wizard_edit.gif` => `actions-open`
- `wizard_list.gif` => `actions-system-list-open`
- `wizard_table.gif` => `content-table`
- `wizard_link.gif` => `actions-wizard-link`
- `wizard_rte.gif` => `actions-wizard-rte`

Breaking: #77693 - Move icons from t3skin

See Issue #77693

Description

Icons from `EXT:t3skin/` have been removed or moved to different locations.

Impact

References of the following images of EXT:t3skin will throw a 404 not found:

- `typo3/sysext/t3skin/icons/gfx/error.png`
- `typo3/sysext/t3skin/icons/gfx/i/_icon_ftp.gif`
- `typo3/sysext/t3skin/icons/gfx/information.png`
- `typo3/sysext/t3skin/icons/gfx/notice.png`
- `typo3/sysext/t3skin/icons/gfx/warning.png`

References of the following images of EXT:t3skin/icons/gfx/i have been moved:

- `typo3/sysext/t3skin/icons/gfx/icon_fatalerror.gif` => `typo3/sysext/backend/Resources/Public/Icons/icon_fatalerror.gif`
- `typo3/sysext/t3skin/images/icons/status/status-edit-read-only.png` => `typo3/sysext/backend/Resources/Public/Icons/status-edit-read-only.png`
- `typo3/sysext/t3skin/images/icons/status/warning-in-use.png` => `typo3/sysext/backend/Resources/Public/Icons/warning-in-use.png`
- `typo3/sysext/t3skin/images/icons/status/warning-lock.png` => `typo3/sysext/backend/Resources/Public/Icons/warning-lock.png`
- `typo3/sysext/t3skin/images/icons/status/status-reference-hard.png` => `typo3/sysext/impexp/Resources/Public/Icons/status-reference-hard.png`
- `typo3/sysext/t3skin/images/icons/status/status-reference-soft.png` => `typo3/sysext/impexp/Resources/Public/Icons/status-reference-soft.png`

Affected Installations

Installations or extensions which have references to icons in EXT:t3skin/icons/.

Migration

No migration

Breaking: #77700 - Extension indexed_search_mysql merged into indexed_search

See Issue #77700

Description

The extension `indexed_search_mysql` has been removed and its functionality was merged into EXT:`indexed_search`. The `MySQL` specific search based on fulltext indexes can be enabled with a feature flag within the extension configuration of the extension manager.

Impact

If extension `indexed_search_mysql` has been loaded, the feature flag `useMysqlFulltext` within the extension configuration of the ExtensionManager `indexed_search` has to be set, otherwise `indexed_search` falls back to the potentially slower non-fulltext based default search algorithm.

Affected Installations

Instances with extension `indexed_search_mysql` loaded.

Migration

Full functionality can be kept by enabling the feature toggle `useMysqlFulltext` within the extension configuration of `indexed_search`.

Breaking: #77728 - Remove obsolete page tree and click menu settings

See Issue #77728

Description

The following obsolete properties have been removed:

- `FileSystemNavigationFrameController->doHighlight`
- `ClickMenu->leftIcons`

The following user TS settings have been removed:

- `options.pageTree.disableTitleHighlight`
- `options.contextMenu.options.leftIcons`

Impact

Extensions which use one of the public properties above will throw a fatal error. Setting above options in UserTSconfig will not impact the tree behaviour.

Affected Installations

All installations with a 3rd party extension using one of the classes above. All installations using one of the UserTSconfig settings above.

Migration

No migration available.

Breaking: #77750 - Return value of ContentObjectRenderer::exec_Query changed

See Issue #77750

Description

The return type of `ContentObjectRenderer::exec_getQuery()` has changed. Instead of returning either `bool`, `\mysqli_result` or `object` the return value always is a `\Doctrine\DBAL\Driver\Statement`.

Impact

Using the mentioned method will no longer yield the expected result type.

Affected Installations

Any installation with a 3rd party extension that uses the named method.

Migration

Change the way the result is being used to conform to the Doctrine API:

```
$result = $this->cObj->exec_getQuery(...);
while ($row = $result->fetch()) {
    // Do something here
}
```

Breaking: #77762 - Extensions dbal and adodb moved to TER

See Issue #77762

Description

The legacy extensions `dbal` and `adodb` have been removed from the TYPO3 CMS core and are only available as TER extensions.

Impact

Tables located on non-MySQL databases stop working until `EXT:adodb` and `EXT:dbal` have been installed from TER if a third party extensions uses the old `TYPO3_DB` API to query those tables.

Affected Installations

Most installations are not affected. Instances are only affected if a loaded extension uses the old `TYPO3_DB` database API, `dbal` and `adodb` have been loaded and if an active table mapping to non-MySQL databases is configured.

Migration

Use the upgrade wizard provided by the install tool to fetch and load the extensions from TER.

Breaking: #77765 - ExtJS notifications and dialogs removed

See Issue #77765

Description

ExtJS notifications with the ExtJS components `TYPO3.Window` and `TYPO3.Dialog` have been removed from the TYPO3 Core.

Impact

Calling any of the JavaScript components above will result in a JavaScript error.

Affected Installations

Any TYPO3 installations using custom extensions based on ExtJS.

Migration

Use the JavaScript-based Modal and Dialog functionality that are part of the TYPO3 Core since v7.

For migration details see the docs

Breaking: #77783 - Removed unused ExtJS JavaScript libraries

See Issue #77783

Description

The ExtJS libraries `app.SearchField`, `grid.RowExpander`, `ux.FitToParent` have been removed from the TYPO3 core after all usages of them have been removed from the TYPO3 core with Feature #74359.

Impact

Including or calling any of the named JavaScript libraries will result in an error.

Affected Installations

Any TYPO3 installations using custom extensions based on ExtJS which rely on the named libraries.

Migration

There is no migration available, consider migrating to a supported modern framework.

Breaking: #77814 - Remove feature subsearch from indexed search

See Issue #77814

Description

The feature subsearch which enabled the possibility to append previously searched words to the current query was removed.

The option TypoScript `plugin.tx_indexedsearch.clearSearchBox` has been removed.

Impact

Frontend output of search results may change.

Setting the TypoScript option has no effect anymore.

Affected Installations

Any installation using this option or using custom indexed search plugin templates.

Migration

Remove the option in your custom TypoScript and remove the functionality in the Fluid plugin template.

Breaking: #77826 - RTEHtmlArea Spellchecker eID removed

See Issue #77826

Description

The RTEHtmlArea eID (`rtehtmlarea_spellchecker`) for using dynamic spellchecking has been removed.

RTEhtmlarea uses the Backend Routing API for Backend and Frontend Editing.

Impact

Calling the eID script will result in a 404 error.

Affected Installations

Installations which use the eID `rtehtmlarea_spellchecker` in a custom extension.

Breaking: #77919 - Changed DateTime::ISO8601 to DateTime::ATOM

See Issue #77919

Description

The format `DateTime::ISO8601` is not compatible with ISO-8601, but is left this way for backward compatibility reasons. The constant `DateTime::ATOM` or `DATE_ATOM` is used instead when rendering JsonViews via Extbase.

See http://php.net/manual/en/class.datetime.php#datetime.constants.iso8601 for more information

Breaking: #77987 - Removal fe_users rendering in page module

See Issue #77987

Description

Because of the deprecation of `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['cms']['db_layout']['addTable`
the records of the table `fe_users` are no longer rendered in the page module.

Affected Installations

All installations.

Migration

By using the following code in the `ext_localconf.php` file, the records can be shown again:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['cms']['db_layout']['addTables']['fe_users
↪'] =
    array (0 => array(
        'MENU' => '',
        'fList' =>  'username,usergroup,name,email,telephone,address,zip,city',
        'icon' => true
    )
);
```

Breaking: #78222 - Extension autoload information is now in typo3conf/autoload

See Issue #78222

Description

To make clear that autoload information is not a cache, the files have been moved from typo3temp to typo3conf.

Impact

TYPO3 deployments which do not take advantage of composer, might need some adaption to also include the new location in typo3conf in the list of to be synced files.

Affected Installations

TYPO3 installation in non composer mode.

Migration

Class loading information can be re-dumped by

- Activating or deactivating an extension
- Running the dump command from the command line
- In install tool (important actions)

The autoload files should never be deleted, but always only be re-dumped.

2.5.2 Features

Feature: #17309 - Access flexform value via TS

See Issue #17309

Description

It is now possible to access properties of a flexform field by using TypoScript.

```
lib.flexformContent = CONTENT
lib.flexformContent {
    table = tt_content
    select {
        pidInList = this
    }

    renderObj = COA
    renderObj {
        10 = TEXT
        10 {
            data = flexform: pi_flexform:settings.categories
        }
    }
}
```

The key `flexform` is followed by the field which holds the flexform data and the name of the property whose content should be retrieved.

Feature: #75691 - Upgrade Analysis - Provide listing of documentation files

See Issue #75691

Description

The install tool now shows all the documentation files that were delivered with the core in the section `Upgrade analysis`. All files can be read inline, but there is no parsing, plain `.rst` is shown to the user.

Impact

The install tool features a new main entry point that lists the documentation files shipped with the core. Filtering by tags provided in the documentation files helps to find interesting changes.

Feature: #76748 - Configure the availability of the elementbrowser

See Issue #76748

Description

The button to open the elementBrowser can be configured to be enabled/disabled for the user.

The button can be disabled by the following TCA setting: `[table_name]['columns'][field_name]['config']['appe = false;`

The button can be disabled by the following pageTs setting: `TCEFORM.table_name.field_name.config.appearance.e = 0`

The button can be disabled by the following userTs setting: `page.TCEFORM.table_name.field_name.config.appearar = 0`

Impact

Default behavior is that the button stays visible, to disable/hide this button the TCA/pageTs/userTs needs to be explicit changed.

Feature: #77589 - EXT: syntax in PageRenderer and Compressor

See Issue #77589

Description

It is now possible to use the `EXT:` prefix for referencing files inside extensions within the `PageRenderer` and `ResourceCompressor` PHP classes for adding JavaSyntax or StyleSheet files.

So you can streamline your code from

```
$this->pageRenderer->addJsFile(ExtensionManagementUtility::extRelPath('core') .
↪'Resources/Public/JavaScript/Contrib/bootstrap/bootstrap.js');
```

to

```
$this->pageRenderer->addJsFile('EXT:core/Resources/Public/JavaScript/Contrib/
↪bootstrap/bootstrap.js');
```

Feature: #77643 - Reimplement SqlSchemaMigrationService using Doctrine SchemaManager

See Issue #77643

Description

The SqlSchemaMigrationService has been reimplemented using a LL(*) Parser for CREATE TABLE statements. The new parser supports MySQL syntax for CREATE TABLE statements. Based on the abstract syntax tree produced by this parser Doctrine Table objects are created that implement a DBMS independent representation of the schema and are used with the Doctrine SchemaManager to handle the schema migrations needs of the TYPO3 core.

Impact

Update suggestions from the new SchemaMigrator are per connection, on all additional connections only explicitly mapped tables are managed. MySQL specific data types are being mapped to the closest matching standard type, for example TINYINT to SMALLINT. The support for foreign keys has been enhanced as a result of the additional capabilities of the Doctrine SchemaManager.

Feature: #77652 - Make sys_language records sortable

See Issue #77652

Description

To improve the usability, the sys_language records are now sortable.

Impact

The languages can now be manually sorted instead of an ordering by its name.

Feature: #77668 - Hide table listing below group element

See Issue #77668

Description

The TCA configuration option `disable_controls` of the type "group" now has a new setting `allowedTables` which can be added to the configuration in order to hide the information which tables are allowed to be referenced in the group field.

Feature: #77799 - Display TCA migration messages in Install Tool

See Issue #77799

Description

Display the TCA migration message(s) in the Install Tool.

Impact

You are now able to show all TCA migration messages from within the Install Tool.

Feature: #77900 - Introduce TypeScript for the core

See Issue #77900

Description

The TYPO3 core has introduced TypeScript for the internal JavaScript handling.

Why we use TypeScript in the core?

TypeScript is a free and open source programming language developed and maintained by Microsoft. It is a strict superset of JavaScript, and adds optional static typing and class-based object-oriented programming to the language.

With TypeScript it is possible to compile JavaScript. TypeScript supports definition files which can contain type information of existing JavaScript libraries.

At the moment the core uses AMD modules for any JavaScript logic in the backend, but what is in 5 years? maybe we want switch to CommonJS, with TypeScript we can recompile all modules with a simple change in a configuration file.

But the main reason to switch to TypeScript is the strict typing and oop structure of the language. We can make use of Interfaces, which is still a missing feature in JavaScript.

Coding Guidelines & Best practice

`/// <amd-dependency path="x" />` informs the compiler about a non-TS module dependency that needs to be injected in the resulting module's require call.

The amd-dependency has a property name which allows passing an optional name for an amd-dependency: `/// <amd-dependency path="x" name="fooBar" />`

An example:

```
/// <amd-dependency path="TYPO3/CMS/Core/Contrib/jquery.minicolors"
name="minicolors">
```

will be compiled to:

```
define(["require","exports","TYPO3/CMS/Core/Contrib/jquery.minicolors"],function
(require,exports,minicolors) {
```

A very simple example is the `EXT:backend/Resources/Private/TypeScript/ColorPicker.ts` file.

TypeScript Linter

The most rules for TypeScript are defined in the rulesets which are checked by the TypeScript Linter. The core provides a configuration file and grunt tasks to ensure a better code quality. For this reason we introduce a new grunt task, which first run the Linter on each TypeScript file before starting the compiler. So if your TypeScript does not follow the rules, the task will fail. The idea is to write clean code, else it will not be compiled.

Additional Rules

For the core we have defined some additional rules which you should know, because not all of them can be checked by the Linter yet:

1. Always define types and return types, also if TypeScript provides a default type. [checked by Linter]
2. Variable scoping: Prefer `let` instead of `var`. [checked by Linter]
3. Optional properties in interfaces are possible but a bad style, this is not allowed for the core. [NOT checked by Linter]
4. An interface will never extend a class. [NOT checked by Linter]
5. Iterables: Use `for (i of list)` if possible instead of `for (i in list)` [NOT checked by Linter]
6. The `implements` keyword is required for any usage, also if TypeScript does not require it. [NOT checked by Linter]

7. Any class or interface must be declared with "export" to ensure re-use or export an instance of the object for existing code which can't be updated now. [NOT checked by Linter]

Contribution workflow

```
# Change to Build directory
cd Build

# Install dependencies
npm install

# Install typings for the core
grunt typings

# Check with Linter and compile ts files from sysext/*/Resources/Private/TypeScript/
↪*.ts
grunt scripts

# File watcher, the watch task also check for *.ts files
grunt watch
```

The grunt task compiles each TypeScript file (*.ts) to a JavaScript file (*.js) and produces an AMD module.

Impact

All AMD modules must be ported to TypeScript to ensure a future proof concept of JavaScript handling. The goal is to migrate all AMD modules to a TypeScript file until CMS 8 LTS is released.

Feature: #78222 - Dump Class Loading Information UI in Install Tool

See Issue #78222

Description

It is now possible to dump class loading information in the install tool.

Impact

Users who are not familiar with the command line can now trigger a dump of class loading information in install tool.

2.5.3 Deprecation

Deprecation: #75363 - Deprecate FormResultCompiler->JStop()

See Issue #75363

Description

The method `JStop()` has been renamed to `addCssFiles()`. The old method name is still present as a deprecated alias, which will be removed in TYPO3 v9.

Keep in mind that the method reads "JS top", not "J Stop".

Impact

Calling `JStop()` method will trigger a deprecation log entry.

Affected Installations

Any extensions calling `JStop()`.

Migration

Instead of `JStop()` use `addCssFiles()`.

Deprecation: #75637 - Deprecate optional parameters of RecyclerUtility::getRecordPath()

See Issue #75637

Description

The following arguments of the method `RecyclerUtility::getRecordPath` have been marked as deprecated:

- `$clause`
- `$titleLimit`
- `$fullTitleLimit`

Impact

Using any of the arguments above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using custom calls to `RecyclerUtility::getRecordPath` using the mentioned arguments.

Migration

No migration available.

Deprecation: #77763 - Deprecate method ClickMenu::DB_editPageProperties()

See Issue #77763

Description

Method `ClickMenu::DB_editPageProperties` has been marked as deprecated.

Impact

Using the method will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using custom calls to `ClickMenu::DB_editPageProperties`

Migration

No migration available.

Deprecation: #77826 - RTEHtmlArea Spellchecker entrypoint

See Issue #77826

Description

The entry point for HTTP Requests `SpellCheckingController->main` within RTEHtmlArea has been marked as deprecated.

Impact

Calling the PHP method above will trigger a deprecation log entry.

Affected Installations

All TYPO3 instances calling this PHP method.

Migration

Use `SpellCheckingController->processRequest` instead.

Deprecation: #77839 - Move TYPO3/CMS/Core/QueryGenerator into EXT:lowlevel and deprecate the old module

See Issue #77839

Description

The AMD module `TYPO3/CMS/Core/QueryGenerator` have been deprecated. The module have been renamed to `TYPO3/CMS/Lowlevel/QueryGenerator` and moved into EXT:lowlevel.

Impact

Using the module will trigger a deprecation log message in the browser console.

Affected Installations

Any TYPO3 installation using custom calls to `TYPO3/CMS/Core/QueryGenerator`

Migration

Use AMD module `TYPO3/CMS/Lowlevel/QueryGenerator` instead.

Deprecation: #77987 - Deprecated record listing in page module

See Issue #77987

Description

The usage of `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['cms']['db_layout']['addTables']` to render records in the page module has been marked as deprecated.

Accessing property `TYPO3\CMS\Backend\Controller\PageLayoutController->externalTables` has been deprecated.

Affected Installations

Instances using `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['cms']['db_layout']['addTables']` or accessing property `TYPO3\CMS\Backend\Controller\PageLayoutController->externalTables`.

Migration

No migration available.

Deprecation: #78096 - Deprecated PageLayoutView::getResult with mysqli_result objects

See Issue #78096

Description

The method `PageLayoutView::getResult` has been marked as deprecated with the usage of mysqli_result objects as first parameter.

Impact

Using the mentioned method with a mysqli_result object will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using extensions which call `PageLayoutView::getResult` with mysqli_result objects.

Migration

The extension should migrate to use the Doctrine API for database queries.

Deprecation: #78193 - ExtensionManagementUtility::extRelPath()

See Issue #78193

Description

The method `ExtensionManagementUtility::extRelPath()` for resolving paths relative to the current script has been marked as deprecated.

Impact

Calling `ExtensionManagementUtility::extRelPath()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance with extensions or third-party scripts resolving paths with the method above.

Migration

Use alternatives for resolving paths. There are the following methods available: * `ExtensionManagementUtility::extPath()` - to resolve the full path of an extension * `ExtensionManagementUtility::siteRelPath()` - to resolve the location of an extension relative to `PATH_site` * `GeneralUtility::getFileAbsFileName()` - to resolve a file/path prefixed with `EXT:myext` * `PathUtility::getAbsoluteWebPath()` - used for output a file location (previously resolved with `GeneralUtility::getFileAbsFileName()`) that is absolutely prefixed for the web folder

Deprecation: #78222 - Late generation of autoload information is deprecated

See Issue #78222

Description

If TYPO3 is in non-composer mode, it used to automatically dump extension class loading information late during the bootstrap. This behavior is now deprecated.

Impact

TYPO3 installations in non-composer mode, now trigger a deprecation log entry in case extension autoload information is missing late in the bootstrap.

Affected Installations

TYPO3 installations in non-composer mode with deleted extension autoload information.

Migration

Class loading information can be re-dumped by

- Activating or deactivating an extension
- Running the dump command from the command line
- In install tool (important actions)

The autoload files should never be deleted, but always only be re-dumped.

Deprecation: #78224 - TYPO3_DB occurrences

See Issue #78224

Description

The TYPO3_DB shorthand functionality has been removed for most of the TYPO3 Core PHP classes, excepted for the following locations:

- AbstractPlugin->databaseConnection (protected property)
- AbstractFunctionModule::getDatabaseConnection()
- BaseScriptClass::getDatabaseConnection()

For these occurrences extensions might extend the base functionality (e.g. for plugins or modules) and the call to the property and protected methods still work.

Calling `$GLOBALS[TYPO3_DB]` is still possible but discouraged.

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instances with references to TYPO3_DB or references to the occurrences mentioned above.

Migration

Use the ConnectionPool and the QueryBuilder classes to achieve future-proof and proper database abstraction for future TYPO3 versions.

2.6 8.3 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.6.1 Breaking Changes

Breaking: #73461 - Import module disabled for non admin users

See Issue #73461

Description

The import module of EXT:impexp has been disabled for non-admin users by default.

Impact

For non-admin users who need that functionality, the userTsConfig option `options.impexp.enableImportForNonAdminUser = 1` can be set. This can become a security problem to the TYPO3 instance in core versions 7.6 and 6.2 and should only be enabled for "trustworthy" backend users in general.

Affected Installations

Installations with non-admin users making active use of the import / export module

Migration

Set userTsConfig option `options.impexp.enableImportForNonAdminUser = 1` to restore the old behavior.

Breaking: #74375 - fe_users.image migrated to FAL

See Issue #74375

Description

The Frontend User field "image" was previously handled via images located under `uploads/pics/`, as simple file references, not able to handle duplicate images etc.

The field is now set up to add references of the File Abstraction Layer avoiding the need to copy all images to uploads/pics/.

Impact

Using the `fe_users.image` field in the frontend or backend may result in unexpected behaviour.

Affected Installations

Any TYPO3 installation using the field "image" within the database table "fe_users", common in third-party extensions using the field for storing images for frontend users (like mm_forum).

Migration

Use the File Abstraction Layer for output and deal with rendering or changing images for frontend users.

Use the migration wizard provided in the install tool to migrate existing data to proper file references.

Breaking: #76108 - Replace ExtJS category tree with D3 and SVG

See Issue #76108

Description

Backend ExtJS category tree has been replaced with one based on D3.js and SVG. The js file `typo3/sysext/backend/Resources/Public/JavaScript/tree.js` has been removed.

The expanded/collapsed state will not be saved to the backend user settings any more. It was not used in the core, as all category trees have setting 'expandAll' set to true. It also polluted backend user settings with tons of data without giving much usability gain.

Impact

Any JS code referencing ExtJS component `TYPO3.Components.Tree` or its sub-components (like `TYPO3.Components.Tree.StandardTree`) will no longer work.

Affected Installations

All installations having extensions which modify the `TYPO3.Components.Tree` (implemented in `typo3/sysext/backend/Resources/Public/JavaScript/tree.js`) component, or rely on the file being present.

Migration

Migration of the JS code to the new `SvgTree` component is recommended.

Breaking: #76259 - Return value of AbstractDatabaseRecordList::makeSearchString changed

See Issue #76259

Description

The value returned by `AbstractDatabaseRecordList::makeSearchString` has been adjusted.

The SQL fragment no longer includes the leading `AND` SQL operator and the method returns "1=1" if no search word is specified or if the table contains no searchable fields.

Impact

3rd Party extensions need to ensure that valid SQL queries are being built using the returned fragment.

Affected Installations

Installations using 3rd party extensions that use `AbstractDatabaseRecordList::makeSearchString` and expect the leading `AND`.

Migration

Migrate your code to use the Doctrine QueryBuilder where the `AND` is no longer needed or prepend the missing `AND` before using the return value.

Breaking: #76259 - Signature of getResult() in PageLayoutView changed

See Issue #76259

Description

As part of migrating the core code to use Doctrine DBAL the signature of the method `PageLayoutView::getResult()` has changed.

Instead of accepting `bool`, `\mysqli_result` or `object` as a result provider only `\Doctrine\DBAL\Driver\Statement` objects are accepted.

The new signature is:

```
public function getResult(\Doctrine\DBAL\Driver\Statement $result, string $table =
↪'tt_content') : array
{
}
```

Impact

3rd party extensions using `PageLayoutView::getResult()` need to provide the correct input type, otherwise exceptions of type `InvalidArgumentException` will be thrown.

Affected Installations

Installations using 3rd party extensions that use `PageLayoutView::getResult()`.

Migration

Refactor all code that works with `PageLayoutView::getResult()` to provide the expected Doctrine Statement object.

Breaking: #76259 - Signature of setTotalItems() in AbstractDatabaseRecordList changed

See Issue #76259

Description

As part of migrating the core code to use Doctrine DBAL the signature of the method `AbstractDatabaseRecordList::setTotalItems()` has changed.

The new signature is:

```
public function setTotalItems(string $table, int $pageId, array $constraints)
{
    $queryBuilder = $this->getQueryBuilder($table, $pageId, $constraints);
    $this->totalItems = (int)$queryBuilder->count('*')
        ->execute()
        ->fetchColumn();
}
```

The parameter `$constraints` is expected to be an array of Doctrine Expressions or SQL fragments.

In case of SQL fragments proper quoting needs to be ensured by the invoking method. SQL fragments should not have a leading ' AND ' SQL operator.

Impact

3rd party extensions using `AbstractDatabaseRecordList::setTotalItems()` need to update the method invocation.

Affected Installations

Installations using 3rd party extensions that use `AbstractDatabaseRecordList::setTotalItems()`.

Migration

Instead of passing an array of parameters built using the deprecated ::php::`makeQueryArray()` method explicitly pass in the table name, page id and any additional query restrictions required.

Breaking: #76259 - Value passed to hook getTable changed

See Issue #76259

Description

The value of `$additionalWhere` passed to the method `getDBlistQuery()` as part of the hook `getTable` in `\TYPO3\CMS\Recordlist\RecordList\DatabaseRecordList` has changed and no longer includes the leading `AND`.

Impact

3rd Party extensions implementing the hook method need to ensure the leading `AND` is no longer present. The leading `AND` should also not be returned anymore.

Affected Installations

Installations using 3rd party extensions that implement the hook method.

Migration

Migrate the hook method to no longer expect or prepend the leading `AND`.

Breaking: #76879 - Remove unused properties from PageTreeView

See Issue #76879

Description

The following unused properties have been removed from the `PageTreeView` class:

`ext_separateNotinmenuPages` `ext_alphasortNotinmenuPages`

Impact

Extensions which use one of the public properties above will throw a fatal error.

Affected Installations

All installations with a 3rd party extension using one of the classes above.

Migration

No migration available.

Breaking: #76879 - Remove unused property pidSelect from AbstractDatabaseRecordList

See Issue #76879

Description

The unused public property `pidSelect` has been removed from the `AbstractDatabaseRecordList` class.

Impact

Extensions which use the public property will throw a fatal error.

Affected Installations

All installations with a 3rd party extension using the `pidSelect` property.

Migration

Use `AbstractDatabaseRecordList::setOverridePageIdList()` to set an array of page ids that should be used to restrict the query.

Breaking: #76891 - syslog lowlevel command

See Issue #76891

Description

The lowlevel cleaner syslog command has been migrated to a Symfony Command to show the latest sys_log entries on the command line.

The new command can be called via `./typo3/sysext/core/bin/typo3 syslog:list`.

Impact

Calling `./typo3/cli_dispatch.phpsh lowlevel_cleaner syslog -r` will not show the expected output anymore as before.

Migration

Use `./typo3/sysext/core/bin/typo3 syslog:list` with the optional verbose parameter instead.

Breaking: #77049 - Remove unused properties from SuggestWizardDefaultReceiver

See Issue #77049

Description

The following unused properties have been removed from the `SuggestWizardDefaultReceiver` class:

- `selectClause`
- `addWhere`

Impact

Extensions which use one of the protected properties above will not work properly as the properties are not used by class methods anymore.

Affected Installations

All installations with a 3rd party extension extending the `SuggestWizardDefaultReceiver` class.

Migration

Don't set the properties in extended classes and make use of the constructor and queryTable method instead, as it is done in the `SuggestWizard` class.

Breaking: #77062 - Example image in TS constants descriptions removed

See Issue #77062

Description

In previous TYPO3 versions it was possible to add help text and and an help image to a certain category or configuration option in the TypoScript Constant Editor of the TYPO3 Backend. This was previously done via an additional Constant Editor option within the `TSConstantEditor` object.

The functionality has been removed without substitution.

Along with that change, the following PHP methods have been removed: - `ExtendedTemplateService::ext_getTSCE_config_image()` - `ConfigurationForm::ext_getTSCE_config_image()`

The following public properties have been removed: - `ExtendedTemplateService::$ext_localGfxPrefix` - `ExtendedTemplateService::$ext_localWebGfxPrefix`

Within `ConfigurationForm::ext_initTSstyleConfig()` the second and third parameter have been removed.

Impact

Setting an option `TSConstantEditor.basic.image = EXT:sys_note/ext_icon.png` for a category or configuration option in TypoScript constants has no effect anymore.

Calling any of the removed methods will result in a fatal PHP error.

Using any of the removed properties will result in a PHP warning.

Calling `ConfigurationForm::ext_initTSstyleConfig()` with the second or third parameter will result in a PHP warning.

Affected Installations

Any TYPO3 installation with extended TypoScript constant editor configuration.

Migration

Remove the affected TypoScript constant editor configuration code, and any reference to the removed PHP methods and properties.

Breaking: #77081 - Removed TCA tree options: width, allowRecursiveMode, autoSizeMax

See Issue #77081

Description

The following three TCA configuration options have been removed from the FormEngine TCA Tree functionality which is e.g. used within the FormEngine for the selection of categories.

TCA Column Config

- [config][treeConfig][appearance][allowRecursiveMode]

The option hasn't been working for a while and the documentation vs. implementation was off - see https://forge.typo3.org/issues/77074

- [config][treeConfig][appearance][width]
- [config][autoSizeMax]

The options have no influence on the rendering of FormEngine select field configured with 'renderType'=>'selectTree' anymore.

The Recursive selection button (the green arrow button located on the category tree toolbar) was not widely used, mostly due that nobody expected the green "refresh" icon was related to recursive selection.

When implemented 4 years ago, the purpose of this button was to ease handling of "record storage page". But now the "recursive" select box can be used for this usage.

The option `autosizemax` has been dropped as the `size` can be used as maximum height.

Impact

The recursive selection mode button is not available any longer.

The options `width` and `autoSizeMax` have no impact on the tree rendering.

The TCA Tree now fills the full width of the parent container.

Instead of using the option `autoSizeMax` the configuration is now using the `size` parameter as maximal height of the TCA tree.

Affected Installations

Any TYPO3 installation using a TCA Tree within FormEngine with one of the options above configured.

Migration

Use the `size` option and tune it to higher value, if it was used in combination with `autoSizeMax`.

Breaking: #77137 - JavaScript API of RTE changed

See Issue #77137

Description

Due to the migration of the RTE from ExtJS to Bootstrap and jQuery, some API methods have been changed or removed.

Impact

ExtJS-based plugins will throw JavaScript errors.

The following methods have been removed: * `onContainerResize` * `getWindowDimensions` * `setTabPanelHeight` * `syncHeight`

The following methods have been changed: * `openContainerWindow` * `buildButtonConfig`

Affected Installations

All installations using custom RTE plugins are affected.

Migration

The former `Ext.Window` objects are replaced by Bootstrap modals.

See the list below for a migration of the changed methods:

openContainerWindow The third parameter `dimensions` which used to be an array has changed to `height`, containing an integer

buildButtonConfig The method takes now two additional arguments: `active` and `severity`. The parameter `active` is a boolean value and declares the button being either active or not. The parameter `severity` is an integer representing the severity of the button. This should always represent the severity of the modal, use one of the severities defined in `TYPO3/CMS/Backend/Severity`.

Breaking: #77137 - RTE option "colors" removed

See Issue #77137

Description

The RTE configuration option `RTEsetup.properties.colors` has been removed.

Impact

The color picker is streamlined with all occurrences in the backend, it shows all available colors.

Affected Installations

All installations configuring `RTEsetup.properties.colors` are affected.

Migration

There is no migration available, the obsolete configuration can be removed.

Breaking: #77156 - TSconfig and TStemplate soft references functionality removed

See Issue #77156

Description

Soft references based on TSconfig, TStemplate and images within fileadmin/ have been removed.

The soft reference keys "TSconfig" and "TStemplate" that could previously be set via `$TCA[$table][columns][$column][config][softref]` are not evaluated anymore.

The soft reference keys "images", "typolink" and "typolink_tag" are not evaluating files within fileadmin/ anymore that are not based on the File Abstraction Layer.

The public PHP property "SoftReferenceIndex::$fileAdminDir" has been removed.

The following PHP methods has been removed without substitution: - `SoftReferenceIndex::findRef_TStemplate()` - `SoftReferenceIndex::findRef_TSconfig()` - `SoftReferenceIndex::fileadminReferences()`

Impact

Setting the softref properties `TSconfig` and `TStemplate` within TCA will not be evaluated anymore and will throw a deprecation message.

Calling any of the PHP methods above will throw a fatal PHP error.

The soft reference index will not be updated with the TSconfig and TStemplate properties anymore, as well as files directly linked or referenced with the fileadmin/ directory.

Affected Installations

If the soft reference index is evaluated in a third-party extension, this might result in unexpected behaviour.

All TYPO3 instances using extensions setting TSconfig or TStemplate soft references in TCA are also affected.

Migration

Remove the softref keys `TStemplate` and `TSconfig` from the TCA definition of the third party extensions.

Breaking: #77180 - Dropped ExtJS support in Frontend TypoScript

See Issue #77180

Description

The following TypoScript options

```
page.javascriptLibs.ExtJs
page.javascriptLibs.ExtJs.debug
page.inlineLanguageLabel
page.extOnReady
```

have been removed.

Impact

Using the settings above will not include ExtJs and inline language labels anymore in the TYPO3 Frontend.

Affected Installations

Any installation using the shipped ExtJS bundle in the frontend.

Migration

Include ExtJS via `page.includeJS` manually if needed or migrate to another supported modern framework.

Breaking: #77182 - Removed BasicFileUtility methods and properties

See Issue #77182

Description

The `BasicFileUtility` class was used for checking file mounts and paths, and is only used for non-FAL files. Now, old legacy functionality has been removed.

The `init()` method hasbeen replaced by a real constructor. A possibility to set the file extension permissions has been added via `setFileExtensionPermissions()`.

The DefaultConfiguration setting `$TYPO3_CONF_VARS[BE][fileExtensions][ftpspace]` has been removed.

The following public properties within BasicFileUtility have been removed: - `getUniqueNamePrefix` - `tempFN` - `f_ext` - `mounts` - `webPath` - `isInit`

The following public methods within `BasicFileUtility` have been removed: - `checkPathAgainstMounts()` - `findFirstWebFolder()` - `slashPath()` - `is_webpath()` - `checkIfFullAccess()` - `init()`

The following public properties within `BasicFileUtility` have been set to have a protected visibility: - `is_directory` - `is_allowed`

Impact

Calling any of the methods above or using one of the properties above will result in PHP errors and warnings respectively.

Using the `TYPO3_CONF_VARS setting` has no effect anymore.

Affected Installations

Any installation using pre - 6.0 core functionality within extensions.

Migration

Use the File Abstraction Layer to achieve the same functionality.

Breaking: #77184 – Various TSFE properties removed

See Issue #77184

Description

The following public properties of the PHP class `TypoScriptFrontendController` have been removed.

- TYPO3_CONF_VARS
- defaultBodyTag
- clientInfo

Additionally, the first parameter of the `TypoScriptFrontendController` constructor has no effect anymore and can be set to null.

Impact

Accessing or setting the properties will throw a PHP warning and have no effect anymore.

Affected Installations

Any installation working with the public property in a third-party extension or instantiating the `TSFE` object itself.

Migration

For any calls to `$TSFE->TYPO3_CONF_VARS` the global array `$GLOBALS['TYPO3_CONF_VARS']` should be used.

For the property `defaultBodyTag` the according TypoScript settings can be used to override the body tag or the page title.

The information previously stored in the clientInfo property can be fetched via `GeneralUtility::clientInfo()`.

Breaking: #77186 - ExtDirect eID entry point removed

See Issue #77186

Description

The frontend eID script to call the `ExtDirect` API for backend calls has been removed.

Impact

Calling `index.php?eID=ExtDirect` will result in an PHP exception.

Affected Installations

Instances using the ExtDirect eID script.

Breaking: #77209 - Adapt default RECORDS tables

See Issue #77209

Description

The value of the constant `styles.content.shortcut.tables` which is set by EXT:fluid_styled_content has been changed from "tt_content,tt_address,tt_news,tx_news_domain_model_news" to "tt_content".

Impact

Shortcuts to records of the tables tt_address, tt_news and tx_news_domain_model_news don't work anymore.

Affected Installations

Installations which use shortcut to records of the tables tt_address, tt_news and tx_news_domain_model_news

Migration

Adopt the TS and use `styles.content.shortcut.tables := addToList(tt_address,tt_news,tx_news_dom`

Breaking: #77280 - Uploads template shows file title in favor of file name

See Issue #77280

Description

The file title is now shown instead of the file name, if the title is present.

Impact

The output of the template changes.

Affected Installations

Every installation using the "File links" content element with files having a specified title is affected.

Migration

Override the template and remove the condition to restore the original behavior.

Breaking: #77342 - Removed templateFile override via FlexForm in EXT:felogin

See Issue #77342

Description

The possibility to override the template file via FlexForm when inserting a Frontend Login plugin has been removed.

The need for the upload folder `uploads/tx_felogin` has been removed, and the folder is not generated anymore.

Impact

The template file cannot be selected anymore from the Frontend Login plugin. Existing installations using this option before will fall back to the TypoScript setting silently.

Affected Installations

TYPO3 instances using the `templateFile` option via FlexForms in Frontend Login plugins.

Migration

Use the TypoScript setting `plugin.tx_felogin.templateFile` to set an alternative template file.

Breaking: #77345 - EXT:form - Remove deprecated IMAGEBUTTON element

See Issue #77345

Description

The `IMAGEBUTTON` element has been removed in TYPO3 v8.

Impact

Using the `IMAGEBUTTON` element is not working anymore, i.e. no `IMAGEBUTTON` element will be rendered.

Affected Installations

All installations using the `IMAGEBUTTON` element.

Migration

Remove all usages of the `IMAGEBUTTON` element.

Breaking: #77390 - Expected return type of hook getResultRows_SQLpointer in Indexed Search changed

See Issue #77390

Description

As part of migrating the core code to use Doctrine DBAL the expected return value of the hook `getResultRows_SQLpointer` in EXT:indexed_search has changed.

It is required that `\Doctrine\DBAL\Driver\Statement` objects are returned instead of the previous types `bool` or `\mysqli_result`.

Impact

3rd party extensions implementing the hook `getResultRows_SQLpointer` need to provide the correct return type, otherwise fatal errors will occur when processing the search results.

Affected Installations

Installations using 3rd party extensions that implement the hook `getResultRows_SQLpointer` for Indexed Search.

Migration

Migrate the implementation of the hook to provide the expected Doctrine Statement object.

Breaking: #77391 - DataHandler method protected

See Issue #77391

Description

Method `doesRecordExist_pageLookUp()` of class `DataHandler` has been changed from public access to protected access and the returned object changed to an instance of `QueryBuilder`.

Impact

Calling the method will trigger a fatal PHP error.

Affected Installations

Extensions that use method `doesRecordExist_pageLookUp()`. This is very unlikely since the method is mostly only useful for core internal handling.

Migration

No migration possible, remove the method call.

Breaking: #77416 - Removed property from DatabaseIntegrityCheck

See Issue #77416

Description

The property `$perms_clause` has been removed from class `DatabaseIntegrityCheck`.

Impact

Setting or reading this property on an instance of `TYPO3\CMS\Core\Integrity\DatabaseIntegrityCheck` will result in a fatal PHP error.

Affected Installations

All installations with a 3rd party extension using this class.

Migration

No migration available.

Breaking: #77453 - Signature of AbstractPlugin::pi_exec_query changed

See Issue #77453

Description

The value returned by `AbstractPlugin::pi_exec_query` has changed.

Instead of returning one of `bool`, `\mysqli_result` or `object` the method always returns a `Doctrine\Dbal\Driver\Statement`.

Impact

3rd Party extensions using `AbstractPlugin::pi_exec_query` need to be modified to work with the new return type.

Affected Installations

Installations using 3rd party extensions that use `AbstractPlugin::pi_exec_query`.

Migration

Migrate your code to use the `Statement` object:

```
$statement = $this->pi_exec_query(...);
while($row = $statement->fetch())
{
    // ... do something here
}
```

Breaking: #77453 - Signature of AbstractPlugin::pi_list_makelist changed

See Issue #77453

Description

The expected result data type of the method `AbstractPlugin::pi_list_makelist` has changed.

Instead of accepting `bool`, `\mysqli_result` or `object` as a result provider only `\Doctrine\DBAL\Driver\Statement` objects are accepted.

Impact

3rd party extensions using `AbstractPlugin::pi_list_makelist` need to provide the correct input type.

Affected Installations

Installations using 3rd party extensions that use `AbstractPlugin::pi_list_makelist`.

Migration

Migrate all code that works with the `AbstractPlugin::pi_list_makelist` to provide the expected Doctrine Statement object.

Breaking: #77460 - Extbase query cache removed

See Issue #77460

Description

The PHP-based query cache functionality within the Extbase persistence layer has been removed.

The following public methods within the Extbase persistence layer have been removed:

- `Typo3DbBackend->quoteTextValueCallback()`
- `Typo3DbBackend->initializeObject()`
- `Typo3DbBackend->injectCacheManager()`
- Interface definition in `QuerySettingsInterface->getUseQueryCache()`

The TypoScript configuration `config.tx_extbase.persistence.useQueryCache` has no effect anymore.

Impact

The according cache configuration set via `$TYPO3_CONF_VARS[SYS][cache][cacheConfigurations][extbase_typ` has no effect anymore.

Affected Installations

Any installation effectively relying on the query cache via a third party extension or explicitly deactivating the query cache of extbase.

Migration

Remove the according lines and migrate to Doctrine.

Breaking: #77481 - Remove favicon from TBE_STYLES

See Issue #77481

Description

The configuration `$GLOBALS['TBE_STYLES']['favicon']` has been removed.

Impact

The configuration `$GLOBALS['TBE_STYLES']['favicon']` is not evaluated anymore.

Affected Installations

Any installation using `$GLOBALS['TBE_STYLES']['favicon']`.

Migration

Define the favicon in the setting of EXT:backend in the extension manager.

Breaking: #77502 - Extbase: pre-parsing of queries removed

See Issue #77502

Description

Extbase's custom implementation to pre-parse and cache queries has been removed in favor of using the RDBMS' native implementation via Doctrine DBAL.

The following public methods have been removed: * `Typo3DbBackend->quoteTextValueCallback()` * `Typo3DbQueryParser->preparseQuery()` * `Typo3DbQueryParser->normalizeParameterIdentifier()` * `Typo3DbQueryParser->addDynamicQueryParts()` * `ComparisonInterface->setParameterIdentifier` * `ComparisonInterface->getParameterIdentifier`

Impact

Calling any of the methods above will result in a fatal PHP error.

Affected Installations

Any TYPO3 installation using custom logic inside Extbase's own Persistence layer within `Typo3DbBackend` or `Typo3DbQueryParser`.

Migration

Remove the functionality and just use `Typo3DbQueryParser->parseQuery()`.

Breaking: #77557 - Signature of QueryView->getQueryResultCode() changed

See Issue #77557

Description

The method signature of `QueryView->getQueryResultCode()` has changed from `getQueryResultCode($mQ,$res,$table)` to `getQueryResultCode($type,array $dataRows,$table)`.

The second argument is no longer a MySQLi or DBAL result object, but an array of rows.

Impact

Extensions using this method will throw a fatal error.

Affected Installations

Extensions using `QueryView->getQueryResultCode()`

Migration

Move away from the method or feed it with an array of database rows.

Breaking: #77558 - PageLayoutController removed methods

See Issue #77558

Description

The following methods have been removed from `PageLayoutController` without substitution:

- `exec_languageQuery()`
- `isColumnEmpty()`
- `getElementsFromColumnAndLanguage()`

All of those methods were internally used within the Page module, the risk that an extension using them is low.

Impact

The methods executed page module specific queries. Extensions calling the method will throw a fatal error.

Affected Installations

Extensions calling one of the aforementioned methods.

Migration

Move away from those methods.

Breaking: #77587 - Removed LiveSearch->getQueryString

See Issue #77587

Description

The public utility method `getQueryString()` within the `LiveSearch` PHP class has been removed.

Impact

Calling the method directly will result in a PHP fatal error.

Affected Installations

Any installation extending TYPO3's internal LiveSearch functionality via an extension.

Migration

Use one of the various quoting options shipped with the Doctrine DBAL.

Breaking: #77591 - Removed WorkspaceService->isOldStyleWorkspaceUsed

See Issue #77591

Description

The method `WorkspaceService->isOldStyleWorkspaceUsed()` was removed without substitution. It existed to identify if Workspaces still were configured for TYPO3 4.4.

Impact

Calling the PHP method directly will result in a fatal PHP error.

Affected Installations

TYPO3 installations using workspaces and extending the workspaces functionality extensively by providing a fallback layer to functionality for TYPO3 4.4 or lower.

Migration

Remove any occurrences to the PHP method.

2.6.2 Features

Feature: #73461 - Enable import module for non admin users

See Issue #73461

Description

The new userTsConfig option `options.impexp.enableImportForNonAdminUser` can be used to enable the import module of EXT:impexp for non admin users.

Impact

This option should be enabled for "trustworthy" backend users only.

Feature: #74365 - Add Linkservice for unified referencing syntax

See Issue #74365

Description

Resources within TYPO3 have been referenced using multiple, different forms of syntax in the past.

TYPO3 now supports a modern and future-proof way of referencing resources using an extensible and expressive syntax which is easy to understand.

In order to understand the syntax, we will guide you through using a simple page link.

`t3://page?uid=13&campaignCode=ABC123`

The syntax consists of three main parts, much like parts on an URL:

Syntax Namespace (t3://) The namespace is set to `t3://` to ensure the `LinkService` should be called to parse the URL. This value is fixed and mandatory.

Resource handler key (page) The resource handler key is a list of available handlers that TYPO3 can work with. At the time of writing these handlers are:

- page
- file
- folder

More keys can be added via `$TYPO3_CONF_VARS['SYS']['linkHandler']` in an associative array where the key is the handler key and the value is a class implementing the LinkHandlerInterface.

Resource parameters(?uid=13&campaignCode=ABC123) These are the specific identification parameters that are used by any handler. Note that these may carry additional parameters in order to configure the behavior of any handler.

Handler syntax

page

The page identifier is a compound string based on several optional settings.

uid int: The **uid** of a page record.

`t3://page?uid=13`

alias string: The **alias** of a page record (as an alternative to the UID).

`t3://page?alias=myfunkyalias`

type int *(optional)*:

`t3://page?uid=13&type=3` will reference page 13 in type 3.

parameters string *(optional, prefixed with &)*:

`t3://page?uid=1313&my=param&will=get&added=here`

fragment string *(optional, prefixed with #)*:

`t3://page?alias=myfunkyalias#c123`

`t3://page?uid=13&type=3#c123`

`t3://page?uid=13&type3?my=param&will=get&added=here#c123`

file

uid int: The UID of a file within the FAL database table `sys_file`. `t3://file?uid=13`

identifier int: The identifier of a file when not indexed in FAL. `t3://file?identifier=folder/myfile.jpg`

folder

identifier string: The identifier of a given folder. `t3://folder?identifier=fileadmin`

storage string: The FAL storage to the given folder (optional). `t3://folder?storage=1&identifier=myfolder`

Examples:

Linking to a page in RTE

The old way of linking to a page in the RTE resulted in the following code in the database: `<link 13?campaignCode=ABC123 _blank class="linkMe" #c1234>Text</link>`

The new way would be the following code in the database: `<a href="t3://page?uid=13&campaignCode=ABC123#c1234" target="_blank" class="linkMe">Text</a>`

As you can see, the syntax is more in line with known markup, thus removing the demand of data processing from or to the RTE component.

Referencing an image in RTE

```
<img src="t3://file?uid=134&renderAs=png" width="200" height="200">
```

In this example we illustrate a **fictional** usecase of identifier configuration (mind the "renderAs" part).

Impact

Currently the impact is rather low, since a fallback mechanism will still be able to work with the old syntax.

Feature: #76107 - Add fluid interceptor registration

See Issue #76107

Description

Interceptors in Fluid Standalone have been introduced to be able to change the template output. The Fluid API already allows for registration of custom interceptors. Now it is possible to define custom interceptors via the following option:

```
$GLOBALS['TYPO3_CONF_VARS']['fluid']['interceptors']
```

Interceptors registered here are added to the Fluid parser configuration.

Impact

Extensions are able to register custom interceptors using the available configuration in `$TYPO3_CONF_VARS[fluid][interceptors]`.

Registered classes have to implement the `\TYPO3Fluid\Fluid\Core\Parser\InterceptorInterface`.

```
// Register an own interceptor to fluid parser configuration
$GLOBALS['TYPO3_CONF_VARS']['SYS']['fluid']['interceptors
↪']['\TYPO3\CMS\Fluid\Core\Parser\Interceptor\DebugInterceptor::class] =
  \TYPO3\CMS\Fluid\Core\Parser\Interceptor\DebugInterceptor::class;
```

```
use TYPO3Fluid\Fluid\Core\Parser\InterceptorInterface;
use TYPO3Fluid\Fluid\Core\Parser\ParsingState;
use TYPO3Fluid\Fluid\Core\Parser\SyntaxTree\NodeInterface;

class DebugInterceptor implements InterceptorInterface
{
```

```
    public function process(NodeInterface $node, $interceptorPosition, ParsingState
↪$parsingState) : NodeInterface
    {
        return $node;
    }

    public function getInterceptionPoints()
    {
        return [];
    }
}
```

Feature: #76108 - Replace ExtJS category tree with D3 and SVG

See Issue #76108

Description

The Backend ExtJS category tree (renderType `selectTree`) has been replaced with one based on D3.js and SVG. Tree implements a 'virtual scroll' pattern, meaning that it renders only as many nodes as fit in the viewport.

Additionally the tree now display icon overlay (e.g. for disabled categories).

Structure

There are three RequireJS modules:

- SvgTree.js - this is a base JS object able to render a SVG based tree. It can expand and collapse child nodes, render icons for each node, and keep track of the select nodes.
- SelectTree.js - extends the SvgTree object (prototype inheritance) with checkboxes
- SvgTreeToolbar.js - toolbar for SvgTree which allows to search, collapse all and expand all tree nodes

Visual Scroll

SvgTree renders only as many nodes as fit in the wrapping container. This requires that the wrapping container has a fixed height set. So e.g. if one node takes 20px height, and the wrapper has 200px, only 10 nodes will be rendered at the time.

Data binding

Thanks to D3, each SVG node representing tree item is bound to the data object. The general idea is that all operations (like showing/hiding/selecting...) are first performed on the dataset, and then the view (SVG) is refreshed. In the `initialize` function SvgTree loads the whole tree as json

Impact

- New tree is faster.
- A new 'indeterminate' state for the category has been introduced introduced. The category is in the 'indeterminate' state if at last one of its descendants is selected (checked).
- Tree data is not rendered inline in HTML any more but fetched via Ajax

Feature: #76209 - Hook to register custom result browsers in AbstractPlugin

See Issue #76209

Description

The hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][\TYPO3\CMS\Frontend\Plugin\AbstractPlug` allows registering custom result browser implementations. This approach allows to override the default implementation of `AbstractPlugin::pi_list_browseresults()` for either all extensions or only for specific ones.

Impact

The hook may be registered in `ext_localconf.php`:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][\TYPO3\CMS\Frontend\Plugin\AbstractPlugin:
↪:class]['pi_list_browseresults'][1463475262] =␣
↪\Vendor\ExtensionKey\Hook\ResultBrowserHook::class
```

The registered class must implement the method `pi_list_browseresults()` with the following arguments:

- int `$showResultCount` Determines how the results of the page browser will be shown
- string `$tableParams` Attributes for the table tag which is wrapped around the table cells containing the browse links
- array `$wrapArr` Array with elements to overwrite the default $wrapper-array
- string `$pointerName` Variable name for the pointer
- bool `$hscText` Enable htmlspecialchars() for the pi_getLL function
- bool `$forceOutput` Forces the output of the page browser if you set this option to `true`
- object `$pObj` The AbstractPlugin instance calling the hook

Feature: #76259 - Introduce buildQueryParametersPostProcess Hook

See Issue #76259

Description

With the migration to Doctrine the hook `buildQueryParameters` has been introduced in the class `DatabaseRecordList`. This hook replaces the hook `makeQueryArray` from the deprecated method `AbstractDatabaseRecordList::makeQueryArray`.

Using this hook allows modifying the parameters used to query the database for records to be shown in the record list view.

The hook-object needs to be registered in `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][\TYPO3\CMS\Recordli` and implement the public method `buildQueryParametersPostProcess`.

The signature of the `buildQueryParametersPostProcess` method is as following:

```
public function buildQueryParametersPostProcess(
    array $parameters,
    string $table,
    int $pageId,
    array $additionalConstraints,
    array $fieldList,
```

```
    AbstractDatabaseRecordList $parentObject
) : void {
}
```

The following fields are part of the `$parameters` array and can be modified:

Key	Type	Description
table	string	The queried tablename
fields	string[]	The columns to retrieve
groupBy	string[]	The columns to group the result by
firstResult	int|null	The offset to start retrieve rows from
maxResults	int|null	The maximum number of rows to retrieve
orderBy	array[]	Array of arrays containing fieldname/sorting pairs
where	string[]	Array of where conditions to apply to the database query.

Feature: #77280 - Render the file title in "file links" content element

See Issue #77280

Description

In the content element "File links" now displays the file title, if present.

Impact

In the frontend the file name will be replaced by the title as soon as a file has a title.

Feature: #77336 - Allow passing an own unit collection to BytesViewHelper

See Issue #77336

Description

The BytesViewHelper accepts a new parameter named `units`. It must be a comma separated list of units.

First example: Use the translation VH

```
{fileSize -> f:format.bytes(units: '{f:translate(\'viewhelper.format.bytes.units\',
↪ \'fluid\')}'}
```

Second example: Provide a plain list

```
<f:format.bytes units="byte, kilo, mega, husel, pusel">{size}</f:format.bytes>
```

results in the currently used collection, provided by the core.

Impact

A custom list of units can be passed to the ViewHelper and will be used for formatting. The existing behaviour has not changed.

Feature: #77349 - Additional locations for extension icons

See Issue #77349

Description

Extensions can now hold their extension icons in additional locations to the existing ones (ext_icon.png, ext_icon.svg, ext_icon.gif):

- `Resources/Public/Icons/Extension.png`
- `Resources/Public/Icons/Extension.svg`
- `Resources/Public/Icons/Extension.gif`

This makes it possible to restrict access to more directories, thus hardening the TYPO3 instance.

Feature: #77481 - Add possibility to define a favicon for the backend

See Issue #77481

Description

The new option `backendFavicon` in the extension manager configuration makes it possible to change the favicon of the backend. This makes it easier to distinguish between different installations and stages.

2.6.3 Deprecation

Deprecation: #76259 - Deprecate method makeQueryArray of AbstractDatabaseRecordList

See Issue #76259

Description

The method `AbstractDatabaseRecordList::makeQueryArray()` has been marked as deprecated.

Impact

Using the method mentioned will trigger a deprecation log entry. The hook `makeQueryArray` provided within this method is no longer called by the core.

Affected Installations

Instances that use the method.

Migration

Migrate your code to the Doctrine based replacement `\TYPO3\CMS\Recordlist\RecordList\AbstractDatabaseReco` and the associated hook `buildQueryParameters`.

Deprecation: #76520 - Deprecate method pages_getTree of PageLayoutView

See Issue #76520

Description

The method `pages_getTree()` of `PageLayoutView` has been marked as deprecated.

Impact

Calling the method `pages_getTree` will trigger a deprecation log entry.

Affected Installations

Any installation with a 3rd party extension calling the method in its PHP code.

Migration

No migration available, implement the required functionality in your own code.

Deprecation: #76804 - Deprecate GeneralUtility::strtoupper & strtolower

See Issue #76804

Description

The following methods within `GeneralUtility` have been marked as deprecated:

- `strtoupper()`
- `strtolower()`

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any installation with a 3rd party extension calling one of the methods in its PHP code.

Migration

Instead of `GeneralUtility::strtoupper($value)` use:

```
mb_strtoupper($value, 'utf-8');
```

Instead of `GeneralUtility::strtolower($value)` use:

```
mb_strtolower($value, 'utf-8');
```

Alternatively use the native implementation of `strtoupper($value)` or `strtolower($value)` if the handled string consists of ascii characters only.

Deprecation: #77164 - ErrorpageMessage and AbstractStandaloneMessage

See Issue #77164

Description

The two PHP classes `ErrorpageMessage` and `AbstractStandaloneMessage` have been marked as deprecated.

Impact

Instantiating one of the PHP classes will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using the two PHP classes directly because of a specialized error handling or exception handling method.

Migration

Use the new Fluid-based ErrorPageController class.

Deprecation: #77405 - PageRepository->getPathFromRootline

See Issue #77405

Description

The PHP method `PageRepository->getPathFromRootline()` has been marked as deprecated.

Impact

Calling the method will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with a third-party extension using this method.

Deprecation: #77432 - Extbase: Prepared Statement Query Option

See Issue #77432

Description

The option to use prepared statements within the Extbase persistence layer has been removed. The method `getUsePreparedStatement()` has been removed from the `QuerySettingsInterface`, as the database abstraction layer will take care of prepared statements automatically.

The implementation of the following properties within `Typo3QuerySettings` has been marked as deprecated:

- `getUsePreparedStatement()`
- `usePreparedStatement()`

The protected property `usePreparedStatement` has been marked as deprecated as well.

Impact

Calling one of the methods above within the `QuerySettings` object within the extbase persistence will trigger a deprecation notice warning.

Affected Installations

Any TYPO3 instance with an extbase extension using custom query settings using the `usePreparedStatement()` option.

Migration

Remove any calls to the methods within the extensions' code, as the TYPO3 abstraction layer will handle them automatically.

Deprecation: #77477 - TemplateService->fileContent

See Issue #77477

Description

The method `fileContent` within the class `TemplateService` has been marked as deprecated.

Impact

Calling the method will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance with a third-party extension calling the method directly.

Migration

Implement the same logic directly in PHP with `getFileName()` and `file_get_contents()`.

Deprecation: #77502 - Extbase: pre-parsing of queries removed

See Issue #77502

Description

The following methods and properties within Extbase's persistence query comparison interface have been marked as deprecated:

- `Comparison->setParameterIdentifier()`
- `Comparison->getParameterIdentifier()`

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using custom logic inside Extbase's own persistence layer with parameters and placeholders within `Typo3DbBackend` or `Typo3DbQueryParser` and actively overwriting parameter identifiers within Extbase.

Migration

Usage of these methods can be replaced by simply using the `DataMapper->getPlainValue()` functionality.

Deprecation: #77557 - Method QueryView->tableWrap()

See Issue #77557

Description

Method `QueryView->tableWrap()` has been marked as deprecated.

Impact

Extensions using this method will trigger a deprecation log entry.

Affected Installations

Extensions using `QueryView->tableWrap()`

Migration

Use `'<pre>' . $str . '</pre>'` instead.

2.6.4 Important

Important: #77411 - Removed extbase table column cache

See Issue #77411

Description

The extbase table column cache "extbase_typo3dbbackend_tablecolumns", which was used to store all database fields of all database tables, hass been removed.

The associated configuration variable `$TYPO3_CONF_VARS[SYS][caching][cacheConfigurations][extbase_typ` can safely be removed.

2.7 8.2 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.7.1 Breaking Changes

Breaking: #75493 - Evaluate "boolean /stdWrap" properties correctly

See Issue #75493

Description

stdWrap sub-properties on boolean properties were not evaluated correctly unless the property itself was explicitly set.

Example: `page.10.value.prioriCalc.wrap =` without `page.10.value.prioriCalc =`

Impact

It is now possible to reliably use stdWrap sub-properties on boolean properties.

Affected Installations

Earlier installations can be affected if they contain TypoScript that triggers the bug.

Test case:

```
page = PAGE
page.10 = TEXT
page.10.value = 1+1
page.10.value.prioriCalc.wrap =
```

Result was: 2 Correct result: 1+1

For some installations, the bug fix will make the TypoScript work as intended. Other installations might accidentally rely on the broken code.

In the latter case, the TypoScript can be changed to:

```
page = PAGE
page.10 = TEXT
page.10.value = 1+1
page.10.value.prioriCalc = 1
```

Migration

The usage of stdWrap sub-properties on boolean properties needs to be checked and possibly adapted to fit the fixed behavior.

Breaking: #75645 - Doctrine: migrate ext:backend/Tree

See Issue #75645

Description

This patch changes all database related functions to use the new Doctrine database API. The method `getDatabaseConnection()` has been removed.

Impact

Calls to the method `AbstractTreeView::getDataInit()` will now return `Statement` objects. All other `AbstractTreeView::getData*` methods now expect such a `Statement` object instead of a SQL resource.

Affected Installations

All installations using TreeViews extending the AbstractTreeView.

Migration

Migrate all calls that work with the result `Statement` from TreeView to be able to handle `Statement` objects.

Breaking: #75710 - RTE-related TSconfig options skipAlign and skipClass removed

See Issue #75710

Description

The two RTE-related TSconfig options `RTE.default.proc.skipAlign` and `RTE.default.proc.skipClass` that don't allow align and class attributes when converting content from the RTE to the database have been removed.

Impact

Setting these options will have no effect anymore.

Affected Installations

Any installation setting custom TSconfig options for `skipAlign`, `skipClass` or setting `keepPDIVattribs` without `class` and `align` as values are affected.

Migration

In order to allow class and align attributes in paragraph tags, the option `keepPDIVattribs` needs to be extended to also include "class" and "align" as values, which is done by default.

If an installation has custom TSconfig settings using `keepPDIVattribs`, the two attributes need to be added accordingly to still allow class and align attributes to be kept when transforming from the RTE to the database.

Breaking: #75747 - EXT:form - Removed useDefaultContentObject setting

See Issue #75747

Description

The TypoScript option `useDefaultContentObject` of the FORM cObject has been removed. Setting this value to 0 allowed the usage of the prehistoric content type `mailform`.

Impact

It is not possible to configure the rendering of the FORM cOject. The setting is not evaluated anymore.

Affected Installations

Any installation that uses the TypoScript option `useDefaultContentObject = 0`.

Migration

Remove the TypoScript option from any TypoScript settings. Migrate manually to use the features of EXT:form.

Breaking: #75760 - Return type of LocalizationRepository::getRecordsToCopyDatabaseResult changed

See Issue #75760

Description

The return type of `LocalizationRepository::getRecordsToCopyDatabaseResult()` has changed. Instead of returning either `bool`, `\mysqli_result` or `object` the return value always is a `\Doctrine\DBAL\Driver\Statement`.

Impact

Using the mentioned method will not yield the expected result type.

Affected Installations

Any installation with a 3rd party extension that uses the named method.

Migration

Change the way the result is being used to conform to the Doctrine API:

```
$result = $this->localizationRepository->getRecordsToCopyDatabaseResult(...);
while ($row = $result->fetch()) {
    // Do something here
}
```

Breaking: #76285 - Popup configuration is moved to TYPO3.settings

See Issue #76285

Description

The popup window configuration has been moved to `TYPO3.settings`.

The following configuration options are not working anymore.

`top.TYPO3.configuration.RTEPopupWindow.width` `top.TYPO3.configuration.RTEPopupWindow.heigh`
`top.TYPO3.configuration.PopupWindow.width` `top.TYPO3.configuration.PopupWindow.height`

Impact

Width and height are not set correctly anymore in JavaScript context.

Affected Installations

Extensions that use one of the mentioned configurations.

Migration

The migration can be done with the following replacements.

`top.TYPO3.configuration.RTEPopupWindow.width` to `TYPO3.settings.Textarea.RTEPopupWindow.wic`

`top.TYPO3.configuration.RTEPopupWindow.height` to `TYPO3.settings.Textarea.RTEPopupWindow.he`

`top.TYPO3.configuration.PopupWindow.width` to `TYPO3.settings.Popup.PopupWindow.width`

`top.TYPO3.configuration.PopupWindow.height` to `TYPO3.settings.Popup.PopupWindow.height`

Breaking: #76469 - Doctrine: migrate ext:ImpExp

See Issue #76469

Description

The return type of `ImportExportController::exec_listQueryPid()` has changed. Instead of returning either `bool`, `\mysqli_result` or `object` the return value always is a `\Doctrine\DBAL\Driver\Statement`.

Impact

Using the mentioned method will not yield the expected result type.

Affected Installations

All installations with a 3rd party extension using `ImportExportController::exec_listQueryPid()`.

Migration

Migrate all calls that work with the result of `ImportExportController::exec_listQueryPid()` to be able to handle `\Doctrine\DBAL\Driver\Statement` objects.

Breaking: #76527 - Cleanup ContextMenu classes

See Issue #76527

Description

The classes of the ContextMenu (used in page tree) have been refactored. The following classes have been removed:

- `\TYPO3\CMS\Backend\ContextMenu\AbstractContextMenu`
- `\TYPO3\CMS\Backend\ContextMenu\AbstractContextMenuDataProvider`
- `\TYPO3\CMS\Backend\ContextMenu\Extdirect\AbstractExtdirectContextMenu`
- `\TYPO3\CMS\Backend\ContextMenu\Renderer\AbstractContextMenuRenderer`

Impact

Extensions which use one of the classes above will stop working.

Affected Installations

All installations with a 3rd party extension using one of the classes above.

Breaking: #76802 - Drop xcache cache backend

See Issue #76802

Description

The `xcache` core cache backend has been dropped because PHP7 does no longer support xcache.

Impact

Instances will throw an exception or a fatal error using a cache with this backend configured.

Affected Installations

Instances that still use a configuration in `LocalConfiguration.php` or `AdditionalConfiguration.php` like:

```
'SYS' => [
    'caching' => [
        'cacheConfigurations' => [
            'aCache' => [ 'backend' => 'TYPO3\CMS\Core\Cache\Backend\XcacheBackend',
```

Migration

Affected instances must switch to a different cache backend, `APCU PHP` module with `ApcuBackend` provides the same local server in memory characteristics like `xcache` and should be considered as alternative.

2.7.2 Features

Feature: #18586 - Configurable width & height for editpanel in feedit

See Issue #18586

Description

It is possible to change the popup's width and height which is used in the edit panel of EXT:feedit by using User TsConfig:

```
options.feedit.popupHeight = 700
options.feedit.popupWidth = 900
```

Feature: #20446 - Clear cache entry in context menu

See Issue #20446

Description

A new entry has been added in the context menu of the page tree to be able to clear the cache of the selected page. The entry is located inside "Page Actions".

Feature: #76008 - Property visibility to DebuggerUtility::var_dump

See Issue #76008

Description

The property visibility information has been added to `DebuggerUtility::var_dump()` for each object property in the dump.

Feature: #76072 - Ogg, flac and opus support

See Issue #76072

Description

Support for the following open formats has been added to the media field: * ogg * flac * opus

Feature: #76458 - Let DebuggerUtility render closures

See Issue #76458

Description

The `DebuggerUtility` of Extbase is now able to render closures.

Impact

If a closure is part of the debugging object, the source code of the closure is rendered.

Feature: #76531 - Add IconForRecordViewHelper

See Issue #76531

Description

A new ViewHelper to render icons for records has been added. Now it is possible to render icons for records in fluid.

```
<core:iconForRecord table="tt_content" row="{recordRow}" />
```

Feature: #76590 - Introduce UnitTests for JavaScript

See Issue #76590

Description

The core uses karma as test runner and jasmine as testing framework for JavaScript UnitTests. The tests will be run on travis with PhantomJS. Locally other browsers like Chrome, Firefox, Safari or IE can be used as well.

To run the UnitTests on a local system the following steps are necessary:

1. Install dependencies

```
cd Build
npm install
cd ..
```

2. Run the tests from your terminal

```
# Execute the tests only once
./Build/node_modules/karma/bin/karma start typo3/sysext/core/Build/Configuration/
↪JSUnit/karma.conf.js --single-run

# Execute the tests for every change (file watcher mode)
./Build/node_modules/karma/bin/karma start typo3/sysext/core/Build/Configuration/
↪JSUnit/karma.conf.js

# Execute the tests for different browser
./Build/node_modules/karma/bin/karma start typo3/sysext/core/Build/Configuration/
↪JSUnit/karma.conf.js --single-run --browsers Chrome,Safari,Firefox
```

Test-Files

Any test file must be located in extension folder `typo3/sysext/<EXTKEY>/Tests/JavaScript/` The filename must end with Test.js, e.g. `GridEditorTest.js` Each test file must be implemented as AMD module, must use strict mode and has to use `describe` with module name as outer wrap for each test. The following code block shows a good example:

```
define(['jquery', 'TYPO3/CMS/Backend/AnyModule'], function($, AnyModule) {
   'use strict';
   // first and outer wrap describe the test class name
   describe('TYPO3/CMS/Backend/AnyModuleTest:', function() {
      // second wrap describe the method to test
      describe('tests for fooAction', function() {
         // the first parameter of each 'it' method describe the test-case.
         it('works for parameter a and b', function() {});
      });
      describe('tests for barAction', function() {
         it('works for parameter a and b', function() {});
      });
   }
}
```

Please take a look at the existing test files and read the jasmine documentation for further information.

DataProvider for tests

For testing a set of values, the core implement a kind of DataProvider. To use the DataProvider you have to use the function `using`. Please take a look at `FormEngineValidationTest.js` for an example.

2.7.3 Deprecation

Deprecation: #15415 - Deprecate removeBadHTML

See Issue #15415

Description

Due to the wrong approach of removeBadHTML it is not 100% complete and does not keep its promise.

- `ContentObjectRenderer::stdWrap_removeBadHTML()`
- `ContentObjectRenderer::removeBadHTML()`
- `stdWrap.removeBadHTML`

Impact

Using the mentioned method or stdWrap property will trigger a deprecation log entry.

Affected Installations

Instances that use the method or stdWrap property.

Migration

Implement a proper encoding by yourself. Use `htmlspecialchars()` or `stdWrap.htmlSpecialChars` in the context of HTML, `GeneralUtility::quoteJSvalue()` or `stdWrap.encodeForJavaScriptValue` in the context of JavaScript.

Deprecation: #71917 - Deprecate the argument 'hsc' for getLL, getLLL, sL and pi_getLL

See Issue #71917

Description

The parameter `$hsc` within the following methods of `TYPO3\CMS\Lang\LanguageService` has been marked as deprecated:

- `getLL()`
- `getLLL()`
- `sL()`

The parameter `$hsc` within the following method of `TYPO3\CMS\Frontend\Plugin\AbstractPlugin` has been marked as deprecated:

- `pi_getLL()`

Impact

Directly or indirectly using any of the methods `getLL()`, `getLLL()`, `sL()` or `pi_getLL()` with the parameter `$hsc` will trigger a deprecation log entry.

Affected Installations

Any installation with a third-party extension calling one of the methods in its PHP code.

Migration

If the return value of these methods is output in HTML context use `htmlspecialchars` directly to properly escape the content.

Deprecation: #72859 - Deprecate methods of DocumentTemplate

See Issue #72859

Description

The following methods within `DocumentTemplate` have been marked as deprecated:

- viewPageIcon()
- getHeader()
- getResourceHeader()
- header()
- icons()
- t3Button()
- wrapScriptTags()
- loadJavascriptLib()
- getContextMenuCode()

The following property within `DocumentTemplate` has been marked as deprecated:

- sectionFlag (is internal)

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any installation with a 3rd party extension calling one of the methods in its PHP code.

Migration

Instead of `wrapScriptTags()` use `GeneralUtility::wrapJS`.

Instead of `getContextMenuCode()` use:

```
$this->getPageRenderer()->loadJquery();
$this->getPageRenderer()->loadRequireJsModule('TYPO3/CMS/Backend/ClickMenu');
```

Deprecation: #75209 - Code cleanup for MenuViewHelperTrait

See Issue #75209

Description

The `MenuViewHelperTrait` has been marked as deprecated. All methods of the Trait have been implemented in a new `AbstractMenuViewHelper` class.

Impact

Using the methods of the `MenuViewHelperTrait` will trigger a deprecation log entry.

Affected Installations

Instances with custom extensions that use the `MenuViewHelperTrait`.

Migration

Extend the new `AbstractMenuViewHelper` which contains all methods instead of using the trait.

Deprecation: #75760 - Deprecate methods of LocalizationRepository

See Issue #75760

Description

The following methods have been marked as deprecated:

- `LocalizationRepository::getExcludeQueryPart()`
- `LocalizationRepository::getAllowedLanguagesForBackendUser()`

Impact

Using the mentioned methods will trigger a deprecation log entry

Affected Installations

Any installation with a 3rd party extension that uses one of the named methods.

Migration

Instead of `LocalizationRepository::getExcludeQueryPart()` configure the query restrictions yourself:

```
$queryBuilder->getRestrictions()
    ->removeAll()
    ->add(GeneralUtility::makeInstance(DeletedRestriction::class))
    ->add(GeneralUtility::makeInstance(BackendWorkspaceRestriction::class));
```

Instead of `LocalizationRepository::getAllowedLanguagesForBackendUser()` add the required conditions to your query yourself.

Deprecation: #75904 - Category Model has icon property but no database field

See Issue #75904

Description

Methods `\TYPO3\CMS\Extbase\Domain\Model\Category::getIcon` and `\TYPO3\CMS\Extbase\Domain\Model\Category::setIcon` have been marked as deprecated.

Impact

Using the methods will trigger a deprecation log entry.

Affected Installations

Instances with custom extensions that use these methods.

Migration

Implement the methods by yourself.

Deprecation: #76101 - remove SoloFieldContainer

See Issue #76101

Description

The `render()` method of the `SoloFieldContainer` class has been marked as deprecated. It is not used in core anymore.

Impact

Using the method will trigger a deprecation log entry.

Affected Installations

Instances with custom extensions that use `render()` from `SoloFieldContainer`.

Migration

Use the render method from the ListOfFieldsContainer class.

Deprecation: #76104 - Deprecated single slash comments in TypoScript

See Issue #76104

Description

Double slash one-line comments are standard in many languages. Make them standard for TypoScript, too.

Define:

```
One-line comments must start with two forward slashes as
the first non-blank characters and should be followed by
a whitespace.
```

Deprecated:

```
/ Line comment headed by single slash
```

Impact

The TypoScript developer receives a deprecation warning with line number.

Affected Installations

Installations using a single slash to indicate a comment.

Migration

Use double slashes (//) or hash (#) for comments in Typoscript.

Deprecation: #76164 - Deprecate RemoveXSS

See Issue #76164

Description

Due to the wrong approach of RemoveXSS it is not 100% secure and does not keep its promise. The following methods have been marked as deprecated:

- `\TYPO3\CMS\Core\Utility\GeneralUtility::removeXSS()`
- `\RemoveXSS::process()`
- `\TYPO3\CMS\Form\Domain\Filter\RemoveXssFilter`

Impact

Using the mentioned methods will trigger a deprecation log entry.

Affected Installations

Instances that use any of these methods.

Migration

Implement a proper encoding by yourself. Use `htmlspecialchars()` in the context of HTML or `GeneralUtility::quoteJSvalue()` in the context of JavaScript.

Deprecation: #76345 - Path prefixes in callUserFunction and getUserObj

See Issue #76345

Description

The two methods `GeneralUtility::callUserFunc()` and `GeneralUtility::getUserObj()` allow the first parameter to contain a file reference to the function/class to be called if prefixed with a colon.

An example would be `EXT:myext/Classes/MyClass.php:Benni\Myext\MyClass` for including the class.

Having the reference to the actual file is not needed since the composer autoloading mechanism takes care of loading everything properly already since TYPO3 6.2.9.

Impact

Calling one of the methods above with a file reference prepended to the class name / function name will trigger a deprecation log entry.

Affected Installations

Any installation with a hook that is registered with the file prefix functionality.

Migration

Remove the file prefix when registering a hook and make use of the common autoloading functionality of composer or via the fallback autoloader by TYPO3 to achieve the same functionality automatically.

Deprecation: #76370 - Deprecate CacheFactory

See Issue #76370

Description

Class `CacheFactory` has been deprecated.

Impact

The class is no longer used or instantiated by the core. Instantiating the class will trigger a deprecation log entry.

Affected Installations

TYPO3 instances and extensions typically make no use of this internal class.

Migration

Nothing notable, do not use this class anymore.

Deprecation: #76383 - Deprecate fontTag

See Issue #76383

Description

Font tags are not used any more in HTML since years.

- `ContentObjectRenderer::stdWrap_fontTag()`
- `stdWrap.fontTag`

Impact

Using the mentioned method or stdWrap property will trigger a deprecation log entry.

Affected Installations

Instances that use the method or stdWrap property.

Migration

Update HTML to not output font tags. Use CSS instead. In case you really want to use the font tag, it can be created by `stdWrap.wrap`.

2.7.4 Important

Important: #75747 - EXT:form - Removed support for compatibility6

See Issue #75747

Description

EXT:compatibility6 contains the prehistoric CType `mailform`. Since compatibility6 is not available for TYPO3 v8 the compatibility between EXT:form and the prehistoric CType `mailform` has been removed.

2.8 8.1 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

2.8.1 Breaking Changes

Breaking: #66861 - Do not automatically append a "/" to the identifier of a folder

See Issue #66861

Description

The `Folder` object automatically appended a `/` to the identifier. But as the `Folder` object should not manipulate the folder identifier this is removed.

Impact

Installations with custom FAL driver(s) could break.

Affected Installations

Installations with a custom FAL driver that relies on the `/` being added by the Folder object.

Migration

All drivers that depend on this `/` being added should be adjusted so the driver handles this.

Breaking: #70056 - Http-related options and HttpRequest class removed

See Issue #70056

Description

The following PHP classes have been removed:

- `TYPO3\CMS\Core\Http\HttpRequest`
- `TYPO3\CMS\Core\Http\Observer\Download`

The following configuration options have been removed:

- $TYPO3_CONF_VARS[SYS][curlUse]
- $TYPO3_CONF_VARS[SYS][curlProxyNTLM]
- $TYPO3_CONF_VARS[SYS][curlProxyServer]
- $TYPO3_CONF_VARS[SYS][curlProxyTunnel]
- $TYPO3_CONF_VARS[SYS][curlProxyUserPass]
- $TYPO3_CONF_VARS[SYS][curlTimeout]
- $TYPO3_CONF_VARS[HTTP][adapter]
- $TYPO3_CONF_VARS[HTTP][protocol_version]
- $TYPO3_CONF_VARS[HTTP][follow_redirects]
- $TYPO3_CONF_VARS[HTTP][max_redirects]
- $TYPO3_CONF_VARS[HTTP][strict_redirects]
- $TYPO3_CONF_VARS[HTTP][proxy_host]
- $TYPO3_CONF_VARS[HTTP][proxy_port]
- $TYPO3_CONF_VARS[HTTP][proxy_user]
- $TYPO3_CONF_VARS[HTTP][proxy_password]
- $TYPO3_CONF_VARS[HTTP][proxy_auth_scheme]

- $TYPO3_CONF_VARS[HTTP][ssl_verify_peer]
- $TYPO3_CONF_VARS[HTTP][ssl_verify_host]
- $TYPO3_CONF_VARS[HTTP][ssl_cafile]
- $TYPO3_CONF_VARS[HTTP][ssl_capath]
- $TYPO3_CONF_VARS[HTTP][ssl_local_cert]
- $TYPO3_CONF_VARS[HTTP][ssl_passphrase]
- $TYPO3_CONF_VARS[HTTP][userAgent]

The following properties have been renamed:

- $TYPO3_CONF_VARS[HTTP][userAgent] is now called $TYPO3_CONF_VARS[HTTP][headers][User-Agent]
- $TYPO3_CONF_VARS[HTTP][protocol_version] is now called $TYPO3_CONF_VARS[HTTP][version]
- All proxy-related options are unified within $TYPO3_CONF_VARS[HTTP][proxy]
- All redirect-related options (HTTP/follow_redirects, HTTP/max_redirects, HTTP/strict_redirects) are unified within $TYPO3_CONF_VARS[HTTP][allow_redirects]
- All options related to SSL private keys (HTTP/ssl_local_cert, HTTP/ssl_passphrase) are merged into $TYPO3_CONF_VARS[HTTP][ssl_key]
- All options related to verify SSL peers are merged into $TYPO3_CONF_VARS[HTTP][verify]

Additionally, the dependency to the PEAR Package "Http_Request2" (composer package name `pear/http_request2`) has been removed in favor of the PHP library Guzzle.

Impact

Calling the mentioned classes above will result in a fatal PHP error.

Using the options in custom PHP code will result in unexpected behavior as the options are non-existent and empty.

Using PHP code that depends on the removed PEAR library "Http_Request2" will result in unexpected behaviour and possibly a fatal PHP error.

Affected Installations

All 3rd party extensions calling the mentioned classes directly or using the configuration options directly, as well as installations depending on the PEAR library "Http_Request2".

Migration

For PHP code previously using the `HttpRequest` and `Download` classes a new object-oriented PSR-7-based approach is introduced, see the Guzzle Feature integration documentation for more details. A new PHP class `TYPO3\CMS\Core\Http\RequestFactory` which generates PSR-7 compliant request objects helps in simplifying the migration process.

All still necessary options will be migrated to new options within `$TYPO3_CONF_VARS[HTTP]` when the install tool is run.

In special cases, the options `$TYPO3_CONF_VARS[HTTP][ssl_verify_host]`, `$TYPO3_CONF_VARS[HTTP][proxy_auth_scheme]` and `$TYPO3_CONF_VARS[HTTP][proxy_host]` need to migrated manually to the newly available options.

Breaking: #72931 - SearchFormController::pi_list_browseresults() has been renamed

See Issue #72931

Description

In order to make Indexed Search pi-based plugin PHP7 compatible, the `SearchFormController::pi_list_browseresults()` method has been renamed to `SearchFormController::renderPagination()`. Parameter types, order and count have been preserved. However the methods visibility has been changed to protected.

Impact

Call to old method name will result in fatal error "Call to undefined method".

Affected Installations

Any installation of TYPO3 7.6 or TYPO3 8 where `SearchFormController` is overloaded (XCLASSed) and the new class contains a call to the old method name and any code that called the public method from outside the class.

Migration

Rename `pi_list_browseresults()` to `renderPagination()`.

Calling the method from outside the class is no longer possible.

Breaking: #75237 - Removal of div ce-bodytext might cause layout issues

See Issue #75237

Description

If neither bodytext nor header were entered in the TextMedia element, the div element containing the class ce-bodytext will be suppressed.

Impact

The missing div could cause layout problems, if the layout and CSS depend on it.

Affected Installations

All installations relying on

Migration

Either change the CSS or use a custom template without all the conditions.

Breaking: #75323 - Removed parameter entryPointPath from main applications

See Issue #75323

Description

The entry point `PHP` classes for `index.php`, `typo3/index.php` and so forth (called "Application classes") now have a parameter not to define the path to the entry point but the number of subdirectories under the main installation path, allowing to not specify the name of the path, but just the levels of subdirectories.

Subsequently, the methods `Bootstrap->baseSetup()`, `Bootstrap->redirectToInstallTool()` and `SystemEnvironmentBuilder::run()` now expect an integer as parameter, instead of the path to the entry point script.

Impact

Calling one of the methods above with a string as parameter instead of an integer will fail because the calculation for PATH_site which is the base for the whole installation will fail.

Affected Installations

Any installation with custom entry points or custom extensions with separate entry points.

Migration

Use the entry point level as integer, instead of the string, in your custom entry points. See `TYPO3\CMS\Backend\Http\Application` for an example.

Breaking: #75324 - ReferenceIndex CLI command changed

See Issue #75324

Description

The Reference Index Updater Command Line command has been changed to use a Symfony Command.

The command to update the reference index on non-composer-mode installations is now called on the command line via `typo3/sysext/core/bin/typo3 referenceindex:update`.

To just check the reference index, the option `-c` (alternatively the property "check" can be used) is used like this `typo3/sysext/core/bin/typo3 referenceindex:update -c`

For installations set up via composer, the typo3 CLI binary is available in the "bin/" directory directly inside the project root.

The command can be used like this:

```
# update the reference index
bin/typo3 referenceindex:update
# check the reference index
bin/typo3 referenceindex:update -c
bin/typo3 referenceindex:update --check
```

The additional option –silent does not output anything when running the CLI command.

Impact

Calling the command via the old syntax `typo3/cli_dispatch.phpsh lowlevel refindex` will not work anymore.

Affected Installations

Any existing installation upgrading to TYPO3 v8 with a (e.g. cron) CLI script, running the reference index update via the `typo3/cli_dispatch.phpsh`.

Migration

Change the CLI scripts inside your installation to the new binary path.

Breaking: #75349 - Move Indexed Search pi-based plugin to compatibility7

See Issue #75349

Description

Indexed Search pi1 plugin (based on AbstractPlugin) has been moved to EXT:compatibility7 and will not be developed further. EXT:compatibility7 will be moved to TER before the release of 8 LTS. The Extbase plugin (pi2) stays in Indexed Search as before.

Impact

Installation of EXT:compatibility7 is required to continue using the pi1 plugin. In the longer run migrating to the Extbase plugin is required.

Affected Installations

All installations using the pi-based indexed search plugin.

Migration

Installations using pi1 should migrate to the Extbase plugin or install EXT:compatibility7.

Breaking: #75355 - FlexForm-related options removed

See Issue #75355

Description

The following options regarding FlexForm handling have been removed.

- $TYPO3_CONF_VARS[BE][niceFlexFormXMLtags]
- $TYPO3_CONF_VARS[BE][compactFlexFormXML]

Storing FlexForms in the database via the TYPO3 Core API does not compact the XML files anymore (it now always uses 4 spaces for indentation inside the XML) and also always uses meaningful tags so it can be validated with DTDs.

Impact

Having the options set will result in new/updated FlexForm data being written with spaces instead of tabs and with meaningful tags while keeping full backwards compatibility when reading.

This was the default for new installations already, but introduced due to legacy installations (pre 4.0) that dealt with FlexForms back in 2004.

Affected Installations

Any installation having these properties set in their LocalConfiguration.php.

Any extension evaluating these parameters on its own.

Breaking: #75357 - $TYPO3_CONF_VARS[BE][lockSSL] option is boolean

See Issue #75357

Description

The setting `$TYPO3_CONF_VARS[BE][lockSSL]` which forces requests to the TYPO3 Backend to be transferred via SSL, has been changed to only allow boolean values.

The settings previously allowed three options: * lockSSL set to "0" - Don't force a SSL connection at all * lockSSL set to "1" - If the incoming request to the TYPO3 backend is a non-SSL request, an exception was thrown * lockSSL set to "2" - If the incoming request is a non-SSL-request, redirect to the SSL-enabled URL

The option "1" has been removed without substitution, allowing the following variants: * lockSSL set to "false" - Don't force a SSL connection at all * lockSSL set to "true" - If the incoming request is a non-SSL-request, redirect to the SSL-enabled URL

Impact

If the option was set previously to "1", the exception is not thrown but a redirect will now happen. The same behavior as the existing option "2".

Affected Installations

TYPO3 instances having the option above set to "1".

Breaking: #75454 - LocalConfiguration DB config structure has changed

See Issue #75454

Description

To provide support for multiple database connections and remapping tables to different database systems within the TYPO3 Core the configuration format for database connections in `LocalConfiguration.php` / `$GLOBALS['TYPO3_CONF_VARS']['DB']` has changed.

The new configuration array structure:

```
'DB' => [
        'Connections' => [
                'Default' => [
                        'driver' => 'mysqli',
                        'dbname' => 'typo3_database',
                        'password' => 'typo3',
                        'host' => '127.0.0.1',
                        'port' => 3306,
                        'user' => 'typo3',
                        'unix_socket' => '',
                        'charset' => 'utf-8',
                ],
        ],
],
```

Be aware that besides the deeper nesting below 'Connections/Default' some of the configuration keys have been renamed. It is required to provide the new configuration key `driver` with a value of `mysqli` explicitly.

The following table lists the changed configuration keys and the appropriate values if these have changed.

Old name	New name
DB/username	DB/Connections/Default/user
DB/password	DB/Connections/Default/password
DB/host	DB/Connections/Default/host
DB/port	DB/Connections/Default/port
DB/socket	DB/Connections/Default/unix_socket
DB/database	DB/Connections/Default/dbname
SYS/setDBinit	DB/Connections/Default/initCommands
SYS/no_pconnect	DB/Connections/Default/persistentConnection
SYS/dbClient-Compress	DB/Connections/Default/driverOptions Valid values for MySQLi connections: 0 compression disabled 32 compression enabled

Impact

Connections to the database will fail with an exception until the configuration has been migrated to the new structure.

Affected Installations

All Installations

Migration

The Install Tool will migrate the configuration information for the default connection to the new format. Installations overriding the database configuration using `AdditionalConfiguration.php` or other means need to ensure the new format is being used.

Breaking: #75454 - TYPO3_db Constants removed

See Issue #75454

Description

The PHP constants `TYPO3_db`, `TYPO3_db_username`, `TYPO3_db_password` and `TYPO3_db_host` which were used when TYPO3 initialized the database connection have been removed.

Impact

Checking for or using the mentioned constants may lead to unexpected behavior or errors. If not checked if the constant even was defined, the application will stop immediately.

Affected Installations

Any installation which uses a third-party extension using these constants.

Migration

Use the configuration data within `$GLOBALS['TYPO3_CONF_VARS']['DB']['Connections']['Default']` to determine the username, password and host information for the default database connection.

Breaking: #75497 - inline backend layout wizard

See Issue #75497

Description

The `BackendLayoutWizardController` has been removed and a new renderType has been added to render the backend layout wizard inline in FormEngine.

Also the backend route `wizard_backend_layout` has been removed.

Impact

Extending or using the `BackendLayoutWizardController` will break installations.

Affected Installations

Any installation which uses an extension which makes use of `BackendLayoutWizardController`

Migration

Use the renderType `belayoutwizard`, which renders the backend layout wizard inline in FormEngine.

Breaking: #75708 - Always store <p> tags in DB from RTE

See Issue #75708

Description

When transforming HTML data from the Rich Text Editor to the database, the RteHtmlParser removed <p> tags from lines when there were no attributes for the <p> tags, otherwise they were kept as <p> tags with their attributes.

The transformation now always keeps <p> tags within the content in order to minimize the transformation overhead between the RTE and the database.

Impact

Every time an RTE field is edited, the <p> tags are now stored inside the database when saving the content.

Affected Installations

All installations using RTE fields or RteHtmlParser transformations.

Migration

An upgrade wizard inside the Install Tool (coming until 8.1) will make sure that any database RTE field is converted.

Breaking: #75711 - Removed DB-related methods and TCA-related options from cObj

See Issue #75711

Description

The following methods have been removed from `ContentObjectRenderer` without substitution:

- DBgetDelete()
- DBgetUpdate()
- DBgetInsert()
- DBmayFEUserEdit()
- DBmayFEUserEditSelect()
- exec_mm_query()
- exec_mm_query_uidList()

The following TCA options have no effect anymore throughout the TYPO3 Core:

- $GLOBALS['TCA'][table]['ctrl']['fe_cruser_id']
- $GLOBALS['TCA'][table]['ctrl']['fe_crgroup_id']
- $GLOBALS['TCA'][table]['ctrl']['fe_admin_lock']

Impact

Calling any of the methods above directly will trigger a PHP fatal error.

Affected Installations

Any TYPO3 installation using DB-related Frontend Administration with the obsolete functionality.

Breaking: #75829 - Removed handling of pre 6.0 files when importing

See Issue #75829

Description

The functionality for importing .t3d files created from installations lower than TYPO3 CMS 6.0 has been removed.

The following public method has been removed: :php‘TYPO3CMSImpexpImport::fixCharsets()‘.

Impact

Importing files into TYPO3 v8 that were created from a TYPO3 4.x installations will result in unexpected behavior, especially when dealing with files and relations.

Calling the PHP method above will result in a fatal PHP error.

Affected Installations

Any installation using the t3d import functionality for importing files that were created from a TYPO3 instance older than TYPO3 6.0.0.

Migration

It is recommended to import files in a 6.x or 7.x installation and export the files from there again to import them in TYPO3 v8.

2.8.2 Features

Feature: #27471 - Allow asterisk for hideTables

See Issue #27471

Description

It is now possible to hide all tables from list view via PageTS-Config.

You want to show only a specific table, you can hide all tables and unhide only the specific one.

```
mod.web_list {
   hideTables = *
   table.tx_cal_event.hideTable = 0
}
```

Feature: #39597 - Multiple locale names for TypoScript config.locale_all

See Issue #39597

Description

The TypoScript option `config.locale_all` now allows to set locale fallbacks as a comma-separated list, as the underlying PHP function `setlocale()` does as well.

```
config.locale_all = de_AT@euro, de_AT, de_DE, deu_deu
```

Feature: #69439 - Enhance SQL query reduction in page tree in workspaces

See Issue #69439

Description

The process of determining whether a page has workspace versions can be extended by custom application code utilizing hooks. This way, the meaning of having versions can be further modified by hooks. For instance the default behavior of the TYPO3 core is to create a workspace version record on persisting the same record in the backend - without any actual changes to the data model.

- $GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['TYPO3\CMS\Workspaces\Service\WorkspaceService']['hasPageRec + $parameters['workspaceId']: The submitted workspace ID + $parameters['pageId']: The submitted page ID + $parameters['versionsOnPageCache']: Reference to the state array
- $GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['TYPO3\CMS\Workspaces\Service\WorkspaceService']['fetchPages\ + $parameters['workspaceId']: The submitted workspace ID + $parameters['pageId']: The submitted page ID + $parameters['pagesWithVersionsInTable']: Reference to the state array

Impact

The hooks introduce the possibility to modify the determined results - only if those hooks are used.

Feature: #70056 - Added PHP library "Guzzle" for HTTP Requests within TYPO3

See Issue #70056

Description

The PHP library `Guzzle` has been added via composer dependency to work as a feature rich solution for creating HTTP requests based on the PSR-7 interfaces already used within TYPO3.

Guzzle auto-detects available underlying adapters available on the system, like cURL or stream wrappers and chooses the best solution for the system.

A TYPO3-specific PHP class called `TYPO3\CMS\Core\Http\RequestFactory` has been added as a simplified wrapper to access Guzzle clients.

All options available under `$TYPO3_CONF_VARS[HTTP]` are automatically applied to the Guzzle clients when using the `RequestFactory` class. The options are a subset to the available options on Guzzle (http://docs.guzzlephp.org/en/latest/request-options.html) but can further be extended.

Existing `$TYPO3_CONF_VARS[HTTP]` options have been removed and/or migrated to the new Guzzle-compliant options.

A full documentation for Guzzle can be found at http://docs.guzzlephp.org/en/latest/.

Although Guzzle can handle Promises/A+ and asynchronous requests, it currently acts as a drop-in replacement for the previous mixed options and implementations within `GeneralUtility::getUrl()` and a PSR-7-based API for HTTP requests.

Impact

The existing TYPO3-specific wrapper `GeneralUtility::getUrl()` now uses Guzzle under the hood automatically for remote files, removing the need to configure settings based on certain implementations like stream wrappers or cURL directly.

The `RequestFactory` class can be used like this:

```
// Initiate the Request Factory, which allows to run multiple requests
/** @var \TYPO3\CMS\Core\Http\RequestFactory $requestFactory */
$requestFactory = GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Http\RequestFactory\RequestFactory::class);
$url = 'https://typo3.com';
$additionalOptions = [
   // Additional headers for this specific request
   'headers' => ['Cache-Control' => 'no-cache'],
   // Additional options, see http://docs.guzzlephp.org/en/latest/request-options.
↪html
   'allow_redirects' => false,
   'cookies' => true
];
// Return a PSR-7 compliant response object
$response = $requestFactory->request($url, 'GET', $additionalOptions);
// Get the content as a string on a successful request
if ($response->getStatusCode() === 200) {
   if ($response->getHeader('Content-Type') === 'text/html') {
      $content = $response->getBody()->getContents();
   }
}
```

Extension authors are advised to use the `RequestFactory` class instead of using the Guzzle API directly in order to ensure a clear upgrade path when updates to the underlying API need to be done.

Feature: #72923 - Configure the number of files shown per page in file list module

See Issue #72923

Description

It is now possible to configure the maximum number of files shown per page in the file list module using User TSconfig.

```
options.file_list {
   filesPerPage = 40
}
```

Feature: #75386 - Get identifier in slide callback

See Issue #75386

Description

The callback of `Wizard.addSlide()` now has a new parameter `identifier`.

Impact

The identifier is passed to the callback function of `Wizard.addSlide()`.

Example code:

```
Wizard.addSlide('my-identifier', 'Foobar', '', Severity.info, function($slide,␣
↪settings, identifier) {
        console.log(identifier); // my-identifier
});
```

Feature: #75454 - Added PHP library "Doctrine DBAL" for Database Connections within TYPO3

See Issue #75454

Description

The PHP library `Doctrine DBAL` has been added via composer dependency to work as a powerful database abstraction layer with many features for database abstraction, schema introspection and schema management within TYPO3.

A TYPO3-specific PHP class called `TYPO3\CMS\Core\Database\ConnectionPool` has been added as a manager for database connections.

All connections configured below `$GLOBALS['TYPO3_CONF_VARS']['DB']['Connections']` are accessible using this manager, enabling the parallel usage of multiple database systems.

By using the database abstraction options and the QueryBuilder provided SQL statements being built will be properly quoted and compatible with different DBMS out of the box as far as possible.

Existing `$GLOBALS['TYPO3_CONF_VARS']['DB']` options have been removed and/or migrated to the new Doctrine-compliant options.

Documentation for Doctrine DBAL can be found at http://www.doctrine-project.org/projects/dbal.html.

The `Connection` class provides convenience methods for `insert`, `select`, `update`, `delete` and `truncate` statements. For `select`, `update` and `delete` only simple equality comparisons (`WHERE "aField" = 'aValue'`) are supported. For complex statements it's required to use the `QueryBuilder`.

It is possible to swap tables out of the default database and use a specific setup (e.g. for logging or caching). This snippet of a LocalConfiguration could for exampe be used to swap the sys_log table out to another database or even another database server:

```
//LocalConfiguration.php
//[...]
'DB' => [
    'Connections' => [
        'Default' => [
            'charset' => 'utf8',
            'dbname' => 'default_dbname',
            'driver' => 'mysqli',
            'host' => 'default_host',
            'password' => '***',
            'port' => 3306,
            'user' => 'default_user',
        ],
        'Syslog' => [
            'charset' => 'utf8',
            'dbname' => 'syslog_dbname',
            'driver' => 'mysqli',
            'host' => 'syslog_host',
            'password' => '***',
```

```
            'port' => 3306,
            'user' => 'syslog_user',
        ],
    ],
    'TableMapping' => [
        'sys_log' => 'Syslog'
    ]
],
//[...]
```

Impact

Currently the `DatabaseConnection` class only uses Doctrine to establish the database connection to MySQL, no advanced options are being used yet.

Connections always need to be requested with a table name so that the abstraction of table names to database connections stays intact.

The `ConnectionPool` class can be used like this:

```
// Get a connection which can be used for multiple operations
/** @var \TYPO3\CMS\Core\Database\Connecction $conn */
$conn = GeneralUtility::makeInstance(ConnectionPool::class)->getConnectionForTable(
↪'aTable');
$affectedRows = $conn->insert(
   'aTable',
   $fields, // Associative array of column/value pairs, automatically quoted &␣
↪escaped
);
```

```
// Get a QueryBuilder, which should only be used a single time
$query = GeneralUtility::makeInstance(ConnectionPool::class)->
↪getQueryBuilderForTable('aTable');
$query->select('*')
   ->from('aTable)
   ->where(
      $query->expr()->eq('aField', $query->createNamedParameter($aValue)),
      $query->expr()->lte(
         'anotherField',
         $query->createNamedParameter($anotherValue)
      )
   );
$rows = $query->execute()->fetchAll();
```

Extension authors are advised to use the `ConnectionPool` and `Connection` classes instead of using the Doctrine DBAL directly in order to ensure a clear upgrade path when updates to the underlying API need to be done.

Feature: #75497 - inline backend layout wizard

See Issue #75497

Description

A new renderType was added to render the backend layout wizard inline in FormEngine.

Impact

The old `BackendLayoutWizardController` which has rendered the backend layout wizard in a popup has been removed.

Use the new renderType `belayoutwizard` to render the backend layout wizard inline in FormEngine.

example TCA configuration:

```
'config' => array(
   'label' => 'LLL:EXT:frontend/Resources/Private/Language/locallang_tca.xlf:
↪backend_layout.config',
   'config' => array(
      'type' => 'text',
      'renderType' => 'belayoutwizard',
   )
)
```

Feature: #75579 - Add markupIdentifier support to JavaScript IconAPI

See Issue #75579

Description

It is now possible to request alternative rendering methods also through the JavaScript IconAPI for the backend. A new parameter has been added to the `getIcon` function that now accepts the `markupIdentifier` for alternative rendering output, as it's also possible within PHP.

Currently this is only used by the `SvgIconProvider` to deliver inline-SVGs instead of linking them in an `img` tag.

Example 1: default, without alternativeMarkup

```
require(['TYPO3/CMS/Backend/Icons'], function(Icons) {
   var iconName = 'actions-view-list-collapse';
   Icons.getIcon(iconName, Icons.sizes.small).done(function(icon) {
      console.log(icon);
   });
});
```

Example 2: with alternativeMarkup = inline

```
require(['TYPO3/CMS/Backend/Icons'], function(Icons) {
   var iconName = 'actions-view-list-collapse';
   Icons.getIcon(iconName, Icons.sizes.small, null, null, 'inline').
↪done(function(icon) {
      console.log(icon);
   });
});
```

Feature: #75581 - Simplify cache clearing

See Issue #75581

Description

The cache clearing system has been simplified by removing options in cache clear menu and install tool.

The cache clear menu in the backend contains now only two options:

- Flush frontend caches Clear frontend and page-related caches, like before.
- Flush all caches Clear all system-related caches, including the class loader, localization, extension configuration file caches and opcode caches. Rebuilding this cache may take some time.

Within the install tool the "Clear all cache" button will now also clear the opcode caches if possible.

Feature: #75827 - Add configuration options to TYPO3CMSExtbasePropertyTypeConverterFloatConverter

See Issue #75827

Description

It is now possible to define the thousands separator and decimal point for `FloatConverter`. This can be used to ensure proper sanitation before converting a string to a float.

You can define the configuration for every property like this:

```
$this->arguments['<argumentName>']
   ->getPropertyMappingConfiguration()
   ->forProperty('<propertyName>') // this line can be skipped in order to specify␣
↪the format for all properties
   ->setTypeConverterOption(
      \TYPO3\CMS\Extbase\Property\TypeConverter\FloatConverter::class,
      \TYPO3\CMS\Extbase\Property\TypeConverter\FloatConverter::CONFIGURATION_
↪THOUSANDS_SEPARATOR,
      '.'
   )
   ->setTypeConverterOption(
      \TYPO3\CMS\Extbase\Property\TypeConverter\FloatConverter::class,
      \TYPO3\CMS\Extbase\Property\TypeConverter\FloatConverter::CONFIGURATION_
↪DECIMAL_POINT,
      ','
   );
```

2.8.3 Deprecation

Deprecation: #73209 - Deprecated flex page TSConfig

See Issue #73209

Description

Setting page TSConfig values `PAGE_TSCONFIG_ID`, `PAGE_TSCONFIG_IDLIST` and ' PAGE_TSCONFIG_STR' for flexform fields globally has been marked as deprecated, specific fields must be set now.

Example for a now deprecated global TSConfig value:

```
TCEFORM.tt_content.pi_flexform.PAGE_TSCONFIG_ID = 42
```

This should now be restricted to specific fields of the flexfrom data structure, if for example the flexform `foreign_table_where` of field `settings.categories` of a `tt_content` plugin's data structure should be set, the new page TSConfig option should look like:

```
TCEFORM.tt_content.pi_flexform.theDataStructure.theSheet.settings\.categories.PAGE_TSCON
= 42
```

Note that any dots within the field name must be escaped with '\', this is a typical scenario for extbase.

Impact

This pageTSConfig cannot be set for section elements anymore: `PAGE_TSCONFIG_ID`, `PAGE_TSCONFIG_IDLIST` and `PAGE_TSCONFIG_STR` do not have any effect on repeatable elements.

Affected Installations

Installations that set `PAGE_TSCONFIG_ID`, `PAGE_TSCONFIG_IDLIST` and `PAGE_TSCONFIG_STR` for flexform fields globally should be restricted to set those values for single elements.

Migration

Search for `PAGE_TSCONFIG_ID`, `PAGE_TSCONFIG_IDLIST` and `PAGE_TSCONFIG_STR` and restrict them to single fields as outlined above.

Deprecation: #73728 - Wizard type colorbox is deprecated

See Issue #73728

Description

The color-picker is now available as dedicated render-type which will integrate an inline color-picker widget based on bootstrap. Thus, the old wizard type `colorbox` has been marked as deprecated.

Impact

Using the TCA wizard type `colorbox` will trigger a deprecation log entry. The possibility to pick the color from a custom image has been removed without substitution together with the possibility to use color names like "red" or "white".

Affected Installations

All TCA fields that are using the wizard type `colorbox`, like e.g.

```
$GLOBALS['TCA']['tableName']['fieldName']['config']['wizards']['colorbox'] = [
   'type' => 'colorbox',
   'script' => 'wizard_colorpicker.php',
   ...
];
```

Migration

Use the new render-type `colorpicker` in the TCA field configuration, like e.g.

```
$GLOBALS['TCA']['tableName']['fieldName']['config']['renderType'] = 'colorpicker';
```

Deprecation: #75327 - $TSFE->csConvObj and $TSFE->csConv()

See Issue #75327

Description

The public property `csConvObj` and the public method `csConv()` inside the TypoScriptFrontendController PHP class have been marked as deprecated.

Impact

Calling `$TSFE->csConv()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation which uses the property or the method directly.

Migration

If a charset conversion is necessary, the conversion can be done directly by instantiating the charset converter class.

```
$from = 'iso-8859-15';
/** @var \TYPO3\CMS\Core\Charset\CharsetConverter $charsetConverter */
$charsetConverter = GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Charset\CharsetConverter::class);
$output = $charsetConverter->conv($str, $charsetConverter->parse_charset($from),
↪'utf-8');
```

Deprecation: #75340 - Methods related to generating traditional Backend AJAX URLs

See Issue #75340

Description

The following methods have been marked as deprecated:

- TYPO3CMSBackendUtilityBackendUtility->getAjaxUrl()
- TYPO3CMSBackendRoutingUriBuilder->buildUriFromAjaxId()

Impact

Calling one of the methods above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance with a third party extension that calls one of the methods above.

Migration

Migrate to UriBuilder routes, which can be registered via Configuration/Backend/AjaxRoutes.php, and can be linked to like this:

```
/** @var \TYPO3\CMS\Backend\Routing\UriBuilder $uriBuilder */
$uriBuilder = GeneralUtility::makeInstance(\TYPO3\CMS\Backend\Routing\UriBuilder::
↪class);
$path = $uriBuilder->buildUriFromRoute('ajax_myroute');
```

Keep in mind that the newly created Ajax routes need to implement PSR-7 as well.

Deprecation: #75371 - array2xml_cs

See Issue #75371

Description

The method `GeneralUtility::array2xml_cs()` has been marked as deprecated.

Impact

Using the method `GeneralUtility::array2xml_cs()` will trigger a deprecation log entry.

Affected Installations

All installations with third party extensions using this method are affected.

Migration

Use `GeneralUtility::array2xml()` instead. The XML declaration must be written manually.

Example code: .. code-block:: php

> // Deprecated $bodyText = GeneralUtility::array2xml_cs($array, 'phparray');
>
> // Migrated $bodyText = '<?xml version="1.0" encoding="utf-8" standalone="yes" ?>' . LF . GeneralUtility::array2xml($array, '', 0, 'phparray');

Deprecation: #75575 - TranslateViewHelper htmlEscape argument marked as deprecated

See Issue #75575

Description

The htmlEscape argument of the TranslateViewHelper has been marked as deprecated.

This ViewHelper now HTML escapes the translation by default. The argument value has no effect anymore.

Impact

Usages of `<f:translate>` view helper with argument set to `false` will have the label HTML escaped anyway.

Usages of `<f:translate>` view helper with argument set to `true` will have the label HTML escaped like before unless the view helper is wrapped with a `<f:format.raw>`

Affected Installations

Installations with usages of `<f:translate>` in a context where HTML escaping is not desired (e.g. JavaScript).

Migration

`<f:translate>` needs to be wrapped by `<f:format.raw>` if the view helper result is needed in a different context than HTML

Deprecation: #75621 - GeneralUtility methods

See Issue #75621

Description

The following methods within `GeneralUtility` have been marked as deprecated:

- compat_version()
- convertMicrotime()
- deHSCentities()
- slashJS()
- rawUrlEncodeJS()
- rawUrlEncodeFP()
- lcfirst()
- getMaximumPathLength()

The second parameter of `GeneralUtility::wrapJS()` has been removed.

Impact

Calling any of the methods above will trigger a deprecation log entry.

Calling `GeneralUtility::wrapJS()` with the second parameter will trigger a PHP notice message.

Affected Installations

Any installation with a third-party extension calling one of the methods in its PHP code.

Migration

For the following methods, use the native PHP methods and constants directly that are used within these methods:

- compat_version()
- convertMicrotime()
- deHSCentities()
- slashJS()
- rawUrlEncodeJS()
- rawUrlEncodeFP()

- lcfirst()
- getMaximumPathLength()

Deprecation: #75625 - Deprecated cache clearing options

See Issue #75625

Description

The following commands have been marked as deprecated and should not be used anymore:

- Method `DataHandler->clear_cacheCmd()` with arguments `system` and `temp_cached`
- `userTSconfig` setting `options.clearCache.system`
- Option `$TYPO3_CONF_VARS['SYS']['clearCacheSystem']` has been removed

Impact

Directly or indirectly using method `clear_cacheCmd` with these arguments will trigger a deprecation log entry.

Affected Installations

All installations with third party extensions using this method are affected.

Migration

If the group of system caches needs to be deleted explicitly, use `flushCachesInGroup('system')` of `CacheManager` directly.

2.8.4 Important

Important: #73041 - PackageStates Includes Only Active Packages

See Issue #73041

Description

The information about available packages in the system located in typo3conf/PackageStates.php was thinned out to only include the extension keys of the active (= installed) extensions.

2.9 8.0 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*

- *Important*

2.9.1 Breaking Changes

Breaking: #43085 - Change GFX settings prefix im_* to generic processor_*

See Issue #43085

Description

Graphics processor settings for Image- or GraphicsMagick have been renamed in `LocalConfiguration.php`. The former prefix `im\_` has been replaced with the unified prefix `processor\_`.

Negative namings like `noScaleUp` have been changed positive counterparts. During the conversion the previous configuration values are negated to reflect the changes in semantics of these options.

In addition references to specific versions of ImageMagick/GraphicsMagick have been removed from settings names and values. For a detailed list of changes please consult the information in the migration section.

The unused configuration option `image\_processing` has been removed without replacement.

The processor specific configuration option `colorspace` has been namespaced below the `processor\_` hierarchy.

Impact

Existing settings in `LocalConfiguration.php` are automatically migrated through a silent upgrader when entering the Install Tool. If you modify the settings in `AdditionalConfiguration.php` or rely on them inside an extension you need to update those.

Affected Installations

Installations which modify those settings directly or access them.

Migration

The following table lists the changed configuration keys and the appropriate values if these have changed.

Old name	New name
im_version_5	processor The configuration value “im6” has been replaced by “ImageMagick”, “gm” by “GraphicsMagick”
im	processor_enabled
im_v5effects	processor_effects
im_noScaleUp	processor_allowUpscaling
im_noFramePrepended	processor_allowFrameSelection
im_mask_temp_ext_gif	processor_allowTemporaryMasksAsPng
im_path	processor_path
im_path_lzw	processor_path_lzw
im_stripProfileCommand	processor_stripColorProfileCommand
im_useStripProfileByDefault	processor_stripColorProfileByDefault
colorspace	processor_colorspace

Breaking: #45899 - Split class ImportExport into classes Import and Export

See Issue #45899

Description

Class TYPO3CMSImpexpImportExport (typo3/sysext/impexp/Classes/ImportExport.php) has been split into a class dedicated for import and another one for export.

Impact

Using and extending the class is not possible anymore.

Affected Installations

Extensions using the class and its methods directly or extending the class.

Migration

Use or extend one or both of the new classes (TYPO3CMSImpexpImport and TYPO3CMSImpexpExport).

Breaking: #45943 - Remove unused Images in "t3skin/icons/gfx/i"

See Issue #45943

Description

A lot of unused icons from `EXT:t3skin/icons/gfx/i` have been removed.

Impact

References to the images of EXT:t3skin/icons/gfx/i will throw a 404 not found.

Affected Installations

Installations or extensions which have references to icons in EXT:t3skin/icons/gfx/i.

Migration

No migration

Breaking: #51099 - Streamline settings/conditions

See Issue #51099

Description

The default TypoScript for the `pi2` (extbase) plugin of EXT:indexed_search sets a TypoScript variable `plugin.tx_indexedsearch.settings.displayRules = 1`, while the default fluid template uses the TypoScript setting `plugin.tx_indexedsearch.settings.showRules`. This change makes the default fluid template for the extbase plugin of EXT:indexed_search use the correct TypoScript setting.

Impact

Instances of the extbase plugin (`pi2`) of EXT:indexed_search using the default fluid template and explicitly configured to *hide* the search rules deliberately using the wrong TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0` *will show* the search rules after this update.

Affected Installations

Installations using the extbase plugin (`pi2`) of EXT:indexed_search with default template and relying on the undocumented TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0`.

Migration

Change all occurrences of the TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0` to `plugin.tx_indexedsearch.settings.displayRules = 1`.

Breaking: #65165 - AdditionalMethodsInFolderInterface

See Issue #65165

Description

The interface `FolderInterface` has received two additional methods. Classes that implement `FolderInterface` have to implement those methods as well. The new methods are:

- `getModificationTime()` - Returns the modification time of the folder as Unix timestamp.
- `getCreationTime()` - Returns the creation time of the folder as Unix timestamp.

Impact

Classes implementing the `FolderInterface` no longer fulfill the requirements of the interface.

Affected Installations

Installations that use custom implementations of the `FolderInterface`.

Migration

Implement the two new methods in custom implementations of the `FolderInterface`.

Breaking: #68890 - Remove dual-use of auth_timeout_field in AbstractUserAuthentication

See Issue #68890

Description

In `AbstractUserAuthentication` class the property `auth_timeout_field` could previously either contain the name of a field or a timeout-value in seconds. To specify a field name the property can be used as before. To specify a timeout-value, a new property called `sessionTimeout` is introduced that can be set to an integer >= 0.

Impact

If some extension reads the value, the default is changed from an integer (0) to an empty string.

Migration

Extensions modifying `auth_timeout_field` to a numeric value should switch to using `sessionTimeout`.

Breaking: #69863 - Changes in ViewHelpers post Standalone-Fluid

See Issue #69863

Description

The following ViewHelpers have changed behaviours in Fluid:

- The `f:case` ViewHelper argument `default` has been marked as deprecated. To indicate which case is the default, use `f:defaultCase`.
- Tag content of `f:render` is no longer ignored and will be output if called with `optional="1"`.
- Arguments `iconOnly` and `styleAttributes` have been removed from `f:be.buttons.csh`.
- Argument `alternateBackgroundColors` has been removed from `f:be.tableList`.
- ViewHelpers no longer use the `escapingInterceptorEnabled` property but instead use `escapeChildren` and `escapeOutput` to control each behavior.
- All ViewHelpers no longer initialize standard arguments ("additionalArguments" and "data") in `__construct()`, but instead do this in `initializeArguments()`. If you override this method, you need to make sure you include a call to `parent::initializeArguments()` in your subclass.

The following ViewHelper classes are now only found in namespace `TYPO3Fluid\Fluid\ViewHelpers` and no longer exist in `TYPO3\CMS\Fluid\ViewHelpers`:

- `TYPO3\CMS\Fluid\ViewHelpers\AliasViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\SwitchViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\CaseViewHelper` (present as deprecated alias until final removal)
- `TYPO3\CMS\Fluid\ViewHelpers\CommentViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\CycleViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\ForViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\GroupedForViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\IfViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\ThenViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\ElseViewHelper`

- `TYPO3\CMS\Fluid\ViewHelpers\LayoutViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\RenderViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\SectionViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\SpacelessViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\Format\CdataViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\Format\HtmlspecialcharsViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\Format\PrintfViewHelper`
- `TYPO3\CMS\Fluid\ViewHelpers\Format\RawViewHelper`

Impact

- A warning about use of an unregistered argument `default` will be displayed if templates contain `f:case` with `default` argument.
- Unexpected template output will be output if templates are rendered which contain `<f:render partial/section optional="1">will be output now</f:render>`.
- A warning about use of an unregistered argument `iconOnly` and/or `styleAttributes` will be displayed if templates contain `f:be.buttons.csh` with either argument.
- A warning about use of an unregistered argument `alternateBackgroundColors` will be displayed if templates contain `f:be.tableList` with that argument.
- Any third-party ViewHelpers subclassing any of the classes listed above must change parent class to new namespace.
- Any third-party ViewHelpers using `escapingInterceptorEnabled` property to disable escaping.

Affected Installations

Any TYPO3 instance that uses a template which contains:

- An `f:case` with `default` argument.
- An `f:render` with `optional="1"` and having content in the `<f:render>` tag.
- An `f:be.buttons.csh` with either `iconOnly` or `styleAttributes` (value irrelevant).
- An `f:be.tableList` with `alternateBackgroundColors` (value irrelevant).
- Any third-party ViewHelper which subclasses any of the classes listed above.
- Any third-party ViewHelper which uses `escapingInterceptorEnabled` property to disable escaping.

Migration

- Remove the `default` option and change `f:case` to `f:defaultCase` for that case.
- Remove the tag contents of `f:render`.
- Remove arguments `iconOnly` and `styleAttributes` from `f:be.buttons.csh` where found.
- Remove argument `alternateBackgroundColors` from `f:be.tableList` where found.
- Update namespace of parent class in ViewHelpers subclassing any of the classes listed above.
- Update ViewHelper class to use `escapeChildren` and/or `escapeOutput` depending on desired behavior.

Breaking: #69863 - Fluid escaping behaviour changed from ENT_COMPAT to ENT_QUOTES

See Issue #69863

Description

The escaping behaviour in Fluid has been changed. Before, `ENT_COMPAT` was used. Now, `ENT_QUOTES` is used.

Impact

Fluid templates which depend on single quotes not being escaped when escaping variables. Affects ObjectAccessor (variable access in general) and calls to `f:format.htmlentities` and `f:format.htmlspecialchars`.

Affected Installations

Any TYPO3 site containing Fluid templates which depend on single quotes not being escaped.

Migration

Change template to not depend on single quotes being escaped in any ObjectAccessor, consider adding `{variable -> f:format.htmlspecialchars(keepQuotes: 1)}` or `<f:format.htmlspecialchars keepQuotes="1">{variable}</f:format.htmlentities>` when accessing variables but be aware of possible XSS implications due to incomplete escaping.

Breaking: #69863 - Removed deprecated code from EXT:fluid

See Issue #69863

Description

Removed deprecated code from EXT:fluid

The ChangeLog file has been removed.

The renderMode option in `FlashMessagesViewHelper` has been removed.

The following methods have been removed:

```
StandaloneView::setLayoutRootPath          StandaloneView::getLayoutRootPath
StandaloneView::setPartialRootPath         StandaloneView::getPartialRootPath
AbstractFormFieldViewHelper::getValue
```

The following class has been removed:

```
IconViewHelper
```

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Relying on the renderMode option might lead to different frontend output.

Affected Installations

Instances which use calls to the methods above, use the removed `IconViewHelper or use the renderMode option in `FlashMessagesViewHelper`.`

Migration

For `StandaloneView::setLayoutRootPath` use `StandaloneView::setLayoutRootPaths` instead. For `StandaloneView::getLayoutRootPath` use `StandaloneView::getLayoutRootPaths` instead. For `StandaloneView::setPartialRootPath` use `StandaloneView::setPartialRootPaths` instead. For `StandaloneView::getPartialRootPath` use `StandaloneView::setPartialRootPaths` instead.

Keep in mind that these methods expect an **array** instead of a string.

For `IconViewHelper` use `\TYPO3\CMS\Core\ViewHelpers\IconViewHelper` instead.

Breaking: #71458 - FullQuoteArray can't handle boolean values for $noQuote

See Issue #71458

Description

The API for `fullQuoteArray` allows the parameter `$noQuote` to be boolean but converted it automatically to false as `$noQuote` is neither a string nor an array. This behavior has been fixed, passing `true` for `$noQuote` now disables quoting of any passed in values.

Impact

Passing in boolean `true` results in escaping being disabled for all values.

Affected Installations

All installations making use of `INSERTmultipleRows()`, `INSERTquery()`, `UPDATEquery()` or `fullQuoteArray()` and relying on the fact that quoting remains enabled when `true` is passed as value for `$noQuote`.

Migration

Pass the correct list of fields to disable quoting for unless none of the fields should be quoted.

Breaking: #71521 - Property userAuthentication removed from CommandController

See Issue #71521

Description

The property `$userAuthentication` was removed from the Extbase `CommandController` class and has been migrated to the newly introduced `getBackendUserAuthentication()` method.

Impact

All command controllers deriving from `CommandController` with see a fatal error when accessing properties or methods of the removed `$userAuthentication` property.

Migration

Use the newly introduced `getBackendUserAuthentication()` method.

Breaking: #72022 - Removed class loading fallback in cObject

See Issue #72022

Description

The method `ContentObjectRenderer->isClassAvailable()` was used internally to check for a TypoScript property `plugin.tx_myextension_pi1.includeLibs` that included a PHP file when `class_exists()` failed.

With TYPO3 CMS 7, the spl_autoload mechanism checks for all places within extensions, alternatively composer does this on build-time. All needed classes are known, making this check and the option obsolete.

The functionality was introduced in TYPO3 4.3 before autoloading was available, and has now been removed.

Impact

The option `.includeLibs` on a plugin TypoScript object has no effect anymore.

Breaking: #72293 - API change in ExceptionHandlerInterface

See Issue #72293

Description

The class `\Throwable` was added in PHP7 as new parent of `\Exceptions`. So we were in need to remove the type hint in #72117, which we now read as `\Throwable` instead of `\Exception` as we only support PHP 7.0 or newer. See http://php.net/manual/en/migration70.incompatible.php

Impact

A fatal error will be thrown if you use own ExceptionHandlers implementing `TYPO3\CMS\Core\Error\ExceptionHandlerInterface` "Fatal error: Declaration of ... must be compatible with ..."

Affected Installations

Installations which use an own ExceptionHandler implementing TYPO3s `ExceptionHandlerInterface`.

Migration

Add `\Throwable` as type hinting in your implementation of `ExceptionHandlerInterface`. Check the API/type hinting of the method where you process the exception.

Breaking: #72310 - EXT:form - Outsource labels and legends to own partials

See Issue #72310

Description

Labels and legends have been outsourced to their own partials. This step is slightly (but thankfully) breaking.

With this change the duplication of code can be avoided. This helps the integrator to customize the labels/ legends with just one small and central override.

Impact

No deep impact. If an EXT:form template was overridden, it mostly contains the `label` and/or `legend` tags and acts like it used to do.

Affected Installations

Any installation using EXT:form since TYPO3 7.5.

Migration

Overridden EXT:form partials could be migrated to use the new central label/ legend partials.

Example changes for `Resources/Private/Partials/Default/Show/FlatElements/Checkbox.html`.

Old:

```
<label for="{model.additionalArguments.id}">
    {model.additionalArguments.label}
    <f:if condition="{model.mandatoryValidationMessages}">
        <em><f:for each="{model.mandatoryValidationMessages}" as=
↪"mandatoryValidationMessage" iteration="iterator">{mandatoryValidationMessage}<f:
↪if condition="{iterator.isLast}"><f:else> - </f:else></f:if></f:for></em>
    </f:if>
    <f:if condition="{model.validationErrorMessages}">
        <strong><f:for each="{model.validationErrorMessages}" as=
↪"errorValidationMessage" iteration="iterator">{errorValidationMessage}<f:if␣
↪condition="{iterator.isLast}"><f:else> - </f:else></f:if></f:for></strong>
    </f:if>
</label>
```

New:

```
{f:render(partial: '{themeName}/Show/AdditionalElements/Label', arguments: {model:␣
↪model, themeName: themeName})}
```

Breaking: #72334 - Removed utf8 conversion in EXT:recycler

See Issue #72334

Description

The recycler module previously handled conversions of labels to and from UTF-8 in order to send proper UTF-8 encoded data via JavaScript. The TYPO3 backend is running with UTF-8 since TYPO3 4.5.

The logic and the according functions have been removed as they are not needed anymore.

Impact

The following methods have been removed:

```
RecyclerUtility::getUtf8String()
RecyclerUtility::isNotUtf8Charset()
RecyclerUtility::getCurrentCharset()
```

Affected Installations

Any TYPO3 instance directly accessing any of the mentioned `RecyclerUtility` methods above via a custom extension.

Migration

Remove the usages to these methods, and use the strings directly.

Breaking: #72338 - Removed GraphicalFunctions->nativeCharset

See Issue #72338

Description

The property `nativeCharset` to allow GifBuilder to use other character-sets than UTF-8 for rendering text for has been removed. The default behaviour is now to always consider multi-byte strings via CharsetConverter, as the data is expected to be UTF-8 at all times.

Additionally the methods `recodeString()` and `singleChars()` have been removed as the direct equivalent from CharsetConverter is used.

Impact

Calling any of the two methods above directly in PHP will result in a fatal error.

Setting `$nativeCharset` to something else than UTF-8 will have no effect anymore.

Affected Installations

Installations with custom setups and third-party PHP code using GifBuilder or GraphicalFunctions and the `$nativeCharset` option.

Migration

Use `CharsetConverter->utf8_to_numberarray()` instead of the method `singleChars()`.

Breaking: #72342 - Removed deprecated code from GeneralUtility

See Issue #72342

Description

The following methods have been removed from `GeneralUtility`

`gif_compress()` `png_to_gif_by_imagemagick()` `read_png_gif()` `modifyHTMLColor()` `modifyHTMLColorAll()` `isBrokenEmailEnvironment()` `normalizeMailAddress()` `formatForTextarea()` `inArray()` `removeArrayEntryByValue()` `keepItemsInArray()` `addSlashesOnArray()` `stripSlashesOnArray()` `slashArray()` `remapArrayKeys()` `array_merge()` `arrayDiffAssocRecursive()` `naturalKeySortRecursive()` `getThisUrl()` `readLLfile()` `quoted_printable()` `encodeHeader()` `substUrlsInPlainText()` `cleanOutputBuffers()`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the methods above.

Migration

For `gif_compress()` use `\TYPO3\CMS\Core\Imaging\GraphicalFunctions::gifCompress()` instead. For `png_to_gif_by_imagemagick()` use `\TYPO3\CMS\Core\Imaging\GraphicalFunctions::pngToG` instead. For `read_png_gif()` use `\TYPO3\CMS\Core\Imaging\GraphicalFunctions::readPngGif()` instead. For `inArray()` use `ArrayUtility::inArray()` instead. For `removeArrayEntryByValue()` use `ArrayUtility::removeArrayEntryByValue()` instead. For `keepItemsInArray()` use `ArrayUtility::keepItemsInArray()` instead. For `remapArrayKeys()` use `ArrayUtility::remapArrayKeys()` instead. For `array_merge()` use native php '+' operator instead. For `arrayDiffAssocRecursive()` use `ArrayUtility::arrayDiffAssocRecursive()` instead. For `naturalKeySortRecursive()` use `ArrayUtility::naturalKeySortRecursive()` instead. For `getThisUrl()` use `GeneralUtility::getIndpEnv*` instead. For `quoted_printable()` use mailer API instead. For `encodeHeader()` use mailer API instead. For `substUrlsInPlainText()` use mailer API instead. For `cleanOutputBuffers()` use ob_* functions directly or `self::flushOutputBuffers`.

Breaking: #72360 - Removed deprecated entry point fallback

See Issue #72360

Description

The entry point fallback mechanism introduced with #68812 has been removed.

The following entry points won't work anymore.

```
typo3/ajax.php
typo3/alt_clickmenu.php
typo3/alt_db_navframe.php
typo3/alt_doc.php
```

```
typo3/alt_file_navframe.php
typo3/browser.php
typo3/db_new.php
typo3/dummy.php
typo3/init.php
typo3/login_frameset.php
typo3/logout.php
typo3/mod.php
typo3/move_el.php
typo3/show_item.php
typo3/tce_db.php
typo3/tce_file.php
typo3/thumbs.php
```

Impact

All references / links to these entry points directly without using the proper API calls will result in a 404 error.

Affected Installations

Installations with third-party extensions that link directly to these files.

Migration

Move all existing code in extensions that link to the deprecated entry points to use methods like `BackendUtility::getModuleUrl()` and `BackendUtility::getAjaxUrl()` or the UriBuilder class.

Breaking: #72361 - Removed deprecated content object wrappers

See Issue #72361

Description

The following wrapper methods for ContentObject rendering within ContentObjectRenderer have been removed:

```
FLOWPLAYER()
TEXT()
CLEARGIF()
COBJ_ARRAY()
USER()
FILE()
FILES()
IMAGE()
IMG_RESOURCE()
IMGTEXT()
CONTENT()
RECORDS()
HMENU()
CTABLE()
OTABLE()
COLUMNS()
HRULER()
CASEFUNC()
LOAD_REGISTER()
```

```
FORM()
SEARCHRESULT()
TEMPLATE()
FLUIDTEMPLATE()
MULTIMEDIA()
MEDIA()
SWFOBJECT()
QTOBJECT()
SVG()
```

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to ContentObjects via the methods above.

Migration

Replace the direct method calls to `$contentObject->COBJECT()` with the common method.

Example for the SVG() ContentObject call:

```
$cObj->cObjGetSingle('SVG', $conf);
```

Breaking: #72361 - Removed deprecated methods in ContentObjectRenderer

See Issue #72361

Description

The following methods have been removed:

- `cleanFormName`
- `stdWrap_offsetWrap`
- `textStyle`
- `tableStyle`

The TypoScript `jumpurl` configuration has been removed for file links. The TypoScript property `andWhere` from `.select` has been removed.

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to ContentObjects via the methods above.

Migration

`cleanFormName` has been moved to `FormContentObject`. `stdWrap_offsetWrap` has not been replaced by a new function. `textStyle` TypoScript option should be done with CSS. `tableStyle` TypoScript option should be done with CSS.

TypoScript option `jumpurl` can be passed in the `typolinkConfiguration` property.

TypoScript property `andWhere` can be migrated to `where`.

Breaking: #72368 - TYPO3 Constants removed

See Issue #72368

Description

The PHP constants `TYPO3_enterInstallScript` and `TYPO3_cliMode` and the global variable `$GLOBALS['TYPO3_AJAX']` which were used when a TYPO3 Request was initialized have been removed. They have been replaced by an alternative to use the `TYPO3_REQUESTTYPE` constant at the very beginning of each TYPO3 request.

Impact

Checking for the mentioned constants and global variable have no effect anymore and may lead to unexpected behaviour.

If not checked if the constant even was defined, the application will stop immediately.

Affected Installations

Any installation which uses a third-party extension using these constants.

Migration

Use `TYPO3_REQUESTTYPE & TYPO3_REQUESTTYPE_CLI` or `TYPO3_REQUESTTYPE & TYPO3_REQUESTTYPE_INSTALL` instead.

Breaking: #72370 - Removed deprecated code from extbase

See Issue #72370

Description

Remove deprecated code from extbase

The ChangeLog file has been removed. `TYPO3\CMS\Extbase\DomainObject\AbstractDomainObject::__wakeup()` has been removed. `TYPO3\CMS\Extbase\Utility\ExtensionUtility::configureModule()` has been removed.

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the methods above.

Migration

Use the according method in `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::configureModule()` instead of the removed one from ext:extbase. Objects are instantiated differently calling `parent::__wakeup()` is no longer necessary. No migration needed.

Breaking: #72372 - Removed deprecated code from beuser

See Issue #72372

Description

The following methods have been removed from `PermissionAjaxController`

`renderOwnername` `renderPermissions` `renderGroupname`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the methods above.

Migration

Migrate your code that calls one of the methods to Fluid templates.

Breaking: #72373 - Removed deprecated code from css_styled_content

See Issue #72373

Description

The following methods have been removed from `CssStyledContentController`

`render_bullets` `render_uploads` `beautifyFileLink`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the methods above.

Migration

Use default TypoScript from CSS Styled Content derived from the current version.

Breaking: #72378 - Removed CSS Styled Content TypoScript for 6.2

See Issue #72378

Description

The compatibility TypoScript code for CSS Styled Content, which renders the Frontend output to behave like TYPO3 CMS 6.2, has been removed.

Impact

Referencing the file via sys_template or including the TypoScript files in `EXT:css_styled_content/static/6.2/` will not work anymore.

Affected Installations

Any installation that still uses the compatibility TypoScript code from TYPO3 CMS 6.2.

Migration

Use the current TypoScript used in CSS Styled Content.

Breaking: #72381 - Removed deprecated code from EXT:dbal

See Issue #72381

Description

The following methods of `\TYPO3\CMS\Dbal\Database\DatabaseConnection` have been removed:

- `MySQLActualType`
- `MySQLMetaType`
- `MetaType`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to DatabaseConnection class via the methods above.

Migration

`MySQLActualType` call `dbmsSpecifics->getNativeFieldType` instead `MySQLMetaType` call `dbmsSpecifics->getMetaFieldType` instead `MetaType` call `getMetadata` instead

Breaking: #72384 - Removed deprecated code from HtmlParser

See Issue #72384

Description

The following methods have been removed from HtmlParser:

- `getSubpart()`
- `substituteSubpart()`
- `substituteSubpartArray()`
- `substituteMarker()`
- `substituteMarkerArray()`
- `substituteMarkerAndSubpartArrayRecursive()`
- `XHTML_clean()`
- `processTag()`
- `processContent()`

The following method has been removed from RteHtmlParser:

- `evalWriteFile`

The TSconfig option `xhtml_cleaning` has been removed as well.

Impact

Using the methods above directly in any third party extension will result in a fatal error. Setting the xhtml processing option when parsing HTML has no effect anymore as well.

Affected Installations

Instances which use custom calls to HtmlParser via the methods above.

Migration

`getSubpart()` use `MarkerBasedTemplateService::getSubpart()` instead `substituteSubpart()` call `MarkerBasedTemplateService::substituteSubpart()` instead `substituteSubpartArray()` call `MarkerBasedTemplateService::substituteSubpartArray()` instead `substituteMarker()` call `MarkerBasedTemplateService::substituteMarker()` instead `substituteMarkerArray()` call `MarkerBasedTemplateService::substituteMarkerArray()` instead `substituteMarkerAndSubpartArrayRecursive()` call `MarkerBasedTemplateService::substitute` instead `XHTML_clean()` call `HtmlParser::HTMLcleaner()` instead

Breaking: #72385 - Removed deprecated code from DataHandler

See Issue #72385

Description

Removed deprecated code from DataHandler

The following properties have been removed:

`stripslashes_values` `clear_flexFormData_vDEFbase` `include_filefunctions`

The following methods have been removed:

`checkValue_text()` `checkValue_input()` `checkValue_check()` `checkValue_radio()` `checkValue_group_select()` `checkValue_flex()`

Additionally, the method `getLocalTCE()` does not accept any parameter anymore.

Impact

Using one of these methods or properties will throw a fatal error.

Affected Installations

An extension relying on one of these public properties or methods will fail.

Migration

These functions are internal and should not be used outside of the core.

Breaking: #72390 - Removed deprecated code from EXT:rtehtmlarea

See Issue #72390

Description

The following methods have been removed:

- `UserElementsController::main()`
- `UserElementsController::printContent()`
- `ParseHtmlController::main()`
- `ParseHtmlController::printContent()`

Furthermore a JavaScript function has been removed:

- `initEventListeners()`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to UserElementsController, ParseHtmlController via the methods above, or rely on the JavaScript function mentioned above being executed.

Migration

`initEventListener()` no replacement for this `UserElementsController::main()` call `UserElementsController::main_user()` instead `UserElementsController::printContent()` call `UserElementsController::mainAction()` instead `ParseHtmlController::main()` call `ParseHtmlController::main_parse_html()` instead `ParseHtmlController::printContent()` call `ParseHtmlController::mainAction()` instead

Breaking: #72392 - Removed deprecated code from DocumentTemplate

See Issue #72392

Description

Remove deprecated code from DocumentTemplate

The following properties have been removed:

`JScodeLibArray` `docType` `inDocStyles` `endJS` `bgColor` `bgColor2` `bgColor3` `bgColor4` `bgColor5` `bgColor6` `hoverColor` `backGroundImage` `inDocStyles_TBEstyle` `parseTimeFlag` `charset`

The following methods have been removed:

`getPageRenderer()` `wrapClickMenuOnIcon()` `issueCommand()` `formatTime()` `parseTime()` `spacer()` `endPageJS()` `dfw()` `rfw()` `table()` `menuTable()` `getDynamicTabMenu()` `getDynTabMenu()` `getDynTabMenuId()` `collapseableSection()`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to any of the above mentioned methods.

Migration

For `wrapClickMenuOnIcon()` use `BackendUtility::wrapClickMenuOnIcon()` instead. For `issueCommand()` use `BackendUtility::getLinkToDataHandlerAction()` instead. For `formatTime()` and `parseTime()` use the corresponding methods in BackendUtility. For `rfw()` and `dfw()` use proper HTML directly instead. For `getDynamicTabMenu()` use `getDynamicTabMenu()` from ModuleTemplate instead. For `collapseableSection()` use HTML bootstrap classes, localStorage etc.

Breaking: #72398 - Removed deprecated code from EXT:recordlist

See Issue #72398

Description

The following deprecated methods have been removed:

- `RecordList::printContent()`

- `ElementBrowserFramesetController::printContent()`

The following deprecated data members have been removed:

- `RecordList::$MCONF`

Support for multiple UIDs in the URL parameter `act` in `AbstractLinkBrowserController::initVariables()` has been removed.

Impact

Using the methods or variables above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to RecordList, AbstractLinkBrowserController, ElementBrowserFramesetController via the methods above, or use one of the variables mentioned above.

Migration

`$MCONF` no replacement for this `RecordList::printContent()` use `RecordList::mainAction()` instead `AbstractLinkBrowserController::initVariables()` no replacement for using multiple UIDs `ElementBrowserFramesetController::printContent()` use `ElementBrowserFramesetController::mainAction()` instead

Breaking: #72399 - Removed deprecated code from BackendUtility

See Issue #72399

Description

Remove deprecated code from BackendUtility

The following methods have been removed:

`getExcludeFields()` `getExplicitAuthFieldValues()` `getSystemLanguages()` `getRegisteredFlexForms()` `implodeTSParams()` `getThumbNail()` `helpTextIcon()` `getUrlToken()` `exec_foreign_table_where_query()` `replaceMarkersInWhereClause()` `RTEgetObj()` `countVersionsOfRecordsOnPage()` `getPathType_web_nonweb()` `isTableMovePlaceholderAware()`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to one of the above mentioned methods.

Migration

For `helpTextIcon()` use `cshItem()` instead. For `isTableMovePlaceholderAware()` use `isTableWorkspaceEnabled()` directly. For `countVersionsOfRecordsOnPage()` use `\TYPO3\CMS\Workspaces\Service\WorkspaceService::hasPageRecordVersions` to check for record versions.

Breaking: #72400 - Removed deprecated IconUtility and SpriteManager

See Issue #72400

Description

Removed deprecated IconUtility class completely. All SpriteManager related code has been removed from the core.

The Install Tool option `BE/spriteIconGenerator_handler` has no effect anymore.

`Bootstrap::initializeSpriteManager()` has been removed.

Impact

Using the static class IconUtility or the SpriteManager will result in a fatal error.

Affected Installations

Instances which use custom calls to one of the above mentioned classes.

Migration

Use the new introduced IconAPI that is available since 7LTS.

Breaking: #72405 - Removed traditional BE modules handling

See Issue #72405

Description

The traditional way of registering backend modules done via custom `mod1/index.php` and `mod1/conf.php` has been removed.

Impact

Calling `ExtensionManagementUtility::addModulePath()` will result in a fatal error. Additionally, all modules that are registered via `ExtensionManagementUtility::addModule()` and setting a path will not be registered properly anymore.

`$TBE_MODULES['_PATHS']` is always empty now. Additionally, the options `script` and `navFrameScript` and `navFrameScriptParam` will have no effect anymore when registering a module.

Affected Installations

Any installation using an extension that registers a module via the traditional way using standalone scripts.

Migration

Use the option `routeTarget` when registering a module, and PSR-7 equivalent entry-points in module controllers.

Breaking: #72412 - Removed deprecated code from language processing functions

See Issue #72412

Description

The following deprecated code has been removed:

- `LocalizationFactory::getParsedData` no support for moved language files
- class `LocallangArrayParser` has been removed completely

The following deprecated methods have been removed:

- `LanguageService::localizedFileRef()`

Impact

Using old locations of language file will result in no text being displayed. Using the removed class will result in a fatal error. Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use old locations of language files, instances which use the removed class LocallangArrayParser, instances which use calls to the methods above.

Migration

`LocalizationFactory::getParsedData` only supports the new location of language files `LocallangArrayParser` use XLIFF language files now `\TYPO3\CMS\Lang\LanguageService::localizedFileRef` no replacement; not needed when XLIFF files are used

Breaking: #72416 - Remove EXT:t3skin/stylesheets/sprites/

See Issue #72416

Description

The sprites and related icons from EXT:t3skin/stylesheets/sprites/ have been removed.

Impact

References to the sprites or images of EXT:t3skin/ will throw a 404 not found.

Affected Installations

Installations or extensions which have references to icons in EXT:t3skin/images/icons/* or EXT:t3skin/stylesheets/sprites/.

Migration

No migration, remove all references and use the IconFactory for all icon related stuff.

Breaking: #72417 - Removed old locking API

See Issue #72417

Description

The old locking mechanism was replaced by a more sophisticated a robust LockFactory, and is now completely removed from the TYPO3 Core.

Impact

Using the `Locker` class will result in a fatal error. The option `$TYPO3_CONF_VARS[SYS][lockingMode]` has no effect anymore.

Breaking: #72418 - Deprecated backend-related PHP classes

See Issue #72418

Description

The following PHP classes have been removed:

- `TYPO3\CMS\Backend\Module\ModuleController`
- `TYPO3\CMS\Backend\Module\ModuleSettings`
- `TYPO3\CMS\Backend\View\LogoView`
- `TYPO3\CMS\Backend\View\ModuleMenuView`
- `TYPO3\CMS\Backend\View\PageLayout\ExtDirect\ExtdirectPageCommands`
- `TYPO3\CMS\Backend\View\ThumbnailView`

Impact

Calling any of these PHP classes directly will result in a fatal error.

Affected Installations

Any installation with a custom PHP code accessing these PHP classes.

Breaking: #72419 - Remove deprecated code from backend controllers

See Issue #72419

Description

Remove deprecated code from backend controllers

The following methods have been removed:

`BackendController::getPageRenderer` `BackendController::addToolbarItem` `ClickMenuController::init` `ClickMenuController::main` `ClickMenuController::printContent` `ElementInformationController::printContent` `MoveElementController::printContent` `NewContentElementController::getWizardItems` `DummyController::printContent` `EditDocumentController::printContent` `EditDocumentController::editRegularContentFromId` `FileSystemNavigationFrameController::printContent` `LoginFramesetController::printContent` `NewRecordController::printContent` `SimpleDataHandlerController::finish` `ColorpickerController::printContent` `EditController::closeWindow` `RteController::printContent` `TableController::printContent`

The following classes have been removed completely:

`ListFrameLoaderController` `PageTreeNavigationController`

Impact

Using the one of the methods or classes above will result in a fatal error.

Affected Installations

Instances which use custom calls to the methods or classes above.

Migration

For `BackendController::addToolbarItem` Toolbar items are registered in $GLOBALS['TYPO3_CONF_VARS']['BE']['toolbarItems'] now.

Breaking: #72421 - Removed deprecated code from database and query functions

See Issue #72421

Description

Removed deprecated code from database and query functions

The following methods have been removed:

`DatabaseConnection::splitGroupOrderLimit` `QueryGenerator::formatQ` `QueryGenerator::JSbottom` `ReferenceIndex::error` `RelationHandler::convertPosNeg`

The following properties have been removed:

`QueryGenerator::$extJSCODE` `ReferenceIndex::$errorLog`

The option to set soft reference parsers has been removed.

Impact

Using the one of the methods or properties above will result in a fatal error.

Affected Installations

Instances which use custom calls to the methods or classes above.

Breaking: #72424 - Removed deprecated TypoScriptFrontendController options and methods

See Issue #72424

Description

The following methods from `TypoScriptFrontendController` have been removed:

- `getPageRenderer()`
- `setExternalJumpUrl()`
- `jumpUrl()`
- `acquirePageGenerationLock()`
- `releasePageGenerationLock()`
- `doXHTML_cleaning()`
- `doLocalAnchorFix()`
- `checkFileInclude()`
- `prefixLocalAnchorsWithScript()`
- `getStorageSiterootPids()`

Additionally, the public properties `jumpurl`, `JSeventFuncCalls` and `anchorPrefix` have been removed. The request parameter `jumpurl` is not evaluated anymore.

The TypoScript property `config.additionalHeaders` has been removed.

Impact

Calling any of the PHP methods directly will result in a fatal error. Accessing the properties will result in a PHP warning. Setting the TypoScript property has no effect anymore.

Additionally, if EXT:felogin is misconfigured and lacks the `storagePid` property, an exception will be thrown.

Affected Installations

Any installation using the TypoScript property above, or a TYPO3 instance having third-party extensions calling the methods or properties directly.

Migration

Use the TER extension `jumpurl` to implement the jumpurl functionality.

Use the `config.additionalHeaders` subproperties (see https://docs.typo3.org/typo3cms/TyposcriptReference/Setup/Config/Index.html#additionalheaders for details) to add the additional header lines.

Breaking: #72426 - Removed deprecated code from file and image processing functions

See Issue #72426

Description

The following deprecated methods have been removed:

- `LocalImageProcessor::getTemporaryImageWithText`
- `ResourceCompressor::compressCssPregCallback`
- `FileList::getButtonsAndOtherMarkers`
- `GraphicalFunctions::pngToGifByImagemagick`

The following deprecated data members have been removed:

- `DuplicationBehavior::$legacyValueMap`
- `ExtendeFileUtillity::$dontCheckForUnique`
- `FileListController::$MCONF`

Impact

Using the methods or variables above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to LocalImageProcessor, ResourceCompressor, FileList, GraphicalFunctions via the methods above, or use one of the variables mentioned above.

Migration

`LocalImageProcessor::getTemporaryImageWithText` use `\TYPO3\CMS\Core\Imaging\GraphicalFunctio` instead `ResourceCompressor::compressCssPregCallback` no replacement, functionality is implemented in a different way `ExtendeFileUtillity::$dontCheckForUnique` use `setExistingFilesConflictMode(DuplicationBehavior::REPLACE)` instead `FileListController::$MCONF` no replacement, configuration is done when registering the module in ext_tables.php `FileList::getButtonsAndOtherMarkers` buttons are now defined in FileListController `GraphicalFunctions::pngToGifByImagemagick` no replacement, the png_to_gif option has been removed

Breaking: #72427 - Removed TypoScript-related methods and properties

See Issue #72427

Description

The following methods and properties have been removed:

- `TYPO3\CMS\Core\TypoScript\ConfigurationForm::ext_getKeyImage()`
- `TYPO3\CMS\Core\TypoScript\ExtendedTemplateService::ext_noSpecialCharsOnLabels`
- `TYPO3\CMS\Core\TypoScript\ExtendedTemplateService::makeHtmlspecialchars()`
- `TYPO3\CMS\Core\TypoScript\ExtendedTemplateService::ext_getKeyImage()`
- `TYPO3\CMS\Core\TypoScript\TemplateService::tempPath`
- `TYPO3\CMS\Core\TypoScript\TemplateService::wrap()`
- `TYPO3\CMS\T3editor\T3editor::isEnabled()`
- `TYPO3\CMS\Tstemplate\Controller\TypoScriptTemplateObjectBrowserModuleFunctionControl`

The TypoScript conditions "browser", "version", "device", "system" and "useragent" have been removed.

Impact

Calling the methods above will result in a PHP fatal error.

Using the removed TypoScript conditions will have no effect anymore.

Breaking: #72431 - Remove deprecated code from lowlevel and utility functions

See Issue #72431

Description

The following deprecated methods have been removed:

- `ConfigurationView->printContent()`
- `DatabaseIntegrityView->printContent()`
- `StringUtility::isLastPartOfString()`
- `Bootstrap->executeExtTablesAdditionalFile()`
- `DatabaseTreeDataProvider->emitDeprecatedPostProcessTreeDataSignal()`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the methods above. Instances which use TYPO3_extTableDef_script for TCA overrides. Instances which use the signal `TYPO3\\CMS\\Core\\Tree\\TableConfiguration\\TableConfiguration\\DatabaseTreeDataProvider`

Migration

- `StringUtility::isLastPartOfString()` use endsWith() instead
- `Bootstrap->executeExtTablesAdditionalFile()` (TYPO3_extTableDef_script) Move your TCA overrides to Configuration/TCA/Overrides of your project specific extension, or slot the signal "tcaIsBeingBuilt" for further processing.
- `DatabaseTreeDataProvider->emitDeprecatedPostProcessTreeDataSignal()` Update the signal name to TYPO3\CMS\Core\Tree\TableConfiguration\DatabaseTreeDataProvider.

Breaking: #72438 - Remove deprecated code from FlashMessage

See Issue #72438

Description

The deprecated `render()` method has been removed.

Impact

Using the `render()` method directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to the `render()` method.

Migration

For FlashMessages that are displayed on top of a page you can replace the `render()` method with code that enqueues the message to the FlashMessageService.

Replace

```
$flashMessage = GeneralUtility::makeInstance(\TYPO3\CMS\Core\Messaging\FlashMessage:
↪:class,
$message,
$title,
\TYPO3\CMS\Core\Messaging\FlashMessage::ERROR);
$content .= $flashMessage->render();
```

with

```
$flashMessage = GeneralUtility::makeInstance(\TYPO3\CMS\Core\Messaging\FlashMessage:
↪:class,
$message,
$title,
\TYPO3\CMS\Core\Messaging\FlashMessage::ERROR);
$flashMessageService = GeneralUtility::makeInstance(FlashMessageService::class);
$defaultFlashMessageQueue = $flashMessageService->getMessageQueueByIdentifier();
$defaultFlashMessageQueue->enqueue($flashMessage);
```

FlashMessages that are used as inline notification should be removed and replaced with custom HTML code.

For the core we have defined output and usage for messages:

1) FlashMessages

FlashMessages are designed to inform a user about success or failure of an action, which was **triggered** by the user. Example: If the user deletes a record, a FlashMessage informs the user about success or failure. This kind of information is not static, it is a temporary and volatile information and triggered by a user action.

Keep in mind that you **must not** use HTML markup here, since this information might be shown in a context different from HTML, like processing it via Javascript or showing the message on the command line.

2) Callouts (InfoBox-ViewHelper)

Callouts are designed to display permanent information, a very good example is the usage in the Page-Module. If a user opens a system folder with the page module, the callout explains: 'Hey, you try to use the page module on a sysfolder, please switch to the list module'. This ViewHelper can also be used to show some help or instruction how to use a backend module.

3) Any other information

For any other information e.g. a list of files which has changed, must be handled in the action / view of the module or plugin. This is not a use case for a FlashMessage or Callout! Example: Display a list of a hundred files within a FlashMessage or Callout is a bad idea, build custom markup in the view to handle this kind of message.

Breaking: #72451 - Removed deprecated code from backend functions

See Issue #72451

Description

Removed deprecated code from backend functions

The following methods have been removed:

`ClickMenu::wrapColorTableCM` `ClickMenu::excludeIcon` `ContextMenuAction::getClass` `ContextMenuAction::setClass` `SuggestWizardDefaultReceiver::getIcon` `BackendUserAuthentication::checkCLIuser` `PageFunctionsController::printContent` `InfoModuleController::printContent`

The following display condition option have been removed:

- Evaluates conditions concerning extensions
- Evaluates whether the field is a value for the default language.

The property `$OS` has been removed from `BackendUserAuthentication`

The property `$doc` has been removed from `InfoModuleController`

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use custom calls to one of the methods above.

Migration

For property `$OS` use the constant TYPO3_OS directly.

Breaking: #72462 - Removed deprecated JavaScript code

See Issue #72462

Description

Removed deprecated JavaScript code

The following JavaScript functions have been removed:

`showClickmenu_raw` `Clickmenu.show` `Clickmenu.populateData` `ShortcutManager.createShortcut` `jsfunc.tbe_editor.getBackendPath`

Impact

Using one of the methods above will result in JavaScript errors in the TYPO3 CMS backend.

Affected Installations

Instances which use custom calls to one of the methods above.

Migration

For `Clickmenu.show` use `TYPO3.ClickMenu` instead. For `Clickmenu.populateData` use `TYPO3.ClickMenu` instead. For `showClickmenu_raw` use `TYPO3.ClickMenu` instead. For `ShortcutManager.createShortcut` use `TYPO3.ShortcutMenu` directly.

Breaking: #72464 - Removed deprecated code from EXT:workspaces

See Issue #72464

Description

The following components have been changed during the development of TYPO3 CMS 7 and lead to deprecated code which is removed in TYPO3 CMS 8.

- remove rewritten toolbar item (#62800)
- remove rewritten notification parts (#35245)

Impact

Using or extending `\TYPO3\CMS\Workspaces\ExtDirect\ToolbarMenu` will fail since it has been removed.

Using TYPO3CMSWorkspacesServiceStagesService::getNotificationMode($stageId) will fail.

Relying on the following database fields in the tables sys_workspace and sys_workspace_stage will fail: * sys_workspace.edit_notification_mode * sys_workspace.publish_notification_mode * sys_workspace.execute_notification_mode * sys_workspace_stage.notification_mode

Affected Installations

All installations using workspaces and notifications that have not been migrated to TYPO3 CMS 7, yet.

Migration

First migrate to TYPO3 CMS 7 and use the accordant upgrade wizard (WorkspacesNotificationSettingsUpdate) and then upgrade to TYPO3 CMS 8.

Breaking: #72473 - Removed deprecated miscellaneous functions

See Issue #72473

Description

Removed deprecated miscellaneous functions

The following methods have been removed:

```
FlexFormTools::getAvailableLanguages      AbstractPlugin::pi_list_searchBox
ImportExportController::printContent    SearchFormController::checkExistance
SearchFormController::checkExistence      SchedulerModuleController::render
SchedulerModuleController::checkDate     SetupModuleController::printContent
TypoScriptTemplateModuleController::printContent VersionModuleController::printContent
```

Last parameter `addTofeInterface` for `ExtensionManagementUtility::addTCAcolumns` has been removed.

The following options in the Install Tool have been removed:

```
FE\strictFormmail       FE\secureFormmail       FE\formmailMaxAttachmentSize
SC_OPTIONS\GLOBAL\softRefParser_GL'
```

Impact

Using the methods above directly in any third party extension will result in a fatal error.

Using one of the options from the Install Tool won't have any effect anymore.

Affected Installations

Instances which use one of the methods above or use one of the removed Install Tool options.

Breaking: #72474 - RequestHandler only works with Routes

See Issue #72474

Description

The default backend RequestHandler had a fallback that triggered the backend initialization without having a `&route=` request parameter set. This was used for the transition when deprecating the traditional entry-scripts. The logic has been removed.

Impact

Any regular backend request (non-module and non-AJAX) will now require a `&route=` request parameter, otherwise will fallback to the default route (login) when not provided.

Migration

For all backend-related calls, either use a custom RequestHandler or switch to Backend Routing.

Breaking: #72476 - PHP Constant TYPO3_PROCEED_IF_NO_USER removed

See Issue #72476

Description

The PHP constant `TYPO3_PROCEED_IF_NO_USER` has been removed.

Impact

Any checks on this constant will result in a fatal PHP error.

Any definition in custom entry-scripts of extensions will have no effect anymore.

Affected Installations

Installations with custom entry-points for the TYPO3 Backend.

Migration

Use a custom RequestHandler, the Backend Routing, AJAX Registration or the Module Configuration for skipping the user authentication when necessary.

Breaking: #72492 - Removed XHTML2 support

See Issue #72492

Description

The support for XHTML2 documents in the TYPO3 Frontend has been removed.

Impact

The TypoScript option `config.doctype = xhtml_2` has no effect anymore.

Affected Installations

Any TYPO3 instance using XHTML2 for frontend rendering.

Migration

Disable all header code and set your own doctype.

Breaking: #72493 - Removed TypoScript property page.bgImg

See Issue #72493

Description

The TypoScript property `page.bgImg` has been removed.

Impact

The option has no effect anymore and will not be evaluated anymore.

Migration

Use the TypoScript property `page.bodyTagCObject` instead.

Breaking: #72497 - Removed recode support for Charset Conversion

See Issue #72497

Description

The support for GNU-recode when converting from one charset to another has been dropped. The CharsetConverter now only supports `mbstring`. In case `mbstring` is not present a polyfill will kick in.

Impact

Setting `$TYPO3_CONF_VARS[SYS][t3lib_cs_convMethod] = 'recode';` will have no effect anymore. Conversion is then done through the default TYPO3-internal conversion.

Affected Installations

Installations that have the option `$TYPO3_CONF_VARS[SYS][t3lib_cs_convMethod]` set to `recode`.

Migration

No migration.

Breaking: #72572 - Remove more deprecated miscellaneous functions and options

See Issue #72572

Description

Removed more deprecated miscellaneous functions and options:

- Linking to a php file directly in `ClickMenu` has been removed.
- Functionality of `$TBE_STYLES['background']` has been removed.
- The option `DocumentTemplate->JScodeLibArray` has been removed.
- The option `$TYPO3_CONF_VARS[SYS][displayErrors]` set to "2" will throw an exception.
- The deprecated icons fallback for `actions-system-refresh` and `actions-system-extension-update-disabled` has been removed.
- An extension may not refer to `ext:cms` in composer.json or ext_emconf.php file. The fallback has been removed.
- The method `loadNewTcaColumnsConfigFiles` has been removed.
- Usage of the field "static_lang_isocode" has stopped working. Use the built-in language field "language_isocode" in sys_language records.

Impact

Using one of the mentioned options or methods will result in a fatal error or won't have any effect anymore.

Registration of `TCA` within `ext_tables.php` now finally stops working and code to resolve `dynamicConfigFile` option from `TCA` has been removed. Extensions still relying on this will fail.

Affected Installations

Instances which use one of the methods above or use one of the removed options.

Migration

For `DocumentTemplate->JScodeLibArray` use PageRenderer instead.

If the option `$TYPO3_CONF_VARS[SYS][displayErrors]` is set to "2" use "-1" instead.

All table definitions should be moved to <your_extension>/Configuration/TCA/<table_name>

Breaking: #72602 - Removed unzip functionality

See Issue #72602

Description

The legacy functionality to unzip files from outside the document root has been removed.

Additionally, the corresponding option `$TYPO3_CONF_VARS[BE][unzip_path]` and the class member `ExtendedFileUtility::$unzipPath` have been removed as well.

Legacy methods from the Extbase domain model BackendUser named `isFileUnzipAllowed` and `setFileUnzipAllowed` have been removed.

Impact

Calling the entry point `FileController` using unzip action will have no effect anymore.

Using the Extbase domain model methods will result in a fatal PHP error.

Migration

Use a third-party extension to integrate unzip functionality into TYPO3.

Breaking: #72604 - Remove option maxFileNameLength

See Issue #72604

Description

The option `$TYPO3_CONF_VARS[SYS][maxFileNameLength]` was removed.

Impact

Using the option will have no effect anymore.

Breaking: #72661 - RTE Transformation ts_strip removed

See Issue #72661

Description

The Rich Text Editor transformation that removes all HTML tags except a hard-coded white-list of allowed HTML tags when saving data from the RTE to the database - called `ts_strip` - has been removed.

Impact

Using the command `ts_strip` in the list of transformations via PageTSconfig or TCA directly will result in keeping the HTML tags inside the database.

Affected Installations

TYPO3 installations that use `ts_strip` explicitly in their TSconfig options, or instances with extensions that set this option.

Migration

Use TSconfig options like `RTE.default.removeTags` to specify which tags should be removed when saving data to the database, or even better `RTE.default.proc.allowTags` which tags are whitelisted to the database together with the default RTE processing command `ts_css`.

Breaking: #72666 - RTE: Remove relative path calculations

See Issue #72666

Description

Since the removal of the feature editing static files with the Rich Text Editor (option "static_file_edit"), the path calculations for files within the HtmlParser including the method `RteHtmlParser->setRelPath()` have been removed as well.

Impact

Using the method `RteHtmlParser->setRelPath()` will result in a fatal PHP error.

Affected Installations

Any installations with custom RTE transformations that use a custom implementation of the RteHtmlParser PHP class.

Breaking: #72667 - RTE: Unused internal methods removed

See Issue #72667

Description

The HTML parsing features for the Rich Text Editor feature related to xhtml_cleaning have been removed. The following now obsolete methods have been removed as well:

- `HtmlParser->checkTagTypeCounts()`
- `HtmlParser->unprotectTags()`
- `HtmlParser->get_tag_attributes_classic()`
- `HtmlParser->cleanFontTags()`
- `HtmlParser->indentLines()`

Additionally, the third parameter for the method `HtmlParser->getAllParts()` has been removed, resulting that the method will always include the parsed tags in the result set.

Impact

Calling any of the methods will result in a fatal PHP error.

Affected Installations

Any installation which uses a third-party extension that modifies the HtmlParsing via PHP.

Breaking: #72671 - Extension "aboutmodules" removed

See Issue #72671

Description

The system extension "aboutmodules" has been removed, as all functionality has been moved to the system extension "about".

Impact

It is not possible to have just "aboutmodules" or "about" installed without having the other module show as well.

Breaking: #72686 - Removed RteHtmlParser methods

See Issue #72686

Description

The following methods within `RteHtmlParser` have been removed without substitution:

- `RteHtmlParser->siteUrl()`
- `RteHtmlParser->getUrl()`

The second method parameter of the following methods have been removed as they have no effect anymore:

- `RteHtmlParser->HTMLcleaner_db()`
- `RteHtmlParser->getKeepTags()`

Impact

Calling either `RteHtmlParser->siteUrl()` or `RteHtmlParser->getUrl()` will result in a PHP fatal error.

Calling `RteHtmlParser->HTMLcleaner_db()` or `RteHtmlParser->getKeepTags()` with a second parameter will have no effect anymore.

Affected Installations

TYPO3 instances which use RteHtmlParser methods directly within a third-party extension for HTML transformation.

Migration

Use `GeneralUtility::getUrl()` instead of `RteHtmlParser->getUrl()`.

Use `GeneralUtility::getIndpEnv('TYPO3_SITE_URL')` instead of `RteHtmlParser->siteUrl()`.

Breaking: #72701 - Remove unused properties in DocumentTemplate

See Issue #72701

Description

Remove deprecated code from DocumentTemplate

The following properties have been removed:

`tableLayout` `table_TR` `table_TABLE`

Impact

Using the properties above directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use calls to any of the above mentioned properties.

Migration

No migration available.

Breaking: #72711 - Remove property strict in TypoScriptParser

See Issue #72711

Description

The property `strict` of `\TYPO3\CMS\Core\TypoScript\Parser\TypoScriptParser` has been removed.

Impact

Using the property directly in any third party extension will result in a fatal error.

Affected Installations

Instances which call the above mentioned property.

Migration

No migration available.

Breaking: #72783 - Removed RTE transformation option preserveTables

See Issue #72783

Description

The RTE transformation option `preserveTables` that allowed keeping HTML table tags and their contents has been removed.

Additionally, the methods `RteHtmlParser->removeTables` and `HtmlParser->getAllParts` have been removed without substitution.

Impact

When the RTE is configured to use `overruleMode = ts` instead of the default "ts_css" the option `RTE.default.proc.preserveTables = 1` will have no effect anymore.

Calling `RteHtmlParser->removeTables` or `HtmlParser->getAllParts` inside a custom extension will result in a fatal PHP error.

Affected Installations

Any TYPO3 instance with a legacy-mode (overruleMode = ts) from TYPO3 3.x or an extension doing custom transformations by using `RteHtmlParser->removeTables`.

Migration

Use the overruleMode `ts_css` instead which keeps the tables as they are. If tables should be disallowed inside the RTE the option `RTE.default.proc.denyTags := addToList(table)` can be used instead.

Breaking: #72826 - Removed custom charset configuration for locales

See Issue #72826

Description

The TYPO3 Frontend resolved the TypoScript option `config.locale_all` and stored the charset part within `$TSFE->localeCharset`. If the option `locale_all` did not provide a charset (e.g. when it is set to `de_AT` instead of `de_AT.UTF-8` a "best guess" was done based on a static list set up in 2004.

The option `$TSFE->localeCharset` has been removed, along with the following calculation options and methods available in the CharsetConverter class:

- CharsetConverter->lang_to_script
- CharsetConverter->script_to_charset_unix
- CharsetConverter->script_to_charset_windows
- CharsetConverter->locale_to_charset
- CharsetConverter->get_locale_charset()

The localeCharset option was solely used within the TypoScript functionality `stdWrap.strftime` when no custom character set was given, and a character set conversion from the "localeCharset" (based on the best guess or explicitly set via `config.locale_all = de_AT.UTF-8` and it was different than the renderCharset option of the TYPO3 Frontend.

Impact

When custom locales are configured in TypoScript which are not present on the server, or the character set of `config.locale_all` differs from the `config.renderCharset`, or `config.locale_all` does not set a character set, could lead to unexpected output in the TYPO3 Frontend.

Affected Installations

Instances which have a different `config.locale_all` character set given than set via `config.renderCharset`, or on servers that don't have the charset of the locale available but the output should be a certain but not given character set.

Migration

As this is a misconfiguration and only necessary if e.g. can not handle UTF-8 locales, `config.set_locale` can explicitly be set to `de_AT@iso-8859-15` and the output should be renderCharset. On instances where `stdWrap.strftime`is used,the subproperty `charset` can be set to the custom character set (e.g. `iso-8859-15`).

In each case, it should be configured that the `config.locale_all` option should have a character set given, to avoid any side-effects with the TypoScript stdWrap option `strftime`.

Breaking: #72830 - Removed deprecated RTE transformations ts & ts_transform

See Issue #72830

Description

The transformation option "ts" and "ts_transform" are outdated since the introduction of CSS Styled Content and have been removed from the TYPO3 Core.

The two related PHP methods `TS_transform_rte` and `TS_transform_db` had the second method parameter dropped.

Impact

Setting the TSconfig option `RTE.proc.overruleMethod = ts` or `RTE.proc.overruleMethod = ts_transform` will result in not having any transformation applied to the content anymore.

Affected Installations

Any installation with custom RTE transformation options to render legacy HTML code.

Migration

Use `ts_css` instead, which is set by default since TYPO3 4.0.

Breaking: #72837 - RTE transformations: Allow div sections by default and remove font-specific parsing

See Issue #72837

Description

The TSconfig `RTE.default.proc` options `preserveDIVSections` and `allowedFontColors` for transforming data between the Rich Text Editor and the database have been removed.

The `preserveDIVSections` option is now built-in by default "always on", and DIV tags are always treated as block elements.

Special handling for `<font>` tags is done via the regular tag processing options like any other tag.

Impact

Setting the TSconfig option `RTE.default.proc.preserveDIVSections = 0` or `RTE.default.proc.allowedFontColors` will have no effect anymore.

Affected Installations

Any installation using custom TSconfig configurations for the RTE and using `RTE.default.proc.preserveDIVSections` set to 0 or `RTE.default.proc.allowedFontColors` to any value.

Migration

If DIV HTML elements should not treated like block elements, the RTE option `RTE.proc.blockElementList` can be manually customized to not include DIV elements.

If the option `allowedFontColors` is still needed, the existing functionality can be achieved by using the `keepTags` functionality to sort out correct values for a property.

Breaking: #72853 - Remove unused Images from core

See Issue #72853

Description

A lot of unused images from the core have been removed. Although it is not a good style, some extensions use references to one or more files.

Impact

References to the images listed below can throw a 404 not found.

Deleted images:

typo3/sysext/backend/Resources/Public/Images/Overlay/default.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_deleted.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_group.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_hidden.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_hidden_timing.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_no_icon_found.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_sub5.gif
typo3/sysext/backend/Resources/Public/Images/Overlay/overlay_timing.gif
typo3/sysext/filemetadata/Resources/Public/Icons/status_1.png typo3/sysext/filemetadata/Resources/Public/Icons/status_2.png
typo3/sysext/filemetadata/Resources/Public/Icons/status_3.png typo3/sysext/frontend/Resources/Public/Icons/ContentElementW
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/div.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/filelinks.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/html.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/images_only.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/login_form.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/mailform.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/multimedia.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/regular_header.gif

typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/regular_text.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/searchform.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/shortcut.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/sitemap.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/sitemap2.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/table.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/text_image_below.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/text_image_right.gif
typo3/sysext/frontend/Resources/Public/Icons/ContentElementWizard/user_defined.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/above_center.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/above_left.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/above_right.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/below_center.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/below_left.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/below_right.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/intext_left.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/intext_left_nowrap.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/intext_right.gif
typo3/sysext/frontend/Resources/Public/Icons/ImageOrientation/intext_right_nowrap.gif
typo3/sysext/frontend/Resources/Public/Images/wizard_backend_layout.png
typo3/sysext/opendocs/Resources/Public/Icons/opendocs.png typo3/sysext/opendocs/Resources/Public/Images/toolbar_item_ac
typo3/sysext/tstemplate/Resources/Public/gfx/BUG_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/BUG_menu.gif
typo3/sysext/tstemplate/Resources/Public/gfx/BUG_menu2.gif typo3/sysext/tstemplate/Resources/Public/gfx/BUSINESS_basic
typo3/sysext/tstemplate/Resources/Public/gfx/BUSINESS_menu.gif
typo3/sysext/tstemplate/Resources/Public/gfx/CANDIDATE_basic.gif
typo3/sysext/tstemplate/Resources/Public/gfx/CANDIDATE_menu.gif
typo3/sysext/tstemplate/Resources/Public/gfx/CANDIDATE_page.gif
typo3/sysext/tstemplate/Resources/Public/gfx/CrCPH_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/FIRST_basic.gif
typo3/sysext/tstemplate/Resources/Public/gfx/FIRST_menu.gif typo3/sysext/tstemplate/Resources/Public/gfx/GLCK_basic.gif
typo3/sysext/tstemplate/Resources/Public/gfx/GLCK_columns.gif typo3/sysext/tstemplate/Resources/Public/gfx/GLCK_menu.
typo3/sysext/tstemplate/Resources/Public/gfx/GREEN_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/GREEN_menu.
typo3/sysext/tstemplate/Resources/Public/gfx/GREEN_menu2.gif typo3/sysext/tstemplate/Resources/Public/gfx/GREEN_menu
typo3/sysext/tstemplate/Resources/Public/gfx/HYPER_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/HYPER_menu.
typo3/sysext/tstemplate/Resources/Public/gfx/HYPER_menu2.gif typo3/sysext/tstemplate/Resources/Public/gfx/HYPER_page.
typo3/sysext/tstemplate/Resources/Public/gfx/HYPER_toptitle.gif typo3/sysext/tstemplate/Resources/Public/gfx/MM_basic.gif
typo3/sysext/tstemplate/Resources/Public/gfx/MM_menu.gif typo3/sysext/tstemplate/Resources/Public/gfx/MM_right.gif
typo3/sysext/tstemplate/Resources/Public/gfx/NEWSLETTER_basic.gif
typo3/sysext/tstemplate/Resources/Public/gfx/RE_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/RE_leftmenu.gif
typo3/sysext/tstemplate/Resources/Public/gfx/RE_menu.gif typo3/sysext/tstemplate/Resources/Public/gfx/RE_top.gif
typo3/sysext/tstemplate/Resources/Public/gfx/TU_basic.gif typo3/sysext/tstemplate/Resources/Public/gfx/TU_menu.gif

Affected Installations

Installations or extensions which have references to this images.

Migration

No migration

Breaking: #72861 - EXT:form - Remove deprecated code

See Issue #72861

Description

The compatibility layer of EXT:form has been removed. The layer made sure that EXT:form acts almost like in TYPO3 6.2. This mainly applies to the layout configuration and rendering. In the former days integrators could use `.layout` settings on different levels to change the output of the form elements in the frontend. Nowadays, changing the frontend output is only possible by utilizing own fluid templates.

Furthermore the SELECT, TEXTAREA and TEXTBLOCK elements have been adjusted. The automatic transformation of the `.data` (SELECT, TEXTAREA) and `.content` (TEXTBLOCK) attribute has been removed. That way `.text` is the only valid attribute for adding a human readable text to the above mentioned elements.

Impact

Using `.layout` will have no effect anymore.

Using the `.data` attribute for SELECT, TEXTAREA and `.content` for TEXTBLOCK elements will also have no effect.

Affected Installations

Any installation using `.layout` and/ or `.data` and/ or `.content` settings. Most of the older installations (mainly 6.2 LTS) will be affected when upgrading to 8 LTS.

Migration

All `.layout` settings have to be removed and ported to own fluid templates.

All occurrences of the `.data` and `.content` attribute have to be substituted by `.text`.

Breaking: #72866 - Removed RTE processing option to use div tags instead of p tags

See Issue #72866

Description

The Rich Text Editor TSconfig processing instructions `RTE.default.proc.useDIVasParagraphTagForRTE` and `RTE.default.proc.remapParagraphTag` have been removed.

The Rich Text Editor is now always getting HTML content wrapped with <p> tags instead of the optional tags.

Impact

Using any of the options above will have no effect anymore.

Affected Installations

TYPO3 instances with custom Rich Text Editors (EXT:rtehtmlarea is not affected).

Breaking: #72870 - Removed RTE transformation ts_preserve and preserveTags

See Issue #72870

Description

The RTE configuration TSconfig option `RTE.default.proc.preserveTags` to preserve special tags has been removed.

The RTE transformation mode "ts_preserve" to change special preserved tags and migrate to tags has been removed.

The according methods `TS_preserve_db` and `TS_preserve_rte` within RteHtmlParser have been removed.

Impact

Setting the TSconfig option or the RTE transformation mode has no effect anymore.

Calling the removed PHP methods directly will result in fatal PHP errors.

Affected Installations

TYPO3 instances with custom RTE transformations using the removed "ts" transformation mode, or a custom transformation mode.

Migration

Use the RTE processing option `RTE.default.proc.allowTags` to include the tags without rewriting them to custom tags.

If special handling is still necessary, an existing hook can be used to re-implement the logic.

Breaking: #72888 - Removed HtmlParser mapTags functionality

See Issue #72888

Description

The functionality to map tags explicitly from the HtmlParser code has been removed:

- `HtmlParser::mapTags()`
- `RteHtmlParser::defaultTStagMapping()`

Impact

Calling one of the two methods above will result in a PHP fatal error.

Affected Installations

Any installation using custom RTE transformation and wanting to remap tags while parsing HTML.

Migration

Use the "remap" functionality of the `keepTags` logic within HtmlParser to achieve the same in custom transformations.

Breaking: #72889 - Removed RteHtmlParser htmlspecialchars() transformation options

See Issue #72889

Description

The TSconfig options `RTE.default.proc.dontHSC_rte` and `RTE.default.proc.dontUndoHSC_db` have been removed from the TYPO3 Core.

Impact

Setting these options has no effect anymore.

Affected Installations

Any installation using these options for properly applying htmlspecialchars() to the RTE content when cleaning the HTML input from an RTE and vice versa.

Migration

Use `entryHtmlParser` and `exitHtmlParser` to apply htmlspecialchars while transforming content from the RTE or to the RTE.

Breaking: #72897 - RteHtmlParser: Dropped ts_reglinks transformation

See Issue #72897

Description

The RTE transformation mode `ts_reglinks` which transforms relative links to absolute links when using the Rich Text Editor has been removed, along with the according PHP method `RteHtmlParser->TS_reglinks()`.

Impact

Using the transformation mode, set via TCA or TSconfig (`RTE.default.proc.mode = ts_reglinks`) will have no effect anymore.

Calling the PHP method will result in a PHP fatal error.

Affected Installations

Any installation using obsolete transformation mode or special RTE transformations.

Breaking: #73044 - JSON for ClickMenu in Backend

See Issue #73044

Description

The ClickMenu in the TYPO3 Backend now uses JSON to transport data between the server and the client.

Before, a proprietary <t3ajax> syntax with XML was used to transport the contents of the ClickMenu.

Impact

Using ClickMenu to implement a custom ClickMenu JavaScript handler instead of the default ClickMenu.js could result in unexpected behaviour.

Affected Installations

Installations with extensions that use custom ClickMenu.js behaviour.

Migration

Adapt the custom code to handle JSON responses instead of XML responses.

Breaking: #73046 - Alias AbstractNode -> ViewHelperNode for backwards compatibility

See Issue #73046

Description

`ViewHelperInterface::compile()` in standalone Fluid now requires an actual `ViewHelperNode` instead of `AbstractNode` as it was before. The reason for changing this should be fairly obvious. In order to preserve signature compatibility an alias is put in place.

The alias is created so that `TYPO3\CMS\Fluid\Core\Parser\SyntaxTree\AbstractNode` becomes an alias of `TYPO3Fluid\Fluid\Core\Parser\SyntaxTree\ViewHelperNode`. This is obviously less than ideal but solves the problem immediately and prioritises not breaking the public API by breaking the non-public API.

Impact

This signature change means that ViewHelpers which implement a custom `compile()` method must update their signature (does not apply to ViewHelpers implementing only renderStatic). Two ViewHelpers in TYPO3 CMS Fluid were migrated as part of the standalone Fluid merge - but third-party ViewHelpers would require either migration or an alias.

- Overridden TemplateParsers (before standalone Fluid merge) might not work, depending on the nature of overrides (conditions checking class names may fail)
- Custom Nodes (implemented via an overridden TemplateParser, before standalone Fluid) would break either in function or form (incompatible constructor signatures on PHP7 at least)

Affected Installations

Any TYPO3 site using an extension which replaces or directly interacts with TemplateParser or TemplateCompiler from Fluid. Only known affected community extension is at this point EXT:builder, specifically the template validation/analysis feature.

Migration

Change any reference to classes in `TYPO3\CMS\Fluid\Core\TemplateParser` to `TYPO3Fluid\Fluid\Core\TemplateParser`.

Breaking: #73106 - Convert thumbnails only for non-image files

See Issue #73106

Description

`$TYPO3_CONF_VARS[GFX][thumbnails_png]` must be taken into account only for non-image files.

Impact

`$TYPO3_CONF_VARS[GFX][thumbnails_png]` is now a boolean value. If the value is true, the processed thumbnails that are not an image will be converted in png, otherwise these will be converted to gif. It is not possible anymore to convert the processed thumbnails that are png or gif to jpg files.

Affected Installations

Installations who have set the value of `$TYPO3_CONF_VARS[GFX][thumbnails_png]` to 2 or 3.

Migration

The install tool automatically sets the value of `$TYPO3_CONF_VARS[GFX][thumbnails_png]` to true, if it has been set to 1, 2 or 3 previously.

Breaking: #73152 - Symfony console helpers replaced

See Issue #73152

Description

By upgrading to Symfony Console 3.x the `DialogHelper`, `ProgressHelper` and `TableHelper` have been replaced. The internal getter methods for these classes have been replaced in Extbase `ConsoleOutput`.

Impact

Calling the following methods with result in a fatal error:

- `ConsoleOutput::getDialogHelper()`
- `ConsoleOutput::getProgressHelper()`
- `ConsoleOutput::getTableHelper()`

The 2nd argument of the following methods is ignored now:

- `ConsoleOutput::progressAdvance()`
- `ConsoleOutput::progressSet()`

Affected Installations

Extensions which have directly called these methods in favor of the Extbase `ConsoleOutput` helper methods.

Migration

Use the following methods instead:

- `ConsoleOutput::getQuestionHelper()`
- `ConsoleOutput::getProgressBar()`
- `ConsoleOutput::getTable()`

Remove the 2nd argument when calling these methods:

- `ConsoleOutput::progressAdvance()`
- `ConsoleOutput::progressSet()`

Breaking: #73445 - Remove flashmessage_compatibility.js from core

See Issue #73445

Description

The `flashmessage_compatibility.js` has been removed from the core.

Impact

Extensions which make use of `TYPO3.Flashmessage` JavaScript function will not work anymore.

Migration

Use `top.TYPO3.Notification.*` to create flash messages within JavaScript for the TYPO3 backend.

Breaking: #73504 - Make TimeTracker a singleton

See Issue #73504

Description

The class `\TYPO3\CMS\Core\TimeTracker\TimeTracker` has been marked as singleton and is no longer stored in `$GLOBALS['TT']`.

Impact

Using methods of `$GLOBALS['TT']` will result in a fatal error.

Affected Installations

All installations or 3rd party extensions using `$GLOBALS['TT']`.

Migration

Use `\TYPO3\CMS\Core\Utility\GeneralUtility::makeInstance(\TYPO3\CMS\Core\TimeTracker\Time` instead.

Breaking: #73514 - TypoScript property "includeLibs" removed

See Issue #73514

Description

The TypoScript property to load additional PHP libraries via `.includeLibs` has been removed from the Content Objects `COA/COA_INT` and `USER/USER_INT`.

Impact

Setting the `.includeLibs` property will have no effect anymore.

Affected Installations

Any installation using a very old pi_based extension that does not ship proper class naming or autoloading information.

Migration

Make sure everything that was previously loaded via includeLibs is now encapsulated in proper PHP classes, which is referenced by USER/USER_INT when needed.

Breaking: #73516 - GeneralUtility::getFileAbsFileName allows for typo3/ maindir specific paths

See Issue #73516

Description

The PHP method `GeneralUtility::getFileAbsFileName` used for resolving absolute paths has the option removed to only resolve relative paths or paths to the typo3/ main directory.

Impact

The two removed parameters are not evaluated anymore, thus always resolving any path, and additionally always relative to the `PATH_site` variable (the installations' base directory).

Affected Installations

Any installation with an extension using the removed options to fetch data relative to the typo3/ directory.

Migration

Use the `EXT:` syntax everywhere to resolve files within extension directories. If the path relative to the typo3/ main directory is explicitly needed, the constant `TYPO3_mainDir` can be used as a prefix to the file.

Breaking: #73602 - Short-URL without ?id=ID removed

See Issue #73602

Description

The support for resolving URLs using `index.php?23` instead of `index.php?id=23` with no real GET parameter given has been removed.

The method `$TSFE->setIDfromArgV()` has been removed as well.

Impact

Calling a frontend page with the short-handed URL will result in not detecting a page ID at all.

Calling `$TSFE->setIDfromArgV()` directly within PHP will result in a fatal PHP error.

Affected Installations

Any TYPO3 installation with an extension using the pre-4.0 syntax.

Migration

Use the proper `index.php?id=23` when using URLs to be called in the frontend.

Breaking: #73611 - Removed ResourceCompressor relative path methods

See Issue #73611

Description

The methods `ResourceCompressor::setInitialPaths()` and `ResourceCompressor::setRelativePath()` have been removed.

Impact

Calling one of the methods above will result in a fatal PHP error.

Affected Installations

Any TYPO3 instance with custom extensions manually using the ResourceCompressor instead of using the PageRenderer API.

Migration

Simply remove the methods from the affected code, as these methods are not needed anymore. All calculations for the paths are now done automatically.

Breaking: #73655 - PHP 7 required

See Issue #73655

Description

TYPO3 CMS 8 requires PHP 7. The system will exit with a message if this requirement is not fulfilled.

Migration

Upgrade to at least PHP 7.0.0.

Breaking: #73698 - Streamline layout of FlashMessages

See Issue #73698

Description

The layout and usage of FlashMessages has been streamlined in the TYPO3 backend. All FlashMessages in the TYPO3 backend are now rendered as markup and contain an icon, the message and an optional title.

Example:

```
<div class="alert alert-danger">
   <div class="media">
      <div class="media-left">
         <span class="fa-stack fa-lg">
            <i class="fa fa-circle fa-stack-2x"></i>
            <i class="fa fa-times fa-stack-1x"></i>
         </span>
      </div>
      <div class="media-body">
         <h4 class="alert-title">The optional title</h4>
         <p class="alert-message">The message goes here</p>
      </div>
   </div>
</div>
```

FlashMessages that are used as inline notification should be removed and replaced with custom HTML code. For the core we have defined output and usage for messages:

1) FlashMessages

FlashMessages are designed to inform a user about success or failure of an action, which was **triggered** by the user. Example: If the user deletes a record, a FlashMessage informs the user about success or failure. This kind of information is not static, it is a temporary and volatile information and triggered by a user action.

Keep in mind that you **must not** use HTML markup here, since this information might be shown in a context different from HTML, like processing it via Javascript or showing the message on the command line.

2) Callouts (InfoBox-ViewHelper)

Callouts are designed to display permanent information, a very good example is the usage in the Page-Module. If a user opens a system folder with the page module, the callout explains: 'Hey, you try to use the page module on a sys folder, please switch to the list module'. This ViewHelper can also be used to show some help or instruction how to use a backend module.

3) Any other information

For any other information e.g. a list of files which has changed, must be handled in the action / view of the module or plugin. This is not a use case for a FlashMessage or Callout! Example: Display a list of a hundred files within a FlashMessage or Callout is a bad idea, build custom markup in the view to handle this kind of message.

Impact

Extensions which use the FlashMessageViewHelper with the default rendering will now get a list of -messages instead of a <ul>-list.

Migration

No migration needed, the generated output should be as expected. If the rendering is broken please consider about the correct usage of FlashMessages and read the explanation about message types above.

Breaking: #73711 - Removed deprecated code from Form Domain Model Element

See Issue #73711

Description

The protected variable `$layout` is deprecated and has been removed together with their getter and setter.

Impact

Using the methods `getLayout()` and `setLayout()` directly in any third party extension will result in a fatal error.

Affected Installations

Instances which use the methods above to access the protected `$layout` variable.

Breaking: #73719 - Unused JavaScript configuration options for the Backend removed

See Issue #73719

Description

The following JavaScript options from TYPO3.configuration and the global JavaScript property `TS` have been removed:

```
TYPO3.configuration.PATH_typo3
TYPO3.configuration.PATH_typo3_enc
TYPO3.configuration.userUid
TYPO3.configuration.securityLevel
TYPO3.configuration.TYPO3_mainDir
TYPO3.configuration.veriCode
TYPO3.configuration.denyFileTypes
TS.PATH_typo3
TS.PATH_typo3_enc
TS.securityLevel
TS.veriCode
TS.denyFileTypes
TS.decimalSign
```

Impact

Calling any of the JavaScript options above from within JavaScript will result in a undefined JavaScript error.

Affected Installations

Any TYPO3 installation using a legacy TYPO3 extension that uses these options within JavaScript.

Migration

Remove the values from the JavaScript code or provide a PHP alternative to make the options available again for JavaScript, if needed.

Breaking: #73763 - Removed backPath from PageRenderer

See Issue #73763

Description

The PageRenderer class responsible for rendering Frontend output and Backend modules has no option to resolve the so-called backPath anymore. The second parameter has been dropped from the constructor method. Additionally the public property `backPath` as well as the method `PageRenderer->setBackPath()` have been removed.

Impact

Calling the constructor of PageRenderer with a second parameter, or setting PageRenderer->backPath has no effect anymore. Calling `PageRenderer->setBackPath()` directly will result in a PHP error.

Affected Installations

Custom installations using the PageRenderer API directly in an extension.

Migration

Simply remove the call to `PageRenderer->setBackPath()` in your own scripts.

Breaking: #73793 - Removed AbstractPlugin->LOCAL_LANG_charset

See Issue #73793

Description

The public property `$LOCAL_LANG_charset` within AbstractPlugin (formerly known as pi_base) has been removed.

The property was used to hold information which labels, coming from TypoScript that override Language files, need to be converted to UTF-8.

Impact

Accessing the property within own extensions has no effect anymore.

Affected Installations

Installations with extensions that provide plugins derived from `AbstractPlugin` and use this property.

Migration

No migration needed. Make sure that all external TypoScript configuration files are stored with UTF-8 character set.

Breaking: #73794 - renderCharset option removed

See Issue #73794

Description

The TypoScript option `config.renderCharset` which was used as character set for internal conversion within a frontend request has been removed, as internal conversions are always set to UTF-8 now. The property `$TSFE->renderCharset` is now always set to `utf-8`, and is not used within the TYPO3 Core anymore.

Please note that it is still possible to define the character set of the returned content via `config.metaCharset`.

Impact

Using TYPO3 with a different datasource which is not UTF-8 (e.g. a database with latin1) might return unexpected results. Non-UTF-8 databases work as expected if the connection charset is still UTF-8, as the DBMS takes care of converting the data to UTF-8.

Affected Installations

Any installation that uses the `config.renderCharset` TypoScript option with a different value than `utf-8`.

Migration

Remove the TypoScript option from any TypoScript settings. If data sources (files, database input) are used that are different than UTF-8, a manual conversion via the CharsetConverter PHP class is needed.

Breaking: #74029 - Remove ModuleLoader->getRelativePath()

See Issue #74029

Description

The method `ModuleLoader->getRelativePath()` has been removed. It was previously part when registering traditional script-based modules, which did not use the new Icon API for the backend.

Impact

Calling the method above directly will result in a fatal PHP error.

Affected Installations

Any installation working with extensions that set up the ModuleLoader class and call the method `getRelativePath()` directly.

Migration

Use `PathUtility::getRelativePath()` when the functionality is still needed.

Breaking: #74031 - CharsetConverter parameters removed

See Issue #74031

Description

The second parameter for the method `CharsetConverter->entities_to_utf8()` has been removed.

The second and the third parameter of `CharsetConverter->utf8_to_numberarray()` has been removed.

Impact

When calling `CharsetConverter->entities_to_utf8()`, string-HTML entities (like & or £) will be converted to UTF-8 as well at all times. Previously this behaviour was configurable.

When calling `CharsetConverter->utf8_to_numberarray()`, string-HTML entities (like & or £) will be converted to UTF-8 as well at all times. Additionally instead of integer numbers the real UTF-8 char is returned at any times. Previously these behaviours were configurable.

Affected Installations

Installations with custom extensions that used these methods directly in PHP code.

Migration

Remove these parameters from the calling PHP code.

Breaking: #74124 - Removed sys_file_reference field downloadname

See Issue #74124

Description

The database table `sys_file_reference` comes with the database field `downloadname` but it is not used or displayed anywhere.

The database field has been removed.

Impact

Using this field in SQL statements will result in no output of files and potentially SQL errors.

Affected Installations

Any installation with an extension using this database field. Any installation with TypoScript using this field directly in the SQL definition when fetching a sys_file_reference record.

Migration

If this field was used before and is still needed, re-create this field in the `ext_tables.sql` of your extension.

Breaking: #75150 - Removed TypoScript option includeJSlibs

See Issue #75150

Description

The deprecated TypoScript property `page.includeJSlibs` has been removed.

Impact

Setting parameters to this property will have no effect anymore.

Affected Installations

Any installation using this TypoScript option.

Migration

Use the TypoScript property `page.includeJSLibs` instead, which is in line with naming of `includeCSSLibs`.

Breaking: #76155 - ViewHelper Namespace imports with `xmlns` are now singular

See Issue #76155

Description

Fluid templates which use **multiple** ´ ‘ in one template file are affected. Only the first such node is now detected and respected. The reasons for this new limitation are the reworked internals of Fluid since the standalone version.

Before, namespace nodes were handled internally by the TemplateParser. They worked in a way that the namespace dictated by the node would **only apply inside that node**.

After, namespace nodes are handled by template pre-processing and handed off to the ViewHelperResolver without going through the TemplateParser first. Since it happens in pre-processing, namespaces dictated by such nodes now **apply across the entire template file**.

This has two effects on template parsing:

1. Extraction of `xmlns:xyz` style imports happens **once** and detects only a single node.
2. Nesting no longer matters; a namespace imported with this method will apply to the entire template file since it is extracted during pre-processing and is not recursive.

Most templates will not be affected by this since the norm is already to include a single such namespace import and put it as the outermost surrounding tag, then add all imported namespaces on that single node. This usage is still fully supported.

Impact

The change affects template files which fulfill one or both of the following conditions:

1. Any template file which contains multiple nodes with `xmlns:xyz` imports will see only the first node detected.
2. Any template file which assumes an imported namespace is removed when the enclosing tag is closed and uses a previously imported namespace after the closing node will likely see errors with `ViewHelper could not be resolved` since Fluid will attempt to translate matching XHTML nodes with namespace prefixes to ViewHelper classes.

Affected Installations

TYPO3 8.0 and above, any site matching conditions stated in `Impact`.

Migration

There is one migration for each of the conditions above:

1. Templates with multiple `xmlns:xyz` nodes can be migrated by combining all those nodes into one.
2. Templates which assume a closing container tag removes the namespace will have to migrate by extracting the XHTML that collides with the Fluid namespace and placing it in a separate template file (e.g. Partial template).

2.9.2 Features

Feature: #1835 - Recover pages recursively to top of rootline

See Issue #1835

Description

The Recycler now supports the recursive recovery of deleted pages to the top of the rootline. This feature is available for admin users only due to internal permission restrictions.

Feature: #19157 - Add option to exclude all hidden records in EXT:impexp

See Issue #19157

Description

The export configuration of EXT:impexp has been extended to allow to completely deactivate exporting of hidden/deactivated records. This behaviour can be controlled via a new option which is checked by default.

Furthermore, if the inclusion of hidden records is activated (which is now an explicit choice), then an additional button is shown, allowing users to preselect all hidden records for manual exclusion.

Feature: #28230 - Add support for PBKDF2 to saltedpasswords

See Issue #28230

Description

A new password hashing algorithm `PBKDF2` has been added to the system extension `saltedpasswords`. PBKDF2 is designed to be computationally expensive to resist brute force password cracking.

Impact

None, the new hashing algorithm needs to be enabled by the system administrator in the extension configuration and will upgrade existing passwords transparently on login.

Feature: #54887 - Post-processing of the previewUrl

See Issue #54887

Description

An additional hook has been added to the method `BackendUtility::viewOnClick()` to post-process the preview url.

The hook is called with the following signature:

```
/**
 * @param string $previewUrl
 * @param int $pageUid
 * @param array $rootLine
 * @param string $anchorSection
 * @param string $viewScript
 * @param string $additionalGetVars
 * @param bool $switchFocus
 * @return string The processed preview URL
 */
function postProcess($previewUrl, $pageUid, $rootLine, $anchorSection, $viewScript,
 ↪ $additionalGetVars, $switchFocus)
```

Register the hook

Register a hook class which implements the method with the name `postProcess`:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_befunc.php']['
↪'viewOnClickClass'][] = \VENDOR\MyExt\Hooks\BackendUtilityHook::class;
```

Feature: #67236 - Added "allowedTags" argument to f:format.stripTags ViewHelper

See Issue #67236

Description

The argument `allowedTags` containing a list of HTML tags which will not be stripped can now be used on `f:format.stripTags`. Tag list syntax is identical to second parameter of PHP function `strip_tags`.

Impact

New ViewHelper argument (string) `allowedTags` becomes available, allowing passing the tag list as second parameter to `strip_tags`.

Feature: #69394 - EXT:form - Directly load form wizard as inline wizard

See Issue #69394

Description

The wizard of EXT:form is loaded directly as inline wizard. There is no need anymore to save and reload the newly created content element in order to be able to open the wizard. This is a huge usability improvement. Additionally there is no need to provide individual doc headers. Instead, the centralized doc headers of the module template API are used.

The whole integration utilizes the nodeRegistry of formEngine and registers the wizard as new render type.

Furthermore, all JavaScript is loaded via require.js.

Since integrators and editors had massive problems with overridden form configuration the wizard cannot be deactivated anymore. Instead, the integrator can configure whether to load the form wizard by default or not. The following UserTS is integrated by default:

```
setup.default.tx_form.showWizardByDefault = 1
```

Feature: #69794 - Support pecl-memcached in MemcachedBackend

See Issue #69794

Description

Support for the PECL module "memcached" has been added to the MemcachedBackend of the Caching Framework.

Impact

The MemcachedBackend checks if either "memcache" or "memcached" is installed. If both plugins are installed, the MemcachedBackend uses "memcache" over "memcached" to avoid being a breaking change. An integrator may set the option `peclModule` to use the preferred PECL module.

Example code:

```
$GLOBALS['TYPO3_CONF_VARS']['SYS']['caching']['cacheConfigurations']['my_memcached
↪'] = [
        'frontend' => \TYPO3\CMS\Core\Cache\Frontend\VariableFrontend::class
        'backend' => \TYPO3\CMS\Core\Cache\Backend\MemcachedBackend::class,
        'options' => [
                'peclModule' => 'memcached',
                'servers' => [
                   'localhost',
                   'server2:port'
                ]
        ]
];
```

Feature: #69863 - Use new standalone Fluid as composer dependency

See Issue #69863

Description

The Fluid rendering engine of TYPO3 CMS is replaced by the standalone capable Fluid which is now included as composer dependency. The old Fluid extension is converted to a so-called Fluid adapter which allows TYPO3 CMS to use standalone Fluid with the many new features this facilitates.

Impact

New features/capabilities have been added in nearly all areas of Fluid. Most importantly: several of the Fluid components which in the past were completely internal and impossible to replace, are now easy to replace and have been fitted with a public API. Unless noted otherwise each feature below is part of such a new API component which you can replace. This gives you unprecedented control over almost all of Fluid's behaviours in a way that can be controlled via each View instance, for example from within a controller's initializeView method.

Developers working with just the template side of Fluid will not notice many differences. Those that they will notice are, for the most part, hidden away beneath a compatibility flag that is set via the View, allowing each extension to opt-in to the extended capabilities or to stick with the legacy behaviour of Fluid. The new features that relate to these new behaviours are:

RenderingContext

The most important new piece of public API is the RenderingContext. The previously internal-only RenderingContext used by Fluid has been expanded to be responsible for a vital new Fluid feature: implementation provisioning. This enables a developer to change a range of classes Fluid uses for parsing, resolving, caching etc. by either including a custom RenderingContext or manipulating the default RenderingContext by public methods, a developer is able to change a range of classes Fluid uses for parsing, resolving, caching etc.

Each component which can be replaced this way is described in further detail below.

Impact for template developers

The following behaviours can all be controlled by manipulating the RenderingContext. By default, none of them are enabled - but calling a simple method (via your View instance) allows you to enable them:

```
$view->getRenderingContext()->setLegacyMode(false);
```

Doing so causes the RenderingContext to deliver different implementations, which simply means that in addition to what you already know Fluid to be capable of (variable access, inline syntax etc.) you gain access to the following features:

ExpressionNodes

Expression Nodes are a new type of Fluid syntax structures which all share a common trait: they only work inside the curly braces previously only used by variable accessing and inline syntax of ViewHelpers. You can define the exact collection of ExpressionNodes that are active for your rendering process, via the View instance:

```
$view->getRenderingContext()->setExpressionNodeTypes(array(
        'Class\Number\One',
        'Class\Number\Two'
));
```

When added to this collection these Expression Node types allow new syntaxes such as `{myVariable + 1}` or `{myArrayLikeObject as array}`. When the legacy mode toggle is set to `false` this will enable the following expression types:

1. CastingExpressionNode - this type allows casting a variable to certain types, for example to guarantee an integer or a boolean. It is used simply with an `as` keyword: `{myStringVariable as boolean}`, `{myBooleanVariable as integer}` and so on. Attempting to cast a variable to an incompatible type causes a standard Fluid error.
2. MathExpressionNode - this type allows basic mathematical operations on variables, for example `{myNumber + 1}`, `{myPercent / 100}`, `{myNumber * 100}` and so on. An impossible expression returns an empty output.
3. TernaryExpressionNode - this type allows an inline ternary condition which only operates on variables. The use case is "if this variable then use that variable else use another variable". It is used as `{myToggleVariable ? myThenVariable : myElseVariable}`. Note that it does not support any nested expressions, inline ViewHelper syntaxes or similar inside it - it must be used only with standard variables as input.

Developers can add their own additional ExpressionNodeTypes. Each one consists of a pattern to be matched and methods dictated by an interface to process the matches - any existing ExpressionNode type can be used as reference.

Namespaces are extensible

Fluid now allows each namespace alias (for example `f:`) to be extended by adding to it additional PHP namespaces that are also checked for the presence of ViewHelper classes. This is what allows TYPO3 CMS to transparently add just the ViewHelpers that are unique to TYPO3 CMS and let Fluid add the rest. It also means that developers can override individual ViewHelpers with custom versions and have their ViewHelpers called when the `f:` namespace is used.

This change also implies that namespaces are no longer monadic - any time you use `{namespace f=My\Extension\ViewHelpers}` you will no longer receive an error with "namespace already registered". Fluid will instead add this PHP namespace and look for ViewHelpers there as well. Additional namespaces are checked from the bottom up, allowing the additional namespaces to override ViewHelper classes by placing them in the same scope (e.g. `f:format.nl2br` can be overridden with `My\Extension\ViewHelpers\Format\Nl2brViewHelper` given the namespace registration example above.

The behaviour is used both for legacy namespace registration in curly braces and the modern `xmlns` approach using a container HTML tag.

Rendering using f:render

This specific ViewHelper is fundamentally different in the standalone Fluid version - in the default (current) usage scenarios it behaves completely like you are used to, but has been fitted with some major impact features that you can use whenever you want to, in any template.

There are two specific changes both documented in their respective commits:

1. Default content (when section/partial is missing) now possible - https://github.com/TYPO3Fluid/Fluid/commit/cd67f9d974bc489058bde1c4272b480eb349da09
2. Tag content of `f:render` can now be passed as a variable to the section/partial being rendered (essentially becoming a wrapping/block strategy) - https://github.com/TYPO3Fluid/Fluid/commit/454121cba81baed4e3fe526412ff3e14f7c499a9

All TagBasedViewHelpers natively support data- prefixed attributes

Simply put - any TagBasedViewHelper can now receive `data-` prefixed attributes without requiring those attributes to be declared by the ViewHelper. Any suffix can be used as long as the prefix is `data-`.

Complex conditional statements

As a forced new feature - which is backwards compatible - Fluid now supports any degree of complex conditional statements with nesting and grouping:

```
<f:if condition="({variableOne} && {variableTwo}) || {variableThree} ||
↪{variableFour}">
        // Done if both variable one and two evaluate to true, or if either␣
↪variable three or four do.
</f:if>
```

In addition, `f:else` has been fitted with an "elseif"-like behavior:

```
<f:if condition="{variableOne}">
        <f:then>Do this</f:then>
        <f:else if="{variableTwo}">Do this instead if variable two evals true</f:
↪else>
        <f:else if="{variableThree}">Or do this if variable three evals true</f:
↪else>
```

```
        <f:else>Or do this if nothing above is true</f:else>
</f:if>
```

Dynamic variable name parts

Another forced new feature, likewise backwards compatible, is the added ability to use sub-variable references when accessing your variables. Consider the following Fluid template variables array:

```
$mykey = 'foo'; // or 'bar', set by any source
$view->assign('data', ['foo' => 1, 'bar' => 2]);
$view->assign('key', $mykey);
```

With the following Fluid template:

```
You chose: {data.{key}}.
(output: "1" if key is "foo" or "2" if key is "bar")
```

The same approach can also be used to generate dynamic parts of a string variable name:

```
$mydynamicpart = 'First'; // or 'Second', set by any source
$view->assign('myFirstVariable', 1);
$view->assign('mySecondVariable', 2);
$view->assign('which', $mydynamicpart);
```

With the following Fluid template:

```
You chose: {my{which}Variable}.
(output: "1" if which is "First" or "2" if which is "Second")
```

This syntax can be used anywhere a variable is referenced, with one exception: variables passed as pure variable accessors cannot contain dynamic parts, e.g. the following will **NOT** work:

```
{f:if(condition: my{which}Variable, then: 'this', else: 'that')}
```

Whereas the following **will** work because the variables are accessed wrapped in a text node:

```
{f:if(condition: '{my{which}Variable}', then: 'this', else: 'that')}
```

In other words: unless your outer variable reference is enclosed with curly braces, Fluid does not detect that you are referencing a dynamic variable and will instead assume you meant a variable actually named `my{which}Variable` which was added as `$view->assign('my{which}Variable','value')`.

New ViewHelpers

A few new ViewHelpers have been added to the collection as part of standalone Fluid and as such are also available in TYPO3 from now on:

- `f:or` which is a shorter way to write (chained) conditions. It supports syntax like `{variableOne -> f:or(alternative: variableTwo) -> f:or(alternative: variableThree)}` which checks each variable and outputs the first one that's not empty.
- `f:spaceless` which can be used in tag-mode around template code to eliminate redundant whitespace and blank lines for example caused by indenting ViewHelper usages.

Improved error reporting

Syntax errors or problems with required arguments or incorrect argument types will now be reported with line number and template code example from the line that fails. Any ViewHelper Exception is turned into this improved

error type by converting it to a special syntax error and attaching the original Exception to it.

An example error could be:

```
TYPO3Fluid\Fluid\Core\Parser\Exception: Fluid parse error in template
Default_action_Default_1cb8dc11e29962882f629f79c0b9113ff33d6219,line
11 at character 3. Error: The ViewHelper "<f:serender>" could not
be resolved. Based on your spelling,the system would load the class
"TYPO3Fluid\Fluid\ViewHelpers\SerenderViewHelper",however this class
does not exist. (1407060572). Template code: <f:serender section="Foo"
optional="1">. A stack trace is still included if TYPO3 does not run in Production context.
```

Impact for extension developers

Extension developers are affected mainly by gaining access to a range of new APIs that control Fluid's behavior. These new APIs can all be accessed via the RenderingContext which is available in Views and ViewHelpers (also when compiled). Developers can provide custom implementations or manipulate the standard implementations by retrieving each API through the RenderingContext and using methods of those.

There are no significant changes to best practices and the ViewHelper API (which you use when creating custom ViewHelpers) remains largely untouched. The most notable change is that `$this->renderingContext` in ViewHelpers and Views now allows direct access to on-the-fly changes in Fluid's behavior.

RenderingContext as implementation API

Rather than just being a simple context which hangs on to variables, the RenderingContext has been given a completely new and even more vital role in Fluid - it is now the API for delivering custom implementations for a range of features that until now were only possible to achieve via means like XCLASSing. A RenderingContext now delivers the following components:

- The VariableProvider (previously known as TemplateVariableContainer, see below) used in rendering
- The ViewHelperVariableContainer (already known) used in rendering
- The ViewHelperResolver (new pattern) responsible for handling namespaces and resolving/creating ViewHelper instances and arguments
- The ViewHelperInvoker (new pattern) responsible for calling ViewHelpers (circumvented when ViewHelpers implement a custom `compile()` method)
- The TemplatePaths (new pattern) which is a template file resolving class that now contains resolving methods previously found on the View itself
- The TemplateParser (already known) which is responsible for parsing the template and creating a ParsedTemplate
- The TemplateCompiler (already known) which is responsible for converting a ParsedTemplate to a native PHP class
- The FluidCache (new pattern) which is a custom caching implementation compatible with TYPO3 caching frontends/backends storing PHP files
- An array of ExpressionNodeTypes (class names, new pattern) - see description of those above
- An array of TemplateProcessors (instances, new pattern) which pre-process template source code before it gets handed off to the TemplateParser, allowing things like extracting registered namespaces in custom ways.
- The controller name, if one applies to the context
- The controller action name, if one applies to the context
- And for TYPO3 CMS only, the Extbase ControllerContext (which is as it has always been; contains a Request etc.).

All (!) of which can be replaced with custom implementations and all of which are accessible through View and ViewHelpers alike. Just a few of the capabilities you gain:

- You can create custom VariableProvider implementations which retrieve variables in new ways from new sources - Fluid itself now includes a JSON-based VariableProvider as well as a ChainedVariableProvider which allows "plugging" multiple variable sources.
- You can create a custom ViewHelperResolver implementation which can do things like automatically register namespaces that are always available or change the way ViewHelper classes are detected, instantiated, how arguments are detected, and more.
- You can create a custom ViewHelperInvoker implementation which calls ViewHelpers in new ways - combined with a custom ViewHelperResolver this can for example allow non-ViewHelper classes to be used as if they actually were ViewHelpers.
- You can create custom TemplatePaths implementations which for example read template sources not from the local file system but from database, remote storage, zip files, whatever you desire.
- You can replace the TemplateParser itself (but be careful if you do, obviously). There are no current use cases for this, but the possibility exists.
- You can replace the TemplateCompiler (be careful here too). No use case exists but this could be used to compile Fluid templates to other things than PHP.
- You can replace the Cache implementation - for example to cache compiled Fluid templates in memcache or a distributed cache accessible by PHP opcache.
- You can change which Expression Node types are possible to use in templates rendered with your context, for example disabling ternary expressions or adding a custom type of expression of your own.
- You can change which TemplateProcessors will be used to process templates when rendered with your context, to do whatever you like - transform, analyse and so on the template source.

All of these parts are possible to replace via the provided RenderingContext - you don't necessarily have to create your own - but when creating multiple implementations it is often easier to combine those in a custom RenderingContext and just provide that for your View.

But perhaps most importantly, because all of these components are contained in the RenderingContext which is available to Views and ViewHelpers alike (also once compiled!), it becomes possible for your View or ViewHelpers to actually interact with the Fluid environment in powerful ways. To illustrate how powerful, you could create a single ViewHelper which: manipulates the Expression Node types usable in its tag content, changes the paths used to resolve Partials, registers a number of other ViewHelper namespaces, changes the variable source to be a JSON file or URL and adds a pre-processing class that triggers on every template source read from within the ViewHelper's tag contents, to strip some undesired namespace from third party Partials. And it could restore the context afterwards so that all of this only applies inside that ViewHelper's tag content.

ViewHelper namespaces can be extended also from PHP

By accessing the ViewHelperResolver of the RenderingContext, developers can change the ViewHelper namespace inclusions on a global (read: per View instance) basis:

```
$resolver = $view->getRenderingContext()->getViewHelperResolver();
// equivalent of registering namespace in template(s):
$resolver->registerNamespace('news', 'GeorgRinger\News\ViewHelpers');
// adding additional PHP namespaces to check when resolving ViewHelpers:
$resolver->extendNamespace('f', 'My\Extension\ViewHelpers');
// setting all namespaces in advance, globally, before template parsing:
$resolver->setNamespaces(array(
        'f' => array(
                'TYPO3Fluid\\Fluid\\ViewHelpers',
                'TYPO3\\CMS\\Fluid\\ViewHelpers',
                'My\\Extension\\ViewHelpers'
        ),
        'vhs' => array(
```

```
            'FluidTYPO3\\Vhs\\ViewHelpers',
            'My\\Extension\\ViewHelpers'
        ),
        'news' => array(
                'GeorgRinger\\News\\ViewHelpers',
        );
));
```

By "extending" a namespace Fluid adds additional lookup namespaces when detecting ViewHelper classes and uses the last added path first, allowing you to replace ViewHelpers by placing a class with the same sub-name in your own ViewHelpers namespace that extends Fluid's. Doing so also allows you to change the arguments the ViewHelper accepts/requires.

ViewHelpers can accept arbitrary arguments

This feature allows your ViewHelper class to receive any number of additional arguments using any names you desire. It works by separating the arguments that are passed to each ViewHelper into two groups: those that are declared using `registerArgument` (or render method arguments), and those that are not. Those that are not declared are then passed to a special function - `handleAdditionalArguments` - on the ViewHelper class, which in the default implementation throws an error if additional arguments exist. So by overriding this method in your ViewHelper you can change if and when the ViewHelper should throw an error on receiving unregistered arguments.

This feature is also the one allowing TagBasedViewHelpers to freely accept arbitrary `data-` prefixed arguments without failing - on TagBased ViewHelpers, the `handleAdditionalArguments` method simply adds new attributes to the tag that gets generated and throws an error if any additional arguments which are neither registered nor prefixed with `data-` are given.

ViewHelpers automatically compilable

All ViewHelpers, including those you write yourself, are now automatically compilable. This means you no longer have to care about implementing the CompilableInterface or a custom `compile()` function, and that every Fluid template can now be cached to a compiled PHP script regardless of ViewHelpers.

ViewHelpers still are able to define a custom `compile()` function but are no longer required to do so. When they don't define such a method, an execution is chosen which is identical in performance to calling the ViewHelper from a template that before this could not be compiled. The ViewHelpers that do define a custom compiling method can further increase performance.

When you explicitly require a ViewHelper of yours to prevent template caching it is possible to implement a custom `compile()` method which calls `$templateParser->disable();` and nothing else. Doing this disables the compiling inside the scope (template, partial or section) currently being rendered.

New and more efficient escaping

Contrary to earlier versions of Fluid which used a ViewHelperNode for `f:format.htmlentities` around other nodes it wished to escape, standalone Fluid has implemented a custom SyntaxTreeNode type which does the escaping in a more efficient manner (directly using `htmlentities`). Although it means you cannot override this escaping behaviour by overriding the `f:format.htmlentities` ViewHelper (which is completely possible to do with Fluid now) it should mean a significant boost to performance as it avoids an excessive amount of ViewHelper resolving and -rendering operations, replacing them with a single PHP function call wrapped in a tiny class, which compiles also to a single function call and which compiles in a way that it wraps the compiled output of the Node it escapes as a pure string operation.

Escaping interception is still contained within the `Configuration` instance given to the TemplateParser - and those can be manipulated with a custom RenderingContext (see above).

Feature: #71331 - Make indexed_search extbase plugin form target Pid configurable

See Issue #71331

Description

The search form target page of the extbase variant of EXT:indexed_search can now be configured by using the TypoScript option `plugin.tx_indexedsearch.settings.targetPid = 123`.

If it is empty, the current page will be used.

Feature: #71876 - Make new content element wizard tab sort order configurable

See Issue #71876

Description

It is possible to influence the order of the tabs in the new content element wizard by setting `before` and `after` values in Page TSconfig:

```
mod.wizards.newContentElement.wizardItems.special.before = common
mod.wizards.newContentElement.wizardItems.forms.after = common,special
```

Feature: #72045 - HTMLparser.stripEmptyTags.keepTags

See Issue #72045

Description

A new option for the `HTMLparser.stripEmptyTags` configuration is added. It allows keeping configured tags. Before this change only a list of tags could be provided that should be removed.

The following example will strip all empty tags **except** `tr` and `td` tags.

```
HTMLparser.stripEmptyTags = 1
HTMLparser.stripEmptyTags.keepTags = tr,td
```

Important! If this setting is used the `stripEmptyTags.tags` configuration will have no effect any more. You can only use one option at a time.

Impact

Unless the configuration of the `HTMLparser is changed`, the stripEmptyTags feature will work as before.

Feature: #72309 - EXT:form - Integration of Predefined Forms

See Issue #72309

Description

The content element of EXT:form now allows the integration of predefined forms. An integrator can define forms - for example within a site package - using `plugin.tx_form.predefinedForms`. An editor can add a new `mailform` content element to a page and choose a form from a list of predefined elements.

There are even more advantages:

- Integrators can build there forms with TypoScript which offers much more possibilities than doing it within the form wizard. Especially, the integrator is able to use stdWrap functionality which are not available when using the form wizard (for security reasons).
- There is no need anymore for editors to use the form wizard. They can choose the predefined forms which are optimized layout-wise.
- Forms can be re-used throughout the whole installation.
- Forms can be stored outside the DB and versioned.

In order to be able to select the pre-defined form in the backend, the form has to be registered using PageTS.

```
TCEFORM.tt_content.tx_form_predefinedform.addItems.contactForm = LLL:EXT:my_theme/
↪Resources/Private/Language/locallang.xlf:contactForm
```

Example form:

```
plugin.tx_form.predefinedForms.contactForm = FORM
plugin.tx_form.predefinedForms.contactForm {
   enctype = multipart/form-data
   method = post
   prefix = contact
   confirmation = 1

   postProcessor {
      1 = mail
      1 {
         recipientEmail = test@mail.com
         senderEmail = test@mail.com
         subject {
            value = Contact form
            lang.de = Kontakt Formular
         }
      }
   }

   10 = TEXTLINE
   10 {
      name = name
      type = text
      required = required
      label {
         value = Name
         lang.de = Name
      }
      placeholder {
         value = Enter your name
         lang.de = Name eingeben
      }
   }

   20 = TEXTLINE
   20 {
      name = email
      type = email
```

```
        required = required
        label {
            value = Email
            lang.de = E-Mail
        }
        placeholder {
            value = Enter your email address
            lang.de = E-Mail Adresse eingeben
        }
    }

    30 = TEXTAREA
    30 {
        name = message
        cols = 40
        rows = 5
        required = required
        label {
            value = Message
            lang.de = Nachricht
        }
        placeholder {
            value = Enter your message
            lang.de = Nachricht eingeben
        }
    }

    40 = SUBMIT
    40 {
        name = 5
        type = submit
        value {
            value = Send
            lang.de = Senden
        }
    }
}
```

Feature: #72337 - Charset Conversion Autodetection

See Issue #72337

Description

The Charset Converter which is used to handle multi-byte charset conversions now always uses `mbstring`. In case `mbstring` is not present a polyfill will kick in.

Impact

The options `$TYPO3_CONF_VARS['SYS'][t3lib_cs_utils]` and `$TYPO3_CONF_VARS[SYS][t3lib_cs_convMet` have no effect anymore and can be removed.

Feature: #72505 - Introduce hook to override a record overlay

See Issue #72505

Description

Prior to TYPO3 7 LTS, it was possible to override a record overlay in Web > List. This patch introduces a new hook with the old functionality.

The hook is called with the following signature:

```
/**
 * @param string $table
 * @param array $row
 * @param array $status
 * @param string $iconName
 * @return string the new (or given) $iconName
 */
function postOverlayPriorityLookup($table, array $row, array $status, $iconName)
```

Register the hook

Register the hook class which implements the method with the name `postOverlayPriorityLookup`:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][IconFactory::class]['overrideIconOverlay
↪'][] = \VENDOR\MyExt\Hooks\IconFactoryHook::class;
```

Feature: #72904 - Add preProcessStorage signal to ResourceFactory

See Issue #72904

Description

This patch introduces a new signal before a resource storage is initialized.

Register the class which implements your logic in `ext_localconf.php`:

```
$dispatcher = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Extbase\SignalSlot\Dispatcher::class);
$dispatcher->connect(
    \TYPO3\CMS\Core\Resource\ResourceFactory::class,
    ResourceFactoryInterface::SIGNAL_PreProcessStorage,
    \MY\ExtKey\Slots\ResourceFactorySlot::class,
    'preProcessStorage'
);
```

The method is called with the following arguments:

- int `$uid` the uid of the record
- array `$recordData` all record data as array
- string `$fileIdentifier` the file identifier

Feature: #73042 - Introduce native support for Symfony Console

See Issue #73042

Description

TYPO3 supports the Symfony Console component out-of-the-box now by providing a new Command Line script located in `typo3/sysext/core/bin/typo3`. On TYPO3 instances installed via Composer, the binary is linked into the `bin-dir`, e.g. `bin/typo3`.

The new binary still supports the existing command-line arguments when no proper Symfony Console command was found as a fallback.

Registering a command to be available via the `typo3` command line tool works by putting a `Configuration/Commands.php` file into any installed extension. This lists the Symfony/Console/Command classes to be executed by `typo3` is an associative array. The key is the name of the command to be called as the first argument to `typo3`.

A required parameter when registering a command is the `class` property which should inherit from Symfony's base Command class.

A `Configuration/Commands.php` could look like this:

```
return [
    'backend:lock' => [
        'class' => \TYPO3\CMS\Backend\Command\LockBackendCommand::class
    ],
    'referenceindex:update' => [
        'class' => \TYPO3\CMS\Backend\Command\ReferenceIndexUpdateCommand::class
    ]
];
```

An example call could look like:

```
bin/typo3 backend:lock http://www.mydomain.com/maintenance.html
```

For a non-Composer installation:

```
typo3/sysext/core/bin/typo3 backend:lock http://www.mydomain.com/maintenance.html
```

Impact

Using Symfony Commands and calling `typo3` instead of using `typo3/cli_dispatch.phpsh` is now the preferred way for writing command line code.

Feature: #73050 - Add a CSPRNG API

See Issue #73050

Description

A new cryptographically secure pseudo-random number generator (CSPRNG) has been introduced in TYPO3 core. It takes advantage of the new CSPRNG functions in PHP 7.

API overview

The API resides in the class `\TYPO3\CMS\Core\Crypto\Random`. It provides several methods. Here is a brief overview of the interface:

```
class Random {
    /**
     * Generates cryptographic secure pseudo-random bytes
     */
    public function generateRandomBytes($length);

    /**
     * Generates cryptographic secure pseudo-random integers
     */
    public function generateRandomInteger($min, $max);

    /**
     * Generates cryptographic secure pseudo-random hex string
     */
    public function generateRandomHexString($length);
}
```

Example

```
use \TYPO3\CMS\Core\Crypto\Random;
use \TYPO3\CMS\Core\Utility\GeneralUtility;

// Retrieving random bytes
$someRandomString = GeneralUtility::makeInstance(Random::class)->
↪generateRandomBytes(64);

// Rolling the dice..
$tossedValue = GeneralUtility::makeInstance(Random::class)->generateRandomInteger(1,
↪ 6);
```

Impact

None, you can start to use the CSPRNG in your code by now.

Feature: #73429 - Wizard component has been added

See Issue #73429

Description

A new wizard component has been added. This component may be used for user-guided interactions. The RequireJS module can be used by including `TYPO3/CMS/Backend/Wizard`.

The wizard supports straight forward actions only, junctions are not possible yet.

Impact

The wizard component has the following public methods:

- `addSlide(identifier,title,content,severity,callback)`
- `addFinalProcessingSlide(callback)`
- `set(key,value)`
- `show()`

- `dismiss()`
- `getComponent()`
- `lockNextStep()`
- `unlockNextStep()`

addSlide

Adds a slide to the wizard.

Name	DataType	Mandatory	Description
identifier	string	Yes	The internal identifier of the slide
title	string	Yes	The title of the slide
content	string	Yes	The content of the slide
severity	int		Represents the severity of a slide. Please see TYPO3.Severity. Default is `TYPO3.Severity.info`.
callback	function		Callback method run after the slide appeared. The callback receives two parameters: `$slide`: The current slide as a jQuery object `settings`: The settings defined via `Wizard.set()`

addFinalProcessingSlide

Adds a slide to the wizard containing a spinner. This should always be the latest slide. This method returns a Promise object due to internal handling. This means you have to add a `done()` callback containing `Wizard.show()` and `Wizard.getComponent()` please see the example below.

Name	DataType	Mandatory	Description
callback	function		Callback method run after the slide appeared. If no callback method is given, the wizard dismisses without any further action.

Example code:

```
Wizard.addFinalProcessingSlide().done(function() {
    Wizard.show();

    Wizard.getComponent().on('click', '.my-element', function(e) {
        e.preventDefault();
        $(this).doSomething();
    });
});
```

set

Adds values to the internal settings stack usable in other slides.

Name	DataType	Mandatory	Description
key	string	Yes	The key of the setting
value	string	Yes	The value of the setting

Events

The event `wizard-visible` is fired when the wizard rendering has finished.

Example code:

```
Wizard.getComponent().on('wizard-visible', function() {
    Wizard.unlockNextButton();
});
```

Wizards can be closed by firing the `wizard-dismiss` event.

Example code:

```
Wizard.getComponent().trigger('wizard-dismiss');
```

Wizards fire the `wizard-dismissed` event if the wizard is closed. You can integrate your own listener by using `Wizard.getComponent()`.

Example code:

```
Wizard.getComponent().on('wizard-dismissed', function() {
    // Calculate the answer of life the universe and everything
});
```

Feature: #73720 - Trigger event after modal window dismissed

See Issue #73720

Description

A new event `modal-destroyed` has been added that will be triggered after a modal window closed.

Impact

Bind to the event `modal-destroyed` to achieve custom actions after the modal has been dismissed.

Example code:

```
var $modal = Modal.confirm(); // stub code
$modal.on('modal-destroyed', function() {
        // Reset the selection
        $someCombobox.val('');
});
```

Feature: #73752 - Allow accessing ObjectStorage as array in Fluid and other places

See Issue #73752

Description

Creates an alias of `toArray()` allowing the method to be called as `getArray()` which in turn allows the method to be called transparently from `ObjectAccess::getPropertyPath`, enabling access in Fluid and other places.

Impact

By creating an extremely simple aliasing of `toArray()` on ObjectStorage allowing it to be called as `getArray()` enables:

```
ObjectAccess::getPropertyPath($subject, 'objectstorageproperty.array.4') to get␣
↪the 4th element
```

```
{myObject.objectstorageproperty.array.4} in Fluid (including {myObject.
↪objectstorageproperty.array.{dynamicIndex}} in v8)
```

Feature: #74038 - Report for checking database character set

See Issue #74038

Description

If a database has e.g. latin1 as default character set, new tables or fields created by TYPO3 will be created with this default charset as utf-8 is not enforced. A report has been added that checks the default charset and warns administrators if a wrong charset is used.

Impact

If the default database character set is not utf-8, the report warns administrators about a wrong charset.

Feature: #74109 - Set the alternative Backend Logo via Extension Manager

See Issue #74109

Description

The Backend Logo in the upper left corner can now be configured in the Extension Configuration of EXT:backend within the Extension Manager. A relative path to the TYPO3 installation ("PATH_site"), e.g. "fileadmin/myfile.jpg" or a path to an extension, e.g. "EXT:my_theme/Resources/Public/Icons/Logo.png" can be configured there.

The configuration option within the Backend extension (EXT:backend) is called `backendLogo`.

Impact

The previously available `$GLOBALS[TBE_STYLES][logo]` option has no effect anymore.

Feature: #74179 - Page Module Drag & Drop Can Do Copies Via CTRL Key Now

See Issue #74179

Description

Additionally to the usual drag and drop in the page module, that was able to move content elements from position A to B, you can do copies via drag & drop now. Just press the CTRL-key while dropping to create a copy of the dragged element at the drop zone position. After dropping is finished the page module will reload to make sure the new element will be generated with all necessary information.

Feature: #74319 - Default database character set and update wizard for non UTF-8

See Issue #74319

Description

If you install TYPO3 to an existing database with a default charset other than utf-8, TYPO3 will create tables with the default charset of that database. The install tool should check the default charset and notify the user if it is not utf-8.

Furthermore the install tool should check for this issue too and provide an update wizard to fix this (=set the default charset to utf-8 and NOT convert existing tables to utf-8). A default charset set other than utf-8 leads to non-utf-8 tables when updating the database via the install tool or installing extensions.

Impact

During installation on database select the default charset of the database is checked. If it is not utf-8 the installation will not proceed and the user is notified of the issue.

For existing installations the install tool also provides an environment check and an upgrade wizard which changes the default database character set. The update wizard will NOT convert any existing tables though!

2.9.3 Deprecation

Deprecation: #68748 - Deprecate AbstractContentObject::getContentObject()

See Issue #68748

Description

The method has been renamed to `getContentObjectRenderer()`. The old method name is still present as a deprecated alias, which will be removed in TYPO3 v9.

Impact

Calling this method will trigger a deprecation log entry.

Affected Installations

Any extensions calling `getContentObject()`.

Migration

Replace calls to `getContentObject()` with `getContentObjectRenderer()`.

Deprecation: #69863 - Deprecate getTemplateVariableContainer function

See Issue #69863

Description

`RenderingContext->getTemplateVariableContainer` has been marked as deprecated in favor of getVariableProvider due to a changed concept for variable provisioning in standalone fluid. It now does more than just contain variables so a change in naming is necessary.

Impact

Calling this method directly will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using `getTemplateVariableContainer` method directly within an extension or third-party code.

Migration

Use `getVariableProvider` instead of `getTemplateVariableContainer`.

Deprecation: #71255 - ExtendedFileUtility::pushErrorMessagesToFlashMessageQueue()

See Issue #71255

Description

Method `\TYPO3\CMS\Core\Utility\File\ExtendedFileUtility::pushErrorMessagesToFlashMessageQ`
has been marked as deprecated.

Impact

Using the method will trigger a deprecation log entry.

Affected Installations

Instances with custom backend modules that use this method.

Migration

Implement the method by yourself.

Deprecation: #71153 - Several DocumentTemplate methods deprecated

See Issue #71153

Description

The following methods from `TYPO3\CMS\Backend\Template\DocumentTemplate` have been marked as deprecated.

`section` `divider` `sectionHeader` `sectionBegin` `sectionEnd`

Impact

Using these methods will trigger a deprecation log entry.

Affected Installations

Instances with custom backend modules that use one of the aforementioned methods.

Migration

Use plain HTML instead.

Deprecation: #71916 - LanguageService->makeEntities

See Issue #71916

Description

The method `LanguageService->makeEntities()` was used when the TYPO3 Backend ran with non-utf8 characters in order to convert UTF-8 characters to latin1. This is not needed anymore as all is UTF-8 now.

Impact

Using `$GLOBALS['LANG']->makeEntities()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using a third-party extension using the PHP method above.

Deprecation: #72340 - Moved moduleLabels from LanguageService to ModuleLoader

See Issue #72340

Description

Labels for registered modules were previously stored within the LanguageService class. The logic has been moved to the ModuleLoader class. The method `LanguageService->addModuleLabels()` and the property `LanguageService->moduleLabels` have been marked as deprecated.

Impact

Calling `LanguageService->addModuleLabels()` will trigger a deprecation log entry. The property `LanguageService->moduleLabels` will no longer contain the expected values anymore.

Affected Installations

Any installation with extensions that directly access the labels for a given module.

Migration

Use `ModuleLoader->addLabelsForModule()` and `ModuleLoader->getLabelsForModule` instead.

Deprecation: #72496 - Deprecated $LANG->overrideLL

See Issue #72496

Description

The method `LanguageService::overrideLL()` has been marked as deprecated.

Impact

Calling this method directly will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using the LanguageService method directly within an extension or third-party code.

Deprecation: #72733 - Deprecate more methods of DocumentTemplate

See Issue #72733

Description

The following methods from `TYPO3\CMS\Backend\Template\DocumentTemplate` have been marked as deprecated:

- `wrapInCData`
- `funcMenu`
- `getDragDropCode`
- `getTabMenu`
- `getVersionSelector`

The following method from `TYPO3\CMS\Backend\Template\ModuleTemplate` have been marked as deprecated:

- `getVersionSelector`

Impact

Calling one of the aforementioned methods will trigger a deprecation log entry.

Affected Installations

Instances with custom backend modules that use one of the aforementioned methods.

Migration

Some replacements are available in the `\TYPO3\CMS\Backend\Template\ModuleTemplate` class.

Some other functionality is moved to separate classes when explicitly calling the version selector.

Deprecation: #72827 - Module Icon configuration via [labels][tabs_images][tab]

See Issue #72827

Description

When registering a non-extbase module, the option to configure an icon was previously done with the module configuration option `[labels][tabs_images][tab]`. To clarify the naming, the property "icon" is introduced which expects a reference to the icon via the `EXT:myextension/path/to/the/file.png` syntax.

The old option `[labels][tabs_images][tab]` has been marked as deprecated.

Impact

When using the old configuration property `[labels][tabs_images][tab]`, a deprecation message is thrown.

Affected Installations

Installations with custom backend non-extbase modules of third-party-extensions that still use the old configuration property.

Migration

Replace the `[labels][tabs_images][tab]` with `[icon]` in `ext_tables.php` in your extension like this:

```
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::addModule(
    'system',
    'dbint',
    '',
    '',
    array(
        'routeTarget' => \TYPO3\CMS\Lowlevel\View\DatabaseIntegrityView::class . '::
↪mainAction',
        'access' => 'admin',
        'name' => 'system_dbint',
        'workspaces' => 'online',
        'icon' => 'EXT:lowlevel/Resources/Public/Icons/module-dbint.svg',
        'labels' => 'LLL:EXT:lowlevel/Resources/Private/Language/locallang_mod.xlf'
    )
);
```

Deprecation: #72851 - Deprecate some functions not in use anymore in the core

See Issue #72851

Description

The following unused methods have been marked as deprecated:

`BackendUtility::processParams()` `BackendUtility::makeConfigForm()`
`BackendUtility::titleAltAttrib()` `BackendUtility::getSQLselectableList()`
`ContentObjectRenderer->processParams()`

Impact

Calling one of the aforementioned methods will trigger a deprecation log entry.

Affected Installations

Instances with custom backend modules that use one of the aforementioned methods.

Deprecation: #72856 - Removed RTE "modes" option

See Issue #72856

Description

The RTE "modes" option that was added to a RTE enabled TCA field in the "defaultExtras" section has been removed.

The RTE is now loaded via the configuration from TSconfig, usually set by "modes" or "overruleMode" (used by default), and loaded even without the RTE mode set in the TCA field defaultExtras section.

Impact

Extension authors do not need to set the defaultExtras "mode=ts_css" parameter explicitly.

Migration

When configuring a RTE field in a TYPO3 extension the defaultExtras part should bet set to `richtext:rte_transform` instead of `richtext:rte_transform[mode=ts_css]` in order to render the RTE.

Flexform

Example for an RTE Field, used in a Flexform with CMS 8 after migration

```
<text>
    <TCEforms>
        <label>LLL:EXT:extension_name/Resources/Private/Language/locallang_db.xlf:
↪flexform.text.element.labelname</label>
        <config>
            <type>text</type>
            <size>10</size>
            <rows>5</rows>
            <enableRichtext>true</enableRichtext>
        </config>
        <defaultExtras>
```

```
            <richtext>rte_transform</richtext>
        </defaultExtras>
    </TCEforms>
</text>
```

Deprecation: #73050 - Deprecated random generator methods in GeneralUtility

See Issue #73050

Description

The method `\TYPO3\CMS\Core\Utility\GeneralUtility::generateRandomBytes()` has been marked as deprecated in favor of `\TYPO3\CMS\Core\Crypto\Random->generateRandomBytes()`.

Also the method `\TYPO3\CMS\Core\Utility\GeneralUtility::getRandomHexString()` has been marked as deprecated in favor of `\TYPO3\CMS\Core\Crypto\Random->generateRandomHexString()`.

Impact

Calling this methods directly will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance that use `GeneralUtility::generateRandomBytes()` or `GeneralUtility::getRandomHexString()` directly within an extension or third-party code.

Migration

Replace calls to `GeneralUtility::generateRandomBytes()` with `GeneralUtility::makeInstance(Random::class)->generateRandomBytes()`.

Also replace calls to `GeneralUtility::getRandomHexString()` with `GeneralUtility::makeInstance(Random::class)->generateRandomHexString()`.

Deprecation: #73067 - Deprecate GeneralUtility::requireOnce and GeneralUtility::requireFile

See Issue #73067

Description

The following methods from `TYPO3\CMS\Core\Utility\GeneralUtility` have been marked as deprecated.

`GeneralUtility::requireOnce()` `GeneralUtility::requireFile()`

Impact

Using aforementioned methods will trigger a deprecation log entry.

Affected Installations

Instances which use one of the aforementioned methods.

Migration

Use native require_once if needed, e.g. if autoloading does not work.

Deprecation: #73068 - Deprecated "default" argument on f:case

See Issue #73068

Description

Due to the switch to Fluid standalone, the following template markup is not supported anymore:

```
<f:case default="true"> ... </f:case>
```

It must be changed to read:

```
<f:defaultCase> ... </f:defaultCase>
```

A compatibility layer is in place which logs the deprecation.

Impact

Use of "default" argument on f:case will trigger a deprecation log entry.

Affected Installations

Any TYPO3 site which uses f:case with argument "default" regardless of the argument value.

Migration

Switch `f:case` to `f:defaultCase` for the case node that is your default case.

Deprecation: #73185 - Deprecate NullTimeTracker

See Issue #73185

Description

The class `\TYPO3\CMS\Core\TimeTracker\NullTimeTracker` has been marked as deprecated in favor of the TimeTracker class.

Impact

Calling this class directly will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using `\TYPO3\CMS\Core\TimeTracker\NullTimeTracker` directly within an extension or third-party code.

Migration

Initialize `\TYPO3\CMS\Core\TimeTracker\TimeTracker` with false as first parameter.

Deprecation: #73190 - Deprecate BackendUtility::getListViewLink()

See Issue #73190

Description

The method `BackendUtility::getListViewLink()` has been marked as deprecated and will be removed in TYPO3 v9.

Impact

Calling the method above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with a third-party extension calling this method directly.

Migration

Use the `IconFactory` and `BackendUtility::getModuleUrl()` API methods directly instead to create links.

Deprecation: #73352 - Deprecate old-school AJAX requests

See Issue #73352

Description

The class `\TYPO3\CMS\Core\Http\AjaxRequestHandler` and the associated methods `ExtensionManagementUtility::registerAjaxHandler()` and `\TYPO3\CMS\Backend\Http\AjaxRequestHandler::dispatchTraditionalAjaxRequest()` have been marked as deprecated and will be removed in TYPO3 v9.

Impact

Calling any of the methods will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with a third-party extensions using the class `\TYPO3\CMS\Core\Http\AjaxRequestHandler` or calling any of the methods directly.

Migration

Use the Backend Ajax Routes logic instead.

Deprecation: #73442 - Modal.getSeverityClass has been moved to the Severity module

See Issue #73442

Description

The method `Modal.getSeverityClass` has been moved to `Severity.getCssClass`. `Modal.getSeverityClass` has been marked as deprecated.

Impact

Calling `Modal.getSeverityClass` will trigger a console warning in the browser.

Affected Installations

All 3rd party extensions using `Modal.getSeverityClass` are affected.

Migration

Change the calls to `Severity.getCssClass(severity)`.

Deprecation: #73482 - $LANG->csConvObj and $LANG->parserFactory

See Issue #73482

Description

The properties of LanguageService (also known as `$GLOBALS[LANG]`) csConvObj and parserFactory have been marked as deprecated. Since these three PHP classes are not dependent on each other, they can be decoupled. The getter method `getParserFactory()` has thus been marked as deprecated as well.

Impact

These properties will be removed in TYPO3 v9. Calling `LanguageService->getParserFactory()` will trigger a deprecation log entry.

Affected Installations

Installations with custom extension accessing the LanguageService properties and method above.

Migration

Instantiate CharsetConverter (csConvObj) and LocalizationFactory (parserFactory) via `GeneralUtility::makeInstance` directly if needed, as they are Singleton objects and then fetched from the General Utility Object container functionalities.

Deprecation: #73511 - BrowserLanguage detection moved to Locales

See Issue #73511

Description

The CharsetConverter contained the calculation to find the corrent language based on the browsers language settings at the backend login screen.

The according code has been moved to TYPO3CMSLocalizationLocales. The method `CharsetConverter::getPreferredClientLanguage()` and the property `CharsetConverter::charSetArray` have been marked as deprecated.

Impact

Calling `CharsetConverter::getPreferredClientLanguage()` will trigger a deprecation log entry.

Affected Installations

All installations with a third-party extension using the CharsetConverter language resolving directly.

Migration

Use the method `Locales->getPreferredClientLanguage()` instead.

Deprecation: #73514 - IncludeLibrary Methods

See Issue #73514

Description

The PHP methods `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController->includeLibrarie` and `\TYPO3\CMS\Frontend\ContentObject\ContentObjectRenderer->includeLibs()` to include PHP libraries during frontend output have been marked as deprecated.

Impact

Calling any of the methods above will trigger a deprecation log entry.

Affected Installations

Any installation using custom Frontend output modified via PHP and TypoScript through e.g. a custom CType provided by a special extension.

Migration

Use proper object orientation and class loading methods to load code in the Frontend when necessary.

Deprecation: #73516 - Various GeneralUtility methods

See Issue #73516

Description

The following methods within GeneralUtility have been marked as deprecated:

```
GeneralUtility::flushOutputBuffers()
GeneralUtility::xmlGetHeaderAttribs()
GeneralUtility::imageMagickCommand()
```

The second and third parameter of `GeneralUtility::getFileAbsFileName()` have been removed as well.

Impact

Calling any of the methods above will trigger a deprecation log entry. Calling `GeneralUtility::getFileAbsFileName()` with the second and third parameter set will also trigger a deprecation log entry.

Affected Installations

Any installation using any third-party extension calling any of these methods.

Migration

For `GeneralUtility::flushOutputBuffers()` use `ob_clean()`.

For `GeneralUtility::imageMagickCommand()` use `CommandUtility::imageMagickCommand`.

Deprecation: #72585 - Deprecate TYPO3CMSCoreResourceUtilityBackendUtility::getFlashMessageForMissingFile

See Issue #72585

Description

`TYPO3\CMS\Core\Resource\Utility\BackendUtility::getFlashMessageForMissingFile` has been marked as deprecated.

Impact

Calling this method will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using `getFlashMessageForMissingFile` method within an extension or third-party code.

Migration

No migration

Deprecation: #73606 - Deprecate IconRegistry::getDeprecationSettings

See Issue #73606

Description

`IconRegistry::getDeprecationSettings` has been marked as deprecated.

Impact

Using `IconRegistry::getDeprecationSettings()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using a third-party extension using the PHP method above.

Deprecation: #73744 - Deprecate Clipboard->confirmMsg()

See Issue #73744

Description

`Clipboard::confirmMsg` has been marked as deprecated.

Impact

Using `Clipboard::confirmMsg()` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 instance using a third-party extension using the PHP method above.

Migration

Use `Clipboard::confirmMsgText()` to get the confirm message and make use of the Modal API.

Deprecation: #73794 - SearchFormController->utf8_to_currentCharset and TSFE->renderCharset

See Issue #73794

Description

The public method `SearchFormController->utf8_to_currentCharset()` within indexed search has been marked as deprecated.

The public property `TypoScriptFrontendController->renderCharset` has been marked as deprecated.

Impact

Calling the method above will result in a deprecation log entry.

Affected Installations

Any installation that extends Indexed Search and uses this method, and any extension that accesses `TSFE->renderCharset` via TypoScript or PHP directly.

Migration

Remove any calls to the method and the public property.

Deprecation: #74022 - GraphicalFunctions->prependAbsolutePath()

See Issue #74022

Description

The method `GraphicalFunctions->prependAbsolutePath()` has been marked as deprecated.

Impact

Calling the method above will trigger a deprecation log entry.

Affected Installations

Any installation with custom extensions that use GraphicalFunctions and the method directly.

Migration

Use `GeneralUtility::getFileAbsFileName()` instead.

Deprecation: #74156 - TemplateService::sortedKeyList and TemplateService->removeQueryString

See Issue #74156

Description

The methods `TemplateService::sortedKeyList()` and `TemplateService->removeQueryString()` have been marked as deprecated.

Impact

Calling one of the methods above will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with a custom extension that uses these PHP methods.

Migration

Use `ArrayUtility::filterAndSortByNumericKeys` and `rtrim($url,'?')` as drop-in replacements.

2.9.4 Important

Important: #22858 - Filelist: Creating a new file and opening it immediately for editing

See Issue #22858

Description

A little usability improvement has been added when creating new files in fileadmin. Pressing the button "Create file" creates a new file and immediately opens the file for editing.

Important: #70849 - Make search levels in live search and list search consistent

See Issue #70849

Description

In order to make the searchlevel handling consistent between live and list search a new PageTS option has been added.

```
mod.web_list.searchLevel.items {
        -1 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.infinite
        0 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.0
        1 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.1
        2 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.2
        3 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.3
        4 = EXT:lang/Resources/Private/Language/locallang_core.xlf:labels.
↪searchLevel.4
}
```

This makes it possible to add custom search level entries.

Important: #71521 - Internal changes in CommandController

See Issue #71521

Description

The `CommandController::processRequest()` method has been changed to initialize arguments and output.

If this method was overridden without calling the parent method, these changes must be copied to prevent errors.

Important: #72290 - Move install tool update flags to system registry

See Issue #72290

Description

The flags to mark install tool updates (aka "wizards") as done have been moved from `LocalConfiguration.php` to the system registry in the database.

The update can be performed from the install tool.

Important: #72580 - Publicly accessible generated asset files moved to typo3temp/assets/

See Issue #72580

Description

The folder structure within typo3temp/ was changed to separate assets that need to be accessed by the client from the files that are temporary created for e.g. caching or locking purposes and only need server-side access.

These assets were moved from the folders `\_processed\_`, `compressor`, `GB`, `temp`, `Language`, `pics` and organized into `typo3temp/assets/js/`, `typo3temp/assets/css/`, `typo3temp/assets/compressed/` and `typo3temp/assets/images/`.

Important: #72697 - Remove thumbnail functionality of impexp

See Issue #72697

Description

Previously it was possible to add additional thumbnails to exports of the EXT:impexp. This functionality is completely outdated and not needed anymore.

CHAPTER

THREE

CHANGELOG V7

Every change to the TYPO3 Core which might affect your site is documented here.

3.1 7.6.x Changes

Changes after 7.6.0 LTS release.

Table of contents

- *Breaking Changes*
- *Features*
- *Important*

3.1.1 Breaking Changes

Breaking: #72931 - SearchFormController::pi_list_browseresults() has been renamed

See Issue #72931

Description

In order to make Indexed Search pi-based plugin PHP7 compatible, the `SearchFormController::pi_list_browseresults()` method has been renamed to `SearchFormController::renderPagination()`. Parameter types, order and count has been preserved. However the methods visibility has been changed to protected.

Impact

Call to old method name will result in fatal error "Call to undefined method".

Affected Installations

Any installation of TYPO3 7.6 or TYPO3 8 where SearchFormController is overloaded (XCLASSed) and new class contains call to old method name and any code that called the public method from outside the class.

Migration

Rename `pi_list_browseresults()` to `renderPagination()`.

Calling the method from outside the class is no longer possible.

Breaking: #73461 - Import module disabled for non admin users

See Issue #73461

Description

The import module of extension "impexp" has been disabled for non-admin users by default.

Impact

For non-admin users who need that functionality, the userTsConfig option `options.impexp.enableImportForNonAdminUser = 1` must be set. This can have a negative security impact to the TYPO3 instance in core versions 7.6 and 6.2 and should only be enabled for "trustworthy" backend users in general.

Affected Installations

Installations with non-admin users making active use of the import / export module

Migration

Set userTsConfig option `options.impexp.enableImportForNonAdminUser = 1` to restore the old behavior.

3.1.2 Features

Feature: #73461 - Enable import module for non admin users

See Issue #73461

Description

The new userTsConfig option `options.impexp.enableImportForNonAdminUser` can be used to enable the import module of EXT:impexp for non admin users.

Impact

This option should be enabled for "trustworthy" backend users only.

3.1.3 Important

Important: #17904 - showAccessRestrictedPages does not work with special menus

See Issue #17904

Description

HMENU setting `showAccessRestrictedPages=NONE` now acts as documented in Common properties.

Before: using the option renders `<a>Page title</a>` when page is inaccessible.

After: using the option renders `<a href="index.php?id=123">Page title</a>` when page is not accessible.

Important: #75400 - New DataHandler command 'copyToLanguage'

See Issue #75400

Description

A new DataHandler command 'copyToLanguage' has been introduced. It behaves like 'localize' command (both record and child records are copied to given language), but does not set transOrigPointerField fields (e.g. l10n_parent).

The 'copyToLanguage' command should be used when localizing records in the "Free Mode". This command is used when localizing content elements using translation wizard's "Copy" strategy.

The 'localize' DataHandler command should be used when translating records in "Connected Mode" (strict translation of records from the default language). This command is used when selecting "Translate" strategy in content elements translation wizard.

Important: #77411 - Removed extbase table column cache

See Issue #77411

Description

The extbase table column cache "extbase_typo3dbbackend_tablecolumns", which was used to store all database fields of all database tables, was removed.

The associated configuration variable `$TYPO3_CONF_VARS[SYS][caching][cacheConfigurations][extbase_typ` can be removed.

Important: #77830 - CSC-HeaderLinkRespectsGlobalPageTarget

See Issue #77830

Description

Setting the global configuration `lib.parseTarget` was not respected by the header_link field. Now the configuration is properly applied and might change the output in the frontend.

Impact

If the global setting `lib.parseTarget` is set, the field header_link will now respect it. If in addition the target of header_link is set in a content element, it will take precedence over `lib.parseTarget`.

3.2 7.6 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

3.2.1 Breaking Changes

Breaking: #24449 - Use move placeholders as default in workspaces

See Issue #24449

Description

The ctrl section of each TCA table has the property "versioningWS" which might be set to "1" (enabled) or to "2" which enables "move placeholders" functionality.

The "move placeholders" are now active by default, removing the possibility to have a "simple workspace" concept which does not consider sorting records inside a workspace.

Impact

All checks in TYPO3 consider all TCA tables that have workspaces enabled ("versioningWS") to be move-placeholder-aware. All TCA tables that only have non-moveable-records in workspace now need the DB table field "t3ver_moveid" to be added.

All existing TCA configurations with "versioningWS" can now simply be set to TRUE instead of "2".

Affected Installations

Any installation with third-party extensions that use workspace functionality but do not have move-placeholder-enabled records.

Migration

Make all TCA tables "move-placeholders" aware by adding the necessary database field "t3ver_moveid".

Breaking: #45899 - Split class ImportExport into classes Import and Export

See Issue #45899

Description

Class TYPO3CMSImpexpImportExport (typo3/sysext/impexp/Classes/ImportExport.php) is split into a class dedicated for import and another one for export.

Impact

Using and extending the class is not possible any more.

Affected Installations

Those which use the class and its methods directly or extend the class.

Migration

Use or extend one or both of the new classes (TYPO3CMSImpexpImport and TYPO3CMSImpexpExport).

Breaking: #51099 - Streamline settings/conditions

See Issue #51099

Description

The default TypoScript for the `pi2` (extbase) plugin of EXT:indexed_search sets a TypoScript variable `plugin.tx_indexedsearch.settings.displayRules = 1`, while the default fluid template uses the TypoScript setting `plugin.tx_indexedsearch.settings.showRules`. This change makes the default fluid template for the extbase plugin of EXT:indexed_search use the correct TypoScript setting.

Impact

Instances of the extbase plugin (`pi2`) of EXT:indexed_search using the default fluid template and explicitly configured to *hide* the search rules deliberately using the wrong TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0` *will show* the search rules after this update.

Affected Installations

Installations using the extbase plugin (`pi2`) of EXT:indexed_search with default template and relying on the undocumented TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0`.

Migration

Change all occurrences of the TypoScript setting `plugin.tx_indexedsearch.settings.showRules = 0` to `plugin.tx_indexedsearch.settings.displayRules = 1`.

Breaking: #62812 - Resolve URLs to "Link to external URL"-pages directly

See Issue #62812

Description

The behaviour of pages with the type "Link to External URL" has changed for menus. Those pages link now directly to the provided url instead of linking the internal page with a redirect afterwards.

Breaking: #63406 - Respect rootLevel configuration in extbase queries

See Issue #63406

Description

The rootLevel of a table can be configured to 0, 1 or -1 in TCA, to define where records of a table can be found in the system:

- 0: In the page tree only
- 1: Only on the root page (pid 0)
- -1: Both, on the root page and in the page tree

Currently only 0 and 1 are respected by the `Typo3DbQueryParser` when building the pageId statement. This means that a rootLevel of -1 does not get any pageId statement at all and therefore ignores any `storagePid` configuration for extbase plugins.

Impact

Custom records that have a configuration like this `$GLOBALS['TCA']['tx_myext_domain_model_record']['ctrl'] = -1` and are used in extbase plugins might have trouble finding the records if `plugin.tx_myext.persistence.storagePid` is not configured properly.

Affected Installations

Third party code using `$GLOBALS['TCA']['tx_myext_domain_model_record']['ctrl']['rootLevel'] = -1` with records within the page tree and without a proper `storagePid` configuration.

Migration

Set `plugin.tx_myext.persistence.storagePid` to the page ids you want to find records from. 0 does not need to be included as it is added to the statement automatically.

Breaking: #66369 - Removed ElementBrowser related classes

See Issue #66369

Description

The following classes have been removed without replacement

- TYPO3CMSCoreElementBrowserElementBrowserHookInterface
- TYPO3CMSRecordlistBrowserElementBrowser
- TYPO3CMSRtehtmlareaBrowseLinks
- TYPO3CMSRtehtmlareaFolderTree
- TYPO3CMSRtehtmlareaPageTree

Impact

Any code still using the aforementioned classes will cause a fatal error.

Affected Installations

Any code still using the aforementioned classes.

Migration

Use the new API for adding element browsers or link handlers.

Breaking: #69227 - Strings for like are not properly escaped

See Issue #69227

Description

The following constants have been removed:

- `\TYPO3\CMS\IndexedSearch\Controller\SearchFormController::WILDCARD_LEFT`
- `\TYPO3\CMS\IndexedSearch\Controller\SearchFormController::WILDCARD_RIGHT`
- `\TYPO3\CMS\IndexedSearch\Domain\Repository\IndexSearchRepository::WILDCARD_LEFT`
- `\TYPO3\CMS\IndexedSearch\Domain\Repository\IndexSearchRepository::WILDCARD_RIGHT`

Impact

The mentioned constants don't exist anymore.

Affected Installations

Third party code using the mentioned constants.

Migration

Use the new constants:

- `\TYPO3\CMS\IndexedSearch\Utility\LikeWildcard::LEFT`
- `\TYPO3\CMS\IndexedSearch\Utility\LikeWildcard::RIGHT`

Breaking: #69916 - Hook ajaxSaveCode of t3editor changed

See Issue #69916

Description

The `$ajaxObj` parameter has been replaced by PSR-7-compliant `$request` and `$response` objects.

Impact

Using the `$ajaxObj` parameter will result in a fatal error.

Affected Installations

All 3rd party extensions using the `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ext/t3editor/classes/cl` hook are affected.

Migration

Make use of ServerRequestInterface and ResponseInterface, see `typo3/sysext/t3editor/Classes/Hook/FileEditHoo` for reference.

Breaking: #69916 - Registered AJAX handlers replaced by routes

See Issue #69916

Description

AJAX handlers registered in the core by `ExtensionManagementUtility::registerAjaxHandler()` have been replaced by AJAX routes, which are registered inside any extension under Configuration/Backend/AjaxRoutes.php.

The routes registered in AjaxRoutes.php are available via JavaScript via `TYPO3.settings.ajaxUrls[routeIdentifier]`.

Impact

Calling removed AJAX identifiers will result in an error. Please see the table below for migration.

Affected Installations

All 3rd party extensions using one of the removed handlers is affected.

Migration

Please see the table to get the new AJAX identifier.

EXT:backend

Old identifier	New identifier	New AJAX ID
SC_alt_db_navframe::expandCollapse	sc_alt_db_navframe_expandtoggle	/ajax/sc-alt-db-navframe/expandtoggle
SC_alt_file_navframe::expandCollapse	sc_alt_file_navframe_expandtoggle	/ajax/sc-alt-file-navframe/expandtoggle
TYPO3_tcefile::process	file_process	/ajax/file/process
TYPO3_tcefile::fileExists	file_exists	/ajax/file/exists
t3lib_TCEforms_inline::createNewRecord	record_inline_create	/ajax/inline/create
t3lib_TCEforms_inline::getRecordDetails	record_inline_details	/ajax/inline/record-details
t3lib_TCEforms_inline::synchronizeLocalizeRecords	record_inline_synchronizelocalize	/ajax/inline/synchronizelocalize
t3lib_TCEforms_inline::setExpandedCollapsedState	record_inline_expandcollapse	/ajax/inline/expandcollapse
t3lib_TCEforms_suggest::searchRecord	record_suggest	/ajax/wizard/suggest/search
ShortcutMenu::getShortcutEditForm	shortcut_editform	/ajax/shortcut/editform
ShortcutMenu::saveShortcut	shortcut_saveform	/ajax/shortcut/saveform
ShortcutMenu::render	shortcut_list	/ajax/shortcut/list
ShortcutMenu::delete	shortcut_remove	/ajax/shortcut/remove
ShortcutMenu::create	shortcut_create	/ajax/shortcut/create
SystemInformationMenu::load	systeminformation_render	/ajax/system-information/render
ModuleMenu::reload	modulemenu	/ajax/module-menu
BackendLogin::login	login	/ajax/login
BackendLogin::logout	logout	/ajax/logout
BackendLogin::refreshLogin	login_refresh	/ajax/login/refresh
BackendLogin::isTimedOut	login_timedout	/ajax/login/timedout
ExtDirect::getAPI	ext_direct_api	/ajax/ext-direct/api
ExtDirect::route	ext_direct_route	/ajax/ext-direct/route
DocumentTemplate::getFlashMessages	flashmessages_render	/ajax/flashmessages/render
ContextMenu::load	contextmenu	/ajax/context-menu
DataHandler::process	record_process	/ajax/record/process
UserSettings::process	usersettings_process	/ajax/user-settings/process
ImageManipulationWizard::getHtmlForImageManipulationWizard	wizard_image_manipulation	/ajax/wizard/image-manipulation
LiveSearch	livesearch	/ajax/livesearch
OnlineMedia::add	online_media_create	/ajax/online-media/create

EXT:beuser

Old identifier	New identifier	New AJAX ID
PermissionAjaxController::dispatch	user_access_permissions	/users/access/permissions

EXT:context_help

Old identifier	New identifier	New AJAX ID
ContextHelpAjaxController::dispatch	context_help	/context-help

EXT:opendocs

Old identifier	New identifier	New AJAX ID
TxOpendocs::renderMenu	opendocs_menu	/opendocs/menu
TxOpendocs::closeDocument	opendocs_close	/opendocs/close

EXT:recycler

Old identifier	New identifier	New AJAX ID
RecyclerAjaxController::dispatch	recycler	/recycler

EXT:rsaauth

Old identifier	New identifier	New AJAX ID
BackendLogin::getRsaPublicKey	rsa_publickey	/rsa/publickey
RsaEncryption::getRsaPublicKey	rsa_publickey	/rsa/publickey

EXT:rtehtmlarea

Old identifier	New identifier	New AJAX ID
rtehtmlarea::spellchecker	rtehtmlarea_spellchecker	/rte/spellchecker

EXT:t3editor

Old identifier	New identifier	New AJAX ID
T3Editor::saveCode	t3editor_save	/t3editor/save
T3Editor::getPlugins	t3editor_get_plugins	/t3editor/get-plugins
T3Editor_TSrefLoader::getTypes	t3editor_tsref	/t3editor/tsref
T3Editor_TSrefLoader::getDescription	t3editor_tsref	/t3editor/tsref
CodeCompletion::loadTemplates	t3editor_codecompletion_loadtemplates	/t3editor/codecompletion/load-templates

- T3Editor_TSrefLoader::getTypes and T3Editor_TSrefLoader::getDescription have been combined. The separation is done by the new parameter `fetch` being either "types" or "description".

EXT:taskcenter

Old identifier	New identifier	New AJAX ID
Taskcenter::saveCollapseState	taskcenter_collapse	/taskcenter/collapse
Taskcenter::saveSortingState	taskcenter_sort	/taskcenter/sort

EXT:workspaces

Old identifier	New identifier	New AJAX ID
Workspaces::setWorkspace	workspace_switch	/workspaces/switch

Breaking: #69916 - Removed BackendLogin::getRsaPublicKey AJAX handler

See Issue #69916

Description

The deprecated AJAX handler `BackendLogin::getRsaPublicKey` has been removed in favor of `rsa_publickey`. As `getRsaPublicKey` was the only method in this class, the file `typo3/sysext/rsaauth/Classes/Backend/AjaxLoginHandler.php` has been removed without substitution.

Impact

Calling the removed handler will result in an error.

Affected Installations

All 3rd party extensions using the removed handler are affected.

Migration

Use the AJAX handler `rsa_publickey` instead of `BackendLogin::getRsaPublicKey`.

Breaking: #70033 - TCA icon options have been removed

See Issue #70033

Description

The `TCA` configurations `noIconsBelowSelect`, `foreign_table_loadIcons` and `suppress_icons` for select fields with the render type `selectSingle` have been removed.

Impact

The old TCA settings `noIconsBelowSelect`, `foreign_table_loadIcons` and `suppress_icons` are ignored and deprecation log entries will be triggered. A migration handles the update of the settings.

Affected Installations

All installations with extensions that configure the icon table visibility of TCA select fields with one of the old settings.

Migration

Extension authors need to use the new option `showIconTable` to define the visibility of the icon table for their select fields.

Breaking: #70055 - Override New Content Element Wizard via page TSconfig

See Issue #70055

Description

In the past it was possible to override the "New Content Element Wizard" via custom scripts when using page TSconfig via `mod.web_list.newContentWiz.overrideWithExtension = myextension` to define an extension, which then needed a file placed under `mod1/db_new_content_el.php`. The script was then called with certain parameters instead of the wizard.

The new way of handling entry-points and custom scripts is now built via modules and routes. The former option `mod.web_list.newContentWiz.overrideWithExtension` has been removed and a new option

`mod.newContentElementWizard.override` has been introduced instead. Instead of setting the option to a certain extension key, a custom module or route has to be specified.

Example:

```
mod.newContentElementWizard.override = my_custom_module
```

Impact

Using the old TSconfig option `mod.web_list.newContentWiz.overrideWithExtension` has no effect anymore and will fallback to the regular new content element wizard provided by the TYPO3 Core.

Affected Installations

Any installation using this option with extensions providing custom New Content Element Wizards, e.g. templavoila.

Migration

The extension providing the script must be changed to register a route or module and set the TSconfig option to the route identifier, instead of a raw PHP script. Any usages in TSconfig need to be adapted to use the new TSconfig option.

Breaking: #70132 - FormEngine custom functions

See Issue #70132

Description

Due to the refactoring of the backend FormEngine code the "low end" extension API to manipulate data has changed. Affected are especially the `type=user` `TCA` element, any `userFunc` configured in `TCA` as well as the `itemsProcFunc` to manipulate single items in select, group and other types.

In general data given to those custom functions has changed and extensions that rely on this data may fail. For instance, if a `itemsProcFunc` was defined for a field within a flex form, the `row` array argument contained the full parent database row in the past. This is no longer the case and the parent database row is now transferred as `flexParentDatabaseRow`. In other cases data previously handed over to custom functions may no longer be available at all.

Impact

Custom functions receive less or different options than before and may stop working.

Affected Installations

Extensions using the `TCA` with `type=user` fields, extensions using `TCA` with `userFunc` and extensions using `itemsProcFunc`.

Migration

Developers using this API have to debug the data given to custom functions and adapt accordingly.

If the data given is not sufficient it is possible to register own element classes with the `NodeFactory` or to manipulate data by adding a custom `FormDataProvider`. While the current API will be mostly stable throughout further TYPO3 CMS 7 LTS patch releases, it may however happen that the given API and data breaks again with the development of the TYPO3 CMS 8 path to make the FormEngine code more powerful and reliable in the end.

Breaking: #70229 - BE-lockSSL = 3 option removed

See Issue #70229

Description

The global option `$TYPO3_CONF_VARS[BE][lockSSL]` allows to lock the backend usage to be worked completely over SSL. Setting this option to "3" allowed to have only the backend login transmitted via SSL, but the rest forced to work via plain HTTP. Option "3" has been removed in favor of having a full SSL session for all communication between the server and the client / browser.

Impact

Installations having `lockSSL` set to "3" will now behave just as it would be lockSSL=1.

Affected Installations

Any installation that has `$TYPO3_CONF_VARS[BE][lockSSL]` set to 3, only having SSL for the Backend login page.

Migration

It is recommended to set the `$TYPO3_CONF_VARS[BE][lockSSL]` option to 1 or 2, depending on the environment and the possibilities of having SSL available.

Breaking: #70444 - EXT:form - Form attributes are not rendered in FE

See Issue #70444

Description

The TypoScript configuration of EXT:form has been streamlined. Useless attributes for the specific form elements have been removed. Additionally, missing attributes have been added. Furthermore, the array notation of `htmlAttributes` and `htmlAttributesUsedByTheViewHelperDirectly` has changed. The whole cleanup was done to provide a solid configuration for the LTS version.

Impact

The removed attributes will not be available anymore out of the box for the specific form element. Custom TypoScript which copied, referenced or removed certain attribute configurations will not work anymore.

Affected Installations

Any installation that relies on the structure of `htmlAttributes` and `htmlAttributesUsedByTheViewHelperDirectly`. Since the whole configuration has not been documented yet and the functionality has been introduced with 7.5 the possibility that a lot of installations customize the configuration is very low.

Migration

Affected installations have to re-add the missing attributes manually and adopt the new array notation.

Breaking: #70503 - EXT:form - Remove breakOnError option from wizard

See Issue #70503

Description

The validation option `breakOnError` is not supported anymore. The option has been removed completely.

Impact

The validation process cannot be interrupted anymore, i.e. the whole form will be validated and all error messages will be shown.

Affected Installations

Any installation that implements the `breakOnError` functionality.

Migration

The function has been removed without any substitution. Therefore there is no migration possible. The attribute can be manually removed from the form configuration. The wizard ignores the existence of `breakOnError`. Since TYPO3 7.5 it is possible to utilize HTML5 attributes to validate form elements on the fly. This could be used to emulate the behaviour.

Breaking: #70574 - Form Wizard Save Handling Changed

See Issue #70574

Description

The ExtJS wizard of EXT:form in the backend has been using an old "module" to load and to save the data from the wizard, but has been misusing this functionality as AJAX responses. All AJAX requests for the wizard are now built with AJAX Routes and PSR-7-based Request/Response objects.

All obsolete WizardView PHP classes have been removed without substitution:

- TYPO3CMSFormViewWizardAbstractWizardView
- TYPO3CMSFormViewWizardLoadWizardView
- TYPO3CMSFormViewWizardSaveWizardView

Impact

Using these now non-existent PHP classes will result in fatal errors or wrong results when calling them directly.

Affected Installations

Any installations with extensions that hook into the wizard views of EXT:form.

Migration

Use the AJAX routes available via `TYPO3.settings.ajaxUrls['formwizard_load']` and `TYPO3.settings.ajaxUrls['formwizard_save']`.

Breaking: #70578 - JumpURL functionality removed from the TYPO3 Core

See Issue #70578

Description

The handling and generation of so-called Jump URLs has been moved into its own extension called "jumpurl" and is now available in the TYPO3 Extension Repository (TER), and available via composer as the package name "friendsoftypo3/jumpurl".

Impact

If the functionality was used in an installation before (e.g. with Direct Mail or via TypoScript), this functionality is not working anymore.

Affected Installations

All installations where Jump URLs were used.

Migration

Download and install the extension "jumpurl" from the TER.

Breaking: #71110 - TYPO3-specific Upload Limit removed

See Issue #71110

Description

TYPO3 has a specific upload limit setting, that is set to 10MB by default, to manually limit down the PHP-specific setting `max_upload_limit`. If not configured properly the PHP limit was lower than the TYPO3-specific limit.

The TYPO3 setting `$TYPO3_CONF_VARS['BE']['maxFileSize']` has been removed and the PHP-internal limit is now the upper barrier.

Impact

Setting the option mentioned above has no effect anymore. The PHP limit is used instead.

The TCA setting `max_size` for `fe_users.image` has been removed, allowing editors to upload images with a size up to the PHP-specific limit.

Affected Installations

Extensions that use `$GLOBALS['TYPO3_CONF_VARS']['BE']['maxFileSize']` as default for the TCA setting `max_size` need adjustment, if the PHP-specific upload limit is higher than `$TYPO3_CONF_VARS['BE']['maxFileSize']`.

Migration

Explicitly set a value for `max_size` or drop those lines from your TCA configuration.

Breaking: #72117 - API change in ExceptionHandlerInterface

See Issue #72117

Description

The class Throwable was added in PHP7 as new parent of Exceptions. This leads to the issue that ExceptionHandlers need to change the API of their exception handling method. To support PHP 5.5, 5.6 and 7.0 we need to remove the type hint. It will later be set to Throwable if we only support PHP 7.0 and newer. See http://php.net/manual/en/migration70.incompatible.php

Impact

A fatal error will be thrown if you use own ExceptionHandlers implementing TYPO3CMSCoreErrorExceptionHandlerInterface "Fatal error: Declaration of ... must be compatible with ..."

Affected Installations

Installations which use an own ExceptionHandler implementing TYPO3s ExceptionHandlerInterface.

Migration

Remove the type hinting in your implementation of ExceptionHandlerInterface. If you switch to PHP 7 you may also get instances from Throwable, so check the API/type hinting of the function were you process the exception.

Breaking: #77344 - EXT:form - Rename configuration for confirmation view

See Issue #77344

Description

The typoscript key `configuration` for the confirmation configuration has been renamed.

Up until now the layout settings of the confirmation view could be addressed via `tt_content.mailform.20.confirmation.layout`. This setting was introduced with patch 28526 but never documented.

Besides this, the confirmation view enable setting can be set via `tt_content.mailform.20.confirmation = 1`.

To keep the meaning of the settings clear, it was decided to rename the configuration of the confirmation view.

Impact

Having the confirmation view enabled and the confirmation configuration customized, a naming collision occurs. As a result, the confirmation step has been disabled. Since the configuration was never documented, only few people know about this setting.

Affected Installations

All installations enabling the confirmation view and customizing the layout of this view.

Migration

All occurences of `tt_content.mailform.20.confirmation.layout` have to be migrated to `tt_content.mailform.20.confirmationView.layout`.

3.2.2 Features

Feature: #20875 - Make hardcoded indexed_search parameters configurable via TS

See Issue #20875

Description

The following TS properties can now be configured for indexed search

```
[plugin.tx_indexedsearch.results. || plugin.tx_indexedsearch.settings.results.]
titleCropAfter = 50
titleCropSignifier = ...
summaryCropAfter = 180
summaryCropSignifier =
hrefInSummaryCropAfter = 60
hrefInSummaryCropSignifier = ...
markupSW_summaryMax = 300
markupSW_postPreLgd = 60
markupSW_postPreLgd_offset = 5
markupSW_divider = ...
```

Every TS property has the stdWrap property, too.

Impact

Default settings do not change old behaviour.

Feature: #23156 - Indexed search: Make path separator of search result configurable

See Issue #23156

Description

A new TypoScript configuration option `breadcrumbWrap` has been added. It allows to configure the page path separator used in breadcrumbs in Indexed Search results. This option supports TypoScript option split syntax.

Impact

By default Indexed Search is configured to use "/" as a path separator, so it's backward compatible. Use following configuration for Indexed Search Extbase plugin:

```
plugin.tx_indexedsearch.settings.breadcrumbWrap = / || /
```

For plugin based on AbstractPlugin use:

```
plugin.tx_indexedsearch.breadcrumbWrap = / || /
```

Feature: #27057 - Relations to the same table in Extbase

See Issue #27057

Description

It is now possible to use a domain model where an object is connected to another object of the same class directly

```
namespace \Vendor\Extension\Domain\Model;
class A {
        /**
        * @var \Vendor\Extension\Domain\Model\A
        */
        protected $parent;
```

as well as using a domain model where an object has multiple relations to objects of the same class

```
namespace \Vendor\Extension\Domain\Model;
class A {
        /**
        * @var \Vendor\Extension\Domain\Model\B
        */
        protected $x;

        /**
        * @var \Vendor\Extension\Domain\Model\B
        */
        protected $y;
```

as well as indirectly

```
namespace \Vendor\Extension\Domain\Model;
class A {
        /**
        * @var \Vendor\Extension\Domain\Model\B
        */
        protected $b;
```

```
        /**
        * @var \Vendor\Extension\Domain\Model\C
        */
        protected $c;

namespace \Vendor\Extension\Domain\Model;
class B {
        /**
        * @var \Vendor\Extension\Domain\Model\C
        */
        protected $c;
```

Using this kind of relations before was only possible by overriding the Extbase query builder and doing manual queries because the Extbase query builder created wrong SQL statements. Now Extbase properly supports these cases.

Impact

Extbase now correctly handles relations to objects of the same class.

Feature: #35245 - Rework workspace notification settings

See Issue #35245

Description

The current notification settings have some drawbacks and are not easy to understand if it comes the the expected behavior in the workspace module. The settings are defined in each sys_workspace and sys_workspace_stage record and are evaluated in the workspace module if sending a particular element to be reviewed to the previous or next stage.

Currently there are the following notification settings:

- on stages
  - "edit stage": takes recipients from "adminusers" field (workspace owners)
  - "ready to publish" stage: takes recipients from "members" field (workspace members)
- on preselection of recipients
  - "all (non-strict)": if users from workspace setting (field "adminusers" or "members") are also in the specific "default_users" setting for the stage, the checkbox is enabled by default and cannot be changed, otherwise it's not checked
  - "all (strict)": all users from workspace setting (field "adminusers" or "members") are checked and cannot be changed
  - **"some (strict)": all users from workspace setting (field "adminusers"** or "members") are checked, but still can be changed
- behavior
  - sending to "edit" stage: members are notified per default
  - sending to "ready to publish" stage: owners are notified per default

The changes extends the possibilities to define notification settings:

- on stages
  - add settings for "publish-execute" stage (actual publishing process)

- on preselection of recipients
  - remove modes
  - replace settings for showing the dialog and whether modifying the preselection is allowed at all (getting rid of the "strict" modes)
  - add possibilities to defined notification recipients
    - owner & members as defined in the accordant fields
    - editors that have been working on a particular element
    - responsible persons (on custom stages only)

Impact

The meaning and behavior of the workspaces notification settings concerning preselected recipients and the possibility to modify the selection on moving an element to a particular change is different now. However, an upgrade wizard helps to upgrade the settings to the new definitions.

Feature: #44127 - Introduced two new Hooks for OpenID

See Issue #44127

Description

Two hooks were added to the OpenIdService. They make it possible to modify the request sent to the OpenID Server, or to modify/create backend users on the fly during OpenID login.

Hooks

The following hooks were introduced:

- **`$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['openid']['getUserRecord']`**
  Modifies the userRecord after it has been fetched (or none was found). Can be used to e.g. create a new record if none was found or update an existing one. The following parameters are passed to the hook: `record`, `response`, `authInfo`.
- **`$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['openid']['authRequest']`**
  Modifies the Authentication Request, before it's sent. Can be used to e.g. request additional attributes like a nickname from the OpenID Server. The following parameters are passed to the hook: `authRequest`, `authInfo`.

Feature: #47613 - Indexed Search: make no_cache parameter for forwardSearchWordsInResultLink configurable

See Issue #47613

Description

A new TypoScript configuration option `forwardSearchWordsInResultLink.no_cache` has been added. It controls whether the `no_cache` parameter should be added to page links together with search words.

Use following configuration for Indexed Search Extbase plugin:

```
plugin.tx_indexedsearch.settings.forwardSearchWordsInResultLink.no_cache = 1
```

For plugin based on AbstractPlugin use:

```
plugin.tx_indexedsearch.forwardSearchWordsInResultLink.no_cache = 1
```

Impact

The default value is set to 1, so it's backward compatible.

Feature: #56633 - Form protection API for frontend usage

See Issue #56633

Description

As of now frontend plugins needed to implement CSRF protection on their own. This change introduces a new class to allow usage of the FormProtection (CSRF protection) API in the frontend.

Usage is the same as in backend context:

```
$formToken = \TYPO3\CMS\Core\FormProtection\FormProtectionFactory::get()
        ->getFormProtection()->generateToken('news', 'edit', $uid);


if ($dataHasBeenSubmitted
        && \TYPO3\CMS\Core\FormProtection\FormProtectionFactory::get()->
↪validateToken(
                \TYPO3\CMS\Core\Utility\GeneralUtility::_POST('formToken'),
                'User setup',
                'edit'
        )
) {
        // Processes the data.
} else {
        // Create a flash message for the invalid token or just discard this␣
↪request.
}
```

Impact

FormProtection API can now also be used in frontend context.

Feature: #64286 - Added absolute url option to uri.image and image viewHelper

See Issue #64286

Description

The ImageViewhelper and Uri/ImageViewHelper got a new option `absolute`. With this option you are able to force the ViewHelpers to output an absolute url.

Examples:

```
<code title="ImageViewHelper">
    <f:image image="{file}" width="400" height="375" absolute="1" />
</code>
<output>
    <img alt="alt set in image record" src="http://www.mydomain.com/fileadmin/_
↪processed_/323223424.png" width="400" height="375" />
</output>

<code title="Uri/ImageViewHelper">
    <f:uri.image image="{file}" width="400" height="375" absolute="1" />
</code>
<output>
    http://www.mydomain.com/fileadmin/_processed_/323223424.png
</output>
```

Feature: #66369 - Added LinkBrowser APIs

See Issue #66369

Description

This new feature allows to extend the link browser with new tabs, which allow to implement custom link functionality in a generic way in a so called LinkHandler. Since the LinkBrowser is used by FormEngine and RTE, the new API ensures that your custom LinkHandler works with those two, and possible future, usages flawlessly.

Each tab rendered in the link browser has an associated link handler, responsible for rendering the tab and for creating and editing of links belonging to this tab.

Tab registration

Link browser tabs are registered in page TSconfig like this:

```
TCEMAIN.linkHandler.<tabIdentifier> {
        handler = TYPO3\CMS\Recordlist\LinkHandler\FileLinkHandler
        label = LLL:EXT:lang/locallang_browse_links.xlf:file
        displayAfter = page
        scanAfter = page
        configuration {
                customConfig = passed to the handler
        }
}
```

The options `displayBefore` and `displayAfter` define the order how the various tabs are displayed in the link browser.

The options `scanBefore` and `scanAfter` define the order in which handlers are queried when determining the responsible tab for an existing link. Most likely your links will start with a specific prefix to identify them. Therefore you should register your tab at least before the 'url' handler, so your handler can advertise itself as responsible for the given link. The 'url' handler should be treated as last resort as it will work with any link.

Handler implementation

A link handler has to implement the `\TYPO3\CMS\Recordlist\LinkHandler\LinkHandlerInterface` interface, which defines all necessary methods for communication with the link browser.

Additionally, each link handler should also provide a Javascript module (requireJS), which takes care of passing a link to the link browser. A minimal implementation of such a module looks like this:

```
define(['jquery', 'TYPO3/CMS/Recordlist/LinkBrowser'], function($, LinkBrowser) {

        var myModule = {};

        myModule.createMyLink = function() {
                var val = $('.myElmeent').val();

                // optional: If your link points to some external resource you␣
↪should set this attribute
                LinkBrowser.setAdditionalLinkAttribute('data-htmlarea-external', '1
↪');

                LinkBrowser.finalizeFunction('mylink:' + val);
        };

        myModule.initialize = function() {
                // todo add necessary event handlers, which will propably call␣
↪myModule.createMyLink
        };

        $(myModule.initialize);

        return myModule;
}
```

Notice the call to `LinkBrowser.finalizeFunction`, which is the point where the link is handed over to the link browser for further processing and storage.

Hooks

You may have the need to modify the list of available link handlers based on some dynamic value. For this purpose you can register hooks.

The registration of a link browser hook generally happens in your `ext_tables.php` and looks like:

```
if (TYPO3_MODE === 'BE') {
        $GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['LinkBrowser']['hooks
↪']['1444048118] = [
                'handler' => \Vendor\Ext\MyClass::class,
                'before' => [], // optional
                'after' => [] // optional
        ];
}
```

The `before` and `after` elements allow to control the execution order of all registered hooks.

Currently the following list of hooks is implemented:

- modifyLinkHandlers(linkHandlers, currentLinkParts): May modify the list of available link handlers and has to return the final list.
- modifyAllowedItems(allowedTabs, currentLinkParts): May modify the list of available tabs and has to return the final list.

Feature: #66369 - Added new element browser API

See Issue #66369

Description

The former code monster class `ElementBrowser` has been split into dedicated parts of functionality. Specifically the functionality of selecting elements for the FormEngine and the code parts for creating links, used in FormEngine and RTE, have been moved into separate APIs.

Each type of element, which can be selected in FormEngine, has its own element browser class. You may add your own special type by registering your own element browser in your `ext_tables.php` as follows:

```
if (TYPO3_MODE === 'BE') {
        $GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['ElementBrowsers'][<identifier>]␣
↪= \Vendor\Ext\TheClass::class;
}
```

The registered class is expected to implement the `\TYPO3\CMS\Recordlist\Browser\ElementBrowserInterface` interface.

Feature: #68771 - Add contentObject functionality to form MailPostProcessor and introduce replyToEmail

See Issue #68771

Description

If the form configuration is defined by TypoScript the following items for the MailPostProcessor in ext:form have contentObject functionality now: - subject - senderEmail - senderName - recipientEmail - ccEmail - replyToEmail (newly introduced, replyToEmailField as fallback) - priority - organization

This feature is not available when building the form with the help of the wizard. The functionality can only be used be setting up the form via TypoScript.

Usage

In the mail postProcessor configuration you could do something like this (depending on the names of the form elements):

```
replyToEmail = TEXT
replyToEmail {
        data = GP:tx_form_form|tx_form|e-mail
        htmlSpecialChars = 1
}
subject = TEXT
subject {
        data = GP:tx_form_form|tx_form|subject
        htmlSpecialChars = 1
        noTrimWrap = |Mail from Form: ||
}
```

Feature: #68895 - Introduced hook in BackendUserAuthentication::getDefaultUploadFolder

See Issue #68895

Description

It is now possible to change the upload folder returned by `BackendUserAuthentication::getDefaultUploadFolder()` by registering a hook. This makes it possible to set a different upload folder for fields with direct upload enabled in the backend.

Register own getDefaultUploadFolder hook

To use your own hook to manipulate the upload folder you need to register the function in `ext_localconf.php` of your extension.

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_userauthgroup.php']['getDefaultUploadFolder'][] =
        \Vendor\MyExtension\Hooks\DefaultUploadFolder::class . '->getDefaultUploadFolder';
```

Example getDefaultUploadFolder hook

```
<?php
namespace Vendor\MyExtension\Hooks;

use TYPO3\CMS\Core\Authentication\BackendUserAuthentication;
use TYPO3\CMS\Core\Resource\Folder;

/**
 * Class DefaultUploadFolder
 */
class DefaultUploadFolder {

        /**
         * Get default upload folder
         *
         * If there is a folder present with the same name as the last part of the table name use that folder.
         *
         * @param array $params
         * @param BackendUserAuthentication $backendUserAuthentication
         * @return Folder
         */
        public function getDefaultUploadFolder($params, BackendUserAuthentication $backendUserAuthentication) {

                /** @var Folder $uploadFolder */
                $uploadFolder = $params['uploadFolder'];
                $pid = $params['pid'];
                $table = $params['table'];
                $field = $params['field'];

                $matches = [];
                if (!empty($uploadFolder) && preg_match('/_([a-z]+)$/', $table, $matches)) {
                        $folderName = $matches[1];
                        if ($uploadFolder->hasFolder($folderName)) {
                                $uploadFolder = $uploadFolder->getSubfolder($folderName);
                        }
                }
                return $uploadFolder;
        }
}
```

Feature: #69120 - Add basic file search in element browser

See Issue #69120

Description

A file search has been added to the TYPO3 Element Browser.

The search happens recursively from the currently selected folder in the folder tree. This way it is possible to search whole mount points or just single folders with a lot of files.

Feature: #69706 - Add support for alternative (inline) icon markup

See Issue #69706

Description

It is now possible to set alternative markups for an `Icon`. By default icon is rendered as `<img src="..."/>` tag with path to the icon file in the src attribute. With this change it's possible to render svg icon inline in the html e.g. `<svg>...</svg>`.

Placing SVG images inline allows to manipulate them using CSS or JS.

```
$icon->setAlternativeMarkup(SvgIconProvider::MARKUP_IDENTIFIER_INLINE, '<svg>...</
↪svg>');
```

Impact

An IconProvider can now add multiple markup variants for an icon.

Feature: #69764 - Introduced file icon detection by mime-type

See Issue #69764

Description

The IconRegistry has been extended with a mapping of file icons by mime-type. It is possible to register full mime-types `main-type/sub-type` but also a fallback for only the main part of the mime-type `main-type/*`. The core provides these fallbacks for `audio/*`, `video/*`, `image/*` and `text/*`.

Impact

It is now possible to register or overwrite the iconIdentifier for a file mime-type.

```
$iconRegistry = GeneralUtility::makeInstance(IconRegistry::class);
$iconRegistry->registerMimeTypeIcon('video/my-custom-type', 'icon-identifier-for-my-
↪custom-type');
```

Feature: #69814 - ModuleTemplate API

See Issue #69814

Challenge

Currently all DocHeaders are implemented on their own.

This means we have about 80 DocHeaders which are equal but not the same.

The main challenge is to provide extension developers with all tools they need to build decent backend modules while maintaining control of the docHeader itself.

Solution

We will provide a replacement for DocumentTemplate which provides an easy-to-use API which is on the other hand flexible enough to tackle all tasks we currently think of.

At the same time we will remove the amount of duplicate marker based templates.

The API uses the Fluent-API approach and has been built to supply maximum IDE code completion support.

Parts of a docHeader Currently a typical docHeader is split up into the following sections:

- Top Bar
  - Context Sensitive Help Icon
  - Select Menu(s)
  - Path
  - RecordInformation incl. Clickmenu
- Bottom Bar
  - Left Button Bar
  - Right Button Bar

API Components

Buttons

InputButton Used to generate a <button> element.

LinkButton Used to generate links

SplitButton A mixed component accepting multiple button objects and renders them into a condensed form.

FullyRenderedButton Displays arbitrary HTML code and we highly recommend to use these.

Menus

Creating menus is pretty simple. Ask the `DocHeaderComponent` for the `MenuRegistry` and ask the `MenuRegistry` to create a `Menu` for you.

The `Menu` in return can create `MenuItems` for you.

A `Menu` can have several **Types** which are represented by their respective Fluid Partials in EXT:backend/Resources/Private/Partials/Menu/.

Examples of usages

Adding a button

```
$openInNewWindowButton = $this->moduleTemplate->getDocHeaderComponent()->
↪getButtonBar()
    ->makeLinkButton()
    ->setHref('#')
    ->setTitle($this->getLanguageService()->sL('LLL:EXT:lang/locallang_core.xlf:
↪labels.openInNewWindow', TRUE))
    ->setIcon($this->iconFactory->getIcon('actions-window-open', Icon::SIZE_SMALL))
    ->setOnClick($aOnClick);

$this->moduleTemplate->getDocHeaderComponent()->getButtonBar()
    ->addButton($openInNewWindowButton, ButtonBar::BUTTON_POSITION_RIGHT);
```

Adding a menu with menu items

```
$languageMenu = $this->moduleTemplate->getDocHeaderComponent()->
↪getModuleMenuRegistry()->makeMenu()
    ->setIdentifier('_langSelector')
    ->setLabel($this->getLanguageService()->sL('LLL:EXT:lang/locallang_general.xlf:
↪LGL.language', TRUE));
$menuItem = $languageMenu->makeMenuItem()
    ->setTitle($lang['title'] . $newTranslation)
    ->setHref($href);
if((int)$lang['uid'] === $currentLanguage) {
    $menuItem->setActive(TRUE);
}
$languageMenu->addMenuItem($menuItem);
$this->moduleTemplate->getDocHeaderComponent()->getModuleMenuRegistry()->addMenu(
↪$languageMenu);
```

ButtonBar Hook

The old module rendering knew a `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['typo3/template.php']['d` hook to manipulate buttons. A similar hook is available in ModuleTemplate API as `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['Backend\Template\Components\ButtonBar']['getB`

Registering your own hook

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['Backend\Template\Components\ButtonBar']['
↪getButtonsHook']['MyExt'] =
    \MyVendor\MyExt\Hooks\ButtonBarHook::class . '->getButtons';
```

Example usage of the hook

```
class ButtonBarHook {

    /**
     * Get buttons
     *
     * @param array $params
     * @param ButtonBar $buttonBar
     * @return array
     */
    public function getButtons(array $params, ButtonBar $buttonBar) {
        $buttons = $params['buttons'];

        $iconFactory = GeneralUtility::makeInstance(IconFactory::class);
        $button = $buttonBar->makeLinkButton();
```

```
        $button->setIcon($iconFactory->getIcon('my-custom-icon', Icon::SIZE_SMALL));
↪
        $button->setTitle('My custom docHeader button');
        $button->setOnClick('alert("Hook works");return false;');

        $buttons[ButtonBar::BUTTON_POSITION_LEFT][1][] = $button;

        return $buttons;
    }
}
```

Feature: #69916 - PSR-7-based Routing for Backend AJAX Requests

See Issue #69916

Description

Support for PSR-7-based Routing for Backend AJAX requests has been added.

Impact

To add a route for an AJAX request, create the `Configuration/Backend/AjaxRoutes.php` of your extension:

```
return [
        // Does something
        'unique_route_name' => [
                'path' => '/toolcollection/some-action',
                'target' => \ACME\Controller\SomeController::class . '::myAction',
        ]
];
```

The unique_route_name (route identifier) parameter acts as the previously known key to call `BackendUtility::getAjaxUrl()` passed as parameter to the action refers to the route path, **not** to the route identifier itself. AJAX handlers configured in `AjaxRoutes.php` are **not** compatible with definitions in `ext_localconf.php` registered by `ExtensionManagementUtility::registerAjaxHandler()` due to different method signatures in the target actions, using PSR-7.

The route identifier is used in `BackendUtility::getAjaxUrl()` as `$ajaxIdentifier` and as key in the global `TYPO3.settings.ajaxUrls` JavaScript object.

Feature: #70033 - Introduced TCA option showIconTable for selectSingle fields

See Issue #70033

Description

A new option `showIconTable` has been introduced for select fields with render type `selectSingle` to enforce or prevent the icon table underneath the field. By default the icon table is not shown.

Example

```
// Enforce icon table showing flags
$GLOBALS['TCA']['tt_content']['columns']['sys_language_uid']['config']['
↪showIconTable'] = true;
```

Feature: #70126 - Introduce TCA option to add autocomplete attribute to input fields

See Issue #70126

Description

It is now possible to enforce or disable the auto completion for input fields in edit mode. The option is called `autocomplete` and can be set to TRUE or FALSE in the config section of a field.

Example

```
// Prevent auto completion of username field for be_users records
$GLOBALS['TCA']['be_users']['columns']['username']['config']['autocomplete'] =␣
↪FALSE;
```

Feature: #70170 - ViewHelper to strip whitespace between HTML tags

See Issue #70170

Description

Removes redundant spaces between HTML tags while preserving the whitespace that may be inside HTML tags. Trims the final result before output.

Heavily inspired by Twig's corresponding node type.

```
<code title="Usage of f:spaceless">
<f:spaceless>
<div>
    <div>
        <div>text

text</div>
</div>
</div>
</f:spaceless>
</code>
<output>
<div><div><div>text

text</div></div></div>
</output>
```

Feature: #70332 - EXT:form - Add HTML4 / HTML5 attributes to the wizard

See Issue #70332

Description

The form wizard should support a wide set of attributes. At the moment the wizard is able to set some attributes, but this attribute-set is not complete and does not include HTML5 attributes.

The patch extends the wizard to set all universal HTML and HTML5 attributes based on selfhtml documentation version 8.1.2.

Currently supported attributes

`accept,acceptcharset,accesskey,action,alt,checked,class,cols,dir,disabled,enctype,id,lab`

New attributes

`autocomplete,autofocus,contenteditable,contextmenu,draggable,dropzone,height,hidden,inpu`

The **type attribute** will be extended with the following HTML5 types:

`color,date,datetime,datetime-local,email,month,number,range,search,tel,time,url,week`

Each element is now able to set the HTML universal attributes and element specific attributes.

Universal attributes are:

`accesskey,class,contenteditable,contextmenu,dir,draggable,dropzone,hidden,id,lang,spellc`

FORM

Currently supported attributes

`accept,accept-charset,action,class,dir,enctype,id,lang,method,style,title`

New attributes

Universal attributes + `accept,accept-charset,action,autocomplete,enctype,method,novalidate`

BUTTON

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,style,tabindex,title,type,value`

New attributes

Universal attributes + `autofocus,disabled,name,type,value`

SELECT

Currently supported attributes

`class,disabled,id,lang,multiple,name,size,style,tabindex,title`

New attributes

Universal attributes + `autofocus,disabled,multiple,name,required,size`

TEXTAREA

Currently supported attributes

`accesskey,class,cols,dir,disabled,id,lang,name,placeholder,readonly,rows,style,tabindex,`

New attributes

Universal attributes + `autofocus,cols,disabled,inputmode,maxlength,minlength,name,placeholder,rea`

SUBMIT

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,style,tabindex,title,type,value`

New attributes

Universal attributes + `autofocus,disabled,name,type,value`

RADIO

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,style,tabindex,title,type,value`

New attributes

Universal attributes + `autofocus,checked,disabled,name,readonly,required,type,value`

PASSWORD

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,maxlength,name,placeholder,readonly,size,style,`

New attributes

Universal attributes + `autocomplete,autofocus,disabled,maxlength,minlength,name,pattern,placehol`

HIDDEN

Currently supported attributes

`class,id,lang,name,style,type,value`

New attributes

Universal attributes + `name,type,value`

FILEUPLOAD

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,size,style,tabindex,title,type`

New attributes

Universal attributes + `accept,autofocus,disabled,multiple name,readonly,required,type,value`

RESET

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,style,tabindex,title,type,value`

New attributes

Universal attributes + `autofocus,disabled,name,type,value`

TEXTLINE

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,maxlength,name,placeholder,readonly,size,style,`

New attributes

Universal attributes + `autocomplete,autofocus,disabled,inputmode,list,maxlength,minlength,name,pa`
CHECKBOX

Currently supported attributes

`accesskey,alt,class,dir,disabled,id,lang,name,style,tabindex,title,type,value`

New attributes

Universal attributes + `autofocus`, `checked`, `disabled`, `name`, `readonly`, `required`, `type`, `value`

Feature: #70531 - RequireJS module for split buttons

See Issue #70531

Description

A RequireJS module for split buttons has been added. The module can be used in another RequireJS modules to add callbacks being executed before the submit takes place. As the callback receives the click event, the submit can be modified, e.g aborting the submit.

Impact

To use the `SplitButtons` module, include it in your own RequireJS module:

```
define('Vendor/Ext/Module', ['TYPO3/CMS/Backend/SplitButtons'],␣
↪function(SplitButtons) {
        // Your code...
});
```

Callbacks will be added by calling `SplitButtons.addPreSubmitCallback`:

```
SplitButtons.addPreSubmitCallback(function(e) {
        // Code being executed as callback before submit
});
```

Feature: #70583 - Introduced Icon API in JavaScript

See Issue #70583

Description

A JavaScript-based icon API based on the PHP API has been introduced. The methods `getIcon()` and `getIcons()` can be called in a RequireJS module.

When imported in a RequireJS module, a developer can fetch icons via JavaScript with the same parameters as in PHP. The methods `getIcon()` and `getIcons()` return `Promise` objects.

Importing

```
define(['jquery', 'TYPO3/CMS/Backend/Icons'], function($, Icons) {
});
```

Get icons

A single icon can be fetched by `getIcon()` which takes four parameters:

identifier

The icon identifier.

size

The size of the icon. Please use the properties of the `Icons.sizes` object.

overlayIdentifier

An overlay identifier rendered on the icon.

state

The state of the icon. Please use the properties of the `Icons.states` object.

To use the fetched icons, chain the `done()` method to the promise.

Examples

```
// Get a single icon
Icons.getIcon('spinner-circle-light', Icons.sizes.small).done(function(spinner) {
        $toolbarItemIcon.replaceWith(spinner);
});
```

Feature: #71196 - Disallow localization mixtures

See Issue #71196

Description

The PageLayout UI will now inform users if a mixture of translated content and standalone content is used in the page module since this is a major source of confusion for both administrators and editors.

In case an integrator knows what he/she is doing, we introduce a `PageTSConfig` setting to turn these warnings off to allow further usage of inconsistent translation handling.

`mod.web_layout.allowInconsistentLanguageHandling = 1`

Impact

Upon setting `mod.web_layout.allowInconsistentLanguageHandling` to `1` the page module will behave as before and allow inconsistent mixups of languages in a certain language.

Feature: #71251 - Add FlashMessage support in ModuleTemplate API

See Issue #71251

Description

Flash messages have different queues, depending on in which context they are enqueued. The FlashMessageService defaults the queue to `core.template.flashMessages`, Extbase defaults the queue to `extbase.flashmessages .$randomPluginQueue`.

Support for flash messages in ModuleTemplate has been added to enqueue flash messages automatically in the correct queue identifier.

Impact

Flash messages can be enqueued by the following code:

```
$this->moduleTemplate->addFlashMessage('I am a message body', 'Title',␣
↪\TYPO3\CMS\Core\Messaging\AbstractMessage::OK, true);
```

Feature: #72505 - Introduce hook to override a record overlay

See Issue #72505

Description

Prior to TYPO3 7 LTS, it was possible to override a record overlay in Web > List. This patch introduces a new hook with the old functionality.

The hook is called with the following signature:

```
/**
 * @param string $table
 * @param array $row
 * @param array $status
 * @param string $iconName
 * @return string the new (or given) $iconName
 */
function postOverlayPriorityLookup($table, array $row, array $status, $iconName)
```

Register the hook

Register the hook class which implements the method with the name `postOverlayPriorityLookup`:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][IconFactory::class]['overrideIconOverlay
↪'][] = \VENDOR\MyExt\Hooks\IconFactoryHook::class;
```

3.2.3 Deprecation

Deprecation: #51482 - Script-based modules

See Issue #51482

Description

Pseudo-modules that are registered via `ExtensionManagementUtility::addModulePath()` and modules that are registered via `ExtensionManagementUtility::addModule()` using the fourth parameter as a custom script-path have been marked as deprecated.

The method `ExtensionManagementUtility::addModulePath()` itself has been marked as deprecated.

Impact

All existing modules which are not registered via Routing will trigger a deprecation entry on registration of the module and when calling the module directly.

Affected Installations

All third-party extensions registering a wizard, module or route without using routeTarget or Routes.php, which have been introduced with TYPO3 CMS 7.

Migration

Use Configuration/Backend/Routes.php to register wizards and use `ExtensionManagementUtility::addModule()` when registering a routePath option in the fifth parameter to use the proper PSR-7 compatible way of registering and calling modules. Make sure to use UriBuilder and `BackendUtility::getModuleUrl()` to link to these modules instead of hard-linking to the script names.

Deprecation: #60712 - DocumentTemplate->getDynamicTabMenu()

See Issue #60712

Description

Methods `TYPO3\CMS\Backend\Template\DocumentTemplate::getDynamicTabMenu()` and `TYPO3\CMS\Backend\Template\DocumentTemplate::getDynTabMenuId()` have been marked as deprecated.

Affected Installations

Instances with custom backend modules that use these methods.

Migration

Use `TYPO3\CMS\Backend\Utility\ModuleTemplate::getDynamicTabMenu()` instead.

Deprecation: #65728 - DocumentTemplate->issueCommand()

See Issue #65728

Description

Method `TYPO3\CMS\Backend\Template\DocumentTemplate::issueCommand()` has been marked as deprecated.

Affected Installations

Instances with custom backend modules that use this method.

Migration

Use `TYPO3\CMS\Backend\Utility\BackendUtility::getLinkToDataHandlerAction()` instead.

Deprecation: #69369 – Use property text instead of data

See Issue #69369

Description

The FORM elements `TEXTAREA` and `OPTION` currently use "data" as property name to define default "values" which are used as human readable "labels" inside the specific tag. Furthermore, the `TEXTBLOCK` element uses the "content" property to define custom text. All other `FORM` elements use "value". Since "data" implies the possibility to use computed values, this patch deprecates "data" and adds a new property called "text".

Impact

Using the property "data" will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation using `TEXTAREA`, `OPTION` and `TEXTBLOCK` elements which use the property "data".

Migration

Remove usage of the "data" property and use "text" instead. Opening a specific form with the form wizard and storing the form again will also migrate from "data" to "text".

Deprecation: #69822 – Deprecate TCA settings of select fields

See Issue #69822

Description

Using the TCA field type `select` without specifying a valid `renderType` has been marked as deprecated.

Additionally the usage of `renderMode` for select fields has been marked as deprecated.

These `renderType` settings are available:

Key

renderType

Datatype

string

Description

This setting specifies how the select field should be displayed. Available options are:

- `selectSingle` - Normal select field for selecting a single value.
- `selectSingleBox` - Normal select field for selecting multiple values.
- `selectCheckBox` - List of checkboxes for selecting muliple values.
- `selectMultipleSideBySide` - Two select fields, items can be selected from the right field, selected items are displayed in the left select.
- `selectTree` - A tree for selecting hierarchical data.

Scope

Display

'renderType' => 'selectMultipleSideBySide', 'renderType' => 'selectCheckBox', 'renderType' => 'selectSingle', 'renderType' => 'selectSingleBox', 'renderType' => 'selectTree',

Note: If a field has no `renderType` set but `maxitems` is set, the migration will set `renderType` to `selectSingle` in case of `maxitems` is <= 1 otherwise `renderType` is set to `selectMultipleSideBySide`

Impact

The old TCA settings can still be used. A migration handles the update of the settings.

Affected Installations

All installations with extensions that configure TCA select fields in the old format.

Migration

Extension authors need to add the correct `renderType` setting to their select field definitions.

Deprecation: #69877 - Use ModuleTemplate API for ext:filelist

See Issue #69877

Description

Method `getButtonsAndOtherMarkers` of class `\TYPO3\CMS\Filelist\FileList` has been marked as deprecated.

Impact

The method should not be used any longer and will be removed with TYPO3 CMS 8.

Affected Installations

All third party extensions using the mentioned method.

Migration

Use the ModuleTemplate API instead.

Deprecation: #70052 - TCA Display condition EXT LOADED

See Issue #70052

Description

In TCA the EXT:anExtension:LOADED display condition has been marked as deprecated.

Affected Installations

Extensions that use LOADED display conditions. Those can be located by searching for LOADED in the backend module Configuration TCA section, example match from rtehtmlarea:

```
'static_lang_isocode' => array(
    'displayCond' => 'EXT:static_info_tables:LOADED:true',
    'config' => ...
    ...
),
```

Migration

Do not use any longer. TCA works additive, so the extension that is referenced in EXT:LOADED:extensionName should instead add columns definition instead of the referring extension defining the TCA conditional. In the example above, the column definition of static_lang_isocode was removed from extension rtehtmlarea and moved to extension static_info_tables, adding the field in an Configuration/TCA/Overrides file to the affected table. To ensure the load order of extensions is correct, static_info_tables could set a suggest dependency rtehtmlarea.

Deprecation: #70138 - Flex form language handling

See Issue #70138

Description

TCA flex fields had an own language handling that allowed to store field localization within the flex data itself and not within the usual database driven overlay records. This was mainly introduced for TemplaVoila and usually only used in this context.

The whole flex form specific language handling has been moved from core extensions to extension compatibility6 and will not be delivered with TYPO3 CMS 7 LTS anymore.

The following flex XML data structure keys have been marked as deprecated and are supported by compatiblity6 only:

- <meta><langDisable>
- <meta><langChildren>
- <meta><currentLangId>

The following PageTSConfig options have been dropped and are ignored if compatibility6 is not loaded:

- TCEFORM.[tableName].[field].[dataStructureKey].langDisable
- TCEFORM.[tableName].[field].[dataStructureKey].langChildren

The following UserTSConfig options have been dropped and are ignored if compatibility6 is not loaded:

- options.checkPageLanguageOverlay

The following LocalConfiguration value has been marked as deprecated, will be removed by the install tool and is set by compatibility6 to TRUE:

- $GLOBALS['TYPO3_CONF_VARS']['BE']['flexFormXMLincludeDiffBase']

The following method has been marked as deprecated and is cloned by `compatibility6` in an `XCLASS`:

- `TYPO3\CMS\Core\Configaration\FlexForm\FlexFormTools->getAvailableLanguages()`

The following property has been marked as deprecated:

- `TYPO3\CMS\Core\DataHandling\DataHandler->clear_flexFormData_vDEFbase`

Impact

If this feature is needed, the `compatibility6` extension must be loaded to keep compatibility with older versions after upgrading from `TYPO3 CMS 6.2`. If the extension is not loaded, flex form fields can no longer be localized on flex data level.

Affected Installations

All multi language installations that use flex form with meta field `langDisable` not set to 1 in their data structure definition and that make active use of the flex localization feature. This is the case if records with flex form fields show flex forms multiple times with different language flags.

Migration

Load extension `compatibility6` for a compatibility layer in `TYPO3 CMS 7`, or migrate affected features to use the record based localization feature. The flex field based language handling will most likely vanish with `TYPO3 CMS 8` altogether. In case the feature is needed for a longer time the code from `compatibility6` could be used as a kick start for an implementation within an own extension.

In case `compatibility6` is loaded, some core content elements may start showing flex field language overlays since the `langDisable` meta definition has been removed from their data structure XML. This can be suppressed with this `PageTSConfig` snippet:

```
TCEFORM.tt_content.pi_flexform.table.langDisable = 1
TCEFORM.tt_content.pi_flexform.login.langDisable = 1
TCEFORM.tt_content.pi_flexform.media.langDisable = 1
```

Deprecation: #70477 - Deprecate SpriteIcon classes

See Issue #70477

Description

The following classes have been marked as deprecated.

```
\TYPO3\CMS\Backend\Sprite\AbstractSpriteHandler
\TYPO3\CMS\Backend\Sprite\SimpleSpriteHandler
\TYPO3\CMS\Backend\Sprite\SpriteBuildingHandler
```

Impact

Any usage of these classes will trigger a deprecation log entry.

Affected Installations

Extensions that use these PHP classes.

Migration

Use the `IconRegistry` to register icons.

Deprecation: #70494 - DocumentTemplate->wrapClickMenuOnIcon()

See Issue #70494

Description

Method `TYPO3\CMS\Backend\Template\DocumentTemplate::wrapClickMenuOnIcon()` has been marked as deprecated.

Affected Installations

Instances with custom backend modules that use this method.

Migration

Use `TYPO3\CMS\Backend\Utility\BackendUtility::wrapClickMenuOnIcon()` instead.

Deprecation: #70514 - dynamicConfigFile is deprecated

See Issue #70514

Description

The `TCA` configuration `dynamicConfigFile` within the `ctrl` section of a table has been marked as deprecated and must not be used any longer.

Impact

Using `dynamicConfigFile` within the `ctrl` section of a table will trigger a deprecation log entry.

Migration

The setting is typically used in `ext_tables.php` files of extensions. The table configuration (`TCA`) must be moved to an own file in `Configuration/TCA/<table_name>.php`. The `dynamicConfigFile` setting isn't needed anymore since the whole `TCA` array definition is in this file.

Furthermore, any other `TCA` manipulation of third party tables must be moved to `Configuration/TCA/Overrides` and no `TCA` setting must remain in `ext_tables.php`. This is highly encouraged since TYPO3 CMS 6.2 already for performance reasons. If this change is not applied to extensions, extension `compatibility6` must be loaded or further migration may not be applied to this portion of `TCA` leading to all sorts of possible issues.

Deprecation: #71153 - DocumentTemplate->spacer()

See Issue #71153

Description

Method `TYPO3\CMS\Backend\Template\DocumentTemplate::spacer()` has been marked as deprecated.

Affected Installations

Instances with custom backend modules that use this method.

Migration

Add the needed margin as HTML / CSS.

Deprecation: #71249 - Deprecate render method of FlashMessage class

See Issue #71249

Description

Method `TYPO3\CMS\Core\Messaging\FlashMessage::render()` has been marked as deprecated.

Affected Installations

Instances with custom backend modules that use this method.

Migration

Use custom render code, the `<f:flashMessage />` ViewHelper or the `ModuleTemplate` for backend modules to render FlashMessages. It is suggested not to include HTML in flash messages. Flash messages should be short notifications on user interactions. If you need more elaborate or persistent messages, use `<f:be.infobox />` view helper or HTML similar to that.

3.2.4 Important

Important: #36166 - Move access right parameters from BE to SYS configuration

See Issue #36166

Description

The access permission parameters (fileCreateMask, folderCreateMask, createGroup) have been moved from `$GLOBALS['TYPO3_CONF_VARS']['BE']` to `$GLOBALS['TYPO3_CONF_VARS']['SYS']`. An Upgrade Wizard ensures the correct migrations of the settings.

Important: #53681 - Change wording for User Settings "Reset Configuration and Clear Temporary Data"

See Issue #53681

Description

To simplify the user settings form, the button "Clear Temporary Data" has been removed. The button to reset the configuration and clear the temporary data has been renamed to "Reset user settings to default state".

Important: #68079 - Extension "mediace" moved to TER

See Issue #68079

Description

The previously available "mediace" extension has been moved to the TYPO3 Extension Repository (TER) and will be managed on GitHub (https://github.com/FriendsOfTYPO3/mediace).

An upgrade wizard in the Install Tool will check if the extension is needed. If so, it is downloaded from the TER and installed if necessary.

Important: #70956 - Behavior of Page TCconfig options keepItems, addItems and removeItems changed

See Issue #70956

Description

The behavior of Page TSconfig options `keepItems`, `addItems` and `removeItems` has been restored to state of TYPO3 CMS 6.2-7.4 and the execution order of these options has been formalized.

The first option to be evaluated is `keepItems`, followed in turn by `addItems` and `removeItems`. All three options are evaluated after items have been added to the configuration by sources like folders or foreign tables.

Important: #71126 - Allow to define multiple inlineLocalizeSynchronize commands

See Issue #71126

Description

The inlineLocalizeSynchronize command in DataHandler currently supports these formats:

- [parent][13][inlineLocalizeSynchronize] = field,14
- [parent][13][inlineLocalizeSynchronize] = field,localize
- [parent][13][inlineLocalizeSynchronize] = field,synchronize

The current string configuration format is changed to be an array, legacy configurations are converted to the new format:

```
[parent][13][inlineLocalizeSynchronize] = [
  field: name of the parent field,
  language: id of the target language,
  action: either "localize" or "synchronize",
  ids: array of child-ids to be localized [1, 2, 3]
]
```

Either "action" or "ids" must be defined.

Important: #72697 - Remove thumbnail functionality of impexp

See Issue #72697

Description

Previously it was possible to add additional thumbnails to exports of the EXT:impexp. This functionality is completely outdated and not needed anymore.

Important: #73565 - AbstractConditionViewHelper no longer automatically compilable

See Issue #73565

Description

CompilableInterface is removed from the abstract AbstractConditionViewHelper and is now implemented by each of the condition ViewHelpers. The base class still provides every method it did before but third-party ViewHelpers must now indicate that they are compilable by implementing the required interface.

The change is related to a previous change which made `evaluateCondition` the API to evaluate the input argument conditions in any condition ViewHelper. The change is done as a preventative measure, preventing issues when third-party condition ViewHelpers were compiled but did not implement the `evaluateCondition` method. Now, such ViewHelpers will prevent compiling entirely until the author of the class has implemented CompilableInterface and the `evaluateCondition` method.

Making such third-party ViewHelpers no longer compilable (and thus decreasing performance when they are used) is chosen in favor of preserving the current behavior where such ViewHelpers would be completely unable to correctly evaluate the condition at all.

3.3 7.5 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

3.3.1 Breaking Changes

Breaking: #24186 - HTMLparser - fixAttrib.['class'].list does not assign first element, when attribute value not in list

See Issue #24186

Description

The HTMLparser now assigns the first class of `fixAttrib.class.list` when none of the given class name values are found in the configured list. Until now the class attribute of the rendered HTML tag was just empty in that case.

Impact

A HTML element that had no class before could now have been assigned a class.

Migration

Add a class from the configured list to the HTML element or add a class at the first position of `fixAttrib.class.list`.

Breaking: #30863 - Streamlined parameters for adding inline language files

See Issue #30863

Description

The method `addInlineLanguageLabelFile` of the `PageRenderer` handles the optional parameter `$stripFromSelectionName`, a string that should be removed from any label key in the given file. This did not work until now, so the label keys were never stripped. As this functionality is now working it could end up with different label keys in the output.

Impact

Inline Javascript label keys could have changed.

Affected Installations

Any third party code using `PageRenderer->addInlineLanguageLabelFile()` with the parameter `$stripFromSelectionName` set to anything but an empty string.

Migration

Change the call to `PageRenderer->addInlineLanguageLabelFile()` with `$stripFromSelectionName = ''` or adjust your Javascript to handle the now correctly rendered label keys.

Breaking: #52156 - Replaced JumpURL features with hooks

See Issue #52156

Description

JumpURL handling

The generation and handling of JumpURLs has been removed from the frontend extension and has been moved to a new core extension called "jumpurl".

URL handler hooks

New hooks were introduced in `TYPO3\CMS\Frontend\ContentObject\ContentObjectRenderer` and `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController` that allow custom URL generation and handling.

This is how you can register a hook for manipulating URLs during link generation:

```
// Place this in your ext_localconf.php file
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['urlProcessing']['urlHandlers']['myext_
↪myidentifier']['handler'] =
        \Company\MyExt\MyUrlHandler::class;

// The class needs to implement the UrlHandlerInterface:
class MyUrlHandler implements \TYPO3\CMS\Frontend\Http\UrlHandlerInterface {}
```

This is how you can handle URLs in a custom way:

```
// Place this in your ext_localconf.php file
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['urlProcessing']['urlProcessors']['myext_
↪myidentifier']['processor']
        = \Company\MyExt\MyUrlProcessor::class;

// The class needs to implement the UrlProcessorInterface:
class MyUrlProcessor implements \TYPO3\CMS\Frontend\Http\UrlProcessorInterface {}
```

External URL page handling

The core functionality for redirecting the user to an external URL when he hits a page with doktype "external" is moved from the `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController` to the `\TYPO3\CMS\Frontend\Page\ExternalPageUrlHandler` class.

ResourceStorage adjustment

The method `\TYPO3\CMS\Core\Resource\ResourceStorage::dumpFileContents()` accepts an additional parameter for overriding the mime type that is sent in the `Content-Type` header when delivering a file.

Impact

Unless the jumpurl extension is installed, no JumpURL related feature will work anymore.

If an extension tightly integrates into the JumpURL process it might break, because some of the related methods have been removed, disabled or changed.

These methods have been removed and their functionality has been moved to the new jumpurl extension:

`\TYPO3\CMS\Frontend\ContentObject\ContentObjectRenderer::locDataJU()`

`\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::locDataCheck()`

The `$initP` parameter of the method `\TYPO3\CMS\Frontend\ContentObject\ContentObjectRenderer::getMa` has been removed.

The method `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::setExternalJumpUrl` has been marked as deprecated and is an alias for the new `initializeRedirectUrlHandlers()` method that does no jumpurl handling any more. The new method only checks if the current page is a link to an external URL and sets the `redirectUrl` property.

The method `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::jumpUrl()` has also been marked as deprecated and is an alias for the new `redirectToExternalUrl()` method. The jumpurl handling has been removed from this method. It loops over all registered URL handlers and handles the redirection to the `redirectUrl`.

Affected installations

All CMS 7.4 installations that use the JumpURL features or that use Extensions that rely on these features or one of the removed methods.

Migration

If you want to use the JumpURL features you need to install the jumpurl extension. Your configuration should work as before.

Please note that the configuration of the filelink TypoScript function has changed. Passing the `jumpurl` parameter in the configuration has been marked as deprecated and will be removed in future versions.

You can now pass arbitrary configuration options for the typolink call that is used to generate the file link in the `typolinkConfiguration` parameter:

```
lib.myfilelink = TEXT
lib.myfilelink.value = fileadmin/myfile.txt
lib.myfilelink.filelink {
        typolinkConfiguration.jumpurl = 1
        typolinkConfiguration.jumpurl.secure = 1
}
```

Breaking: #63000 - Migrate EXT:cshmanual to Extbase

See Issue #63000

Description

The extension "cshmanual" has been migrated to a newer code base by using Extbase and Fluid.

Impact

Any call to the previous public methods of the old controller HelpModuleController will fail as the code base changed.

Affected installations

Any installation using an extension which calls the previously available methods directly.

Migration

Use the Extbase controller or Repository class.

Breaking: #65317 - TypoScriptParser sortList sanitizes input on numerical sort

See Issue #65317

Description

When calling the `:= sortList()` with a "numeric" modifier of the TypoScript parser with a string, the `sort()` method differs between PHP versions. In order to make this behavior more strict, a check is done before the elements are sorted to only have numeric values in the list, otherwise an Exception is thrown.

Impact

An exception is thrown if non-numerical values are given for a numeric sort in TypoScripts `sortList`.

Affected Installations

All installations using `sortList` numeric with non-numerical values.

Migration

Either remove the non-numerical values from the list or change the sort order to be non-numerical (ascending / descending).

Breaking: #66190 - Remove flash and chart from ExtJS

See Issue #66190

Description

The flash and chart module is removed from ExtJS. In order to reduce ExtJS components this is a first step.

Impact

Extensions which use the flash and chart module from ExtJS will not work anymore.

Affected Installations

Installations that use flash or chart module of ExtJS.

Migration

Don't use cores ExtJS anymore, as we migrate away from it. Use other JS frameworks which implement such functionality for you.

Breaking: #67098 - Correct required-parameter in TextfieldViewHelper

See Issue #67098

Description

The value comparison of the required parameter has been corrected. Prior to this change, a textfield was required as soon as it had a parameter "required" set to any value even if this value was set to FALSE, the textfield was still required.

Impact

Textfields with required="FALSE" are not required any longer.

Affected Installations

Every installation that uses the textfield viewhelper with the required attribute.

Migration

No migration is necessary.

Breaking: #68354 - Uniform extension directory structure of EXT:indexed_search

See Issue #68354

Description

The directory structure of the extension "Indexed Search" has been streamlined.

Impact

All language files are now located in directory Resources/Private/Language, the template files in Resources/Private/Templates. Icons from pi/res directory have been moved to Resources/Public/Icons, images to Resources/Public/Images.

Affected Installations

Installations that use EXT:indexed_search that depend on paths that have been moved.

Migration

Make sure your configuration matches with new directory structure.

Breaking: #68401 - SqlParser moved into EXT:dbal

See Issue #68401

Description

The SQL Parser included with the core has not been in use by anything except EXT:dbal for some time. The SQL parser has been merged with the version in EXT:dbal which now provides parsing and compiling of SQL statements for MySQL as well as other DBMS.

Impact

There is no impact for the core as EXT:dbal was the sole user of the SQL parser and it has been migrated into EXT:dbal.

As the parsing and the compiling of SQL statements has been separated into multiple classes the non-public interface of `SqlParser` has changed. Classes extending SqlParser need to be adjusted to the new interface.

Affected Installations

Installations with 3rd party extensions that use `\TYPO3\CMS\Core\Database\SqlParser`.

Migration

Update the code to use `\TYPO3\CMS\Dbal\Database\SqlParser` instead of `\TYPO3\CMS\Core\Database\SqlParser` or install EXT:compatibility6 which maps the old class names to the new ones in EXT:dbal.

Breaking: #68562 - Bool values need to be cast to integer for MySQL strict mode

See Issue #68562

Description

MySQL strict mode doesn't accept '' as a valid value to store in an integer column if the MySQL server is running in strict mode.

mysqli_real_escape() casts boolean values to string using '1' (for `TRUE`) and '' (for `FALSE`). Due to this special handling is required for boolean values to result in '0' and '1' for FALSE/TRUE.

Impact

All TYPO3 CMS installations using MySQL as DBMS.

Affected Installations

Installations where 3rd party extension are relying on `FALSE` being cast to '' when they are storing boolean values in character type columns. In this case new values will get stored as '0'

Migration

Adjust the code to either store boolean values in integer type columns or manually cast the boolean value to string before storing it in the database.

Breaking: #68571 - Removed method ElementBrowser->getMsgBox

See Issue #68571

Description

The removed method `getMsgBox` in `ElementBrowser` used table based styling. The method has been removed since we have better means to display this kind of messages: FlashMessages or Callouts.

Impact

A fatal error will be thrown if the method `getMsgBox` is used.

Affected Installations

Third party code using the removed method.

Migration

Remove the call to the method and replace the message with a FlashMessage.

Breaking: #68812 - Old Backend Entrypoints moved to deprecation layer

See Issue #68812

Description

The backend entry points within the typo3/ directory which have been marked as deprecated in favor of using typo3/index.php directly as Entry Point via Request Handling, have been moved to a deprecation.php file.

The following files have therefore been removed from the typo3/ directory directly:

```
typo3/ajax.php
typo3/alt_clickmenu.php
typo3/alt_db_navframe.php
typo3/alt_doc.php
typo3/alt_file_navframe.php
typo3/browser.php
typo3/db_new.php
typo3/dummy.php
typo3/init.php
typo3/login_frameset.php
typo3/logout.php
typo3/mod.php
typo3/move_el.php
typo3/show_item.php
typo3/tce_db.php
typo3/tce_file.php
typo3/thumbs.php
```

The typo3/install/ entrypoint is now also redirected with a rewrite rule.

Impact

All references / links to these entry points directly without using the proper API calls will result in a 404 error.

If an Apache webserver is used with the enabled mod_rewrite module, a .htaccess file placed inside typo3/ will rewrite the URLs to the deprecated.php and throw a deprecation warning.

For Nginx and IIS an alternative for the rewrite rules in the shipped typo3/.htaccess within needs to be added.

Affected Installations

Installations with third-party extensions that link directly to these files.

Migration

Move all existing code in extensions that link to the deprecated entry points to use methods like `BackendUtility::getModuleUrl()` and `BackendUtility::getAjaxUrl()` or the UriBuilder class.

Breaking: #68814 - Remove of base constant TYPO3_URL_ORG

See Issue #68814

Description

Base constant TYPO3_URL_ORG defined in SystemEnvironmentBuilder::defineBaseConstants() has been removed. It was for internal usage only and defined at 2 places in the core.

Impact

Constant TYPO3_URL_ORG no longer exists.

Migration

Use TYPO3_URL_GENERAL instead.

Breaking: #69028 - TCA type select - Drop neg_foreign_table

See Issue #69028

Description

The following `TCA` keys for `type select` have been dropped and are no longer handled by the core:

- neg_foreign_table
- neg_foreign_table_where
- neg_foreign_table_prefix
- neg_foreign_table_loadIcons
- neg_foreign_table_imposeValueField

These setting were used in `select` for comma separated value relations in addition to `foreign_table` to allow a second connected table. Relations for `neg_foreign_table` were stored as negative uids in the field to distinguish them from relations to the table defined in `foreign_table`.

The functionality has been dropped without substitution and is no longer handled by the TYPO3 core.

Impact

Existing relations to the table defined in `neg_foreign_table` will be discarded when a record with such a `TCA` configuration is saved to the database. The display of existing connected records may be misleading.

Affected Installations

This old school feature was never documented well and used by a very small amount of extensions. Searching an instance for the keyword `neg_foreign_table` will reveal usages.

Migration

In case records from multiple different tables must still be supported, the `TCA` configuration should be adapted to use a `MM` intermediate table. For existing migrations a database migration is required.

Breaking: #69057 - Deprecate IconUtility and move methods into IconFactory

See Issue #69057

Description

While refactoring the `IconUtility` to the new `IconFactory` class, several methods have been marked as deprecated. In some cases parameters of the old `IconUtility` methods are not used anymore. The following list describes the possible breaking changes.

The second parameter `$options` of method `IconUtility::getSpriteIconForFile()` is not used anymore. The third parameter `$options` of method `IconUtility::getSpriteIconForRecord()` is not used anymore.

The `IconUtility` signals `buildSpriteIconClasses` and `buildSpriteHtmlIconTag` have been dropped and will not be emitted anymore. The `IconUtility` hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_iconworks.php']['overrideIc` has been dropped and will not be called anymore.

Impact

Extensions could break if the methods, signals or hooks above are used.

Affected Installations

Extensions that call the methods with the `$options` parameter or make use of the signals and hook.

Migration

Make use of the new `IconFactory` class.

Breaking: #69083 - Renamed identifier for filelist module

See Issue #69083

Description

The filelist module was rewritten to use Extbase. Therefore the module identifier has been changed from `file_list` to `file_FilelistList`.

Impact

All links pointing to the filelist module using the old identifier will break.

Affected Installations

All installations that reference the filelist module by its old name.

Migration

There is a upgrade wizard to change the backend user settings of users whose start module is the filelist module. All other links to the module have to be changed manually to use `file_FilelistList` as module identifier.

Breaking: #69148 - Backend Module Dispatching removed

See Issue #69148

Description

Dispatching Backend modules through custom dispatchers have been removed. The corresponding Extbase functionality called "ModuleRunner" and its Interface have been removed as well.

Impact

Any dispatcher registered via `$TBE_MODULES['_dispatcher']` is not evaluated anymore.

Affected Installations

All TYPO3 Instances with an extension that registers a custom backend module dispatcher.

Migration

Use a custom RequestHandler.

Breaking: #69161 - Removed includeCsh setting from ContainerViewHelper

See Issue #69161

Description

Include CSH setting in `<f:be.container>` is not needed anymore and has therefore been removed. The JavaScript will be loaded automatically when ext:cshmanual is enabled.

Impact

Using `<f:be.container>` ViewHelpers in a custom Backend module, setting the includeCsh property, will result in a fatal error.

Affected Installations

Extensions that use `<f:be.container>` which set the setting `includeCsh`

Migration

Remove the property from the template. When ext:cshmanual is enabled the JavaScript is loaded automatically.

Breaking: #69168 - Removed non-tabbed view of Content Element Wizard

See Issue #69168

Description

The "New Content Element Wizard" view to show possible content elements to create now only shows the elements in a tabbed view. The non-tabbed view variant has been removed without substitution.

The TSconfig option `mod.wizards.newContentElement.renderMode` has been removed.

Migration

Remove the TSconfig option `mod.wizards.newContentElement.renderMode` from any configuration settings.

Breaking: #69224 - Fix wrong usage of enumerations in InformationStatus::mapStatusToInt()

See Issue #69224

Description

The `InformationStatus` enumeration provides a `mapStatusToInt()` method. * The method expects a string but should expect an enum of itself. * The method logic is not what is expected from an enumeration method as it does not do any logic comparison. Therefore it has been replaced by `isGreaterThan()` as this was the logic that has been checked everywhere `mapStatusToInt()` has been used.

Impact

The method `InformationStatus::mapStatusToInt()` has been replaced by `InformationStatus::isGreaterThan()` and all usages have been replaced by the new method / logic. As the `InformationStatus` Enum has been introduced in 7.4 it should not be used by any public API and therefore the change should not have much impact.

Breaking: #69276 - ElementBrowserController::$browser removed

See Issue #69276

Description

The `$browser` member variable of `\TYPO3\CMS\Recordlist\Controller\ElementBrowserController` has been removed.

Impact

Any third party code accessing `$GLOBAL['SOBE']->browser` will break.

Affected Installations

Installations using third party code, which accesses `$GLOBAL['SOBE']->browser`.

Migration

If the code is extending one of the ElementBrowser tree classes, the protected member variable `$elementBrowser` can be used to access the underlying ElementBrowser instance.

If your code is using the ElementBrowser tree classes, an instance of `ElementBrowser` has to be injected using the setter.

Breaking: #69291 - Changed registration of backend module icons

See Issue #69291

Description

The sprite icon support for backend modules introduced with CMS 7.3 has been adjusted again. The configuration has been streamlined.

Impact

The specified icon will not be recognized.

Affected Installations

Any installation running TYPO3 CMS 7.3+ having third party extensions which use sprite icons for backend modules.

Migration

Change the configuration from

```
'configuration' => array(
        'icon' => 'module-web',
),
```

to

```
'iconIdentifier' => 'module-web',
```

Breaking: #69315 - ElementBrowser::main_* protected

See Issue #69315

Description

The `ElementBrowser::main_*` methods have been marked protected as the new `render` method is the main entry point to the class. Additionally the public member `ElementBrowserController::mode` has been protected as well.

The `ElementBrowserController::content` member and the `ElementBrowserController::printContent()` method have been removed.

Impact

Any code calling the protected or removed methods or using the protected member will fail with a fatal error. Any code using the removed member will receive only an empty value. (PHP fallback for non-existing class members)

Affected Installations

Any installation using third party code calling the mentioned methods or member.

Migration

Ensure the intended mode is passed in via the `mode` GET-parameter and call the new `ElementBrowser::render` method.

Breaking: #69401 - Adopt form to support the Extbase/ Fluid MVC stack

See Issue #69401

Description

The `postProcessor` interface and the mail postProcessor have changed.

Validators and filters have been moved to other folders and both class names and algorithms have changed.

Impact

Own postProcessors, validators and filters will possibly fail with an error.

Affected Installations

Installations with own postProcessors, validators and filters.

Migration

Adopt own postProcessors, validators and filters to comply with the current implementation.

Breaking: #69561 - Replace sprite icons with IconFactory in ContextMenu

See Issue #69561

Description

SpriteIcon and standalone image support have been replaced with `IconFactory` in the context menu. All menu icons now need to be registered through the `IconRegistry`.

Impact

The `UserTsConfig` options for items `icon` and `spriteIcon` have no effect anymore, and will deliver a blank placeholder image if `iconName` is not set.

Affected Installations

All installations that add or modify items in the ContextMenu.

Migration

Register the icon through the `IconRegistry` and set the `iconName` in the item configuration.

```
// Register Icon
$iconRegistry = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Imaging\IconRegistry::class);
$iconRegistry->registerIcon(
        'contextmenu-example',
        \TYPO3\CMS\Core\Imaging\IconProvider\SvgIconProvider::class,
        array(
                'source' => 'EXT:example/Resources/Public/Icons/contextmenu-example.
↪svg'
        ))
);
```

```
options.contextMenu.table {
        virtual_root.items {
                9999 = ITEM
                9999 {
                        name = contextmenuExample
                        label = LLL:EXT:example/Resources/Private/Language/
↪locallang.xlf:contextmenu-example
                        iconName = contextmenu-example
                        callbackAction = exampleCallback
                }
        }
}
```

Breaking: #69568 - FormEngine related classes

See Issue #69568

Description

The following classes have been removed:

- `\TYPO3\CMS\Backend\Form\DataPreprocessor`
- `\TYPO3\CMS\Backend\Form\FormEngine`
- `\TYPO3\CMS\Backend\Form\FlexFormsHelper`

The following hook has been removed:

- `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_tceforms.php']['getMai`

Impact

Code trying to instantiate these classes will fatal.

Affected Installations

A rather low number of extensions should be affected by this change. Searching for the above class names should reveal them.

Migration

The methods and classes have been moved to different classes and solutions. Extensions needs adaption.

The hook `getMainFieldsClass` has been substituted with a much more fine grained and flexible API. Use `FormDataProvider` to change data given to the render engine of FormEngine from now on.

Breaking: #69699 - TCA ctrl typeicons removed

See Issue #69699

Description

The `TCA['ctrl']['typeicons']` key has been removed.

Impact

If still used, a fallback default icon may be displayed instead.

Affected Installations

Searching for `typeicons` keyword should reveal extensions using this functionality.

Migration

Until further works on the icon API have been finished, `TCA['ctrl']['typeicon_classes']` should be used as documented in the TCA reference.

Breaking: #69795 - Unused DTM Tabmenu code removed

See Issue #69795

Description

All DynTabMenu JavaScript and CSS code which was previously used to render Tab Menus in the TYPO3 Backend has been removed without substitution.

Impact

All logic that requires EXT:backend/Resources/Public/JavaScript/tabmenu.js directly and/or use the JavaScript code of `DTM_activate()` or `DTM_toggle()` directly have been removed.

Affected Installations

TYPO3 Installations with custom extensions that use the logic mentioned above.

Migration

Use ModuleTemplate::getDynamicTabMenu() directly to use the Bootstrap-based API shipped with the TYPO3 Core.

Breaking: #69904 - Remove Setting diff_path from DefaultConfiguration

See Issue #69904

Description

Creating a diff view of text has been replaced with a PHP library instead of using `diff` on the command line. Thus we no longer need to be able to configure the path.

Impact

The setting `[BE][diff_path]` will no longer have any effect.

Affected Installations

Any Installation that had to define a path different than `diff`

Migration

Delete the line from LocalConfiguration.php if the UpgradeWizard should fail to do so.

Breaking: #69930 - Remove option "serverTimeZone"

See Issue #69930

Description

The option `$TYPO3_CONF_VARS['SYS']['serverTimeZone']` which was introduced when there was no clean way to fetch the timezone option in the PHP4 environment, has been removed. It was solved in PHP 5.1.0 which introduced `date_default_timezone_get()` which is used by the TYPO3 Core by default.

Impact

Accessing the option will result in a PHP notice, as it has been removed in TYPO3 CMS 7. Extensions making use of this option will result in an unexpected behaviour as possible calculations are wrong.

Affected Installations

Any TYPO3 installation which uses a third-party extensions that uses this option.

Migration

Use native timezone support by PHP directly. See `date_default_timezone_get()` for more information.

3.3.2 Features

Feature: #16525 - Add conditions to INCLUDE_TYPOSCRIPT

See Issue #16525

Description

The INCLUDE_TYPOSCRIPT tag now has an extra (optional) property "condition" which causes the file/directory to be included only if the condition is met.

As usual a condition is enclosed in square brackets, but if these are not present they will be added. Any double quotes must be escaped by adding backslashes and any backslash must be doubled.

Example

```
<INCLUDE_TYPOSCRIPT: source="FILE:EXT:my_extension/Configuration/TypoScript/firefox.
↪ts" condition="[loginUser = *]">
```

Condition with square brackets. File will only be included if a frontend user is logged in.

```
<INCLUDE_TYPOSCRIPT: source="FILE:EXT:my_extension/Configuration/TypoScript/staging.
↪ts" condition="applicationContext = /^Production\\/Staging\\/Server\\d+$/">
```

Condition without square brackets, backslashes doubled inside the condition. File will only be included in application context Production/Staging/Server followed by at least one digit.

Feature: #19494 - Add SELECTmmQuery method to DatabaseConnection

See Issue #19494

Description

A new method `SELECT_mm_query` has been added to the `DatabaseConnection` class. This method has been extracted from `exec_SELECT_mm_query` to separate the building and execution of M:M queries.

This enables the use of the query building in the database abstraction layer.

Example:

```
$query = SELECT_mm_query('*', 'table1', 'table1_table2_mm', 'table2', 'AND table1.
↪uid = 1', '', 'table1.title DESC');
```

Feature: #25341 - Scheduler task to optimize database tables

See Issue #25341

Description

A scheduler task to run the `OPTIMIZE TABLE` command on selected database tables has been added. The `OPTIMIZE TABLE` command reorganizes the physical storage of table data and associated index data to reduce storage space and improve I/O efficiency when accessing the table. The exact changes made to each table depend on the storage engine used by that table. For more information see the MySQL manual.

The scheduler task is meant for the MySQL database system and only shows tables matching the MySQL storage engines MyISAM, InnoDB and ARCHIVE. Using this task with DBAL and other DBMS is not supported as the commands used are MySQL specific.

Impact

Optimizing tables is I/O intensive. On MySQL < 5.6.17 it also locks the tables for the whole time, which can severely impact the website while it is running. When considering whether or not to run optimize, consider the workload of transactions that your server will process as InnoDB tables do not suffer from fragmentation in the same way that MyISAM tables do.

Feature: #28243 - Introduce TCA option to disable age display of dates per field

See Issue #28243

Description

It is now possible to disable the display of the age (p.e. "2015-08-30 (-27 days)") of date fields in record listings by a new TCA option. The option is called `disableAgeDisplay` and can be set in the config section of a field. It will be respected if the field has the type `input` and its eval is set to `date`.

Example

```
// disables the display of " (-27 days)" p.e.
$GLOBALS['TCA']['tt_content']['columns']['date']['config']['disableAgeDisplay'] =␣
↪true;
```

Feature: #31100 - Integrate multiline support for TEXTBLOCK in form wizard

See Issue #31100

Description

The form object `TEXTBLOCK` is now rendered as multiline input field in the form wizard. Line breaks are automatically converted to `<br>` tags in the wizard preview and the frontend.

Feature: #38732 - Fluid-based Content Elements Introduced

See Issue #38732

Description

A new system extension called "Fluid Styled Content" has been added to the core, which ships with a trimmed down and simplified set of Content Elements which are rendered by Fluid Templates. This extension is installed by default on new installations.

In order to have Fluid Styled Content running, add the TypoScript file inside the Template module. The Page TSconfig is loaded automatically when the extension is installed. The autoloading of this file can be disabled by deactivating the `loadContentElementWizardTsConfig` option in the extension configuration of the extension manager. You have then to load the Page TSConfig by yourself on the page properties.

It is possible to overwrite the templates by adding your own paths in the TypoScript setup:

```
lib.fluidContent.templateRootPaths.50 = EXT:site_example/Resources/Private/
↪Templates/
lib.fluidContent.partialRootPaths.50 = EXT:site_example/Resources/Private/Partials/
lib.fluidContent.layoutRootPaths.50 = EXT:site_example/Resources/Private/Layouts/
```

The new CType `textmedia` adds support for rendering media elements and image elements side by side.

Impact

Please note that this extension is still in an early stage and breaking changes are still possible until TYPO3 CMS 7 LTS, so be aware of changes to TCA, Templates, Behaviour and Feature set.

Some conflicts regarding CSS Styled Content and Fluid Styled Content might still exist.

Feature: #47812 - Query support for BETWEEN added

See Issue #47812

Description

Support for `between` has been added to the Extbase `Query` object. As there is no performance advantage to using BETWEEN on the DBMS side (the optimizer converts it to `(min <= expr AND expr <= max)` this function replicates the DBMS behaviour by building a logical AND condition that has the advantage of working on all DBMS.

Example:

```
$query->matching(
        $query->between('uid', 3, 5)
);
```

Feature - #52217: Signal for pre processing linkvalidator records

See Issue #52217

Description

This signal allows for additional processing upon initialization of a specific record, e.g. getting content data from plugin configuration in record.

Registering the signal: (in ext_localconf.php)

```
$signalSlotDispatcher = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Extbase\SignalSlot\Dispatcher::class);
$signalSlotDispatcher->connect(
    \TYPO3\CMS\Linkvalidator\LinkAnalyzer::class,
    'beforeAnalyzeRecord',
    \Vendor\Package\Slots\RecordAnalyzerSlot::class,
    'beforeAnalyzeRecord'
);
```

The slot class:

```
namespace Vendor\Package\Slots;

use TYPO3\CMS\Linkvalidator\LinkAnalyzer;

class RecordAnalyzerSlot {

    /**
     * Receives a signal before the record is analyzed
     *
     * @param array $results Array of broken links
     * @param array $record Record to analyse
     * @param string $table Table name of the record
     * @param array $fields Array of fields to analyze
     * @param LinkAnalyzer $parentObject Parent object
     * @return array
     */
    public function beforeAnalyzeRecord($results, $record, $table, $fields,␣
↪LinkAnalyzer $parentObject) {
        // Processing here
        return array(
            $results,
            $record
        );
    }
}
```

Impact

Extensions may now perform any kind of processing for every record when validating content links.

Feature: #53406 - Add placeholder attribute to some textfields in wizard

See Issue #53406

Description

The form wizard is now able to add a placeholder attribute to the following elements: TEXTLINE, TEXTAREA, PASSWORD and the preset "email". TypoScript config is properly read and written.

Feature: #56282 - Language selector for pageview module

See Issue #56282

Description

The pageview module now has a dropdown to select a language for the page preview.

In case you switch languages based on something different than a parameter called `L` you can disable the selector by using the following PageTSConfig:

```
mod.SHARED.view.disableLanguageSelector = 1
```

Feature: #56726 - Trigger metadata extraction after file upload

See Issue #56726

Description

Before #56726 the metadata extraction was only called through the extract metadata scheduler task. So when a editor uploaded a new file he had to wait until the scheduler task had been triggered again and extracted the metadata.

Now the metadata extraction is by default triggered after adding/uploading a file in the BE or when the FAL API is used `ResourceStorage::addFile()`, `ResourceStorage::replaceFile()` and `ResourceStorage::addUploadedFile()`.

In some special situations it isn't desired to have metadata extraction direct after file upload/adding a file to the storage. For these cases the automatic extraction can be disabled in File Storage configuration.

Impact

The flag is by default set for all existing and a new storage. When you have some special use-case where automatic extraction of metadata is not desired the flag can be disabled in File Storage configuration.

Feature: #57632 - Include inline language label files with TypoScript

See Issue #57632

Description

It is now possible to add inline language label files with TypoScript.

Usage

If you want to include inline labels from a XLF file, you have to specify that file in your TypoScript with a custom key in the new `inlineLanguageLabelFiles` section. In addition to the file you can configure three optional parameters:

- `selectionPrefix`: Only label keys that start with this prefix will be included (default: '')
- `stripFromSelectionName`: A string that will be removed from any included label key (default: '')
- `errorMode`: Error mode if the file could not be found: 0 - syslog entry, 1 - do nothing, 2 - throw an exception (default: 0)

Example

```
page = PAGE
page.inlineLanguageLabelFiles {
        someLabels = EXT:myExt/Resources/Private/Language/locallang.xlf
        someLabels.selectionPrefix = idPrefix
        someLabels.stripFromSelectionName = strip_me
        someLabels.errorMode = 2
}
```

Output in the HTML head:

```
var TYPO3 = TYPO3 || {};
TYPO3.lang = {"firstLabel":[{"source":"first Label","target":"erstes Label"}],
↪"secondLabel":[{"source":"second Label","target":"zweites Label"}]};
```

Feature: #59144 - Previewing workspace records using Page TSconfig

See Issue #59144

Description

Per default TYPO3 only creates preview links for the tables tt_content, pages and pages_language_overlay. To avoid utilizing a hook for each table, creating preview links can be triggered using Page TSconfig.

```
# Using page 123 for previewing workspaces records (in general)
options.workspaces.previewPageId = 123

# Using the pid field of each record for previewing (in general)
options.workspaces.previewPageId = field:pid

# Using page 123 for previewing workspaces records (for table tx_myext_table)
options.workspaces.previewPageId.tx_myext_table = 123

# Using the pid field of each record for previewing (or table tx_myext_table)
options.workspaces.previewPageId.tx_myext_table = field:pid
```

Feature: #59591 - Image quality definable per sourceCollection

See Issue #59591

Description

The image quality of each `sourceCollection` entry can be configured. Integrators can already render images with the predefined quality set by LocalConfiguration.php. To decrease the quality of larger images (e.g. double density) in order to lower the file size, integrators can configure the parameter `quality` of the matching `sourceCollection` now.

The TypoScript setup can be configured as followed (e.g.):

```
# for small retina images
tt_content.image.20.1.sourceCollection.smallRetina.quality = 80

# for large retina images
tt_content.image.20.1.sourceCollection.largeRetina.quality = 65
```

If the new parameter is not set, TYPO3 will use to the default quality of LocalConfiguration.

Impact

The rendering of `sourceCollection` stays as it is. Users can additionally selectively control the quality of jpeg by TypoScript setup.

Feature: #61799 - Improved handling of online media

See Issue #61799

Description

Editors can now use YouTube and Vimeo videos (online media) just like a any other file, organising them just like any other file in the file list and selecting them in element browser to use in a CE or any other record. Adding new online media files is done providing the URL to online media. The matching helper class will fetch the needed metadata and supply an image that will be used as preview if available.

YouTube and Vimeo support

The core provides an `OnlineMediaHelper` and a `FileRenderer` class for YouTube and Vimeo.

Adding YouTube videos can be done by providing a URL in one of the following formats (with and without http(s)://):

- youtu.be/<code> # Share URL
- www.youtube.com/watch?v=<code> # Normal web link
- www.youtube.com/v/<code>
- www.youtube-nocookie.com/v/<code> # youtube-nocookie.com web link
- www.youtube.com/embed/<code> # URL form iframe embed code, can also get code from full iframe snippet

Adding Vimeo videos can be done by providing a URL in one of the following formats (with and without http(s)://):

- vimeo.com/<code> # Share URL
- player.vimeo.com/video/<code> # URL form iframe embed code, can also get code from full iframe snippet

Each renderer has some custom configuration options:

YouTubeRenderer

- `bool autoplay` default = FALSE; when set video starts immediately after loading of the page
- `int controls` default = 2; see https://developers.google.com/youtube/player_parameters#controls
- `bool loop` default = FALSE; if set video starts over again from te beginning when finished
- `bool enablejsapi` default = TRUE; see https://developers.google.com/youtube/player_parameters#enablejsapi
- `bool showinfo` default = FALSE; show video title and uploader before video starts playing
- `bool no-cookie` default = FALSE; use domain youtube-nocookie.com instead of youtube.com when embedding a video

Example of setting the YouTubeRenderer options with the MediaViewHelper:

```
<!-- enable js api and set no-cookie support for YouTube videos -->
<f:media file="{file}" additionalConfig="{enablejsapi:1, 'no-cookie': true}" />
```

VimeoRenderer

- `bool autoplay` default = FALSE; when set video starts immediately after loading of the page
- `bool loop` default = FALSE; if set video starts over again from te beginning when finished
- `bool showinfo` default = FALSE; show video title and uploader before video starts playing

Example of setting the YouTubeRenderer options with the MediaViewHelper:

```
<!-- show title and uploader for YouTube and Vimeo before video starts playing -->
<f:media file="{file}" additionalConfig="{showinfo:1}" />
```

Register your own online media service

For every service you need an `OnlineMediaHelper` class that implements `OnlineMediaHelperInterface` and a `FileRenderer` class (see #61800) that implements `FileRendererInterface`. The online media helper is responsible for translating the input given by the editor to a `onlineMediaId` that is known to the service. The renderer is responsible for turning the `onlineMediaId` to the correct HTML output to show the media item.

The `onlineMediaId` is stored in a plain text file that only holds this ID. By giving this file a custom file extension TYPO3 knows which `OnlineMediaHelper` and `FileRenderer` belong to it. To further tell TYPO3 what kind of "file" (text, image, audio, video, application, other) this online media holds we also need to bind a custom mime-type to this file extension.

With adding this custom file extension to `$GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext']` (see #69543) your custom online media file can be used throughout the backend every where all media files are allowed.

Example of registering your own online media file/service:

```
// Register your own online video service (the used key is also the bind file␣
↪extension name)
$GLOBALS['TYPO3_CONF_VARS']['SYS']['fal']['onlineMediaHelpers']['myvideo'] =␣
↪\MyCompany\Myextension\Helpers\MyVideoHelper::class;

$rendererRegistry = \TYPO3\CMS\Core\Resource\Rendering\RendererRegistry::
↪getInstance();
$rendererRegistry->registerRendererClass(
    \MyCompany\Myextension\Rendering\MyVideoRenderer::class
);

// Register an custom mime-type for your videos
$GLOBALS['TYPO3_CONF_VARS']['SYS']['FileInfo']['fileExtensionToMimeType']['myvideo
↪'] = 'video/myvideo';

// Register your custom file extension as allowed media file
$GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext'] .= ',myvideo';
```

Override core Helper class with your own helper class

The helper classed provided by the core use the `oEmbed` web service provided by YouTube and Vimeo to gather some basic metadata for the provided video urls. The upside is that you do not need an API user/key to use their webservice as these services are publicly available. But the downside is that the gathered info is kind of scarce. So if you have an API user/key for these services, you could create an own helper class which provides more meta data.

```
// Register your own online custom youtube helper class
$GLOBALS['TYPO3_CONF_VARS']['SYS']['fal']['onlineMediaHelpers']['youtube'] =␣
↪\MyCompany\Myextension\Helpers\YouTubeHelper::class;
```

Feature: #61993 - _CSS_PAGE_STYLE is now only included on the affected page

See Issue #61993

Description

CSS set via the TypoScript property `_CSS_PAGE_STYLE` was concatenated and compressed with the non-page-specific CSS and therefore loaded on pages it did not affect at all.

Impact

The behaviour from now on is that `_CSS_PAGE_STYLE` is included only on the affected page. Depending on your configuration it will be written in an external file and included on the page or directly added as inline CSS block. Compression for page specific CSS also depends on the global `config.compressCss` setting.

Feature: #63395 - HTML5 video poster preview image

See Issue #63395

Description

Enable Video preview image "poster" in media in the old fashion way.

If there is a jpg, jpeg or png image in the same folder with the same name as the video file it will be shown with the HTML5 poster attribute.

The file extensions are checked exactly in the order mentioned above.

Feature: #64535 - IRRE: Suppress and override useCombination warning via TCA settings

See Issue #64535

Description

When using `useCombination=TRUE` there is always a FlashMessage warning displayed. It is now possible to override the default warning message with a custom message or to suppress the FlashMessage completely via TCA setting.

Example to suppress `useCombination` warning message:

```
$GLOBALS['TCA']['tx_demo_domain_model_demoinline']['columns']['irre_records']['
↪'config'] = array(
        'foreign_types_combination' => array(
                '1' => array(
                        'showitem' => 'title'
                )
        )
        'appearance' => array(
                'suppressCombinationWarning' => TRUE
                'useCombination' => TRUE
        )
)
```

Example to override `useCombination` warning message:

```
$GLOBALS['TCA']['tx_demo_domain_model_demoinline']['columns']['irre_records']['
↪'config'] = array(
        'foreign_types_combination' => array(
                '1' => array(
                        'showitem' => 'title'
                )
        )
        'appearance' => array(
                'overwriteCombinationWarningMessage' => 'LLL:EXT:demo/Resources/
↪Private/Language/locallang_db.xlf:tx_demo_domain_model_demoinline.irre_records.
↪useCombinationWarning'
                'useCombination' => TRUE
        )
)
```

Feature: #64726 - Added support for multiple FlashMessage queues

See Issue #64726

Description

In Extbase ControllerContext the method `getFlashMessageQueue()` now optionally allows to specify which queue to fetch. If none is specified the `default-messagequeue` for the current controller/plugin will be used.

```
$this->controllerContext->getFlashMessageQueue($queueIdentifier);
```

In Fluid the flashMessages-ViewHelper also allows to specify a queue to use.

```
<f:flashMessages queueIdentifier="myQueue" />
```

Impact

Extensions may now render foreign flash message queues and add messages to them.

Feature: #58621 - Unified Backend Routing

See Issue #58621

Description

A new Routing component has been added to the TYPO3 Backend which handles addressing different calls / modules inside TYPO3.

A **Route** is the smallest entity consisting of a path (e.g. "/records/edit/") as well as an identifier for addressing the route, and the information about how to dispatch the route to a PHP controller.

A **Route** can be a module, wizard or any page inside the TYPO3 Backend. The Router contains the public API for matching paths to fetch a Route and is resolved inside the RequestHandler of the Backend.

The entry point for Routes is `typo3/index.php?route=myroute&token=.....` The main RequestHandler for all Backend requests detects if a route parameter from the server is given and uses this as the route identifier and then resolves to a controller defined inside the Route.

Routes are defined inside the file "Configuration/Backend/Routes.php" of any extension.

Example of a Configuration/Backend/Routes.php file:

```
return [
        'myRouteIdentifier' => [
                'path' => '/document/edit',
                'controller' => Acme\MyExtension\Controller\MyExampleController::
↪class . '::methodToCall'
        ]
];
```

The called method in the controller to be called receives a PSR-7 compliant Request object and a PSR-7 Response object, and has to return a PSR-7 Response object. The UriBuilder generates any kind of URL for the Backend, may it be a module, a typical route or an AJAX call. The UriBuilder returns a PSR-7-conform Uri object that can be casted to string when needed.

Usage:

```
$uriBuilder = GeneralUtility::makeInstance(UriBuilder::class);
$uri = $uriBuilder->buildUriFromRoute('myRouteIdentifier', array('foo' => 'bar'));
```

See http://wiki.typo3.org/Blueprints/BackendRouting for more details.

Impact

Handling of existing modules works the same as before and fully transparent. Any existing registration of entrypoints can be moved to the new registration file in Configuration/Backend/Routes.php.

Feature: #65791 - Use PHP configured sendmail path, if [MAIL][transport] = sendmail is active

See Issue #65791

Description

The install tool setting `[MAIL][transport_sendmail_command]` is now retrieved automatically from PHP runtime configuration `sendmail_path` during installation (instead of '/usr/sbin/sendmail -bs').

Impact

There are no impacts on current installations.

New installations will have `[MAIL][transport_sendmail_command]` automatically set during installation using `sendmail_path` from PHP runtime configuration. It can still be changed manually.

As this setting is only used if `[MAIL][transport]` is set to `sendmail`, it doesn't have impact on other transport schemes.

Feature: #66366 - Introduced MediaViewHelper

See Issue #66366

Description

In order to comfortably render video, audio and all other file types with a registered Renderer class (`RenderingRegistry` introduced with #61800) in FE, the `MediaViewHelper` has been added.

The `MediaViewHelper` first checks if there is a Renderer present for the given file. If not, it will as fallback render a image tag. This way it is a replacement for the `ImageViewHelper` in most cases when rendering video and audio tags.

Examples:

```
<code title="Image Object">
    <f:media file="{file}" width="400" height="375" />
</code>
<output>
    <img alt="alt set in image record" src="fileadmin/_processed_/323223424.png"␣
↪width="396" height="375" />
</output>

<code title="MP4 Video Object">
    <f:media file="{file}" width="400" height="375" />
</code>
<output>
    <video width="400" height="375" controls><source src="fileadmin/user_upload/my-
↪video.mp4" type="video/mp4"></video>
</output>

<code title="MP4 Video Object with loop and autoplay option set">
    <f:media file="{file}" width="400" height="375" additionalConfig="{loop: '1',␣
↪autoplay: '1'}" />
</code>
<output>
    <video width="400" height="375" controls loop><source src="fileadmin/user_
↪upload/my-video.mp4" type="video/mp4"></video>
</output>
```

Feature: #66371 - Introduce autoplay option for video and audio files

See Issue #66371

Description

The `RenderingRegistry` added with #61800 introduced the option to render video and audio tags with the new `MediaViewHelper` added with #66366. To improve the usability of this feature an autoplay checkbox has been added to the `sys_file_reference` records to enable the editor to configure this option on a per file basis.

To make the autoplay option available in sys_file_reference records, make use of the new palettes `videoOverlayPalette` and `audioOverlayPalette` in your TCA.

However, the autoplay property of the `sys_file_reference` is only taken into account if the view helper does not explicitly specify an autoplay option.

Examples:

Example config of an sys_file_reference field in TCA:

```
'media' => array(
        'exclude' => 1,
        'label' => 'Media',
        'config' => \TYPO3\CMS\Core\Utility\ExtensionManagementUtility::
↪getFileFieldTCAConfig(
                'media',
```

```
                array(
                        'foreign_types' => array(
                                \TYPO3\CMS\Core\Resource\File::FILETYPE_AUDIO =>␣
↪array(
                                        'showitem' => '
                                                --palette--;;audioOverlayPalette,
                                                --palette--;;filePalette',
                                ),
                                \TYPO3\CMS\Core\Resource\File::FILETYPE_VIDEO =>␣
↪array(
                                        'showitem' => '
                                                --palette--;;videoOverlayPalette,
                                                --palette--;;filePalette',
                                )
                        )
                ),
                'wav,mpeg,mp4,ogg'
        )
)
```

```
<code title="MP4 Video Object with autoplay option set regardless of sys_file_
↪reference checkbox">
        <f:media file="{file}" width="400" height="375" additionalConfig="{autoplay:
↪ '1'}" />
</code>
<output>
        <video width="400" height="375" controls autoplay><source src="fileadmin/
↪user_upload/my-video.mp4" type="video/mp4"></video>
</output>

 <code title="MP4 Video Object without autoplay option set will respect the␣
↪configuration of the sys_file_reference record">
        <f:media file="{file}" width="400" height="375" />
</code>
<output>
        <video width="400" height="375" controls><source src="fileadmin/user_upload/
↪my-video.mp4" type="video/mp4"></video>
</output>
```

Feature: #67056 - Add option to disable move buttons TCA group type

See Issue #67056

Description

The move buttons of the TCA type `group` can now be explicitly disabled with the `hideMoveIcons` option. Before these icons where only automatically removed if `maxitems` was set to 1.

```
'options' => array(
        'label' => 'Options',
        'config' => array(
                'type' => 'group',
                'internal_type' => 'db',
                'allowed' => 'pages',
                'maxitems' => 9999,
                'hideMoveIcons' => TRUE,
        ),
),
```

Impact

Move buttons can now always be hidden for `group` fields

Feature: #67875 - Override CategoryRegistry entries

See Issue #67875

Description

The `makeCategorizable` method of the `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility` class has been extended with a new parameter `override` to set a new category configuration for an already registered table / field combination.

If the parameter is set to `TRUE`, previously defined registry entries are cleared for the current table / field combination before adding the new configuration.

The intended usecase for this method is to add additional TCA types for a previously registered table.

A good example is the `tt_content` table:

1. The basic TCA is defined in the `frontend` Extension.
2. After the processing of the normal TCA definition, the default categorized tables (from the install tool setting `SYS/defaultCategorizedTables`) are initialized and the categories tab is added to the `showitem` configuration for all TCA types that exist so far.
3. Now the TCA overrides are processed. The `css_styled_content` Extension defines additional TCA types. After the types are defined the `addOrOverride()` method is called to add the category tab to them.

Impact

The current behavior of the existing functionality is not changed. Only new functionality is added.

Example

```
// This example is from the tt_content TCA overrides file from the css_styled_
↪content Extension.
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::makeCategorizable('css_styled_
↪content', 'tt_content', 'categories', array(), TRUE);
```

Feature: #67880 - Added count to split

See Issue #67880

Description

A new property `returnCount` is added to the stdWrap property `split`.

When dealing with comma separated values like the content of `field:records` or similar, we might need to know how many items are present inside the csv.

Example:

```
# should return 9
1 = TEXT
1 {
        value = x,y,z,1,2,3,a,b,c
        split.token = ,
        split.returnCount = 1
}
```

Feature: #67932 - RenderType for rsa encrypted input fields

See Issue #67932

Description

EXT:rsaauth defines an own render type for TCA input fields. Those fields will be encrypted before submitting the form.

Impact

In the Backend password fields for backend and frontend users are automatically encoded before the form is submitted.

Usage

To encrypt your own TCA fields you can add define the render type `rsaInput`.

```
```

$GLOBALS['TCA']['be_users']['columns']['password']['config']['renderType'] = 'rsaInput';

Feature: #68429 - Introduced AvatarProvider API

See Issue #68429

Description

To make providing an avatar image for BE users more flexible an API has been introduced so you can register AvatarProviders. The core provides the `DefaultAvatarProvider` by default to handle the image defined in the user settings.

When an avatar is rendered in the BE the available `AvatarProviders` are asked if they can provide an `TYPO3\CMS\Backend\Backend\Avatar\Image` for the given `be_users` record in the requested size. The first `TYPO3\CMS\Backend\Backend\Avatar\Image` that gets returned is used.

Registering an avatar provider

An avatar provider can be registered within your `ext_localconf.php` file like this:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']['avatarProviders'][
↪'myCustomProvider'] = [
        'provider' => \MyVendor\MyExtension\AvatarProvider\CompanyAvatarProvider::
↪class,
```

```
        'before' => ['defaultAvatarProvider']
];
```

The settings are defined as:

- `provider`: The avatar provider class name, which must implement `TYPO3\CMS\Backend\Backend\Avatar\AvatarProviderInterface`.
- `before/after`: You can define the ordering how providers are executed. This is used to get the order in which the providers are executed.

For a new avatar provider you have to register a **new key** in `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']`. If your avatar provider extends another one, you may only overwrite necessary settings. An example would be to extend an existing provider and replace its registered 'provider' class with your new class name.

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']['avatarProviders']['
↪'defaultAvatarProvider']['provider'] =␣
↪\MyVendor\MyExtension\AvatarProvider\CustomAvatarProvider::class;
```

AvatarProviderInterface

The AvatarProviderInterface contains only one method:

`public function getImage(array $backendUser,$size);`

The parameters are defined as:

- `$backendUser`: The record from `be_users` database table.
- `$size`: The requested size of the avatar image.

The return value of the method is expected to be an instance of `TYPO3\CMS\Backend\Backend\Avatar\Image` or NULL when the provider can not provide an image.

An `TYPO3\CMS\Backend\Backend\Image` object has 3 properties:

- `$url`: Url of avatar image. Needs to be relative to the website root or an absolute URL.
- `$width`: The width of the image.
- `$height`: The height of the image.

Feature: #68700 - Autoload definition can be provided in ext_emconf.php

See Issue #68700

Description

It is now possible for extensions to provide one or more PSR-4 definitions, in the ext_emconf.php file.

While it was possible to define a psr-4 section in a composer.json before already, now it is also possible to define an autoload/psr-4 section in the ext_emconf.php file as well, so that extension authors do not need to provide a composer.json just for that any more.

This is the new recommended way to register classes for TYPO3.

Example ext_emconf.php:

```
<?php
$EM_CONF[$_EXTKEY] = array (
  'title' => 'Extension skeleton for TYPO3 7',
  'description' => 'Description for ext',
```

```
  'category' => 'Example Extensions',
  'author' => 'Helmut Hummel',
  'author_email' => 'info@helhum.io',
  'author_company' => 'helhum.io',
  'shy' => '',
  'priority' => '',
  'module' => '',
  'state' => 'stable',
  'internal' => '',
  'uploadfolder' => '0',
  'createDirs' => '',
  'modify_tables' => '',
  'clearCacheOnLoad' => 0,
  'lockType' => '',
  'version' => '0.0.1',
  'constraints' =>
  array (
        'depends' =>
        array (
          'typo3' => '7.5.0-7.99.99',
        ),
        'conflicts' =>
        array (
        ),
        'suggests' =>
        array (
        ),
  ),
  'autoload' =>
  array(
        'psr-4' =>
        array(
          'Helhum\\ExtScaffold\\' => 'Classes'
        )
  )
);
```

Impact

Without providing an autoload section, TYPO3 scans the complete extension directory for PHP class files and registers them all. This includes test classes or classes of third party libraries, which might lead to unexpected results.

Therefore it is recommended to provide such an autoload section in an extension. It will be ignored in older TYPO3 versions, so there will be no issue with backwards compatibility.

Feature: #68724 - EM: "Get preconfigured distribution" shows only distributions that suite the current TYPO3 version

See Issue #68724

Description

As the list of distributions gets longer, it is filtered now to those distributions that suite the current TYPO3 version.

Impact

Distributions that require different TYPO3 versions are not shown anymore.

Feature: #68741 - Introduce new IconFactory as base to replace the icon skinning API

See Issue #68741

Description

The logic for working with icons, icon sizes and icon overlays is now bundled into the new `IconFactory` class. The new icon factory will replace the old icon skinning API step by step.

All core icons will be registered directly in the `IconRegistry` class, third party extensions must use `IconRegistry::registerIcon()` to override existing icons or add additional icons to the icon factory.

The `IconFactory` takes care of the correct icon and overlay size and the markup.

IconProvider

The core implements three icon provider classes, which all implement the `IconProviderInterface`.

- `BitmapIconProvider` for all kind of bitmap icons for gif, png and jpg files
- `FontawesomeIconProvider` for font icons from fontawesome.io
- `SvgIconProvider` for svg icons

Third party extensions can provide own icon provider classes, each class must implement the `IconProviderInterface`.

BitmapIconProvider

The `BitmapIconProvider` has the following option

- `source` The path to the bitmap file, this may also contain the EXT: prefix

FontawesomeIconProvider

The `FontawesomeIconProvider` has the following option

- `name` The name of the icon without the icon prefix e.g. `check` instead of `fa-check`

SvgIconProvider

The `SvgIconProvider` has the following option

- `source` The path to the svg file, this may also contains the EXT: prefix

Register an icon

```
/*
 * Put the following code into your ext_localconf.php file of your extension.
 *
 * @param string $identifier the icon identifier
 * @param string $iconProviderClassName the icon provider class name
 * @param array $options provider specific options, please reference the icon␣
↪provider class
 */
$iconRegistry = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Imaging\IconRegistry::class);
$iconRegistry->registerIcon($identifier, $iconProviderClassName, array $options =␣
↪array());
```

Use an icon

To use an icon, you need at least the icon identifier. The default size is currently 32x32 px. The third parameter can be used to add an additional icon as overlay, which can be any registered icon.

The `Icon` class provides only the following constants for Icon sizes:

- `Icon::SIZE_SMALL` which currently means 16x16 px
- `Icon::SIZE_DEFAULT` which currently means 32x32 px
- `Icon::SIZE_LARGE` which currently means 48x48 px

All the sizes can change in future, so please make use of the constants for an unified layout.

```
$iconFactory = GeneralUtility::makeInstance(IconFactory::class);
$iconFactory->getIcon($identifier, Icon::SIZE_SMALL, $overlay)->render();
```

ViewHelper

The core provides a fluid ViewHelper which makes it really easy to use icons within a fluid view.

```
{namespace core = TYPO3\CMS\Core\ViewHelpers}
<core:icon identifier="my-icon-identifier" />
<!-- use the "small" size if none given ->
<core:icon identifier="my-icon-identifier" />
<core:icon identifier="my-icon-identifier" size="large" />
<core:icon identifier="my-icon-identifier" overlay="overlay-identifier" />
<core:icon identifier="my-icon-identifier" size="default" overlay="overlay-
↪identifier" />
<core:icon identifier="my-icon-identifier" size="large" overlay="overlay-identifier
↪" />
```

Impact

No impact

Feature: #68746 - Add annotation for CLI only commands

See Issue #68746

Description

The PHPDoc annotation `@cli` was added to indicate Extbase CommandController commands to be usable on CLI only. In general each defined CommandController can be selected within the Extbase CommandController Task in the scheduler. For some commands like `extbase:help:help` running in a scheduler task is not wanted or needed. Now those commands can be excluded from the scheduler command selection.

Impact

Extbase `CommandController` commands annotated with `@cli` are not shown as command in the scheduler task.

Feature: #68756 - Add config "base" to stdWrap

See Issue #68756

Description

The following function was updated and added with a new optional parameter $base: `TYPO3\CMS\Core\Utility\GeneralUtility::formatSize($sizeInBytes,$labels = '',$base = 0)`

This affects the function: `TYPO3\CMS\Frontend\ContentObject\ContentObjectRenderer->stdWrap_bytes`

Until now it was not possible to set the base parameter via TypoScript if you add custom labels.

Impact

The ability to set the base (1000 or 1024) via TypoScript configuration has been added.

With the `base` property it can be defined whether to use a base of 1000 or 1024 to calculate with

Thus:: bytes.labels = " | K| M| G" bytes.base = 1000

Feature: #68757 - Provide untouched newPassword in felogin password_changed hook

See Issue #68757

Description

The new parameter `newPasswordUnencrypted` in the EXT:felogin password_changed hook won't be salted if EXT:saltedpaswords is enabled. It is now possible to work with the real new password.

Feature: #68773 - Show a special image for official distributions in Extension Manager

See Issue #68773

Description

For official distributions a special image is displayed in the "Get preconfigured distribution" list. So it is easier now to distinguish the official distributions from others.

Feature: #68804 - Colored output for CLI-relevant error messages

See Issue #68804

Description

Calling `typo3/cli_dispatch.phpsh` via the command line will now show a colored error message when an invalid or no CLI key as first parameter is given.

Feature: #68837 - Closures for Command Line Scripts

See Issue #68837

Description

For registering new command line scripts through the CLI API ("cliKey"), it is now possible to use PHP closures instead of reference to PHP scripts.

Example usage inside ext_localconf.php:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['GLOBAL']['cliKeys']['myclikey'] = array(
        function() {
                $controller = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\Acme\MyExtension\CommandLineTool::class);
                $controller->main();
        },
        '_CLI_lowlevel'
);
```

Feature: #69095 - Introduce icon state for IconFactory

See Issue #69095

Description

A state (default or disabled) for icons has been added. The state "disabled" marks an icon as disabled and shows the icon with 50% opacity.

Use an icon

The method `IconFactory::getIcon()` has now a fourth parameter for the state.

The `\TYPO3\CMS\Core\Type\Icon\IconState` class provides only the following constants for icon states:

- `State::STATE_DEFAULT` which currently means 100% opacity
- `State::STATE_DISABLED` which currently means 50% opacity

The states may change in future, so please make use of the constants for an unified layout.

```
$iconFactory = GeneralUtility::makeInstance(IconFactory::class);
$iconFactory->getIcon($identifier, Icon::SIZE_SMALL, $overlay, IconState::
↪cast(IconState::STATE_DEFAULT))->render();
```

ViewHelper

The core provides a Fluid ViewHelper which makes it really easy to use icons within a Fluid view. This ViewHelper has an argument for the new state parameter.

```
{namespace core = TYPO3\CMS\Core\ViewHelpers}
<core:icon identifier="my-icon-identifier" state="disabled" />
```

Feature: #69119 - Add a basic search to the filelist module

See Issue #69119

Description

A basic recursive file search by file name has been added to be able to search for files in the file list module like it was possible with EXT:dam.

The search happens recursively from the currently chosen folder in the folder tree. This way it is possible to search whole mount points or just single folders with a lot of files.

The search results will be displayed similar to the regular file list although some features of the regular list view are missing. There is no possibility to order the search results yet. Also the buttons for localization and clipboard commands are missing in this first implementation. Regular file command links like editing, renaming and deleting are already implemented. By default the search results are ordered by the file identifier, i.e. the file path ascending from A-Z.

Feature: #69389 - Add spinning feature for icon

See Issue #69389

Description

The Icon API has now a support for spinning icons. While registering an icon a new property `spinning` is available:

```
$iconRegistry = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Imaging\IconRegistry::class);
$iconRegistry->registerIcon(
        'my-spinning-icon',
        \TYPO3\CMS\Core\Imaging\IconProvider\FontawesomeIconProvider::class,
        array(
                'name' => 'times',
                'spinning' => TRUE
        )
);
```

Impact

Icons can now be animated.

Feature: #69401 - Adopt form to support the Extbase/ Fluid MVC stack

See Issue #69401

Description

Short summery

The form extension - including the custom data model, controller logic, property validation, views and templating - has been adopted to support the Extbase/ Fluid MVC stack. This allows better customization and control of the generated behavior and markup by simply modifying Fluid templates or utilizing own custom view helper logic. At the same time the rewrite must not break current setups, i.e. the frontend rendering must be as compatible as possible.

Details

Fluid Rendering

The rendering of the frontend output is based on Fluid. Form relies on the native Fluid viewhelpers of the core and ships 2 new viewhelpers for optimal rendering of the SELECT object including the support of OPTGROUP objects. Furthermore a viewhelper is included to optimize the output of text mails.

To support existing setups a compatibility mode is introduced. The mode is activated by default. This has no impact on the rendering as long as no "old" TypoScript settings (like .layout =) are present. If old rendering settings are used a compatibility theme is loaded which guarantees maximum backwards compatibility including all the different wrap-abilities like `containerWrap` and `elementWrap`.

For new installations it is recommended to switch off the compatibility mode and use own Fluid templates to customize the output.

For each form object and view a Fluid partial is available. There are 3 views: the form itself (show), the confirmation page (confirmation) and the email (postProcessor/ mail). The patch allows to customize the frontend output for every single view, e.g. one can have a custom Fluid partial for the BUTTON object of the confirmation view.

The `partialRootPath` can be overridden or extended to customize the form objects on a global scope. Furthermore it is possible to set a partial path for each form element on a local scope.

```
10 = BUTTON
10 {
        label = My button
        name = myButton
        partialPath = FlatElements/MyButton
}
```

The Fluid rendering would look for a MyButton.html located in the defined `partialRootPath`.

In addition it is now possible to decide if an element should be rendered for a specific view. The visualisation can be adopted by using the TypoScript settings `visibleInShowAction`, `visibleInConfirmationAction` and `visibleInMail`. As an example, this is utilized to hide the FIELDSET object on the confirmation page which was the default behaviour in earlier versions of form.

Extensibility

It is now possible to register custom form objects and attributes easily only by using TypoScript and Fluid. Form attributes can now be cObjects and use stdWrap. This is only possible if the form was not designed within the form wizard.

Furthermore 2 new signal slots are implemented to allow the manipulation of the form objects and the submitted data.

Validation

The validators are now using the extbase property mapping validation process.

Additional information

The session handling was dropped since it was unstable (see #58765). Now form relies on the concepts of Extbase.

The unit tests have been adopted to reflect the code changes.

Future

Further patches are needed to adopt the form wizard. The wizard still works as it used to after applying this patch but it is not able to reflect the new features like choosing a partial path for a single element.

Another patch will take care of the documentation.

A few more patches will come which will fix some issues regarding the validators and filters.

Feature: #69409 - Allow value attribute for OPTION object in wizard

See Issue #69409

Description

Now it is possible to add a value to the OPTION object using the wizard. Furthermore the table within the field options of the SELECT configuration has been improved. There were no labels shown which has been fixed now.

Feature: #69416 - Plugins (AbstractPlugin) can load custom language file

See Issue #69416

Description

`AbstractPlugin::pi_loadLL` takes an optional argument specifying path to a language file. It allows placing language files in other paths like in Extbase structure "Resources/Private/Language". Previously language file had to be located in the directory set in the `scriptRelPath` property.

Impact

Possibility to put language label files in other paths.

Feature: #69459 - Show tt_content preview in page module via Fluid

See Issue #69459

Description

A new PageTSconfig option allows to render a preview of a single content element in the Backend via Fluid.

The following option allows to override the default output of a content element via PageTSconfig:

```
mod.web_layout.tt_content.preview.media = EXT:site_mysite/Resources/Private/
↪Templates/Preview/Media.html
```

All content elements with CType "media" will then be rendered by the Fluid Template which can be rendered like:

```
<h4>{header}</h4>
<f:format.crop length="200">{bodytext}</f:format.crop>
```

All properties of the tt_content record are available in the template directly. Any data of the flexform field `pi_flexform` is available with the property `pi_flexform_transformed` as an array.

Note: If a PHP hook already is set to render the element, it will take precedence over the Fluid-based preview.

Feature: #69496 - Extract title from PDF when indexing

See Issue #69496

Description

Indexed search is now able to natively extract the title when indexing PDF documents.

Impact

The title of indexed PDF documents is now extracted.

Feature: #69512 - Support *.typoscript files as text file type

See Issue #69512

Description

The file type **typoscript** has been added to the list of text file types.

Impact

Files with the type **typoscript** can now be edited through the file list module.

Feature: #69543 - Introduced $GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext']

See Issue #69543

Description

Now we got the `RendererRegistry` with the `VideoTagRenderer`, `AudioTagRenderer` and `MediaViewHelper` in the core we needed also a way to define a list of file extensions of the files that can be handled by these. This list can then be used in the TCA for allowing sys_file_references to these files.

```
// Comma list of file extensions perceived as media files by TYPO3.
// Lowercase and no spaces between
$GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext'] = 'gif,jpg,jpeg,bmp,png,pdf,svg,
↪ai,mov,avi';
```

TCA example:

```
'media' => array(
        'label' => 'LLL:EXT:lang/locallang_general.xlf:LGL.media',
        'config' => \TYPO3\CMS\Core\Utility\ExtensionManagementUtility::
↪getFileFieldTCAConfig('media', array(
                'foreign_types' => array(
                        '0' => array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        ),
                        \TYPO3\CMS\Core\Resource\File::FILETYPE_TEXT => array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        ),
                        \TYPO3\CMS\Core\Resource\File::FILETYPE_IMAGE => array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        ),
                        \TYPO3\CMS\Core\Resource\File::FILETYPE_AUDIO => array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        ),
                        \TYPO3\CMS\Core\Resource\File::FILETYPE_VIDEO => array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        ),
                        \TYPO3\CMS\Core\Resource\File::FILETYPE_APPLICATION =>␣
↪array(
                                'showitem' => '
                                        --palette--;LLL:EXT:lang/locallang_tca.xlf:
↪sys_file_reference.imageoverlayPalette;imageoverlayPalette,
                                        --palette--;;filePalette'
                        )
                )
        ), $GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext'])
),
```

Extending this list:

If you want to extend this list you can add the desired extension name to list in the `ext_localconf.php` of your extension.

```
$GLOBALS['TYPO3_CONF_VARS']['SYS']['mediafile_ext'] .= ',myext';
```

Feature: #69568 - FormEngine data processing

See Issue #69568

Description

Warning: The `FormEngine` data structure will change in the future and extensions must not rely on array or class structures at the moment.

The FormEngine construct to render records has been split to two main parts where the first data processing part takes care of gathering and processing all data needed for the second part to render final form data.

The data processing is done via `FormDataCompiler` that returns a data array that can be given to the outer most render container. The array contains all main data required by the render part like final `TCA` as well as the processed database row.

Extensions can change the data processing by registering additional items in the data processing chain. For casual `TCA` based database records, the `FormDataGroup TcaDatabaseRecord` is used to define relevant data provider within `$GLOBALS['TYPO3_CONF_VARS']['SYS']['formEngine']['formDataGroup']['tcaDatabaseRecord']`. Additional data provider can be added at specific positions using the `depends` and `before` keywords relative to other providers.

Feature: #69602 - Simplify handling of backend layouts in frontend

See Issue #69602

Description

To avoid complex TypoScript for integrators, the handling of backend layouts has been simplified for the frontend.

To get the correct backend layout, the following TypoScript code can be used:

```
page.10 = FLUIDTEMPLATE
page.10 {
  file.stdWrap.cObject = CASE
  file.stdWrap.cObject {
        key.data = pagelayout

        default = TEXT
        default.value = EXT:sitepackage/Resources/Private/Templates/Home.html

        3 = TEXT
        3.value = EXT:sitepackage/Resources/Private/Templates/1-col.html

        4 = TEXT
        4.value = EXT:sitepackage/Resources/Private/Templates/2-col.html
  }
}
```

Using `data = pagelayout` is the same as using as

```
field = backend_layout
ifEmpty.data = levelfield:-2,backend_layout_next_level,slide
ifEmpty.ifEmpty = default
```

Feature: #69730 - Introduce uniqueId generator

See Issue #69730

Description

A new method `getUniqueId()` has been added to the StringUtility class. Now there is a common way to generate an unique identifier which can be used even in HTML tag attributes as it removes the invalid dot character.

```
$uniqueId = \TYPO3\CMS\Core\Utility\StringUtility::getUniqueId('Prefix');
```

Impact

No need to remove the dot manually anymore.

Feature: #69855 - Dispatcher for Backend Routing added

See Issue #69855

Description

The previously introduced Backend Routing is updated so that Routes must be defined with a class name and method name, or a Closure / callable. The controller/action or closure is now named as "target".

Example from `EXT:backend/Configuration/Backend/Routes.php`

```
// Logout script for the TYPO3 Backend
'logout' => [
        'path' => '/logout',
        'target' => Controller\LogoutController::class . '::logoutAction'
]
```

Impact

Each method that is registered will receive both the Request object and the Response object which can be manipulated for output.

The fixed `ControllerInterface` is not needed anymore and will be removed.

Feature: #69918 - Add PSR-7-based dispatching for Backend Modules

See Issue #69918

Description

Built on the PSR-7 principle and the routing concepts, it is now possible to register backend modules which are dispatched to a callable string instead of pointing to an index.php file in `EXT:myextension/Modules/MyModule/index.php`.

The method which is called, receives a PSR-compatible request and response object and must return a response object which is outputted to the browser.

An example registration uses the option `routeTarget` to resolve the method to be called when rendering the module:

```
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::addModule(
        'web',
        'layout',
        'top',
        '',
        array(
                'routeTarget' => \TYPO3\CMS\Backend\Controller\PageLayoutController:
↪:class . '::mainAction',
                'access' => 'user,group',
                'name' => 'web_layout',
                'labels' => array(
                        'tabs_images' => array(
                                'tab' => 'EXT:backend/Resources/Public/Icons/module-
↪page.svg',
                        ),
                        'll_ref' => 'LLL:EXT:backend/Resources/Private/Language/
↪locallang_mod.xlf',
                ),
        )
);
```

Feature: #70002 - Make it possible to register own icons for file extensions

See Issue #70002

Description

The IconRegistry has been extended with a mapping of file extensions.

Impact

It is now possible to register or overwrite the iconIdentifier for a file extension.

```
$iconRegistry = GeneralUtility::makeInstance(IconRegistry::class);
$iconRegistry->registerFileExtension('log', 'icon-identiifer-for-log-files');
```

Feature: #70078 - Extensions can provide a class map for class loading

See Issue #70078

Description

With the old class loader it was possible for extension authors to register several classes in an ext_autoload.php file.

This possibility was completely removed with introduction of composer class loading. In composer mode, one can fully benefit from composer and its class loading options. However TYPO3 installations in non composer mode (extracted and symlinked archive of sources), lack this functionality completely.

Now it is possible to provide a class map section in either the composer.json file or the ext_emconf.php file. This section will be evaluated and used also in non composer mode.

Example ext_emconf.php file:

```
<?php
$EM_CONF[$_EXTKEY] = array (
  'title' => 'Extension skeleton for TYPO3 7',
  'description' => 'Description for ext',
  'category' => 'Example Extensions',
  'author' => 'Helmut Hummel',
  'author_email' => 'info@helhum.io',
  'author_company' => 'helhum.io',
  'shy' => '',
  'priority' => '',
  'module' => '',
  'state' => 'stable',
  'internal' => '',
  'uploadfolder' => '0',
  'createDirs' => '',
  'modify_tables' => '',
  'clearCacheOnLoad' => 0,
  'lockType' => '',
  'version' => '0.0.1',
  'constraints' =>
  array (
        'depends' =>
        array (
          'typo3' => '7.5.0-7.99.99',
        ),
        'conflicts' =>
        array (
        ),
        'suggests' =>
        array (
        ),
  ),
  'autoload' =>
  array(
        'psr-4' =>
        array(
          'Helhum\\ExtScaffold\\' => 'Classes'
        ),
        'classmap' =>
        array(
          'Resources/PHP/Libs'
        )
  )
);
```

In the example configuration the path `Resources/PHP/Libs` is parsed for PHP files which are automatically added to the class loader.

Impact

Extensions that target TYPO3 6.2 LTS and 7 LTS can now provide a class map in ext_emconf.php which is only evaluated in TYPO3 7 LTS and an ext_autoload.php which is only evaluated in 6.2 LTS for maximum flexibility and compatibility.

Feature: #7098 - Severity-filtering for FlashMessageQueue

See Issue #7098

Description

So far only all messages of the FlashMessageQueue could be fetched and/or removed. With this addition it's possible to do that for a given severity only. The existing functions get an optional $severity-parameter.

Usage:

```
FlashMessageQueue::getAllMessages($severity);
FlashMessageQueue::getAllMessagesAndFlush($severity);
FlashMessageQueue::removeAllFlashMessagesFromSession($severity);
FlashMessageQueue::clear($severity);
```

3.3.3 Deprecation

Deprecation: #55419 - Streamline file conflict mode handling

See Issue #55419

Description

Conflicts in file names and folder names when uploading new files or creating new folders are now handled uniformly with constants within the core. Therefore a new enumeration has been introduced to provide the available values: `\TYPO3\CMS\Core\Resource\DuplicationBehavior`.

Provided constants are:

- `DuplicationBehavior::CANCEL`
- `DuplicationBehavior::REPLACE`
- `DuplicationBehavior::RENAME`

Before this change there were two sets of strings used to define the behavior upon conflicts.

- Set1: `cancel`, `replace` and `changeName`
- Set2: `cancel`, `overrideExistingFile` and `renameNewFile`

As they are redundant they are now represented by a new set of constants:

- `CANCEL`, `REPLACE` and `RENAME`

All usages of strings of the former sets have been replaced with their counterparts from the new set. In the enumeration the former values have been mapped to the new values and marked for deprecation.

Impact

Using `changeName`, `overrideExistingFile` or `renameNewFile` for file conflict handling will result in a deprecation log entry.

Affected Installations

All third party code that calls one of the listed methods with `$conflictMode` either set to `changeName`, `overrideExistingFile` or `renameNewFile`.

Migration

Use the provided enumeration `\TYPO3\CMS\Core\Resource\DuplicationBehavior` instead.

Example

```
$resourceStorage->copyFile($file, $targetFolder, 'target-file-name',␣
↪DuplicationBehavior::RENAME);
```

Deprecation: #66588 - POST Data in selectviewhelper should have higher priority than "value" value

See Issue #66588

Description

Submitted form data has precedence over value arguments.

This adjusts the behavior of all Form ViewHelpers so that any submitted value is redisplayed even if a "value" argument has been specified.

The issue with this, however, was that upon re-display of the form due to property-mapping or validation errors the value argument had precedence over the previously submitted value.

Impact

This is a breaking change if you expect the previous behavior of form ViewHelpers always being pre-populated with the specified value attribute / bound object property even when re-displaying the form upon validation errors.

Besides this the change marks `AbstractFormFieldViewHelper::getValue()` as deprecated. If you call that method in your custom ViewHelpers you should use `AbstractFormFieldViewHelper::getValueAttribute()` instead and call `AbstractFormFieldViewHelper::addAdditionalIdentityPropertiesIfNeeded()` explicitly if the ViewHelper might be bound to (sub)entities.

The default usage of getValueAttribute() did not respect the submitted form data, because not every viewhelper needs this feature. But you can enable the usage of the form data by setting `AbstractFormFieldViewHelper::respectSubmittedDataValue` to TRUE.

Deprecation: #68128 - GeneralUtility slash-related methods

See Issue #68128

Description

The following methods within `GeneralUtility` used to add or remove slashes have been marked as deprecated.

```
GeneralUtility::addSlashesOnArray()
GeneralUtility::stripSlashesOnArray()
GeneralUtility::slashArray()
```

Impact

Any usage of these methods will trigger a deprecation log entry.

Affected Installations

Extensions that call these PHP methods directly.

Migration

Remove usage of these methods from custom extensions.

Deprecation: #68760 - Deprecate class ModuleSettings

See Issue #68760

Description

In older TYPO3 versions `t3lib_modSettings` (as ModuleSettings class was called before) was used to save the current settings of backend modules. This kind of settings is nowadays stored in backend users uc array. For that reason ModuleSettings is now marked for removal in TYPO3 CMS 8.

Impact

Using `ModuleSettings` will trigger a deprecation log entry.

Affected Installations

Any TYPO3 installation with custom extensions using this class and its methods.

Migration

Remove usage of this class from custom extensions.

Deprecation: #68804 - CLI-related constants and methods

See Issue #68804

Description

Logic regarding regular CLI-based scripts with the CLIkey option has been moved into the CliRequestHandler.

Therefore, the following method has been marked as deprecated:

```
BackendUserAuthentication->checkCLIuser()
```

Additionally, the following constants and global parameters have been marked for deprecation in CLI context.

```
const TYPO3_cliKey
const TYPO3_cliInclude
$GLOBALS['MCONF']['name']
$GLOBALS['temp_cliScriptPath']
$GLOBALS['temp_cliKey']
```

The method, constants and variables will be removed in TYPO3 CMS 8.

Impact

Calling `BackendUserAuthentication->checkCLIuser()` directly will now trigger a deprecation log entry.

Affected Installations

Installations with custom entry points in a CLI environment that make use of the method, constants or variables above.

Migration

Use the native `$_SERVER['argv']` or the given `Input` object directly in your code to detect the current CLI-relevant data.

Deprecation: #68860 - Deprecate SelectImage.initEventHandler

See Issue #68860

Description

Removes the calls of `SelectImage.initEventHandler` method. This was limited to WebKit UserAgents and the provided implementation of `require` was wrong, so that the EventListener was not registered at all. Nevertheless the functionality of drag and drop is not broken without the initEventHandler.

Impact

Throws console log with deprecation message.

Affected Installations

All installations calling `SelectImage.initEventHandler`.

Migration

Remove the call of `SelectImage.initEventHandler`.

Deprecation: #69028 - RelationHandler convertPosNeg()

See Issue #69028

Description

Method `convertPosNeg()` of class `TYPO3\CMS\Core\Database\RelationHandler` has been marked as deprecated.

Impact

The method should not be used any longer and will be removed with TYPO3 CMS 8.

Affected Installations

The method is rather internal and relatively unlikely to be used by third party modules. Searching for the string `convertPosNeg` may reveal possible usages.

Migration

The method was used together with the dropped `neg_foreign_table` setting for `TCA select` fields. If this functionality is still needed, the method could be copied over to the third party application that uses it.

Deprecation: #69057 - Deprecate IconUtility and move methods into IconFactory

See Issue #69057

Description

The `IconUtility` class will be removed with TYPO3 CMS 8. All public methods of this class have been marked as deprecated:

- `IconUtility::skinImg()`
- `IconUtility::getIcon()`
- `IconUtility::getSpriteIcon()`
- `IconUtility::getSpriteIconForFile()`
- `IconUtility::getSpriteIconForRecord()`
- `IconUtility::getSpriteIconForResource()`
- `IconUtility::getSpriteIconClasses()`

The PageTSConfig setting `mod.wizards.newContentElement.wizardItems.*.elements.*.icon` also has been marked as deprecated.

The `IconUtilityOverrideResourceIconHookInterface` interface will be removed with TYPO3 CMS 8.

Impact

Any usage of these methods will trigger a deprecation log entry.

Affected Installations

Extensions that call these PHP methods directly. Extensions that register own content elements with an icon for the new content element wizard.

Migration

Use the new `IconFactory` class instead of `IconUtility`.

For content element wizard register your icon in `IconRegistry::registerIcon()` and use the new setting: `mod.wizards.newContentElement.wizardItems.*.elements.*.iconIdentifier`

Deprecation: #69078 - TemplateService::$tempPath

See Issue #69078

Description

The `\TYPO3\CMS\Core\TypoScript\TemplateService::$tempPath` member variable is not used anymore inside the core, therefore it has been marked as deprecated and will be removed with CMS 8.

Affected Installations

Any installation using third party code, which accesses `TemplateService::$tempPath`.

Migration

Remove any reference to `TemplateService::$tempPath`.

Deprecation: #69262 - Move marker substitution functionality to own class

See Issue #69262

Description

The marker substitution functionality has been moved from `core/Classes/Html/HtmlParser.php` to it's own class `core/Classes/Service/MarkerBasedTemplateService.php`

The following methods within HtmlParser have been marked as deprecated.

```
HtmlParser::getSubpart()
HtmlParser::substituteSubpart()
HtmlParser::substituteSubpartArray()
HtmlParser::substituteMarker()
HtmlParser::substituteMarkerArray()
HtmlParser::substituteMarkerAndSubpartArrayRecursive()
```

Impact

Any usage of these methods will trigger a deprecation log entry.

Affected Installations

Extensions that call these PHP methods directly.

Migration

Change the use statement from `TYPO3\CMS\Core\Html\HtmlParser` to `TYPO3\CMS\Core\Service\MarkerBasedTemplateService` and create an instance of this service class. The methods are not static anymore, but named as before.

```
$templateService = GeneralUtility::makeInstance(MarkerBasedTemplateService::class);
$templateService->getSubpart()
$templateService->substituteSubpart()
$templateService->substituteSubpartArray()
$templateService->substituteMarker()
$templateService->substituteMarkerArray()
$templateService->substituteMarkerAndSubpartArrayRecursive()
```

Deprecation: #69269 - Deprecate BackendUtility::getPathType_web_nonweb

See Issue #69269

Description

Method `getPathType_web_nonweb()` of class `TYPO3\CMS\Backend\Utility\BackendUtility` has been marked as deprecated.

Impact

The method should not be used any longer and will be removed with TYPO3 CMS 8.

Affected Installations

The method is unused in the core since at least TYPO3 CMS 6.2. Searching for the string `getPathType_web_nonweb` may reveal possible usages.

Migration

Use path functions from `TYPO3\CMS\Core\Utility\PathUtility`.

Deprecation: #69371 - Form element IMAGEBUTTON

See Issue #69371

Description

Form element `IMAGEBUTTON` of class `TYPO3\CMS\Form\Domain\Model\Element\ImagebuttonElement` has been marked as deprecated. The related attribute `scr` of class `TYPO3\CMS\Form\Domain\Model\Attribute\SrcAttribute` has been marked as deprecated.

Impact

The element `IMAGEBUTTON` should not be used any longer because it is outdated and will be removed with TYPO3 CMS 8.

Affected Installations

All installations which use the form element `IMAGEBUTTON`. All installations which use a form typoscript like this:

```
10 = IMAGEBUTTON
10 {
        label = Image button
        src = /typo3conf/ext/someExt/some/picture.png
        value = value
}
```

Migration

No Migration is planned.

Deprecation: #69401 - Adopt form to support the Extbase/ Fluid MVC stack

See Issue #69401

Description

Form is now based on Extbase/ Fluid for frontend rendering. Therefore all TypoScript based layout settings have been marked as deprecated. Using the following code is not recommended anymore:

```
10 = FORM
10 {
        layout {
                containerWrap = <div><elements /></div>
                elementWrap = <div><element /></div>
        }
}
```

Impact

All `.layout` TypoScript properties should not be used anymore. Backward compatibility algorithms will be removed with TYPO3 CMS 8.

Affected Installations

All installations using `.layout` TypoScript properties.

Migration

Move away from `.layout` TypoScript properties and move to Fluid based templating.

Deprecation: #69535 - Deprecate TYPO3CMSFluidViewHelpersBeButtonsIconViewHelper

See Issue #69535

Description

`\TYPO3\CMS\Fluid\ViewHelpers\Be\Buttons\IconViewHelper` has been marked as deprecated.

Impact

The viewhelper should not be used any longer and will be removed with TYPO3 CMS 8.

Affected Installations

Extensions which use the viewhelper.

Migration

Use the core icon viewhelper `\TYPO3\CMS\Core\ViewHelpers\IconViewHelper` instead.

Example: Instead of `<f:be.buttons.icon icon="apps-pagetree-collapse" />` use `<core:icon identifier="apps-pagetree-collapse" />`

Deprecation: #69561 - Replace sprite icons with IconFactory in ContextMenu

See Issue #69561

Description

The `\TYPO3\CMS\Backend\ContextMenu\ContextMenuAction::$class` member variable is not used anymore inside Core, therefore it has been marked as deprecated and will be removed with CMS 8.

Affected Installations

Any installation using third party code, which accesses `ContextMenuAction::$class`.

Migration

Remove any reference to `ContextMenuAction::$class`.

Deprecation: #69562 - Deprecate helper methods for redundant CSRF protection

See Issue #69562

Description

The method `BackendUtility::getUrlToken` has been introduced as shortcut to protect data manipulating entry points `tce_db.php` `tce_file.php` and `alt_doc.php` from CSRF attacks. These entry points have been replaced with proper modules or routing, which are CSRF protected by default. With this `BackendUtility::getUrlToken` is not needed anymore and therefore has been marked as deprecated.

Impact

Third party code using `BackendUtility::getUrlToken` will trigger deprecation log entries.

Affected Installations

Extensions using the above code.

Migration

These method calls can safely be removed, when generating links to former entry points `tce_db.php` `tce_file.php` and `alt_doc.php` with the API method calls : `BackendUtility::getModuleUrl('tce_db')`, `BackendUtility::getModuleUrl('tce_file')` or `BackendUtility::getModuleUrl('record_edit')`.

Deprecation: #69568 - Various FormEngine related methods

See Issue #69568

Description

The following methods have been marked as deprecated and should not no longer be used:

- `BackendUtility::getExcludeFields()`
- `BackendUtility::getExplicitAuthFieldValues()`
- `BackendUtility::getSystemLanguages()`
- `BackendUtility::getRegisteredFlexForms()`
- `BackendUtility::exec_foreign_table_where_query()`
- `BackendUtility::replaceMarkersInWhereClause()`

Impact

Using those methods will trigger a deprecation log entry.

Affected Installations

The impact is rather low in general since those methods were mostly internal in the first place and only used within FormEngine scope. It is unlikely extensions are affected by this change.

Migration

If still used, extensions should switch to own solutions for those methods.

Deprecation: #69705 - Add unified refresh icon

See Issue #69705

Description

The icon `actions-system-refresh` has been marked as deprecated in `TYPO3\CMS\Core\Imaging\IconRegistry` and will be removed with TYPO3 CMS 8. All requests for `actions-system-refresh` will now show `actions-refresh`.

Impact

Using `IconUtility` or `IconFactory` to fetch the icon `actions-system-refresh` will trigger a deprecation log entry.

Affected Installations

Installations with third party extensions that use the icon `actions-system-refresh`.

Migration

Use the icon `actions-refresh` instead.

Deprecation: #69736 - Select option iconsInOptionTags removed

See Issue #69736

Description

The option `iconsInOptionTags` of TCA `select` fields has been removed due to little support in browsers.

Impact

The usage of this option triggers a deprecation log entry and is automatically removed in TCA tables during bootstrap.

Affected Installations

Any `TCA` configuration using `iconsInOptionTags`.

Migration

Remove usage of this option.

Deprecation: #69754 - Deprecate relative path to extension directory and using filename only in TCA ctrl iconfile

See Issue #69754

Description

- Using relative paths to refer to the extension directory for iconfiles in `TCA['ctrl']['iconfile']` has been marked as deprecated.
- Using filenames only to refer to an iconfile in TCA['ctrl'] has been marked as deprecated.

Impact

- TCA definitions in `TCA['ctrl']['iconfile']` containing `'../typo3conf/ext/'` or calls to `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::extRelPath()` will trigger a deprecation log entry.
- TCA definitions in `TCA['ctrl']['iconfile']` containing a filename only will trigger a deprecation log entry.

Affected Installations

Any installation with extensions defining `TCA['ctrl']['iconfile']` by using `../typo3conf/ext/` or only a filename.

Migration

Relative paths

Use `EXT:` instead of relative path `'../typo3conf/ext/'` or `\TYPO3\CMS\Core\Utility\ExtensionManagementUt` e.g.

```
'ctrl' => array(
        'iconfile' => \TYPO3\CMS\Core\Utility\ExtensionManagementUtility::
↪extRelPath('my_extension') . 'Resources/Public/Icons/image.png'
),
```

has to be migrated to

```
'ctrl' => array(
        'iconfile' => 'EXT:my_extension/Resources/Public/Icons/image.png'
),
```

Filename only

Use a full absolute path or an `EXT:` definition instead of a filename only:

```
'ctrl' => array(
        'iconfile' => '_icon_ftp.gif'
),
```

has to be migrated to

```
'ctrl' => array(
        'iconfile' => 'EXT:t3skin/icons/gfx/i/_icon_ftp.gif'
),
```

or

```
'ctrl' => array(
        // You can use absolute paths (to your web root folder) to the icons but
        // it is discouraged to do so as these icons belong to an extension they
        // should also be stored in this extension
        'iconfile' => '/fileadmin/icons/_icon_ftp.gif'
),
```

Deprecation: #69938 - HIDE_L10N_SIBLINGS FlexFormdisplayCond

See Issue #69938

Description

The flexform `HIDE_L10N_SIBLINGS display` condition has been marked as deprecated and will be removed with CMS 8. The condition could only be used with translation mode `langChildren=1` to only show the field for the default language.

Impact

FlexForms using this condition will show the field separately for each language again.

3.3.4 Important

Important: #67954 - Migrate CTypes text, image and textpic to textmedia

See Issue #67954

Description

EXT:fluid_styled_content simplifies the available CTypes which results in types `text`, `image` and `textpic` being dropped in favor of type `textmedia`. The relation field is changed from `image` to `media_references`.

Impact

When EXT:fluid_styled_content is installed and EXT:css_styled_content isn't installed an "Upgrade Wizard" is available in the Install Tool to migrate all CE of type `text`, `image` or `textpic` to type `textmedia`. Furthermore the relations to field `image` will be adjusted to `media_references` for the migrated CE's.

The frontend rendering has to be adjusted so the new type is rendered after the migration is done. Migration will not happen automatically but has to be triggered manually.

Affected Installations

All installations where EXT:fluid_styled_content is installed and EXT:css_styled_content isn't installed.

Migration

First un-install EXT:css_styled_content and install EXT:fluid_styled_content. After that an "Upgrade Wizard" will be available in the install tool to migrate all existing CE elements of type `text`, `image` or `textpic` to type `textmedia`.

Important: #68128 - PHP Magic Quote Handling removed

See Issue #68128

Description

In PHP versions prior to PHP 5.4 the option of adding slashes (magic quotes) to the superglobals `$_GET` and `$_POST` was causing inconsistent data handling. TYPO3 therefore always added slashes to these variables on every request to streamline the handling of the superglobals. The corresponding methods `GeneralUtility::_GET()`, `GeneralUtility::_GP()` and `GeneralUtility::_POST()` have been changed to not strip off slashes anymore.

Since the PHP option was completely removed, TYPO3 is not adding slashes anymore, and also does not strip the slashes anymore when using the methods within GeneralUtility.

Important: #68758 - Command controllers allowed in subfolders

See Issue #68758

Description

Extbase command controllers can now reside in arbitary subfolders within the `Command` folder. This allows for better grouping and namespacing of commands.

Given a command controller resides in `my_ext/Classes/Command/Hello/WorldCommandController.php`, it can now be invoked like this:

```
typo3/cli_dispatch.sh extbase my_ext:hello:world <arguments>
```

Important: #68917 - Updated jQuery to 2.x

See Issue #68917

Description

With the step from jQuery 1.x to 2.x support for older browsers and "dirty workarounds" have been dropped. Since TYPO3 CMS doesn't support "older" browsers (IE < 9) anymore, jQuery was updated from 1.x to 2.x. For extensions that ran fine with an up-to-date 1.x-version that shouldn't really make much difference. There might however be edge-cases where extension-authors might need to adapt their jQuery scripts for 2.x.

Important: #69084 - Adding Extbase Objects with NOT NULL columns has changed

See Issue #69084

Description

To better support databases that don't silently convert `NULL` values to an empty default value for database columns defined as `NOT NULL` the `insertObject()` method tries to determine the appropriate value for a column.

Extbase object properties that have a value of `NULL` will be skipped when preparing the record to enable the DBMS default value to be used. This behavior has not changed compared to TYPO3 CMS 7.4 but allows proper support for DBMS that are strict about `NOT NULL` columns by defining appropriate default values for properties in the models.

Example database schema:

```
CREATE TABLE tx_blogexample_domain_model_blog (
        title varchar(255) DEFAULT '' NOT NULL
);
```

Example model definition:

```
class Blog extends \TYPO3\CMS\Extbase\DomainObject\AbstractEntity {

        /**
         * The blog's title.
         *
         * @var string
         */
        protected $title = '';

}
```

Important: #69137 - Link Wizard popup width and height fields removed

See Issue #69137

Description

Opening links in popups with width/height definition is a very rare usecase nowadays.

For user convenience and to have less clutter in the UI, the width and height fields have been removed. The editor is not able to select a width and height anymore, it can still be entered manually though.

The RTE option `buttons.link.popupSelector.disabled` has no effect anymore.

Important: #69531 - Remove SpriteManagerIconViewHelper

See Issue #69531

Description

The `\TYPO3\CMS\Backend\ViewHelpers\SpriteManagerIconViewHelper` is not used anymore and has been removed. Use `\TYPO3\CMS\Core\ViewHelpers\IconViewHelper` instead

Important: #69846 - Have eIDs with PSR-7 without ControllerInterface

See Issue #69846

Description

In order to allow the same logic as with the routing and the direct information which method to call, implementing `ControllerInterface` is not mandatory anymore.

Remove the `implements ControllerInterface` instruction in the affected class. The former `processRequest` method may (and should) be changed to:

```
public function anyMethodNameYouLike(ServerRequestInterface $request,␣
↪\Psr\Http\Message\ResponseInterface $response) {
}
```

Please note that the `$response` object is now passed into the method directly, thus you must not create a new object by `$response = GeneralUtility::makeInstance(Response::class);` any more.

The eID_include registration in `ext_localconf.php` must be changed in such case to

```
$GLOBALS['TYPO3_CONF_VARS']['FE']['eID_include']['identifier'] = \Foo\Bar::class .
↪'::anyMethodNameYouLike';
```

Important: #69909 - FAL-based Database Fields moved to integer

See Issue #69909

Description

The FAL-related fields in the database tables `pages`, `pages_language_overlay` and `tt_content` that contained a comma-separated list of paths to files were migrated at 6.2 with the shipped update wizards to IRRE fields. The database fields `pages.media`, `pages_language_overlay.media`, `tt_content.image` and `tt_content.media` now only contain numeric values, which are handled by the DataHandler and the Reference Index, holding the number of references. The database fields are now changed to be only int fields, instead fields of type `text`.

3.4 7.4 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

3.4.1 Breaking Changes

Breaking: #39721 - Prototype.js and Scriptaculous removed

See Issue #39721

Description

The JavaScript libraries prototype.js and scriptaculous have been removed from the TYPO3 Core.

Impact

Any TYPO3 Extension that uses prototype.js or scriptaculous for usage in the Backend or Frontend will not work anymore.

The TypoScript properties `page.javascriptLibs.Prototype` and `page.javascriptLibs.Scriptaculous.*` have been removed and have no effect anymore, leading to not including prototype in websites where this TypoScript option is set. This might lead to broken websites when updating.

The shipped Fluid ViewHelper for backend modules does no longer have the according properties anymore and will throw a fatal error:

```
<f:be.container loadPrototype="false" loadScriptaculous="false"␣
↪scriptaculousModule="someModule,someOtherModule">
```

Using the PageRenderer directly in any module, and calling one of the related methods will result in a fatal error.

```
PageRenderer->setPrototypePath()
PageRenderer->setScriptaculousPath()
PageRenderer->getPrototypePath()
PageRenderer->getScriptaculousPath()
PageRenderer->loadPrototype()
PageRenderer->loadScriptaculous()
```

Including the JavaScript files in a custom extension or custom frontend without using the API will lead to a 404 error when referencing one of the files.

Affected Installations

Instances that use prototype.js or scriptaculous in the Frontend.

Instances with third-party extensions that

require these libraries or set one of the options in the mentioned ViewHelper.

Instances that link to one of the JavaScript files directly.

Instances that use the PageRenderer API directly and use on of the methods above.

Migration

The preferred substitution is jQuery and RequireJS that are loaded by default in any TYPO3 Backend module, if any third-party code is needed based on prototype and/or scriptaculous. Alternatively, it is possible to ship a separate prototype.js and scriptaculous library in a third-party extension if no migration is possible with jQuery.

Breaking: #52705 - Default log configuration is changed

See Issue #52705

Description

FileWriter behavior has changed

The FileWriter of the logging Framework now appends a hash to its default log file which is used when no log file name is provided in the configuration.

The new default log file might now look like this (the hash depends on the current encryption key):

```
typo3temp/logs/typo3_7ac500bce5.log
```

Default configuration has changed

For security reasons we want the default log file of TYPO3 to contain a random hash to make guessing the file name harder.

Therefore the `logFile` configuration is removed for the default `FileWriter` configuration.

Additionally the "deprecated" `FileWriter` configuration is removed because it is not used by the core.

Impact

If the log file configuration is **not** overwritten the TYPO3 default log file will change from `typo3temp/logs/typo3.log` to `typo3temp/logs/typo3_<hash>.log`.

Installations with Extensions making use of the changed / removed log configurations might break.

Affected Installations

All instances that expect the default log file to be `typo3temp/logs/typo3.log` for some reason.

All instances that expect the `logFile` configuration to be present in the `writerConfiguration` for some reason.

All instances that use the "deprecated" log configuration in their Extensions.

All instances that use Extensions which extend the FileWriter and access the `$defaultLogFile` class property which is replaced by `$defaultLogFileTemplate` and the `getDefaultLogFileName()` method.

Migration

Adjust the log configuration according to your needs in your `LocalConfiguration.php`.

Adjust any Extension code affected by the changes if needed.

Breaking: #55759 - HTML in link titles not working anymore

See Issue #55759

Description

By introducing proper handling of double quotes in link titles (TypoLink fields) the processing of the link title is adjusted. Escaping will be done automatically now.

Impact

Existing link titles, which contain HTML escape sequences, will not be shown correctly anymore in Frontend.

Example: A link title `Some "special" title` will be output as `Some "special" title`

Affected Installations

Any installation using links with titles containing HTML escape sequences like `"` or `>`

Migration

Change the affected link titles to contain the plain characters, the correct encoding will be taken care of automatically.

Example: `Some "special" title`

If you need to encode a TypoLink manually in code, use the `TypoLinkCodecService` class, which provides a convenient way to encode a TypoLink from its fragments.

Breaking: #56133 - New BE user permission "Files: replace"

See Issue #56133

Description

A new feature was introduced to replace files in the file list. For this feature an new permission was introduce "Files: replace". This permission is now also checked when a BE user uploads a file with the same name.

Impact

BE users need the permission "Files: replace" before they are allowed to replace a file by uploading a file with the same name.

Affected Installations

All installations.

Migration

An upgrade wizard was added to set this permission for all BE users that already are allowed to write files as this was the old permissions check.

Breaking: #56951 - Remove unused methods in PagePositionMap

See Issue #56951

Description

Remove unused methods in PagePositionMap

Impact

A fatal error will be thrown if one of the removed methods is used. The removed methods are:

`insertQuadLines` `JSimgFunc`

Affected Installations

Installations that use one of the removed methods.

Migration

Use proper styling for a tree list.

Breaking: #63838 - Changed OpcodeCacheUtility being a service class

See Issue #63838

Description

The `OpcodeCacheUtility` has been migrated to a service class called `OpcodeCacheService`, all methods are not static anymore.

Impact

Calling `OpcodeCacheUtility` will throw a fatal error.

Affected Installations

All third-party extensions using the utility class will be affected.

Migration

Create an instance of `OpcodeCacheService` and call its method by the object operator `->`.

Example:

```
GeneralUtility::makeInstance(OpcodeCacheService::class)->clearAllActive(
↪$cacheEntryPathAndFilename);
```

Breaking: #64200 - Custom [cObject].cache.* TypoScript evaluation

See Issue #64200

Description

The `stdWrap.cache.` property is now available as first-class function to all content objects. This skips the rendering even for content objects that evaluate `stdWrap` after rendering (e.g. `COA`).

Additionally, stdWrap support is added to key, lifetime and tags.

Impact

If you've previously used the `cache.` property in your custom cObject, this will now fail, because `cache.` is unset to avoid double caching.

`stdWrap.cache` continues to exist and can be used as before. However the top level `stdWrap` of certain cObjects (e.g. `TEXT` cObject) will not evaluate `cache.` as part of `stdWrap`, but before starting the rendering of the cObject. In conjunction the storing will happen after the `stdWrap` processing right before the content is returned.

Top level `cache.` will not evaluate the hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tslib/class.tsl` any more.

Affected Installations

All installations with custom `cObject` implementations which make use of the `cache.` property.

Installations that purposely rely on the content object being evaluated before the cache is tried.

Installations that rely on the order of the `cache.` evaluation.

Installations that make use of the hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tslib/class.tslib_co` on top level `cache.`.

Migration

Rename your property or rely on the Core implementation.

If you need `cache.` being evaluated as part of `stdWrap`, please move it down one level by writing `stdWrap.cache` instead.

If you used the hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tslib/class.tslib_content.php']` please use `stdWrap` and the available hooks inside `stdWrap` to achieve your goal.

Breaking: #64714 - Catch exceptions for inaccessible storages

See Issue #64714

Description

Changed `\TYPO3\CMS\Core\Resource\Exception\ResourcePermissionsUnavailableException` to extend from `\TYPO3\CMS\Core\Resource\Exception` instead of `\RuntimeExtension`

Impact

If a call to `\TYPO3\CMS\Core\Resource\Driver\LocalDriver->getPermissions()` throws an exception and your extension catches `\RuntimeExtension` it breaks on permission read errors.

Migration

To write a compatible extension you can do following in your catch block:

```
} catch(\RuntimeException $e) {
        // Do exception handling
} catch(\TYPO3\CMS\Core\Resource\Exception\ResourcePermissionsUnavailableException
↪$e) {
        // Do same exception handling
}
```

Breaking: #65305 - DriverInterface has been extended

See Issue #65305

Description

The `getFolderInFolder()` and `getFileInFolder()` functions have been added to `DriverInterface`.

Impact

Any FAL driver extension will stop working due to the change in the interface.

Affected Installations

Any installation with a custom FAL driver, like WebDAV or Dropbox.

Migration

The functions `getFolderInFolder()` and `getFileInFolder()` must be added to the custom FAL driver. A non-hierarchical driver needs to throw a "not implemented" exception when calling these functions.

Breaking: #66602 - Check jumpUrl referer has been removed

See Issue #66602

Description

The following method has been removed:

```
TypoScriptFrontendController::checkJumpUrlReferer()
```

Impact

Calls to this method will result in a fatal error.

Affected Installations

Instances with third-party extensions calling this method.

Breaking: #67545 - PreFileAdd signal behaviour changed

See Issue #67545

Description

In order to check whether an uploaded file exists already before uploading or to determine user preferences about present files only when needed, the final name of the file is needed. As the final name can be altered by the `PreFileAdd` signal, which originally always received the temporary uploaded file path as parameter, the signal will now receive an empty string in `$sourceFilePath`.

Impact

All `PreFileAdd` slot methods that depend on the `$sourceFilePath` param must be adapted to handle the new empty string value.

Affected Installations

All extensions that use the `PreFileAdd` signal and depend on the `$sourceFilePath` param.

Breaking: #67546 - Cleanup Flash message rendering in FlashMessageQueue

See Issue #67546

Description

The rendering of flash messages has changed when using the view helper. Now the rendering output of `\TYPO3\CMS\Core\Messaging\FlashMessageQueue::renderFlashMessages()` is adapted to being exactly the same.

Impact

Extensions using the rendered output of `\TYPO3\CMS\Core\Messaging\FlashMessageQueue::renderFlashMessag` and in addition using HTML tags in flash messages for styling purposes will get their HTML flash message output properly HTML encoded. Thus the HTML tags will be visible in the rendered flash message output.

Since `\TYPO3\CMS\Backend\Template\DocumentTemplate` also uses this rendering type, modules using this class will also be affected.

Affected Installations

All extensions that use modules with `\TYPO3\CMS\Backend\Template\DocumentTemplate` or are using `\TYPO3\CMS\Core\Messaging\FlashMessageQueue::renderFlashMessages()` directly.

Migration

Remove all HTML from flash messages.

Breaking: #67557 - Language file of Opendocs was moved

See Issue #67557

Description

The language file `locallang_opendocs.xlf` of EXT:opendocs has been moved to `Resources/Private/locallang.xlf`.

Impact

Inclusion of the file via `$this->getLanguageService()->includeLLFile()` or usage in `<f:translate key="LLL:EXT:opendocs/locallang_opendocs.xlf:foobar" />`. will fail.

Affected Installations

Every extension relying on the existence of `locallang_opendocs.xlf` will be affected.

Migration

Use `Resources/Private/Language/locallang.xlf`, if required.

Breaking: #67565 - Deprecated backend related methods removed

See Issue #67565

Description

The following deprecated methods have been removed:

```
DocumentTemplate::formWidthText()
PageLayoutView::getBackendLayoutConfiguration()
PageLayoutView::wordWrapper()
```

Impact

Calls to these methods will result in a fatal error.

Affected Installations

Instances with third-party extensions calling one of these methods.

Breaking: #67577 - rte_enabled and flag handling

See Issue #67577

Description

Content elements of type `text` and `text with image` contained a field "RTE enabled" that could be unchecked to disable the rich text editor. This field has been removed together with the `TCA` richtext `flag` handling.

Impact

The field is removed from database and the flag information is lost.

Affected Installations

All instances will no longer show the "RTE enabled" field below `text` and `text with image` content elements below the text field, the `TCA flag` is obsolete, see example below.

Migration

A typical rich text configuration in `TCA` looked like:

```
'content' => array(
        'label' => 'LLL:EXT:lang/locallang_general.xlf:LGL.text',
        'config' => array(
                'type' => 'text',
                'cols' => '48',
                'rows' => '5',
                'wizards' => array(
                        'RTE' => array(
                                ...
```

```
                        )
                )
        ),
        'defaultExtras' => 'richtext:rte_transform[flag=otherField|mode=ts_css]',
),
```

With this configuration RTE was only rendered if `otherField` had the value 1. This flag is obsolete now:

```
'defaultExtras' => 'richtext:rte_transform[mode=ts_css]',
```

Breaking: #67646 - PHP library inclusion in frontend removed

See Issue #67646

Description

The PHP library inclusion into the TYPO3 Frontend has been removed without substitution. Previously it was used to include plain PHP scripts during the Frontend request.

The method `PageGenerator::getIncFiles()` has been removed.

Impact

The TypoScript options `config.includeLibrary` and `config.includeLibs` have no effect anymore. Any calls to `PageGenerator::getIncFiles()` will result in a fatal error.

Affected Installations

Any installation using the TypoScript options named above. Any third party code using the method named above.

Migration

Use hooks during the Frontend set up to execute custom PHP code.

Breaking: #67654 - Remove $GLOBALS[FILEICONS] functionality

See Issue #67654

Description

The global variable `$GLOBALS['FILEICONS']` was in use for displaying icons before the sprite icons for files were introduced in TYPO3 v4.4.

The `$FILEICONS` has been removed completely as well as the function call `BackendUtility::getFileIcon()`.

Impact

Any usage on `$GLOBALS['FILEICONS']` will have no effect anymore.

Any calls on `BackendUtility::getFileIcon()` will result in a fatal error.

Affected Installations

Instances that populate or make use of `$GLOBALS['FILEICONS']` or installations with extensions calling `BackendUtility::getFileIcon()` directly.

Migration

Use sprite icons via `IconUtility::getSpriteIconForFile()`.

Breaking: #67749 - Force class auto loading for various hooks

See Issue #67749

Description

Some hook registrations now rely on class auto loading.

Impact

Hooks may not be called any longer if class auto loader can not find the class.

Affected Installations

Instances using

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tce']['formevals']
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['setup']['accessLevelCheck']
```

Migration

Make sure the hook classes are found with the casual auto loading mechanism that is also used for all other PHP classes. The hook registration can be simplified to an empty value, example:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tce']['formevals
↪']['\TYPO3\CMS\Saltedpasswords\Evaluation\FrontendEvaluator::class] = '';
```

Breaking: #67753 - Drop "Show secondary options"

See Issue #67753

Description

When editing records, the checkbox at the bottom "Show secondary options (palettes)" has been dropped, palettes are now always shown and the collapse buttons are no longer rendered.

Impact

PageTSconfig

Setting `options.enableShowPalettes` has no effect anymore and can be removed from `PageTSconfig`.

TCA

Setting `canNotCollapse` in `ctrl` and `palettes` section are obsolete and can be dropped:

```
$GLOBALS['TCA']['aTable']['ctrl']['canNotCollapse'] = 1; // Obsolete
$GLOBALS['TCA']['aTable']['palettes']['aPaletteName']['canNotCollapse'] = 1; //␣
↪Obsolete
```

PHP

The following method has been dropped. If an extension calls it, a PHP fatal error will be thrown. This was an internal method and external usage is unlikely:

```
\TYPO3\CMS\Backend\Controller\EditDocumentController->functionMenus()
```

The following properties have been dropped, calling those may trigger a PHP warning level error, external usage is unlikely:

```
\TYPO3\CMS\Backend\Controller\EditDocumentController->MOD_MENU
\TYPO3\CMS\Backend\Controller\EditDocumentController->MOD_SETTINGS
\TYPO3\CMS\Backend\Form\FormEngine->palettesCollapsed
```

Affected Installations

In the rare case that an extension uses one of the above methods or properties, a fatal PHP error may be triggered.

Migration

The above properties can be dropped, the `PageTS` and `TCA` settings have no effect anymore.

Breaking: #67792 - Class aliases of Indexed Search removed

See Issue #67792

Description

The class aliases of the Indexed Search extension are removed.

Impact

These class aliases are not available anymore:

- `tx_indexedsearch`
- `Tx_IndexedSearch_Controller_SearchController`

Affected Installations

Any third party code using the old class aliases.

Migration

Use the new class names:

- `\TYPO3\CMS\IndexedSearch\Controller\SearchFormController`
- `\TYPO3\CMS\IndexedSearch\Controller\SearchController`

Breaking: #67811 - Rte API

See Issue #67811

Description

The RTE implementation was based on the main classes `\TYPO3\CMS\Backend\Rte\AbstractRte`, `\TYPO3\CMS\Rtehtmlarea\RteHtmlAreaBase` and `\TYPO3\CMS\Rtehtmlarea\RteHtmlAreaApi`. These three main API were removed or changed method signatures and internal method calls.

The functionality to render RTE standalone and out of a context of `FormEngine` was dropped.

Impact

Main API changes

- Method `TYPO3\CMS\Backend\Utility\BackendUtility::RTEgetObj()` is deprecated and no longer used. `FormEngine` now creates a `RichTextElement` with `NodeFactory` and `makeInstance()`, the created object is not a singleton but a prototype.
- With the deprecation of `RTEgetObj` method `transformContent` from `AbstractRte` has been inlined to `DataHandler`.
- Method `isAvailable` from `AbstractRte` has been dropped. Every valid browser and browser version for TYPO3 CMS 7 can render the default richtext editor. Custom checks may be implement via `NodeResolverInterface` in `FormEngine`.
- Property `RTE_errors` in `TYPO3\CMS\Core\Authentication\BackendUserAuthentication` has been dropped along with the `RTEgetObj()` deprecation.

RTE registration

- Different richtext implementations can no longer register in `$GLOBALS['TYPO3_CONF_VARS']['BE']['RTE_reg']`. Instead, registration must be done in `FormEngine` via `NodeFactory` API, the method `drawRTE` has been dropped.
- Transformations are not available via `AbstractRte` anymore, hooks within `RteHtmlParser` can be used for custom transformations.

PHP classes

- `\TYPO3\CMS\Backend\Rte\AbstractRte` has been dropped.
- `\TYPO3\CMS\Rtehtmlarea\RteHtmlAreaBase` has been dropped and its functionality was moved to `\TYPO3\CMS\Rtehtmlarea\Form\Element\RichtextElement`. All methods and properties except the main entry method `render()` used by `FormEngine` are protected.
- `\TYPO3\CMS\Rtehtmlarea\RteHtmlAreaApi` has been refactored. Method `main()` receives a configuration array instead of an instance of the parent object. Some methods were dropped and are no longer called.

RTE Plugin Configuration

- Parameter `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['anExtensionKey']['plugins']['aPluginN` was dropped, plugin property `relativePathToSkin` is no longer evaluated.
- A couple of helper methods were added to `RteHtmlAreaApi`
- This API may get further changes in the future.

Affected Installations

Extensions that extend one of the above mentioned extensions or API.

Migration

Adapt the code using these methods.

Breaking: #67815 - Remove tceforms.js because we don't need it anymore

See Issue #67815

Description

Since the value slider is based on bootstrap, the last code from `tceforms.js` is not needed anymore.

Impact

All instances which include `sysext/backend/Resources/Public/JavaScript/tceforms.js` will produce a 404 Not Found error.

Affected Installations

All instances which include `sysext/backend/Resources/Public/JavaScript/tceforms.js`.

Migration

Remove all references to the file.

Breaking: #67824 - typo3/ext folder removed

See Issue #67824

Description

The folder `typo3/ext` does not exist in the default core package anymore. The functionality to have global extensions in this directory is not touched.

Impact

In case global extensions are moved to this directory during deployment or rollout, the directory must be created before, now.

Affected Installations

Instances that use global extensions within `typo3/ext`.

Migration

Create directory `typo3/ext` before moving extensions into this folder.

Breaking: #67825 - Remove colorpicker options "dim" and "tableStyle"

See Issue #67825

Description

`TCA` colorpicker options "dim" and "tableStyle" have been removed.

Impact

The TCA options won't have any effect anymore.

Affected Installations

Any extension that has a colorpicker wizard configured in `TCA` and uses `dim` or `tableStyle` options is effected.

Migration

Both options can de safely removed.

Breaking: #67890 - Redesign FluidTemplateDataProcessorInterface to DataProcessorInterface

See Issue #67890

Description

The `FluidTemplateDataProcessorInterface` introduced with #66907 has been refactored to `DataProcessorInterface`.

This decouples it from the Fluid StandaloneView and makes the ContentObjectRenderer available in the process method so the different DataProcessor classes do no have to initiate it on their own. Instead of manipulating the `$data` property of the `ContentObjectRenderer` a new key/value store can be filled/manipulated by the different dataProcessor classes.

The new interface expects the following `process()` method:

```
/**
 * Process content object data
 *
 * @param ContentObjectRenderer $cObj The data of the content element or page
 * @param array $processorConfiguration The configuration of this processor
 * @param array $contentObjectConfiguration The configuration of Content Object
 * @param array $processedData Key/value store of processed data (e.g. to be␣
↪passed to a Fluid View)
 * @return array the processed data as key/value store
 */
public function process(
        ContentObjectRenderer $cObj,
        array $contentObjectConfiguration,
        array $processorConfiguration,
        array $processedData
);
```

Impact

This will break all frontend rendering for TYPO3 7.3 installations that use `FLUIDTEMPLATE.dataProcessing`.

Affected Installations

All TYPO3 7.3 installations that already use the new `FLUIDTEMPLATE.dataProcessing` option.

Migration

Change the interface of all DataProcessor classes from `FluidTemplateDataProcessorInterface` to the new `DataProcessorInterface` and adjust the `process()` method to match the new parameters and make sure it returns the processed data as the processed data.

Breaking: #67932 - felogin template has been changed for RSA encryption

See Issue #67932

Description

Due to the introduction of the new rsaauth API the felogin template has been changed.

A new HTML data-attribute had to be added to the password field in order to enable the RSA encryption Javascript code.

Impact

If rsaauth is enabled and the template is not updated, no login is possible.

Affected Installations

Any installation using a custom felogin template and having rsaauth enabled for frontend.

Migration

The template has to be adjusted and a `data-rsa-encryption` attribute has to be added to the password field in `<!--###TEMPLATE_LOGIN###-->`

The field definition in your template has to be updated to like this:

```
<input type="password" id="pass" name="pass" value="" data-rsa-encryption="" />
```

Breaking: #67946 - LowLevel Cleaner Scripts Removed

See Issue #67946

Description

The shell scripts for checking and cleaning a TYPO3 installation for its integrity have been removed from the Core. They were previously located under `typo3/cleaner_check.sh` and `typo3/cleaner_fix.sh`. The contents have been moved to the documentation for EXT:lowlevel in the respective `README.rst`.

Impact

Any regular system jobs that execute these scripts will exit with the script not found anymore.

Affected Installations

Any installation that uses the shell scripts above from the command line directly.

Migration

If such a list is needed, create the files manually again in your system (outside your document root). The code can be found inside `EXT:lowlevel/README.rst`.

Breaking: #67987 - Removed entry script handling

See Issue #67987

Description

Definition and handling of constants `TYPO3_MOD_PATH` and `PATH_typo3_mod` have been dropped. These constants were important for modules that were still not called through the `mod.php _DISPATCH` system that was introduced in TYPO3 CMS version 4.2. It is required to route modules through `typo3/mod.php` from now on in case the module relies on the definition of those constants.

The following old entry scripts were removed:

- typo3/sysext/cms/layout/db_layout.php, use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl` to link to the module
- typo3/sysext/cms/layout/db_new_content_el.php, use `\TYPO3\CMS\Backend\Utility\BackendUtility::getMo` to link to the module

Impact

Path resolving may fail for scripts that rely on entry points different from `typo3/mod.php` Constants `TYPO3_MOD_PATH` and `PATH_typo3_mod` are not defined anymore and scripts may throw a PHP warning if they try to access these constants.

Affected Installations

Installations may fail if linking to modules that use an entry script with a `conf.php` file that do not use `$MCONF['script'] = '_DISPATCH';`. Those modules must be adapted to the `mod.php` entry point and may need adaption of further references that are defined relative to the entry script.

Searching for extensions with backend modules that define `TYPO3_MOD_PATH` is a good start to find unwanted usages.

Breaking: #68001 - Removed ExtJS Core and ExtJS Adapters

See Issue #68001

Description

The functionality of "ExtCore", which acts as a slim ExtJS base functionality similar to what libraries like underscore.js or jQuery do, but is just dated, has been completely removed from the TYPO3 Core.

The custom adapters which have previously been shipped with ExtJS to allow jQuery functionality to be used with ExtJS underneath have been removed without substitution. The adapters are not compatible with supported jQuery, prototype.js or YUI versions anymore and their usages have been slower than ExtJS's base library natively.

Impact

Using TypoScript options `page.javascriptLibs.ExtCore = 1`, `page.javascriptLibs.ExtCore.debug = 1` and `page.javascriptLibs.ExtJs.adapter` have no effect anymore.

Using `<f:be.container>` ViewHelpers in a custom Backend module, setting the extJsAdapter, property will result in a fatal error.

Calling `$pageRenderer->loadExtJS()` with a custom third parameter will have no effect anymore.

Calling the methods `loadExtCore()`, `enableExtCoreDebug()`, `getExtCorePath()` and `setExtCorePath()` of `PageRenderer` will result in fatal errors.

Affected Installations

Instances that use ExtCore in the TYPO3 Frontend

Extensions that use `<f:be.container>` with an ExtJS Adapter

Extensions that use the `PageRenderer` object directly to load custom ExtJS or ExtCore.

Migration

Use alternatives for ExtCore or adapters in custom extensions.

Breaking: #68010 - T3Editor - Event callbacks for codecompletion have changed

See Issue #68010

Description

Due to the rewrite of T3Editor to jQuery, the event callbacks for codecompletion have changed.

Impact

Plugins for codecompletion written in Prototype will not work anymore.

Affected Installations

Every third-party extension providing a T3Editor plugin extending the codecompletion.

Migration

Port the plugin to an AMD module. The event callbacks are now part of the module object and not a standalone function anymore.

Example code:

```
CoolPlugin.afterKeyDown = function(currWordObj, compResult) {
        CoolPlugin.somethingFunky(currWordObj, compResult);
};
```

Breaking: #68010 - T3Editor - Plugin registration for codecompletion has changed

See Issue #68010

Description

Due to the rewrite of T3Editor to jQuery, the plugin registration for codecompletion has changed.

Impact

Plugins for codecompletion written in Prototype will not work anymore.

Affected Installations

Every third-party extension providing a T3Editor plugin extending the codecompletion.

Migration

Port the plugin to an AMD module. The plugin must have an `init` method with a configuration object as only parameter. Every parameter that was passed to the old Prototype function must be in that configuration object. Please see the example code below or consult `EXT:t3editor/Resources/Public/JavaScript/Plugins/CodeCompletion/DescriptionPlugin.js`.

Example code:

```
define('Awesome/Extension/Plugins/CodeCompletion/CoolPlugin', [
        'jquery',
        'TYPO3/CMS/T3editor/Plugins/CodeCompletion/TsRef',
        'TYPO3/CMS/T3editor/Plugins/CodeCompletion/TsParser'
], function ($, TsRef, TsParser) {
        var CoolPlugin = {
                codeCompleteBox: null,
                codemirror: null
        };

        CoolPlugin.init = function(configuration) {
                DescriptionPlugin.codeCompleteBox = configuration.codeCompleteBox;
                DescriptionPlugin.codemirror = configuration.codemirror;

                DescriptionPlugin.codeCompleteBox.parent().append($('<div />',
↪{class: 'foomatic'}));
        };

        return CoolPlugin;
});
```

Breaking: #68020 - Dropped DisableBigButtons

See Issue #68020

Description

The TSconfig option `mod.web_layout.disableBigButtons` has been dropped, setting it to 0 has no effect anymore.

Impact

The option is ignored and instances using this will not get the buttons rendered in page module anymore.

These methods have been removed, but it is very unlikely an extension is affected:

- `TYPO3\CMS\Backend\View\PageLayoutView->linkRTEbutton()`
- `TYPO3\CMS\Backend\View\PageLayoutView->isRTEforField()`
- `TYPO3\CMS\Backend\View\PageLayoutView->getSpecConfForField()`

Affected Installations

Instances that had User / Page TSconfig with this option may have a slightly different Web -> Page view.

Breaking: #68092 - TCA: Remove wizard hideParent and _HIDDENFIELD

See Issue #68092

Description

Wizards defined in `TCA` for display in `FormEngine` allowed to hide the "parent" field with the configuration options `_HIDDENFIELD` on main wizard level, and with the `hideParent` option for single wizards.

Both options have been dropped.

Impact

The configuration options have no effect anymore, the main field will show up.

Affected Installations

A search through the TER code showed not a single extension that used the above options. A 3rd party extension is affected if a `TCA` column configuration is used like:

```
'aField' => array(
        'config' => array(
                ...
                'wizards' => array(
                        '_HIDDENFIELD' => TRUE,
                        'aWizard' => array(
                                'hideParent' => array(
                                        ...
                                ),
                        ),
                ),
        ),
),
```

Migration

Wizards can not trigger that a main field is not rendered anymore. If this kind of functionality is needed, it is recommended to register an own `renderType` in the `NodeFactory` for this type of field instead to route the element rendering to an own class.

Breaking: #68116 - Drop RTE.userLinks functionality

See Issue #68116

Description

Drop RTE.userLinks functionality from the ElementBrowser. The option was broken since 6.0 and has been removed now.

Impact

The special option won't show up in the ElementBrowser anymore.

Affected Installations

All installations which use the option `RTE.userLinks`

Migration

Use the newly added Tabbing API to add your custom link selection tab.

Breaking: #68131 - Streamline error and exception handling

See Issue #68131

Description

It is not possible any more to change error and exception handling configuration in an ext_localconf.php of an extension.

Impact

Error or exception handling configuration overridden in ext_localonf.php files will not work any more.

Affected Installations

All installations with extension that set error or exception handling configuration in ext_localconf.php files.

Migration

Configure error and exception handling in LocalConfiguration.php or AdditionalConfiguration.php

Breaking: #68150 - $GLOBALS['CLIENT']

See Issue #68150

Description

The initialization of the `$GLOBALS['CLIENT']` variable has been dropped.

Impact

Extensions that use `$GLOBALS['CLIENT']` will cause a PHP notice or may not function properly any more.

Affected Installations

Installations with extensions that use `$GLOBALS['CLIENT']` are affected.

Migration

Extensions can still use `GeneralUtility::clientInfo()` API to retrieve the same information.

Breaking: #68178 - Drop $GLOBALS['TYPO3_CONF_VARS']['SYS']['form_enctype']

See Issue #68178

Description

Setting `$GLOBALS['TYPO3_CONF_VARS']['SYS']['form_enctype']` has been dropped without replacement.

Impact

Extensions that used this setting in forms may end up with an empty `enctype` attribute in `HTML form` fields.

Affected Installations

Extensions that rely on this variable being set.

Migration

Substitute the variable access with `multipart/form-data`.

Breaking: #68186 - Adjusted and removed methods in eID area

See Issue #68186

Description

eID scripts now use Request and Response objects to retrieve and output data.

Due to adjustments of the Core eID scripts a few methods have been adjusted.

`ExtDirectEidController::routeAction` is now protected and has a changed signature.

The following methods have been removed:

- `ExtDirectEidController::actionIsAllowed()`
- `ExtDirectEidController::render()`
- `EidUtility::isEidRequest()`
- `EidUtility::getEidScriptPath()`

Additionally calling an non-existent eID key will result in a fatal error.

Impact

All third party code using those methods will cause a fatal PHP error.

Affected Installations

All installations using third party code accessing one of the adjusted (or removed) methods.

Migration

No replacement for the mentioned methods is provided. Consider migrating your eID scripts to the new PSR-7 compliant model.

Breaking: #68193 - Drop removeLoginpagesWithContentHash from Indexer.php

See Issue #68193

Description

The method `\TYPO3\CMS\IndexedSearch\Indexer::removeLoginpagesWithContentHash()` was not used within the core since 6.0 and has been removed.

Impact

Calling `\TYPO3\CMS\IndexedSearch\Indexer::removeLoginpagesWithContentHash()` will throw a fatal error.

Affected Installations

All installations with third party code using the mentioned method.

Migration

No migration is available.

Breaking: #68206 - Remove usage of typolist and typohead in RTE

See Issue #68206

Description

The transformation for the tags `typolist` and `typohead` have been removed from the RTE. The option and method `internalizeFontTags()` from RteHtmlParser have been removed.

Impact

The tags are not processed anymore by the RteHtmlParser. Fonts are not internalized anymore.

Affected Installations

All installations using the custom tags `typolist` and `typohead`. All installations that use the method `internalizeFontTags()` will throw an fatal error.

Migration

Substitute the tags by a tag `ul``or `header`. Remove any usage of `internalizeFontTags()`

Breaking: #68243 - Move not used FrontendDocumentTemplate

See Issue #68243

Description

Move unused FrontendDocumentTemplate to ext:compatibility6.

Impact

Installations still using FrontendDocumentTemplate require ext:compatibility6 to be installed.

Affected Installations

Installations still using FrontendDocumentTemplate.

Migration

Install ext:compatibility6 or adapt the code to not use the FrontendDocumentTemplate functionality.

Adapting the code is highly recommended.

Breaking: #68276 - Remove ExtJS Quicktips if possible

See Issue #68276

Description

The method `PageRenderer::enableExtJSQuickTips()`, which was used to enable ExtJS quicktips, has been removed. In some places like RTE or workspaces the Quicktips are still in use, but will be removed as soon as possible.

Impact

All calls to the PHP method `PageRenderer::enableExtJSQuickTips()` will throw a fatal error.

Affected Installations

Instances which make use of `PageRenderer::enableExtJSQuickTips()`.

Migration

No migration, use bootstrap tooltips, which work out of the box as alternative. Simple add `data-toggle="tooltip"` and `data-title="your tooltip"` to any element you want.

Example

```
<a href="#" data-toggle="tooltip" data-title="My very nice title">My Link</a>
```

Breaking: #68321 - Move language and images in rtehtmlarea

See Issue #68321

Description

All language files and images have been moved.

Impact

Labels and images may be not displayed if TypoScript from the sample files is used.

Affected Installations

All installations that use the TypoScript settings from the provided sample files.

Migration

Adjust the path to language and image files according the sample TypoScript files provided by the extension.

3.4.2 Features

Feature: #20194 - Configuration for displaying the "Save & View" button

See Issue #20194

Description

The "Save & View" button is configurable by TSConfig "TCEMAIN.preview.disableButtonForDokType" (CSV of "doktype" IDs) to disable the button for custom page "doktypes". The default value is set in the PHP implementation: "254, 255, 199" (Storage Folder, Recycler and Menu Seperator)

Impact

The "Save & View" button is no longer displayed in folders and recycler pages.

Feature: #22175 - Support IEC/SI units in file size formatting

See Issue #22175

Description

Size formatting supports two keywords additionally to the list of labels: - iec: uses the Ki, Mi, etc prefixes and binary base (power of two, 1024) - si: uses the k, M, etc prefixes and decimal base (power of ten, 1000)

The default formatting is set to "iec" base size calculations on the same base as before. The fractional part, when present, is changed to two numbers instead of only one.

The list of labels is still supported and defaults to using binary base. It is also possible to explicitly choose between binary or decimal base when it is used.

Impact

Default formatted output of file sizes changes, see example below.

TypoScript `stdWrap` property `bytes` defaults to a different label set. `bytes.labels = iec`, a specifically defined label string with pipe separated label keywords is obsolete, but can still be used if required. The keyword `iec` resolves to ' | Ki| Mi| Gi| Ti| Pi| Ei| Zi| Yi' (binary base) and `si` resolves to ' | k| M| G| T| P| E| Z| Y' (based on ten).

Example

```
echo GeneralUtility::formatSize(85123);
// => Before "83.1 K"
// => Now "83.13 Ki"
```

Feature: #33071 - Add the http header "Content-Language" when rendering a page

See Issue #33071

Description

By default a header "Content-language: XX" is sent where "XX" is the iso code of the sys_language_content if that is properly defined by the sys_language record representing the sys_language_uid. The "config.disableLanguageHeader" will disable that.

Impact

By default in new and existing installations a header "Content-language: XX" is sent where "XX" is the iso code of the sys_language_content if that is properly defined by the sys_language record representing the sys_language_uid. You must set "config.disableLanguageHeader" to disable that and get previous behavior (no header).

Feature: #34922 - Allow .ts file extension for static TypoScript templates

See Issue #34922

Description

Only these TypoScript file names were allowed:

- constants.txt
- setup.txt

- include_static.txt
- include_static_files.txt

The ts file extension has been allowed for constants and setup and is prioritised over txt.

Impact

There is a little performance impact when loading the TypoScript from scratch like in the backend and frontend without cache as the new file extension is always tested.

Feature: #43984 - Add stdWrap functionality to TreatIdAsReference TypoScript

See Issue #43984

Description

The new TypoScript property "treatIdAsReference" for the getImgResource object can now be used with stdWrap properties.

Feature: #45725 - Added recursive option to folder based file collections

See Issue #45725

Description

Folder based file collections have now an option to fetch all files recursively for the given folder. The option is also available in the TypoScript Object `FILES`.

Usage:

```
filecollection = FILES
filecollection {
        folders = 1:images/
        folders.recursive = 1

        renderObj = IMAGE
        renderObj {
                file.import.data = file:current:uid
        }
}
```

Feature: #48947 - Avatars for backend users

See Issue #48947

Description

To improve the user experience in collaborative content editing, avatars for backend users have been introduced. Avatars will be either user defined by uploading a custom image or generated by the system.

Impact

The avatars will be shown in the topbar, users list and other places.

Feature: #56133 - Replace file feature for fal file list

See Issue #56133

Description

Now its possible to replace files for a specific record at the extended view in the FAL record list.

Impact

Provides a new button "replace" at the extended view in FAL equal to DAM. It's possible to replace a file * with a new one -> old file will be overwritten; identifier of the file object will be kept * with a new one -> old file will be deleted; identifier of the file object will be changed to the new filename

The file replacing also respects unique file names.

Feature: #56644 - Hook for InlineRecordContainer::checkAccess()

See Issue #56644

Description

Hook to post-process `InlineRecordContainer::checkAccess` result. `InlineRecordContainer::checkAccess` is used to check the access to related inline records. It's implemented in the same way as the hook $GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['typo3/alt_doc.php']['makeEditForm_accessCheck'] in the EditDocumentController.

Register it like this:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_tceforms_inline.php']['checkAccess'][] = 'My\\Package\\HookClass->hookMethod';
```

Feature: #59231 - Hook for AbstractUserAuthentication::checkAuthentication()

See Issue #59231

Description

Hook to post-process login failures in `AbstractUserAuthentication::checkAuthentication`. By default the process sleeps for five seconds in case of failing. By using this hook, different solutions for brute force protection can be implemented.

Register like this:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_userauth.php']['postLoginFailureProcessing'][] = 'My\\Package\\HookClass->hookMethod';
```

Feature: #59384 - XML parser options for xml2tree()

See Issue #59384

Description

`GeneralUtility::xml2tree()` gets an optional parameter: an array that can hold options for the parser. Those will simply be passed through to the PHP-function xml_parser_set_option().

```
GeneralUtility::xml2tree($xmlData, 999, array(XML_OPTION_SKIP_WHITE => 1));
```

Impact

It's just an optional parameter. If you don't specify it, simply no additional initialisation of the XML-parser will be done.

Feature: #59570 - Add description-field for filemounts

See Issue #59570

Description

A new field "description" has been added to the table sys_filemounts. With the new field it is possible to give a short description what a certain filemount should be used for or which documents it contains, etc.

Impact

The new field is optional and can be used in all projects.

Feature: #61308 - Allow Placeholder attribute

See Issue #61308

Description

System extension `form` allows a placeholder attribute in textfields by adding it in a form configuration. However, there is no option to add placeholder using the form wizard, yet.

Feature: #61903 - PageTS dataprovider for backend layouts

See Issue #61903

Description

Over the last year, several extensions appeared on TER that implemented the very same basic feature: Deploying backend layouts without database records by providing them via PageTS.

Implement a generic PageTS provider for backend layouts to unify those approaches and to make backend layouts reusable across installations.

Impact

It is now possible to define backend layouts via PageTSConfig on every page.

Example

```
mod {
        web_layout {
                BackendLayouts {
                        exampleKey {
                                title = Example
                                config {
                                        backend_layout {
                                                colCount = 1
                                                rowCount = 2
                                                rows {
                                                        1 {
                                                                columns {
                                                                        1 {
↪name = LLL:EXT:frontend/Resources/Private/Language/locallang_ttc.xlf:colPos.I.3
↪colPos = 3
↪colspan = 1
                                                                        }
                                                                }
                                                        }
                                                        2 {
                                                                columns {
                                                                        1 {
↪name = Main
↪colPos = 0
↪colspan = 1
                                                                        }
                                                                }
                                                        }
                                                }
                                        }
                                }
                                icon = EXT:example_extension/Resources/Public/
↪Images/BackendLayouts/default.gif
                        }
                }
        }
}
```

Feature: #64200 - Allow individual content caching

See Issue #64200

Description

The `stdWrap.cache.` property is now available as first-class function to all content objects. This skips the rendering even for content objects that evaluate `stdWrap` after rendering (e.g. `COA`).

Usage:

```
page = PAGE
page.10 = COA
page.10 {
        cache.key = coaout
        cache.lifetime = 60
        #stdWrap.cache.key = coastdWrap
        #stdWrap.cache.lifetime = 60
        10 = TEXT
        10 {
                cache.key = mycurrenttimestamp
                cache.lifetime = 60
                data = date : U
                strftime = %H:%M:%S
                noTrimWrap = |10: | |
        }
        20 = TEXT
        20 {
                data = date : U
                strftime = %H:%M:%S
                noTrimWrap = |20: | |
        }
}
```

The commented part is `stdWrap.cache.` property available since 4.7, that does not stop the rendering of `COA` including all sub-cObjects.

Additionally, stdWrap support is added to key, lifetime and tags.

Impact

If you've previously used the `cache.` property in your custom cObject, this will now fail, because `cache.` is unset to avoid double caching. You are encouraged to rely on the core methods for caching cObjects or rename your property.

`stdWrap.cache` continues to exists and can be used as before. However the top level `stdWrap` of certain cObjects (e.g. `TEXT` cObject) will not evaluate `cache.` as part of `stdWrap`, but before starting the rendering of the cObject. In conjunction the storing will happen after the `stdWrap` processing right before the content is returned.

Top level `cache.` will not evaluate the hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['tslib/class.tsl` any more.

Feature: #65550 - Make table display order configurable in List module

See Issue #65550

Description

The new `PageTSconfig` configuration option `mod.web_list.tableDisplayOrder` has been added for the List module to allow flexible configuration of the order in which tables are displayed. The keywords `before` and `after` can be used to specify an order relative to other table names.

Example:

```
mod.web_list.tableDisplayOrder.<tableName> {
  before = <tableA>, <tableB>, ...
  after = <tableA>, <tableB>, ...
}
```

Feature: #65698 - Additional localization files in backend workspace module

See Issue #65698

Description

The AdditionalResourceService of the workspace module in the backend is extended by the functionality to register custom localization files that are forwarded to the PageRenderer in the end. This way, labels can be accessed in JavaScript using the TYPO3.l10n.localize() function for instance.

```
\TYPO3\CMS\Workspaces\Service\AdditionalResourceService::getInstance()->
↪addLocalizationResource(
        'EXT:my_extension/Resources/Private/Language/locallang.xlf'
);
```

Feature: #66070 - Configure anchor for pagination widget

See Issue #66070

Description

This feature allows to add a key "section" to the configuration of a fluid pagination widget. The anchor gets appended to every link of the pagination widget. The "widget.link" viewHelper used by the pagination widget already supports this.

Examples

The following example will render the page browser having a section parameter "#archive" appended to every link

```
<f:widget.paginate objects="{plantpestWarnings}" as="paginatedWarnings"␣
↪configuration="{section: 'archive', itemsPerPage: 10, insertAbove: 0, insertBelow:
↪ 1, maximumNumberOfLinks: 10}">
[...]
</f:widget.paginate>
```

Impact

If the "section" attribute does not get specified or no configuration is supplied at all then no section parameter (#section) will get appended to the links and the pagination widget behaves as usual.

Feature: #67228 - Emit Signal when an IndexRecord is marked as missing

See Issue #67228

Description

The new signal `recordMarkedAsMissing` is emitted when the FAL indexer encounters a sys_file record which does not have a corresponding filesystem entry and marks it as missing. It passes the sys_file record uid.

Impact

This can be used by extensions that provide or extend file management capabilities (versioning, synchronizations, recovery etc).

Feature: #67290 - DBAL: DBMS specific conversion between Meta/MySQL field types

See Issue #67290

Description

DBAL did a generic translation between MySQL native and DBMS specific field types.

The translation of field types has been enhanced to allow more specific conversions per DBMS driver. Overrides for PostgreSQL have been added with optimized mappings for BLOB, SERIAL, DOUBLE and INTEGER columns.

Impact

Running PostgreSQL, the Upgrade Wizard in the Install Tool will show a lot of field alterations as the optimized mappings will be used.

Feature: #67293 - Dependency ordering service

See Issue #67293

Description

In many cases it is necessary to establish a sorted list of items from a set of "dependencies". The ordered list is then used to execute actions in the given order.

Some examples from the Core are: - Hook execution order - Extension loading order - Listing of menu items

The dependencies are therefore specified in a relative manner, outlining that an item has to be executed/loaded/listed "before" or "after" some other item.

Typical use case:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['someExt']['someHook'][<some id>] = [
        'handler' => someClass::class,
        'runBefore' => [ <some other ID> ],
        'runAfter' => [ ... ],
        ...
];
```

In order to evaluate such relative dependencies to finally have a sorted list for `['someHook']`, we introduced a new helper class `\TYPO3\CMS\Core\Service\DependencyOrderingService`, which does the evaluation work for you.

Example usage:

```
$hooks = $GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['someExt']['someHook'];
$sortedHooks = GeneralUtility:makeInstance(DependencyOrderingService::class)->
↪orderByDependencies($hooks , 'runBefore', 'runAfter');
```

`$sortedHooks` will then contain the content of `$hooks`, but sorted according to the dependencies.

The `DependencyOrderingService` class also detects cycles in the dependencies and will throw an Exception in case conflicting dependencies have been defined.

In case the initial list does not specify a dependency for an item, those items will be put last in the final sorted list.

Feature: #67360 - Custom attribute name and multiple values for meta tags

See Issue #67360

Description

`page.meta` is extented to support different attribute names like `property` used for OG tags. You may also supply multiple values for one name, which results in multiple meta tags with the same name to be rendered.

See http://ogp.me/ for more information about the Open Graph protocol and its properties.

```
page {
    meta {
        X-UA-Compatible = IE=edge,chrome=1
        X-UA-Compatible.attribute = http-equiv

        keywords = TYPO3

        og:site_name = TYPO3
        og:site_name.attribute = property

        description = Inspiring people to share Normal

        dc\.description = Inspiring people to share [DC tags]

        og:description = Inspiring people to share [OpenGraph]
        og:description.attribute = property

        og:locale = en_GB
        og:locale.attribute = property

        og:locale:alternate {
            attribute = property
            value {
                1 = fr_FR
                2 = de_DE
            }
        }

        refresh = 5; url=http://example.com/
        refresh.attribute = http-equiv

    }
}
```

Impact

Meta tags with a different attribute name are supported now like the Open Graph meta tags.

Feature: #67545 - AJAX call to check whether file exists

See Issue #67545

Description

A Backend AJAX call to check whether a file exists has been added. The call needs two parameters to work properly.

Impact

The method can be called with `TYPO3.settings.ajaxUrls['file_exists']`. The parameters `fileName` and `fileTarget` are required:

- fileName: Name of the file
- fileTarget: Combined identifier of target directory for the file

Feature: #67574 - Display online status in backend user list

See Issue #67574

Description

A badge that shows the online status of a backend user has been integrated.

Impact

The badge will be shown next to the username in the backend user list.

Feature: #67578 - Add description-field for backend-users

See Issue #67578

Description

A new field "description" has been added to the table be_users. With the new field it is possible to give a short description about a user, like there is already for be_groups.

Impact

The new field is optional and can be used in all projects.

Feature: #67603 - Introduce TCA > ctrl > descriptionColumn

See Issue #67603

Description

To annotate database table column fields as internal description for editors and admins a new setting for TCA is introduced. Setting is called `['TCA']['ctrl']['descriptionColumn']` and holds column name.

This description should only displayed in the backend to guide editors and admins.

Usage of descriptionColumn is added under different issues.

Impact

None, since annotation itself is added only. Does not impact.

Feature: #67658 - Introduce DataProcessors for splitting values

See Issue #67658

Description

Two new DataProcessors are added to allow flexible processing for comma-separated values. To use e.g. with the FLUIDTEMPLATE content object.

The SplitProcessor allows to split values separated with a delimiter inside a single database field into an array to loop over it.

The CommaSeparatedValueProcessor allows to split values into a two-dimensional array used for CSV files or tt_content records of CType "table".

Using the SplitProcessor the following scenario is possible:

```
page.10 = FLUIDTEMPLATE
page.10.file = EXT:site_default/Resources/Private/Template/Default.html
page.10.dataProcessing.2 = TYPO3\CMS\Frontend\DataProcessing\SplitProcessor
page.10.dataProcessing.2 {
        if.isTrue.field = bodytext
        delimiter = ,
        fieldName = bodytext
        removeEmptyEntries = 1
        filterIntegers = 1
        filterUnique = 1
        as = keywords
}
```

In the Fluid template then iterate over the split data:

```
<f:for each="{keywords}" as="keyword">
        <li>Keyword: {keyword}</li>
</f:for>
```

Using the CommaSeparatedValueProcessor the following scenario is possible:

```
page.10 = FLUIDTEMPLATE
page.10.file = EXT:site_default/Resources/Private/Template/Default.html
page.10.dataProcessing.4 =␣
↪TYPO3\CMS\Frontend\DataProcessing\CommaSeparatedValueProcessor
page.10.dataProcessing.4 {
        if.isTrue.field = bodytext
        fieldName = bodytext
        fieldDelimiter = |
        fieldEnclosure =
        maximumColumns = 2
        as = table
}
```

In the Fluid template then iterate over the processed data:

```
<table>
        <f:for each="{table}" as="columns">
                <tr>
                        <f:for each="{columns}" as="column">
```

```
                                <td>{column}</td>
                        </f:for>
                <tr>
        </f:for>
</table>
```

Feature: #67662 - DataProcessor for files

See Issue #67662

Description

A new Files DataProcessor has been introduced, which can be used to prepare data to be handled by a ContentObject implementing the processors, e.g. the FLUIDTEMPLATE ContentObject. The FilesProcessor resolves File References, Files, or Files inside a folder or collection to be used for output in the Frontend. A FLUIDTEMPLATE can then simply iterate over processed data automatically.

```
tt_content.image.20 = FLUIDTEMPLATE
tt_content.image.20 {
        file = EXT:myextension/Resources/Private/Templates/ContentObjects/Image.
↪html

        dataProcessing.10 = TYPO3\CMS\Frontend\DataProcessing\FilesProcessor
        dataProcessing.10 {
                # the field name where relations are set
                # + stdWrap
                references.fieldName = image

                # the table name where relations are put, defaults to the␣
↪currently selected record from $cObj->getTable()
                # + stdWrap
                references.table = tt_content

                # A list of sys_file UID records
                # + stdWrap
                files = 21,42

                # A list of File Collection UID records
                # + stdWrap
                collections = 13,14

                # A list of FAL Folder identifiers and files fetched recursive␣
↪from all folders
                # + stdWrap
                folders = 1:introduction/images/,1:introduction/posters/
                folders.recursive = 1

                # Property of which the files should be sorted after they have␣
↪been accumulated
                # can be any property of sys_file, sys_file_metadata
                # + stdWrap
                sorting = description

                # Can be "ascending", "descending" or "random", defaults to
↪"ascending" if none given
                # + stdWrap
                sorting.direction = descending

                # The target variable to be handed to the ContentObject again, can␣
↪be used
```

```
                # in Fluid e.g. to iterate over the objects. defaults to "files"␣
↪when not defined
                # + stdWrap
                as = myfiles
        }
}
```

In the Fluid template then iterate over the files:

```
<ul>
<f:for each="{myfiles}" as="file">
        <li><a href="{file.publicUrl}">{file.name}</a></li>
</f:for>
</ul>
```

Feature: #67663 - Introduce DataProcessor for media galleries

See Issue #67663

Description

The logic for working with galleries and calculating the maximum asset size is done in a separate GalleryProcessor. The GalleryProcessor uses the files already present in the processedData array for his calculations. The FilesProcessor can be used to fetch the files.

```
tt_content.textmedia.20 = FLUIDTEMPLATE
tt_content.textmedia.20 {
        file = EXT:myextension/Resources/Private/Templates/ContentObjects/Image.
↪html

        dataProcessing {

                # Process files
                10 = TYPO3\CMS\Frontend\DataProcessing\FilesProcessor

                # Calculate gallery info
                20 = TYPO3\CMS\Frontend\DataProcessing\GalleryProcessor
                20 {

                        # filesProcessedDataKey :: Key in processedData array that␣
↪holds the files (default: files) + stdWrap
                        filesProcessedDataKey = files

                        # mediaOrientation :: Media orientation, see: TCA[tt_
↪content][column][imageorient] (default: data.imageorient) + stdWrap
                        mediaOrientation.field = imageorient

                        # numberOfColumns :: Number of columns (default: data.
↪imagecols) + stdWrap
                        numberOfColumns.field = imagecols

                        # equalMediaHeight :: Equal media height in pixels (default:
↪ data.imageheight) + stdWrap
                        equalMediaHeight.field = imageheight

                        # equalMediaWidth :: Equal media width in pixels (default:␣
↪data.imagewidth) + stdWrap
                        equalMediaWidth.field = imagewidth
```

```
                        # maxGalleryWidth :: Max gallery width in pixels (default:␣
↪600) + stdWrap
                        maxGalleryWidth = 1000

                        # maxGalleryWidthInText :: Max gallery width in pixels␣
↪when orientation intext (default: 300) + stdWrap
                        maxGalleryWidthInText = 1000

                        # columnSpacing :: Column spacing width in pixels (default:
↪ 0) + stdWrap
                        columnSpacing = 0

                        # borderEnabled :: Border enabled (default: data.
↪imageborder) + stdWrap
                        borderEnabled.field = imageborder

                        # borderWidth :: Border width in pixels (default: 0) +␣
↪stdWrap
                        borderWidth = 0

                        # borderPadding :: Border padding in pixels  (default: 0)␣
↪+ stdWrap
                        borderPadding = 10

                        # as :: Name of key in processedData array where result is␣
↪placed (default: gallery) + stdWrap
                        as = gallery
                }
        }
}
```

Feature: #67765 - Introduce TypoLinkCodecService

See Issue #67765

Description

The new `TypoLinkCodecService` class helps to simplify encoding and decoding of TypoLink strings.

A given TypoLink string can be passed to the `decode` method, which will return an associative array with the decoded parts. The `encode` method takes care of assembling a valid TypoLink string for an array of TypoLink parts.

The encoding uses proper quoting and escaping, which allows safe usage of characters like `"\<space>`.

Feature: #67808 - Introduce Application classes for entry points and equivalent RequestHandlers

See Issue #67808

Description

All entry points are encapsulating all previous initialization code in an Application class depending on the TYPO3_MODE and several context-dependant constraints. Each Application class registers Request Handlers to the TYPO3 Bootstrap to run a certain request type (e.g. eID or TSFE-logic, or AJAX requests in the Backend). Each Application is handed over the Class Loader provided by Composer.

There are four types of Applications provided by the TYPO3 Core:

TYPO3CMSFrontendHttpApplication

All incoming web requests coming to index.php in the main directory, handling all TSFE and eID requests. The Application sets TYPO3_MODE=FE very early. The Application checks if all configuration is given, otherwise redirects to the TYPO3 Install Tool.

TYPO3CMSBackendHttpApplication

All incoming web requests for any regular Backend call inside typo3/*. This handles three types of Request Handlers:

- The AJAX Request Handler, which is triggered on requests with an "ajaxID" GET Parameter given.
- The Backend Module Request Handler, which handles all types of modules triggered on requests with an "M" GET Parameter
- The regular Request handler for typical other backend calls on index.php.

The Application checks if all configuration is given, otherwise redirects to the TYPO3 Install Tool.

TYPO3CMSBackendConsoleApplication

All CLI Requests handled by cli_dispatch.php. Only executes the parts that are necessary for Backend CLI Scripts used with the cliKey syntax. The typical CliRequestHandler is used for handling requests set up by this Application.

TYPO3CMSInstallHttpApplication

The install tool Application only runs with a very limited bootstrap set up with a Failsafe Package Manager not taking the ext_localconf.php scripts of installed extensions into account.

Feature: #67880 - Added count to listNum

See Issue #67880

Description

A new property `returnCount` is added to the stdWrap property `split`.

When dealing with comma separated values like the content of field:records or similar, in some cases we need to know, how many items are present inside the csv.

Example:

```
# should return 9
1 = TEXT
1 {
        value = x,y,z,1,2,3,a,b,c
        split.token = ,
        split.returnCount = 1
}
```

Feature: #67932 - New rsaauth API

See Issue #67932

Description

The rsaauth API has been rewritten to be more generic and can now be used easily in more parts of the core as well as in third party extensions.

Impact

Form fields (e.g. password fields) can be encrypted before transmission. This helps to improve the security of your and your user's data.

Examples

Encode

Encoding is done automatically via a JavaScript function which gets a public key and encrypts the data.

1) Include JavaScript to parse form fields for encryption. You can either choose to include a RequireJS module or a plain Javascript file.

```
$rsaEncryptionEncoder = GeneralUtility::
↪makeInstance(\TYPO3\CMS\Rsaauth\RsaEncryptionEncoder::class);
$rsaEncryptionEncoder->enableRsaEncryption(); // Adds plain JavaScript
$rsaEncryptionEncoder->enableRsaEncryption(TRUE); // Adds RequireJS module
```

2. Activate encryption for your from fields with the data attribute `data-rsa-encryption`.

```
<input type="password" id="pass" name="pass" value="" data-rsa-encryption="" />
```

If you want the encrypted value to be stored in another field, you have to use the RequiredJS module and you can pass the id of that form field as value to the data attribute.

```
<input type="password" id="t3-password" name="p_field" value="" data-rsa-
↪encryption="t3-field-userident" />
<input type="hidden" name="userident" id="t3-field-userident" />
```

Decode

To decode your data you can use the method `TYPO3\CMS\Rsaauth\RsaEncryptionDecoder::decrypt` which can either handle a string or an array as parameter. Data that is handled by TYPO3CMSCoreDataHandlingDataHandler will be decoded automatically before processing.

Notice: A RSA public key can only be used once to decrypt data. If you encrypt multiple fields in your form you have to pass an array to the decrypt function with all data you want to decrypt. The function parses the values for a `rsa:` prefix so you can be sure that non-matching data will not be changed.

```
$rsaEncryptionDecoder = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Rsaauth\RsaEncryptionDecoder::class);

// Decrypt a single string
$password = $loginData['uident'];
$decryptedPassword = $rsaEncryptionDecoder->decrypt($password);

// Decrypt an array
if ($this->isRsaAvailable()) {
        $parameters['be_user_data'] = $this->getRsaEncryptionDecoder()->decrypt(
↪$parameters['be_user_data']);
}
```

Feature: #67950 - Move CE table options from flexform to tt_content

See Issue #67950

Description

The CE table (processing) configuration

- `Table caption`
- `Field delimiter`
- `Text enclosure`
- `Table header position`
- `Use table footer`

were in EXT:css_styled_content configured/saved in a flexform. This has now been moved to regular database fields.

Impact

When EXT:css_styled_content isn't installed a Migration wizard is shown in the install tool to move the flexform values to regular database fields in the tt_content table.

Feature: #68022 - Added base date attribute to DateViewHelper

See Issue #68022

Description

The DateViewHelper has been improved with an optional attribute named `base`. The attribute can be used to define a base-date when using a relative time specification for `date`. If `date` is a `DateTime` object, `base` is ignored.

The possible relative date format specification can be found in: http://www.php.net/manual/en/datetime.formats.relative.php

```
<f:format.date format="Y" base="{dateObject}">-1 year</f:format.date>
```

This will result in the output `2016` assuming the `dateObject` is some date in 2017.

Feature: #68047 - Emit a signal for each mapped object

See Issue #68047

Description

The signal `afterMappingSingleRow` is emitted whenever the DataMapper creates an object.

Feature: #68094 - Database Query DataProcessor

See Issue #68094

Description

A new Database Query DataProcessor has been introduced, which can be used to fetch data from the Database to be handled by a ContentObject implementing the processors, e.g. the FLUIDTEMPLATE ContentObject.

The Database Query Processor works like the code from the Content Object CONTENT, except for just handing over the result as array. A FLUIDTEMPLATE can then simply iterate over processed data automatically.

```
tt_content.mycontent.20 = FLUIDTEMPLATE
tt_content.mycontent.20 {
        file = EXT:myextension/Resources/Private/Templates/ContentObjects/MyContent.
↪html

        dataProcessing.10 =␣
↪TYPO3\CMS\Frontend\DataProcessing\DatabaseQueryProcessor
        dataProcessing.10 {
                # regular if syntax
                if.isTrue.field = records

                # the table name from which the data is fetched from
                # + stdWrap
                table = tt_address

                # All properties from .select can be used directly
                # + stdWrap
                colPos = 1
                pidInList = 13,14

                # The target variable to be handed to the ContentObject again, can␣
↪be used
                # in Fluid e.g. to iterate over the objects. defaults to "records"␣
↪when not defined
                # + stdWrap
                as = myrecords

                # The fetched records can also be processed by DataProcessors.
                # All configured processors are applied to every row of the result.
                dataProcessing {
                        10 = TYPO3\CMS\Frontend\DataProcessing\FilesProcessor
                        10 {
                                references.fieldName = image
                        }
                }
        }
}
```

In the Fluid template then iterate over the files:

```
<ul>
<f:for each="{myrecords}" as="record">
        <li>
                <f:image image="{record.files.0}" />
                <a href="{record.data.www}">{record.data.first_name} {record.data.
↪last_name}</a>
        </li>
</f:for>
</ul>
```

Feature: #68184 - Paths to typo3/cms package and document root can be specified in composer.json

See Issue #68184

Description

With the new composer installer it is possible to specify the path of the document root and also the path of the typo3/cms package.

It can be specified in the extra section of your composer root package like that:

```
{
        "repositories": [
                { "type": "composer", "url": "http://composer.typo3.org/" }
        ],
        "name": "typo3/cms-base-distribution",
        "description" : "TYPO3 CMS Base Distribution",
        "license": "GPL-2.0+",
        "config": {
                "vendor-dir": "Packages/Libraries",
                "bin-dir": "bin"
        },
        "require": {
                "typo3/cms": "dev-master"
        },
        "require-dev": {
                "mikey179/vfsStream": "1.3.*@dev",
                "phpunit/phpunit": "~4.4.0",
                "twbs/bootstrap": "3.3.*",
                "fortawesome/font-awesome": "4.2.*"
        },
        "extra": {
                "typo3/cms": {
                        "cms-package-dir": "{$vendor-dir}/typo3/cms",
                        "web-dir": "web"
                }
        }
}
```

Impact

When specifying the configuration like mentioned above, the directory structure of a `composer install` will change.

Feature: #68186 - PSR-7 support for eID added

See Issue #68186

Description

The eID dispatcher now supports eID scripts using the `TYPO3\CMS\Core\Http\ControllerInterface`. This allows to handle the request using the Request and Response objects.

Feature: #68191 - TypoScript .select option languageField is active by default

See Issue #68191

Description

The TypoScript .select option which is used for Content Objects like "CONTENT", has the property "languageField". This option allows to set the name of the database field that has the information about the sys_language_uid value in order to have only records shown that are translated or set to "-1" (show in all langauges) when showing translated pages.

Previously this functionality had to be set explicitly:

```
config.sys_language_uid = 2
page.10 = CONTENT
page.10 {
        table = tt_content
        select.where = colPos=0
        select.languageField = sys_language_uid
        renderObj = TEXT
        renderObj.field = header
        renderObj.htmlSpecialChars = 1
}
```

The languageField line is not necessary anymore, as the information is now fetched automatically from the TCA information structure:

```
config.sys_language_uid = 2
page.10 = CONTENT
page.10 {
        table = tt_content
        select.where = colPos=0
        renderObj = TEXT
        renderObj.field = header
        renderObj.htmlSpecialChars = 1
}
```

If the functionality should be disabled, this can be achieved like this:

```
config.sys_language_uid = 2
page.10 = CONTENT
page.10 {
        table = tt_content
        select.where = colPos=0
        select.languageField = 0
        renderObj = TEXT
        renderObj.field = header
        renderObj.htmlSpecialChars = 1
}
```

Impact

All records that have language-relevant information in the TCA "ctrl"-section displayed via .select in the frontend on translated pages are now translated by default.

Feature: #68197 - Show a dialog for existing files on upload

See Issue #68197

Description

The "Shall existing files be overwritten?" confirm dialog when uploading files in the "Files" backend module has been replaced by a modal window that appears when there are really file conflicts.

Impact

The user is able to define an action for each conflicted image. A file can be replaced, renamed or skipped.

Feature: #68218 - Lock edit for tt_content

See Issue #68218

Description

Content elements can now be restricted to be editable by admins only on a per record base. This has already been possible for pages with the checkbox "Restrict editing by non-Admins".

Impact

This is useful to generally allow editors to edit content but be able to assure that important elements like the imprint or specific plugins are not changed.

Feature: #68282 - Make DatabaseRecordList configurable to be editable

See Issue #68282

Description

A new property `editable` is added to `\TYPO3\CMS\Recordlist\RecordList\DatabaseRecordList` which is set to TRUE by default. If set to FALSE, the records can't be edited.

Impact

The record list in the Element Browser benefits from the new setting as the localization view is now enabled. This will show editors translated records properly intended below the record with the default language.

Feature: #68315 - Include a pageTSconfig file in page properties like TS static templates

See Issue #68315

Description

In the Page properties an option is added to include a page TSconfig file (the same way as TypoScript static templates are included). The included files from the pages in the rootline are included after the default page TSconfig and before the normal TSconfig from the pages in the rootline. To add files to the selector in the Page properties a new function `registerPageTSConfigFile` is added to `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility` to register a pageTSconfig file.

Impact

No effect on existing installations.

Usage

In `Configuration/TCA/Overrides/pages.php` of any extension, register PageTS config files, which will be shown afterwards at the newly introduced field.

```
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::registerPageTSConfigFile(
↪'extension_name', 'Configuration/PageTS/myPageTSconfigFile.txt', 'My special␣
↪config');
```

Feature: #68395 - Allow real copies of content elements into foreign languages

See Issue #68395

Description

A new button has been added to each column in the "Page" module which allows "real" copies of content element into a language. This allows to create copies from any language into the destination. References, like FAL records, become independent records and are not related to the original record, as there is no parent anymore.

Impact

The button will be either displayed as as standalone button if a page has no records in the default language or as a split button if there are records in the default language.

Creating real copies will cause the loss of any functionality between the copy and the default language (e.g. diff), as the copy is not defined as child of the element where it was copied from.

Feature: #68589 - Add CLI command to dump class loading information

See Issue #68589

Description

In rare cases it is useful to update the additional class loading information TYPO3 writes in non composer mode. We now provide a CLI command which does that.

Impact

By executing `typo3/cli_dispatch.phpsh extbase extension:dumpclassloadinginformation` on the command line, the class loading information of all active extensions is updated.

Feature: #68600 - Introduced ResourceStorage SanitizeFileName signal

See Issue #68600

Description

In order to check whether an uploaded/newly added file already exists before uploading it or to ask for user preferences about already existing files only when needed, the final name for the uploaded file is needed.

In order to let extensions do custom sanitizing of a file name the signal `sanitizeFileName` is introduced in `TYPO3\CMS\Core\Resource\ResourceStorage`. The signal is emitted when `ResourceStorage::sanitizeFileName` or `ResourceStorage::addFile` are called.

Impact

All installations with extensions that use the PreFileAdd signal to change/sanitize a file name. This logic should be moved to the new sanitizeFileName signal.

3.4.3 Deprecation

Deprecation: #50349 - Reduce SQL queries of page tree in workspaces

See Issue #50349

Description

The performance of the calculation of versions of a record has been improved. Therefore the method `\TYPO3\CMS\Backend\Utility::countVersionsOfRecordsOnPage()` has been marked as deprecated and is being replaced with `\TYPO3\CMS\Workspaces\Service\WorkspaceService::hasPageRecordVersio`

Impact

All calls to the PHP method will throw a deprecation warning.

Affected Installations

Instances which make use of `\TYPO3\CMS\Backend\Utility::countVersionsOfRecordsOnPage()`

Migration

Use `\TYPO3\CMS\Workspaces\Service\WorkspaceService::hasPageRecordVersions()` instead.

Deprecation: #63603 - ExtendedFileUtility::$dontCheckForUnique is deprecated

See Issue #63603

Description

The ExtendedFileUtility `$dontCheckForUnique` flag has been marked as deprecated and replaced by `$fileUtility->setExistingFileConflictMode()` with the possible options of the `\TYPO3\CMS\Core\Resource\DuplicationBehavior` enumeration.

Impact

Extensions still using `ExtendedFileUtility::$dontCheckForUnique` will throw a deprecation warning.

Affected Installations

All installations with extensions that use `ExtendedFileUtility::$dontCheckForUnique`.

Migration

Change the `$fileUtility->dontCheckForUnique = TRUE` to `$fileUtility->setExistingFileConflictMo`

Deprecation: #63603 - FileController and FileListController overwriteExistingFiles changed to string value

See Issue #63603

Description

The GET/POST param to tell the FileController and FileListController whether to override a file or not switched from a bool value to a string with the possibilities of the `\TYPO3\CMS\Core\Resource\DuplicationBehavior` enumeration.

Impact

Extensions still using `overwriteExistingFiles = 1` will throw a deprecation warning.

Affected Installations

All installations with extensions that use the BE upload functionality and supply the file override option.

Migration

Change the `<input name="overwriteExistingFiles" value="1">` to `<input name="overwriteExistingFiles" value="replace">`.

Deprecation: #63974 - Deprecate CSS compressor callback method

See Issue #63974

Description

The callback method `compressCssPregCallback()` as defined in EXT:core/Classes/Resource/ResourceCompressor.php has been marked as deprecated due to the overhauled regular expressions.

Impact

Usage of the mentioned method is discouraged and will break as of CMS 8.

Affected installations

All installations or extensions using the `compressCssPregCallback()` callback method.

Migration

No migration possible for the callback method.

Deprecation: #65790 - Remove pages.storage_pid and logic

See Issue #65790

Description

The DB field "pages.storage_pid" and its TCA definition have been moved to the compatibility6 extension as the field and its functionality is discouraged.

Additionally the method `getStorageSiterootPids()` within the PHP class `TypoScriptFrontendController` has been marked as deprecated. The method is currently only used if the Frontend Login plugin is used without setting a specific folder where the fe_users records are stored in.

Impact

Any usage of this field in any TypoScript, page or the usage of the method mentioned above in any third-party extension will only work if the compatibility6 extension is installed.

The Frontend Login functionality will throw a deprecation warning if the TypoScript option `plugin.tx_felogin.storagePid` (via TypoScript directly or the flexform configuration within the plugin) is not set.

Affected installations

All installations making use of `storage_pid` within the pages database table as well as installations using the Frontend Login plugin without having the storagePid option set.

Deprecation: #66904 - $disable Option in PageRepository->getExtURL()

See Issue #66904

Description

The second parameter `$disable` within `PageRepository->getExtURL()` has been marked as deprecated.

Affected Installations

Any installation using the method in a third-party extension above with using the second parameter set to `true` will throw a deprecation warning.

Migration

Check if redirects are enabled before the actual call to `PageRepository->getExtURL()` in a third-party extension.

Deprecation: #67288 - Deprecate DbalDatabaseConnection::MetaType() method

See Issue #67288

Description

The following public function has been marked as deprecated as the bugfix requires a signature change:

- `Dbal\DatabaseConnection->MetaType()`

Impact

Using this function will throw a deprecation warning. Due to missing information the field type cache will be bypassed and the DBMS will be queried for the necessary information on each call.

Migration

Switch to `getMetadata()` and the field name for which you need the ADOdb MetaType information.

Deprecation: #67471 - Deprecate init.php

See Issue #67471

Description

In order to move all unneeded files from typo3/, the often used init.php has been deprecated in favor of using the bootstrap initialization code directly in the TYPO3 Backend.

Impact

All entry points from third-party extensions using init.php will now throw a deprecation warning.

Affected Installations

All instances having extensions that include init.php when not using the mod.php for modules or ajax calls.

Migration

Use the following code instead of the init.php inclusion if you still need custom entry points:

```
define('TYPO3_MODE', 'BE');

require __DIR__ . '/sysext/core/Classes/Core/Bootstrap.php';
\TYPO3\CMS\Core\Core\Bootstrap::getInstance()->run('typo3/');
```

If using a module, use the mod.php to register your own module.

Deprecation: #67506 - Deprecate IconUtility::getIcon

See Issue #67506

Description

The method `IconUtility::getIcon()` which was used for generating overlaid icons for records has been marked as deprecated.

Impact

All calls to the PHP method will throw a deprecation warning.

Affected Installations

Instances with third-party extensions modifying the TYPO3 Backend with a custom module or hook and calling `IconUtility::getIcon()`.

Migration

Use `IconUtility::getSpriteIconForRecord()` instead.

Deprecation: #67670 - Deprecate custom singleton logic in GeneralUtility::getUserObj

See Issue #67670

Description

The functionality of instantiating classes only once by calling `GeneralUtility::getUserObj($className)` multiple times while having a `$className` that is prepended with a ampersand ("&") has been marked as deprecated.

An example of the deprecated behaviour in the ext_localconf.php of an extension:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['indexed_search']['pi1_hooks']['
↪'getResultRows_SQLpointer'] = '&Acme\\MyExtension\\Hooks\\MysqlFulltextIndexHook';
↪
```

Impact

Any calls to `GeneralUtility::getUserObj()` with a prefixed ampersand will throw a deprecation message.

Affected Installations

TYPO3 Instances with extensions that use `getUserObj()` themselves and/or use hooks built with `getUserObj()` and use references.

Migration

Check if the classes that hook into certain parts of your custom extensions really need to be referenced / instantiated once. If so, implement the `SingletonInterface` of the TYPO3 Core, so the underlying function `GeneralUtility::makeInstance()` will register the SingletonInterface only once.

The modified example from above now looks like this:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['indexed_search']['pi1_hooks']['
↪'getResultRows_SQLpointer'] = \Acme\MyExtension\Hooks\MysqlFulltextIndexHook::
↪class;
```

While the class itself implements the SingletonInterface of the TYPO3 Core to only be instantiated once during a single request:

```
<?php
namespace \Acme\MyExtension\Hooks;

class MysqlFulltextIndexHook implements \TYPO3\CMS\Core\Core\SingletonInterface {
        ...
}
```

Deprecation: #67737 - TCA: Drop additional palette

See Issue #67737

Description

The `showitem` string of `TCA types` allowed to define an "additional palette" as third semicolon separated name of a field. Such a palette was then rendered after the main field. This handling has been dropped and existing "additional palettes" were migrated to a "normal" palette definition directly after the field.

Before:

```
'types' => array(
        'aType' => array(
                'showitem' => 'aField;aLabel;anAdditionalPaletteName',
        ),
),
```

The behavior before was: If the field `aField` is rendered, then the palette `anAdditionalPaletteName` is rendered, too. This functionality has been dropped, the migrated field now looks like this:

```
'types' => array(
        'aType' => array(
                'showitem' => 'aField;aLabel, --palette--;;anAdditionalPaletteName',
↪
        ),
),
```

A casual field name in `showitem` now only has a label override as additional information, like `aField;aLabel`, while a palette is referenced as `--palette--;aLabel;paletteName`.

Impact

All extensions that use "additional palette" syntax are migrated to the new syntax, but will throw a deprecation message.

The "additional palette" handling was sometimes misused as "poor-mans-access-control": If access to the main field was not allowed, the palette fields were not rendered either. This changed, the main field and the palette are decoupled, it may happen that additional fields are now rendered for users that should not have access to it. Adapting the `exclude` config definition of the palette fields and user or group access records is necessary in those cases.

Affected Installations

Extensions that use the "additional palette" handling.

Migration

An automatic migration is in place and logged to `typo3conf/deprecation_*`. The migration code will be dropped with TYPO3 CMS 8, a manual fix of the according `showitem` field is required, the deprecation log gives detailed information on how the definition should look like.

Deprecation: #67769 - Deprecate QueryGenerator::formatQ()

See Issue #67769

Description

The method `QueryGenerator::formatQ()` which was used to format a query string, has been marked as deprecated.

Impact

All calls to the PHP method will throw a deprecation warning.

Affected Installations

Instances which make use of `QueryGenerator::formatQ()`.

Migration

No migration, use `htmlspecialchars` as alternative.

Deprecation: #67790 - Deprecate QueryGenerator::JSbottom()

See Issue #67790

Description

The method `QueryGenerator::JSbottom()` which was used to append JavaScript code has been marked as deprecated.

Impact

All calls to the PHP method will throw a deprecation warning.

Affected Installations

Instances which make use of `QueryGenerator::JSbottom()`.

Migration

No migration, use requireJS modules and register the module through `pageRenderer`.

Deprecation: #67932 - Deprecated old rsaauth encryption API

See Issue #67932

Description

The rsaauth API has been rebuilt to be more generic. Therefore the Ajax Handler `BackendLogin::getRsaPublicKey()` has been marked as deprecated and the eID script `FrontendLoginRsaPublicKey` has been removed.

Affected Installations

Any installation using one of the entry points above in a third-party extension.

Migration

There is no reason to use the entry points on your own anymore. Please update your scripts to use the new rsaauth API. For backend requests you should use the provided ajax handler `RsaEncryption::getRsaPublicKey()`. For frontend request you should use the provided eID script `RsaPublicKeyGenerationController`.

Deprecation: #67991 - Removed ext:cms

See Issue #67991

Description

Extension `cms` has been removed. Most functionality has been moved to extension `frontend` with version 6.0 already. The rest of the files have now been moved to other extensions, extension `cms` does not exist anymore.

Impact

Language files from EXT:cms are moved to different places into the core. `LLL:EXT:cms/...` references are rewritten dynamically to the new file locations, but this fallback layer is subject to be removed with CMS 8.

Third party extensions that define a dependency to extension `cms` will get their dependency rewritten to `core` on the fly during extension installation as a compatibility layer.

Affected Installations

All 3rd party extensions using language labels from extension `cms` or defining a dependency to extension `cms` in `ext_emconf.php`.

Migration

Update the dependency constraint in `ext_emconf.php` of the affected extension. A typical substitution is a dependency to extension `frontend`.

Move the following references to the new location of the language file:

- typo3/sysext/cms/web_info/locallang.xlf -> typo3/sysext/frontend/Resources/Private/Language/locallang_webinfo.xlf
- typo3/sysext/cms/locallang_ttc.xlf -> typo3/sysext/frontend/Resources/Private/Language/locallang_ttc.xlf
- typo3/sysext/cms/locallang_tca.xlf -> typo3/sysext/frontend/Resources/Private/Language/locallang_tca.xlf
- typo3/sysext/cms/layout/locallang_db_new_content_el.xlf -> typo3/sysext/backend/Resources/Private/Language/locallang_db_n
- typo3/sysext/cms/layout/locallang.xlf -> typo3/sysext/backend/Resources/Private/Language/locallang_layout.xlf
- typo3/sysext/cms/layout/locallang_mod.xlf -> typo3/sysext/backend/Resources/Private/Language/locallang_mod.xlf
- typo3/sysext/cms/locallang_csh_webinfo.xlf -> typo3/sysext/frontend/Resources/Private/Language/locallang_csh_webinfo.xlf
- typo3/sysext/cms/locallang_csh_weblayout.xlf -> typo3/sysext/frontend/Resources/Private/Language/locallang_csh_weblayout.

Deprecation: #68074 - Deprecate getPageRenderer() methods

See Issue #68074

Description

The following public functions have been marked as deprecated as the instance they return is a singleton:

- `TYPO3\CMS\Backend\Controller\BackendController::getPageRenderer()`
- `TYPO3\CMS\Backend\Template\DocumentTemplate::getPageRenderer()`
- `TYPO3\CMS\Backend\Template\FrontendDocumentTemplate::getPageRenderer()`
- `TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::getPageRenderer()`

Impact

Using one of these functions will throw a deprecation message.

Migration

As the PageRenderer implements a SingletonInterface you can get your own (shared) instance with `\TYPO3\CMS\Core\Utility\GeneralUtility::makeInstance(\TYPO3\CMS\Core\Page\PageRenderer::` and work with that one.

Deprecation: #68098 - Deprecate GeneralUtility methods

See Issue #68098

Description

The following methods within `GeneralUtility` have been marked as deprecated and will be removed in TYPO3 CMS v8.

```
GeneralUtility::modifyHTMLColor()
GeneralUtility::modifyHTMLColorAll()
GeneralUtility::isBrokenEmailEnvironment()
GeneralUtility::normalizeMailAddress()
GeneralUtility::formatForTextarea()
GeneralUtility::getThisUrl()
GeneralUtility::cleanOutputBuffers()
```

The functionality `formatForTextarea()` was used in the older days to actually support IE4 and Netscape 3 properly and can now safely be exchanged by `htmlspecialchars()`.

Impact

All extensions using these methods directly will throw a deprecation message.

Affected Installations

Installations with extensions that use the methods above handling.

Migration

Use corresponding functionality from `getIndpEnv()` instead of `getThisUrl()`.

For the other methods, you can re-implement the functionality yourself in your extension where needed.

Deprecation: #68122 - Deprecate GeneralUtility::readLLfile

See Issue #68122

Description

Method `GeneralUtility::realLLfile()` was just a wrapper around LocalizationFactory and has been marked as deprecated.

Impact

Extensions using `realLLfile()` to parse localization files should switch to an instance of `LocalizationFactory`.

Affected Installations

Extensions using `GeneralUtility::readLLfile()`

Migration

A typical call now should look like:

```
/** @var $languageFactory \TYPO3\CMS\Core\Localization\LocalizationFactory */
$languageFactory = GeneralUtility::
↪makeInstance(\TYPO3\CMS\Core\Localization\LocalizationFactory::class);
$languageFactory->getParsedData($fileToParse, $language, $renderCharset,
↪$errorMode);
```

Deprecation: #68141 - typo3/ajax.php

See Issue #68141

Description

The ajax.php entry-point has been marked as deprecated. All AJAX requests in the Backend using the Ajax API are not affected as they automatically use index.php.

Impact

All extensions directly linking to typo3/ajax.php will throw a deprecation warning.

Affected Installations

Installations with custom extensions that call typo3/ajax.php without using proper API calls from `BackendUtility`.

Migration

Use `BackendUtility::getAjaxUrl()`.

Deprecation: #68183 - typo3/mod.php

See Issue #68183

Description

The mod.php entry-point has been marked as deprecated. All Backend Module requests in the Backend using the Module Url API are not affected as they automatically use index.php.

Impact

All extensions directly linking to typo3/mod.php will throw a deprecation warning.

Affected Installations

Installations with custom extensions that call typo3/mod.php without using proper API calls from `BackendUtility`.

Migration

Use `BackendUtility::getModuleUrl()`.

3.4.4 Important

Important: #67216 - Default minimum log level is set to warning

See Issue #67216

Description

The minimum logging severity for TYPO3 has been raised from DEBUG to WARNING in order to ship strong defaults for production. Log messages of the severities DEBUG, NOTICE and INFO will be suppressed in the default setup.

The previous behavior from TYPO3 <= 7.3 can be achieved with the following configuration:

```
$GLOBALS['TYPO3_CONF_VARS']['LOG']['writerConfiguration'] = array(
        \TYPO3\CMS\Core\Log\LogLevel::DEBUG => array(
                \TYPO3\CMS\Core\Log\Writer\FileWriter::class => array(
                        'logFile' => 'typo3temp/logs/typo3.log'
                ),
        ),
);
```

Important: #67445 - DBAL support for ALTER TABLE ADD/DROP KEY added

See Issue #67445

Description

The prefix used to build the name of indexes in a database schema has been changed. The prefix is used to ensure that an index name is unique within a database schema.

Formerly the requested index name was prepended with the table name to which the index was added. In some cases this results in index names that exceeded the valid identifier length on all DBMS except MS SQL Server. The silent truncation of these identifiers results in non-unique names or index names that can not be matched to the original name.

With TYPO3 7.4 the prefix used for index names has been changed to a unique constant length prefix. Due to this all non-primary indexes need to be dropped and re-created with a new name. The changes to the database will be performed by the Upgrade Wizard in the Install Tool.

Important: #67852 - Remove jsfunc.evalfield.js from FormEngine

See Issue #67852

Description

After 12 years, the usage of `jsfunc.evalfield.js` has been removed from `FormEngine`. The JavaScript has been moved into FormEngineValidation AMD module. Processor and Validator have been split up in two different functions.

Including the `jsfunc.evalfield.js` still works, but will be removed on short notice.

Important: #68290 - Default behavior for TCA suggest wizard changed

See Issue #68290

Description

The suggest wizard by default searches in the whole word instead only the beginning now. This might have performance implications for large sites with a lot of records and/or sites that have a lot of tables that are searched, as the search is done with a `LIKE "%searchterm%"`.

To switch back to the old behavior add `searchWholePhrase = FALSE` to the config of the suggest wizard.

Example to reset it for `page.shortcut`:

```
'shortcut' => array(
        'config' => array(
                'type' => 'group',
                'internal_type' => 'db',
                'allowed' => 'pages',
                'size' => '1',
                'maxitems' => '1',
                'minitems' => '0',
                'show_thumbs' => '1',
                'wizards' => array(
                        'suggest' => array(
                                'type' => 'suggest',
                                'default' => array(
                                        'additionalSearchFields' => 'nav_title,␣
↪alias, url',
                                        'searchWholePhrase' => FALSE
                                )
                        )
                )
        )
),
```

Important: #68600 - Introduced ResourceStorage SanitizeFileName signal

See Issue #68600

Description

In order to check whether an uploaded/newly added file already exists before uploading it or to ask for user preferences about already existing files only when needed, the final name for the uploaded file is needed.

Before #68600 the PreFileAdd signal was documented to have the ability to change the `$targetFileName`, but the signal expects the local file path of the file. Since this information isn't available when checking only by a file name if a file already exists, a new signal has been added to `TYPO3\CMS\Core\Resource\ResourceStorage`, which is emitted when the ResourceStorage is asked to sanitize a file name.

Affected Installations

All installations with extensions that use the PreFileAdd signal to change/sanitize a file name. This logic should be moved to the new sanitizeFileName signal.

3.5 7.3 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*
- *Important*

3.5.1 Breaking Changes

Breaking: #62983 - postProcessMirrorUrl signal has moved

See Issue #62983

Description

While refactoring the Language backend module, the `\TYPO3\CMS\Lang\Service\UpdateTranslationService::pos` signal got lost. Due to the refactoring, it has been integrated in another class.

Impact

Using the old signal will prevent the slot from being called.

Affected Installations

All extensions are affected that use the old `\TYPO3\CMS\Lang\Service\UpdateTranslationService::postProces` signal.

Migration

Change the slot to use the `\TYPO3\CMS\Lang\Service\TranslationService::postProcessMirrorUrl` signal. If it is required to serve multiple TYPO3 versions, use the following code:

```
$signalSlotDispatcher->connect(
        version_compare(TYPO3_version, '7.0', '<')
                ? 'TYPO3\\CMS\\Lang\\Service\\UpdateTranslationService'
                : 'TYPO3\\CMS\\Lang\\Service\\TranslationService',
        'postProcessMirrorUrl',
        'Vendor\\Extension\\Slots\\CustomMirror',
        'postProcessMirrorUrl'
);
```

Breaking: #63453 - Changed rendering of FlashMessagesViewHelper

See Issue #63453

Description

The default (`renderMode="ul"`) rendering output of the `FlashMessagesViewHelper` has been changed.

By default the view helper rendered an unordered list, each list item containing one message. This output has been adjusted and more markup has been added.

Impact

You may see unexpected formatting of flash messages.

Affected Installations

Any template using the `FlashMessagesViewHelper` unless the attribute `renderMode` is set to "div". Be aware that the `renderMode` attribute has been deprecated.

Migration

Add a custom rendering template for the flash messages, like outlined in the example, to obtain the same output as before.

```
<f:flashMessages as="flashMessages">
        <ul class="myFlashMessages">
                <f:for each="{flashMessages}" as="flashMessage">
                        <li>{flashMessage.message}</li>
                </f:for>
        </ul>
</f:flashMessages>
```

Breaking: #63835 - Remove Deprecated Parts in Extbase Persistence Layer

See Issue #63835

Description

The previously deprecated functions `TYPO3\CMS\Extbase\Persistence\Generic\Backend->setDeletedObjects` and `TYPO3\CMS\Extbase\Persistence\Repository->replace()` inside the Persistence Layer of Extbase have been removed. The protected property "session" inside `TYPO3\CMS\Extbase\Persistence\Repository` has been removed as well.

Impact

Any direct calls to the methods will now exit with a PHP Fatal Error.

Breaking: #63846 - FormEngine refactoring

See Issue #63846

Description

FormEngine is the core code structure that renders a record view in the backend. Basically everything that is displayed if elements from page or list module are edited is done by this code.

The main implementation was done thirteen years ago and was never touched on a deep code structure level until now. The according patches were huge and move the whole code to a new level. Stuff like that can not be done without impact on extensions that use this code.

Impact

TCA changes

- Keys `_PADDING`, `_VALIGN` and `DISTANCE` of `TCA['aTable']['columns']['aField']['config']['wizar` have been removed and have no effect anymore.
- Key `TCA['aTable']['ctrl']['mainPalette']` has been dropped and has no effect anymore.

TSconfig changes

- Key `mod.web_layout.tt_content.fieldOrder` has been dropped and has no effect anymore.
- Key `TCEFORM.aTable.aField.linkTitleToSelf` has been dropped and has no effect anymore.

Code level

Methods and properties from FormEngine are not available anymore. Classes like `InlineElement` are gone. New structures like a factory for elements and container have been introduced. While not too many extensions in the wild hook or code with FormEngine, those that do will probably throw fatal errors after upgrade. The hook `getSingleFieldClass` has been removed altogether.

Changed user functions and hooks

- TCA: If format of type `none` is set to `user`, the configured userFunc no longer gets an instance of `FormEngine` as parent object, but an instance of `NoneElement`.
- TCA: Wizards configured as `userFunc` now receive a dummy `FormEngine` object with empty properties instead of the real instance.
- Hooks no longer get the key `form_type`. Use `type` instead.
- Hook `getSingleFieldClass` has been dropped and no longer called.

Breaking interface changes

- The type hint to `FormEngine` as `$pObj` has been removed on the `DatabaseFileIconsHookInterface`. This hook is no longer given an instance of `FormEngine`.
- Method `init()` of `InlineElementHookInterface` has been removed. Classes that implement this interface will no longer get `init()` called.

Affected installations

For most instances, the overall impact is rather low or they are not affected at all. Some very rarely used TCA and TSconfig options have been dropped, those will do no harm. Instances are usually only affected if loaded extensions do fancy stuff with FormEngine with hooks or other related code.

TYPO3 CMS 7 installations with extensions using or hooking into FormEngine and its related classes are likely to break. TCA elements of type user may break. Instances using these parts will quickly show fatal errors at testing. It may help to search for `FormEngine` or `t3lib_tceForms` below the `typo3conf/ext` directory to find affected instances.

Migration

Adapt the extension code. The majority of methods were for internal core usage only, but still public. Please use the existing API to solve needs on FormEngine.

Breaking: #66429 - Remove IdentityMap from persistence

See Issue #66429

Description

The `IdentityMap` class and its usage has been removed from the Extbase persistence.

Impact

Upgraded installations will throw a `ReflectionException`. Accessing the previously existing `IdentityMap` properties within `DataMapper` and `Repository` will now fail. Creating `IdentityMap` instances is not possible anymore.

Affected Installations

All installations, especially extensions using the `IdentityMap` class directly or accessing the properties within `DataMapper` or `Repository`.

Migration

The Extbase reflection cache of existing installations needs to be cleared once.

Existing code can be migrated to the persistence `Session` class which provides a drop-in replacement for the `IdentityMap`.

Usage example

How to use the `Session` class to retrieve objects by an identifier:

```
$session = GeneralUtility::makeInstance(ObjectManager::class)->
 ↪get(\TYPO3\CMS\Extbase\Persistence\Generic\Session::class);
$session->registerObject($object, $identifier);
if ($session->hasIdentifier($identifier)) {
        $object = $session->getObjectByIdentifier($identifier, $className);
}
```

Breaking: #66669 - Backend LoginController refactored

See Issue #66669

Description

The backend login has been completely refactored and a new API has been introduced. The openid form has been extracted and is now using the new API as well.

Impact

All former member variables of the `LoginController` class have been removed or made protected, together with some, now pointless, hooks and their related classes.

The deleted hooks are:

- `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['typo3/index.php']['loginScriptHook']`
- `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['typo3/index.php']['loginFormHook']`

The removed class and its alias:

- `TYPO3\CMS\Rsaauth\Hook\LoginFormHook`
- `tx_rsaauth_loginformhook`

Affected Installations

Any code manipulating the BE login.

Migration

Use the new backend login form API.

Breaking: #66707 - issueCommand() now adds quotes when used in JS context

See Issue #66707

Description

Using `\TYPO3\CMS\Backend\Template\DocumentTemplate::issueCommand()` in JavaScript context (second parameter = -1), now ensures that the URL is properly escaped and quoted for being used in JavaScript code.

Impact

Having additional quotes around the result of the call to `issueCommand()` will lead to JavaScript errors.

Affected Installations

Any installation using third party extensions, which use `issueCommand()` with second parameter set to -1.

Migration

Make sure that you do **not** specify any additional quotes around the result of the call to `issueCommand()`.

Breaking: #66754 - Remove RenderingContextAwareInterface

See Issue #66754

Description

The `RenderingContextAwareInterface` allowed objects to get the `RenderingContext` set while being accessed from inside a Fluid template. This makes optimization of variable access in Fluid difficult and seems to be an unused feature. Therefore it has been removed.

Impact

For implementations of `RenderingContextAwareInterface` the change breaks without any simple replacement. Functionality would have to be replicated in userland code. But as there are no known implementations the expected impact is rather low.

Breaking interface changes

- The `RenderingContextAwareInterface` has been removed. There is no replacement.

Breaking: #66868 - Move usage of BackendUserSettingsDataProvider

See Issue #66868

Description

The ExtDirect API `BackendUserSettingsDataProvider` has been removed.

Impact

Third party code using either `BackendUserSettingsDataProvider` or `top.TYPO3.BackendUserSettings.ExtDirect` will fatal.

Affected Installations

Any installation using `BackendUserSettingsDataProvider` or `top.TYPO3.BackendUserSettings.ExtDirect` is affected.

Migration

In JavaScript, use `TYPO3.Storage.Persistent` API. In PHP, use `\TYPO3\CMS\Backend\Controller\UserSettingsController`:

```
/** @var $userSettingsController␣
↪\TYPO3\CMS\Backend\Controller\UserSettingsController */
$userSettingsController = GeneralUtility::
↪makeInstance(\TYPO3\CMS\Backend\Controller\UserSettingsController::class);
$state = $userSettingsController->process('get', 'BackendComponents.States.' .
↪$stateId);
```

Breaking: #66906 - Automatic PNG to GIF conversion removed

See Issue #66906

Description

The configuration setting `$TYPO3_CONF_VARS[GFX][png_to_gif]` has been removed.

Impact

If the option is set in an installation, then PNG images used in the TYPO3 Frontend will now be kept as PNG, instead of converting them to GIF files.

Affected Installations

Installations having the option `$TYPO3_CONF_VARS[GFX][png_to_gif]` activated.

Breaking: #66991 - TCA value slider based on jQuery

See Issue #66991

Description

The TCA value slider has been ported from ExtJS to jQuery and Bootstrap.

Impact

Since TYPO3 CMS 7 uses a DateTimePicker, the time selection conflicts with the value slider and therefore time-sliding has been dropped.

Affected Installations

All installations are affected whose TCA uses the value slider wizard in combination with `time` evaluation.

Migration

Remove the slider wizard from affected TCA.

Breaking: #66997 - Remove super-/challenged password security

See Issue #66997

Description

TYPO3 CMS supports four possibilities how passwords can be sent from the browser to the server:

- "normal": Plain text
- "challenged": md5 hashed
- "superchallenged": md5 hashed
- "rsa": asymmetric encryption

Since TYPO3 CMS 6.2 the password transmission is protected by the rsaauth-extension by default ("rsa"), which renders the old protection mechanisms "superchallenged" and "challenged" useless.

If the Backend login is accessed via HTTPS protocol, the "rsa" protection is redundant and can be disabled in general.

The super-/challenged options are removed, as "rsa" and "normal" are sufficient. If rsaauth was not installed the default has been "superchallenged". The new default is "normal" now.

Impact

If an installation has rsaauth disabled, the password transfer is now **Plain Text**.

Any code relying on or checking for the "superchallenged" or "challenged" option of `[BE][loginSecurityLevel]` or `[FE][loginSecurityLevel]`, will not work as expected.

Affected Installations

Any installation having set `[BE][loginSecurityLevel]` or `[FE][loginSecurityLevel]` to an empty string or to either of "superchallenged" or "challenged".

Migration

Make sure you access the Backend via HTTPS or install the rsaauth system extension.

Also refer to the TYPO3 Security Guide

Breaking: #67027 - Removed FLOW-compatibility from PackageManager

See Issue #67027

Description

The Package Manager has been simplified and trimmed down to fit the needs of the TYPO3 extensions and typical Composer packages. All shipped code backported from Flow was removed or refactored to be included in the TYPO3 Core natively. Loading classes are done with the Composer class loader or by the conventions of extension namings. All default Composer packages can still be included as usual, however the custom Flow-logic has been removed.

Impact

It is not possible to add custom Package.php loaders into TYPO3 extensions anymore to be called during runtime. It is not possible to configure extensions with custom `Classes/` directories and custom composer.json locations anymore. There is no special handling for "typo3-flow" packages anymore. The `typo3conf/PackageStates.php` file now only contains the parts that are necessary for the TYPO3 system.

Affected Installations

All installations using custom functionality of the PackageManager not in use with the TYPO3 Core, or installations trying to use Flow packages natively in the TYPO3 Core.

Migration

Use Composer packages natively for class loading, or use ext_localconf.php to additionally configure a package.

Breaking: #67204 - DatabaseConnection::exec_SELECTgetRows() may throw exception

See Issue #67204

Description

`DatabaseConnection::exec_SELECTgetRows()` validates `$uidIndexField` parameter now. If the specified field is not present in the database result an `InvalidArgumentException` is thrown.

Impact

This change will affect only broken usages of `DatabaseConnection::exec_SELECTgetRows()` with an invalid last parameter.

It is very unlikely that existing code affected by this change, since using the method in a wrong way had the consequence that it only returned the last row from the result.

Affected Installations

Any code using the `DatabaseConnection::exec_SELECTgetRows()` method with `$uidIndexField` being set to a field name not present in the queried result set.

Migration

Fix your call to the method and correct the `$uidIndexField` parameter.

Breaking: #67212 - Discard TYPO3 class loader

See Issue #67212

Description

The former TYPO3 class loader has been removed in favor of the composer class loader.

Impact

ext_autoload.php files are **not** evaluated any more. Instead all class files are registered automatically during extension installation and written into a class map file. This class map file is not changed during regular requests, but only if the extension list changes (by using the Extension Manager).

These class information files are located in the `typo3temp/autoload/` directory and will also be automatically created if they do not exist.

Non-namespaced classes with `Tx_` naming convention like `Tx_Extension_ClassName` are only resolved through the aforementioned class map, but not dynamically. This means that extension authors need to re-generate the class map files when introducing new classes. Thus it is highly recommended to use a Classes folder with PSR-4 standard class files in there.

When installing TYPO3 with composer, it also means that all extensions need to bring their own `composer.json` file with class loading information or the class loading information of all extensions need to be specified in the root `composer.json` for class loading to work properly.

Affected Installations

All installations are affected.

Migration

No migration is needed during upgrade if TYPO3 is installed in the classic way. If TYPO3 is installed in a distribution via composer, missing class loading information need to be provided in root composer.json for all extensions which do not bring their own composer.json manifest.

```
{
    "autoload": {
        "psr-4": {
            "GeorgRinger\\News\\": "typo3conf/ext/news/Classes/",
            "MyAwesomeNamespace\\IncrediExt\\": "typo3conf/ext/incredible_extension/
↪Resources/PHP/Libraries/lib/"
        }
    }
}
```

Breaking: #67229 - FormEngine related classses

See Issue #67229

Description

With the further development of FormEngine, some minor changes on PHP level have been applied:

- Class `TYPO3\CMS\T3editor\FormWizard` has been removed
- Class `TYPO3\CMS\Rtehtmlarea\Controller\FrontendRteController` has been removed
- The method signature of class `TYPO3\CMS\Utility\BackendUtility` method `getSpecConfParts` has changed

Impact

Using code will fatal or not be called any longer.

Affected Installations

If extensions use above classes or methods. Since these classes are mostly core internal it is quite unlikely any project in the wild is affected.

Migration

Use the newly introduced API.

Breaking: #67402 - Extbase AbstractDomainObject initializeObject

See Issue #67402

Description

Method `initializeObject()` has been removed from `TYPO3\CMS\Extbase\DomainObject\AbstractDomainObje`

Impact

Possible fatal error in Extbase if objects are thawed from persistence.

Affected Installations

Domain objects extending AbstractDomainObject and calling `parent::initializeObject()`. This is relatively unlikely since the default implementation of `initializeObject()` is empty.

Migration

Remove calls to `parent::initializeObject()` from own `initializeObject()` implementations.

Breaking: #67402 - Extbase AbstractDomainObject __wakeup()

See Issue #67402

Description

Method `__wakeup()` in classes extending `TYPO3\CMS\Extbase\DomainObject\AbstractDomainObject` is no longer called if objects are created when fetched from persistence.

Affected Installations

An instance is affected if own domain objects extending AbstractDomainObject implement own `__wakeup()` methods. Those methods are no longer called.

Migration

Move initialization code from `__wakeup()` to `initializeObject()`. As a bonus, dependencies have been injection at this point already.

3.5.2 Features

Feature: #59606 - Integrate Symfony/Console into CommandController

See Issue #59606

Description

The CommandController now makes use of Symfony/Console internally and provides various methods directly from the CommandController's `output` member:

- TableHelper
  - outputTable($rows, $headers = NULL)
- DialogHelper
  - select($question, $choices, $default = NULL, $multiSelect = false, $attempts = FALSE)
  - ask($question, $default = NULL, array $autocomplete = array())
  - askConfirmation($question, $default = TRUE)
  - askHiddenResponse($question, $fallback = TRUE)
  - askAndValidate($question, $validator, $attempts = FALSE, $default = NULL, array $autocomplete = NULL)
  - askHiddenResponseAndValidate($question, $validator, $attempts = FALSE, $fallback = TRUE)
- ProgressHelper
  - progressStart($max = NULL)
  - progressSet($current)
  - progressAdvance($step = 1)
  - progressFinish()

Here's an example showing of some of those functions:

```
namespace Acme\Demo\Command;

use TYPO3\CMS\Extbase\Mvc\Controller\CommandController;

/**
 * My command
 */
class MyCommandController extends CommandController {

        /**
         * @return string
         */
        public function myCommand() {
                // render a table
                $this->output->outputTable(array(
                        array('Bob', 34, 'm'),
                        array('Sally', 21, 'f'),
                        array('Blake', 56, 'm')
                ),
                array('Name', 'Age', 'Gender'));

                // select
                $colors = array('red', 'blue', 'yellow');
                $selectedColorIndex = $this->output->select('Please select one␣
↪color', $colors, 'red');
                $this->outputLine('You choose the color %s.', array($colors[
↪$selectedColorIndex]));

                // ask
                $name = $this->output->ask('What is your name?' . PHP_EOL, 'Bob',␣
↪array('Bob', 'Sally', 'Blake'));
                $this->outputLine('Hello %s.', array($name));
```

```
                // prompt
                $likesDogs = $this->output->askConfirmation('Do you like dogs?');
                if ($likesDogs) {
                        $this->outputLine('You do like dogs!');
                }

                // progress
                $this->output->progressStart(600);
                for ($i = 0; $i < 300; $i ++) {
                        $this->output->progressAdvance();
                        usleep(5000);
                }
                $this->output->progressFinish();

        }
}
```

Impact

This change does not alter the public API so it is not breaking in the strict sense. But it introduces a new behavior: Previously all output was collected in the `Cli\Response` and only rendered to the console at the end of a CLI request. Now all methods producing output (including `output()` and `outputLine()`) render the result directly to the console. If you use `$this->response` directly or let the command method return a string, the rendering is still deferred until the end of the CLI request.

Feature: #62242 - ActionMenuItemGroupViewHelper

See Issue #62242

Description

Using this ViewHelper, OptGroups can be used in the backend select field, which controls which action is selected.

Impact

The new ViewHelper can be used in all new projects. There is no interference with any part of existing code.

Examples

Usage example:

```
<f:be.menus.actionMenu>
        <f:be.menus.actionMenuItem label="Default: Welcome" controller="Default"␣
↪action="index" />
        <f:be.menus.actionMenuItem label="Community: get in touch" controller=
↪"Community" action="index" />

        <f:be.menus.actionMenuItemGroup label="Information">
                <f:be.menus.actionMenuItem label="PHP Information" controller=
↪"Information" action="listPhpInfo" />
                <f:be.menus.actionMenuItem label="Documentation" controller=
↪"Information" action="documentation" />
                <f:be.menus.actionMenuItem label="Hooks" controller="Information"␣
↪action="hooks" />
```

```
                <f:be.menus.actionMenuItem label="Signals" controller="Information
↪" action="signals" />
                <f:be.menus.actionMenuItem label="XClasses" controller="Information
↪" action="xclass" />
        </f:be.menus.actionMenuItemGroup>
</f:be.menus.actionMenu>
```

Feature: #63453 - Template support for FlashMessagesViewHelper

See Issue #63453

Description

Template support for `FlashMessagesViewHelper` has been added. This allows to define a custom rendering for flash messages.

The new attribute `as` for the `FlashMessagesViewHelper` allows to specify a variable name, which can be used within the view helper's child elements to access the flash messages.

Example usage:

```
<f:flashMessages as="flashMessages">
        <ul class="myFlashMessages">
                <f:for each="{flashMessages}" as="flashMessage">
                        <li class="alert {flashMessage.class}">
                                <h4>{flashMessage.title}</h4>
                                <span class="fancy-icon">{flashMessage.message}</
↪span>
                        </li>
                </f:for>
        </ul>
</f:flashMessages>
```

Feature: #63561 - Add TypoScript stdWrap strtotime

See Issue #63561

Description

A new TypoScript property `strtotime` is now available within `stdWrap` which allows for conversion of formatted dates to timestamp, e.g. to perform date calculations.

Possible values are `1` or any time string valid as first argument of the PHP `strtotime()` function.

Basic usage to convert date string to timestamp:

```
date_as_timestamp = TEXT
date_as_timestamp {
        value = 2015-04-15
        strtotime = 1
}
```

Convert incoming date string to timestamp, perform date calculation and output as date string again:

```
next_weekday = TEXT
next_weekday {
        data = GP:selected_date
        strtotime = + 2 weekdays
```

```
        strftime = %Y-%m-%d
}
```

Impact

The new property is available everywhere in TypoScript where `stdWrap` is applied.

Feature: #65250 - TypoScript condition add GPmerged

See Issue #65250

Description

If one uses TypoScript condition with GP then the check is with GeneralUtility::_GP() which means that if I have GET variables beginning with an extbase plugin-namespace and POST variables with the same plugin-namespace, e.g. GET: tx_demo_demo[action]=detail POST: tx_demo_demo[name]=Foo then GeneralUtility::_GP('tx_demo_demo'), as intended, will only return the array of the POST variables for that namespace. However, that results in the issue that if you check for the GET variable the check will fail.

So, instead the check should use GeneralUtility::_GPmerged()

```
[globalVar = GPmerged:tx_demo|foo = 1]
page.90 = TEXT
page.90.value = DEMO
[global]
```

Feature: #66111 - Add TemplateRootPaths support to cObject FLUIDTEMPLATE

See Issue #66111

Description

cObject FLUIDTEMPLATE has been extended with `templateRootPaths` and `templateName`. Now you can set a template name and when rendering the template this name is used together with the set format to find the template in the given templateRootPaths with the same fallback logic as layoutRootPath and partialRootPath

- templateName = string/stdWrap
- templateRootPaths = array of file paths with "EXT:" prefix support

Example 1:

```
lib.stdContent = FLUIDTEMPLATE
lib.stdContent {
        templateName = Default
        layoutRootPaths {
                10 = EXT:frontend/Resources/Private/Layouts
                20 = EXT:sitemodification/Resources/Private/Layouts
        }
        partialRootPaths {
                10 = EXT:frontend/Resources/Private/Partials
                20 = EXT:sitemodification/Resources/Private/Partials
        }
        templateRootPaths {
```

```
                10 = EXT:frontend/Resources/Private/Templates
                20 = EXT:sitemodification/Resources/Private/Templates
        }
        variable {
                foo = bar
        }
}
```

Example 2:

```
lib.stdContent = FLUIDTEMPLATE
lib.stdContent {

        templateName = TEXT
        templateName.stdWrap {
                cObject = TEXT
                cObject {
                        data = levelfield:-2,backend_layout_next_level,slide
                        override.field = backend_layout
                        split {
                                token = frontend__
                                1.current = 1
                                1.wrap = |
                        }
                }
                ifEmpty = Default
        }
        layoutRootPaths {
                10 = EXT:frontend/Resources/Private/Layouts
                20 = EXT:sitemodification/Resources/Private/Layouts
        }
        partialRootPaths {
                10 = EXT:frontend/Resources/Private/Partials
                20 = EXT:sitemodification/Resources/Private/Partials
        }
        templateRootPaths {
                10 = EXT:frontend/Resources/Private/Templates
                20 = EXT:sitemodification/Resources/Private/Templates
        }
        variable {
                foo = bar
        }
}
```

Impact

If templateName and templateRootPaths are set the template and file options are neglected.

Feature: #66173 - Allow page title edit by doubleclick

See Issue #66173

Description

A user can edit the page title in the "Page" and the "List" module by double-clicking the page header.

Impact

If a user has sufficient permissions, the page title will become a text field after double-clicking on it. Pressing the "escape" key aborts the edit, pressing "enter" submits the changes.

Feature: #66269 - Fluid: Remove ViewHelper xmlns-attributes and specified html tag

See Issue #66269

Description

With the introduction of using xmlns:* attributes to include ViewHelpers, it is possible to have IDE support for Fluid templates. However, the problem is that the xmlns:* attributes and the corresponding tag will also be rendered, which is not desired most of the time. A workaround to avoid this is to use sections. However, this solution is counter-intuitive, is not available in layouts and causes extra processing overhead.

```
<html xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers"
                xmlns:n="http://typo3.org/ns/GeorgRinger/News/ViewHelpers">

<f:section name="content">
</f:section>
```

Impact

The xmlns:* attributes for valid ViewHelper namespaces will now be removed before rendering. Such ViewHelper namespaces follow this URI pattern:

```
http://typo3.org/ns/<phpNamespace>
```

Examples:

```
http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers
http://typo3.org/ns/GeorgRinger/News/ViewHelpers
```

xmlns attributes for non-ViewHelper namespaces will be preserved.

Furthermore an additional data-attribute to HTML-Tags is introduced.

```
data-namespace-typo3-fluid="true"
```

If this attribute is specified on the HTML-Tag, the HTML-tag itself won't be rendered as well. (Also a corresponding closing tag will not be rendered for that template.) This is useful for various IDEs and HTML auto-completion.

Examples

Include ViewHelper namespaces on an existing tag (e.g. root xml tag) via xmlns attributes for Fluid and News extension.

```
<?xml version="1.0" encoding="utf-8"?>
<root xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers"
                xmlns:n="http://typo3.org/ns/GeorgRinger/News/ViewHelpers"
                xmlns:foo="http://typo3.org/foo">

        <f:if condition="{newsItem.title}">
                <f:then>
                        <n:titleTag>{newsItem.title}</n:titleTag>
```

```
                </f:then>
                <f:else>
                        <n:titleTag>News-Detail</n:titleTag>
                </f:else>
        </f:if>
</root>
```

Output is then

```
<root xmlns:foo="http://typo3.org/foo" >
        ...
</root>
```

Include ViewHelper namespaces with HTML-tag and a data-namespace-typo3-fluid="true" attribute via xmlns attributes for Fluid and News extension.

```
<html data-namespace-typo3-fluid="true"
                xmlns:f="http://typo3.org/ns/TYPO3/CMS/Fluid/ViewHelpers"
                xmlns:n="http://typo3.org/ns/GeorgRinger/News/ViewHelpers">

        <f:if condition="{newsItem.title}">
                <f:then>
                        <n:titleTag>{newsItem.title}</n:titleTag>
                </f:then>
                <f:else>
                        <n:titleTag>News-Detail</n:titleTag>
                </f:else>
        </f:if>
</html>
```

The output contains everything excluding the HTML-tag.

Feature: #66669 - BE login form API

See Issue #66669

Description

The backend login has been completely refactored and a new API has been introduced. The OpenID form has been extracted and is now using the new API as well and is completely independent of the central Core classes for the first time.

Registering a login provider

The concept of the new backend login is based on "login providers". A login provider can be registered within your `ext_localconf.php` file like this:

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']['loginProviders'][1433416020] = [
        'provider' =>␣
↪\TYPO3\CMS\Backend\LoginProvider\UsernamePasswordLoginProvider::class,
        'sorting' => 50,
        'icon-class' => 'fa-key',
        'label' => 'LLL:EXT:backend/Resources/Private/Language/locallang.xlf:login.
↪link'
];
```

The settings are defined as:

- `provider`: The login provider class name, which must implement `TYPO3\CMS\Backend\LoginProvider\LoginProviderInterface`.
- `sorting`: The sorting is important for the ordering of the links to the possible login providers on the login screen.
- `icon-class`: The font-awesome icon name for the link on the login screen.
- `label`: The label for the login provider link on the login screen.

For a new login provider you have to register a **new key** - a unix timestamp - in `$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']['loginProviders']`. If your login provider extends another one, you may only overwrite necessary settings. An example would be to extend an existing provider and replace its registered 'provider' class with your new class name.

```
$GLOBALS['TYPO3_CONF_VARS']['EXTCONF']['backend']['loginProviders'][1433416020][
↪'provider'] = CustomProviderExtendingUsernamePasswordLoginProvider::class
```

LoginProviderInterface

The LoginProviderInterface contains only one method:

```
public function render(StandaloneView $view,PageRenderer
$pageRenderer,LoginController $loginController);
```

The parameters are defined as:

- `$view`: The Fluid StandaloneView which renders the login screen. You have to set the template file and you may add variables to the view according to your needs.
- `$pageRenderer`: The PageRenderer instance provides possibility to add necessary JavaScript resources.
- `$loginController`: The LoginController instance.

The View

As mentioned above, the `render` method gets the Fluid StandaloneView as first parameter. You have to set the template path and filename using the methods of this object. The template file must only contain the form fields, not the form-tag. Later on, the view renders the complete login screen.

View requirements:

- The template must use the `Login`-layout provided by the Core `<f:layout name="Login">`.
- Form fields must be provided within the section `<f:section name="loginFormFields">`.

```
<f:layout name="Login" />
<f:section name="loginFormFields">
        <div class="form-group t3js-login-openid-section" id="t3-login-openid_url-
↪section">
                <div class="input-group">
                        <input type="text" id="openid_url" name="openid_url" value=
↪"{presetOpenId}" autofocus="autofocus" placeholder="{f:translate(key: 'openId',␣
↪extensionName: 'openid')}" class="form-control input-login t3js-clearable t3js-
↪login-openid-field" />
                        <div class="input-group-addon">
                                <span class="fa fa-openid"></span>
                        </div>
                </div>
        </div>
</f:section>
```

Examples

Within the Core you can find two best practice implementations:

1. EXT:backend, which implements the `UsernamePasswordLoginProvider` (the default)
2. EXT:openid, which implements the `OpenIdLoginProvider` and adds a second login option

Impact

All extensions which add additional fields to the login form must be updated and make use of the new BE login form API.

Feature: #66681 - CategoryRegistry: add options to set l10n_mode and l10n_display

See Issue #66681

Description

Class `CategoryRegistry->addTcaColumn` got options to set `l10n_mode` and `l10n_display`. The values can be set via:

```
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::makeCategorizable(
        $extensionKey,
        $tableName,
        'categories',
        array(
                'l10n_mode' => 'string (keyword)',
                'l10n_display' => 'list of keywords'
        )
);
```

Feature: #66697 - Add uppercamelcase and lowercamelcase to stdWrap.case

See Issue #66697

Description

To make it possible to change a value from underscored to "UpperCamelCase" or "lowerCamelCase" the options `uppercamelcase` and `lowercamelcase` are added to `stdWrap.case`.

```
tt_content = CASE
tt_content {
        key {
                field = CType
        }

        my_custom_ctype =< lib.userContent
        my_custom_ctype {
                file = EXT:site_base/Resources/Private/Templates/SomeOtherTemplate.
↪html
                settings {
                        extraParam = 1
                }
        }
```

```
        default =< lib.userContent
        default {
                file = TEXT
                file.field = CType
                file.stdWrap.case = uppercamelcase
                file.wrap = EXT:site_base/Resources/Private/Templates/|.html
        }
}
```

Feature: #66698 - Add integrity property to JavaScript files

See Issue #66698

Description

Add a property `integrity="some-hash"` to JavaScript files via TypoScript

`page.includeJSLibs.<array>.integrity = some-hash`

This patch affects the TypoScript PAGE properties

- includeJSLibs
- includeJSFooterLibs
- includeJS
- includeJSFooter

Usage:

```
page {
        includeJS {
                jQuery = fileadmin/jquery-1.10.2.min.js
                jQuery.disableCompression = 1
                jQuery.excludeFromConcatenation = 1
                jQuery.integrity = sha256-C6CB9UYIS9UJeqinPHWTHVqh/
↪E1uhG5Twh+Y5qFQmYg=
        }
}
```

Hint: Integrity hashes may be generated using https://srihash.org/.

Feature: #66709 - Add TemplateRootPaths support to Fluid/View/StandaloneView

See Issue #66709

Description

The StandaloneView is extended with `setTemplateRootPaths($templatePaths)` and `setTemplate($templateName,$throwException = TRUE)`. Now you can set a template by name.

When `setTemplate($templateName)` is called the `$templateName` is used to find the template in the given templateRootPaths with the same fallback logic as layoutRootPath and partialRootPath.

Basic example:

```
$view = GeneralUtility::makeInstance(StandaloneView::class);
$view->setLayoutRootPaths($layoutPaths);
$view->setPartialRootPaths($partialPaths);
$view->setTemplateRootPaths($templatePaths);

try {
        $view->setTemplate($templateName);
} catch (InvalidTemplateResourceException $e) {
        // no template $templateName found in given $templatePaths
        exit($e->getMessage());
}

$content = $view->render();
```

Example of rendering a email template:

```
$view = GeneralUtility::makeInstance(StandaloneView::class);
$view->setLayoutRootPaths(array(GeneralUtility::getFileAbsFileName('EXT:my_
↪extension/Resources/Private/Layouts')));
$view->setPartialRootPaths(array(GeneralUtility::getFileAbsFileName('EXT:my_
↪extension/Resources/Private/Partials')));
$view->setTemplateRootPaths(array(GeneralUtility::getFileAbsFileName('EXT:my_
↪extension/Resources/Private/Templates')));
$view->setTemplate('Email/Notification');

$emailBody = $view->render();
```

Impact

The public API of `TYPO3\CMS\Fluid\View\StandaloneView` is enhanced with the methods `setTemplateRootPaths($templatePaths)` and `setTemplate($templateName,$throwException = TRUE)`

Feature: #66822 - Allow Sprites For Backend Modules

See Issue #66822

Description

Backend Modules (both Main Modules like "Web" and Submodules like "Filelist") can now use sprites instead of images for displaying the icons in the module menu on the left side of the TYPO3 Backend.

Registering a module can now look like this (as an example the "Page" module):

```
\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::addModule(
        'web',
        'layout',
        'top',
        \TYPO3\CMS\Core\Utility\ExtensionManagementUtility::extPath($_EXTKEY) .
↪'Modules/Layout/',
        array(
                'script' => '_DISPATCH',
                'access' => 'user,group',
                'name' => 'web_layout',
                'iconIdentifier' => 'module-web',
                'labels' => array(
                        'll_ref' => 'LLL:EXT:backend/Resources/Private/Language/
↪locallang_mod.xlf',
```

```
                ),
        )
);
```

One can use any available sprite icon known to TYPO3.

Feature: #66907 - Add Data Processing to FLUIDTEMPLATE content object

See Issue #66907

Description

cObject FLUIDTEMPLATE has been extended with `dataProcessing`. This setting can be used to add one or multiple processors to manipulate data of the currently rendered content object, like tt_content or page, and fill a key/value store that will be passed as variables to the Fluid template, where every key of the key/value store will be available as variable in the Fluid template.

- dataProcessing = array of class references by full namespace

Example:

```
my_custom_ctype = FLUIDTEMPLATE
my_custom_ctype {
        templateRootPaths {
                10 = EXT:your_extension_key/Resources/Private/Templates
        }
        templateName = CustomName
        settings {
                extraParam = 1
        }
        dataProcessing {
                1 = Vendor\YourExtensionKey\DataProcessing\MyFirstCustomProcessor
                2 =␣
↪Vendor2\AnotherExtensionKey\DataProcessing\MySecondCustomProcessor
                2 {
                        options {
                                myOption = SomeValue
                        }
                }
        }
}
```

Impact

The data processors can be used in all new projects. There is no interference with any part of existing code.

Feature: #67071 - Processed files cleanup tool added in Install Tool

See Issue #67071

Description

The Install Tool now provides a new tool to remove processed files (e.g. image thumbnails) from FAL in its "Clean up" section.

The tool is useful if you change graphic-related settings or after updating GraphicsMagick/ImageMagick on the server and you want all files to be regenerated.

Feature: #67229 - FormEngine NodeFactory API

See Issue #67229

Description

The FormEngine class construct was moved to a tree approach with container classes as inner nodes and element classes (the rendering widgets) as leaves. Finding, instantiation and preparation of those classes is done via `TYPO3\CMS\Backend\Form\NodeFactory`.

This class was extended with an API to allow flexible overriding and adding of containers and elements:

Registration of new nodes and overwriting existing nodes

```
$GLOBALS['TYPO3_CONF_VARS']['SYS']['formEngine']['nodeRegistry'][1433196792] =␣
↪array(
        'nodeName' => 'input',
        'priority' => 40,
        'class' => \MyVendor\MyExtension\Form\Element\T3editorElement::class,
);
```

This registers the class `MyVendor\MyExtension\Form\Element\T3editorElement` as render class for the type `input`. It will be called to render elements of this type and must implement the interface `TYPO3\CMS\Backend\Form\NodeInterface`. The array key is the unix timestamp of the date when an registry element is added and is just used to have a unique key that is very unlikely to collide with others - this is the same logic that is used for exception codes. If more than one registry element for the same type is registered, the element with highest priority wins. Priority must be set between 0 and 100. Two elements with same priority for the same type will throw an exception.

The core extension t3editor uses this API to substitute a `type=text` field with `renderType=t3editor` from the default `TextElement` to its own `T3editorElement`.

This registry both allows completely overriding existing implementations of any existing given type as well as registration of a new `renderType` for own fancy elements. A TCA configuration for a new renderType and its nodeRegistry could look like:

```
'columns' => array(
        'bodytext' => array(
                'config' => array(
                        'type' => 'text',
                        'renderType' => '3dCloud',
                ),
        ),
),

$GLOBALS['TYPO3_CONF_VARS']['SYS']['formEngine']['nodeRegistry'][1433197759] =␣
↪array(
        'nodeName' => '3dCloud',
        'priority' => 40,
        'class' => \MyVendor\MyExtension\Form\Element\ShowTextAs3dCloudElement::
↪class,
);
```

Resolve class resolution to different render classes

In case the above API is not flexible enough, another class can be registered to resolve the final class that renders a certain element or container differently:

```
// Register FormEngine node type resolver hook to render RTE in FormEngine if␣
↪enabled
$GLOBALS['TYPO3_CONF_VARS']['SYS']['formEngine']['nodeResolver'][1433198160] =␣
↪array(
        'nodeName' => 'text',
        'priority' => 50,
        'class' => \MyVendor\MyExtension\Form\Resolver\MyTextNodeResolver::class,
);
```

This registers a resolver class at priority 50 if the type `text` should be rendered. This class must implement `TYPO3\CMS\Backend\Form\NodeResolverInterface` and can return a different class name that is called as render class. The render class in turn must implement `TYPO3\CMS\Backend\Form\NodeInterface`.

The array key is a unix timestamp of the date when this resolver code is registered. Multiple resolvers are a chain, the resolver with highest priority is asked first, and the chain is called until one resolver returns a new class name. If no resolver returns anything, the default class name will be instantiated and rendered.

Priority is between 0 and 100 and two resolvers for the same type and same priority will throw an exception.

The resolver will receive the full `globalOptions` array with all settings to make a resolve decision on all incoming values.

This API is used by core extension rtehtmlarea to route the rendering of `type=text` to its own `RichTextElement` class in case the editor is enabled for this field and for the user.

This API allows fine grained resolution of render-nodes based on any need, for instance it would be easily possible to call another different richtext implementation (eg. TinyMCE) for specific fields of own extensions based on moon phases or your fathers birthday, by adding a resolver class with a higher priority.

Warning

The internal data given to the resolver class still may change. Both the `globalOptions` and the current `renderType` values are subject to change without further notice until TYPO3 CMS 7 LTS.

Feature: #67319 - Add field "copyright" to EXT:filemetadata

See Issue #67319

Description

The field "copyright" has been added to the meta data of a FAL record

Impact

The new field can be used in all new projects. There is no interference with any part of existing code.

3.5.3 Deprecation

Deprecation: #61829 - Deprecate config.classFile DBAL option

See Issue #61829

Description

The DBAL option `config.classFile` has been marked for deprecation, and will be removed with TYPO3 CMS 8.

Impact

Using `config.classFile` option will throw a deprecation message.

Affected Installations

Installations which use a user-defined DBAL database-handler.

Migration

Load the class using the autoloader.

Deprecation: #63453 - Deprecate renderMode attribute of FlashMessagesViewHelper

See Issue #63453

Description

Deprecated `renderMode` in favor of a flexible deferred rendering of flash messages in the Fluid template. This means that flash messages should no longer contain HTML, but the HTML output can and should be adjusted in the Fluid template.

Impact

Using `renderMode` on FlashMessage output will throw a deprecation warning.

Affected Installations

All instances using the renderMode attribute in FlashMessage output.

Migration

Adjust flash messages to contain only plain text and remove the renderMode attribute in the output Templates.

```
<f:flashMessages as="flashMessages">
        <ul class="typo3-flashMessages">
                <f:for each="{flashMessages}" as="flashMessage">
                        <li class="alert {flashMessage.class}">
                                <h4>{flashMessage.title}</h4>
                                {flashMessage.message}
                        </li>
                </f:for>
        </ul>
</f:flashMessages>
```

Deprecation: #63735 - Deprecate DataHandler->checkValue_*-methods

See Issue #63735

Description

The following internal but currently public functions have been marked as deprecated:

- DataHandler->checkValue_text
- DataHandler->checkValue_input
- DataHandler->checkValue_check
- DataHandler->checkValue_radio
- DataHandler->checkValue_group_select
- DataHandler->checkValue_flex

Impact

Using these functions will throw a deprecation message.

Migration

These functions are internal and should not be used outside of the core.

Deprecation: #65344 - typo3conf/extTables.php deprecated

See Issue #65344

Description

The file `typo3conf/extTables.php` which could be used for local TCA modifications has been marked as deprecated.

Setting `$GLOBALS['TYPO3_CONF_VARS']['DB']['extTablesDefinitionScript']` together with the constant `TYPO3_extTableDef_script` are deprecated and should not be used any longer.

Impact

The options and files are typically used for "poor man" `$GLOBALS['TCA']` overrides. This is discouraged and shouldn't be used any longer.

Migration

There are two options to migrate away from `typo3conf/extTables.php` usage, the first one should be preferred:

- It is good practice to have a project / site specific extension that contains templates, TypoScript and other stuff. Create one or more dedicated extensions and use TCA overrides to apply the desired modifications. Something like `$GLOBALS['TCA']['pages']['ctrl']['hideAtCopy'] = FALSE;` should be moved from `typo3conf/extTables.php` to `typo3conf/ext/<your_extension>/Configuration/TCA/Overrides/pages.php`.

- Slot the signal `tcaIsBeingBuilt` that is emitted in `ExtensionManagementUtility.php`.

Deprecation: #66789 - options deprecated in CshViewHelper

See Issue #66789

Description

The two unused options `iconOnly` and `styleAttributes` have been marked as deprecated.

Impact

The options in the CshViewHelper have been marked as deprecated and will be removed in TYPO3 CMS 8.

Migration

Remove the options where the CshViewHelper is used.

Deprecation: #66823 - Deprecate Extbase ExtensionUtility->configureModule method

See Issue #66823

Description

The method `TYPO3\CMS\Extbase\Utility\ExtensionUtility->configureModule()` has been marked for deprecation, and will be removed with TYPO3 CMS 8.

Impact

Calling `TYPO3\CMS\Extbase\Utility\ExtensionUtility->configureModule()` will throw a deprecation message.

Affected Installations

Any installation with a third-party extension making use of `ExtensionUtility->configureModule()` directly inside e.g. ext_tables.php.

Migration

Use the 1:1 functionality in `TYPO3\CMS\Core\Utility\ExtensionManagementUtility->configureModule()` directly.

Deprecation: #66905 - Deprecate uc->classicPageEditMode and editRegularContentFromId option

See Issue #66905

Description

The BE-User uc option "classicPageEditMode" which was used prior to TYPO3 CMS 4.0 has been removed some time ago. The functionality `editRegularContentFromId` which was then triggered in EditDocumentController has been marked for deprecation.

Impact

Any direct calls using `editRegularContentFromId` via GET parameter or calling `editRegularContentFromId()` directly from a third-party extension will trigger a deprecation message.

Affected Installations

Any installation using third-party code to restore the old behaviour.

Migration

Remove calls to the functionality.

Deprecation: #66906 - Functionality for png_to_gif conversion

See Issue #66906

Description

The global option `$TYPO3_CONF_VARS[GFX][png_to_gif]` has been removed. The according functionality within `GraphicalFunctions->pngToGifByImagemagick()` has been marked for deprecation.

Impact

Any direct calls using `pngToGifByImagemagick()` will now throw a deprecation warning. All installations having the option `png_to_gif` activated will now always show png files instead of gifs when resizing PNG images in the TYPO3 Frontend.

Affected Installations

Any installation having png_to_gif activated or having third-party extensions calling `GraphicalFunctions->pngToGifByImagemagick()` directly.

Migration

Remove calls to the functionality, as the result will be a PNG. If GIF conversion is needed, the functionality needs to be implemented in a custom FAL Processor inside an extension.

Deprecation: #67029 - Deprecate page.bgImg option

See Issue #67029

Description

The option `page.bgImg` has been marked for deprecation and will be removed with TYPO3 CMS 8.

Impact

Using `page.bgImg` will throw a deprecation message.

Affected Installations

Any installation which uses this TypoScript option.

Migration

Use CSS to set a background on the body.

Deprecation: #37171 - Deprecate t3editor->isEnabled()

See Issue #37171

Description

TYPO3CMST3editorT3editor->isEnabled() has been marked as deprecated and should not be called anymore.

Impact

The method isEnabled() always returns TRUE and will be removed with TYPO3 CMS 8.

Affected Installations

Any installation using third-party code that works with t3editor and calls isEnabled().

Migration

The method call should be removed.

Deprecation: #65290 - TCA changes

See Issue #65290

Description

Some details in the main `Table Configuration Array`, `TCA`, known on PHP side as `$GLOBALS['TCA']` changed.

Simplified `types showitem` configuration using `columnsOverrides`

If a field is configured as `type` in `TCA ctrl` section, the value of this database field determines which fields are shown if opening a record in the backend. The shown fields are configured in `TCA` section `types showitem` and is a comma separated list of field names. Each field name can have 4 additional semicolon separated options, from which the last two have been dropped and moved:

Before:

```
'types' => array(
        'aType' => array(
                'showitem' => 'aField,anotherField;otherLabel;aPalette;special:
↪configuration;a-style-indicator,thirdField',
        ),
),
```

If a record is opened that has the type field set to `aType`, it would show the three fields `aField`, `anotherField` and `thirdField`. The second field `anotherField` has further configuration and shows a different label, adds an additional palette below the field referenced as `aPalette`, adds `special:configuration` as special configuration and changes the style with its last field. The last two parameters were changed: The style configuration is obsolete since 7.1 and has been removed. The special configuration is identical to the `defaultExtras` field of a `columns` field section and can be added with this name in a newly introduced array `columnsOverrides` that is parallel to `showitem` of this type:

```
'types' => array(
        'aType' => array(
                'showitem' => 'aField,anotherField;otherLabel;aPalette,thirdField',
                'columnsOverrides` => array(
                        'anotherField' => array(
                                'defaultExtras' => 'special:configuration',
                        ),
                ),
        ),
),
```

So, the 4th parameter has been transferred to `columnsOverrides` while the 5th parameter has been removed.

This change enables more flexible overrides of column configuration based on a given type. This is currently used in `FormEngine` only, so only view-related parameters must be overwritten here. It is not supported to change data handling related parameters like `type=text` to `type=select` or similar, but it is possible to change for example the number of rows shown in a `type=text` column field:

```
'types' => array(
        'aType' => array(
                'showitem' => 'aField,anotherField;otherLabel;aPalette,thirdField',
                'columnsOverrides` => array(
                        'anotherField' => array(
                                'config' => array(
                                        'rows' => 42,
                                ),
                        ),
                ),
        ),
),
```

It is also possible to remove a given configuration from the default configuration using the `__UNSET` keyword. Again, this is only supported for view-related configuration options. Changing for instance an `eval` option may cripple the PHP-side validation done by the DataHandler that checks and stores values.

```
'types' => array(
        'aType' => array(
                'columnsOverrides` => array(
```

```
                        'bodytext' => array(
                                'config' => array(
                                        'rows' => '__UNSET',
                                ),
                        ),
                ),
        ),
),
```

The above example would remove the `rows` parameter of the `bodytext` field columns configuration, so a default value would be used instead.

Simplified t3editor configuration

t3editor is no longer configured and enabled as wizard.

Configuration for a column field looked like this before:

```
'bodytext' => array(
        'config' => array(
                'type' => 'text',
                'rows' => 42,
                'wizards' => array(
                        't3editor' => array(
                                'type' => 'userFunc',
                                'userFunc' => 'TYPO3\CMS\T3editor\FormWizard->main',
↪
                                'title' => 't3editor',
                                'icon' => 'wizard_table.gif',
                                'module' => array(
                                        'name' => 'wizard_table'
                                ),
                                'params' => array(
                                        'format' => 'html',
                                        'style' => 'width:98%; height: 60%;'
                                ),
                        ),
                ),
        ),
),
```

The new configuration is simplified to:

```
'bodytext' => array(
        'exclude' => 1,
        'label' => 'aLabel',
        'config' => array(
                'type' => 'text',
                'renderType' => 't3editor',
                'format' => 'html',
                'rows' => 42,
        ),
),
```

In case t3editor was only enabled for a specific type, this was previously done with `enableByTypeConfig` within the wizard configuration and `wizards[theWizardName]` as the 4th semicolon separated parameter of the according field in section `showitem` of the `type` where t3editor should be enabled. Old configuration was:

```
'columns' => array(
        'bodytext' => array(
                'exclude' => 1,
```

```
                'label' => 'aLabel',
                'config' => array(
                        'type' => 'text',
                        'rows' => 42,
                        'wizards' => array(
                                't3editorHtml' => array(
                                        'type' => 'userFunc',
                                        'userFunc' =>
↪'TYPO3\CMS\T3editor\FormWizard->main',
                                        'enableByTypeConfig' => 1,
                                        'title' => 't3editor',
                                        'icon' => 'wizard_table.gif',
                                        'module' => array(
                                                'name' => 'wizard_table'
                                        ),
                                        'params' => array(
                                                'format' => 'html',
                                                'style' => 'width:98%; height: 60%;
↪'
                                        ),
                                ),
                        ),
                ),
        ),
),
'types' => array(
        'firstType' => array(
                'showitem' => 'bodytext;;;wizards[t3editorHtml]',
        ),
),
```

This now uses the new `columnsOverrides` feature parallel to `showitem`:

```
'columns' => array(
        'bodytext' => array(
                'config' => array(
                        'type' => 'text',
                        'rows' => 42,
                ),
        ),
),
'types' => array(
        'firstType' => array(
                'showitem' => 'bodytext',
                'columnsOverrides' => array(
                        'bodytext' => array(
                                'config' => array(
                                        'format' => 'typoscript',
                                        'renderType' => 't3editor',
                                ),
                        ),
                ),
        ),
```

Impact

TCA is automatically migrated during bootstrap of the TYPO3 core and the result is cached. In case TCA is still registered or changed in extensions with entries in `ext_tables.php`, an automatic migration of this part of `TCA` is only triggered if extension `compatibility6` is loaded. This has a performance penalty since the migration in `compatibility6` is then done on every frontend and backend script call and is not cached. It

is **strongly** advised to move remaining `TCA` changes from `ext_tables.php` to `Configuration/TCA` or `Configuration/TCA/Overrides` of the according extension and to unload `compatibility6`.

Migration

An automatic migration is in place. It throws deprecation log entries in case `TCA` had to be changed on the fly. The migration logs give hints on what exactly has changed and the final `TCA` can be inspected in the backend configuration module. If outdated flexforms are used, the migration is done within the FormEngine class construct on the fly and will throw deprecation warnings as soon as a record with outdated `TCA` flexforms is opened in the backend.

Typical migration of the 4th `showitem` parameter involves moving a RTE configuration like `richtext:rte_transform[mode=ts_css]` or the `type=text` flags `nowrap`, `fixed-font` and `enabled-tab` to `columnsOverrides`.

Deprecation: #67297 - MySQL / DBMS field type conversion

See Issue #67297

Description

The DbalDatabaseConnection class provides generic functions that translate between native MySQL field types and ADOdb meta field types. The generic functions `MySQLActualType()` and `MySQLMetaType` have been marked as deprecated and should not be used any longer.

Impact

Although these are public functions the use was probably limited to the DBAL Extension. If used however, they will trigger a deprecation message.

Migration

Use the functions `getNativeFieldType()` and `getMetaFieldType()` provided by the DBMS specifics class.

Deprecation: #67402 - Extbase AbstractDomainObject __wakeup()

See Issue #67402

Description

Method `__wakeup()` has been marked as deprecated in `TYPO3\CMS\Extbase\DomainObject\AbstractDomainObjec`

Affected Installations

An instance is affected if own domain objects extending AbstractDomainObject implement `__wakeup()` and call `parent::__wakeup()` as documented.

Migration

Remove calls to `parent::__wakeup()` from own `__wakeup()` implementations.

3.5.4 Important

Important: #66614 - Checksums for processed files have been changed

See Issue #66614

Description

The base data used for the checksum calculation of processed files has been changed. The checksum is used to identify changes which require regeneration of processed files.

Formerly the `GFX` section of the `TYPO3_CONF_VARS` was included in this base data, which caused weird problems in some cases.

With TYPO3 CMS 7.3 (and 6.2.13) this has been changed. In case you are adjusting `GFX` settings and you want processed files to be regenerated, you need to manually clean the existing processed files by using the Clean up utility in the Install Tool.

Since the base data is different now, the Core would not recognize the existing processed files as valid files and would delete those and build a new version. In case you are having a large installation, you might want to avoid this costly operation. The Install Tool provides a dedicated Upgrade Wizard for you, which avoids the expensive regeneration of processed files by updating the checksum of all existing processed files.

Note: The Upgrade Wizard is only relevant for you if you're upgrading from any TYPO3 CMS version below 7.3 or 6.2.13. Any upgrade from 7.3 or later or from 6.2.13 or later to a newer version does **not** require to run the wizard.

Important: #67248 - Clean up DataMapper::convertClassNameToTableName

See Issue #67248

Description

As a side-effect of cleaning up `DataMapper::convertClassNameToTableName` the argument `$className` is now mandatory.

Important: #67401 - Dependency Injection is now done before initializeObject()

See Issue #67401

Description

Formerly `initializeObject()` was called before the dependencies were injected when retrieving an Extbase Domain Model. This behavior didn't match either the documentation nor the behavior when using the `ObjectManager`.

With TYPO3 CMS 7.3 this has been changed, dependency injection using `@inject` annotations and `inject*()` methods is now performed **before** calling `initializeObject()` when retrieving Domain Models.

This may have impact on extensions that are relying on the reversed call order. In these cases adjustments are required to take into account that the injected objects are available.

3.6 7.2 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*

3.6.1 Breaking Changes

Breaking: #56746 - Add count methods and sort functionality to FAL drivers

See Issue #56746

Description

To improve the performance of the file list when showing (remote) storages with a lot of files and folders the sorting and ordering needs to be done by the driver. Also the pagination of the file list can be improved by moving the counting to the driver instead of fetching all files and folders objects to count them.

Impact

Installations with custom FAL drivers will break after update.

Affected installations

TYPO3 CMS 7 installations using custom FAL drivers.

Migration

The custom FAL drivers need to be updated to be in line with the updated DriverInterface.

2 new functions need to be implemented:

- `countFoldersInFolder()`
- `countFilesInFolder()`

2 functions need to be extended with the parameters $sort and $sortRev:

- `getFilesInFolder(...,$sort,$sortRev)`
- `getFoldersInFolder(...,$sort,$sortRev)`

Breaking: #63784 - Visibility and type of DataHandler->exclude_array

See Issue #63784

Description

The internal but formerly public property DataHandler->exclude_array is replaced by the protected property DataHandler->excludedTablesAndFields, which contains the combination of excluded table and field as key instead. This improves performance especially for bulk editing since many in_array()-checks can be avoided.

Impact

Extensions using the DataHandler (former TCEMain) and changing the (former public) exclude_array to change access to tables and fields cannot do so anymore. Users need to have their access-rights set properly instead.

Affected installations

Installations using extensions that read or write the undocumented array exclude_array.

Migration

Remove code accessing DataHandler->exclude_array and configure the BE-User properly.

Breaking: #64719 - Multimedia and Media cObjects and Content Types are moved to new system extension

See Issue #64719

Description

The Content Element Types "media" and "multimedia" have been extracted into one single place, which is an extension called "mediace". This extension is not installed by default but is shipped with the core.

The following Content Objects are not available anymore by default:

- MULTIMEDIA
- MEDIA
- SWFOBJECT
- FLOWPLAYER
- QTOBJECT

The Content Types "media" and "multimedia" are not available anymore by default.

The table column `tt_content.multimedia` is not available anymore by default.

Impact

Any TypoScript using any of the cObjects directly or Content Elements with the CType "media" or "multimedia" will result in no output. Existing Content Elements of this type cannot be edited anymore.

Affected installations

TYPO3 CMS 7 installations still using any of the cObjects or having Content Elements of CType "media" or "multimedia".

Migration

Install the system extension "mediace" to regain all functionality as it was before.

Breaking: #65432 - Storage of module URI in global variable has been removed

See Issue #65432

Description

Previously the URI to a module which was dispatched through mod.php was stored in a global variable `$GLOBALS['MCONF']['_']`.

In terms of cleanup of global variable usage and module configuration cleanup and streamlining, this functionality has been removed without substitution.

Impact

Any backend module code which accesses `$GLOBALS['MCONF']['_']` to get the module URI will not work any more.

Affected installations

TYPO3 CMS 7 installations using extensions with backend modules which use `$GLOBALS['MCONF']['_']`.

Migration

Extension code needs to be changed in a way that the API `BackendUtility::getModuleUrl('module_name')` is used instead of accessing `$GLOBALS['MCONF']['_']`.

Breaking: #65727 - Don't provide access to localPath of FAL files

See Issue #65727

Description

It was possible to retrieve the local path of a FAL file via TypoScript

```
a = TEXT
a.value.data = file:current:localPath
```

The localPath property has been dropped for the following reasons:

- The implementation used allow write access to the file and hence created a local copy which created useless file garbage.
- Changing this to read-only access would cause the LocalDriver to return the true local path to the file, which would open the possibility to file manipulation via "side channel" of FAL. This would make the FAL data inconsistent.

Impact

Any TypoScript using this file-property will stop working.

Affected Installations

Any installation with TypoScript using this file-property

Migration

There is no other possibility to retrieve this information. Use the FAL API.

Breaking: #65778 - MediaWizard functionality is moved to system extension "mediace"

See Issue #65778

Description

The Media Wizard Provider for the "media" Content Element Type has been moved to the same system extension.

Impact

Any extensions registering their own Media Wizards need to install the system extension "mediace" and define a dependency to this extension.

Affected installations

TYPO3 CMS 7 installations using the "MEDIA" cObject or having Content Elements of CType "media" or "multimedia" with custom media wizard providers.

Migration

Make sure to install the system extension "mediace" and rename the function calls to use the new classes, see the file `ext_localconf.php` of the extension "mediace" for example usage.

Breaking: #65922 - Move unused tt_content TCA fields to compatibility6

See Issue #65922

Description

There are some database fields defined in 'frontend' which are not used by `frontend` or `css_styled_content`. These fields have been moved to `compatibility6`.

- altText
- imagecaption
- imagecaption_position
- image_link
- longdescURL
- titleText

Affected Installations

All installations with extensions installed depending on these fields. For instance installations still using `css_styled_content` static templates of the TYPO3 CMS 4.* versions.

Migration

Add the moved TCA and SQL definitions to your own extension or install the compatibility extension `compatibility6`. The latter is not recommended and should be considered a short-term solution.

Breaking: #65939 - Backend Login Refactoring

See Issue #65939

Description

For the refactoring of the backend login we introduce Fluid as template engine and remove the old marker template.

It was necessary to remove the signal `LoginController::SIGNAL_RenderLoginForm` which will no longer be emitted.

Additionally the following methods of `LoginController` have been removed:

- `LoginController::makeLoginBoxImage`
- `LoginController::wrapLoginForm`
- `LoginController::makeLoginNews`
- `LoginController::makeLoginForm`
- `LoginController::makeLogoutForm`

Impact

The mentioned methods are no longer available and a fatal error will be triggered if used.

Affected installations

All installations which make use of the `LoginController::SIGNAL_RenderLoginForm` signal or use the removed methods: * `LoginController::makeLoginBoxImage` * `LoginController::wrapLoginForm` * `LoginController::makeLoginNews` * `LoginController::makeLoginForm` * `LoginController::makeLogoutForm`

Migration

Use the introduced Fluid view to adapt the login screen to your demands.

Breaking: #65962 - Third-party library "websvg" and the according API has been removed

See Issue #65962

Description

The third-party library "websvg" has been removed from the TYPO3 CMS Core. The according TypoScript options and the public methods within PageRenderer have been removed without substitution.

The following PHP methods within PageRenderer have been removed:

```
$pageRenderer->setSvgPath()
$pageRenderer->getSvgPath()
$pageRenderer->loadSvg()
$pageRenderer->enableSvgDebug()
$pageRenderer->svgForceFlash()
```

The following TypoScript options are removed:

```
page.javascriptLibs.SVG
page.javascriptLibs.SVG.debug
page.javascriptLibs.SVG.forceFlash
```

Impact

Any installation using one of the methods above in an extension will fail.

Any installation using `page.javascriptLibs.SVG = 1` will not include the websvg library anymore and might lead to SVGs not being displayed anymore in certain browsers. Using the SVG Content Object will lead to the same result.

Affected installations

TYPO3 CMS 7 installations using the TypoScript options, the SVG Content Object or the pageRenderer methods directly.

Migration

Affected installations should include the "websvg" library directly from the library owner, and in their setups.

Breaking: #66001 - Third-party libraries installed via composer are now in vendor

See Issue #66001

Description

All composer-installed libraries which the TYPO3 Core uses are now installed under `vendor` (composer default vendor directory) when running `composer install`. This way the packaging process for releasing TYPO3 CMS as tarball or zip can trigger a fully working installation without having to ship Packages/ for third-party libraries. Before composer installed all third-party libraries in the folder `Packages/Libraries`.

Impact

Any existing installation that is set up via composer based on the composer.json of the TYPO3.CMS.git repository will break if Unit Tests or Functional Tests via `bin/phpunit` will fail unless composer dependencies have been completely rebuilt.

Affected installations

Installations using a Packages/ directory within the typo3_src/ folder structure, most commonly by having checked out the TYPO3.CMS.git Repository and having run `composer install` after that. Any installations using the common TYPO3 distribution from composer.typo3.org are not affected.

Migration

Running `rm -rf vendor/ bin/ Packages/Libraries/ composer.lock; composer install` will make PHPunit work again.

Breaking: #66034 - Drop Content Adapter

See Issue #66034

Description

The TYPO3 configuration option [FE][activateContentAdapter] and the associated code has been dropped from the TYPO3 core. This option was used to transform FAL fields back to old file fields.

Impact

Any installation using TypoScript referring to old file columns as present *before* TYPO3 CMS 6.x will stop working as expected.

Migration

Change your TypoScript to use the new content object FILES to retrieve files.

Breaking: #66286 - Page TSconfig options to hide Web Info modules renamed

See Issue #66286

Description

WEB > Info options in the function menu have new names in Page TSconfig properties to hide these modules

Impact

Page TSconfig options in `mod.web_info.menu.function` use new class names.

Affected Installations

Installation which have options in the Info module disabled by using Page TSconfig `mod.web_info.menu.function`.

Migration

The following properties under `mod.web_info.menu.function` have to be renamed:

- tx_cms_webinfo_page -> TYPO3CMSFrontendControllerPageInformationController
- tx_cms_webinfo_lang -> TYPO3CMSFrontendControllerTranslationStatusController
- tx_belog_webinfo -> TYPO3CMSBelogModuleBackendLogModuleBootstrap
- tx_infopagetsconfig_webinfo -> TYPO3CMSInfoPagetsconfigControllerInfoPageTyposcriptConfigController
- tx_linkvalidator_ModFuncReport -> TYPO3CMSLinkvalidatorReportLinkValidatorReport
- tx_indexedsearch_modfunc1 : removed, indexed_search has its own module
- tx_indexedsearch_modfunc2 : removed, indexed_search has its own module

Breaking: #66431 - New Login Screen

See Issue #66431

Description

To keep the focus on the most recent news, the news are reduced to a single view carousel where you can slide through the latest news.

A title for the news section is not needed anymore, in result $GLOBALS['TYPO3_CONF_VARS']['BE']['loginNewsTitle'] is superfluous and was removed completely without replacement.

Impact

The news section title has been dropped without replacement. There will be no alternative section title displayed.

Affected Installations

Installations that used $GLOBALS['TYPO3_CONF_VARS']['BE']['loginNewsTitle'] to set an alternative section title for the news.

Migration

If an entry for BE/loginNewsTitle exists in your local configuration it will be removed by the SilentConfigurationUpgradeService automatically.

3.6.2 Features

Feature: #20555 - Strip empty HTML tags in HtmlParser

See Issue #20555

Description

A new functionality is introduced in the HtmlParser that allows the stripping of empty HTML tags.

It can be used in the Frontend by using the HTMLparser TypoScript configuration of stdWrap:

```
stdWrap {

        # If this is set all empty tags are stripped, unless a list of tags is␣
↪provided below.
        HTMLparser.stripEmptyTags = 1

        # This setting can be used to filter the tags that should be stripped if␣
↪they are empty.
        HTMLparser.stripEmptyTags.tags = h2, h3
}
```

It is also possible to use it in the HTMLparser_rte or HTMLparser_db in Page TSconfig:

```
# For rtehtmlarea we need to use the entry parser because otherwise the p tags will
# be converted to linebreaks during the RTE transformation.
RTE.default.proc.entryHTMLparser_db {
        stripEmptyTags = 1
        stripEmptyTags.tags = p

        # Since rtehtmlarea adds non breaking spaces in empty <p> tags we need to
        # tell the parser that   should be treated as an empty string:
        stripEmptyTags.treatNonBreakingSpaceAsEmpty = 1
}
```

Tip: Please note that the HTMLparser will strip all unknown tags by default. If you **only** want to strip empty tags, you need to set `keepNonMatchedTags` to TRUE or configure the allowed tags:

```
stdWrap {
        HTMLparser.keepNonMatchedTags = 1
        HTMLparser.stripEmptyTags = 1
        HTMLparser.stripEmptyTags.tags = h2, h3
}
```

Impact

If the configuration is not set, the HtmlParser behaves like before so there is no impact to existing systems (unless they already have used the stripEmptyTags setting for whatever reason).

Feature: #32651 - Add scheduler task to remove deleted records

See Issue #32651

Description

A new scheduler task for removing deleted records has been added. The maximum age and the affected tables are configurable in the task's settings.

Feature: #36743 - Registry for adding text extractor classes

See Issue #36743

Description

Text extraction from files is a complex task. Thus it would be un-wise to implement that over and over again if needed. By providing a registry, text extraction services can be provided to other extensions.

It is expected that there won't ever be a lot of implementations for text extractors. The core ships with an extractor for plain text files (.txt file extension).

When asking the registry to provide a text extractor for a file it will "ask" the registered text extractors whether they can read the file. The first text extractor returning TRUE will be returned and can then be used to actually read/extract text from the file.

Every registered text extractor class needs to implements the TextExtractorInterface with the following methods:

- canExtractText() gets a file reference and returns TRUE if the text extractor can extract text from that file. How this is determined is up to the text extractor, f.e. by using MIME type or file extension as indicators.
- extractText() gets a file reference and is expected to return the file's text content as string.

It is possible to register your own text extractor classes in the ext_localconf.php of an extension.

Examples

Text extractor registration

```
$textExtractorRegistry =␣
↪\TYPO3\CMS\Core\Resource\TextExtraction\TextExtractorRegistry::getInstance();
$textExtractorRegistry->registerTextExtractor(
        \TYPO3\CMS\Core\Resource\TextExtraction\PlainTextExtractor::class
);
```

Usage

```
$textExtractorRegistry =␣
↪\TYPO3\CMS\Core\Resource\TextExtraction\TextExtractorRegistry::getInstance();
$extractor = $textExtractorRegistry->getTextExtractor($file);
if ($extractor !== NULL) {
        $content = $extractor->extractText($file);
}
```

Impact

The registry on its own doesn't do anything. It provides a facility in the core that allows extensions to provide text extraction services to be used by other extensions.

Feature: #47712 - New Locking API

See Issue #47712

Description

The new Locking API follows a new approach. Due to the problem of a very scattered support of locking methods in the various operating systems, the new API introduces a locking service, which provides access to the various

locking methods. Some basic methods are shipped with the Core, but the available methods may be extended by extensions.

A locking method has to implement the `LockingStrategyInterface`. Each method has a set of capabilities, which may vary depending on the current system, and a priority.

If a function requires a lock, the locking service is asked for the best fitting mechanism matching the requested capabilities. e.g. Semaphore locking is only available on Linux systems.

Usage example

Acquire a simple exclusive lock:

```
$lockFactory = GeneralUtility::makeInstance(LockFactory::class);
$locker = $lockFactory->createLocker('someId');
$locker->acquire() || die('ups, lock couldn\'t be acquired. That should never␣
↪happen.');
...
$locker->release();
```

Some methods also support non-blocking locks:

```
$lockFactory = GeneralUtility::makeInstance(LockFactory::class);
$locker = $lockFactory->createLocker(
        'someId',
        LockingStrategyInterface::LOCK_CAPABILITY_SHARED | LockingStrategyInterface:
↪:LOCK_CAPABILITY_NOBLOCK
);
try {
        $result = $locker->acquire(LockingStrategyInterface::LOCK_CAPABILITY_
↪SHARED | LockingStrategyInterface::LOCK_CAPABILITY_NOBLOCK);
catch (LockAcquireWouldBlockException $e) {
        // some process owns the lock, let's do something else meanwhile
}
if ($result) {
        $locker->release();
}
```

Feature: #50136 - Add SVG support

See Issue #50136

Description

Added rendering support for SVG images. When an SVG image is scaled there is no processed file created but only a sys_file_processedfile record with the calculated new dimensions.

When a mask of explicit cropping is set for an SVG image, the a processed file is created like for all other images.

An extra fallback is added to ImageInfo to determine SVG dimensions when IM/GM fails. The new fallback reads the contents of the SVG file as a normal XML file and tries to find width and height in the outer tag. When no width and height are found viewBox is checked and when present the 3th and 4th value are used as width and height.

Impact

SVG is added as default supported image file extension to `$GLOBALS['TYPO3_CONF_VARS']['GFX']['imagefile_ext'`

Feature: #50501 - Extension Manager: Disable automatic installation

See Issue #50501

Description

The extension manager currently downloads and installs an extension from TER directly. Some people want to do audits or modifications before installing an extension, therefore there is now an option to disable the automatic installation process and just download an extension.

The option is located in the settings of the extension manager itself (Extensions > Installed > Extension Manager).

Feature: #59646 - Add TSFE property $requestedId

See Issue #59646

Description

A new property within the main TypoScriptFrontendController for the frontend called $requestedId stores the information about the page ID which is set before the page ID processing and resolving. It is accessible via `$TSFE->getRequestedId()`. Also see `$TSFE->fetch_the_id()` method.

Feature: #58621 - FormatCaseViewHelper

See Issue #58621

Description

Add a format case view helper to change casing of strings.

Possible modes are: * `upper` Transforms the input string to its uppercase representation * `lower` Transforms the input string to its lowercase representation * `capital` Transforms the input string to its first letter upper-cased * `uncapital` Transforms the input string to its first letter lower-cased

```
<f:format.case>Some Text with miXed case</f:format.case> renders "SOME TEXT WITH␣
↪MIXED CASE"

<f:format.case mode="capital">someString</f:format.case> renders "SomeString"
```

Impact

The new ViewHelper can be used in all new projects. There is no interference with any part of existing code.

Feature: #59646 - Add RTE configuration property buttons.link.[*type*].properties.target.default

See Issue #59646

Description

New RTE configuration property `buttons.link.[*type*].properties.target.default` may be used in Page TSconfig to configure a default target for links of a given type.

Possible link types are: page, file, url, mail, spec. More types may be provided by extensions.

Feature: #59712 - Additional params for HTMLparser userFunc

See Issue #59712

Description

It is now possible to supply additional parameters to a userFunc of the HTMLparser:

```
myobj = TEXT
myobj.value = <a href="/" class="myclass">MyText</a>
myobj.HTMLparser.tags.a.fixAttrib.class {
        userFunc = Tx\MyExt\Myclass->htmlUserFunc
        userFunc.myparam = test1
}
```

By default only the value of the processed attribute is passed to the userFunc as the first parameter:

```
function htmlUserFunc($attributeValue, HtmlParser $htmlParser) {
        // $attributeValue is set to the current attribute value "myclass"
}
```

When additional options are provided as described above, these options will be passed in the first function parameter as an array. The attribute value is passed in the array with the `attributeValue` array key.

```
function htmlUserFunc(array $params, HtmlParser $htmlParser) {
        // $params['attributeValue'] contains the current attribute value "myclass".
↪
        // $params['myparam'] is set to "test" in the current example.
}
```

Impact

If additional parameters are provided to the HTMLparser userFunc setting the first parameter passed to the called function changes from a string with the attribute value to an array containing the attributeValue key and all additional settings.

This has an impact to all installations where additional parameters are used in the userFunc setting of the HTMLparser.

Feature: #61463 - Allow processed folders in different storage

See Issue #61463

Description

The processing folder of a storage can now be a combined identifier. This makes it possible to have the processed files outside of the storage in case of a read-only storage for instance.

Impact

For existing systems there is no impact. When the processing folder is changed to a folder in a different storage you need to make sure the folder exists and is writable.

Feature: #63040 - Add RTE configuration property buttons.abbreviation.removeFieldsets

See Issue #63040

Description

The new property `buttons.abbreviation.removeFieldsets` may be used in Page TSconfig to configure the abbreviation dialogue.

If set, the listed fieldsets of the Abbreviation dialogue are not shown.

Possible values in the list are: acronym, definedAcronym, abbreviation, definedAbbreviation

Impact

The acronym tag is deprecated in HTML5. Installations that want to use the Abbreviation feature of the RTE, but do not wish to use the acronym setting tab of the Abbreviation dialogue, may set this property in the Page TSconfig of the RTE, specifying `buttons.abbreviation.removeFieldsets = acronym,definedAcronym`

Feature: #63703 - Add option to stop a running task in the scheduler

See Issue #63703

Description

The scheduler CLI has received an option to stop a running task. The new option can be used with "-s".

Feature: #64686 - Add backend user groups to backend user module

See Issue #64686

Description

The backend user groups can now be maintained in a sub-module of the "Backend users" backend module.

Feature: #65584 - Add image cropping

See Issue #65584

Description

A new functionality is introduced that allows the editor to define image cropping settings to a *sys_file_reference*.

The current support crop setting is a comma separated string defining: offsetX,offsetY,width,height

Impact

The value set for a *sys_file_reference* will be passed through to the image rendering of TYPO3 by default. The new option of *sys_file_reference* is defined as exclude field in TCA so it needs to be enabled for editors.

Disable cropping of image when used with *typoscript* rendering:

```
# Disable cropping for all images
tt_content.image.20.1.file.crop =
```

Set custom cropping setting for when used with *typoscript* rendering:

```
# Overrule/set cropping for all images
tt_content.image.20.1.file.crop = 50,50,100,100
```

Disable cropping of image when used in *fluid*:

```
<f:image image="{imageObject}" crop="" />
```

Set custom cropping setting for image when used in *fluid*:

```
<f:image image="{imageObject}" crop="50,50,100,100" />
```

Feature: #65585 - Add TCA type imageManipulation

See Issue #65585

Description

TCA type `imageManipulation` brings a image manipulation wizard to the core.

This first version brings image cropping with the possibility to set a certain aspect ratio for the cropped area. The sys_file_reference.crop property is extended and can now also hold a json string to describe the image manipulation.

The `LocalCropScaleMaskHelper` that is used by the core to create adjusted images is also adjusted to handle the new format.

Impact

There is an new TCA type column type `imageManipulation` it supports the following config:

- file_field: string, default `uid_local`
- enableZoom: bool, default `FALSE`
- allowedExtensions: string, default `$GLOBALS['TYPO3_CONF_VARS']['GFX']['imagefile_ext']`
- ratios: array, default
  - '1.7777777777777777' => '16:9',
  - '1.3333333333333333' => '4:3',
  - '1' => '1:1',
  - 'NaN' => 'Free',

When `ratios` is set in TCA the defaults are neglected.

Property `sys_file_reference.crop` can now hold a string representing a json object. `LocalCropScaleMaskHelper` checks if the it can parse the string as json. If it can it assumes it holds the properties: `x`, `y`, `width` and `height`.

Feature: #65767 - System Information Dropdown

See Issue #65767

Description

A new, extensible flyout menu item is introduced that contains information about the system TYPO3 is installed on.

Impact

In a default installation. the new flyout item will be placed between the "help" and the "user" flyout items and is accessible by administrators only.

Items

It is possible to add own system information items by creating a slot. The slot must be registered in an extension's ext_localconf.php

```
$signalSlotDispatcher = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Extbase\SignalSlot\Dispatcher::class);
$signalSlotDispatcher->connect(
        \TYPO3\CMS\Backend\Backend\ToolbarItems\SystemInformationToolbarItem::class,
↪
        'getSystemInformation',
        \Vendor\Extension\SystemInformation\Item::class,
        'getItem'
);
```

This requires the class `Item` and its method `getItem()` in EXT:extensionClassesSystemInformationItem.php:

```
class Item {
        public function getItem() {
                return array(array(
                        'title' => 'The title shown on hover',
                        'value' => 'Description shown in the list',
                        'status' => SystemInformationHookInterface::STATUS_OK,
                        'count' => 4,
                        'icon' => \TYPO3\CMS\Backend\Utility\IconUtility::
↪getSpriteIcon('extensions-example-information-icon')
                ));
        }
}
```

Due to the SignalSlot internals, the data array must be encapsulated with another array! If there is no data to return, return `NULL`.

The icon `extensions-example-information-icon` must be registered in ext_localconf.php:

```
\TYPO3\CMS\Backend\Sprite\SpriteManager::addSingleIcons(
        array(
                'information-icon' =>␣
↪\TYPO3\CMS\Core\Utility\ExtensionManagementUtility::extRelPath($_EXTKEY) .
↪'Resources/Public/Images/Icons/information-icon.png'
        ),
        $_EXTKEY
);
```

"extensions-" is a hardcoded prefix, combined with `$_EXTKEY` (e.g. "example") creates the prefix "extensions-example-" to be used with every icon being registered. Since the first parameter of `SpriteManager::addSingleIcons()` is an array, multiple icons can be registered at once.

Messages

Messages are shown at the bottom og the dropdown. An extension can provide its own slot to fill the messages:

```
$signalSlotDispatcher = \TYPO3\CMS\Core\Utility\GeneralUtility::
↪makeInstance(\TYPO3\CMS\Extbase\SignalSlot\Dispatcher::class);
$signalSlotDispatcher->connect(
        \TYPO3\CMS\Backend\Backend\ToolbarItems\SystemInformationToolbarItem::class,
↪
        'loadMessages',
        \Vendor\Extension\SystemInformation\Message::class,
        'getMessage'
);
```

This requires the class `Message` and its method `getMessage()` in EXT:extensionClassesSystemInformationMessage.php:

```
class Message {
        public function getMessage() {
                return array(array(
                        'status' => SystemInformationHookInterface::STATUS_OK,
                        'text' => 'Something went somewhere terribly wrong. Take a␣
↪look at the reports module.'
                ));
        }
}
```

Due to the SignalSlot internals, the data array must be encapsulated with another array! If there is no data to return, return `NULL`.

Feature: #65996 - Show confirm message on closing an edit form with unsaved changes

See Issue #65996

Description

A new functionality is introduced that asks the editor to confirm closing an edit form with unsaved changes.

Impact

When clicking the close icon of an edit form with unsaved changes, a confirmation modal will be shown.

Feature: #66029 - Show remaining characters below text fields

See Issue #66029

Description

The amount of remaining characters is displayed below text fields.

Impact

When focusing a form field, the amount of remaining characters gets rendered below the form field, if a TCA field definition has the "max" definition.

Feature: #66042 - Web Libraries are included via bower

See Issue #66042

Description

Web libraries like Bootstrap, jQuery or Font Awesome are now installed via bower package management, see http://bower.io/ for details on how bower is set up.

All third-party libraries needed to build final CSS or JS files that will be shipped with the core are excluded from the TYPO3 Core Git and installed via bower when building e.g. a new CSS file out of less.

For setting up a development environment working with web libraries bower (which can be installed via npm on a local machine) installs all needed dependencies defined in bower.json. The file .bowerrc describes where the files are put. To set up the third-party libraries and their dependencies, execute the following command.

```
bower install
```

For updating the code-base to a new version, the bower.json in the root directory can be adapted. Executing `bower update` will then update the third-party libraries.

Impact

Setting up a development environment when working with frontend libraries (e.g. LESS) requires npm and bower to be installed on the local machine.

Feature: #66047 - Introduce JavaScript notification API

See Issue #66047

Description

The Flashmessages API has been moved from `TYPO3.Flashmessages` to `top.TYPO3.Flashmessages` in TYPO3 CMS 7.0.

Now we introduce the new JavaScript Notification API and remove the refactoring of the FlashMessage API which was made for TYPO3 CMS 7.0.

The compatibility layer for TYPO3.FlashMessage has changed to use the new Notification API and will be removed with TYPO3 CMS 9 as before.

The new Notification API works similar to old Flashmessages, you can use it from the Top-Frame, where is it loaded one time for the complete backend.

Please look at the examples section in this document for more details.

Migration

The affected 3rd party extensions must be modified to use `top.TYPO3.Notification` instead of `top.TYPO3.Flashmessages`.

Examples:

1. Old and new syntax in general

```
// Old and deprecated:
top.TYPO3.Flashmessages.display(TYPO3.Severity.notice)

// New and the only correct way:
top.TYPO3.Notification.notice(title, message)
```

2. Notice notification

```
// duration is optional, default is 5 seconds
top.TYPO3.Notification.notice(title, message, duration)
```

3. Info notification

```
// duration is optional, default is 5 seconds
top.TYPO3.Notification.info(title, message, duration)
```

4. Success notification

```
// duration is optional, default is 5 seconds
top.TYPO3.Notification.success(title, message, duration)
```

5. Warning notification

```
// duration is optional, default is 5 seconds
top.TYPO3.Notification.warning(title, message, duration)
```

6. Error notification

```
// duration is optional, default is 0 seconds which means sticky!
top.TYPO3.Notification.error(title, message, duration)
```

Feature: #66077 - Introduce callouts to replace content alerts

See Issue #66077

Description

In several places alerts (flash messages) were used to display context information. We introduce content info boxes and replace all occurrences where flash messages were used.

Impact

We introduced a new layout for context information and added a ViewHelper to render the markup.

Examples

Simple info box with a title. Please note that the title will always be HTML encoded by the ViewHelper

```
<f:be.infobox title="Message title">your box content</f:be.infobox>
```

All options of the ViewHelper. If you pass your message as ViewHelper argument, it will also be HTML encoded.

```
<f:be.infobox title="Message title" message="your box content" state="-2" iconName=
↪"check" disableIcon="TRUE" />
```

If you really need to output HTML in your message, use the closing variant. All children of the ViewHelper will be used as message.

```
<f:be.infobox title="Message title" state="-2" iconName="check" disableIcon="TRUE">
        <h1>{AlertMessage}</h1>
</f:be.infobox>
```

Feature: #66370 - Add flexible Preview URL configuration

See Issue #66370

Description

It is now possible to configure the preview link generated for the save+view button in Backend.

This allows to have different preview URLs depending on the record type.

Common usecase is to have previews for blog or news records, but this feature now allows you to define a different preview page for content elements as well, which might be handy if those are stored in a sysfolder.

Impact

New page TSconfig is introduced. The options are:

```
TCEMAIN.preview {
        <table name> {
                previewPageId = 123
                useDefaultLanguageRecord = 0
                fieldToParameterMap {
                        uid = tx_myext_pi1[showUid]
                }
                additionalGetParameters {
                        tx_myext_pi1.special = HELLO # results in tx_myext_
↪pi1[special]
                }
        }
}
```

The `previewPageId` is the uid of the page to use for preview. If this setting is omitted the current page will be used. If the current page is not a normal page, the root page will be chosen.

The `useDefaultLanguageRecord` defaults to `1` and ensures that translated records will use the uid of the default record for the preview link. You may disable this, if your extension can deal with the uid of translated records.

The `fieldToParameterMap` is a mapping which allows you to select fields of the record to be included as GET-parameters in the preview link. The key specifies the field name and the value specifies the GET-parameter name.

Finally `additionalGetParameters` allow you to add arbitrary GET-parameters and even override others.

Predefined GET-parameters

The Core automatically sets the `no_cache` and the `L` parameter. The language matches the language of the current record. You may override each parameter by using the `additionalGetParameters` configuration option.

Feature: #66445 - Add file extension to mimeType mapping

See Issue #66445

Description

As a fix for wrong mimeType detection for SVG files without XML prologue we added a new setting to map known file extensions to mimeTypes. The new setting is `$GLOBALS['TYPO3_CONF_VARS']['SYS']['FileInfo']['fileExtensionToMimeType']` which contains an array:

```
array(
        'svg' => 'image/svg+xml'
)
```

Impact

The automatic detection for mimeTypes works great, but in some special cases not. This new setting should only be used, if the automatic detection fails.

3.6.3 Deprecation

Deprecation: #47712 - Deprecate old Locking API

See Issue #47712

Description

The old class `\TYPO3\CMS\Core\Locking\Locker` has been marked as deprecated.

The configuration option `[SYS][lockingMode]` is now marked as deprecated and only affects the old Locker class, which is unused in the Core now.

Moreover two unused methods of `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController` have b

- acquirePageGenerationLock()
- releasePageGenerationLock()

Impact

Using the old class will trigger deprecation log messages.

Migration

Use the new Locking Service API instead.

Deprecation: #51360 - Deprecate mod.tx_linkvalidator namespace in scheduler settings

See Issue #51360

Description

Using the `mod.tx_linkvalidator` namespace in the linkvalidator scheduler task settings has been marked as deprecated. To make the setting consistent with TSconfig the namespace is changed to `mod.linkvalidator`.

Impact

Using `mod.tx_linkvalidator` in the scheduler task settings will throw an deprecation log entry.

Affected installations

Instances which use the linkvalidator scheduler task and use custom TSconfig in the task settings.

Migration

Replace `mod.tx_linkvalidator` with `mod.linkvalidator` in all affected scheduler tasks.

Deprecation: #64068 - Deprecate thumbs.php and ThumbnailView

See Issue #64068

Description

Prior to the File Abstraction Layer (FAL) there was `typo3/thumbs.php` generating all preview images for the TYPO3 Backend resources. This functionality is now marked for removal in TYPO3 CMS 8, as all functionality in the core already uses the File Abstraction Layer.

Impact

Using `ThumbnailView`, `thumbs.php` or `BackendUtility::getThumbNail()` will throw a deprecation warning.

Affected installations

Any TYPO3 installation with custom extensions using one of the files / methods mentioned.

Migration

Use the File Abstraction Layer for any custom works. See `BackendUtility::thumbCode()` for inspiration.

Deprecation: #64598 - Deprecate PagePositionMap::JSimgFunc

See Issue #64598

Description

The following function has been marked as deprecated:

- `\TYPO3\CMS\Backend\Tree\View\PagePositionMap::JSimgFunc`

This method was used only in class PagePositionMap. The implemented "onmouseover" / "onmouseout" behaviour to switch between two images was dropped entirely for now. If this is needed, it should done a different way.

Impact

Using this function in a backend module will throw a deprecation message.

Affected Installations

Every Extension that uses the deprecated function.

Migration

Write own JavaScript functions for your extension to handle onmouseover and onmouseout events to switch between two images.

Deprecation: #65111 - getDynTabMenu

See Issue #65111

Description

The DocumentTemplate method `getDynTabMenu()` has been marked as deprecated.

Impact

The method has been refactored and renamed. The new method `getDynamicTabMenu()` should be used. The method `getDynTabMenu()` is now marked as deprecated.

Affected installations

All installations which make use of `DocumentTemplate::getDynTabMenu()`

Migration

Use `ModuleTemplate::getDynamicTabMenu()` instead of `DocumentTemplate::getDynTabMenu()`

Deprecation: #65283 - Deprecate show item entry point

See Issue #65283

Description

The following entry point has been marked as deprecated:

- typo3/show_item.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/show_item.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility::getModu`

Deprecation: #65288 - Deprecate "new record" entry point

See Issue #65288

Description

The following entry point has been marked as deprecated:

- typo3/db_new.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/db_new.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleU`

Deprecation: #65289 - Deprecate browser entry point

See Issue #65289

Description

The following entry point has been marked as deprecated:

- typo3/browser.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/browser.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility::getModule`

Deprecation: #65290 - Deprecate dummy entry point

See Issue #65290

Description

The following entry point has been marked as deprecated:

- typo3/dummy.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/dummy.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUr`

Deprecation: #65283 - Deprecate logout entry point

See Issue #65283

Description

The following entry point has been marked as deprecated:

- typo3/logout.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/logout.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleU`

Deprecation: #65293 - Deprecate file navigation frame entry point

See Issue #65293

Description

The following entry point has been marked as deprecated:

- typo3/alt_file_navframe.php

Impact

Using this entry point in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

`typo3/alt_file_navframe.php` will have to be refactored to `\TYPO3\CMS\Backend\Utility\BackendUtility:`

Deprecation: #64134 - Deprecate $BE_USER->OS

See Issue #64134

Description

The public property in the global object `$BE_USER->OS` has been marked as deprecated.

Affected installations

Instances with extensions that make use of the public property directly.

Migration

Use the constant `TYPO3_OS` directly.

Deprecation: #65360 - Deprecate wrong class name used in PostProcessTree Signal call

See Issue #65360

Description

In DatabaseTreeDataProvider there is a PostProcessTree signal called via SignalSlot dispatcher. The wrong class name `TYPO3\CMS\Core\Tree\TableConfiguration\TableConfiguration\DatabaseTreeDataProvider` was used prior to this change. This class name has now been marked as deprecated. The correct name is `TYPO3\CMS\Core\Tree\TableConfiguration\DatabaseTreeDataProvider`

Impact

Wrong class name was used for the PostProcessTree signal. The old one is now deprecated.

Affected installations

All installations which have signals connected to the old/wrong class name `TYPO3\CMS\Core\Tree\TableConfiguration\TableConfiguration\DatabaseTreeDataProvider`.

Migration

- Use `TYPO3\CMS\Core\Tree\TableConfiguration\DatabaseTreeDataProvider::class` instead of `TYPO3\CMS\Core\Tree\TableConfiguration\TableConfiguration\DatabaseTreeDataPro

Deprecation: #65381 - Deprecate DataHandler property "stripslashes_values"

See Issue #65381

Description

The DataHandler property `stripslashes_values` has been marked as deprecated.

Impact

A deprecation message is logged for every time DataHandler processes data if this property is set to TRUE.

Affected installations

All installations or extensions relying on the DataHandler property `stripslashes_values`.

Migration

Set the `stripslashes_values` property to FALSE and apply `stripslashes()` in the code that prepares the data if it was expected that DataHandler stripped the slashes from incoming data.

Deprecation: #65422 - Alias cObjects COBJ_ARRAY and CASEFUNC

See Issue #65422

Description

The cObject aliases `COBJ_ARRAY` (alias for `COA`) and `CASEFUNC` (alias for `CASE`) have been moved to the compatibility6 extension. The use of these aliases have been marked for deprecation.

Impact

Any usage of TypoScript using `COBJ_ARRAY` and `CASEFUNC` will not work anymore unless the compatibility6 extension is installed.

Affected installations

All installations with TypoScript in `COBJ_ARRAY` and `CASEFUNC`.

Migration

Use `COA` instead of `COBJ_ARRAY` and `CASE` instead of `CASEFUNC` in all TypoScript code. Installing ext:compatibility6 can be used as a short-term solution, although this is discouraged.

Deprecation: #65465 - Deprecate errorLog in ReferenceIndex

See Issue #65465

Description

The method `\TYPO3\CMS\Core\Database\ReferenceIndex::error()` and the according property `\TYPO3\CMS\Core\Database\ReferenceIndex::errorLog` have been marked as deprecated. It was not used and always empty.

Impact

Calling `ReferenceIndex::error()` will throw a deprecation message. This function should not be used from outside the core.

Migration

Extensions that used this property to log errors have to use their own errorLog.

Deprecation: #65913 - Deprecate $TSFE->checkFileInclude

See Issue #65913

Description

The public method in the global frontend controller `$TSFE->checkFileInclude()` has been marked as deprecated.

Affected installations

Instances with extensions that make use of the method directly.

Migration

Use the autoloader for classes or `$TSFE->tmpl->getFileName()` if needed.

Deprecation: #65934 - "Prefix Local Anchors" functionality moved to legacy extension

See Issue #65934

Description

Prefixing local anchors is not considered best practice in web sites anymore as the same is achieved with absolute prefixes for links (see `config.absRefPrefix`). Therefore the according functionality has been moved to the legacy extension EXT:compatibility6.

The following TypoScript option has been marked for deprecation:

```
config.prefixLocalAnchors
```

The following PHP methods have been marked for deprecation:

```
TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::
↪prefixLocalAnchorsWithScript()
TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::doLocalAnchorFix()
```

Impact

The content output of the TYPO3 frontend is not prefixed with local anchors anymore unless EXT:compatibility6 is loaded.

Affected installations

Any installation having the TypoScript option `config.prefixLocalAnchors` set will have different behaviour in the frontend rendering.

Migration

For TYPO3 CMS 7, installing EXT:compatibility6 brings back the existing functionality.

Deprecation: #65938 - Discourage usage of "$TSFE->anchorPrefix"

See Issue #65938

Description

The property "anchorPrefix" within TypoScriptFrontendController is set to the relative path from the public site root when `config.baseURL` is set, and can be used to prefix local anchors with that prefix. The option has been marked as deprecated in favor of using `config.absRefPrefix` when this functionality is needed.

Affected installations

All installations or extensions relying on the TypoScriptFrontendController property "anchorPrefix" running with the TypoScript option `config.baseURL` enabled.

Migration

Use the PHP code below directly to fetch the information when needing baseURL and the anchorPrefix option.

```
GeneralUtility::getIndpEnv('TYPO3_REQUEST_URL'), strlen(GeneralUtility::getIndpEnv(
↪'TYPO3_SITE_URL'))
```

Alternatively, use `config.absRefPrefix` to achieve the same result.

Deprecation: #65956 - $returnHTML parameter of DebugUtility::debugRows()

See Issue #65956

Description

The parameter `$returnHTML` of the method `\TYPO3\CMS\Core\Utility\DebugUtility::debugRows()` is not used anymore and has been marked for deprecation.

Impact

The parameter is not used anymore.

Affected installations

All method calls using this parameter are affected.

Migration

Remove the parameter in the method call.

Deprecation: #66065 - Backend Logo View Deprecated

See Issue #66065

Description

The logo view class responsible for the rendering of the TYPO3 Logo in the left corner of the backend is not in use anymore and marked for deprecation. The logic for exchanging the logo via TBE_STYLES is still available.

Impact

Installations extending `TYPO3\CMS\Backend\View\LogoView` as an XCLASS will not see any modified output anymore.

Affected Installations

Installations extending `TYPO3\CMS\Backend\View\LogoView` as an XCLASS.

Migration

As the same logic is now done in the BackendController and the main Backend Fluid Template, the template can be modified to fit the installations' needs.

Deprecation: #66223 - Backend parseTime deprecated

See Issue #66223

Description

The option to show the parse time of the rendered script on the bottom of the HTML page has been marked for deprecation and is not in use anymore.

Impact

Debug information is not shown anymore on a backend page if the member var `$parseTimeFlag` is enabled.

Affected Installations

Installations with 3rd-party extensions manually activating this option by e.g. XCLASSing or hooking into the main document template class.

Migration

Do not set the member var to `TRUE`.

Deprecation: #66431 - New Login Screen

See Issue #66431

Description

The login screen is now supporting background images and adjustable highlight colors out of the box. Settings for the login screen can now be accessed in the backend extension settings.

Since the needed settings for the login screen were moved to the backend extension configuration, `$GLOBALS['TBE_STYLES']['logo_login']` is only used as fallback. The option has been marked as deprecated and will be removed with TYPO3 CMS 8.

Impact

`$GLOBALS['TBE_STYLES']['logo_login']` will add a deprecation log message and is still used as fallback to the new option but will be removed with TYPO3 CMS 8.

Affected Installations

Installations that use `$GLOBALS['TBE_STYLES']['logo_login']` to set an alternative logo for the backend login.

Migration

Remove the `$GLOBALS['TBE_STYLES']['logo_login']` from your setup and go to the extension manager to edit the configuration for the backend extension.

3.7 7.1 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*

3.7.1 Breaking Changes

Breaking: #24900 - Remove $TYPO3_CONF_VARS[SYS][compat_version] option

See Issue #24900

Description

The option `$TYPO3_CONF_VARS[SYS][compat_version]`, which was modified on update in the Install Tool wizard, has been removed.

Any checks on `GeneralUtility::compat_version` are now made against the common constant `TYPO3_branch` instead of the former TYPO3_CONF_VARS option.

Impact

Any usage of `$TYPO3_CONF_VARS[SYS][compat_version]` where the value is different than `TYPO3_branch` will result in unexpected behaviour.

TypoScript conditions which check for older compat_version will have different behaviour now.

Affected installations

Any installation where `$TYPO3_CONF_VARS[SYS][compat_version]` was not set to the currently running version or where the value of compat_version was used to simulate behaviour of an older version. E.g. TypoScript conditions or `GeneralUtility::compat_version` in extensions.

Migration

Remove any direct usage of the option, and use the "compat_version" method within `GeneralUtility` as well as the TypoScript condition "compat_version" which gives more accurate results.

Breaking: #44879 - TypoScript inline styles from blockquote tag removed

See Issue #44879

Description

CSS Styled Content renders blockquote tags using `lib.parseFunc_RTE` TypoScript. For TYPO3 CMS 7.1, the following TypoScript lines have been removed without substitution:

lib.parseFunc_RTE.externalBlocks.blockquote.callRecursive.tagStdWrap.HTMLparser = 1
lib.parseFunc_RTE.externalBlocks.blockquote.callRecursive.tagStdWrap.HTMLparser.tags.blockquote.overrideAttribs = style="margin-bottom:0;margin-top:0;"

The effect is that the following inline styles have been removed from blockquote tags without substitution:

```
margin-bottom:0;
margin-top:0;
```

Impact

Styling of blockquote tags based on CSS Styled Content could differ after an upgrade to CMS 7.1.

Affected installations

All CMS 7.1 installations that were upgraded from 7.0 and below which use blockquote tags rendered by `lib.parseFunc_RTE`.

Migration

It is recommended to fix the margins inside your website CSS. Alternatively, you can re-add the above TypoScript lines to your website TypoScript template (not recommended).

Breaking: #57089 - Behaviour of page shortcut to "Parent of selected or current page"

See Issue #57089

Description

In former versions of TYPO3 CMS the page shortcut type "Parent of selected or current page" had a misleading label as the selected page was never taken into account, it always chose the parent of the current page.

This has been changed and the selected page is now considered, so the parent of the selected page will be used as the target page.

Impact

Assuming the supplied upgrade wizard was run, the behaviour of existing shortcuts will not change.

If you fail to run the upgrade wizard, the target of a shortcut is changed to the parent of the selected page as the selected page is now respected.

Affected installations

Any installation using shortcut pages with shortcut type "Parent of selected or current page"

Migration

Run the supplied upgrade wizard in the Install Tool if it shows up.

Breaking: #61510 - Improvement of indexed_search

See Issue #61510

Description

The extension indexed_search is improved in the backend and frontend.

Backend

Previously the functionality of indexed_search has been scattered to multiple modules. Information about indexed_search was available in a custom module in "Admin tools" and 2 sections in the "Info" module.

The complete code has been moved to a central place, which is now a custom module in the area "Web" and has been rewritten by using Extbase & Fluid. Translations and a modern UI have been added as well.

Impact

Changes in the Backend

The previous user configuration for indexed_search modules is not working anymore. Therefore editors won't see the module anymore after login.

Changes in the Frontend

The TypoScript configuration changed. If indexed_search is installed, it is automatically activated:

```
config.index_enable = 1
config.index_externals = 1
```

Affected installations

All installations using indexed_search

Migration

Backend

Reconfigure the backend users and groups if users need to see the module of indexed_search.

Breaking: #62415 - Remove deprecated disable_autocreate field in workspaces

See Issue #62415

Description

The field "disable_autocreate" from ext:workspaces has been removed.

Impact

If a 3rd party extension relies on the removed field an SQL error will be thrown.

Affected installations

An installation is affected if a 3rd party extension relies on the removed field in the database.

Migration

Remove any usage of the removed field in 3rd party extensions.

Breaking: #62886 - Removed setting config.meaningfulTempFilePrefix

See Issue #62886

Description

The setting *config.meaningfulTempFilePrefix* which was still used for images built by GIFBUILDER that were saved in typo3temp/GB/ has been removed. With this option it was possible to configure a meaningful file prefix limiting the filename prefix to a certain character length.

Impact

All files will always have their original filename fully prepended when stored in the folder typo3temp/GB/.

Affected Installations

All installations using GIFBUILDER to generate images. The files within typo3temp/GB/ will now be called with the full name of each original file before the hash and the file extension.

Migration

The setting should be removed from any TypoScript configuration because it has no impact anymore.

Breaking: #62925 - ExtJS Ext.ux.DateTimePicker removed

See Issue #62925

Description

The old ExtJS component Ext.ux.DateTimePicker has been removed and replaced with a bootstrap alternative. For technical reasons, the feature had to be removed. Thus the possibility to use "+3d" or "today" in input fields is no longer available.

Impact

Extensions which rely on Ext.ux.DateTimePicker will break.

Migration

Use the new bootstrap DateTimePicker component which can be loaded with require.js. Example implementations can be found in EXT:belog, EXT:scheduler and the FormEngine component of the TYPO3 CMS core.

Breaking: #63296 - Deprecated typo3/ files removed

See Issue #63296

Description

The following script entry points have been removed without substitution:

- typo3/file_edit.php
- typo3/file_newfolder.php
- typo3/file_rename.php
- typo3/file_upload.php
- typo3/show_rechis.php
- typo3/listframe_loader.php

The corresponding ListFrameLoaderController class is now marked as deprecated.

Impact

Any script pointing to one of these file resources will trigger a 404 server response.

Affected installations

An extension needs to be adapted in the unlikely case that it uses a link to any of the files.

Migration

The functionality of these scripts (except listframe_loader.php which is not used at all any more) have been moved to "modules". Use BackendUtility::getModuleUrl() to link to them. The module name is identical to the file name without the ".php" suffix.

e.g. BackendUtility::getModuleUrl('file_edit');

Breaking: #63310 - Web=>Functions=>Wizards moved

See Issue #63310

Description

The two module functions "Create Pages" and "Sort Pages" located within the extensions "wizard_crpages" and "wizard_sortpages" were located under Web => Functions => Wizards. This structure is now simplified as the wizards are moved one level up in Web => Functions.

Impact

Any options set via TSconfig for these wizards for the module menu have changed. The existing options don't work anymore.

Affected installations

Any installation using TSconfig like "web_func.menu.wiz" needs to be adapted.

Migration

The respective options "web_func.menu.wiz" have been moved to "web_func.menu.functions".

Breaking: #63431 - Backend toolbar refactored

See Issue #63431

Description

The upper right backend toolbar has been refactored with TYPO3 CMS 7.0. A new PHP interface and a new registration were introduced.

Impact

Extensions that are not adapted to the new interface will not show up in the toolbar anymore, but will not throw a fatal PHP error.

Method BackendController::addToolbarItem() has been marked as deprecated.

Affected installations

If a TYPO3 CMS instance uses extensions based on the old interface and registration, the according items will no longer show up in the toolbar.

Migration

Extensions must implement the new interface TYPO3CMSBackendToolbarToolbarItemInterface and must be registered in $GLOBALS['TYPO3_CONF_VARS']['BE']['toolbarItems'].

Warning: The new interface is not 100% finished, method `checkAccess()` will probably be substituted by two other methods in later versions of TYPO3 CMS.

Breaking: #63437 - Class aliases moved to legacy extension

See Issue #63437

Description

With the switch to namespaced classes in TYPO3 CMS 6.0, a class alias mechanism has been set up to support the old class names as aliases for a while. All those class aliases have been moved to the dedicated extension "compatibility6". This extension will be removed at some point during the development of TYPO3 CMS 7.

Removing the class aliases from the core results in a significant performance gain especially during first load with empty caches.

Impact

If extensions still rely on old non-namespaced class names, EXT:compatibility6 can be loaded to keep further backwards compatibility for now - with the side-effect of drained performance.

Affected installations

TYPO3 CMS 7 installations need EXT:compatibility6 loaded if old extensions are used that are still not adapted to the namespaced core classes.

Migration

During an upgrade, the "Extension check" of the install tool may find old extensions that still rely on old class names and can unload those. The backend may work again to load EXT:compatibility6, or to unload further extensions that rely on old class names.

Another option is to manually set all extensions that rely on old class names to "inactive" in typo3conf/PackageStates.php, or to set EXT:compatibility6 to "active". If a manual change is done, typo3temp/Cache directory have to be flushed afterwards.

Breaking: #63464 - Remove include_once inclusions inside ModuleFunctions

See Issue #63464

Description

The functionality to include PHP files within module functions (e.g. info module) via an `include_once` array has been removed. The API did not use the include_once array anymore and certain places were marked as deprecated since TYPO3 CMS 6.2. All module functions are using the common autoloading functionality via namespaced classes.

The following `include_once` arrays within the following modules have been removed:

- Web => Page
- Web => Page - New Content Element Wizard
- Web => Functions
- Web => Info
- Web => Template
- Web => Recycler
- User => Task Center
- System => Scheduler

Impact

Any non-API usage of the `include_once` array in any custom module function will fail.

Affected installations

Any installation with an extension using the property `$include_once` to load additional files via direct access instead of using the API via `ExtensionManagementUtility::insertModuleFunction()`.

Migration

Use the autoloader to load any custom classes inside your code, or any hooks if available in the custom module functions to include any file.

Breaking: #63687 - Web=>Functions=>Wizards moved to legacy extension

See Issue #63687

Description

Within the "Web" => "Functions" module was a nested layer called "Wizards" where "Sort pages" and "Bulk-create new pages" resided until TYPO3 CMS 7.1. These are now moved directly underneath "Web" => "Functions", so the "Wizards" module function becomes obsolete, and with it the whole extension called "func_wizards" that provided this nested layer. The module function "Wizards" has been moved to the legacy extension EXT:compatibility6. The extension "func_wizards" has been completely removed from the core.

Impact

The existing "Functions" provided by the TYPO3 CMS Core are now directly dependant and hooked into "Web" => "Functions". Any extensions using "Web" => "Functions" => "Wizards" will not show up anymore.

Affected installations

TYPO3 CMS 7 installations need EXT:compatibility6 extension loaded if old extensions are still hooked into "Web" => "Functions" => "Wizards".

Migration

Any extension hooking into "Web" => "Functions" => "Wizards" need to be adapted. In their ext_tables.php the `insertModuleFunction` call does not need to have a 5th parameter given. EXT:compatibility6 is then no longer needed.

Breaking: #63780 - Remove public properties words and word_strings from ReferenceIndex

See Issue #63780

Description

Public properties `words` and `word_strings` have been removed from class `\TYPO3\CMS\Core\Database\ReferenceIndex`. `ReferenceIndex->words` was always an empty array and `ReferenceIndex->word_strings` contained strings from input- and text fields of every record that was given to this class instance.

Impact

An extension relying on one of these public properties will fail.

Affected installations

It is unlikely that any extension used the properties words or word_strings. An instance could be checked by searching for usages of class `ReferenceIndex`.

Migration

The according logic needs to be re-implemented in an extension that used the content of these properties.

Breaking: #63818 - Removed Static file edit functionality

See Issue #63818

Description

A special TCA configuration enabled RTE fields to write its content to sections within files. The configuration was done in `defaultExtras` array, `static_write` section and documented in TCA reference->AdditionalFeatures->SpecialConfigurationOptions.

This functionality has been removed without substitution.

Impact

Content of RTE fields can no longer be written to files.

Affected installations

In the unlikely case that this feature was used by anyone its usage can be located by searching for `static_write` keyword in TCA configuration.

Migration

Move this logic elsewhere, eg. use hooks in DataHandler to write out DB content.

Breaking: #64059 - Rewritten Javascript Tree Components

See Issue #64059

Description

In the process of refactoring prototype/scripta.culo.us code and migrate these to an AMD module, the tree component and its drag&drop parts have been migrated to a RequireJS / jQuery module.

The page tree filter functionality of the prototype tree, which is not used in the TYPO3 core, was removed from the tree component.

The file `typo3/js/tree.js` has been removed, the replacement code, based on jQuery is located under EXT:backend/Resources/Public/JavaScript/LegacyTree.js.

Impact

Any usages in third party extensions that include `js/tree.js` will fail, as the tree component has been removed. Any extension using the filter part of the tree.js component will not work.

Affected installations

Any installation with its own backend module using the tree component from the core.

Migration

Rewrite any needed logic for filtering, and include the RequireJS module like in e.g. FileSystemNavigationFrameController.php, to use the tree component. If the old code is needed, the `tree.js` file and prototype need to be included as part of the extension, not from the core.

Breaking: #64070 - Removed global variable WEBMOUNTS

See Issue #64070

Description

The global variable WEBMOUNTS has been removed, as the same data from the WEBMOUNTS can always be fetched via `$GLOBALS['BE_USER']->returnWebmounts()`.

Impact

The variable `$GLOBALS['WEBMOUNTS']` will no longer be filled.

Affected installations

Any installation using `$GLOBALS['WEBMOUNTS']` directly within an extension will produce a wrong result.

Migration

Replace all occurrences of `$GLOBALS['WEBMOUNTS']` with `$GLOBALS['BE_USER']->returnWebmounts()`.

Breaking: #64102 - Move t3-table and t3-button to bootstrap

See Issue #64102

Description

In transition to full bootstrap coverage and streamlining the backend, we are dropping the support for the CSS classes .t3-table and .t3-button. We are replacing them with the corresponding bootstrap CSS classes for tables and buttons. See http://getbootstrap.com/css/#tables and http://getbootstrap.com/css/#buttons for more details.

Impact

Custom implementations of tables and buttons in backend modules will lose the TYPO3 default styling.

Affected installations

Extensions that provide custom backend modules that are using the css classes `.t3-table` or `.t3-button`.

Migration

For tables we recommend the usage of the CSS class combination `table table-striped table-hover` instead of `t3-table`.

For buttons we recommend the usage of the CSS class combination `btn btn-default` instead of `t3-button`.

Breaking: #64131 - Resizable Textarea option removed

See Issue #64131

Description

The User Setting "Resizable Textarea" for FormEngine textarea fields has been removed.

Impact

The option and its feature are no longer available.

Breaking: #64143 - Language / Country Flag files moved

See Issue #64143

Description

The static GIF file collection representing flags within `typo3/gfx/flags/` has been removed. All PNG flag images from EXT:t3skin are moved from `typo3/sysext/t3skin/images/flags/` to `typo3/sysext/core/Resources/Public/Icons/flags/`. The according stylesheets for the that `flags-*` icon files have been moved to EXT:core as well.

Impact

Any hard-coded reference on any of the files within `typo3/gfx/flags/` and `typo3/sysext/core/Resources/Public/Icons/flags/` will result in an error.

Changing the EXT:t3skin flags sprite now means changing the flags sprite of EXT:core.

Affected installations

Any installation using third-party extensions that access `typo3/gfx/flags/` or `typo3/sysext/t3skin/images/flags/` will fail.

Breaking: #64190 - FormEngine Checkbox Element limitation of cols setting

See Issue #64190

Description

The TCA configuration for checkbox cols has been changed. We reduced the number of accepted values to 1, 2, 3, 4 and 6 to provide a responsive experience.

For usecases like checkboxes for weekdays like mo, tu, we, th, fr, sa, su we introduced a new value `inline`.

Impact

For values equals 5 or above 6 the rendering of 6 will be used.

Affected installations

Installations with TCA column configurations for checkboxes with values equals 5 or above 6.

Migration

Choose between one of the supported values or change the display to `inline`.

Breaking: #64226 - Option $TYPO3_CONF_VARS[BE][accessListRenderMode] removed

See Issue #64226

Description

The `$GLOBALS[TYPO3_CONF_VARS][BE][accessListRenderMode]` option, which acted as a shorthand function to set permission-related fields for modules and excludeFields, has been removed.

All corresponding fields in TCA tables be_users and be_groups are set to the already-default value `checkbox` given in the DefaultConfiguration.php file.

Impact

All respective fields will show up as a checkbox selection within FormEngine (implying the value `renderMethod=checkbox`).

Affected installations

Installations having this option set to something different than `checkbox` will result in a having the fields displayed as checkboxes.

Migration

Choose between the default value `checkbox` (no change required then) or set the following values inside the `Configuration/TCA/Overrides` files of your project specific extension to the option of your needs.

```
$GLOBALS['TCA']['be_users']['columns']['file_permissions']['config']['renderMode']␣
↪= 'singlebox';
$GLOBALS['TCA']['be_users']['columns']['userMods']['config']['renderMode'] =
↪'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['file_permissions']['config']['renderMode
↪'] = 'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['pagetypes_select']['config']['renderMode
↪'] = 'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['tables_select']['config']['renderMode'] =
↪'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['tables_modify']['config']['renderMode'] =
↪'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['non_exclude_fields']['config']['renderMode
↪'] = 'singlebox';
$GLOBALS['TCA']['be_groups']['columns']['userMods']['config']['renderMode'] =
↪'singlebox';
```

Breaking: #64229 - Trim submitted login-form-data before usage

See Issue #64229

Description

Data submitted through the login-forms (frontend and backend) will now be trimmed before the login is performed. So now all fields (like username or password) with leading/trailing whitespace will have those removed. Any whitespace inside fields will not be touched.

Impact

Users that have had whitespace at the beginning or end of their username or password will not be able to log in anymore.

Affected installations

Any installation relying on whitespace at the beginning or end of either a username or a password. Please note that the TYPO3 backend didn't allow whitespace for username in frontend or backend and only allowed whitespace for passwords of backend users until now.

Migration

Update usernames and/or passwords.

Breaking: #64637 - Compatibility CSS Styled Content TypoScript templates removed

See Issue #64637

Description

CSS Styled Content ships compatibility TypoScript templates for being compatible with older versions. For TYPO3 CMS 7, all templates to render a compatibility frontend for the following versions have been removed without substitution.

- TYPO3 CMS 6.1
- TYPO3 CMS 6.0
- TYPO3 CMS 4.7
- TYPO3 CMS 4.6
- TYPO3 CMS 4.5

Impact

It is not possible to have the TYPO3 CMS 7 Frontend rendering based on CSS Styled Content to behave like a version prior to TYPO3 CMS 6.2.

Affected installations

All installations using the TypoScript templates to have the TYPO3 frontend powered via CSS Styled Content with a compatibility TypoScript will result in no Frontend output.

Migration

For installations still needing the old templates, an old version of the template (e.g. from TYPO3 CMS 6.2) need to be included separately. Any other installation still using the old TypoScript templates should migrate to the latest TypoScript templates of CSS Styled Content by choosing the correct version inside the Web=>Template module.

Breaking: #63687 - Outdated ContentObjects moved to legacy extension

See Issue #63687

Description

The TypoScript Content Objects (cObjects) CLEARGIF, COLUMNS, CTABLE, OTABLE and HRULER have been moved into the legacy extension EXT:compatibility6.

Impact

Any TypoScript using the cObjects above will result in an empty output in the TYPO3 Frontend.

Affected installations

TYPO3 CMS 7 installations still using the cObjects need EXT:compatibility6 to be loaded if the rendering should be continued.

Migration

Any installation should migrate to alternatives such as FLUIDTEMPLATE to customize the output of the content.

Breaking: #64643 - Remove functionality for enable_typo3temp_db_tracking

See Issue #64643

Description

The logic and the database table for tracking generated typo3temp/ images by GraphicalFunctions have been removed without substitution. The option to enable this functionality `$TYPO3_CONF_VARS[GFX][enable_typo3temp_db_tracking]` has been removed.

Impact

Images generated with GraphicalFunctions with the option above enabled will not be tracked in the database anymore.

Affected installations

Any installation having the option `$TYPO3_CONF_VARS[GFX][enable_typo3temp_db_tracking]` enabled.

Migration

If the functionality or a similar functionality is needed for some edge-cases, an own implementation via a third-party extension is necessary.

Breaking: #64668 - Content Element mailform moved to legacy extension

See Issue #64668

Description

The fallback "mailform" functionality, containing the `FORM` ContentObject, the submission logic for sending mailform and the content element (CType=mailform) has been moved to the legacy extension "compatibility6". This mailform was available when the "Form" extension, introduced in TYPO3 CMS 4.7, was not installed.

The following options have been marked for deprecation:

```
$TYPO3_CONF_VARS][FE][secureFormmail]
$TYPO3_CONF_VARS][FE][strictFormmail]
$TYPO3_CONF_VARS][FE][formmailMaxAttachmentSize]
```

The following methods within TypoScriptFrontendController have been removed:

```
protected checkDataSubmission()
protected sendFormmail()
public extractRecipientCopy()
public codeString()
protected roundTripCryptString()
```

Impact

Mailform elements are missing and not rendered in the frontend anymore unless EXT:compatibility6 is loaded.

Affected installations

Any installation using the fallback "mailform" Content Element, the FrontendDataSubmissionController or the FORM Content Object directly will break. Additionally, any third party extension using the TypoScriptFrontendController methods directly will stop working with a fatal error.

Migration

For TYPO3 CMS 7, installing EXT:compatibility6 brings back the existing functionality. For the long term the affected installations should migrate to a different, better suited solution for sending mails and building forms.

Breaking: #64671 - Outdated ContentObject IMGTEXT moved to legacy extension

See Issue #64671

Description

The TypoScript Content Object IMGTEXT has been moved to the legacy extension "compatibility6".

Impact

Any TypoScript using the cObject directly or via CSS Styled Content configured using `renderMethod=table` will result in an empty output in the TYPO3 Frontend.

Affected installations

TYPO3 CMS 7 installations still using the cObject need EXT:compatibility6 to be loaded.

Migration

Any installation should migrate to alternatives such as CSS Styled Content without a table-based rendering for text w/ image elements to customize the output of content.

Breaking: #64696 - Content Element "search" moved to legacy extension

See Issue #64696

Description

The TYPO3 default `search` functionality, which was based on the `FORM` and `SEARCHRESULTS` ContentObjects and the content element CType=search itself has been moved to the legacy extension EXT:compatibility6.

Impact

Content elements of the Type "Search" are gone and will no longer be rendered in the frontend unless EXT:compatibility6 is loaded. TypoScript using `SEARCHRESULTS` directly will return nothing.

Affected installations

Any installation using the simple "search" content element or the `SEARCHRESULTS` ContentObject directly will break.

Migration

For TYPO3 CMS 7, installing EXT:compatibility6 brings back the existing functionality. For the long term the affected installations should be migrate to a better suited solution for searching.

3.7.2 Features

Feature: #15619 - Access module: Allow selector as "unchanged"

See Issue #15619

Description

In order to only overwrite permissions in the access module and not the owner/group it is now possible to set the selector to " - leave unchanged - ".

Feature: #16794 - Linking of Indexed Search result sections

See Issue #16794

Description

Per default the section headlines of indexed search results are links. It is possible to disable those links, thus having the sections displayed as simple text.

```
plugin.tx_indexedsearch.linkSectionTitles = 0
```

Will result in not linked section headlines. The setting is per default set to 1 in order to preserve current behaviour.

Feature: #20767 - Allow nested array access on getData of field

See Issue #20767

Description

Right now the `getData` type in TS only allows to access nested arrays in types GPVar and TSFE. Now the same is allowed for `field` too.

If the field value is `array('somekey' => array('level1' => array('level2' => 'somevalue')));`, you can get the `somevalue` by configuring the following TypoScript.

```
10 = TEXT
10.data = field:fieldname|level1|level2
```

Impact

You can now access nested keys via getData `field:`. Nested keys are not available with the default set of content objects, however just content objects and `USER` object may return such a field structure.

Feature: #22086 - Add .stdWrap to page.headTag option

See Issue #22086

Description

The TypoScript setting page.headTag now has stdWrap functionality available.

The new option can be set like this:

```
page = PAGE
page.headTag = <head>
page.headTag.override = <head class="special">
page.headTag.override.if {
  isInList.field = uid
  value = 24
}
```

Feature: #24906 - Configuration for maximum chars in TextElement

See Issue #24906

Description

TCA type `text` now supports the HTML5 attribute `maxlength` to restrict text to a given maximum length. Line breaks are usually counted as two characters.

Not every browser supports this, see sites like w3schools.com for details.

The new option can be used like this:

```
'teaser' => array(
        'label' => 'Teaser',
        'config' => array(
                'type' => 'text',
                'cols' => 60,
                'rows' => 2,
                'max' => '30',
        )
),
```

Feature: #28382 - Add async property to JavaScript files

See Issue #28382

Description

Add a property `async="async"` to JavaScript files via TypoScript

```
page.includeJSlibs.<array>.async = 1
```

This patch affects the TypoScript PAGE properties

- includeJSLibs
- includeJSFooterlibs
- includeJS
- includeJSFooter

Usage:

```
page {
        includeJS {
                jsFile = /Path/To/jsFile.js
                jsFile.async = 1
        }
}
```

Feature: #33491 - Add stdWrap functionality to <title> tag

See Issue #33491

Description

The <title> tag of a frontend page can already be controlled by various settings via TypoScript. However, the stdWrap part was not available yet, but is now compliant and available as an option `config.pageTitle`. This option will be executed after all other existing processing options like `config.titleTagFunction` and `config.pageTitleFirst`.

The new option can be used like this, e.g. in order to write everything in uppercase in the title tag:

```
page = PAGE
page.config.pageTitle.case = upper
```

Feature: #34944 - PaginateViewHelper handles non-query-result objects

See Issue #34944

Description

The PaginateViewHelper accepts input collections of following types:

- `QueryResultInterface`
- `ObjectStorage`
- `\ArrayAccess`
- `array`

```
<f:widget.paginate objects="{blogs}" as="paginatedBlogs">
        <f:for each="{paginatedBlogs}" as="blog">
                <h4>{blog.title}</h4>
        </f:for>
</f:widget.paginate>
```

Feature: #35891 - FormEngine: Possibility to add icons via PageTSconfig

See Issue #35891

Description

The possibility to add a new value/label pair for a select field in FormEngine is given via the pageTSconfig option "addItems". Now, it is also possible to give the items an icon. Either with the .icon subproperty or with the separate option "altIcons".

```
TCEFORM.pages.doktype.addItems {
        13 = My Label
        13.icon = EXT:t3skin/icons/gfx/i/pages.gif
}
TCEFORM.pages.doktype.altIcons {
        123 = EXT:myext/icon.gif
}
```

If the path is not prefixed with "EXT:" it needs to be relative to the typo3/ directory.

Feature: #46624 - HMENU item selection via additionalWhere

See Issue #46624

Description

The TypoScript Content Object HMENU menu options have a new property called "additionalWhere" to allow for a more specific database query based on any page properties.

```
lib.authormenu = HMENU
lib.authormenu.1 = TMENU
lib.authormenu.1.additionalWhere = AND author!=""
...
```

Feature: #47666 - Attribute "multiple" for f:form.upload Viewhelper

See Issue #47666

Description

The Viewhelper allows now an attribute "multiple", that will provide a possibility to upload several files at once.

```
<f:form.upload property="files" multiple="multiple" />
```

Will result in the according HTML tag providing the field content as array.

Be aware, that you need to prepare the incoming value for the property mapping yourself, by writing your own TypeConverter.

Feature: #49060 - MySql Comments reflected in SchemaMigrator

See Issue #49060

Description

Comments in the `*.sql` files are stored the database table and shown by the Schema Migration Feature in Install Tool.

Existing comments are overriden with changes provided by other extensions.

Feature: #50780 - Append element browser mount points

See Issue #50780

Description

A new UserTSconfig option is introduced:

```
options.pageTree.altElementBrowserMountPoints.append
```

This option allows administrators to add additional mount points in the RTE and the Wizard element browser instead of replacing the configured database mount points of the user when using the existing UserTSconfig option:

```
options.pageTree.altElementBrowserMountPoints
```

Usage:

Set these options in User TSconfig:

```
options.pageTree.altElementBrowserMountPoints = 20,31
options.pageTree.altElementBrowserMountPoints.append = 1
```

Impact

Mount point overriding is centralized in the BackendUser object and used by element browsers of rtehtmlarea and recordlist for calculating the page tree mount points that are displayed to the user.

Feature: #52131 - Hook for end of PageRepository->init()

See Issue #52131

Description

A new hook at the very end of the `PageRepository->init()` has been introduced. This function allows manipulation of where clause in order to modify select queries that involve visibility of pages.

Register the hook as follows:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][\TYPO3\CMS\Frontend\Page\PageRepository::
↪class]['init']
```

The hook class must implement the interface `\TYPO3\CMS\Frontend\Page\PageRepositoryInitHookInterface`.

Feature: #56236 - Multiple HTTP headers of the same type in Frontend Output

See Issue #56236

Description

It is now possible to use `config.additionalHeaders` as a TypoScript array object to add multiple headers at the same time

Usage:

```
config.additionalHeaders {
        10 {
                # the header string
                header = WWW-Authenticate: Negotiate

                # replace previous headers with the same name
    # optional, default is "on"
                replace = 0

                # optional, force the HTTP response code
                httpResponseCode = 401
        }
        # always set cache headers to private, overwriting the sophisticated TYPO3␣
↪option
        20.header = Cache-control: Private
}
```

See also: http://php.net/header

Impact

The previous option `config.additionalHeaders = X-Header: ABC|X-Header2: DEF` is deprecated in favor of the more flexible solution.

Feature: #56529 - Support "has*" Functions in ObjectAccess

See Issue #56529

Description

ObjectAccess supports "is*" and "get*" functions to return property values. Another convenience method is "has*", which can be used in the same way.

Feature: #46624 - Additional HMENU browse menus

See Issue #46624

Description

The TypoScript Content Object HMENU with a `special=browse` option does not exclude "not in menu" pages nor pages that have a "no search" checkbox set. The two new options allow for more fine-grained selection of the items within the menu.

The existing option "includeNotInMenu" was not available yet for the `HMENU` with `special=browse` enabled.

```
lib.browsemenu = HMENU
lib.browsemenu.special = browse
lib.browsemenu.special.excludeNoSearchPages = 1
lib.browsemenu.includeNotInMenu = 1
...
```

Feature: #50039 - Configurable width of the Element Browser

See Issue #50039

Description

It is now possible to set the width and height of the element browser on a per-user basis via UserTSConfig.

Usage:

:: options.popupWindowSize = 400x900 options.RTE.popupWindowSize = 200x200

Impact

The Element Browser width and height can be set for better usability and user experience.

Feature: #58033 - Enable label override of checkbox and radio buttons by TSconfig

See Issue #58033

Description

Use TSconfig to override labels of radio buttons and checkboxes used in FormEngine.

For single checkboxes the key `default` is used:

```
TCEFORM.pages.hidden.altLabels.default = individual label
```

```
TCEFORM.pages.hidden.altLabels.default = LLL:path/to/languagefile.xlf:
↪individualLabel
```

For fields with multiple checkboxes, the value or the corresponding numeration (0,1,2,3) of the checkbox is used:

```
TCEFORM.pages.l18n_cfg.altLabels.0 = individual label for the first checkbox
TCEFORM.pages.l18n_cfg.altLabels.1 = individual label for the second checkbox
```

The same functionality works on radio buttons, where the "key" is the value of the radio button.

As seen in the example, hard-coded strings or references to language files are allowed.

Impact

The feature enables even more customization for FormEngine for any custom crafted backend instance.

Feature: #58366 - Add "auto" Option for config.absRefPrefix

See Issue #58366

Description

The TypoScript setting `config.absRefPrefix` can be used to allow URL rewriting like giving a hard prefix for all relative paths. As an alternative to `config.baseURL` to be set to a specific domain absRefPrefix can autodetect the site root and use that instead of manually setting this option.

Frontend:

The new option can be set like this:

```
config.absRefPrefix = auto
```

instead of hardcoded values for different environments or when moving installations in subfolders.

```
[ApplicationContext = Production]
config.absRefPrefix = /

[ApplicationContext = Testing]
config.absRefPrefix = /my_site_root/
```

As the feature only works with path prefixes, and not with host name variables from the server, the new option is also safe for multi-domain environments to avoid duplicate caching mechanism.

Impact

The new special option can be used to automatically set up installations and distributions like the Introduction Package where a site configuration is shipped with the system but might need to be adjusted.

Feature: #58929 - PageLayoutView: Add hook for tt_content_drawFooter

See Issue #58929

Description

A hook has been added to the PageLayoutView to manipulate the rendering of the footer of a content element.

Feature: #60019 - New SplFileInfo implementation with new mimeTypeGuessers hook

See Issue #60019

Description

A new class `\TYPO3\CMS\Core\Type\File\FileInfo` which extends `SplFileInfo` is now available as an API for fetching meta information from files.

Besides the native .. _SplFileInfo API: http://php.net/manual/en/class.splfileinfo.php, it provides a new method `getMimeType()` to get the mime type of a file, e.g. text/html. It uses the native PHP function `finfo_file()` and `mime_content_type()` as a fallback.

Example: Get the MIME type of a file

```
$fileIdentifier = '/tmp/foo.html';
$fileInfo = GeneralUtility::makeInstance(\TYPO3\CMS\Core\Type\File\FileInfo::class,
↪ $fileIdentifier);
echo $fileInfo->getMimeType();
// text/html
```

New Hook 'mimeTypeGuessers'

Custom implementations to determine the MIME type can be added with a hook. Register the hook as follows:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS'][\TYPO3\CMS\Core\Type\File\FileInfo::
↪class]['mimeTypeGuessers']
```

Feature: #61542 - Add two-letter ISO 639-1 keys to sys_language

See Issue #61542

Description

The handling of the languages is done by the sys_language database table, which is usually referenced via the common sys_language_uid. The commonly referenced ISO 639-1 two-letter-code is only in use when static_info_tables is installed, which brings all ISO 639-1 letter-codes in a separate table. The CMS Core uses a hard-coded dependency on the extension to retrieve the ISO codes where needed, however, already ships an empty and invisible field "static_lang_isocode" which is already supplied by the CMS Core.

As a first step to identify languages by their proper ISO 639-1 two-letter code a new DB field for sys_language called "language_isocode" is introduced, which is used in all places of the TYPO3 CMS Core.

Additionally the new TypoScript option `config.sys_language_isocode` can be used to set the existing `$TSFE->sys_language_isocode` variable via TypoScript. Previously this was done via static_info_tables.

The ISO code is also used for the language attribute of the HTML tag. Therefore the setting `config.htmlTag_langKey` is not needed anymore if it is the same as the ISO code.

Impact

Frontend: The value `$TSFE->sys_language_isocode` is now filled at any time. It can be set via TypoScript, or is automatically set if the `config.sys_language_uid` parameter is set > 0 from the language_isocode DB field.

```
# danish by default
config.sys_language_uid = 0
config.sys_language_isocode_default = da

[globalVar = GP:L = 1]
        # isocode is filled by the respective DB value from sys_language (uid 1)
        config.sys_language_uid = 1

        # you can override this of course
        config.sys_language_isocode = fr
[GLOBAL]
```

The new field can be used in any TypoScript variable like

```
page.10 = TEXT
page.10.data = TSFE:sys_language_isocode
page.10.wrap = <div class="main" data-language="|">
```

Backend:

All ISO code usages based on sys_language in the Backend (FormEngine, Translation Tools) is now done via the new field but still uses static_info_tables as fallback if already in use.

Each sys_language record is now editable with the new DB field to select the respective ISO 639-1 code.

Migration

The upgrade wizard makes sure that the new DB field is filled properly so no deprecation warnings are written if static_info_tables in conjunction with the DB field "static_lang_isocode" was used before. If this field is used in a 3rd party extension, the extension developers and site integrators need to make sure it is switched to the new DB field sys_language.language_isocode.

If a site uses multiple languages without static_info_tables each sys_language record should be modified to select the proper ISO 639-1 code for the languages.

Feature: #61725 - Hook for BackendUtility::countVersionsOfRecordsOnPage()

See Issue #61725

Description

Hook to post-process `BackendUtility::countVersionsOfRecordsOnPage` result. `BackendUtility::countVersionsOfRecordsOnPage` is used to visualize workspace states in e.g. the page tree.

Register like this:

```
$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/class.t3lib_befunc.php']['countVersionsOfRecordsOnPage'][] = 'My\Package\HookClass->hookMethod';
```

Feature: #62944 - UserFunc available as Display Condition

See Issue #62944

Description

Being able to use userFunc as `displayCondition` makes it possible to check on any imaginable condition or state. If any situation can not be evaluated with any of the existing checks the developer is free to add an own user function which provides a boolean result whether to show or hide the TCA field.

```
$GLOBALS['TCA']['tt_content']['columns']['bodytext']['displayCond'] = 'USER:Evoweb\\Example\\User\\ElementConditionMatcher->checkHeaderGiven:any:more:information';
```

Any parameters can be added (separated by colons) that are sent to the ConditionMatcher class.

Feature: #62960 - Signal for mailer initialization

See Issue #62960

Description

This signal allows for additional processing upon initialization of a mailer object, e.g. registering a Swift mailer plugin.

Registering the signal:

```
$signalSlotDispatcher = \\TYPO3\\CMS\\Core\\Utility\\GeneralUtility::
↪makeInstance(\\TYPO3\\CMS\\Extbase\\SignalSlot\\Dispatcher::class);
$signalSlotDispatcher->connect(
        \\TYPO3\\CMS\\Core\\Mail\\Mailer::class,
        'postInitializeMailer',
        \\Vendor\\Package\\Slots\\MailerSlot::class,
        'registerPlugin'
);
```

The slot class:

```
<?php
namespace Vendor\\Package\\Slots;

use TYPO3\\CMS\\Core\\Mail\\Mailer;

class MailerSlot {
        /**
         * @param Mailer $mailer
         * @return void
         */
        public function registerPlugin(Mailer $mailer) {
                // Processing here
        }
}
```

Impact

Extensions may now perform arbitrary processing for every mail.

Feature: #63207 - Split buttons into two groups

See Issue #63207

Description

The action buttons in Web>List for pages and content elements have been split into two groups to organize the actions as primary and secondary actions. Primary actions are common RUD actions: - Show - Edit - Hide - Delete - Move up/down

Secondary actions keep any other action. If "Extended view" is disabled, the primary actions are now still displayed, the secondary action are collapsed but can be expanded by clicking the expand trigger.

Impact

Existing hooks will work like before. If an action is added to one of the two sections, the icon on rootlevel must be reset, please see *Migration*.

Migration

```
unset($cells['edit']);
$cells['primary']['edit'] = '<a class="btn btn-default"><span class="t3-icon fa fa-
↪trash"></span></a>';
$cells['secondary']['edit'] = '<a class="btn btn-default"><span class="t3-icon fa␣
↪fa-trash"></span></a>';
```

Icons, that are not added into a section, will be sorted into the secondary group.

Feature: #61489 - Allow own TypoScript Conditions in Backend as well

See Issue #61489

Description

It is now possible to add own TypoScript conditions via a separate API in the Backend. The API for the frontend was introduced with TYPO3 CMS 7.0.

An extension/package can now ship an implementation of a new abstract class AbstractCondition. Via the existing TypoScript Condition Syntax the class is called by the simple fully namespaced class name. The class' main function "matchCondition" can flexibly evaluate any parameters given after the class name.

Usage:

```
[BigCompanyName\TypoScriptLovePackage\MyCustomTypoScriptCondition]

[BigCompanyName\TypoScriptLovePackage\MyCustomTypoScriptCondition = 7]

[BigCompanyName\TypoScriptLovePackage\MyCustomTypoScriptCondition = 7, != 6]

[BigCompanyName\TypoScriptLovePackage\MyCustomTypoScriptCondition = {$mysite.
↪myconstant}]
```

where the TypoScript Condition class deals with =/!= etc itself.

Impact

If you've previously used the `userFunc` condition, you are encouraged to use this new API for your own TypoScript conditions.

Feature: #63729 - API for Twitter Bootstrap modals

See Issue #63729

Description

Actions that require a users' attention must be visualized by modal windows. TWBS provides those, but extension authors or core developers must take care of its creation and handling themselves.

This API provides a basis to create modal windows with severity representation. For a better UX, if actions (buttons) are attached to the modal, one button must be a positive action. This button should get a btnClass and set as active.

Modals should be used rarely and only for confirmations. For information the TYPO3.Flashmessage API should be used. For complex content, like forms or a lot of information, please use normal pages.

Impact

API

The API provides only two public methods:

1. `TYPO3.Modal.confirm(title,content,severity,buttons)`
2. `TYPO3.Modal.dismiss()`

Modal Settings

Name	DataType	Mandatory	Description
title	string	Yes	The title displayed in the modal
content	string|jQuery	Yes	The content displayed in the modal
severity	int		Represents the severity of a modal. Please see TYPO3.Severity. Default is `TYPO3.Severity.info`.
buttons	object[]		Actions rendered into the modal footer. If empty, the footer is not rendered. See table below.

Button Settings

Name	DataType	Mandatory	Description
text	string	Yes	The text rendered into the button.
trigger	function	Yes	Callback that's triggered on button click.
active	bool		Marks the button as active. If true, the button gets the focus.
btnClass	string		The css class for the button

Data-Attributes

It is also possible to use data-attributes to trigger a modal. e.g. on an anchor element, which prevents the default behavior.

Name	Description
data-title	the title text for the modal
data-content	the content text for the modal
data-severity	the severity for the modal, default is info (see TYPO3.Severity.*)
data-href	the target URL, default is the href attribute of the element
data-button-close-text	button text for the close/cancel button
data-button-ok-text	button text for the ok button

`class="t3js-modal-trigger"` marks the element as modal trigger

Examples

A basic modal without any specials can be created this way:

```
TYPO3.Modal.confirm('The title of the modal', 'This the the body of the modal');
```

A modal as warning with button:

```
TYPO3.Modal.confirm('Warning', 'You may break the internet!', TYPO3.Severity.
↪warning, [
        {
                text: 'Break it',
                active: true,
                trigger: function() {
                        // break the net
                }
        }, {
                text: 'Abort!',
                trigger: function() {
                        TYPO3.Modal.dismiss();
                }
```

```
        }
]);
```

A modal as warning:

```
TYPO3.Modal.confirm('Warning', 'You may break the internet!', TYPO3.Severity.
↪warning);
```

A modal triggered on an anchor element:

```
<a href="delete.php" class="t3js-modal-trigger" data-title="Delete" data-content=
↪"Really delete?">delete</a>
```

Feature: #63913 - Allow ContainerViewHelper to load RequireJS modules

See Issue #63913

Description

The ContainerViewHelper can load RequireJS modules via the `includeRequireJsModules` attribute. The scripts are passed as array.

```
<f:be.container pageTitle="Extension Module" loadJQuery="true"
        includeRequireJsModules="{
        0:'TYPO3/CMS/Extension/Module',
        1:'TYPO3/CMS/Extension/Module2',
        2:'TYPO3/CMS/Extension/Module3',
        3:'TYPO3/CMS/Extension/Module4'
}">
```

Feature: #64031 - JavaScript Storage API

See Issue #64031

Description

Accessing the Backend User configuration ($BE_USER->uc) can be handled in JavaScript with a common and simple key-value storage manner, allowing to store any data. Additionally, the use of HTML5s localStorage allows to store any data in the same way inside the browser. All localStorage data is prefixed with "t3-" in order to avoid collisions with other data from the same browserStorage.

Impact

API Methods

The API provides two objects available in the top frame attached to the global TYPO3 object:

1. `top.TYPO3.Storage.Client`
2. `top.TYPO3.Storage.Persistent`

Each object has the following API methods

- `get(key)` To fetch the data behind the key.
- `set(key,value)` To set/override a key with any arbitrary content.

- `isset(key)` (bool) checks if the key is in use.
- `clear()` to empty all data inside the storage.

Examples

To fetch data from the persistent user configuration, simple use the key known already:

```
top.TYPO3.Storage.Persistent.get('startModule');
```

Storing / Updating data in the storage works like this, and can contain any data type.

```
top.TYPO3.Storage.Persistent.set('startModule', 'web_info');
```

The same is possible for browserStorage using top.TYPO3.Storage.Client.

Feature: #64190 - Inline rendering for FormEngine Checkbox Element

See Issue #64190

Description

The checkbox setting `inline` for cols can be used to render the checkboxes directly next to each other to reduce the amount of used space.

Example Configuration:

```
'weekdays' => array(
        'label' => 'Weekdays',
        'config' => array(
                'type' => 'check',
                'items' => array(
                        array('Mo', ''),
                        array('Tu', ''),
                        array('We', ''),
                        array('Th', ''),
                        array('Fr', ''),
                        array('Sa', ''),
                        array('Su', ''),
                ),
                'cols' => 'inline',
        ),
),
```

Impact

Checkboxes will be placed directly next to each other to reduce the amount of used space.

Feature: #64257 - Support multiple UID in PageRepository::getMenu()

See Issue #64257

Description

An array of `uid` values can be passed to `PageRepository::getMenu()`, providing the chance to build menus from several roots.

Example: Fetch children of pages #2 and #3:

```
$pageRepository = new \TYPO3\CMS\Frontend\Page\PageRepository();
$pageRepository->init(false);
$rows = $pageRepository->getMenu(array(2, 3));
```

Feature: #64386 - Public Content Object Registration

See Issue #64386

Description

A new global option is now available to register and/or extend/overwrite content objects like TEXT. A list of all available Content Objects that was previously registered within the main ContentObjectRenderer class is now moved to the global array `$GLOBALS['TYPO3_CONF_VARS']['FE']['ContentObjects']` which allows for modifications via third-party extensions.

Example: Register a new Content Object EXAMPLE in a third-party extension

```
$GLOBALS['TYPO3_CONF_VARS']['FE']['ContentObjects']['EXAMPLE'] =␣
↪Acme\MyExtension\ContentObject\ExampleContentObject::class
```

The registered class must be a subclass of `TYPO3\CMS\Frontend\ContentObject\AbstractContentObject`.

For future autoloading mechanisms, it is encouraged to place the custom ContentObject class inside `EXT:myextension/Classes/ContentObject/`.

Feature: #64921 - Needed changes for flexible configuration of submodules

See Issue #64921

Description

The additional configuration for submodules can now be configured with an extra parameter in `ExtensionManagementUtility::addModule()` since https://forge.typo3.org/issues/62880. This makes it possible to remove the conf.php file by setting the configuration within `ExtensionManagementUtility::addModule()` in ext_tables.php.

When doing so you might have some issues with $this->MCONF not being set. This happens if your backend module extends from `\TYPO3\CMS\Backend\Module\BaseScriptClass`.

To fix this problem you need to add the module name in $this->MCONF.

```
/**
 * The name of the module
 *
 * @var string
 */
protected $moduleName = 'file_list';

/**
 * Constructor
 */
```

```
public function __construct() {
        $this->MCONF = array(
                'name' => $this->moduleName
        );
}
```

Feature: #63729 - Introduce GruntJS

See Issue #63729

Description

In order to unify build processes in the backend we introduce GruntJS as a central taskrunner. It will provide a global config that takes over the responsibility for all build processes in the future starting with the less files of the backend skin.

The build files are located in the *Build* folder located in the root directory.

For detailed informations about setting up GruntJS please head to http://gruntjs.com/.

Initial setup

```
npm install
npm install -g grunt-cli
```

Registered Tasks

Compiling Less files:

```
grunt less
```

Watching Less files:

```
grunt watch
```

3.7.3 Deprecation

Deprecation: #24387 - TypoScript option config.xhtmlDoctype=xhtml_2

See Issue #24387

Description

The TypoScript option `config.xhtmlDoctype = xhtml_2` is marked for removal in CMS 8, due to the W3C decision to fully work on HTML5 instead of XHTML2. See http://www.w3.org/MarkUp/ and http://www.w3.org/News/2010.html#entry-8982 for more details.

Affected installations

Any TYPO3 installation with TypoScript that relies on `config.xhtmlDoctype = xhtml_2`.

Migration

Use other doctypes like html5 to render the frontend of the TYPO3 site.

Deprecation: #25112 - Deprecate TypoScript property "andWhere"

See Issue #25112

Description

The select-property `andWhere` has been marked as deprecated.

Impact

A deprecation message is logged for every usage of this property.

Affected installations

All installations or extensions using the select-property `andWhere`.

Migration

Use the properties `where` and `markers` instead.

```
page.30 = CONTENT
page.30 {
        table = tt_content
        select {
                pidInList = this
                orderBy = sorting
                where {
                        dataWrap = sorting>{field:sorting}
                }
        }
}
page.60 = CONTENT
page.60 {
        table = tt_content
        select {
                pidInList = 73
                where = header != ###whatever###
                orderBy = ###sortfield###
                markers {
                        whatever.data = GP:first
                        sortfield.value = sor
                        sortfield.wrap = |ting
                }
        }
}
```

Deprecation: #46523 - BackendUtility::implodeTSParams()

See Issue #46523

Description

The method `TYPO3\CMS\Backend\Utility\BackendUtility::implodeTSParams()` has been marked as deprecated and will be removed in TYPO3 CMS 8.

Impact

Any installation with third-party extensions using this method will throw a deprecation warning.

Affected installations

Any installation with third-party extensions using the method.

Deprecation: #46770 - Deprecate LocalImageProcessor::getTemporaryImageWithText

See Issue #46770

Description

The public method `LocalImageProcessor::getTemporaryImageWithText()` has been marked as deprecated, it is directly replaced by `\TYPO3\CMS\Core\Imaging\GraphicalFunctions::getTemporaryImageWit`

Impact

Calling `LocalImageProcessor::getTemporaryImageWithText()` will trigger a deprecation log message.

Affected installations

TYPO3 Installations with custom extensions using the API of the File Abstraction Layer for custom image processing.

Migration

Replace all calls to the method to the `LocalImageProcessor` with an instantiation of `GraphicalFunctions` and a call to `getTemporaryImageWithText()` on the `GraphicalFunctions` object.

Deprecation: #49247 - Deprecate TypoScript functions "textStyle" and "tableStyle"

See Issue #49247

Description

The following stdWrap methods have been marked as deprecated:

- textStyle
- tableStyle

Impact

Usage of the mentioned stdWrap methods is deprecated.

Affected installations

All installations or extensions using the stdWrap methods `textStyle` and `tableStyle`.

Migration

Use CSS instead.

Deprecation: #60559 - makeLoginBoxImage()

See Issue #60559

Description

Method `TYPO3\CMS\Backend\Controller::makeLoginBoxImage()` has been marked as deprecated.

Impact

Backend login images are no longer rendered. The method body is empty and does not return rendered HTML any longer.

Affected installations

The method was unused with default backend login screen for a long time already, an installation is only affected if a 3rd party extension was loaded that changed the default login screen and used `makeLoginBoxImage()` or the template marker `LOGINBOX_IMAGE`.

Migration

Free an affected 3rd party extension from using this method or unload the extension.

Deprecation: #61605 - Change naming of TypoScript property page.includeJSlibs

See Issue #61605

Description

The existing TypoScript option `page.includeJSlibs` has been renamed to `page.includeJSLibs` to follow the lower camel case naming scheme. The existing property has been marked as deprecated.

Impact

The old property will be removed with CMS 8 and should be avoided if it has been used before.

Affected Installations

Any installation using the `page.includeJSlibs` option.

Migration

Search and replace all TypoScript code of the installation from `includeJSlibs` to `includeJSLibs`.

Deprecation: #62329 - Deprecate DocumentTable::table()

See Issue #62329

Description

`DocumentTable::table()` has been marked as deprecated.

Impact

Calling `table()` of the `DocumentTable` class will trigger a deprecation log message.

Affected installations

Instances which use `DocumentTable::table()` for rendering tables.

Migration

Use fluid for rendering instead.

Deprecation: #62855 - "XHTML cleaning" functionality moved to legacy extension

See Issue #62855

Description

XHTML cleaning is not necessary anymore, since modern technology bases completely on HTML5. Therefore the according functionality has been moved to the legacy extension EXT:compatibility6.

The following TypoScript option has been marked for deprecation:

```
config.xhtml_cleaning
```

The following PHP method has been marked for deprecation:

```
TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::doXHTML_cleaning()
```

Impact

The content output of the TYPO3 frontend is not cleaned anymore unless EXT:compatibility6 is loaded.

Affected installations

Any installation having the TypoScript option `config.xhtml_cleaning` set will have different behaviour in the frontend rendering.

Migration

For TYPO3 CMS 7, installing EXT:compatibility6 brings back the existing functionality.

Deprecation: #62864 - DataHandler->include_filefunctions deprecated

See Issue #62864

Description

The public property `->include_filefunctions` has been marked as deprecated. It has not been formally defined and was only created dynamically in the code.

Impact

In the history of the core this property has never been used (read). Extensions might have used it.

Affected installations

All installations running extensions that rely on reading this property. Currently no affected extensions are known.

Migration

If your extension needs to know whether the BasicFileUtility has been instantiated in DataHandler it could use `$datahandler->fileFunc instanceof \TYPO3\CMS\Core\Utility\File\BasicFileUtility`

Deprecation: #63522 - Deprecate the "device" TypoScript condition

See Issue #63522

Description

Client related TypoScript conditions the `device` type condition have been marked as deprecated.

Impact

Using a condition like `[device = wap]` is considered outdated and should be solved differently.

Affected installations

Instances with TypoScript that rely on `[device = ...]`.

Migration

- Most usual conditions for specific browsers can nowadays be turned into conditional CSS includes
- Use libraries such as modernizr for browser support
- If conditions for specific clients or devices are still needed, they should be done with a userFunc condition and a project like WURFL that keep the device information more recent than the current core code like matching AMIGA

Deprecation: #64059 - Non-ExtJS Page Tree Navigation Frame

See Issue #64059

Description

The non-ExtJS page tree navigation frame which was used in the core until TYPO3 CMS 4.5, is still available and can be included within a module, if the module is registering a navFrameScript in ext_tables.php:

```
'navFrameScript' => 'alt_db_navframe.php'
```

Impact

Usage of the PHP class, and the entry script typo3/alt_db_navframe.php has been marked as deprecated.

Affected installations

All installations with extensions using modules with the non-ExtJS page tree navigation frame.

Migration

Use the ExtJS navigationComponentID instead within the module registration.

Deprecation: #64109 - Deprecate global hook softRefParser_GL

See Issue #64109

Description

The hook `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['GLOBAL']['softRefParser_GL']` has been marked as deprecated. It was a hook to add a general softRefParser which parsed every SoftReference regardless of its type. The `softRefParser_GL`-hook was undocumented and used neither in core nor in any known extension.

Impact

Creating a global softRefParser by adding a hook to `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['GLOBAL']['s` will trigger a deprecation log message.

Affected installations

Instances with extensions using a `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['GLOBAL']['softRefParser` hook

Migration

A `softRefParser_GL` hook in an extension has to be replaced with multiple `softRefParser` hooks for each type the parser can handle.

Deprecation: #64134 - Deprecate TypoScriptTemplateObjectBrowserModuleFunctionController::verify_TSobjects()

See Issue #64134

Description

`TypoScriptTemplateObjectBrowserModuleFunctionController::verify_TSobjects()` has been marked as deprecated.

Impact

Using `TypoScriptTemplateObjectBrowserModuleFunctionController::verify_TSobjects()` will trigger a deprecation log message.

Affected installations

Instances which use of `TypoScriptTemplateObjectBrowserModuleFunctionController::verify_TSobjects`

Migration

No migration possible.

Deprecation: #64147 - TemplateService->ext_getKeyImage

See Issue #64147

Description

`ExtendedTemplateService::ext_getKeyImage()` has been marked as deprecated. `ConfigurationForm::ext_getKeyImage()` has been marked as deprecated.

Impact

Using the two methods will throw a deprecation message.

Affected installations

TYPO3 installations with extensions that call the methods above directly.

Migration

As in the methods directly, plain HTML based on Twitter bootstrap can be used. Example: `<span class="label label-danger">3</span>`.

Deprecation: #64361 - Composer Class Loading

See Issue #64361

Description

TYPO3 CMS started integrating composer support and by that embracing PHP standards PSR-0 and PSR-4 for class loading that comes with composer. The old class loader is still present and registered and will handle loading extension classes that do not follow the above mentioned standards.

For the time being a pre-compiled composer class loader is shipped with the git repository, so that people using the TYPO3 sources directly from there can use it without requiring a `composer install` step.

It is possible however to do a `composer install` in the TYPO3 sources directory or a top level distribution directory to gain full control over class loading of your TYPO3 installation. This step is optional and currently requires thorough knowledge of composer and as such currently is only recommended for people familiar with this process.

Further technical details can be found in the wiki.

Impact

Third party libraries added to a TYPO3 installation via composer can immediately be used without further manual intervention. The classes cache for most core classes will be gone. Resolving classes is slowly shifted from a run time task - executed and monitored in every request - to an installation task with composer.

Affected installations

Some installations could be affected which were previously installed via composer, but now not properly updated using composer. And outdated Packages/Libraries/autoload.php file present in the system will lead to fatal errors.

Migration

If you previously installed TYPO3 via composer, make sure you perform a `composer update` command to reflect the current changes in your Packages folder.

Deprecation: #64388 - Direct ContentObject methods within ContentObjectRenderer

See Issue #64388

Description

The following wrapper methods for ContentObject rendering within ContentObjectRenderer have been marked for removal for TYPO3 CMS 8.

```
FLOWPLAYER()
TEXT()
CLEARGIF()
COBJ_ARRAY()
USER()
FILE()
FILES()
IMAGE()
IMG_RESOURCE()
IMGTEXT()
CONTENT()
RECORDS()
HMENU()
CTABLE()
OTABLE()
COLUMNS()
HRULER()
CASEFUNC()
LOAD_REGISTER()
FORM()
SEARCHRESULT()
TEMPLATE()
FLUIDTEMPLATE()
MULTIMEDIA()
MEDIA()
SWFOBJECT()
QTOBJECT()
SVG()
```

Impact

Using the methods above directly in any third party extension will trigger a deprecation log message.

Affected installations

Instances which use custom calls to ContentObjects via the methods above.

Migration

Replace the direct method calls to `$contentObject->COBJECT()` with the common method.

Example for the SVG() ContentObject call:

```
$cObj->cObjGetSingle('SVG', $conf);
```

Deprecation: #64711 - Various methods within CSS Styled Content Controller

See Issue #64711

Description

The following methods within the main CSS Styled Content Controller responsible for rendering custom HTML due to lack of TypoScript logic in the past have been marked for removal for TYPO3 CMS 8. They are not part of the default CSS Styled Content TypoScript code since TYPO3 CMS 6.

```
CssStyledContentController->render_bullets()
CssStyledContentController->render_uploads()
CssStyledContentController->beautifyFileLink()
```

Impact

Using the methods in custom TypoScript code or CSS Styled Content methods will throw a deprecation message.

Migration

Use default TypoScript from CSS Styled Content derived from the current version.

Deprecation: #64922 - Deprecated entry points

See Issue #64922

Description

The following entry points have been marked as deprecated:

- typo3/tce_file.php
- typo3/move_el.php
- typo3/tce_db.php
- typo3/login_frameset.php
- typo3/sysext/cms/layout/db_new_content_el.php
- typo3/sysext/cms/layout/db_layout.php

Impact

Using one of the entry points in a backend module will throw a deprecation message.

Migration

Use `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl()` instead with the according module name.

typo3/tce_file.php `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl('tce_file')`

typo3/move_el.php `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl('move_element')`

typo3/tce_db.php `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl('tce_db')`

typo3/login_frameset.php `\TYPO3\CMS\Backend\Utility\BackendUtility::getModuleUrl('login_frameset`

3.8 7.0 Changes

Table of contents

- *Breaking Changes*
- *Features*
- *Deprecation*

3.8.1 Breaking Changes

Breaking: #19737 - Prefer root templates for pages

See Issue #19737

Description

The Core now gives templates having set the root-flag precedence over normal extension templates. This ignores the fact that normally the first template of the current page is chosen, following their sorting.

This improves user experience as the user expects the root template to be the one with highest priority.

Impact

Pages where multiple templates are present not having a root-template as the topmost template in the list will encounter different results when templates are evaluated.

Affected installations

Installations with pages where multiple templates are present not having a/the root-template as the topmost template.

Migration

Ensure the templates have correct flags set.

Breaking: #33805 - ClickMenu Rewrite

See Issue #33805

Description

The ClickMenu has seen some major changes under the hood. This implies some refactoring within JavaScript where existing functionality is replaced by a AMD Module based on jQuery. The following JS methods are now replaced by respective methods:

- showClickmenu_raw()
- Clickmenu.show()
- Clickmenu.populateData()

The new functionality is available via a global JavaScript object called TYPO3.ClickMenu which has equal functions.

Additionally the ClickMenu is now used via AJAX completely, all non-AJAX calls are not supported anymore.

Impact

All third-party extensions using alt_clickmenu.php directly in the backend, or using the above JavaScript calls directly.

Affected installations

Any installation using extensions having Backend modules using JavaScript functions for the ClickMenu inline and installations using extensions using alt_clickmenu.php directly.

Migration

Any use of "Clickmenu.show()" etc should be avoided and channelled through the according DocumentTemplate methods.

- BackendUtility::wrapClickMenuOnIcon()
- DocumentTemplate->getContextMenuCode()

If a backend module without a DocumentTemplate (with e.g. Extbase/Fluid) is used, this is done with a separate class and related data attribute:

```
<a href="#" class="t3-js-clickmenutrigger" data-table="be_users" data-uid="{record.
↪uid}" data-listframe="1">
```

Breaking: #42543 - Default TypoScript Removed

See Issue #42543

Description

The default TypoScript `styles.insertContent` was removed without substitution.

Impact

Frontend output may change.

Affected installations

A TYPO3 instance is affected if its TypoScript operates with `styles.insertContent`. This is pretty unlikely since the styles segment was hidden in the TypoScript Object Browser.

Migration

Either remove usage of `styles.insertContent` or add a snippet at an early point in TypoScript for backwards compatibility.

```
styles.insertContent = CONTENT
styles.insertContent {
  table = tt_content
  select {
    orderBy = sorting
    where = colPos=0
```

```
    languageField = sys_language_uid
  }
}
```

Breaking: #53542 - Removal of deprecated code in sysext fluid

See Issue #53542

Description

ContainerViewHelper

The following options have been removed from the ViewHelper:

- `enableJumpToUrl`
- `addCssFile`, use `includeCssFiles` instead
- `addJsFile`, use `includeJsFiles` instead

AbstractBackendViewHelper

The usage of `$GLOBALS['SOBE']` is removed for retrieving the DocumentTemplate instance. Use `->getDocInstance()` instead.

TemplateView

The following methods have been removed:

- `getTemplateRootPath()` is removed, use `getTemplateRootPaths()` instead
- `getPartialRootPath()` is removed, use `setPartialRootPaths()` instead
- `getLayoutRootPath()` is removed, use `getLayoutRootPaths()` instead

Impact

A call to any of the aforementioned methods by third party code will result in a fatal PHP error.

Affected installations

Any installation which contains third party code still using these deprecated methods.

Migration

Replace the calls with the suggestions outlined above.

Breaking: #53658 - option alternateBgColors removed

See Issue #53658

Description

The PageTSConfig option `mod.web_list.alternateBgColors` is removed without substitution.

Impact

Extensions that extend the DatabaseRecordList and are using the property `alternateBgColors`

The option in the TableListViewHelper has been deprecated and will be removed in TYPO3 CMS 8.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed property.

Migration

Remove the call to the removed property.

Breaking: #54409 - RTE "acronym" button was renamed "abbreviation"

See Issue #54409

Description

The "acronym" tag being deprecated, the RTE "acronym" button was renamed "abbreviation". Accordingly, the RTE Acronym plugin was renamed Abbreviation.

Impact

The "abbreviation" button may not appear in the RTE toolbar, if configured as "acronym" in Page TSconfig, TCA special configuration options and/or User TSconfig. Possible undefined PHP class errors. Possible Javascript or file not found errors.

Affected installations

An installation is affected if the "acronym" button was configured in Page TSconfig and/or User TSconfig. An installation is affected if a 3rd party extension refers to the "acronym" button in TCA special configuration options. An installation is affected if a 3rd party extension refers to class TYPO3CMSRtehtmlareaExtensionAcronym An installation is affected if a 3rd party extension loads the JavaScript file of the Acronym plugin: EXT:rtehtmlarea/Resources/Public/Javascript/Plugins/Acronym.js

Migration

There is no immediate impact on the RTE configuration in Page TSconfig and TCA special configuration options until the automatic conversion of existing references to "acronym" is removed in TYPO3 CMS 8.0. Intallations may run the upgrade wizard of the Install tool to migrate the contents of Page TSconfig, replacing "acronym" by "abbreviation". Note that this string replacement will apply to all contents of PageTSconfig. The migration of PageTSconfig may also be done manually.

User TSconfig must be modified to refer to "abbreviation" instead of "acronym".

Any affected 3rd party extension must be modified to refer to the "abbreviation" button rather than "acronym" in TCA special configuration options. Any affected 3rd party extension must be modified to refer to class TYPO3CMSRtehtmlareaExtensionAbbreviation rather than TYPO3CMSRtehtmlareaExtensionAcronym Any affected 3rd party extension must be modified to load EXT:rtehtmlarea/Resources/Public/Javascript/Plugins/Abbreviation.js rather than EXT:rtehtmlarea/Resources/Public/Javascript/Plugins/Acronym.js

Breaking: #57382 - Remove old flash message API

See Issue #57382

Description

The old flash message API is removed.

Impact

Extensions relying on the old (static) flash message queue API will not work anymore. Extbase removes the protected old flashMessageContainer.

Affected installations

Any installation that uses an extension relying on the old API.

Migration

Change the API calls to not be of static kind anymore. Extbase extensions have to use `getFlashMessageQueue()` of the controllerContext

Breaking: #57396 - Deprecated Extbase Property Mapper was removed

See Issue #57396

Description

A new Property Mapper that mapped request arguments to controller action arguments was introduced in Extbase 1.4 and the old one was deprecated at the same time. Along with the mapping, the validation API has also been changed. The rewritten property mapper is turned on by default since TYPO3 6.2

Now the old mapping and validation API is completely removed.

Impact

Extbase extensions that relied on the internal behaviour of the deprecated property mapper or make use of the old validation API will stop working or may not work as expected any more.

Affected installations

Extbase extensions that turned off the introduced feature switch with the TypoScript setting `features.rewrittenPropertyMapper = 0` because they relied on internal behavior of the old property mapper will stop working.

Migration

Manual migration of extension code might be required, especially when own validators using the old validation API were used.

Breaking: #59659 - Removal of deprecated code in sysext backend

See Issue #59659

Description

Flexforms

Flexform xml that still contains the old "<tx_templavoila><title>" code has to be adapted. The "<tx_templavoila>" elements needs to be removed.

Wizard registration

Wizard registration in TCA must not contain the "script=some/path/script.php" definition anymore. The new API for registering wizards is to set "module[name]=module_name".

Removed PHP methods

- `AbstractRecordList::writeBottom()` is removed without replacement. The functionality is not needed anymore.
- `SpriteGenerator::setOmmitSpriteNameInIconName()` is removed in favor of `setOmitSpriteNameInIconName()`
- `DocumentTemplate::isCMlayers()` is removed without replacement. The functionality is obsolete.
- `DocumentTemplate::getFileheader()` is removed. Use `getResourceHeader()` instead.
- `BackendUtility::displayWarningMessages()` is removed without replacement. The functionality was moved to ext:aboutmodules.
- `IconUtility::getIconImage()` is removed without replacement. Use sprite icon API instead.
- `PageLayoutView::getSelectedBackendLayoutUid()` is removed. Use `BackendLayoutView::getSelectedCombinedIdentifier()` instead.
- `ClickMenu::menuItemsForClickMenu()` is removed without replacement. The functionality is obsolete.

Removed JS functions

- `showClickmenu_noajax()` is removed. Use `Clickmenu.ajax = false; showClickmenu_raw();` instead.
- `setLayerObj()` is replaced with `Clickmenu.populateData()`.
- `hideEmpty()` is replaced with `Clickmenu.hideAll()`.
- `hideSpecific()` is replaced with `Clickmenu.hide()`. E.g. `Clickmenu.hide('contentMenu1');`
- `showHideSelectorBoxes()` is replaced with `toggleSelectorBoxes()`.

Impact

A call to any of the aforementioned methods by third party code will result in a fatal PHP error.

Affected installations

Any installation which contains third party code still using these deprecated methods.

Migration

Replace the calls with the suggestions outlined above.

Breaking: #59966 - Extension Configuration cache-flushing changed

See Issue #59966

Description

On saving the configuration of an extension, the system cache group has been flushed. This is inefficient as this includes also the classes cache, but most changes will never need this cache to be cleared. We optimize this for the common case and stop flushing caches after configuration changes completely.

Impact

Extensions which relied on cache-clearing after configuration changes may require a manual cache flush.

Affected installations

Any installation that uses extensions relying on automatic cache flush after extension configuration changes.

Migration

Extensions requiring a cache flush after configuration changes need to implement a slot for the `afterExtensionConfigurationWrite` signal which allows individual cache flush actions.

Breaking: #60063 - Felogin Plugin Removed

See Issue #60063

Description

File EXT:felogin/pi1/class.tx_felogin_pi1.php was removed.

Impact

- A require in PHP of this file throws a fatal error.
- An `includeLibs` TypoScript setting to this file raises a warning.

Affected installations

A TYPO3 instance is affected if a 3rd party extension requires EXT:felogin/pi1/class.tx_felogin_pi1.php or if an includeLibs TypoScript setting to this file is set.

Migration

Remove the require line in PHP and includeLibs line in TypoScript, they are obsolete.

Breaking: #60135 - Recursive stdWrap is now only called once

See Issue #60135

Description

If recursive stdWrap configuration was used, the stdWrap method was called twice, although the content was only rendered once. This performance overhead is now removed.

Impact

If a recursive stdWrap configuration is used, which sets or acts on a global state like registers, the resulting rendering can now be different because the global state is now modified only once.

Affected installations

A TYPO3 instance is affected if there is TypoScript code like:

```
page.1 = TEXT
page.1 {
  value = Counter:
  append = TEXT
  append.data = register:Counter
  stdWrap.append = LOAD_REGISTER
  stdWrap.append {
    Counter.cObject = TEXT
    Counter.cObject.data = register:Counter
    Counter.cObject.wrap = |+1
    Counter.prioriCalc = 1
  }
}
```

This now correctly outputs `Counter:1` instead of `Counter:2`

Migration

The usage of recursive stdWrap TypoScript configuration needs to be checked and probably adapted to fit the fixed behavior.

Breaking: #60152 - GeneralUtility::formatSize is now locale aware

See Issue #60152

Description

The `GeneralUtility::formatSize()` method now adheres to the currently set locale and selects the correct decimal separator symbol. This also applies to the TypoScript option `stdWrap.bytes`, which uses the method internally, as well as the Filelist content element type.

Impact

All output generated for locales, where the decimal separator is not a dot, will change to use the correct symbol. e.g. comma for German.

Affected installations

Any installation that uses the `formatSize()` method in one of the ways mentioned.

Migration

If you think you get the wrong decimal separator, ensure the locale is configured correctly and the locale really exists on the server.

TypoScript option: `config.locale` Commandline: `locale -a`

Breaking: #60559 - Dropped Backend Login Options

See Issue #60559

Description

Handling of `$GLOBALS['TBE_STYLES']['loginBoxImage_rotationFolder']` and `$GLOBALS['TBE_STYLES']['loginBoxImage_author']` was dropped.

Impact

Setting those options has no effect anymore.

Affected installations

These options had no effect with standard core internal login screen based on t3skin for a long time already. Instances are only affected if a 3rd party extension is loaded that changes the backend login screen and operates with these settings.

Migration

Remove these options and their usage from the affected 3rd party extension or unload the extension.

Breaking: #60559 - T3skin Backend Login Javascript File Moved

See Issue #60559

Description

Javascript files of the backend login form moved from EXT:t3skin to EXT:backend.

Impact

Javascript or file not found errors.

Affected installations

An installation is affected if a 3rd party extension includes EXT:t3skin/Resources/Public/JavaScript/login.js

Migration

Include EXT:backend/Resources/Public/JavaScript/login.js instead or refactor the affected extension to free it from the dependency to this core internal file.

Breaking: #60559 - T3skin Backend Login Template File Moved

See Issue #60559

Description

The HTML template file for the backend login screen was moved from EXT:t3skin to EXT:backend.

Impact

Broken login screen.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses file EXT:t3skin/Resources/Private/Templates/login.html

Migration

Use file EXT:backend/Resources/Private/Templates/login.html instead or refactor the affected extension to free it from the dependency to this core internal file.

Breaking: #60561 - Default TypoScript Constants Removed

See Issue #60561

Description

These default TypoScript constants were dropped:

- `{$_clear}`
- `{$_blackBorderWrap}`
- `{$_tableWrap}`
- `{$_tableWrap_DEBUG}`
- `{$_stdFrameParams}`
- `{$_stdFramesetParams}`

Impact

Frontend output may change.

Affected installations

A TYPO3 instance is affected if own TypoScript uses the above mentioned TypoScript constants.

Migration

Either remove usage of the above constants or add a snippet at an early point in TypoScript for backwards compatibility.

```
_clear = <img src="clear.gif" width="1" height="1" alt="" />
_blackBorderWrap = <table border="0" bgcolor="black" cellspacing="0" cellpadding="1
↪"><tr><td> | </td></tr></table>
_tableWrap = <table border="0" cellspacing="0" cellpadding="0"> | </table>
_tableWrap_DEBUG = <table border="1" cellspacing="0" cellpadding="0"> | </table>
_stdFrameParams = frameborder="no" marginheight="0" marginwidth="0" noresize=
↪"noresize"
_stdFramesetParams = 'border="0" framespacing="0" frameborder="no"
```

Breaking: #60582 - Rsaauth Javascript Files Moved

See Issue #60582

Description

Javascript files from EXT:rsaauth were moved from resources/ to Resources/Public/JavaScript.

Impact

Javascript or file not found errors.

Affected installations

An installation is affected if a 3rd party extension includes Javascript files from rsaauth.

Migration

Change affected extension to include Javascript files from resources/ to Resources/Public/JavaScript/.

Breaking: #60609 - Configuration Manager Signal Changed

See Issue #60609

Description

The extension for which the configuration was written was added to the signal emitted in the ConfigurationManager of the ExtensionManager as first parameter as the whole signal was unusable without this information.

Impact

The arguments for a method listening to this signal have changed.

Affected installations

A TYPO3 instance is affected if there is code using the signal "afterExtensionConfigurationWrite".

Migration

Rewrite the listening function to use the extension key as first parameter.

Breaking: #60630 - Scheduler Javascript File Moved

See Issue #60630

Description

Javascript files of the scheduler module moved from EXT:scheduler/res/tx_scheduler_be.js to EXT:scheduler/Resources/Public/JavaScript/Scheduler.js

Impact

Javascript or file not found errors.

Affected installations

An installation is affected if a 3rd party extension includes EXT:scheduler/res/tx_scheduler_be.js

Migration

Include EXT:scheduler/Resources/Public/JavaScript/Scheduler.js instead or refactor the affected extension to free it from the dependency to this scheduler internal file.

Breaking: #60630 - Scheduler Language Files Moved

See Issue #60630

Description

The language files of the scheduler extension are moved to EXT:scheduler/Resources/Private/Language/

Impact

Labels are not translated when being fetched from old file location.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses a language file from EXT:scheduler or if localization overrides of these files are registered. Those overridden labels may not work anymore.

Migration

Use new path to language file instead or create/copy the labels to an own language file and adapt existing overrides to the new file locations.

Breaking: #60630 - Scheduler Module Template File Moved

See Issue #60630

Description

The HTML template file for the scheduler module was moved from EXT:scheduler/mod1/mod_template.html to EXT:scheduler/Resources/Private/Templates/Module.html.

Impact

Broken module screen.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses file EXT:scheduler/mod1/mod_template.html

Migration

Use file EXT:scheduler/Resources/Private/Templates/Module.html instead or refactor the affected extension to free it from the dependency to this scheduler internal file.

Breaking: #61459 - Removal of tslib directory and constant

See Issue #61459

Description

The tslib/ directory and the constant PATH_tslib were removed.

Impact

Extensions that still use PATH_tslib constant, and reference typo3/sysext/cms/tslib/index_ts.php directly won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses `index_ts.php` directly, or if the main `index.php` is not replaced with the TYPO3 Update (used on certain install types). The index.php file must be replaced with the current version from the TYPO3 CMS Core.

Besides scripts are affected that access the time tracking (`$TT`) or typoscript frontend controller (`$TSFE`) objects instead of using `$GLOBALS['TT']` respectively `$GLOBALS['TSFE']`.

Migration

Remove the constant PATH_tslib from the 3rd party extension, use a current version of index.php and use `$GLOBALS['TT']` and `$GLOBALS['TSFE']` where necessary.

Breaking: #61471 - EXT:t3skin CSS files moved to less

See Issue #61471

Description

All CSS files in EXT:t3skin are moved to less files and handled by less CSS pre processor and merged to a single CSS file.

Impact

Single CSS files can not be included anymore. This may result in broken layouts.

Affected installations

A TYPO3 instance is affected if an extension loads single CSS files from EXT:t3skin. Backend modules of extensions usually get CSS core stuff loaded by default, which will not be a problem. An extension is only affected if single CSS files are explicitly referenced. This should be a rare case.

Migration

Most simple solution is to copy over the "old" CSS file from an older instance. Directly including those files from t3skin is discouraged. A better solution is to refactor the extension to use the full t3skin CSS file and to overlay it with required changes in an own file.

Breaking: #61781 - include_once array in ClickMenuController removed

See Issue #61781

Description

The include_once array of the ClickMenuController, which is filled with paths from the `$GLOBALS['TBE_MODULES_EXT']['xMOD_alt_clickmenu']['extendCMclasses']['path']` setting, has been removed.

Impact

Extension classes relying on the `$GLOBALS['TBE_MODULES_EXT']['xMOD_alt_clickmenu']['extendCMclasses'` registration for autoloading will not be loaded anymore.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed class loading registration method.

Migration

All classes are autoloaded automatically by TYPO3 CMS Core.

Breaking: #61782 - deprecated DocumentTemplate classes removed

See Issue #61782

Description

The following deprecated classes have been removed:

`\TYPO3\CMS\Backend\Template\MediumDocumentTemplate` `\TYPO3\CMS\Backend\Template\SmallDocu`
`\TYPO3\CMS\Backend\Template\StandardDocumentTemplate`

Impact

Extensions that still use one of the removed classes for their backend module won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses one of the removed classes.

Migration

Use `\TYPO3\CMS\Backend\Template\DocumentTemplate` instead.

Breaking: #61783 - Removed deprecated mailing API

See Issue #61783

Description

The deprecated methods to send email are removed. This includes the `MailUtility::mail()` method, the mail delivery substitution API and the `plainMailEncoded()` methods.

Impact

Any call to `MailUtility::mail()` or `GeneralUtility::plainMailEncoded()` will result in a fatal error. The option `$GLOBALS['TYPO3_CONF_VARS']['SC_OPTIONS']['t3lib/utility/class.t3lib_utility_` does not have any effect.

Affected installations

Any installation using an extension still using the deprecated API will fail.

Migration

Use the `\TYPO3\CMS\Core\Mail\Mailer` API for sending email.

Breaking: #61785 - getCompressedTCarray and includeTCA from TypoScriptFrontendController removed

See Issue #61785

Description

Methods `getCompressedTCarray()` and `includeTCA()` from `\TYPO3\CMS\Frontend\Controller\TypoScript` have been removed.

Impact

Extensions that still use `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::getCompress` or `\TYPO3\CMS\Frontend\Controller\TypoScriptFrontendController::includeTCA()` will trigger a fatal error.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses one of the removed functions.

Migration

Full TCA is always loaded during bootstrap in FE, the methods are obsolete. If an eid script calls this method to load TCA, use `\TYPO3\CMS\Frontend\Utility\EidUtility::initTCA()` instead.

Breaking: #61785 - loadTCA function in GeneralUtility removed

See Issue #61785

Description

Method `loadTCA()` from `\TYPO3\CMS\Core\Utility\GeneralUtility` is removed.

Impact

Extensions that still use `\TYPO3\CMS\Core\Utility\GeneralUtility::loadTCA()` will trigger a fatal error.

Affected installations

A TYPO3 instance is affected if a 3rd party extension still calls `loadTCA`.

Migration

The method is obsolete, full TCA is always loaded in all context except eID. It is safe to remove the method call.

Breaking: #61786 - remove deprecated TypeHandlingService in extbase

See Issue #61786

Description

The TypeHandlingService class is removed from the extbase extension.

Impact

Extensions that still use `\TYPO3\CMS\Extbase\Service\TypeHandlingService` won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed class `\TYPO3\CMS\Extbase\Service\TypeHandlingService`.

Migration

Replace all calls to `\TYPO3\CMS\Extbase\Service\TypeHandlingService` functions to their new static functions in `\TYPO3\CMS\Extbase\Utility\TypeHandlingUtility`

Breaking: #61802 - deprecated isLocalconfWritable function removed

See Issue #61802

Description

The function `isLocalconfWritable()` from `\TYPO3\CMS\Core\Utility\ExtensionManagementUtility` has been removed. The boostrap now just checks for the existence of the file and redirects to the install tool if it doesn't exist.

Impact

Extensions that still use the function `isLocalconfWritable()` won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed function.

Migration

Remove the call to this function. The bootstrap takes care to check the existence of the file.

Breaking: #61820 - deprecated PhpOptionsUtility functions removed

See Issue #61820

Description

The `\TYPO3\CMS\Core\Utility\PhpOptionsUtility` functions isSafeModeEnabled and isMagicQuotesGpcEnabled are removed.

Impact

Extensions that still use one of the removed functions won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses one of the removed functions.

Migration

Remove the call to `\TYPO3\CMS\Core\Utility\PhpOptionsUtility::isSafeModeEnabled()` or `\TYPO3\CMS\Core\Utility\PhpOptionsUtility::isMagicQuotesGpcEnabled()` functions. The Install Tool takes care of the removed checks now.

Breaking: #61821 - classFile option in makeInstanceService removed

See Issue #61821

Description

The option "classFile" in `\TYPO3\CMS\Core\Utility\GeneralUtility\makeInstanceService()` is removed. This should now be done by the respective ext_autoload.php of each extension.

Impact

Extension classes relying on the "classFile" registration for autoloading will not be loaded anymore.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed class loading registration method.

Migration

Use the ext_autoload.php file to autoload the class.

Breaking: #61822 - deprecated function getUniqueFields() removed

See Issue #61822

Description

The function `getUniqueFields()` from `\TYPO3\CMS\Core\DataHandling\DataHandler` has been removed. The function is available in `\TYPO3\CMS\Version\Hook\DataHandlerHook`.

Impact

Extensions that still use the function `getUniqueFields()` won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed function.

Migration

Replace all calls to `\TYPO3\CMS\Core\DataHandling\DataHandler::getUniqueFields()` with calls to `\TYPO3\CMS\Version\Hook\DataHandlerHook::getUniqueFields()`

Breaking: #61823 - Remove magic setter for $fromTC

See Issue #61823

Description

The magic setter for `$fromTC` in TYPO3CMSCoreDatabaseRelationHandler is removed.

Impact

Directly setting the protected property `$fromTC` will trigger a PHP warning.

Affected installations

Any installation using an extension that sets `$fromTC` property directly.

Migration

Use `\TYPO3\CMS\Core\Database\RelationHandler::setFetchAllFields()` instead.

Breaking: #61859 - deprecated file type FILETYPE_SOFTWARE removed

See Issue #61859

Description

The constant `\TYPO3\CMS\Core\Resource\AbstractFile::FILETYPE_SOFTWARE` has been removed.

Impact

Using the removed constant will result in a fatal error.

Affected installations

Any installation using an extension that uses the constant `\TYPO3\CMS\Core\Resource\AbstractFile::FILETYPE_SOF`

Migration

Use `\TYPO3\CMS\Core\Resource\AbstractFile::FILETYPE_APPLICATION` instead, which matches the Iana standard.

Breaking: #61860 - deprecated function int_from_ver removed

See Issue #61860

Description

Function `int_from_ver()` from `\TYPO3\CMS\Core\Utility\GeneralUtility` has been removed.

Impact

Extensions that still use the function `int_from_ver()` won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed function.

Migration

Replace the usage of the removed function with `\TYPO3\CMS\Core\Utility\VersionNumberUtility::convertVers`

Breaking: #61863 - deprecated connectDB from EidUtility removed

See Issue #61863

Description

Method `connectDB()` from `\TYPO3\CMS\Frontend\Utility\EidUtility` has been removed.

Impact

Extensions that still use the function `connectDB()` will trigger a fatal PHP error when an eID script is called.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed function.

Migration

The function can be removed safely. The database will connect itself if needed.

Breaking: #61890 - TBE Styling removed from FormEngine and TCA

See Issue #61890

Description

The styling functionality of FormEngine was based on a mixture of loose variables within `$GLOBALS['TBE_STYLES']` overridden by hardcoded values in various Backend PHP classes. This setup, additionally mixed with CSS classes that followed a very complicated syntax to render certain fields differently, has been removed in order to allow Backend styling for FormEngine completely based on CSS/LESS.

Impact

Using the following CSS classes within FormEngine don't have any effect anymore:

- class-main
- class-main1
- class-main2
- class-main3
- class-main4
- class-main5
- class-main11
- class-main12
- class-main13
- class-main14
- class-main15

- class-main21
- class-main22
- class-main23
- class-main24
- class-main25
- class-main31
- class-main32
- class-main33
- class-main34
- class-main35
- class-main41
- class-main42
- class-main43
- class-main44
- class-main45
- class-main51
- class-main52
- class-main53
- class-main54
- class-main55
- wrapperTable
- wrapperTable1
- wrapperTable2
- wrapperTable3
- wrapperTable4
- wrapperTable5
- formField
- formField1
- formField2
- formField3
- formField4
- formField5

Additionally, the following keys of $TBE_STYLES have no effect anymore:

- $TBE_STYLES['colorschemes']
- $TBE_STYLES['styleschemes']
- $TBE_STYLES['borderschemes']

They can safely removed from any third party extension.

The 5th parameter defining custom styleschemes in any field defined in $TCA[mytable][types][mytype][showitem] or $TCA[mytable][palettes][mypalette][showitem] has no effect anymore and can be removed from any third party extension (e.g. myfield;mylabel;usedpalette;extraDefinition;stylescheme).

Any styling is now done solely via LESS.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the CSS classes for styling or dynamic HTML via JavaScript.

Migration

Use the available CSS classes for custom styling and modifying FormEngine. Clean up any custom TCA definitions with a stylescheme in 3rd party extensions, where the fifth parameter of a field definition in $TCA[mytable][types][mytype][showitem] or $TCA[mytable][palettes][mypalette][showitem] is used.

Breaking: #61959 - Move flash message output to alerts

See Issue #61959

Description

Flash messages are now styled by using the native CSS classes of Twitter Bootstrap. The changed classes are:

- "typo3-message message-notice" => "alert alert-notice"
- "typo3-message message-information" => "alert alert-info"
- "typo3-message message-ok" => "alert alert-success"
- "typo3-message message-warning" => "alert alert-warning"
- "typo3-message message-error" => "alert alert-danger"

Impact

Extensions which use the old classes like "typo3-message message-information" rely on deprecated CSS classes which might lead to non styled output.

Affected installations

Any installation that uses the HTML of flash messages without calling the API.

Migration

Change the used CSS classes to the new ones.

Breaking: #62038 - Deprecated DocumentTemplate functionality

See Issue #62038

Description

The Backend/DocumentTemplate class contains various options that have no effect in the core anymore:

`$doc->JScodeLibArray` `$doc->docType` (as rendering is always as HTML5 by default) `$doc->inDocStyles` (use inDocStylesArray) `$doc->inDocStyles_TBEstyle` (now used as inDocStylesArray[tbeStyle] `$doc->charset` (always utf-8)

The methods `$doc->formatTime()` and `$doc->menuTable()` have also been deprecated as they are not used anymore.

Impact

Extensions that still use the properties of DocumentTemplate will not see any changes in the output of the code anymore.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the unused variables.

Migration

The variables can be removed safely, any modifications is possible via hooks in DocumentTemplate.

Breaking: #62039 - Removed TBE_STYLES[mainColors]

See Issue #62039

Description

The values within `$TBE_STYLES[mainColors]` are redundant and can be completely defined via CSS nowadays. The corresponding PHP leftovers are removed from the core and have no effect anymore.

Impact

Setting the variables within `$TBE_STYLES[mainColors]` and using the `$doc->bgColor*` and `$doc->hoverColor` properties of DocumentTemplate have no effect anymore.

Affected installations

Any installation using an extension that is overriding skin info via `$TBE_STYLES[mainColors]`.

Migration

Use CSS directly to modify the appearance of the Backend.

Breaking: #62291 - RTE Deprecated JavaScript methods removed

See Issue #62291

Description

The following RTE JavaScript methods, deprecated since TYPO3 4.7, have been removed:

HTMLArea.Editor.forceRedraw: use HTMLArea.Framework.doLayout instead

HTMLArea.Editor.convertNode: use HTMLArea.DOM.convertNode instead HTMLArea.Editor.getBlockAncestors: use HTMLArea.DOM.getBlockAncestors instead HTMLArea.getInnerText: use HTMLArea.DOM.getInnerText instead HTMLArea.hasAllowedAttributes: use HTMLArea.DOM.hasAllowedAttributes instead HTMLArea.isBlockElement: use HTMLArea.DOM.isBlockElement instead HTMLArea.needsClosingTag: use HTMLArea.DOM.needsClosingTag instead HTMLArea.Editor.rangeIntersectsNode: use HTMLArea.DOM.rangeIntersectsNode instead HTMLArea.removeFromParent: use HTMLArea.DOM.removeFromParent instead

HTMLArea.Editor.cleanAppleStyleSpans: use HTMLArea.DOM.Node.cleanAppleStyleSpans instead HTMLArea.Editor.removeMarkup: use HTMLArea.DOM.Node.removeMarkup instead HTMLArea.Editor.wrapWithInlineElement: use HTMLArea.DOM.Node.wrapWithInlineElement instead

HTMLArea.Editor.addRangeToSelection: use HTMLArea.DOM.Selection.addRange instead HTMLArea.Editor._createRange: use HTMLArea.DOM.Selection.createRange instead HTMLArea.Editor.emptySelection: use HTMLArea.DOM.Selection.empty instead HTMLArea.Editor.endPointsInSameBlock: use HTMLArea.DOM.Selection.endPointsInSameBlock instead HTMLArea.Editor.execCommand: use HTMLArea.DOM.Selection.execCommand instead HTMLArea.Editor._getSelection: use HTMLArea.DOM.Selection.get instead HTMLArea.Editor.getAllAncestors: use HTMLArea.DOM.Selection.getAllAncestors instead HTMLArea.Editor.getSelectedElement: use HTMLArea.DOM.Selection.getElement instead HTMLArea.Editor.getEndBlocks: use HTMLArea.DOM.Selection.getEndBlocks instead HTMLArea.Editor._getFirstAncestor: use HTMLArea.DOM.Selection.getFirstAncestorOfType instead HTMLArea.Editor.getFullySelectedNode: use HTMLArea.DOM.Selection.getFullySelectedNode instead HTMLArea.Editor.getSelectedHTML: use HTMLArea.DOM.Selection.getHtml instead HTMLArea.Editor.getSelectedHTMLContents: use HTMLArea.DOM.Selection.getHtml instead HTMLArea.Editor.getParentElement: use HTMLArea.DOM.Selection.getParentElement instead HTMLArea.Editor.getSelectionRanges: use HTMLArea.DOM.Selection.getRanges instead HTMLArea.Editor.getSelectionType: use HTMLArea.DOM.Selection.getType instead HTMLArea.Editor.insertHTML: use HTMLArea.DOM.Selection.insertHtml instead HTMLArea.Editor.insertNodeAtSelection: use HTMLArea.DOM.Selection.insertNode instead HTMLArea.Editor._selectionEmpty: use HTMLArea.DOM.Selection.isEmpty instead HTMLArea.Editor.hasSelectedText: use !HTMLArea.DOM.Selection.isEmpty instead HTMLArea.Editor.selectNode: use HTMLArea.DOM.Selection.selectNode instead HTMLArea.Editor.selectNodeContents: use HTMLArea.DOM.Selection.selectNodeContents instead HTMLArea.Editor.selectRange: use HTMLArea.DOM.Selection.selectRange instead HTMLArea.Editor.setSelectionRanges: use HTMLArea.DOM.Selection.setRanges instead HTMLArea.Editor.surroundHTML: use HTMLArea.DOM.Selection.surroundHtml instead

HTMLArea.Editor.getBookmark: use HTMLArea.DOM.BookMark.get instead HTMLArea.Editor.getBookmarkNode: use HTMLArea.DOM.BookMark.getEndPoint instead HTMLArea.Editor.moveToBookmark: use HTMLArea.DOM.BookMark.moveTo instead

HTMLArea.htmlDecode: use HTMLArea.util.htmlDecode instead HTMLArea.htmlEncode: use HTMLArea.util.htmlEncode instead

Impact

3rd party extensions adding plugins to the RTE and using the removed methods will fail.

Affected installations

A TYPO3 instance is affected if a 3rd party extension of the RTE refers to the removed methods.

Migration

The affected 3rd party extensions must be modified to use the replacement methods.

Breaking: #62339 - Move EXT:perm into EXT:beuser and remove EXT:perm

See Issue #62339

Description

The extension EXT:perm is removed from core, the perms module is moved into EXT:beuser. The BE module moved from "Web Access" to "System Access"

Impact

Extensions that use EXT:perm or maybe depends on it will cause problems

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses parts the removed extension.

Migration

The logic is moved into EXT:beuser. No special migration is necessary.

Breaking: #62416 - Removal of deprecated code in sysext core

See Issue #62416

Description

DataHandler

`DataHandler::clear_cache()` has been removed. Use `->clear_cacheCmd()` instead. Alternatively you can call `->registerPageCacheClearing()` from a hook to not immediately clear the cache but register clearing after DataHandler operation finishes.

`DataHandler::internal_clearPageCache()` has been removed, please use the cache manager directly.

The hook ['t3lib/class.t3lib_tcemain.php']['clearAllCache_additionalTables'] has been removed, use the caching framework with database backend instead.

DatabaseConnection

All connection parameters (host, username, password, db) for methods sql_connect(), sql_select_db() and connectDB() have been removed.

PackageManager

The globally defined REQUIRED_EXTENSIONS constant has been removed.

ExtensionManagementUtility

Parameter classPath of insertModuleFunction() is now unused. Leverage autoloading instead.

Removed PHP classes

- TYPO3CMSCoreResourceServiceIndexerService has been removed without replacement. Indexing is done internally.
- TYPO3CMSCoreCompatibilityGlobalObjectDeprecationDecorator has been removed without replacement. Do not use $GLOBALS['typo3CacheManager'] and $GLOBALS['typo3CacheFactory'] anymore, use GeneralUtility::makeInstance() instead.

Removed PHP class members

- AbstractUserAuthentication::$cookieId has been removed. Use isCookieSet() instead.

Removed PHP methods

- GeneralUtility::array_merge_recursive_overrule() has been removed. Use ArrayUtility::mergeRecursiveWithOverrule() instead. WARNING: The new method changed its signature and does not return the first parameter anymore.
- GeneralUtility::htmlspecialchars_decode() has been removed in favor of the native PHP htmlspecialchars_decode() function.
- CategoryRegistry::get() has been removed. Use isRegistered() instead.
- CategoryRegistry::applyTca() has been removed. The functionality is obsolete.
- DataHandler::clear_cache() has been removed. Use ->clear_cacheCmd instead. Alternatively you can call ->registerPageCacheClearing() from a hook to not immediately clear the cache but register clearing after DataHandler operation finishes.
- DataHandler::internal_clearPageCache() has been removed, use the cache manager directly.
- FileRepository::findByUid() has been removed without replacement.
- FileRepository::addToIndex() has been removed without replacement. Indexing is done transparently.
- FileRepository::getFileIndexRecordsForFolder() has been removed. Use FileIndexRepository::findByFolder() instead.
- FileRepository::getFileIndexRecord() has been removed. Use FileIndexRepository::findOneByFileObject() instead.
- FileRepository::findBySha1Hash() has been removed. Use FileIndexRepository::findByContentHash() instead.
- FileRepository::update() has been removed. Use FileIndexRepository::update() instead.
- ResourceStorage::getFolderByIdentifier() is replaced by getFolder().
- ResourceStorage::getFileByIdentifier() is replaced by getFileInfoByIdentifier().
- ResourceStorage::getFileList() is replaced by getFilesInFolder().
- ResourceStorage::getFolderList() has been removed. Use getFoldersInFolder() instead.
- ResourceStorage::fetchFolderListFromDriver() has been removed. Use getFoldersInFolder() instead.
- BasicFileUtility::getTotalFileInfo() has been removed. Use ResourceStorage instead via $GLOBALS['BE_USER']->getFileStorages()

- BasicFileUtility::checkFileNameLen() has been removed. Use ResourceStorage instead via $GLOBALS['BE_USER']->getFileStorages()
- BasicFileUtility::isPathValid() has been removed in favor of GeneralUtility::validPathStr().
- BasicFileUtility::blindPath() has been removed without replacement.
- BasicFileUtility::findTempFolder() has been removed without replacement.
- BasicFileUtility::rmDoubleSlash() has been removed without replacement.
- BasicFileUtility::cleanDirectoryName() has been removed. Use PathUtility::getCanonicalPath() instead.
- ExtendedFileUtility::init_actionPerms() is replaced with setActionPermissions().
- ExtendedFileUtility::printLogErrorMessages() is replaced with pushErrorMessagesToFlashMessageQueue().
- ExtendedFileUtility::findRecycler() has been removed. Use ResourceStorage instead.
- RteHtmlParser::rteImageStorageDir() and SelectImage::getRTEImageStorageDir() are removed. Use $fileFactory->getFolderObjectFromCombinedIdentifier($GLOBALS['TYPO3_CONF_VARS']['BE']['RTE_imageStorageDir' instead.
- Locales::getTerLocales() has been removed without replacement.
- Locales::getTerLocaleDependencies() has been removed without replacement.
- ExtensionManagementUtility::getRequiredExtensionListArray() has been removed without replacement.
- ExtensionManagementUtility::writeNewExtensionList() has been removed without replacement.
- PhpOptionsUtility::isSqlSafeModeEnabled() has been removed without replacement.
- ClassLoader::getAliasForClassName() is replaced with getAliasesForClassName().

Impact

A call to any of the aforementioned methods by third party code will result in a fatal PHP error.

Affected installations

Any installation which contains third party code still using these deprecated methods.

Migration

Replace the calls with the suggestions outlined above.

Breaking: #62595 - Remove SU change-to mode

See Issue #62595

Description

The permanent user switch has been removed from backend user list for a better UX.

Impact

The parameter "emulate" in the view helper "SwitchUser" is dropped. Using the argument causes an error "Argument "emulate" was not registered".

Affected installations

Any installation using an extension that uses the view helper "SwitchUser" with "emulate" argument.

Migration

Drop the "emulate" argument in the view helper call.

Breaking: #62670 - Removal of deprecated code in multiple sysexts

See Issue #62670

Description

DBAL DatabaseConnection

All connection parameters (host, username, password, db) for methods `sql_connect()` and `sql_select_db()` are removed.

CSS Styled Content

The old frontend plugin file `pi1/class.tx_cssstyledcontent_pi1.php` is removed. Refer to `Classes/Controller/CssStyledContentController.php` instead.

Install Tool

The check for PHP's magic_quotes_gpc settings is removed, as the feature is disabled since PHP 5.4.

Removed files

- lang/lang.php is removed. You don't need to include this file anymore, just use `\TYPO3\CMS\Lang\LanguageService` directly.
- `rtehtmlarea/htmlarea/plugins/DynamicCSS/dynamiccss.css` is removed. The file was not used.

Removed PHP classes

- TYPO3CMSSchedulerTaskFileIndexingTask is removed without replacement.

Removed PHP class members

- TypoScriptFrontendController::$absRefPrefix_force is removed without replacement.

Removed PHP methods

- LanguageService::JScharCode is removed, use GeneralUtility::quoteJSvalue instead.
- ContentObjectRenderer::joinTSarrays is removed without replacement.
- TypoScriptFrontendController::tidyHTML is removed without replacement. You may use the tidy extension from TER.
- ElementBrowser::isWebFolder is removed without replacement.
- ElementBrowser::checkFolder is removed without replacement.
- AbstractDatabaseRecordList::getTreeObject is removed without replacement.
- FileList::dirData is removed without replacement.
- FilesContentObject::stdWrapValue is removed, use ContentObjectRenderer::stdWrapValue instead.
- ImportExportController::userTempFolder is removed, use getDefaultImportExportFolder instead.
- ImportExportController::userSaveFolder is removed, use getDefaultImportExportFolder instead.
- CrawlerHook::loadIndexerClass is removed without replacement.
- DatabaseIntegrityView::func_filesearch is removed without replacement.
- DatabaseIntegrityView::findFile is removed without replacement.
- RteHtmlAreaBase::buildStyleSheet is removed without replacement.
- RteHtmlAreaBase::loremIpsumInsert is removed without replacement.
- StagesService::checkCustomStagingForWS is removed without replacement.

Removed JS functions

- tx_rsaauth_encrypt is removed without replacement.
- tx_rsaauth_feencrypt is removed without replacement.

Impact

A call to any of the aforementioned methods by third party code will result in a fatal PHP error.

Affected installations

Any installation which contains third party code still using these deprecated methods.

Migration

Replace the calls with the suggestions outlined above.

Breaking: #62673 - Deprecated extbase code is removed

See Issue #62673

Description

GenericQomStatement

You may no longer use bound variables without using a prepared statement.

ActionController

Support for old view configuration options templateRootPath, layoutRootPath and partialRootPath is dropped. Use the new options with fallback mechanism.

Removed PHP classes

- QueryObjectModelConstantsInterface
- QueryObjectModelFactoryInterface

Removed PHP class members

- ActionController::$viewObjectNamePattern is removed without replacement
- Repository::$backend is removed, use persistence manager instead

Removed PHP methods

- ObjectManager::create() is removed, use ObjectManager::get() instead
- ObjectManagerInterface::create() is removed
- PersistenceGenericBackend::replaceObject() is removed without replacement
- QuerySettingsInterface::setReturnRawQueryResult() is removed without replacement
- QuerySettingsInterface::getReturnRawQueryResult() is removed, use the parameter on $query->execute() directly
- Typo3QuerySettings::setSysLanguageUid() is removed, use setLanguageUid() instead
- Typo3QuerySettings::getSysLanguageUid() is removed, use getLanguageUid() instead

Impact

A call to any of the aforementioned methods by third party code will result in a fatal PHP error.

Affected installations

Any installation which contains third party code still using these deprecated methods.

Migration

Replace the calls with the suggestions outlined above.

Breaking: #62291 - RTE wizard classes renamed

See Issue #62291

Description

The following two RTE classes were renamed:

TYPO3CMSRtehtmlareaContentParser renamed to TYPO3CMSRtehtmlareaControllerParseHtmlController
TYPO3CMSRtehtmlareaUser renamed to TYPO3CMSRtehtmlareaControllerUserElementsController

Impact

3rd party extensions referring to an old class name will fail.

Affected installations

A TYPO3 instance is affected if a 3rd party extension refers to a renamed class by its old name.

Migration

The affected 3rd party extensions must be modified to use the new names of these classes.

Breaking: #62733 - RTE Javascript Files Moved

See Issue #62733

Description

Javascript files of the rtehtmlarea extension were moved from EXT:rtehtmlarea/htmlarea/ to EXT:rtehtmlarea/Resources/Public/JavaScript/

Impact

Javascript or file not found errors.

Affected installations

An installation is affected if a 3rd party extension loads any JavaScript file from EXT:rtehtmlarea/htmlarea/

Migration

Any affected 3rd party extension must be modified to load any JavaScript file from EXT:rtehtmlarea/Resources/Public/JavaScript/ instead.

Breaking: #62793 - TypoScript config.notification_* removed

See Issue #62793

Description

The following TypoScript options were removed:

`config.notification_email_charset` `config.notification_email_encoding` `config.notification_email_urlmode`

Impact

Using those options have no effect anymore.

Affected installations

Instances that set these options in FE TypoScript.

Migration

Those options can be safely removed. They were used with old mail handling and are substituted with a different engine that rendered those useless.

Breaking: #62804 - RTE JavaScript method HTMLArea.Editor::getNodeByPosition was moved

See Issue #62804

Description

RTE JavaScript method `getNodeByPosition()` was moved from HTMLArea.Editor to HTMLArea.DOM.Node where it belongs.

Impact

3rd party extensions referring to `HTMLArea.Editor::getNodeByPosition()` will fail.

Affected installations

A TYPO3 instance is affected if a 3rd party extension refers to the method `HTMLArea.Editor::getNodeByPosition()`.

Migration

The affected 3rd party extensions must be modified to use method `HTMLArea.DOM.Node::getNodeByPosition()` instead.

Breaking: #62819 - Remove php Localization Support

See Issue #62819

Description

The support for php files in localization, e.g. locallang.php files was deprecated with TYPO3 CMS 4.6. All translations are done with XLF in the core, XML files are still supported.

The parsing of PHP localization files is now disabled by default, the parsing class is now deprecated.

Impact

Extensions using locallang.php files for translation will not show labels anymore.

Affected installations

All installations with third-party extensions using locallang.php translation files.

Migration

Third-party extensions should migrate their translation files to the XLIFF format (XLF file extension). The extension development extension (Extension Key "extdeveval") can be used to transform locallang.php files to according XLF files.

Until this is done, it is possible to enable this option again by adding the following lines to typo3conf/AdditionalConfiguration.php:

```
$GLOBALS['TYPO3_CONF_VARS']['SYS']['lang']['format']['priority'] = 'xlf,xml,php';
$GLOBALS['TYPO3_CONF_VARS']['SYS']['lang']['format']['parser']['php'] =
↪'TYPO3\\CMS\\Core\\Localization\\Parser\\LocallangArrayParser';
```

Breaking: #62833 - Removed dividers2tabs functionality

See Issue #62833

Description

The "dividers2tabs" option in the ctrl section of TCA allows to show tabs in FormEngine while editing records, instead of showing all fields in one long column. This behaviour is the default since some TYPO3 versions.

This option has no effect anymore, as "dividers2tabs" is removed for TYPO3 CMS Core. The option can also be safely removed from any extension that adds TCA data.

Impact

A third-party extension that overrides the dividers2tabs option for an existing table or that adds a TCA table with this option disabled will have a record editing with tabs from now on.

Affected installations

Installations with 3rd-party extensions with TCA tables that have "dividers2tabs" disabled.

Breaking: #62859 - Removal of doc:link.action view helper

See Issue #62859

Description

The view helper `\TYPO3\CMS\Documentation\ViewHelpers\Link\Action` has been removed.

Impact

Extensions that rely on existence of `\TYPO3\CMS\Documentation\ViewHelpers\Link\Action` won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed view helper.

Migration

Either use f:be.buttons.icon or any of f:uri.* view helpers.

Breaking: #62888 - Remove config.uniqueLinkVars

See Issue #62888

Description

The setting `config.uniqueLinkVars` has been removed as the behaviour is now always used.

Migration

Remove the setting `config.uniqueLinkVars` from the TypoScript.

Breaking: #62914 - Early check for PHP 5.5 in Install Tool

See Issue #62914

Description

PHP 5.5 or above is a requirement for TYPO3 CMS7. As code is using specific PHP 5.5 features, an early check is required in Install Tool.

Impact

Install Tool will throw an exception if PHP 5.5 or above is not detected.

Affected installations

Any installation without at least PHP 5.5.0.

Migration

Upgrade to PHP 5.5 or above.

Breaking: #62987 - Remove CSH Glossary

See Issue #62987

Description

Early TYPO3 versions featured a glossary functionality, served by an extension, with built in switches in the core. Those switches are now removed.

Impact

The functionality is no longer available.

Affected installations

Any installation using the CSH Glossary functionality.

Breaking: #63056 - Remove Template Selection Hack

See Issue #63056

Description

There has been an ancient hack for the former "freesite" extension, which enabled selection of the template via GET variable. This technique is outdated and has been removed.

Impact

Any extension using this hack will not be able to select the template this way anymore.

Affected installations

Installations with third party extensions using the hack.

Migration

No migration path intended.

Breaking: #63069 - Removed compatibility layer for submodules of func and info modules

See Issue #63069

Description

The web_info and web_func modules use the module dispatcher now and do not have their own index scripts. Therefore any submodule for those modules need to adjust links accordingly.

Impact

Any third party code creating links to either web_info or web_func module using the old entry scripts, will not work anymore.

Migration

Use `BackendUtility::getModuleUrl()` instead to get the correct target for your links.

Breaking: #63110 - alt_doc_nodoc.php removed

See Issue #63110

Description

Script entry point typo3/alt_doc_nodoc.php and NoDocumentsOpenController class are removed without substitution.

Impact

A script pointing to this file resource will trigger a 404 server response and a script instantiating the class will cause a fatal error.

Affected installations

An extension needs to be adapted in the unlikely case that it uses this code.

Migration

Redirecting to typo3/dummy.php instead.

Breaking: #75942 - BigDocumentTemplate class removed

See Issue #75942

Description

The following class has been removed:

`\TYPO3\CMS\Backend\Template\BigDocumentTemplate`

Impact

Extensions that still use the removed class for their backend module won't work.

Affected installations

A TYPO3 instance is affected if a 3rd party extension uses the removed class.

Migration

Use `\TYPO3\CMS\Backend\Template\DocumentTemplate` instead.

3.8.2 Features

Feature: #47919 - Possibility to configure an exception handler when rendering TypoScript content objects

See Issue #47919

Description

Exceptions which occur during rendering of content objects (typically plugins) will now be caught by default in production context and an error message is shown as rendered output. If this is done, the page will remain available while the section of the page that produces an error (throws an exception) will show a configurable error message. By default this error message contains a random code which references the exception which is also logged by the logging framework for developer reference.

Usage:

```
# Use 1 for the default exception handler (enabled by default in production␣
↪context)
config.contentObjectExceptionHandler = 1

# Use a class name for individual exception handlers
config.contentObjectExceptionHandler =␣
↪TYPO3\CMS\Frontend\ContentObject\Exception\ProductionExceptionHandler

# Customize the error message. A randomly generated code is replaced within the␣
↪message if needed.
config.contentObjectExceptionHandler.errorMessage = Oops an error occurred. Code:
↪%s

# Configure exception codes which will not be handled, but bubble up again (useful␣
↪for temporary fatal errors)
tt_content.login.20.exceptionHandler.ignoreCodes.10 = 1414512813

# Disable the exception handling for an individual plugin/ content object
tt_content.login.20.exceptionHandler = 0

# ignoreCodes and errorMessage can be both configured globally ...
config.contentObjectExceptionHandler.errorMessage = Oops an error occurred. Code:
↪%s
config.contentObjectExceptionHandler.ignoreCodes.10 = 1414512813

# ... or locally for individual content objects
tt_content.login.20.exceptionHandler.errorMessage = Oops an error occurred. Code:
↪%s
tt_content.login.20.exceptionHandler.ignoreCodes.10 = 1414512813
```

Impact

Instead of breaking the whole page when an exception occurs, an error message is shown for the part of the page that is broken. Be aware that unlike before, it is now possible that a page with error message gets cached. To get rid of the error message not only the actual error needs to be fixed, but the cache must be cleared for this page.

Feature: #50039 - Multiple CSS Files in Rich Text Editor

See Issue #50039

Description

It is now possible to import more than one CSS file for the Rich Text Editor.

New syntax is:

```
RTE.default.contentCSS {
        file1 = fileadmin/myStylesheet1.css
        file2 = fileadmin/myStylesheet2.css
}
```

Impact

The old syntax may still be used. If no CSS files are set, the RTE default CSS file is used as before.

Feature: #51905 - Add dependencies between classes in the Rich Text Editor

See Issue #51905

Description

It is now possible to configure a class as requiring other classes.

The syntax of this new property is

```
RTE.classes.[ *classname* ] {
        .requires = list of class names; list of classes that are required by the␣
↪class;
                if this property, in combination with others, produces a circular␣
↪relationship, it is ignored;
                when a class is added on an element, the classes it requires are␣
↪also added, possibly recursively;
                when a class is removed from an element, any non-selectable class␣
↪that is not required by any of the classes remaining on the element is also␣
↪removed.
}
```

Impact

There is no impact on previous configurations.

Feature: #54518 - Provide TSconfig to link checkers

See Issue #54518

Description

The active TSconfig of the linkvalidator is stored in the LinkAnalyser and made publicly available to the link checkers.

The TSconfig is read either from the currently active TSconfig in the Backend when the linkvalidator is used in the info module or from the configuration provided in the linkvalidator scheduler task.

This allows passing configuration to the different link checkers.

Usage:

```
# The configuration in mod.linkvalidator can be read by the link checkers.
mod.linkvalidator.mychecker.myvar = 1
```

Impact

The method signature of `\TYPO3\CMS\Linkvalidator::LinkAnalyser::init()` is changed. A new parameter has been added for submitting the current TSconfig. This can break third party code that extends this method.

Feature: #54519 - Report links to disabled linkhandler records

See Issue #54519

Description

A new configuration option has been introduced for the linkhandler link checker in the linkvalidator extension:

```
mod.linkvalidator.linkhandler.reportHiddenRecords = 1
```

When enabled links will be considered invalid when they point to disabled records. By default only links to deleted records are reported.

Impact

The `\TYPO3\CMS\Linkvalidator\Linktype::checkLink()` method has been restructured and will now determine if the linked record is deleted or hidden and report a error depending on the `reportHiddenRecords` configuration.

Feature: #58122 - Configure class as non-selectable in Rich Text Editor

See Issue #58122

Description

It is now possible to configure a class as non-selectable in the style selectors of the Rich Text Editor.

The syntax of this new property is

```
RTE.classes.[ *classname* ] {
        .selectable = boolean; if set to 0, the class is not selectable in the␣
↪style selectors; if the property is omitted or set to 1, the class is selectable␣
↪in the style selectors
}
```

Impact

There is no impact on previous configurations.

Feature: #59396 - TypolinkViewHelper

See Issue #59396

Description

Adding Link and Uri ViewHelper that cope with the contents of any field that was filled with a link wizard in TYPO3 CMS Backend. Those fields contain various parts split by a space and being escaped to provide input for the typoLink function. In order to use those fields natively in Fluid without the need of TypoScript in between, this ViewHelper was introduced. It takes the field content as a whole and can additionally take some parameters directly from Fluid.

The full parameter usage in Fluid might look like this, where {link} is the field content:

```
<f:link.typolink parameter="{link}" target="_blank" class="ico-class" title="some␣
↪title" additionalParams="b=u" additionalAttributes="{type:'button'}">

<f:uri.typolink parameter="{link}" additionalParameters="b=u">
```

Only *parameter* is required, all other parameters are optional. While passing additional parameters to the ViewHelper, following rules apply:

- target is overridden, the value from Fluid applies
- class is merged from the values passed from the database and those of *class*
- title is overridden, the value from Fluid applies
- additionalParams is merged from the values passed from the database and those of *additionalParams*
- additionalAttributes is (as usual) added to the resulting tag as *type="button"*

{link} contains *19 _blank - "testtitle with whitespace" &X=y*. In case of the Uri.Typolink Viewhelper, only the first and the fourth parameter of the field content are taken into account, the tag related properties are discarded. For the given examples, the output is:

```
<a href="index.php?id=19&X=y&b=u" title="some title" target="_blank" class="ico-
↪class" type="button">
index.php?id=19&X=y&b=u
</a>
```

Impact

The new ViewHelper can be used in all new projects. There is no interference with any part of existing code.

Feature: #59830 - Introduce read-only column for file mounts

See Issue #59830

Description

File mount records now have a new flag "read only". This flag replaces the virtual flag introduced earlier, so it can be defined natively in the record.

Impact

The impact is low as the old behavior still exists. A storage was never marked as read-only before. There is an option to set this through UserTs, but now it is also possible to set it on the storage record directly.

Feature: #60064 - Logging Framework Introspection Processor

See Issue #60064

Description

The introspection processor of the logging framework has been extended to log the full PHP backtrace and not only the last element of a backtrace.

Two options were added to enable this feature:

- `appendFullBackTrace`, boolean, not mandatory‘ Add full backtrace to the log
- `shiftBackTraceLevel`, integer, default 0, not mandatory‘ Removes the given number of entries from the top of the backtrace stack.

Impact

The introspection processor behaves as before as long as the feature is not explicitly configured.

Feature: #60123 - Unit base test case removes test files

See Issue #60123

Description

Some unit tests need to create test files or directories to check the system under test. Those files should be removed again. A test can now register absolute file paths in `$this->testFilesToDelete`, and the generic `tearDown()` method will then remove them. Only files, links and directories within typo3temp/ are allowed.

Impact

This allows tests to clean up the environment without leaving obsolete test files behind.

Feature: #60567 - Show Styles Segment in TS Object Browser

See Issue #60567

Description

The TypoScript Object Browser now shows the setup segment `styles`.

Impact

The segment is cached in the Frontend and not unset anymore, page cache entries increase slightly in size.

Feature: #60822 - Class annotations in extbase reflection service

See Issue #60822

Description

The extbase reflection service can now return tags/annotations added to a class.

Suppose the given class:

```
/**
 * @SomeClassAnnotation A value
 */
 class Foo {
 }
```

Those annotation can be fetched with the reflection service:

```
$service = new \TYPO3\CMS\Extbase\Reflection\ReflectionService();
$classValues = $service->getClassTagsValues('Foo');
$classValue = $service->getClassTagValue('Foo', 'SomeClassAnnotation');
```

Impact

Getting class tags by ReflectionService is now possible.

Feature: #66185 - Allow Svg Files as Extension icon

See Issue #66185

Description

Extension authors can ship Extensions with an ext_icon file with the suffixes .png, .svg and .gif.

Impact

Extension icons might be rendered better when provided as vector graphics and not as bitmaps.

Feature: #61289 - Signal for IconUtility html tag manipulation

See Issue #61289

Description

This signal allows to manipulate the rendered html code for a sprite icon by an extension.

Currently all sprite icons are rendered as :code: <span class="""> ‘.

Extensions can now adjust the html tag, add or remove attributes and define own content in between the html tags.

Impact

The rendered html code is no longer a span with fixed classes, but can be modified by an extension.

Feature: #61351 - Add data attribute to Fluid ViewHelpers

See Issue #61351

Description

Since HTML5 Elements can contain a generic data attribute, Fluid provides for those elements the possibility to add key-value pairs as array, which will be rendered as `data-$key="$value"`.

```
<f:form.textfield data="{foo: 'bar', baz: 'foos'}" />
```

Impact

Generic data attributes do not need to be passed by the `additionalAttributes` array anymore making the viewhelper more straightforward to use.

Feature: #61361 - Template Path Fallback for Fluid StandaloneView and FLUIDTEMPLATE

See Issue #61361

Description

StandaloneView

Earlier in the development of Fluid, a template fallback was introduced in the TemplateView, providing the possibility to pass a set of possible file locations to the View Configuration, where Templates, Layouts and Partials can be found.

The same functionality is now available in the StandaloneView. It is possible to let the system look up the fitting paths for Partials and Layouts. It is in the nature of the StandaloneView to get a specific template file set, so for Templates there is no lookup requirement.

As a developer or integrator, you can configure your View as follows:

```
$view = $this->objectManager->get(\TYPO3\CMS\Fluid\View\StandaloneView::class);
$view->setFormat('html');
$view->setTemplatePathAndFileName(ExtensionManagementUtility::extPath('myExt') .
↪'Resources/Private/Templates/Email.html');
$view->setLayoutRootPaths(array(
  'default' => ExtensionManagementUtility::extPath('myExt') . 'Resources/Private/
↪Layouts',
  'specific' => ExtensionManagementUtility::extPath('myTemplateExt') . 'Resources/
↪Private/Layouts/MyExt',
));
$view->setPartialRootPaths(array(
  'default' => ExtensionManagementUtility::extPath('myExt') . 'Resources/Private/
↪Partials',
  'specific' => ExtensionManagementUtility::extPath('myTemplateExt') . 'Resources/
↪Private/Layouts/MyExt',
  'evenMoreSpecific' => 'fileAdmin/templates/myExt/Partials',
));
```

With this, the View will first look up the requested layout file in the path with the key `specific`, and in case there is no such file, it will fall back to `default`. For the partials the sequence would be `evenMoreSpecific`, then `specific`, then fall back to `default`.

You are free in the naming of the keys. The paths are searched from bottom to top. In case you choose for numeric array keys, the array is ordered first, then reversed for the lookup, so the highest index is accessed first.

FLUIDTEMPLATE

Additionally the TypoScript Content Object FLUIDTEMPLATE, which is based on StandaloneView, also supports this kind of fallback mechanism. Two new TypoScript options are added for this purpose:

- partialRootPaths
- layoutRootPaths

Example usage:

```
page.10 = FLUIDTEMPLATE
page.10.file = EXT:sitedesign/Resources/Private/Templates/Main.html
page.10.partialRootPaths {
  10 = EXT:sitedesign/Resources/Private/Partials
  20 = EXT:sitemodification/Resources/Private/Partials
}
```

In case you're using the old options (partialRootPath, layoutRootPath) together with the new options, the content of the old options will be placed at the first position (index zero) in the fallback list.

Impact

In order to change the skin of an extension output, provided by the Fluid StandaloneView, you are no longer required to copy the whole Resources folder into fileadmin or to some specific location, but you can pick only the files you want to change. Those need to be organized in folders, which are then configured for the view. The system will fall through all the provided locations, taking the first fitting file it finds.

Feature: #61489 - Allow own TypoScript Condition implementations

See Issue #61489

Description

It is now possible to add own TypoScript conditions via a separate API.

An extension / package can now ship an implementation of a new abstract class AbstractCondition. Via the existing TypoScript Condition Syntax the class is called by the simple full namespaced class name. The class's main function "matchCondition" can flexibly evaluate any parameters given after the class name.

Usage:

```
[BigCompanyName\TypoScriptLovePackage\BennisTypoScriptCondition]

[BigCompanyName\TypoScriptLovePackage\BennisTypoScriptCondition = 7]

[BigCompanyName\TypoScriptLovePackage\BennisTypoScriptCondition = 7, != 6]

[BigCompanyName\TypoScriptLovePackage\BennisTypoScriptCondition = {$mysite.
↪myconstant}]
```

where the TypoScript Condition class deals with =/!= etc itself.

Impact

If you've previously used the "userFunc" condition, you are encouraged to use this new API for your own TypoScript conditions.

Feature: #61529 - Add multiple parameter to f:form.checkbox

See Issue #61529

Description

Introduce parameter "multiple" for f:form.checkbox ViewHelper.

```
<f:form action="create" method="POST" name="pizza" object="{pizza}">
        <f:form.checkbox property="covering" multiple="1" value="salami" /><br />
        <f:form.checkbox property="covering" multiple="1" value="ham" /><br />
        <f:form.checkbox property="covering" multiple="1" value="cheese" /><br />
        <f:form.submit value="Send" />
</f:form>
```

Impact

If you add the parameter "multiple" to your checkboxes, it automatically appends [] to the name of your checkbox.

Feature: #61577 - Backend markup for checkboxes with labels

See Issue #61577

Description

A typical checkbox with label form element should now be rendered as:

```
<div class="checkbox">
        <label for="someId">
                <input type="checkbox" id="someId" />
                Label text
        </label>
</div>
```

Impact

If this HTML markup is applied, CSS styles by the TYPO3 core will take care of optimized view and custom CSS has become obsolete.

Feature: #61668 - Video and audio playback in backend record information

See Issue #61668

Description

The record information popup ("i" symbol) can play certain html5 video and audio types. This is especially useful in the "file list" module.

Impact

Better usability of media files.

Feature: #61800 - Registry for adding file rendering classes

See Issue #61800

Description

To be able to render all kinds of media files a file rendering registry is needed where you can register a "renderer" class that can generate the needed HTML output.

Every renderer has a priority between 1 and 100, 100 is more important than 1. When the rendererRegistry is asked for a renderer that fits a given file all registered renderer classes are "asked", in order of priority, if they are able to render the file. The first renderer class that can render the file an instance is returned by the rendererRegistry.

Every registered renderer class needs to implement the FileRendererInterface. This makes sure the class has a getPriority(), canRender() and render() method.

- getPriority() returns integer between 1 and 100
- canRender() gets a file(Reference) object as parameter and returns TRUE if the class is able to render the file It checks on mime-type but also storage type etc. can be performed to determine if creating the correct output is possible
- render() also gets the file(Reference) object as parameter together with width, height and an optional options array the return value is the HTML output

A AudioTagRenderer and VideoTagRenderer have already been added.

It is possible to register your own renderer classes in the ext_localconf.php of an extension.

Example:

```
$rendererRegistry = \TYPO3\CMS\Core\Resource\Rendering\RendererRegistry::
↪getInstance();
$rendererRegistry->registerRendererClass(
    'MyCompany\\MySpecialMediaFile\\Rendering\\MySpecialMediaFileRenderer'
);
```

Impact

The registry on its own doesn't do anything. Some followup patches are needed to use this registry to find the correct renderer class for rendering videos and other media files in BE preview and FE.

Feature: #62147 - New eval option in TCA: email

See Issue #62147

Description

A new option has been added to the eval field: email. This will check if the entered value is a valid e-mail address server-side. If not, a flash error message will be shown.

Usage:

```
'email' => array(
    'exclude' => 1,
    'label' => 'LLL:EXT:wd_products/Resources/Private/Language/locallang_db.xlf:tx_
↪wdproducts_domain_model_contactperson.email',
    'config' => array(
        'type' => 'input',
        'size' => 30,
        'eval' => 'email,trim'
    ),
)
```

Impact

Users don't have to write their own validation classes for e-mail validation.

3.8.3 Deprecation

Deprecation: #60574 - Client Related Conditions

See Issue #60574

Description

Conditions that depend on client details are unlovely for a number of reasons:

- Per condition or permutation of conditions that matches, the frontend creates a different cache entry. This can lead to a very high number of cache entrys per page
- Conditions based on browser or clients on server side are bad practice.
- The device information in the core is outdated (for example it is possible to match "AMIGA")
- Setups like reverse proxies give additional headaches with these types of conditions
- All client related condition types are deprecated with this patch.

Impact

Usage of client related TypoScript conditions will result in a deprecation log message. Client related conditions are browser, version, system and useragent.

Affected installations

Installations using TypoScript conditions for browser, version, system or useragent.

Migration

- Most usual conditions for specific browsers can nowadays be turned into conditional CSS includes
- Use libraries such as modernizr for browser support
- If conditions for specific clients or devices are still needed, they should be done with a userFunc condition and a project like WURFL that keep the device information more recent than the current core code like matching AMIGA

Deprecation: #61513 - Use native htmlspecialchars in ExtendedTemplateService

See Issue #61513

Description

In previous versions the ExtendedTemplateService used a conditional wrapper method to apply htmlspecialchars() for rendering TypoScript search labels, keys and comments in the backend. This option was never used, so htmlspecialchars() was always activated rendering the option and method redundant and not necessary. Calls to the method are removed. The ExtendedTemplateServer method and property are now marked as deprecated for removal with CMS 8.

Impact

Custom extensions using the flag ExtendedTemplateService->ext_noSpecialCharsOnLabels or a custom implementation of ExtendedTemplateService might get different results when using this switch within the class.

Affected installations

Installations using ExtendedTemplateService in their own extensions.

Deprecation: #62363 - TSFE->JSeventFuncCalls disabled

See Issue #62363

Description

TYPO3 CMS provides a way to register direct JS calls to be added to the body tag of the frontend output to allow several functions to register for e.g. "onload". Nowadays this is done via JS frameworks directly, or via JS variables.

The functionality has been marked as deprecated.

Impact

The core does not use this functionality anymore. Installations with menus using "GMENU_LAYERS", which has been removed from the core a while ago, but still use it via third-party extensions, might not work anymore as expected.

Affected installations

All installations which use the `$TSFE->JSeventFuncCalls` option, e.g. like GMENU_LAYERS.

Migration

Every call of a 3rd party extension to the mentioned method must be changed to use their own JS function registration.

Deprecation: #62794 - Mail methods in GeneralUtility

See Issue #62794

Description

The following methods of the class TYPO3CMSCoreUtilityGeneralUtility have been marked as deprecated:

- quoted_printable()
- encodeHeader()
- substUrlsInPlainText()

Impact

The methods were used together with the old mail API and are now obsolete. Deprecation warnings will be triggered if used.

Affected installations

Installations that still use those methods will trigger deprecations warnings.

Migration

Code that still uses these methods should be refactored to the mail API using TYPO3CMSCoreMailMailer class.

Deprecation: #62795 - DocumentTemplate->endPageJS()

See Issue #62795

Description

Method `TYPO3\CMS\Backend\Template\DocumentTemplate::endPageJS()` and the according property `endJS` have been marked as deprecated.

Impact

None, as it isn't in use anymore since TYPO3 CMS 4.5 and was responsible for notifying the browser that the session is still active.

Affected installations

Installations misusing top.busy until now for their own good will break.

Deprecation: #62800 - Workspaces ToolbarItem via ExtDirect

See Issue #62800

Description

The PHP functionality for switching a workspace was done with the ExtDirect call `TYPO3.Ajax.ExtDirect.ToolbarMenu` until now. This has been replaced by a simple AJAX JSON call, based on jQuery and the refactored ToolbarItem Menu for the workspaces module.

Impact

The core does not use this functionality anymore, and also removed the ExtDirect registration for this class.

Affected installations

All installations which directly used the ExtDirect `TYPO3.Ajax.ExtDirect.ToolbarMenu` to fetch the data.

Migration

Use the new AjaxHandler `Workspaces::setWorkspace()` directly instead.

Deprecation: #62854 - Abstractplugin->pi_list_searchBox()

See Issue #62854

Description

Method `pi_list_searchBox()` of AbstractPlugin (aka pibase) was used with very old search solutions and is hopelessly outdated. It is now discouraged to be used and will be removed with next major version.

Impact

Extensions still using `pi_list_searchBox()` will throw a deprecation warning.

Affected installations

Any extension still using this method needs to be adapted.

Deprecation: #62864 - BackendUtility->helpTextIcon deprecated

See Issue #62864

Description

The function `helpTextIcon()` in BackendUtility has been marked as deprecated.

Impact

The core does not use this functionality anymore.

Affected installations

All installations which use the function `helpTextIcon()`.

Migration

Use `BackendUtility::cshItem()` instead.

Deprecation: #62893 - Flashmessage JavaScript object TYPO3.Flashmessages was moved

See Issue #62893

Description

Flashmessages JavaScript object has been moved from `TYPO3.Flashmessages` to `top.TYPO3.Flashmessages`. The severity constant values has been changed to correspond to the same values (-2,-1,0,1,2) of the constants as in PHP. The constants `TYPO3.Severity.information` have been marked as deprecated. 3rd party extensions referring to `TYPO3.Severity.information` will work until CMS 9. A compatibility file was introduced to map `TYPO3.Flashmessages` to `top.TYPO3.Flashmessages`, will also work until CMS 9.

Impact

If a 3rd party extension calls the mentioned methods directly, a deprecation log will be written to the browser console.

Affected installations

A TYPO3 instance is affected if a 3rd party extension refers to the method `TYPO3.Flashmessages.display()` or uses `TYPO3.Severity.information` constants.

Migration

The affected 3rd party extensions must be modified to use `top.TYPO3.Flashmessages` instead of `TYPO3.Flashmessages`.

Example:

```
// Old and deprecated:
TYPO3.Flashmessages.display(TYPO3.Severity.notice)

// New and the only correct way:
top.TYPO3.Flashmessages.display(top.TYPO3.Severity.notice)
```

The `TYPO3.Severity` object has been moved to `top.TYPO3.Severity`. Use `top.TYPO3.Severity.*` instead.

Deprecation: #62988 - Deprecate unused/non-unified DocumentTemplate code

See Issue #62988

Description

Several functions within DocumentTemplate are not encouraged to be used anymore.

The hard-coded background image setting via `$TBE_STYLES['background']` is removed and its usage is deprecated.

Additionally, the font-wrapping methods *rfw()* and *dfw()* are deprecated. The according CSS was removed from the core.

The method *collapseableSection()*, which was used solely by the reports module for ages in a buggy way, is also deprecated in favor of Bootstrap collapseables and localstorage.

Impact

The core does not use this functionality anymore.

Affected installations

All installations which use the setting `$GLOBALS['TBE_STYLES']['background']` or any of the functions:

- dfw()
- rfw()
- collapseableSection()

Migration

- Use CSS directly instead of `$GLOBALS['TBE_STYLES']['background']`
- Use the CSS class *text-muted* instead of the method `dfw()`
- Use the CSS class *text-danger* instead of the method `rfw()`
- Use HTML bootstrap classes, localStorage etc. instead of `collapseableSection()`

CHAPTER

FOUR

DOCUMENTING CHANGES

4.1 Motivation and goals

Forger

If you want to save yourself some time you can use the ReST-File-Generator at https://forger.typo3.org/utility/rst

Select the type of ReST snippet you want to create, enter your issue number and click the search button.

Spreading information about important core changes has been a problematic issue ever since. With the switch to the new release cycle with version 7 of the TYPO3 CMS core this situation should be improved.

A dedicated changelog of important changes is established to inform users, developers and other core related teams or projects:

- Overview for documentation team which changes trigger documentation changes.
- Overview for Release Managers and other "Spread the word" related works on important changes.
- Hints for extension developers, integrators and project developers whose own code areas may need adjustments on core updates.
- Standardized format for easy usage in scripts like a core migration or release notes system.

This structure replaces the old `NEWS.md` file.

4.2 Different changelog types

A changelog handles one of these change types:

- Breaking change: A patch moved or removed a specific part of core functionality that may break extensions if they use this part. Removal of deprecated code or an interface change are good examples of this type.
- Deprecation: A patch deprecates a certain core functionality for a planned removal.
- Feature: A patch adds new functionality.
- Important: Any other important message.

Casual bug fixes do not need changelog files, but every change that may be of special interest for extension developers or documentation writers should receive an entry. The changelog file must be provided as part of the patch that introduces the change.

4.3 Location

New changelog files should usually be added to the `typo3/sysext/core/Documentation/Changelog/master` directory. If a version is to be released, all files in this directory will be moved to a directory that is named after the release number. This way it can be easily seen which change has been introduced in which released core version.

In rare cases, patches worth a changelog file need to be back ported to stable LTS and / or old stable LTS versions. Those should be put into a different directory, depending on target LTS versions. We'll explain this by example:

Suppose core is currently developing v9, a first 9.0 has been released, so git core branch `master` will become 9.1.0 with next sprint release. Stable LTS version is currently 8.7.9, git core branch `TYPO3_8-7` will become 8.7.10 with next stable LTS patch level release. Old stable LTS version is currently 7.6.23, git core branch `TYPO3_7-6` will become 7.6.24 with next old stable LTS patch level release.

Example scenarios:

- **A feature patch is added to master:** Put the .rst file into the `master/` directory. The core team will re-review files in this directory shortly before 9.1.0 release and will move all files from `master` into `9.1` directory.
- **A patch with an important change in behavior is not only added to master, but also to 8.7:** Put the .rst file into the `8.7.x` directory in `master` branch. The back port to `TYPO3_8-7` branch includes the changelog file into `8.7.x` directory, too. Users upgrading to latest patch level release of 8.7 will then see the new file in the `8.7.x` directory.
- **A patch with an important change in behavior is not only added to master, but back ported to 8.7 and to 7.6:** Put the .rst into `8.7.x` directory, duplicate the file and also add it to `7.6.x` directory in the `master` branch. The back port to `TYPO3_8-7` branch adds these two files as well. The patch to `TYPO3_7-6` branch adds only the file in the `7.6.x` directory, not the file in `8.7.x` directory. Users upgrading to latest 7.6 patch level will then see the new file in `7.6.x` directory, users upgrading to latest 8.7 patch level will see the a new file in `7.6.x` directory, and also in `8.7.x` directory.

The main goal of this approach is to have a consistent state of changelog file across branches. Changelog files from older releases are never deleted in younger branches. They are still rendered in the install tool "View Upgrade Documentation" and are connected to the "Extension scanner". In our example above, master contains all changelog files for v9, and v8 and v7 files, branch `TYPO3_8-7` contains all files for v8 and v7, and branch `TYPO3_7-6` contains all v7 files.

Furthermore, documentation files from older releases should be identical in all branches. If a patch improves some documentation file from a v7 directory, this change should be put into all branches: `master`, `TYPO3_8-7` and `TYPO3_7-6` for consistency. The core team will check for differences of files between branches occasionally and will align them in case they diverged.

4.4 Filename convention

<type>-<forgeIssueNumber>-<UpperCamelCaseDescription>.rst

4.5 File content

Like other documentation, changelog files are done in ReST, see TYPO3 wiki ReST syntax for more details.

- All types contain a "Description" section that should give a short summary on which core part was affected by the change.
- All types contain an "Impact" section that describes the possible impact of a change. An example is "Frontend output may change", "Configuration of xy is easier" or "Backend will throw a fatal error".
- Types "Deprecation" and "Breaking" contain an "Affected installations" section that describes when and if a TYPO3 instance is affected by a change. Example: "Extension xy is in use" or "TypoScript functionality xy is used" or "System is based on PHP 5.3".

- Types "Deprecation" and "Breaking" contain a "Migration" section to describe best practices on how to cope with a specific change.
- All types contain a list of tags, see below.

4.6 Tagging

To provide the possibility to filter ReST files by topics, it is mandatory to equip every RST file with at least two tags. As a rule of thumb a file should have no more than five tags. Please limit yourself to the list provided below. If you are in dearly need to introduce a new tag, you must also add it to the list (and explain it) in this file as a reference for everyone.

The tag list should be located at the end of a ReST file prefixed with the index keyword, example:: `.. index:: Backend,JavaScript,NotScanned`.

List of all possible tags:

- TypoScript - Changes that imply or introduce alterations to some TypoScript settings or modify the behavior of TypoScript itself. Frontend TypoScript only.
- TSConfig - Changes or modifications on the PageTS or UserTS or the behavior of this field.
- TCA - Every change related to TCA.
- FlexForm - Changes affecting FlexForm functionality.
- LocalConfiguration - Changes that affect the LocalConfiguration.php or the subordinated AdditionalConfiguration.php
- Fluid - Changes that alter behavior of Fluid like introducing new tags or modifying already established ones.
- FAL - Changes to File Abstraction Layer.
- Database - Changes that modify behavior of the database abstraction or introduces or removes new fields.
- JavaScript - Modifying or introducing JavaScript.
- PHP-API - Implementations of mandatory changes of the PHP-API.
- Frontend - Changes that will affect the behavior or rendering of the TYPO3 Frontend.
- Backend - Changes that will affect the behavior or rendering of the TYPO3 Backend.
- CLI - Changes affecting CLI functionality.
- RTE - Changes to RTE functionality.
- ext:xyz - Changes on extension xyz. Please refer to this tag only when changing system extensions.

Furthermore, exactly one of the following tags *must* be added for all "Deprecation" and "Breaking" ReST files since core v9 and above:

- NotScanned - If this ReST file is not covered by the extension scanner at all.
- PartiallyScanned - If some parts of the deprecated / removed functionality can be found by the extension scanner.
- FullyScanned - If usages of all deprecated / removed functionality this ReST file is about can be found by the extension scanner. This tag is used by the extension scanner to mark a ReST file as "You are not affected by this in your codebase" if it does not find a match in extensions.

INDEX

B

Backend, 7, 14–16, 19, 27, 28, 32–34, 36–38, 40–42, 46–49, 51, 56, 57, 59, 61–63, 67, 69, 71, 75, 76, 78, 93–96, 100, 103–105, 107–109, 111, 114, 117, 119, 129, 131–133, 135, 137, 149, 151, 153, 157–159, 161–163, 165, 167, 169, 170, 172, 176–180, 184, 185, 187, 188, 190, 192–194, 196–201, 203–207, 220, 225, 228, 230, 231, 234, 242–245, 247–250, 252, 259, 261–263, 266, 268, 269, 272, 276, 286, 293–298, 300, 303–306, 308–312, 315–320, 322, 326–329, 332, 333, 341, 344, 345, 348–350, 354, 356, 358, 359, 364

C

CLI, 14, 21, 28, 35, 39, 57, 66, 67, 71, 97, 109, 110, 134, 156, 158, 160, 161, 163, 164, 166, 177, 214, 258, 322, 345

D

Database, 7, 27, 33, 36, 57, 70, 75, 95, 101, 110, 112, 114, 131, 171, 174, 177, 185, 196, 201, 207, 210–213, 222, 224–227, 233, 238–240, 242, 243, 260–262, 264, 267, 281, 330, 349

E

ext:adodb, 196
ext:backend, 225
ext:dbal, 196
ext:extbase, 174, 189, 225, 226, 237, 239, 250, 269, 282, 289
ext:felogin, 221
ext:fluid_styled_content, 11, 249, 256
ext:form, 16, 19, 49, 51, 56, 68, 78, 109, 117, 119, 170, 222, 241, 253, 283, 317, 333, 343
ext:impexp, 97
ext:indexed_search, 168, 183, 194, 197, 222, 277, 341
ext:indexed_search_mysql, 194
ext:install, 171
ext:lang, 182
ext:lowlevel, 206
ext:recordlist, 294
ext:scheduler, 93
ext:workspaces, 157, 179

F

FAL, 15, 219, 254
FlexForm, 80, 81, 111, 137, 151, 161–163, 179, 199, 259, 270
Fluid, 7, 13, 20, 30, 34, 48, 62, 64, 83, 85, 90, 94, 108, 110, 112, 113, 120, 121, 123, 125, 128, 165, 166, 172, 173, 197, 231, 233, 245, 256, 273, 279–281, 283, 326, 331, 333, 340, 349, 357
Frontend, 5, 6, 9–11, 13, 15, 16, 19, 26, 29, 37, 39, 45, 49, 51, 56, 57, 62, 64, 65, 70, 75, 78–83, 85, 86, 88, 90, 93, 95, 110, 113, 119–121, 123, 125, 127–130, 151–153, 155, 168, 170, 173–177, 185, 188, 191, 197, 210, 221, 233, 240, 242, 245, 247, 249, 250, 252, 253, 264, 271, 273, 276, 287, 288, 290, 291, 295, 297, 300, 303, 307, 314, 316, 317, 324, 329, 341, 365, 369

J

JavaScript, 25, 45, 61, 63, 64, 67, 68, 94, 103, 107, 135, 152, 157–159, 170, 188, 196, 197, 203, 206, 210, 217, 218, 220, 231, 242, 246, 266, 268, 304, 320, 322, 327, 348, 359

L

LocalConfiguration, 6, 9, 46, 78, 108, 151, 154, 172, 187, 194, 198, 206, 219, 225, 231, 232, 239, 244, 255, 259–261, 264, 265, 267, 274, 275, 307–309, 321, 334, 343, 365

P

PHP-API, 9, 14, 26, 27, 34, 36–46, 48, 49, 56–58, 61–63, 65, 75, 95, 96, 103–107, 109, 114, 129–135, 137, 150–153, 161–163, 168, 170–172, 174, 177–179, 183–185, 189–191, 195, 198–201, 203–205, 207, 208, 211–215, 218–220, 222–228, 231–238, 240, 242, 243, 245, 247–258, 261–263, 265, 267, 269–272, 274, 276–278, 281–314, 318, 321–326, 328–330, 333, 334, 344–346, 350–365, 369

R

RTE, 14, 72–74, 108, 111, 135, 137, 185, 197, 205, 217, 230, 241, 262, 293, 309–311, 313–315, 317–319, 356

T

TCA, 7, 10, 28, 32, 47, 69–71, 80, 81, 83, 84, 94, 95, 100, 101, 104, 111, 117, 130, 133, 137, 149, 151, 154, 155, 171, 172, 179, 180, 187, 192, 200, 201, 216, 218, 261, 262, 268, 270, 302, 308
TSConfig, 15, 61, 72–74, 103, 129, 159, 180, 195, 200, 209, 217, 228, 241, 244, 263, 265, 270, 309, 313–315, 317–319, 341, 343, 356, 364
TypoScript, 6, 8, 11, 15, 16, 20, 21, 26, 29, 35, 43, 52, 56, 57, 68, 70, 79–81, 86, 93, 106, 129, 152, 168, 173, 177, 182, 185, 197, 199, 215, 218, 221, 222, 240, 241, 247, 251, 253, 264, 277, 282, 288, 290, 300, 301, 307, 314, 317, 323, 324, 329, 330, 341, 343, 369